PROJECT REPORT

HDFS-Based Distributed File Replication, Deduplication and Storage Optimization System

HDFS Optimizer

Submitted in partial fulfilment of the requirements for the degree of

[Degree and branch]

Submitted by [Student name]

Roll number [Roll number]

Under the guidance of [Supervisor name and designation]

[Department]

[University or institute]

Academic session [Session]

Certificate

This is to certify that the project report titled *HDFS-Based Distributed File Replication, Deduplication and Storage Optimization System* is the work of [Student name], roll number [Roll number], carried out under my supervision in partial fulfilment of the requirements for the degree of [Degree and branch] at [University or institute] during the academic session [Session].

[Supervisor signature]
[Supervisor name]
[Designation]

[Head of department signature]
[Name]
[Department]

Declaration of Originality

I declare that the source code and results presented are from the system described here, and that all external sources are acknowledged in the references. Experimental values are the output of the experiments described, and no dataset, screenshot or measurement has been fabricated. [State your own contribution, and disclose the use of AI assistance in building this project and writing this report, as your institution's policy requires. Review this declaration before signing it.]

[Student signature]

Date: [Date]

Acknowledgements

[Acknowledge your supervisor, department and anyone else who helped.] This project relies on the open-source work of the Apache Hadoop community and on the public-domain library maintained by Project Gutenberg and its volunteers.

Abstract

The Hadoop Distributed File System (HDFS) protects data by replicating blocks, but it has no notion of content: identical files uploaded independently are stored, and replicated, separately. This report presents HDFS Optimizer, an application layer on an unmodified Apache Hadoop 3.5.0 cluster that adds whole-file and fixed-size chunk-level deduplication through content-addressable storage, a relational metadata model, and a web dashboard.

Because HDFS and the metadata database cannot share a transaction, all writes follow a staged protocol, and a file's old copy is removed only after its new form has been read back and verified with SHA-256. Unreferenced content is identified by query rather than by stored counters. HDFS replication is never reduced to obtain savings, and each reported value is labelled as measured, derived or estimated.

The system was evaluated on a four-node containerised cluster, using 15 authentic files from Project Gutenberg with recorded provenance. In a workload of 27 files with two labelled exact copies of each book, whole-file optimization reduced NameNode-reported space consumption from 119.95 MiB to 39.98 MiB, and all 27 files passed SHA-256 verification afterwards. Savings tracked the expected $(N-1)/N$ exactly for N identical copies. Chunk-level deduplication additionally recovered overlap between different files, at a measured cost in throughput and NameNode objects, and it did not survive a one-byte shift, confirming the known weakness of fixed-size chunking. Real DataNode, NameNode and process-kill failures caused no data loss. These savings describe controlled workloads, and they do not predict savings on any production system.

Keywords: Hadoop, HDFS, data deduplication, content-addressable storage, replication, storage optimization, SHA-256, distributed file systems.

Table of Contents

Chapter 1 — Introduction **1**
1.1 Background 1
1.2 Distributed storage 1
1.3 The Hadoop ecosystem 1
1.4 HDFS fundamentals 1
1.5 Replication 1
1.6 Data redundancy 1
1.7 Deduplication 1
1.8 Storage optimization 1
1.9 Motivation 1
1.10 Project scope 1
Chapter 2 — Problem Statement **2**
2.1 Existing storage inefficiencies 2
2.2 Redundancy problems 2
2.3 Duplication versus replication 2
2.4 Operational limitations 2
2.5 Need for the proposed system 2
Chapter 3 — Objectives **3**
3.1 Completed objectives 3
3.2 Proposed future enhancements 3
Chapter 4 — Literature Review **4**
4.1 Distributed file systems 4
4.2 Content-addressable storage 4
4.3 Chunking 4
4.4 Deduplication systems 4
4.5 Contribution of this project 4
Chapter 5 — Dataset Description **5**
5.1 Nature of the input data 5
5.2 Authentic source 5
5.3 Format and size 5
5.4 Download method and provenance 5
5.5 Duplication test methodology 5
5.6 Restrictions and limitations 5
Chapter 6 — Dataset Source **6**
6.1 Source links 6
6.2 Citation and attribution 6
6.3 Why generated data was not used 6
Chapter 7 — Tools and Technologies **7**
7.1 Hadoop, HDFS and WebHDFS 7
7.2 Python and FastAPI 7
7.3 React and Next.js 7
7.4 Metadata database 7
7.5 SHA-256 7
7.6 Docker 7
7.7 Testing libraries 7
7.8 Visualization technologies 7
Chapter 8 — System Requirements **8**
8.1 Functional requirements 8
8.2 Nonfunctional requirements 8
8.3 Hardware requirements 8
8.4 Software requirements 8
8.5 Constraints 8
8.6 Assumptions 8
Chapter 9 — System Architecture **9**
9.1 High-level architecture 9
9.2 HDFS cluster architecture 9
9.3 Upload pipeline 10
9.4 Deduplication pipeline 10

9.5 Chunk-level storage model 10
9.6 File reconstruction 10
9.7 Replication workflow 10
9.8 Database design 10
9.9 Data flow 10
9.10 Sequence of a new upload 11
Chapter 10 — Implementation and Methodology 14
10.1 Environment setup 14
10.2 HDFS initialization 14
10.3 Backend architecture 14
10.4 Database initialization 14
10.5 Frontend integration 14
10.6 File upload and SHA-256 computation 14
10.7 Duplicate identification and content-addressable storage 14
10.8 Chunk processing and file manifests 14
10.9 Reference handling and safe deletion 14
10.10 Safe optimization 14
10.11 Recovery 15
10.12 Replication management 15
10.13 Analytics calculation 15
10.14 Error management 15
Chapter 11 — Complete Source Code 16
11.1 Infrastructure and deployment 16
11.2 Backend: core, models and HDFS client 20
11.3 Backend: deduplication and services 37
11.4 Backend: API routes 74
11.5 Frontend: libraries and shared components 84
11.6 Frontend: pages 105
11.7 Tests 139
11.8 Reproduction and tooling scripts 165
Chapter 12 — Execution Screenshots 195
Chapter 13 — Output and Result Screenshots 207
Chapter 14 — Result Analysis 223
14.1 Experiment methodology 223
14.2 Formulas 223
14.3 Unique files (Experiment A) 223
14.4 Exact duplicates (Experiments B and C) 223
14.5 Chunk-level deduplication (Experiment D) 223
14.6 Boundary shift (Experiment D2) 223
14.7 Before and after optimization (Experiment I) 224
14.8 Restoration (Experiments E and G) 225
14.9 Deletion with shared content (Experiment F) 225
14.10 Replication (Experiment H) 225
14.11 Failure handling (Experiment J) 225
14.12 Small-file overhead 225
14.13 Logical savings versus physical usage 225
14.14 Timing validity 225
14.15 Summary of observations 225
Chapter 15 — Limitations 226
15.1 Small-file overhead 226
15.2 Metadata costs 226
15.3 Docker resource limitations 226
15.4 Single-machine versus multi-machine deployments 226
15.5 Chunking limitations 226
15.6 Hashing overhead 226
15.7 Concurrency limitations 226
15.8 Scalability considerations 226
15.9 HDFS-specific restrictions 226
15.10 Security 226
Chapter 16 — Future Scope 227

16.1 Content-defined chunking 227
16.2 Object packing 227
16.3 Large-scale distributed indexing 227
16.4 Apache Spark integration 227
16.5 Stronger multi-user authorization 227
16.6 Incremental background deduplication 227
16.7 More advanced monitoring 227
16.8 Enterprise deployment 227
Chapter 17 — Conclusion 228
17.1 What was built 228
17.2 What was verified 228
17.3 Results achieved 228
17.4 Technical significance 228
17.5 Future possibilities 228
References 229
Appendix A: Setup Commands 230
Appendix B: API Documentation 231
Appendix C: Configuration 233
Appendix D: Database Schema 234
Appendix E: Test Results 238
Appendix F: Benchmark Records 239
Appendix G: Dataset Provenance 240
Appendix H: Troubleshooting 241

List of Figures

Figure 1 High-level system architecture 9
Figure 2 HDFS cluster architecture as deployed 10
Figure 3 Upload pipeline: bytes are made durable before metadata is committed 10
Figure 4 Deduplication decision for an uploaded file 10
Figure 5 Chunk-level storage model 10
Figure 6 File reconstruction with verification 10
Figure 7 Replication change and verification 10
Figure 8 Database entity-relationship diagram 10
Figure 9 Data flow diagram 11
Figure 10 Sequence diagram for a new whole-file upload 13
Figure 11 HDFS cluster running in Docker, with Hadoop version output 195
Figure 12 `hdfs dfsadmin -report`: live DataNodes and capacity. All DataNodes share one VM disk, so capacity is counted once per node. 196
Figure 13 Apache Hadoop's own NameNode web interface 197
Figure 14 DataNode list in the NameNode interface 198
Figure 15 One DataNode's own web interface 198
Figure 16 Running backend: health check, live statistics and request log 199
Figure 17 Generated OpenAPI documentation, served by the backend 200
Figure 18 Running frontend on an empty system. Undefined metrics show N/A, not 0%. 200
Figure 19 Dataset lab with recorded provenance 201
Figure 20 Dataset ingestion with labelled workload copies 202
Figure 21 File upload through the interface. The second upload of the same bytes under a new name is reported as a duplicate. 202
Figure 22 File explorer after ingestion 203
Figure 23 Deduplication execution: confirmation after a reviewed preview 204
Figure 24 HDFS command-line view of the managed namespace after optimization 204
Figure 25 `hdfs fsck` on the managed namespace, and the replica locations of one object 205
Figure 26 Unit test execution 205
Figure 27 Integration tests on the real HDFS cluster 205
Figure 28 Browser tests in Chrome, on the real cluster 206
Figure 29 Cluster monitoring page with live NameNode and DataNode data 206
Figure 30 Duplicates found: each group has one SHA-256 and shows how logical names map to stored copies. Here the copies were stored without deduplication. 207
Figure 31 Dry-run preview. Figures are labelled as estimates, and nothing has changed yet. 208
Figure 32 Completed whole-file optimization: measured before and after, and every file verified 209
Figure 33 Dry-run preview of chunk-level optimization 210
Figure 34 Completed chunk-level optimization 211
Figure 35 Chunk map: blue marks are chunks shared with another file. A truncated copy and a one-byte edit share almost everything; a one-byte insertion at the start shares nothing. 212
Figure 36 Dashboard with unique content retained and storage savings 213
Figure 37 Analytics page with performance measurements and integrity results 214
Figure 38 Replication after a verified change to factor 2. Measured space fell accordingly. 215
Figure 39 Replication restored to factor 3, with both changes recorded 215
Figure 40 Lowering replication requires explicit confirmation 216
Figure 41 File details: replica locations, and a passed SHA-256 verification 216
Figure 42 Restored file compared with the publisher's bytes: identical SHA-256, and `cmp` reports no difference . 216
Figure 43 Real failure: a stopped DataNode is reported as dead, and all managed blocks are under-replicated . . 217
Figure 44 While a DataNode is dead, the maximum selectable factor falls to 2 217
Figure 45 Real failure: the NameNode is stopped, and the interface reports it 218
Figure 46 An upload during the NameNode outage fails with a clear error, and no file is recorded 218
Figure 47 Experiments run from the lab, showing measured results 219
Figure 48 Audit trail of the session 220
Figure 49 Settings, with real diagnostics and a metadata-to-HDFS comparison 220
Figure 50 Dashboard in dark mode 221
Figure 51 Dashboard at phone width 222
Figure 52 Measured HDFS space for exact duplicates 223
Figure 53 Chunk size trade-off, measured. Bars on throughput show the observed range across three runs. 223
Figure 54 Chunk reuse after three small edits 224

Figure 55 Before and after optimization, as measured by the NameNode and on DataNode disks 224
Figure 56 Verified restore throughput 225

List of Tables

Table 1 Original dataset files as downloaded 5
Table 2 Documented partial-overlap workload files 5
Table 3 Nine authentic books with $N - 1$ labelled exact copies each. Upload shows the mean and observed range of three runs. 223
Table 4 The same files at each chunk size. Disk is du across three DataNodes, including replicas. 223
Table 5 Optimization in place, measured three ways. Optimize shows the mean and range of three runs. 224

Abbreviations

API	Application Programming Interface
CAS	Content-Addressable Storage
CDC	Content-Defined Chunking
CORS	Cross-Origin Resource Sharing
CRC	Cyclic Redundancy Check
CVD	Color-Vision Deficiency
DN	DataNode
ER	Entity-Relationship
FIPS	Federal Information Processing Standards
GC	Garbage Collection
HA	High Availability
HDFS	Hadoop Distributed File System
JMX	Java Management Extensions
NN	NameNode
REST	Representational State Transfer
RPC	Remote Procedure Call
SHA-256	Secure Hash Algorithm, 256-bit
SQL	Structured Query Language
UI	User Interface
VM	Virtual Machine
WAL	Write-Ahead Logging

Chapter 1 — Introduction

1.1 Background

Data volumes grow faster than the budgets that store them, and a large share of stored data is redundant. Distributed file systems make capacity scalable, but they add their own multiplier: each byte is stored several times for durability. Removing accidental redundancy, while keeping deliberate redundancy, is therefore worthwhile.

1.2 Distributed storage

A distributed file system presents one namespace over many machines. Files are split into blocks spread across servers, so capacity and throughput grow by adding machines. Failure is treated as normal: at this scale some disk or machine is always failing, so the system must detect failure and recover automatically [1].

1.3 The Hadoop ecosystem

Apache Hadoop began as an open-source implementation of ideas from Google's distributed systems, and it contains a storage layer (HDFS), a resource manager (YARN) and a batch engine (MapReduce). This project uses HDFS only.

1.4 HDFS fundamentals

HDFS has a primary/worker design [2], [3]. A single active **NameNode** keeps the directory tree and the mapping from files to blocks in memory, and persists changes in an edit log. Many **DataNodes** store blocks as local files. Clients ask the NameNode where blocks are, and transfer data directly with DataNodes, so the NameNode stays out of the data path. Files are written once and then read many times. The default block size is 128 MiB.

1.5 Replication

Each block is copied to several DataNodes, three by default. DataNodes send heartbeats, every 3 seconds by default, and regular block reports. When a DataNode stops reporting, the NameNode schedules new copies of its blocks [3]. Replication is what makes commodity hardware safe, and it is intentional redundancy.

1.6 Data redundancy

A different kind of redundancy comes from users and applications: the same dataset uploaded by two teams, repeated backups, or new versions of a large file. Studies of real systems find substantial duplicate content [4]. HDFS does not detect any of this, since each upload is an unrelated file to it, so each copy is also replicated.

1.7 Deduplication

Deduplication stores identical content once and keeps references to it [5], [6]. Whole-file deduplication compares files by a cryptographic hash. Chunk-level deduplication compares pieces of files, finding overlap between files that differ overall, at the cost of more metadata.

1.8 Storage optimization

Storage optimization here means reducing the bytes retained for a given set of logical files without losing data or weakening fault tolerance. Reducing the replication factor would also save space, but it trades away durability, so this project keeps it as a separate, explicit decision.

1.9 Motivation

A student or small team using HDFS has no easy way to see how much of their storage is redundant, or to reclaim it safely. Deleting duplicates by hand risks data loss. Existing academic work often reports savings by simulation or trace analysis; this project builds the mechanism on a real cluster and measures it.

1.10 Project scope

The project delivers a working application: a backend service, a web dashboard, a real multi-node HDFS deployment, whole-file and fixed-size chunk deduplication, verified restoration, replication management, monitoring and reproducible experiments on authentic files. It does not modify HDFS, does not deduplicate internal HDFS blocks, and is not a production multi-tenant service.

Chapter 2 — Problem Statement

2.1 Existing storage inefficiencies

For N identical copies of a file of size S at replication factor R, plain HDFS consumes about $N \cdot S \cdot R$ bytes. Only $S \cdot R$ is needed to give the same fault tolerance. Nothing in HDFS reveals the difference.

2.2 Redundancy problems

The project addresses ten specific problems: storage wasted by duplicate files; repeated content within different files; difficulty identifying redundant data across a distributed store; poor visibility into storage consumption; difficulty interpreting logical versus replicated physical consumption; the lack of a usable optimization dashboard; integrity risks during deduplication; safe restoration of logically distinct files; monitoring replication health; and measuring efficiency with reproducible experiments.

2.3 Duplication versus replication

These terms are easily confused, so the project keeps them strictly separate. **Replication** is block-level copies that HDFS creates and manages for fault tolerance. **Duplication** is separate files with the same content, created by users. Deduplication removes the second and leaves the first intact: the single remaining copy is still replicated R times.

2.4 Operational limitations

Deduplicating above HDFS raises problems that a built-in feature would not have. HDFS and an external metadata store cannot share a transaction. Files are immutable, so content cannot be merged in place. Every extra file costs NameNode memory [7], which limits how finely data can be chunked. Deleting one logical file must never destroy content that another still uses.

2.5 Need for the proposed system

A layer is needed that identifies duplicates by content, stores each distinct content once, keeps every logical file independently usable, proves that restored bytes are correct, keeps HDFS replication intact, and reports honest numbers. The system described here is that layer.

Chapter 3 — Objectives

3.1 Completed objectives

All of the following are implemented, and each is covered by automated tests or experiments described in this report.

Objective	Evidence
HDFS-backed file management	Ch. 12, 13
Exact duplicate detection with SHA-256	Exp. B, C
Identical-chunk detection across files	Exp. D, D2
Content-addressable storage of unique data	Ch. 10
Logical references preserved	Exp. C, F
Storage optimized without data loss	Exp. I
Exact reconstruction of original files	Exp. E, G
Cryptographic integrity verification	Exp. E, J3
Supported replication settings managed	Exp. H
NameNode and DataNode health monitored	Ch. 12
Storage and deduplication metrics displayed	Ch. 13
Safe deletion and recovery	Exp. F
Evaluation on authentic files	Ch. 5, 14
Visual comparison of storage efficiency	Ch. 13, 14
Architecture and results documented	This report

3.2 Proposed future enhancements

These are **not** implemented: content-defined chunking, chunk packing, a distributed index, background deduplication with Apache Spark, multi-user authorization, and deployment on separate physical machines. See Chapter 16.

Chapter 4 — Literature Review

4.1 Distributed file systems

The Google File System [1] established the design of a single metadata master with many chunkservers, large blocks and replication on commodity hardware. Shvachko et al. [2] describe HDFS, which follows the same model, with a NameNode and DataNodes, and report on its operation at scale. The Apache documentation [3], [8] specifies the behaviour this project relies on, including replication, rename semantics and the REST interface. A Cloudera engineering article [7] explains why many small files are a problem: each file, directory and block is an object in NameNode memory, at about 150 bytes each as a rule of thumb.

4.2 Content-addressable storage

Venti [9] is an early archival system that addresses blocks by the hash of their contents, so identical blocks coalesce automatically and stored data is never overwritten. The object store in this project uses the same principle, at a higher level and using SHA-256 [10] rather than SHA-1, for which practical collisions have since been demonstrated [11].

4.3 Chunking

LBFS [12] introduced content-defined chunking to deduplicated file transfer: it uses Rabin fingerprints to place chunk boundaries by content, so that an insertion affects only nearby chunks. FastCDC [13] reduces the CPU cost of this approach while achieving a similar deduplication ratio. This project uses fixed-size chunking, for simplicity and determinism, and measures the weakness these papers set out to solve.

4.4 Deduplication systems

Zhu et al. [14] describe the Data Domain file system, which avoids an on-disk index lookup for most chunks by using a Bloom filter summary and locality-preserving layout. That is the problem of index scale, which this project's SQLite index does not address. Meyer and Bolosky [4] analyse real desktop file systems and find that whole-file deduplication achieves about three quarters of the savings of aggressive block-level deduplication. That result motivated offering both modes. Paulo and Pereira [5] and Xia et al. [6] survey the field and classify systems by granularity, locality, timing and scope.

4.5 Contribution of this project

This project does not propose a new deduplication algorithm. It contributes a complete, inspectable implementation above an unmodified HDFS cluster, which combines staged writes to keep two independent stores consistent, verify-before-delete conversion of existing data, and query-based reference tracking. It also separates derived, measured and estimated values in both its evaluation and its interface, and it reports results from real experiments, including real failures.

Chapter 5 — Dataset Description

5.1 Nature of the input data

Deduplication is not machine learning, so no labelled dataset is required. The system needs real files: hashing, chunking and transfer should be exercised on authentic bytes with natural structure.

5.2 Authentic source

All original files are public-domain books from Project Gutenberg [15], including the three suggested by the project brief: *Pride and Prejudice* (ebook 1342), *Alice's Adventures in Wonderland* (11) and *The Adventures of Sherlock Holmes* (1661).

5.3 Format and size

The dataset contains 15 original files totalling 39.19 MiB: plain text for all 9 works, plus HTML and EPUB versions of the three primary works. These give a realistic mix: highly compressible text, markup, and already-compressed archives. Table 1 lists every file.

Table 1: Original dataset files as downloaded

Title	ID	Format	Bytes
Pride and Prejudice	1342	txt	772386
Pride and Prejudice	1342	html	852589
Pride and Prejudice	1342	epub	24864457
Alice's Adventures in Wonderland	11	txt	174311
Alice's Adventures in Wonderland	11	html	192539
Alice's Adventures in Wonderland	11	epub	189249
The Adventures of Sherlock Holmes	1661	txt	607606
The Adventures of Sherlock Holmes	1661	html	640229
The Adventures of Sherlock Holmes	1661	epub	379503
Frankenstein; Or, The Modern Prometheus	84	txt	448885
Dracula	345	txt	890348
A Tale of Two Cities	98	txt	807197
Moby Dick; Or, The Whale	2701	txt	1276315
The Complete Works of William Shakespeare	100	txt	5638481
War and Peace	2600	txt	3359610

5.4 Download method and provenance

`scripts/download_dataset.py` fetches each file and writes `datasets/provenance/manifest.json`. For each file it records the title, author, ebook ID, source organization, source page, requested and final URLs, download date, format, size, SHA-256, kind and any transformation. For plain-text files, the title printed inside the file was also compared with the catalog, and all matched.

Project Gutenberg's policy states that its website is for human users and directs automated tools to mirrors [16]. This policy was noticed after the files in this report had been fetched once from `www.gutenberg.org`, at one request every two seconds. The manifest records those URLs truthfully. The tool was then changed to use the mirror operated by the Project Gutenberg Literary Archive Foundation. As a check, `pg11.txt` from that mirror has the same SHA-256 as the recorded file.

5.5 Duplication test methodology

Natural duplicates are not available in a public book collection, so duplicates are created under control, in two labelled ways:

- **Workload copies** are the same bytes ingested under another logical name, labelled `workload_copy`.
- **Transformed files** are byte-level edits of *Pride and Prejudice*, labelled `transformed`, for partial-overlap tests. Table 2 describes them.

Table 2: Documented partial-overlap workload files

File	Transformation	Bytes
`truncate_90`	Kept the first 90% of the original bytes and removed the rest.	695147
`prefix_newline`	Inserted one newline byte (0x0A) before the first original byte, shifting every later byte by one.	772387
`replace_mid_byte`	Replaced the single byte at the file's midpoint with a different byte, keeping the length unchanged.	772386

No text was invented. The only bytes these edits add are one newline or one replaced byte. The originals are never altered.

5.6 Restrictions and limitations

The dataset cannot show behaviour at production scale, because it is about 41 MiB. All files are smaller than one HDFS block, so each occupies a single block. Duplication is created deliberately, so measured savings verify the mechanism on known workloads, and they do not estimate the redundancy of any real system.

Chapter 6 — Dataset Source

6.1 Source links

- Project Gutenberg: https://www.gutenberg.org/
- *Pride and Prejudice*: https://www.gutenberg.org/ebooks/1342
- *Alice's Adventures in Wonderland*: https://www.gutenberg.org/ebooks/11
- *The Adventures of Sherlock Holmes*: https://www.gutenberg.org/ebooks/1661
- License: https://www.gutenberg.org/policy/license.html
- Robot access policy: https://www.gutenberg.org/policy/robot_access.html

The remaining works are linked in the provenance appendix.

6.2 Citation and attribution

Files are cited as Project Gutenberg [15], and are used under the Project Gutenberg License [17]. The system stores files as distributed, with the license header intact, and does not redistribute them with this report.

6.3 Why generated data was not used

Generated data can be shaped to produce any desired result, and a reader cannot independently check it. Every original file here can be traced to a public URL and confirmed by its hash, and every derived file has a stated transformation. Unit tests do use small deterministic byte patterns, but these exercise program logic only, and no evaluation result in this report comes from them.

Chapter 7 — Tools and Technologies

7.1 Hadoop, HDFS and WebHDFS

Apache Hadoop 3.5.0 provides the distributed storage. WebHDFS [8] is its REST interface, where a `CREATE` request first redirects to a DataNode, and the data is then sent there. It lets the Python backend work with no Java client. Monitoring uses the NameNode's JMX and fsck servlets.

7.2 Python and FastAPI

The backend uses Python 3.12 and FastAPI [18], which provides validation through Pydantic and generates OpenAPI documentation. HTTPX is the HTTP client. Uploads are spooled to disk and streamed, so memory use does not grow with file size.

7.3 React and Next.js

The frontend uses Next.js 16 and React 19 with TypeScript and Tailwind CSS, components from shadcn/ui, Recharts for charts, TanStack Query for data fetching, and Motion for restrained transitions.

7.4 Metadata database

SQLite, in write-ahead-logging mode with foreign keys enforced, is accessed through SQLAlchemy 2.0, with schema migrations managed by Alembic. It is a single file and needs no server, which suits a project of this scale, though not a large deployment.

7.5 SHA-256

SHA-256 [10] produces a 256-bit digest. Content identity is defined entirely by this digest and the size. MD5 and SHA-1 were rejected because practical collisions are known [11], which would let a crafted file masquerade as another.

7.6 Docker

HDFS runs in Docker Compose under a Colima VM. The official `apache/hadoop` images are amd64-only, so a Dockerfile builds a native arm64 image from the official Apache release tarball and verifies its SHA-512.

7.7 Testing libraries

Pytest runs the unit and integration tests. Playwright drives real browser tests in Google Chrome.

7.8 Visualization technologies

The dashboard uses Recharts, and the report's charts use Matplotlib, both generated from measured data. Colors come from a categorical palette checked for color-vision deficiency, and every interface chart has a table view. Diagrams are generated from a script as SVG files.

Chapter 8 — System Requirements

8.1 Functional requirements

ID	Requirement
FR1	Upload files by streaming, with progress
FR2	Detect identical content regardless of name
FR3	Store whole-file objects and fixed-size chunks
FR4	Rebuild any file exactly, and verify it
FR5	Convert existing data safely, after a dry run
FR6	Delete recoverably; protect shared content
FR7	Change the replication factor, and verify it
FR8	Show cluster health and storage measurements
FR9	Record an audit trail of all operations
FR10	Import authentic data with provenance, and run experiments

8.2 Nonfunctional requirements

ID	Requirement
NF1	No data loss from any single failure of a process or step
NF2	Memory use independent of file size, up to one chunk
NF3	Numbers must state how they are known
NF4	Responsive, accessible interface in light and dark modes
NF5	Reproducible through scripts and documented steps
NF6	Free to operate, using open-source software

8.3 Hardware requirements

The system was developed and evaluated on an Apple M1 laptop with 8 GB of RAM and 8 CPU cores. The Docker VM needs 4 GB of memory for the standard cluster, of which the four Hadoop daemons used about 700 MiB when idle. About 6 GB of free disk is needed, mostly for the Hadoop image.

8.4 Software requirements

macOS or Linux; Docker with Compose, or Colima; Python 3.12; Node.js 22; Google Chrome for browser tests; Typst to rebuild this report. Exact versions are pinned in `requirements.txt` and `package-lock.json`.

8.5 Constraints

- HDFS files are write-once, so content is immutable.
- WebHDFS offers no multi-file transaction.
- All “machines” share one host in this deployment.

8.6 Assumptions

- One API process runs at a time. Internal locks depend on this.
- SHA-256 collisions are treated as impossible, and only the size is compared as well.
- Users are trusted. Access control is limited to an optional shared token.

Chapter 9 — System Architecture

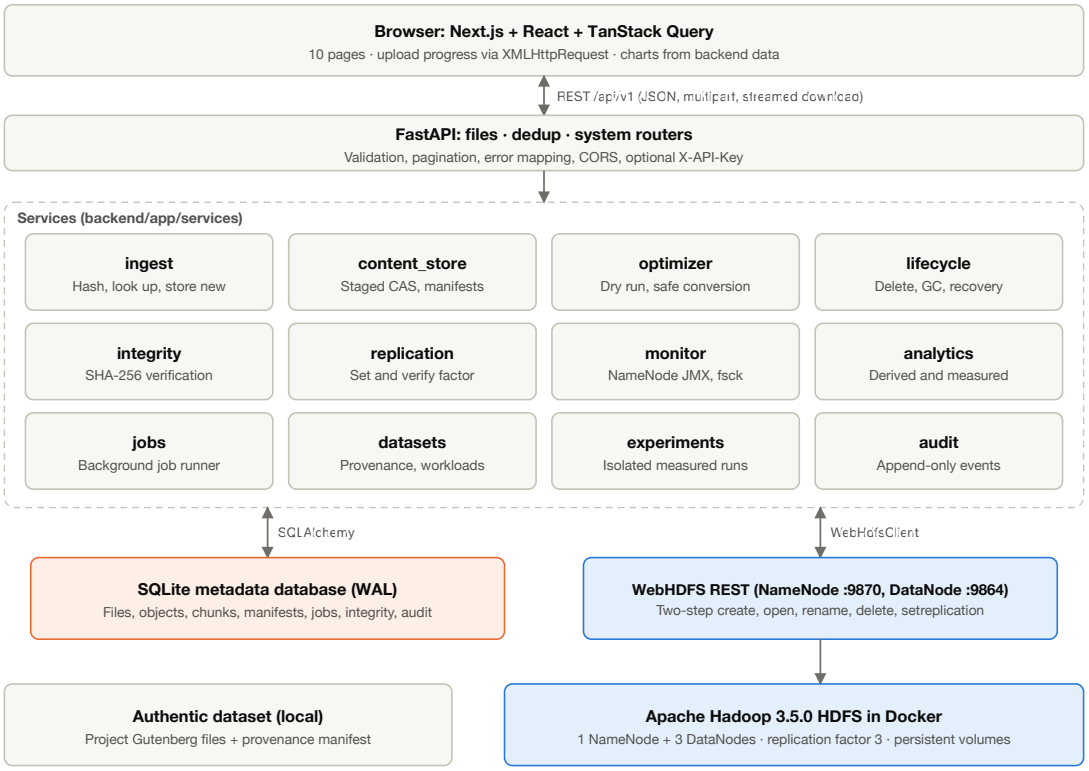

Figure 1: High-level system architecture

9.1 High-level architecture

Figure 1 shows the layers. A browser calls a versioned REST API. Routers validate input and call services. Services use two stores: SQLite for metadata, and HDFS, through a small `HdfsClient` interface, for all content. Every service receives its dependencies through an explicit context object. That is what allows an experiment to run against a private directory and database without touching managed data.

9.2 HDFS cluster architecture

The cluster in Figure 2 has one NameNode and three DataNodes in containers, with persistent volumes. The backend runs outside the Docker network in the standard setup. WebHDFS redirects name DataNodes by container hostname, so the client maps these to published ports. The NameNode formats its directory only if it is empty, so a restart cannot wipe metadata. Dead-node detection was shortened from the default

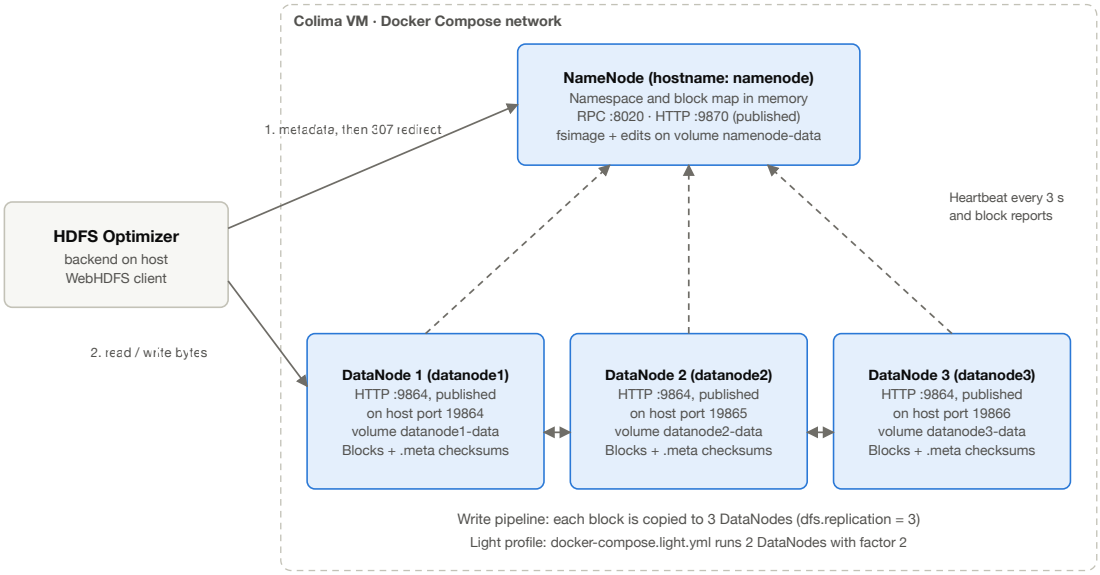

Figure 2: HDFS cluster architecture as deployed

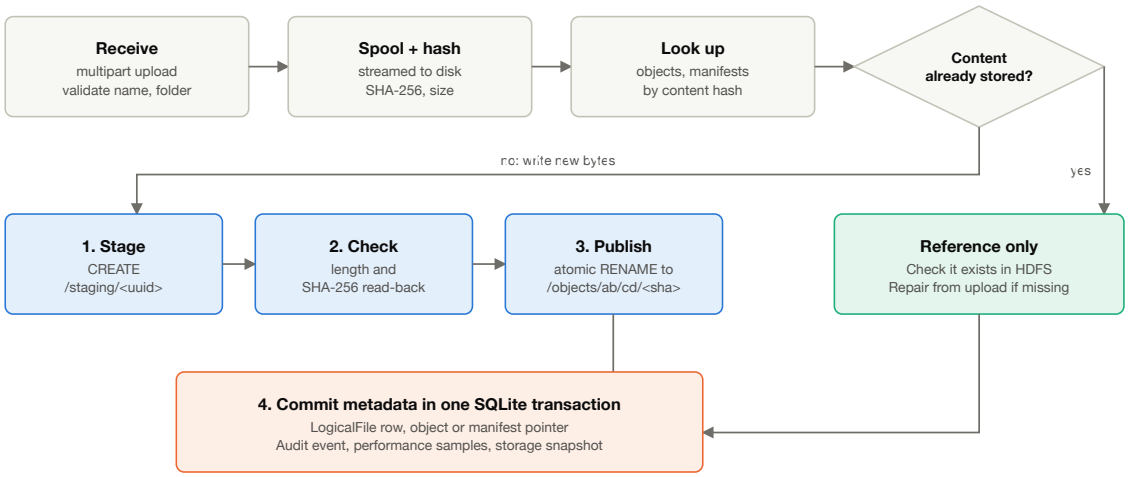

Figure 3: Upload pipeline: bytes are made durable before metadata is committed

10.5 minutes to about 90 s, so failures can be observed in a demonstration.

9.3 Upload pipeline

Figure 3 shows a whole-file upload. Staging and rename make publication atomic: HDFS rename refuses to overwrite an existing path, a behaviour the integration tests confirm on the real cluster.

9.4 Deduplication pipeline

Figure 4 shows how a storage decision is made. If any active form of the content already exists, the upload adds a reference, even if the existing form differs from the requested mode. This avoids storing the same content twice in two forms.

9.5 Chunk-level storage model

In Figure 5, logical files point at manifests, and manifests list chunks in order. Identical files share a manifest. Two files that differ by one chunk share all the others. Each chunk is a separate HDFS file, not an HDFS block.

9.6 File reconstruction

Figure 6 shows a verified restore. Each chunk is checked as it is read, and the whole stream is compared at the end. Since the response is already streaming, a failure aborts the connection, so a client cannot mistake damaged data for a complete download.

9.7 Replication workflow

Figure 7 shows a replication change. HDFS applies changes asynchronously, so success is observed rather than assumed.

9.8 Database design

Figure 8 shows the schema. A `files` row has exactly one storage pointer, enforced by a `CHECK` constraint. Foreign keys to stored content use `ON DELETE RESTRICT`. `file_chunks` rows are the authoritative chunk references, and whether content is still needed is always a query. No counter is stored, so none can drift.

9.9 Data flow

Figure 9 shows how data moves between processes and the two stores.

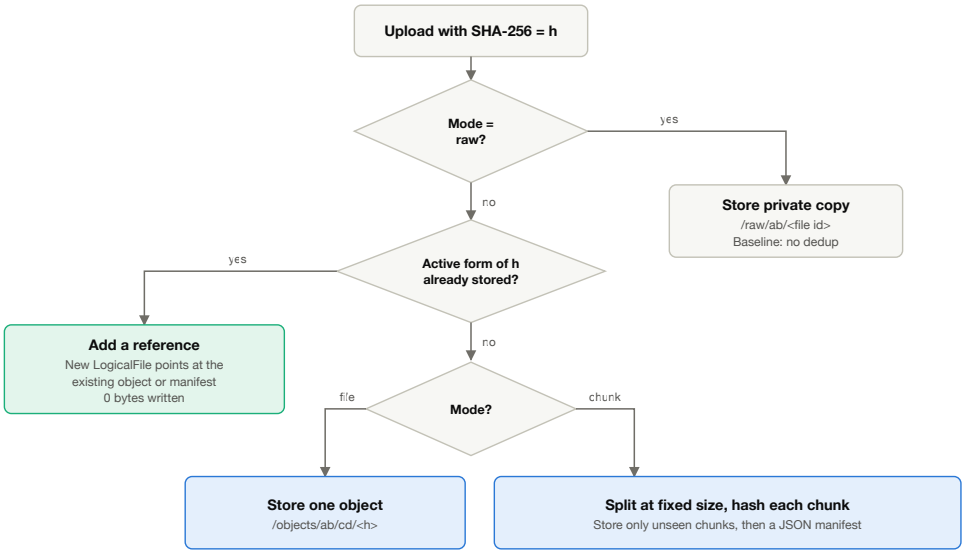

Figure 4: Deduplication decision for an uploaded file

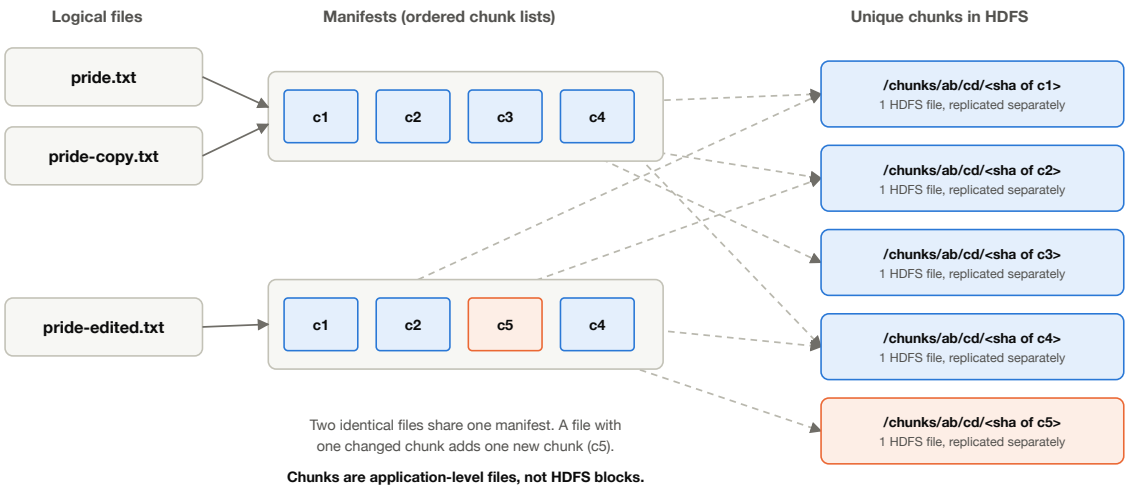

Figure 5: Chunk-level storage model

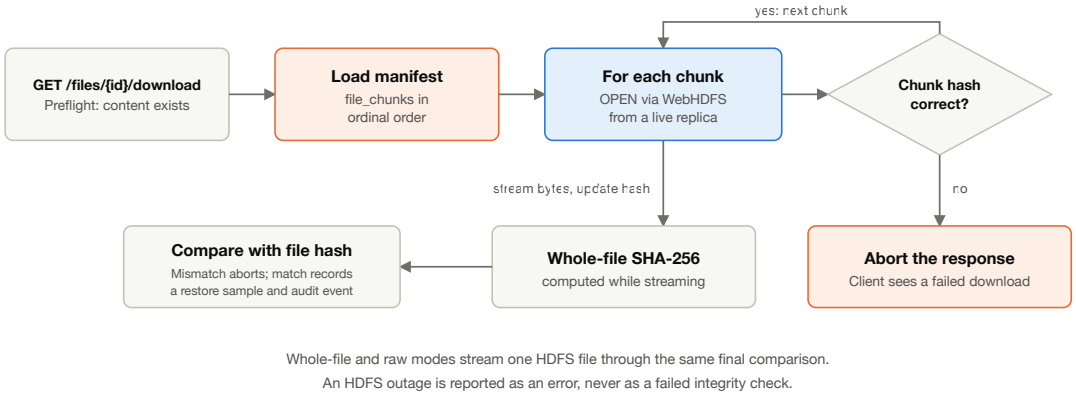

Figure 6: File reconstruction with verification

9.10 Sequence of a new upload

Figure 10 shows the messages exchanged when new whole-file content is uploaded.

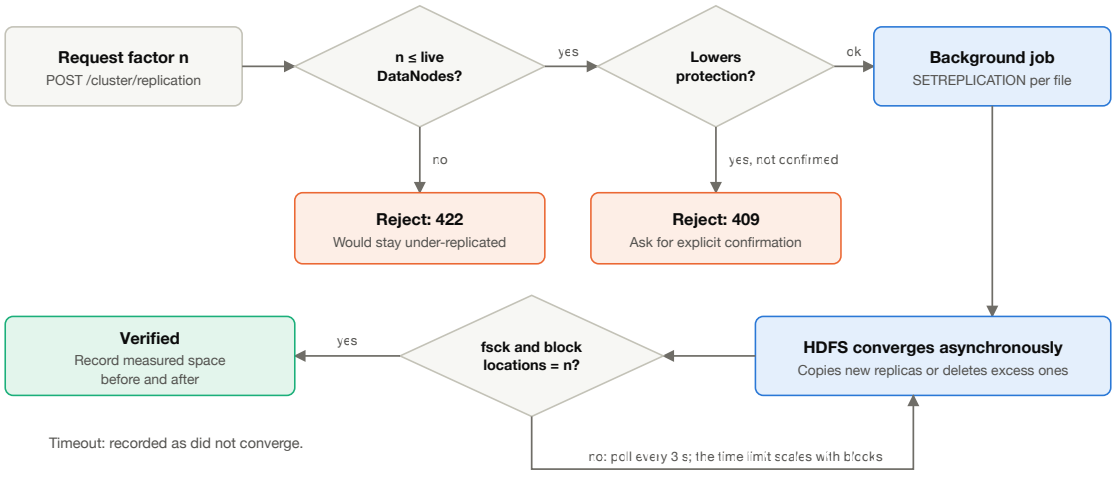

Figure 7: Replication change and verification

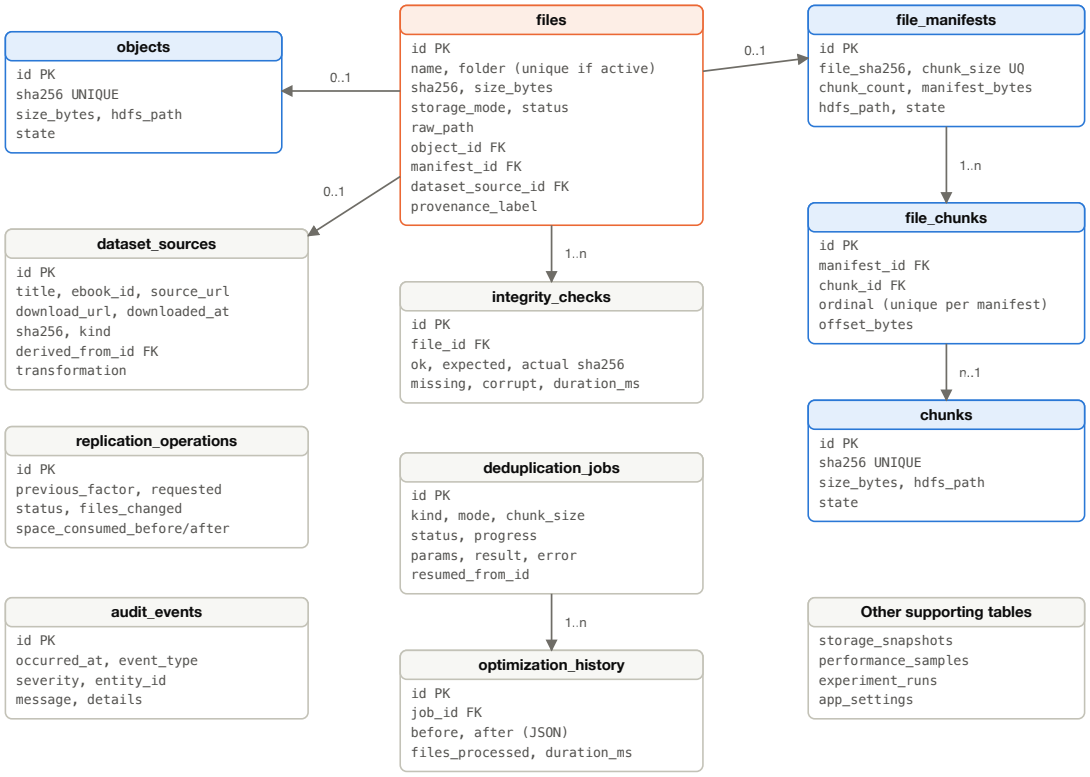

Figure 8: Database entity-relationship diagram

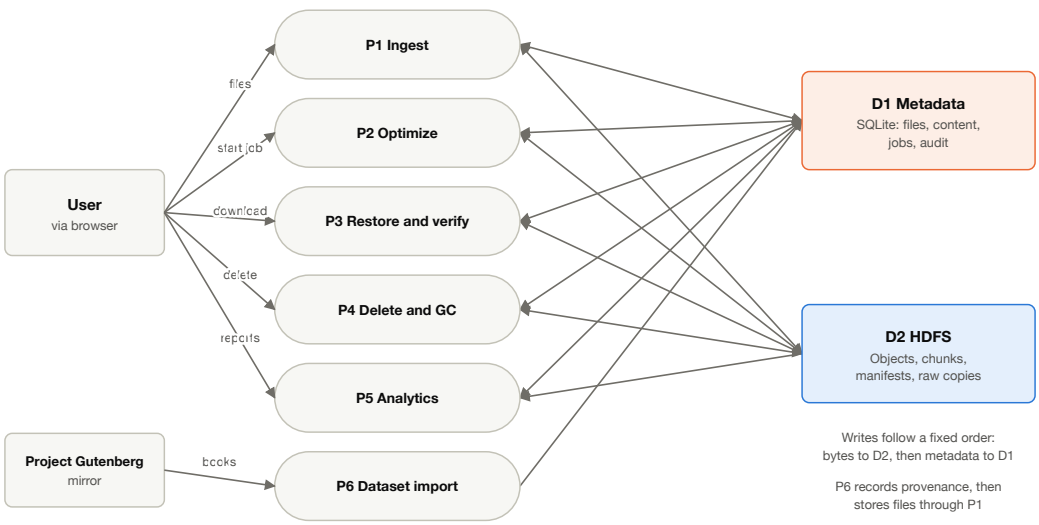

Figure 9: Data flow diagram

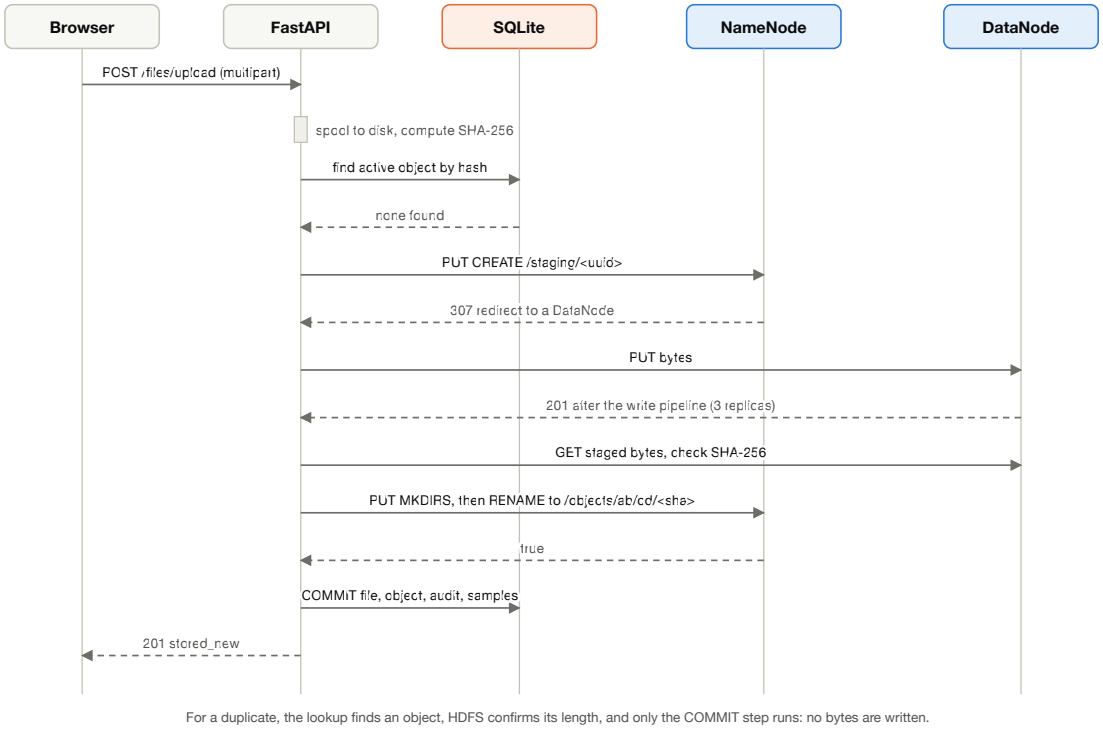

Figure 10: Sequence diagram for a new whole-file upload

Chapter 10 — Implementation and Methodology

10.1 Environment setup

`scripts/setup.sh` creates the Python environment, installs frontend dependencies, starts the container runtime and builds the Hadoop image. A project-local Docker configuration avoids a stale credential helper left behind by a removed Docker Desktop installation.

10.2 HDFS initialization

Compose starts the NameNode, and it waits until a JMX health check passes before starting DataNodes. An entrypoint renders the configuration from templates, including the replication factor. On startup, the backend creates its namespace: `/hdfs-optimizer/{objects, chunks, manifests, staging, recovery, raw}`.

10.3 Backend architecture

Routers contain no storage logic. `WebHdfsClient` maps WebHDFS errors onto typed exceptions, so an outage becomes HTTP 503. All HDFS paths come from hashes or generated IDs, never from user input, and folders are virtual and exist only in metadata. This rules out path traversal by construction, and it is also tested.

10.4 Database initialization

Alembic migrations run at startup, and a test confirms that they build the same tables as the models. Each connection enables foreign keys, write-ahead logging and a 30-second busy timeout.

10.5 Frontend integration

Pages fetch through TanStack Query, and jobs are polled until they finish. Uploads use `XMLHttpRequest`, because it reports the bytes sent, so upload progress is genuine. A browser test confirms that an empty system shows N/A rather than a fabricated value.

10.6 File upload and SHA-256 computation

An upload is spooled to disk in 1 MiB buffers, hashing as it goes. Time is $O(n)$ in the file size, and memory is constant.

10.7 Duplicate identification and content-addressable storage

```
Algorithm 1: INGEST(file, mode)
  h, n ← SHA256(file), SIZE(file)
  acquire content_lock
  if mode = raw:
      PUT_STAGED(/raw/<id>, file, h, n)
      pointer ← raw path
  else if an active object or manifest of h
exists:
      check it exists in HDFS; repair from file
if not
      pointer ← existing form             ▷ 0
bytes written
  else if mode = file:
      PUT_STAGED(/objects/h[0:2]/h[2:4]/h,
file, h, n)
      pointer ← object
  else:
      pointer ← BUILD_MANIFEST(file, h, n,
chunk_size)
  commit LogicalFile(pointer), audit, samples

Algorithm 2: PUT_STAGED(path, data, h, n)
  if path exists and SHA256(path) = h: return
▷ adopt
  s ← /staging/<uuid>
  CREATE(s, data); assert LENGTH(s) = n
  if verify_on_write: assert SHA256(s) = h
  RENAME(s, path)
▷ atomic
```

Lookup uses a unique index, so it is $O(\log m)$ for m stored contents.

10.8 Chunk processing and file manifests

```
Algorithm 3: BUILD_MANIFEST(stream, h, n, c)
  for each block b of exactly c bytes in
stream:
      d ← SHA256(b); append (d, |b|, offset) to
plan
      if d is new in this run:
          if chunk d is active and intact:
reuse
          else PUT_STAGED(/chunks/.../d, b, d,
|b|)
  assert SHA256(stream) = h             ▷
source did not change
  PUT_STAGED(manifest JSON)
  commit chunk rows, manifest, file_chunks in
one transaction
```

Rows are added only after all HDFS writes finish, so the database write lock is held briefly. The manifest also exists as a JSON file in HDFS, and verification checks that it agrees with the database.

10.9 Reference handling and safe deletion

Soft deletion marks the logical file only; its content remains, because the file can still be restored. Permanent removal clears the pointers, then runs collection:

```
Algorithm 4: COLLECT_GARBAGE()
  acquire content_lock
  O ← active objects no non-purged file points
at
  M ← active manifests no non-purged file
points at
  C ← active chunks in no surviving manifest
  mark O ∪ M ∪ C as gc_pending and commit
  for each: DELETE from HDFS, mark removed,
commit
```

The state written first means that an interrupted collection is finished at the next startup. The complexity is that of the anti-join, about $O(k \log k)$ for k references.

10.10 Safe optimization

```
Algorithm 5: OPTIMIZE(mode)
  require a preview of the unchanged catalog
  for each eligible file f:             ▷
independent unit
      make the new form durable in HDFS
      read it back; assert SHA256 = f.sha256
      commit f's new pointer
```

```
      delete the old copy
  COLLECT_GARBAGE()
  record before and after measurements
```

A corrupted source is detected and left untouched, and the other files continue. An HDFS outage stops the job, since per-file failures would be misleading. Because converted files are no longer eligible, re-running the job is safe and processes only what remains.

10.11 Recovery

At startup, active jobs are marked interrupted, pending collections finish, and leftover staging files are removed. A reconciliation tool compares HDFS with the metadata, reporting missing files, and quarantines orphans in `/recovery/orphans` rather than deleting them.

10.12 Replication management

A request above the number of live DataNodes is refused, since those blocks could never be fully replicated. Lowering the factor needs explicit confirmation. A job then calls `SETREPLICATION` on each file and waits for convergence.

Testing on the real cluster revealed that `fsck` stops counting a replica as soon as it is marked excess, which is before the DataNode deletes it. The first implementation therefore reported success early. Verification now also requires `GET_BLOCK_LOCATIONS` to show the requested count.

10.13 Analytics calculation

Two formulas define the savings metrics:

$$\text{Savings } (\%) = \frac{\text{Logical} - \text{Stored}}{\text{Logical}} \times 100 \tag{1}$$

$$\text{Deduplication ratio} = \frac{\text{Logical}}{\text{Stored}} \tag{2}$$

Each is undefined when its denominator is zero, and the interface then shows N/A. Stored bytes are the sum of private raw copies, active objects and active chunks, derived from metadata. Replication-adjusted storage is labelled an estimate. Measured space comes from the NameNode’s `GETCONTENTSUMMARY`.

10.14 Error management

Errors have typed classes and a uniform JSON format. HDFS unavailability is distinguished from content corruption: a verification that cannot reach HDFS raises an error rather than recording a failed check, so an outage cannot be mistaken for data damage. Failures are recorded in the audit trail in their own transaction, so they survive the rollback of the failed operation.

Chapter 11 — Complete Source Code

This chapter contains the complete authored source: 108 files and 16000 lines. It is not a copy. When this PDF is built, each listing is read directly from the repository file, so the report and the repository cannot diverge. Long lines wrap.

The following are deliberately omitted because they are not authored: dependency directories and lock files (`node_modules`, `.venv`, `package-lock.json`), build output (`.next`), the generated UI primitives in `frontend/components/ui/`, which were created by the shadcn/ui command-line tool (their only change is an import path), and generated data such as benchmark results. The database migration is included.

11.1 Infrastructure and deployment

`docker-compose.yml` 100 lines

```yaml
# Standard deployment: 1 NameNode, 3 DataNodes, replication factor 3.
# Start HDFS:           docker compose up -d --wait
# Also containerise the app:  scripts/build-app-images.sh && docker compose --profile app up -d
name: hdfs-optimizer

x-hadoop: &hadoop
  image: hdfs-optimizer/hadoop:3.5.0
  build:
    context: infrastructure/docker/hadoop
    args: { HADOOP_VERSION: "3.5.0" }
  restart: unless-stopped
  init: true

x-datanode-health: &datanode-health
  test: ["CMD-SHELL", "curl -fsS 'http://localhost:9864/jmx?qry=Hadoop:service=DataNode,name=DataNodeInfo' | grep -q ClusterId"]
  interval: 10s
  timeout: 5s
  retries: 12
  start_period: 20s

services:
  namenode:
    <<: *hadoop
    hostname: namenode
    command: ["namenode"]
    environment:
      HDFS_REPLICATION: "${HDFS_REPLICATION:-3}"
      HDFS_NAMENODE_OPTS: "-Xmx512m"
    ports: ["127.0.0.1:9870:9870"]
    volumes:
      - ./infrastructure/hadoop:/templates:ro
      - namenode-data:/data
    healthcheck:
      test: ["CMD-SHELL", "curl -fsS 'http://localhost:9870/jmx?qry=Hadoop:service=NameNode,name=NameNodeStatus' | grep -q active"]
      interval: 10s
      timeout: 5s
      retries: 12
      start_period: 20s

  datanode1: &datanode
    <<: *hadoop
    hostname: datanode1
    command: ["datanode"]
    depends_on: { namenode: { condition: service_healthy } }
    environment: { HDFS_REPLICATION: "${HDFS_REPLICATION:-3}", DATANODE_HOSTNAME: datanode1, HDFS_DATANODE_OPTS: "-Xmx256m" }
    ports: ["127.0.0.1:19864:9864"]
    volumes:
      - ./infrastructure/hadoop:/templates:ro
      - datanode1-data:/data
    healthcheck: *datanode-health

  datanode2:
    <<: *datanode
    hostname: datanode2
    environment: { HDFS_REPLICATION: "${HDFS_REPLICATION:-3}", DATANODE_HOSTNAME: datanode2, HDFS_DATANODE_OPTS: "-Xmx256m" }
    ports: ["127.0.0.1:19865:9864"]
    volumes:
      - ./infrastructure/hadoop:/templates:ro
      - datanode2-data:/data

  datanode3:
    <<: *datanode
    hostname: datanode3
    environment: { HDFS_REPLICATION: "${HDFS_REPLICATION:-3}", DATANODE_HOSTNAME: datanode3, HDFS_DATANODE_OPTS: "-Xmx256m" }
    ports: ["127.0.0.1:19866:9864"]
    volumes:
      - ./infrastructure/hadoop:/templates:ro
      - datanode3-data:/data

  backend:
    profiles: ["app"]
    build: { context: backend }
    depends_on:
      namenode: { condition: service_healthy }
      datanode1: { condition: service_healthy }
```

```
    environment:
      HDFSOPT_NAMENODE_URL: http://namenode:9870
      HDFSOPT_DATABASE_PATH: /data/metadata.db
      HDFSOPT_DATASET_DIR: /datasets/raw
      HDFSOPT_CORS_ORIGINS: http://localhost:3100,http://127.0.0.1:3100
    ports: ["127.0.0.1:8100:8100"]
    volumes:
      - backend-data:/data
      - ./datasets:/datasets

  frontend:
    profiles: ["app"]
    build:
      context: frontend
      args: { NEXT_PUBLIC_API_URL: "http://localhost:8100" }
    depends_on: [backend]
    ports: ["127.0.0.1:3100:3100"]

volumes:
  namenode-data:
  datanode1-data:
  datanode2-data:
  datanode3-data:
  backend-data:
```

docker-compose.light.yml 56 lines

```
# Resource-constrained deployment: 1 NameNode, 2 DataNodes, replication factor 2.
# Replication factor 3 would be permanently under-replicated here, so it is not offered.
name: hdfs-optimizer-light

x-hadoop: &hadoop
  image: hdfs-optimizer/hadoop:3.5.0
  build:
    context: infrastructure/docker/hadoop
    args: { HADOOP_VERSION: "3.5.0" }
  restart: unless-stopped
  init: true

services:
  namenode:
    <<: *hadoop
    hostname: namenode
    command: ["namenode"]
    environment: { HDFS_REPLICATION: "2", HDFS_NAMENODE_OPTS: "-Xmx384m" }
    ports: ["127.0.0.1:9870:9870"]
    volumes:
      - ./infrastructure/hadoop:/templates:ro
      - light-namenode-data:/data
    healthcheck:
      test: ["CMD-SHELL", "curl -fsS 'http://localhost:9870/jmx?qry=Hadoop:service=NameNode,name=NameNodeStatus' | grep -q active"]
      interval: 10s
      timeout: 5s
      retries: 12
      start_period: 20s

  datanode1:
    <<: *hadoop
    hostname: datanode1
    command: ["datanode"]
    depends_on: { namenode: { condition: service_healthy } }
    environment: { HDFS_REPLICATION: "2", DATANODE_HOSTNAME: datanode1, HDFS_DATANODE_OPTS: "-Xmx192m" }
    ports: ["127.0.0.1:19864:9864"]
    volumes:
      - ./infrastructure/hadoop:/templates:ro
      - light-datanode1-data:/data

  datanode2:
    <<: *hadoop
    hostname: datanode2
    command: ["datanode"]
    depends_on: { namenode: { condition: service_healthy } }
    environment: { HDFS_REPLICATION: "2", DATANODE_HOSTNAME: datanode2, HDFS_DATANODE_OPTS: "-Xmx192m" }
    ports: ["127.0.0.1:19865:9864"]
    volumes:
      - ./infrastructure/hadoop:/templates:ro
      - light-datanode2-data:/data

volumes:
  light-namenode-data:
  light-datanode1-data:
  light-datanode2-data:
```

.env.example 23 lines

```
# Copy to backend/.env to override defaults. Every value below shows its default.

# --- HDFS connection ---
HDFSOPT_NAMENODE_URL=http://127.0.0.1:9870
HDFSOPT_HDFS_USER=hadoop
# Maps the DataNode names in WebHDFS redirects to published ports. Leave empty inside the Docker network.
HDFSOPT_DATANODE_HOST_MAP=datanode1=127.0.0.1:19864,datanode2=127.0.0.1:19865,datanode3=127.0.0.1:19866
HDFSOPT_ROOT_PATH=/hdfs-optimizer

# --- API ---
```

```
HDFSOPT_CORS_ORIGINS=http://localhost:3100,http://127.0.0.1:3100
# When set, requests that change state must send this value as X-API-Key. No credential ships with the project.
# HDFSOPT_API_TOKEN=
HDFSOPT_MAX_UPLOAD_BYTES=2147483648

# --- Defaults the Settings page can change at runtime ---
HDFSOPT_DEFAULT_INGEST_MODE=file
HDFSOPT_DEFAULT_CHUNK_SIZE=1048576
HDFSOPT_VERIFY_ON_WRITE=true

# --- Docker Compose ---
# HDFS_REPLICATION=3
```

infrastructure/docker/hadoop/Dockerfile 44 lines

```
# Genuine Apache Hadoop built from the official Apache release tarball.
# The published apache/hadoop images are amd64-only, so this image picks the tarball
# matching the build architecture and verifies the published SHA-512 before use.
FROM eclipse-temurin:17-jre-jammy

ARG HADOOP_VERSION=3.5.0
ENV HADOOP_HOME=/opt/hadoop \
    HADOOP_CONF_DIR=/opt/hadoop/etc/hadoop \
    PATH=/opt/hadoop/bin:/opt/hadoop/sbin:$PATH

RUN apt-get update -qq && apt-get install -y -qq --no-install-recommends curl procps >/dev/null \
 && rm -rf /var/lib/apt/lists/*

RUN set -eu; \
    case "$(uname -m)" in \
      aarch64|arm64) TARBALL="hadoop-${HADOOP_VERSION}-aarch64.tar.gz" ;; \
      x86_64)        TARBALL="hadoop-${HADOOP_VERSION}.tar.gz" ;; \
      *) echo "unsupported architecture $(uname -m)"; exit 1 ;; \
    esac; \
    BASE="https://downloads.apache.org/hadoop/common/hadoop-${HADOOP_VERSION}"; \
    curl -fsSL --retry 3 -o "/tmp/${TARBALL}" "${BASE}/${TARBALL}"; \
    curl -fsSL --retry 3 -o "/tmp/${TARBALL}.sha512" "${BASE}/${TARBALL}.sha512"; \
    EXPECTED="$(grep -oE '[0-9a-fA-F]{128}' "/tmp/${TARBALL}.sha512" | head -1 | tr 'A-F' 'a-f')"; \
    ACTUAL="$(sha512sum "/tmp/${TARBALL}" | cut -d' ' -f1)"; \
    [ "$EXPECTED" = "$ACTUAL" ] || { echo "SHA-512 mismatch for ${TARBALL}"; exit 1; }; \
    mkdir -p /opt && tar -xzf "/tmp/${TARBALL}" -C /opt; \
    mv "/opt/hadoop-${HADOOP_VERSION}" /opt/hadoop; \
    rm -f "/tmp/${TARBALL}" "/tmp/${TARBALL}.sha512"; \
    rm -rf /opt/hadoop/share/doc /opt/hadoop/share/hadoop/yarn /opt/hadoop/share/hadoop/mapreduce \
           /opt/hadoop/share/hadoop/tools

# YARN and MapReduce jars are removed to save space (only HDFS daemons run here), but
# Hadoop's launcher scripts still require these directories to exist.
RUN groupadd -r hadoop && useradd -r -g hadoop -d /opt/hadoop hadoop \
 && mkdir -p /opt/hadoop/share/hadoop/yarn /opt/hadoop/share/hadoop/mapreduce /opt/hadoop/share/hadoop/tools \
 && mkdir -p /data /opt/hadoop/logs && chown -R hadoop:hadoop /data /opt/hadoop

COPY entrypoint.sh /usr/local/bin/hadoop-entrypoint.sh
RUN chmod +x /usr/local/bin/hadoop-entrypoint.sh

USER hadoop
WORKDIR /opt/hadoop
ENTRYPOINT ["/usr/local/bin/hadoop-entrypoint.sh"]
```

infrastructure/docker/hadoop/entrypoint.sh 29 lines

```
#!/usr/bin/env bash
# Renders Hadoop config from mounted templates and starts one HDFS daemon in the foreground.
set -euo pipefail
ROLE="${1:?usage: hadoop-entrypoint.sh namenode|datanode}"
: "${HDFS_REPLICATION:=3}"
: "${DATANODE_HOSTNAME:=$(hostname)}"

cp /templates/core-site.xml "$HADOOP_CONF_DIR/core-site.xml"
sed -e "s|@REPLICATION@|${HDFS_REPLICATION}|g" -e "s|@DATANODE_HOSTNAME@|${DATANODE_HOSTNAME}|g" \
    /templates/hdfs-site.xml > "$HADOOP_CONF_DIR/hdfs-site.xml"
: > "$HADOOP_CONF_DIR/workers"

case "$ROLE" in
  namenode)
    mkdir -p /data/nn
    # Format only an empty name directory, so a restart can never wipe existing metadata.
    if [ ! -f /data/nn/current/VERSION ]; then
      echo "[entrypoint] formatting empty NameNode directory"
      hdfs namenode -format -nonInteractive -force -clusterId "${HDFS_CLUSTER_ID:-hdfs-optimizer}"
    fi
    exec hdfs namenode
    ;;
  datanode)
    mkdir -p /data/dn
    exec hdfs datanode
    ;;
  *) exec "$@" ;;
esac
```

infrastructure/hadoop/core-site.xml 8 lines

```
<?xml version="1.0"?>
<configuration>
  <property><name>fs.defaultFS</name><value>hdfs://namenode:8020</value></property>
  <property><name>hadoop.http.staticuser.user</name><value>hadoop</value></property>
```

```
  <!-- HTrash is not used: deletion and recovery are handled by the application's recovery namespace. -->
  <property><name>fs.trash.interval</name><value>0</value></property>
</configuration>
```

infrastructure/hadoop/hdfs-site.xml 25 lines

```
<?xml version="1.0"?>
<configuration>
  <property><name>dfs.replication</name><value>@REPLICATION@</value></property>
  <property><name>dfs.namenode.name.dir</name><value>file:///data/nn</value></property>
  <property><name>dfs.datanode.data.dir</name><value>file:///data/dn</value></property>
  <property><name>dfs.webhdfs.enabled</name><value>true</value></property>

  <!-- Container networking -->
  <property><name>dfs.namenode.rpc-bind-host</name><value>0.0.0.0</value></property>
  <property><name>dfs.namenode.http-bind-host</name><value>0.0.0.0</value></property>
  <property><name>dfs.namenode.http-address</name><value>namenode:9870</value></property>
  <property><name>dfs.datanode.hostname</name><value>@DATANODE_HOSTNAME@</value></property>
  <property><name>dfs.datanode.http.address</name><value>0.0.0.0:9864</value></property>
  <property><name>dfs.client.use.datanode.hostname</name><value>true</value></property>
  <property><name>dfs.datanode.use.datanode.hostname</name><value>true</value></property>
  <property><name>dfs.namenode.datanode.registration.ip-hostname-check</name><value>false</value></property>

  <!-- Demonstration tuning: a stopped DataNode is declared dead after about 90 s rather than the
       default 10.5 minutes, so re-replication can be observed during a live demonstration. -->
  <property><name>dfs.heartbeat.interval</name><value>3</value></property>
  <property><name>dfs.namenode.heartbeat.recheck-interval</name><value>30000</value></property>
  <property><name>dfs.namenode.safemode.extension</name><value>5000</value></property>
  <property><name>dfs.client.block.write.replace-datanode-on-failure.best-effort</name><value>true</value></property>
</configuration>
```

backend/Dockerfile 26 lines

```
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /srv/backend

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY alembic.ini .
COPY migrations ./migrations
COPY app ./app

RUN useradd --create-home --uid 1000 app && mkdir -p /data /datasets && chown -R app:app /data /datasets
USER app

ENV HDFSOPT_DATABASE_PATH=/data/metadata.db \
    HDFSOPT_SPOOL_DIR=/data/spool \
    HDFSOPT_DATASET_DIR=/datasets/raw \
    HDFSOPT_PROVENANCE_PATH=/datasets/provenance/manifest.json \
    HDFSOPT_DATANODE_HOST_MAP=

EXPOSE 8100
HEALTHCHECK --interval=15s --timeout=5s --start-period=20s \
  CMD python -c "import urllib.request as u; u.urlopen('http://127.0.0.1:8100/api/v1/health', timeout=4)"
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8100"]
```

backend/.dockerignore 8 lines

```
.venv
data
tests
__pycache__
*.pyc
.pytest_cache
.ruff_cache
```

frontend/Dockerfile 20 lines

```
FROM node:22-slim AS build
WORKDIR /app
ENV NEXT_TELEMETRY_DISABLED=1 NEXT_STANDALONE=1
ARG NEXT_PUBLIC_API_URL=http://localhost:8100
ENV NEXT_PUBLIC_API_URL=$NEXT_PUBLIC_API_URL
COPY package.json package-lock.json ./
RUN npm ci --no-audit --no-fund
COPY . .
RUN npm run build

# Runtime image: only the standalone server and its static files.
FROM node:22-slim
WORKDIR /app
ENV NODE_ENV=production NEXT_TELEMETRY_DISABLED=1 PORT=3100 HOSTNAME=0.0.0.0
COPY --from=build /app/.next/standalone ./
COPY --from=build /app/.next/static ./.next/static
USER node
EXPOSE 3100
CMD ["node", "server.js"]
```

frontend/.dockerignore 8 lines

```
node_modules
.next
test-results
playwright-report
e2e
```

```
capture
.env.local
```

11.2 Backend: core, models and HDFS client

backend/requirements-dev.txt 5 lines

```
-r requirements.txt
pytest==8.4.2
pytest-cov==7.0.0
ruff==0.14.5
```

backend/requirements.txt 9 lines

```
fastapi==0.121.2
uvicorn==0.38.0
pydantic==2.12.4
pydantic-settings==2.12.0
sqlalchemy==2.0.44
alembic==1.17.2
httpx==0.28.1
python-multipart==0.0.20
```

backend/alembic.ini 28 lines

```
[alembic]
script_location = migrations
prepend_sys_path = .
path_separator = os
sqlalchemy.url = sqlite:///data/metadata.db

[loggers]
keys = root

[handlers]
keys = console

[formatters]
keys = generic

[logger_root]
level = WARNING
handlers = console

[handler_console]
class = StreamHandler
args = (sys.stderr,)
level = NOTSET
formatter = generic

[formatter_generic]
format = %(levelname)-5.5s [%(name)s] %(message)s
```

backend/pytest.ini 10 lines

```
[pytest]
testpaths = tests
pythonpath = .
markers =
    hdfs: integration test that requires the real HDFS cluster
addopts = --strict-markers
filterwarnings =
    error::sqlalchemy.exc.SAWarning
    ignore:The anyio.abc.BlockingPortal alias:DeprecationWarning
```

backend/app/main.py 109 lines

```
"""Application entry point."""

from __future__ import annotations

import logging
from collections.abc import AsyncIterator
from contextlib import asynccontextmanager
from pathlib import Path

from alembic import command
from alembic.config import Config
from fastapi import Depends, FastAPI
from fastapi.middleware.cors import CORSMiddleware

from app.api import dedup, files, system
from app.api.deps import require_token
from app.core.config import BACKEND_DIR, Settings, get_settings
from app.core.db import make_engine, make_session_factory
from app.core.errors import install_error_handlers
from app.hdfs.webhdfs import WebHdfsClient
from app.services import (
    datasets,
    experiments,
    integrity,
    lifecycle,
    optimizer,
    replication,
)
from app.services.context import AppContext, StoragePaths
from app.services.jobs import JobManager
```

```
from app.services.monitor import ClusterMonitor

log = logging.getLogger("hdfs_optimizer")

JOB_HANDLERS = {
    "analyze": optimizer.run_analysis,
    "optimize": optimizer.run_optimization,
    "replication": replication.run_replication_change,
    "verify_all": integrity.run_verify_all,
    "dataset_import": datasets.run_import,
    "dataset_ingest": datasets.run_ingest,
    "experiment": experiments.run_scenario,
}


def run_migrations(database_path: Path) -> None:
    database_path.parent.mkdir(parents=True, exist_ok=True)
    config = Config(str(BACKEND_DIR / "alembic.ini"))
    config.set_main_option("script_location", str(BACKEND_DIR / "migrations"))
    config.set_main_option("sqlalchemy.url", f"sqlite:///{database_path}")
    command.upgrade(config, "head")


def build_context(settings: Settings) -> AppContext:
    run_migrations(Path(settings.database_path))
    engine = make_engine(settings.database_path)
    hdfs = WebHdfsClient(
        settings.namenode_url,
        user=settings.hdfs_user,
        datanode_map=settings.datanode_map,
        timeout_seconds=settings.http_timeout_seconds,
        connect_timeout_seconds=settings.connect_timeout_seconds,
    )
    ctx = AppContext(
        settings=settings,
        session_factory=make_session_factory(engine),
        hdfs=hdfs,
        paths=StoragePaths(root=settings.root_path),
        monitor=ClusterMonitor(settings.namenode_url, user=settings.hdfs_user),
    )
    ctx.jobs = JobManager(ctx, JOB_HANDLERS)
    return ctx


def create_app(context: AppContext | None = None) -> FastAPI:
    @asynccontextmanager
    async def lifespan(app: FastAPI) -> AsyncIterator[None]:
        ctx = context or build_context(get_settings())
        app.state.ctx = ctx
        summary = lifecycle.recover_on_startup(ctx)
        log.info("startup recovery: %s", summary)
        yield
        if ctx.jobs:
            ctx.jobs.shutdown()

    app = FastAPI(
        title="HDFS Optimizer API",
        version="1.0.0",
        description="Application-managed deduplication and storage optimization on Apache Hadoop HDFS.",
        lifespan=lifespan,
        dependencies=[Depends(require_token)],
    )
    settings = context.settings if context else get_settings()
    app.add_middleware(
        CORSMiddleware,
        allow_origins=settings.cors_origin_list,
        allow_methods=["GET", "POST", "PUT", "DELETE"],
        allow_headers=["Content-Type", "X-API-Key"],
        expose_headers=["X-Content-SHA256", "Content-Disposition"],
    )
    install_error_handlers(app)
    app.include_router(files.router)
    app.include_router(dedup.router)
    app.include_router(system.router)
    return app


app = create_app()
```

backend/app/core/config.py 73 lines

```
"""Environment-based configuration. All values can be overridden with HDFSOPT_* variables."""

from __future__ import annotations

from functools import lru_cache
from pathlib import Path

from pydantic_settings import BaseSettings, SettingsConfigDict

BACKEND_DIR = Path(__file__).resolve().parents[2]
PROJECT_DIR = BACKEND_DIR.parent

KIB = 1024
```

```
MIB = 1024 * 1024


class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_prefix="HDFSOPT_", env_file=".env", extra="ignore")

    # HDFS connection
    namenode_url: str = "http://127.0.0.1:9870"
    hdfs_user: str = "hadoop"
    # WebHDFS redirects name DataNodes by their container hostname. A backend running outside the
    # Docker network maps those names to published ports. Empty inside the Compose network.
    datanode_host_map: str = (
        "datanode1=127.0.0.1:19864,datanode2=127.0.0.1:19865,datanode3=127.0.0.1:19866"
    )
    root_path: str = "/hdfs-optimizer"
    http_timeout_seconds: float = 60.0
    connect_timeout_seconds: float = 5.0

    # Local state
    database_path: Path = BACKEND_DIR / "data" / "metadata.db"
    dataset_dir: Path = PROJECT_DIR / "datasets" / "raw"
    provenance_path: Path = PROJECT_DIR / "datasets" / "provenance" / "manifest.json"
    spool_dir: Path = BACKEND_DIR / "data" / "spool"

    # API
    cors_origins: str = "http://localhost:3100,http://127.0.0.1:3100"
    api_token: str | None = None  # when set, every mutating request must send X-API-Key
    max_upload_bytes: int = 2 * 1024 * MIB

    # Defaults that the settings page can change at runtime (see services/settings_store.py)
    default_ingest_mode: str = "file"
    default_chunk_size: int = 1 * MIB
    verify_on_write: bool = True

    # Limits
    min_chunk_size: int = 4 * KIB
    max_chunk_size: int = 64 * MIB

    @property
    def cors_origin_list(self) -> list[str]:
        return [origin.strip() for origin in self.cors_origins.split(",") if origin.strip()]

    @property
    def datanode_map(self) -> dict[str, str]:
        mapping: dict[str, str] = {}
        for pair in self.datanode_host_map.split(","):
            if "=" in pair:
                name, target = pair.split("=", 1)
                mapping[name.strip()] = target.strip()
        return mapping

    @property
    def database_url(self) -> str:
        return f"sqlite:///{self.database_path}"


@lru_cache
def get_settings() -> Settings:
    return Settings()
```

backend/app/core/db.py 60 lines

```
"""SQLite engine setup and session helpers."""

from __future__ import annotations

from collections.abc import Iterator
from contextlib import contextmanager
from datetime import UTC, datetime
from pathlib import Path

from sqlalchemy import create_engine, event
from sqlalchemy.engine import Engine
from sqlalchemy.orm import DeclarativeBase, Session, sessionmaker


class Base(DeclarativeBase):
    pass


def utcnow() -> datetime:
    """Naive UTC timestamp, the form SQLite stores."""
    return datetime.now(UTC).replace(tzinfo=None)


def make_engine(database_path: Path | str) -> Engine:
    if str(database_path) != ":memory:":
        Path(database_path).parent.mkdir(parents=True, exist_ok=True)
    engine = create_engine(
        f"sqlite:///{database_path}",
        connect_args={"check_same_thread": False, "timeout": 30},
    )

    @event.listens_for(engine, "connect")
```

```
    def _configure(connection, _record):  # noqa: ANN001
        cursor = connection.cursor()
        cursor.execute("PRAGMA foreign_keys=ON")
        cursor.execute("PRAGMA journal_mode=WAL")
        cursor.execute("PRAGMA synchronous=FULL")
        cursor.execute("PRAGMA busy_timeout=30000")
        cursor.close()

    return engine


def make_session_factory(engine: Engine) -> sessionmaker[Session]:
    return sessionmaker(bind=engine, expire_on_commit=False, autoflush=True)


@contextmanager
def session_scope(factory: sessionmaker[Session]) -> Iterator[Session]:
    """Commit on success, roll back on any error."""
    session = factory()
    try:
        yield session
        session.commit()
    except Exception:
        session.rollback()
        raise
    finally:
        session.close()
```

backend/app/core/errors.py 98 lines

```
"""Application errors and their mapping onto HTTP responses."""

from __future__ import annotations

import logging

from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse


class AppError(Exception):
    status_code = 400
    code = "bad_request"

    def __init__(self, message: str, *, details: dict | None = None):
        super().__init__(message)
        self.message = message
        self.details = details or {}


class NotFoundError(AppError):
    status_code = 404
    code = "not_found"


class ConflictError(AppError):
    status_code = 409
    code = "conflict"


class ValidationFailure(AppError):
    status_code = 422
    code = "validation_failed"


class PayloadTooLarge(AppError):
    status_code = 413
    code = "payload_too_large"


class UnauthorizedError(AppError):
    status_code = 401
    code = "unauthorized"


class IntegrityFailure(AppError):
    """Stored bytes do not match the recorded cryptographic hash, or are missing."""

    status_code = 500
    code = "integrity_failure"


class HdfsError(AppError):
    status_code = 502
    code = "hdfs_error"


class HdfsUnavailable(HdfsError):
    status_code = 503
    code = "hdfs_unavailable"


class HdfsFileNotFound(HdfsError):
    status_code = 404
```

```
    code = "hdfs_file_not_found"


class HdfsAlreadyExists(HdfsError):
    status_code = 409
    code = "hdfs_already_exists"


def install_error_handlers(app: FastAPI) -> None:
    @app.exception_handler(AppError)
    async def handle_app_error(_: Request, exc: AppError) -> JSONResponse:
        return JSONResponse(
            status_code=exc.status_code,
            content={"error": {"code": exc.code, "message": exc.message, "details": exc.details}},
        )

    @app.exception_handler(HdfsError)
    async def handle_hdfs_error(request: Request, exc: HdfsError) -> JSONResponse:
        # A storage failure on a request that changes state belongs in the audit trail, in its own
        # transaction, because the operation itself has been rolled back.
        if request.method not in {"GET", "HEAD", "OPTIONS"}:
            from app.services.audit import record_failure

            try:
                record_failure(
                    request.app.state.ctx.session_factory,
                    "request.failed",
                    f"{request.method} {request.url.path} failed: {exc.message}",
                    details={"code": exc.code, "status": exc.status_code},
                )
            except Exception:  # noqa: BLE001 - failing to log must not hide the original error from the client
                logging.getLogger("hdfs_optimizer").exception("could not record failed request")
        return await handle_app_error(request, exc)
```

backend/app/models/entities.py 355 lines

```
"""Relational metadata schema.

Logical files are kept separate from the content they point at. Whether a piece of content is still
needed is always derived from these relations with a query, never from a stored reference counter
that could drift from reality.
"""

from __future__ import annotations

import uuid
from datetime import datetime
from enum import StrEnum

from sqlalchemy import (
    JSON,
    BigInteger,
    Boolean,
    CheckConstraint,
    DateTime,
    Float,
    ForeignKey,
    Index,
    Integer,
    String,
    Text,
    UniqueConstraint,
    text,
)
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.core.db import Base, utcnow


def new_id() -> str:
    return uuid.uuid4().hex


class StorageMode(StrEnum):
    RAW = "raw"  # stored as a private copy, with no deduplication (the baseline, and the optimizer's input)
    FILE = "file"  # points at one whole-file content object
    CHUNK = "chunk"  # points at a manifest of ordered shared chunks


class FileStatus(StrEnum):
    ACTIVE = "active"
    DELETED = "deleted"  # recoverable, and still holds its content
    PURGED = "purged"  # no longer holds content; the row remains for audit


class ContentState(StrEnum):
    ACTIVE = "active"
    GC_PENDING = "gc_pending"  # selected for removal; must not gain new references
    REMOVED = "removed"


class JobStatus(StrEnum):
    QUEUED = "queued"
    RUNNING = "running"
```

```
    SUCCEEDED = "succeeded"
    PARTIAL = "partial"  # finished, but some items failed
    FAILED = "failed"
    INTERRUPTED = "interrupted"  # the process stopped mid-run; safe to resume


class ContentObject(Base):
    """One unique whole-file blob stored at a content-addressed HDFS path."""

    __tablename__ = "objects"

    id: Mapped[str] = mapped_column(String(32), primary_key=True, default=new_id)
    sha256: Mapped[str] = mapped_column(String(64), unique=True, index=True)
    size_bytes: Mapped[int] = mapped_column(BigInteger)
    hdfs_path: Mapped[str] = mapped_column(String(512))
    state: Mapped[str] = mapped_column(String(16), default=ContentState.ACTIVE, index=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow)
    removed_at: Mapped[datetime | None] = mapped_column(DateTime)


class Chunk(Base):
    """One unique application-level chunk. This is not an HDFS block: each chunk is its own HDFS file."""

    __tablename__ = "chunks"

    id: Mapped[str] = mapped_column(String(32), primary_key=True, default=new_id)
    sha256: Mapped[str] = mapped_column(String(64), unique=True, index=True)
    size_bytes: Mapped[int] = mapped_column(BigInteger)
    hdfs_path: Mapped[str] = mapped_column(String(512))
    state: Mapped[str] = mapped_column(String(16), default=ContentState.ACTIVE, index=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow)
    removed_at: Mapped[datetime | None] = mapped_column(DateTime)


class FileManifest(Base):
    """An ordered recipe that rebuilds one distinct content value at one chunk size."""

    __tablename__ = "file_manifests"
    __table_args__ = (UniqueConstraint("file_sha256", "chunk_size", name="uq_manifest_content"),)

    id: Mapped[str] = mapped_column(String(32), primary_key=True, default=new_id)
    file_sha256: Mapped[str] = mapped_column(String(64), index=True)
    file_size: Mapped[int] = mapped_column(BigInteger)
    chunk_size: Mapped[int] = mapped_column(Integer)
    chunk_count: Mapped[int] = mapped_column(Integer)
    manifest_bytes: Mapped[int] = mapped_column(Integer, default=0)
    hdfs_path: Mapped[str] = mapped_column(String(512))
    state: Mapped[str] = mapped_column(String(16), default=ContentState.ACTIVE, index=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow)
    removed_at: Mapped[datetime | None] = mapped_column(DateTime)

    entries: Mapped[list[FileChunk]] = relationship(
        back_populates="manifest",
        order_by="FileChunk.ordinal",
        cascade="all, delete-orphan",
    )


class FileChunk(Base):
    """One position in a manifest. These rows are the authoritative references to chunks."""

    __tablename__ = "file_chunks"
    __table_args__ = (
        UniqueConstraint("manifest_id", "ordinal", name="uq_manifest_ordinal"),
        CheckConstraint("ordinal >= 0", name="ck_ordinal_nonnegative"),
    )

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    manifest_id: Mapped[str] = mapped_column(ForeignKey("file_manifests.id", ondelete="CASCADE"), index=True)
    chunk_id: Mapped[str] = mapped_column(ForeignKey("chunks.id", ondelete="RESTRICT"), index=True)
    ordinal: Mapped[int] = mapped_column(Integer)
    offset_bytes: Mapped[int] = mapped_column(BigInteger)

    manifest: Mapped[FileManifest] = relationship(back_populates="entries")
    chunk: Mapped[Chunk] = relationship()


class DatasetSource(Base):
    """Provenance for an authentic downloaded file, or for a workload file derived from one."""

    __tablename__ = "dataset_sources"

    id: Mapped[str] = mapped_column(String(32), primary_key=True, default=new_id)
    title: Mapped[str] = mapped_column(String(255))
    author: Mapped[str | None] = mapped_column(String(255))
    ebook_id: Mapped[int | None] = mapped_column(Integer)
    source_url: Mapped[str | None] = mapped_column(String(512))
    download_url: Mapped[str | None] = mapped_column(String(512))
    organization: Mapped[str | None] = mapped_column(String(255))
    downloaded_at: Mapped[datetime | None] = mapped_column(DateTime)
    file_format: Mapped[str] = mapped_column(String(32))
    size_bytes: Mapped[int] = mapped_column(BigInteger)
    sha256: Mapped[str] = mapped_column(String(64), index=True)
```

```
    kind: Mapped[str] = mapped_column(String(24))  # original | workload_copy | transformed
    derived_from_id: Mapped[str | None] = mapped_column(ForeignKey("dataset_sources.id", ondelete="SET NULL"))
    transformation: Mapped[str | None] = mapped_column(Text)
    local_path: Mapped[str] = mapped_column(String(512))
    license_note: Mapped[str | None] = mapped_column(Text)

    __table_args__ = (
        CheckConstraint("kind IN ('original','workload_copy','transformed')", name="ck_dataset_kind"),
        CheckConstraint(
            "kind = 'original' OR derived_from_id IS NOT NULL OR transformation IS NOT NULL",
            name="ck_dataset_derived_is_documented",
        ),
    )


class LogicalFile(Base):
    """What a user sees as a file: a name in a folder that resolves to stored content."""

    __tablename__ = "files"
    __table_args__ = (
        CheckConstraint(
            "status = 'purged' OR "
            "(storage_mode = 'raw' AND raw_path IS NOT NULL AND object_id IS NULL AND manifest_id IS NULL) OR "
            "(storage_mode = 'file' AND raw_path IS NULL AND object_id IS NOT NULL AND manifest_id IS NULL) OR "
            "(storage_mode = 'chunk' AND raw_path IS NULL AND object_id IS NULL AND manifest_id IS NOT NULL)",
            name="ck_file_storage_pointer",
        ),
        CheckConstraint("size_bytes >= 0", name="ck_file_size"),
        Index(
            "uq_active_file_name",
            "folder",
            "name",
            unique=True,
            sqlite_where=text("status = 'active'"),
        ),
        Index("ix_files_status_created", "status", "created_at"),
    )

    id: Mapped[str] = mapped_column(String(32), primary_key=True, default=new_id)
    name: Mapped[str] = mapped_column(String(255))
    folder: Mapped[str] = mapped_column(String(512), default="/", index=True)
    size_bytes: Mapped[int] = mapped_column(BigInteger)
    sha256: Mapped[str] = mapped_column(String(64), index=True)
    mime_type: Mapped[str | None] = mapped_column(String(128))
    extension: Mapped[str | None] = mapped_column(String(32))
    storage_mode: Mapped[str] = mapped_column(String(8))
    status: Mapped[str] = mapped_column(String(8), default=FileStatus.ACTIVE, index=True)

    raw_path: Mapped[str | None] = mapped_column(String(512))
    object_id: Mapped[str | None] = mapped_column(ForeignKey("objects.id", ondelete="RESTRICT"), index=True)
    manifest_id: Mapped[str | None] = mapped_column(ForeignKey("file_manifests.id", ondelete="RESTRICT"), index=True)

    provenance_label: Mapped[str] = mapped_column(String(24), default="user_upload")
    dataset_source_id: Mapped[str | None] = mapped_column(ForeignKey("dataset_sources.id", ondelete="SET NULL"))

    created_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow)
    updated_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow, onupdate=utcnow)
    deleted_at: Mapped[datetime | None] = mapped_column(DateTime)
    purged_at: Mapped[datetime | None] = mapped_column(DateTime)
    last_verified_at: Mapped[datetime | None] = mapped_column(DateTime)
    last_verified_ok: Mapped[bool | None] = mapped_column(Boolean)

    content_object: Mapped[ContentObject | None] = relationship()
    manifest: Mapped[FileManifest | None] = relationship()
    dataset_source: Mapped[DatasetSource | None] = relationship()


class DeduplicationJob(Base):
    """A tracked background operation."""

    __tablename__ = "deduplication_jobs"

    id: Mapped[str] = mapped_column(String(32), primary_key=True, default=new_id)
    kind: Mapped[str] = mapped_column(String(24), index=True)  # analyze | optimize | gc | replication | experiment | ...
    mode: Mapped[str | None] = mapped_column(String(8))
    chunk_size: Mapped[int | None] = mapped_column(Integer)
    status: Mapped[str] = mapped_column(String(16), default=JobStatus.QUEUED, index=True)
    progress_done: Mapped[int] = mapped_column(Integer, default=0)
    progress_total: Mapped[int] = mapped_column(Integer, default=0)
    message: Mapped[str | None] = mapped_column(Text)
    params: Mapped[dict | None] = mapped_column(JSON)
    result: Mapped[dict | None] = mapped_column(JSON)
    error: Mapped[str | None] = mapped_column(Text)
    resumed_from_id: Mapped[str | None] = mapped_column(String(32))
    created_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow, index=True)
    started_at: Mapped[datetime | None] = mapped_column(DateTime)
    finished_at: Mapped[datetime | None] = mapped_column(DateTime)
    heartbeat_at: Mapped[datetime | None] = mapped_column(DateTime)


class OptimizationHistory(Base):
    """Verified before-and-after record of a completed optimization job."""
```

```
    __tablename__ = "optimization_history"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    job_id: Mapped[str] = mapped_column(ForeignKey("deduplication_jobs.id", ondelete="CASCADE"), index=True)
    mode: Mapped[str] = mapped_column(String(8))
    chunk_size: Mapped[int | None] = mapped_column(Integer)
    before: Mapped[dict] = mapped_column(JSON)
    after: Mapped[dict] = mapped_column(JSON)
    files_processed: Mapped[int] = mapped_column(Integer, default=0)
    files_failed: Mapped[int] = mapped_column(Integer, default=0)
    bytes_reclaimed_logical: Mapped[int] = mapped_column(BigInteger, default=0)
    duration_ms: Mapped[float] = mapped_column(Float, default=0)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow, index=True)


class StorageSnapshot(Base):
    """A point-in-time metadata measurement, recorded after each operation that changes storage."""

    __tablename__ = "storage_snapshots"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    taken_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow, index=True)
    reason: Mapped[str] = mapped_column(String(48))
    logical_files: Mapped[int] = mapped_column(Integer)
    logical_bytes: Mapped[int] = mapped_column(BigInteger)
    stored_bytes: Mapped[int] = mapped_column(BigInteger)
    unique_files: Mapped[int] = mapped_column(Integer)


class PerformanceSample(Base):
    """One measured duration of a real operation."""

    __tablename__ = "performance_samples"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    operation: Mapped[str] = mapped_column(String(32), index=True)  # upload | hash | chunking | restore | verify ...
    mode: Mapped[str | None] = mapped_column(String(8))
    bytes_processed: Mapped[int] = mapped_column(BigInteger)
    duration_ms: Mapped[float] = mapped_column(Float)
    file_id: Mapped[str | None] = mapped_column(String(32))
    created_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow, index=True)


class ReplicationOperation(Base):
    __tablename__ = "replication_operations"

    id: Mapped[str] = mapped_column(String(32), primary_key=True, default=new_id)
    previous_factor: Mapped[int | None] = mapped_column(Integer)
    requested_factor: Mapped[int] = mapped_column(Integer)
    status: Mapped[str] = mapped_column(String(16), default="pending")  # pending | applied | verified | timed_out | failed
    files_total: Mapped[int] = mapped_column(Integer, default=0)
    files_changed: Mapped[int] = mapped_column(Integer, default=0)
    space_consumed_before: Mapped[int | None] = mapped_column(BigInteger)
    space_consumed_after: Mapped[int | None] = mapped_column(BigInteger)
    details: Mapped[dict | None] = mapped_column(JSON)
    error: Mapped[str | None] = mapped_column(Text)
    requested_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow, index=True)
    completed_at: Mapped[datetime | None] = mapped_column(DateTime)


class IntegrityCheck(Base):
    __tablename__ = "integrity_checks"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    file_id: Mapped[str] = mapped_column(ForeignKey("files.id", ondelete="CASCADE"), index=True)
    ok: Mapped[bool] = mapped_column(Boolean)
    expected_sha256: Mapped[str] = mapped_column(String(64))
    actual_sha256: Mapped[str | None] = mapped_column(String(64))
    bytes_read: Mapped[int] = mapped_column(BigInteger, default=0)
    chunks_checked: Mapped[int] = mapped_column(Integer, default=0)
    missing: Mapped[list | None] = mapped_column(JSON)
    corrupt: Mapped[list | None] = mapped_column(JSON)
    detail: Mapped[str | None] = mapped_column(Text)
    duration_ms: Mapped[float] = mapped_column(Float, default=0)
    checked_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow, index=True)


class AuditEvent(Base):
    __tablename__ = "audit_events"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    occurred_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow, index=True)
    event_type: Mapped[str] = mapped_column(String(48), index=True)
    severity: Mapped[str] = mapped_column(String(8), default="info", index=True)  # info | warning | error
    entity_type: Mapped[str | None] = mapped_column(String(24))
    entity_id: Mapped[str | None] = mapped_column(String(64))
    message: Mapped[str] = mapped_column(Text)
    details: Mapped[dict | None] = mapped_column(JSON)


class ExperimentRun(Base):
    __tablename__ = "experiment_runs"
```

```
    id: Mapped[str] = mapped_column(String(32), primary_key=True, default=new_id)
    scenario: Mapped[str] = mapped_column(String(48), index=True)
    params: Mapped[dict | None] = mapped_column(JSON)
    status: Mapped[str] = mapped_column(String(16), default="running")
    result: Mapped[dict | None] = mapped_column(JSON)
    error: Mapped[str | None] = mapped_column(Text)
    job_id: Mapped[str | None] = mapped_column(String(32))
    started_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow, index=True)
    finished_at: Mapped[datetime | None] = mapped_column(DateTime)


class AppSetting(Base):
    __tablename__ = "app_settings"

    key: Mapped[str] = mapped_column(String(64), primary_key=True)
    value: Mapped[dict | int | str | bool | None] = mapped_column(JSON)
    updated_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow, onupdate=utcnow)
```

backend/app/schemas/api.py 168 lines

```
"""Request and response schemas for the REST API."""

from __future__ import annotations

from datetime import datetime
from typing import Any

from pydantic import BaseModel, ConfigDict, Field


class FileOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: str
    name: str
    folder: str
    size_bytes: int
    sha256: str
    mime_type: str | None
    extension: str | None
    storage_mode: str
    status: str
    provenance_label: str
    created_at: datetime
    deleted_at: datetime | None
    last_verified_at: datetime | None
    last_verified_ok: bool | None
    logical_references: int = 0
    is_duplicate: bool = False


class FilePage(BaseModel):
    items: list[FileOut]
    total: int
    page: int
    page_size: int


class ChunkEntry(BaseModel):
    ordinal: int
    offset_bytes: int
    sha256: str
    size_bytes: int
    shared_by_manifests: int


class IntegrityOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    file_id: str
    ok: bool
    expected_sha256: str
    actual_sha256: str | None
    bytes_read: int
    chunks_checked: int
    missing: list[str] | None
    corrupt: list[str] | None
    detail: str | None
    duration_ms: float
    checked_at: datetime


class FileDetail(FileOut):
    hdfs_paths: list[str]
    same_content_files: list[dict[str, Any]]
    chunk_size: int | None = None
    chunk_count: int | None = None
    chunks: list[ChunkEntry] = []
    integrity_history: list[IntegrityOut] = []
    block_locations: list[dict[str, Any]] | None = None
    dataset_source: dict[str, Any] | None = None
```

```
class UploadResult(BaseModel):
    file: FileOut
    outcome: str = Field(description="stored_new, duplicate_reference, chunks_shared or stored_raw")
    duplicate_of_existing: bool
    bytes_written: int
    chunks_total: int
    chunks_new: int
    chunks_reused: int
    hash_ms: float
    store_ms: float
    total_ms: float


class AnalyzeRequest(BaseModel):
    mode: str = Field(description="file or chunk")
    chunk_size: int | None = None


class OptimizeRequest(BaseModel):
    analysis_job_id: str = Field(description="A successful analysis of the current catalog")


class JobOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: str
    kind: str
    mode: str | None
    chunk_size: int | None
    status: str
    progress_done: int
    progress_total: int
    message: str | None
    params: dict[str, Any] | None
    result: dict[str, Any] | None
    error: str | None
    resumed_from_id: str | None
    created_at: datetime
    started_at: datetime | None
    finished_at: datetime | None
    heartbeat_at: datetime | None


class JobAccepted(BaseModel):
    job_id: str


class ReplicationRequest(BaseModel):
    factor: int = Field(ge=1, le=16)
    acknowledge_reduced_protection: bool = False


class DatasetImportRequest(BaseModel):
    ebook_ids: list[int] | None = None


class DatasetIngestRequest(BaseModel):
    source_ids: list[str]
    copies: int = Field(default=0, ge=0, le=20, description="Exact workload copies per file, labelled as copies")
    mode: str | None = None
    chunk_size: int | None = None
    folder: str = "/datasets"


class ExperimentRequest(BaseModel):
    scenario: str
    options: dict[str, Any] | None = None


class SettingsUpdate(BaseModel):
    default_ingest_mode: str | None = None
    default_chunk_size: int | None = None
    verify_on_write: bool | None = None


class AuditOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    occurred_at: datetime
    event_type: str
    severity: str
    entity_type: str | None
    entity_id: str | None
    message: str
    details: dict[str, Any] | None


class AuditPage(BaseModel):
    items: list[AuditOut]
    total: int
    page: int
    page_size: int
```

backend/app/hdfs/base.py 87 lines

```
"""The storage interface used by every service.

The application implementation is ``WebHdfsClient``, which talks to a real cluster. The only other
implementation lives in ``tests/support`` and is a clearly labelled local-filesystem stand-in used by
fast unit tests. It is not importable from the application.
"""

from __future__ import annotations

from collections.abc import Iterable, Iterator
from dataclasses import dataclass
from typing import Protocol, runtime_checkable


@dataclass(frozen=True)
class FileStatus:
    path: str
    is_dir: bool
    length: int
    replication: int
    block_size: int
    modification_time: int


@dataclass(frozen=True)
class ContentSummary:
    """Directly measured by the NameNode. ``space_consumed`` includes replication."""

    length: int
    space_consumed: int
    file_count: int
    directory_count: int


@dataclass(frozen=True)
class BlockLocation:
    block_id: int
    length: int
    hosts: tuple[str, ...]
    corrupt: bool


@runtime_checkable
class HdfsClient(Protocol):
    def mkdirs(self, path: str) -> None: ...

    def create(
        self,
        path: str,
        data: bytes | Iterable[bytes],
        *,
        length: int | None = None,
        overwrite: bool = False,
        replication: int | None = None,
    ) -> None: ...

    def open(self, path: str) -> Iterator[bytes]: ...

    def status(self, path: str) -> FileStatus | None: ...

    def list_dir(self, path: str) -> list[FileStatus]: ...

    def rename(self, source: str, destination: str) -> bool: ...

    def delete(self, path: str, *, recursive: bool = False) -> bool: ...

    def set_replication(self, path: str, factor: int) -> bool: ...

    def content_summary(self, path: str) -> ContentSummary: ...

    def block_locations(self, path: str) -> list[BlockLocation]: ...

    def default_replication(self) -> int: ...


def walk_files(client: HdfsClient, root: str) -> Iterator[FileStatus]:
    """Yield every file below ``root``. Missing roots yield nothing."""
    if client.status(root) is None:
        return
    stack = [root]
    while stack:
        for entry in client.list_dir(stack.pop()):
            if entry.is_dir:
                stack.append(entry.path)
            else:
                yield entry
```

backend/app/hdfs/webhdfs.py 200 lines

```
"""Client for the Apache Hadoop WebHDFS REST API.

Reference: https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/WebHDFS.html
"""
```

```
from __future__ import annotations

from collections.abc import Iterable, Iterator
from urllib.parse import quote, urlsplit, urlunsplit

import httpx

from app.core.errors import (
    HdfsAlreadyExists,
    HdfsError,
    HdfsFileNotFound,
    HdfsUnavailable,
)
from app.hdfs.base import BlockLocation, ContentSummary, FileStatus

READ_BUFFER = 1024 * 1024


class WebHdfsClient:
    def __init__(
        self,
        namenode_url: str,
        *,
        user: str = "hadoop",
        datanode_map: dict[str, str] | None = None,
        timeout_seconds: float = 60.0,
        connect_timeout_seconds: float = 5.0,
    ):
        self.base_url = namenode_url.rstrip("/")
        self.user = user
        self.datanode_map = datanode_map or {}
        timeout = httpx.Timeout(timeout_seconds, connect=connect_timeout_seconds)
        limits = httpx.Limits(max_keepalive_connections=8, keepalive_expiry=15)
        self._http = httpx.Client(timeout=timeout, limits=limits, follow_redirects=False)

    def close(self) -> None:
        self._http.close()

    # -- request plumbing ---------------------------------------------------------------

    def _url(self, path: str) -> str:
        if not path.startswith("/"):
            raise HdfsError(f"HDFS path must be absolute: {path!r}")
        return f"{self.base_url}/webhdfs/v1{quote(path)}"

    def _rewrite_datanode_url(self, location: str) -> str:
        """Map a redirect to a DataNode's container hostname onto its published port, if configured."""
        parts = urlsplit(location)
        target = self.datanode_map.get(parts.hostname or "")
        if target is None:
            return location
        return urlunsplit((parts.scheme, target, parts.path, parts.query, parts.fragment))

    def _request(self, method: str, path: str, op: str, **params) -> httpx.Response:
        query = {"op": op, "user.name": self.user, **{k: str(v) for k, v in params.items() if v is not None}}
        try:
            response = self._http.request(method, self._url(path), params=query)
        except httpx.TransportError as exc:
            raise HdfsUnavailable(f"Cannot reach the NameNode at {self.base_url}: {exc}") from exc
        return response

    @staticmethod
    def _raise_for_status(response: httpx.Response, path: str) -> None:
        if response.is_success:
            return
        exception, message = "", response.text[:500]
        try:
            remote = response.json()["RemoteException"]
            exception, message = remote.get("exception", ""), remote.get("message", message)
        except (ValueError, KeyError):
            pass
        if response.status_code == 404 or exception == "FileNotFoundException":
            raise HdfsFileNotFound(f"Not found in HDFS: {path}")
        if exception in {"FileAlreadyExistsException", "AlreadyBeingCreatedException"}:
            raise HdfsAlreadyExists(f"Already exists in HDFS: {path}")
        if exception in {"SafeModeException", "RetriableException", "StandbyException"} or response.status_code == 503:
            raise HdfsUnavailable(f"HDFS is not accepting this request ({exception}): {message}")
        raise HdfsError(f"WebHDFS {exception or response.status_code} on {path}: {message}")

    def _redirect(self, method: str, path: str, op: str, **params) -> str:
        response = self._request(method, path, op, **params)
        if response.status_code not in (301, 302, 303, 307, 308):
            self._raise_for_status(response, path)
            raise HdfsError(f"WebHDFS {op} on {path} did not redirect to a DataNode")
        return self._rewrite_datanode_url(response.headers["location"])

    # -- filesystem operations ----------------------------------------------------------

    def mkdirs(self, path: str) -> None:
        response = self._request("PUT", path, "MKDIRS")
        self._raise_for_status(response, path)
```

```
    def create(
        self,
        path: str,
        data: bytes | Iterable[bytes],
        *,
        length: int | None = None,
        overwrite: bool = False,
        replication: int | None = None,
    ) -> None:
        """Two-step WebHDFS create: ask the NameNode where to write, then send the bytes to that DataNode."""
        location = self._redirect(
            "PUT", path, "CREATE", overwrite=str(overwrite).lower(), replication=replication
        )
        headers = {"Content-Type": "application/octet-stream"}
        if length is not None:
            headers["Content-Length"] = str(length)
        try:
            response = self._http.put(location, content=data, headers=headers)
        except httpx.TransportError as exc:
            raise HdfsUnavailable(f"Lost contact with the DataNode writing {path}: {exc}") from exc
        self._raise_for_status(response, path)

    def open(self, path: str) -> Iterator[bytes]:
        location = self._redirect("GET", path, "OPEN", buffersize=READ_BUFFER)
        try:
            with self._http.stream("GET", location) as response:
                if not response.is_success:
                    response.read()
                    self._raise_for_status(response, path)
                yield from response.iter_bytes(READ_BUFFER)
        except httpx.TransportError as exc:
            raise HdfsUnavailable(f"Lost contact with the DataNode reading {path}: {exc}") from exc

    @staticmethod
    def _to_status(parent: str, raw: dict) -> FileStatus:
        suffix = raw.get("pathSuffix") or ""
        full = parent if not suffix else f"{parent.rstrip('/')}/{suffix}"
        return FileStatus(
            path=full,
            is_dir=raw["type"] == "DIRECTORY",
            length=int(raw.get("length", 0)),
            replication=int(raw.get("replication", 0)),
            block_size=int(raw.get("blockSize", 0)),
            modification_time=int(raw.get("modificationTime", 0)),
        )

    def status(self, path: str) -> FileStatus | None:
        response = self._request("GET", path, "GETFILESTATUS")
        if response.status_code == 404:
            return None
        self._raise_for_status(response, path)
        return self._to_status(path, response.json()["FileStatus"])

    def list_dir(self, path: str) -> list[FileStatus]:
        response = self._request("GET", path, "LISTSTATUS")
        self._raise_for_status(response, path)
        return [self._to_status(path, raw) for raw in response.json()["FileStatuses"]["FileStatus"]]

    def rename(self, source: str, destination: str) -> bool:
        response = self._request("PUT", source, "RENAME", destination=destination)
        self._raise_for_status(response, source)
        return bool(response.json()["boolean"])

    def delete(self, path: str, *, recursive: bool = False) -> bool:
        response = self._request("DELETE", path, "DELETE", recursive=str(recursive).lower())
        self._raise_for_status(response, path)
        return bool(response.json()["boolean"])

    def set_replication(self, path: str, factor: int) -> bool:
        response = self._request("PUT", path, "SETREPLICATION", replication=factor)
        self._raise_for_status(response, path)
        return bool(response.json()["boolean"])

    def content_summary(self, path: str) -> ContentSummary:
        response = self._request("GET", path, "GETCONTENTSUMMARY")
        self._raise_for_status(response, path)
        raw = response.json()["ContentSummary"]
        return ContentSummary(
            length=int(raw["length"]),
            space_consumed=int(raw["spaceConsumed"]),
            file_count=int(raw["fileCount"]),
            directory_count=int(raw["directoryCount"]),
        )

    def block_locations(self, path: str) -> list[BlockLocation]:
        response = self._request("GET", path, "GET_BLOCK_LOCATIONS")
        self._raise_for_status(response, path)
        blocks = response.json()["LocatedBlocks"].get("locatedBlocks", [])
        return [
            BlockLocation(
                block_id=int(entry["block"]["blockId"]),
                length=int(entry["block"]["numBytes"]),
                hosts=tuple(sorted(loc["hostName"] for loc in entry.get("locations", []))),
```

```
                corrupt=bool(entry.get("isCorrupt", False)),
            )
            for entry in blocks
        ]

    def default_replication(self) -> int:
        response = self._request("GET", "/", "GETSERVERDEFAULTS")
        self._raise_for_status(response, "/")
        return int(response.json()["FsServerDefaults"]["replication"])
```

backend/migrations/env.py 31 lines

```
"""Alembic environment. SQLite needs batch mode for schema changes."""

from alembic import context
from sqlalchemy import create_engine

from app.core.db import Base
from app.models import entities  # noqa: F401 - registers the tables on Base.metadata

target_metadata = Base.metadata
url = context.config.get_main_option("sqlalchemy.url")


def run_offline() -> None:
    context.configure(url=url, target_metadata=target_metadata, literal_binds=True, render_as_batch=True)
    with context.begin_transaction():
        context.run_migrations()


def run_online() -> None:
    engine = create_engine(url)
    with engine.connect() as connection:
        context.configure(connection=connection, target_metadata=target_metadata, render_as_batch=True)
        with context.begin_transaction():
            context.run_migrations()


if context.is_offline_mode():
    run_offline()
else:
    run_online()
```

backend/migrations/script.py.mako 23 lines

```
"""${message}

Revision ID: ${up_revision}
Revises: ${down_revision | comma,n}
Create Date: ${create_date}
"""
from alembic import op
import sqlalchemy as sa
${imports if imports else ""}

revision = ${repr(up_revision)}
down_revision = ${repr(down_revision)}
branch_labels = ${repr(branch_labels)}
depends_on = ${repr(depends_on)}


def upgrade() -> None:
    ${upgrades if upgrades else "pass"}


def downgrade() -> None:
    ${downgrades if downgrades else "pass"}
```

backend/migrations/versions/a04f640e13c2_initial_schema.py 370 lines

```
"""initial schema

Revision ID: a04f640e13c2
Revises:
Create Date: 2026-11-05 10:39:57.479912
"""
from alembic import op
import sqlalchemy as sa


revision = 'a04f640e13c2'
down_revision = None
branch_labels = None
depends_on = None


def upgrade() -> None:
    # ### commands auto generated by Alembic - please adjust! ###
    op.create_table('app_settings',
    sa.Column('key', sa.String(length=64), nullable=False),
    sa.Column('value', sa.JSON(), nullable=True),
    sa.Column('updated_at', sa.DateTime(), nullable=False),
    sa.PrimaryKeyConstraint('key')
    )
    op.create_table('audit_events',
    sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),
```

```
    sa.Column('occurred_at', sa.DateTime(), nullable=False),
    sa.Column('event_type', sa.String(length=48), nullable=False),
    sa.Column('severity', sa.String(length=8), nullable=False),
    sa.Column('entity_type', sa.String(length=24), nullable=True),
    sa.Column('entity_id', sa.String(length=64), nullable=True),
    sa.Column('message', sa.Text(), nullable=False),
    sa.Column('details', sa.JSON(), nullable=True),
    sa.PrimaryKeyConstraint('id')
    )
    with op.batch_alter_table('audit_events', schema=None) as batch_op:
        batch_op.create_index(batch_op.f('ix_audit_events_event_type'), ['event_type'], unique=False)
        batch_op.create_index(batch_op.f('ix_audit_events_occurred_at'), ['occurred_at'], unique=False)
        batch_op.create_index(batch_op.f('ix_audit_events_severity'), ['severity'], unique=False)

    op.create_table('chunks',
    sa.Column('id', sa.String(length=32), nullable=False),
    sa.Column('sha256', sa.String(length=64), nullable=False),
    sa.Column('size_bytes', sa.BigInteger(), nullable=False),
    sa.Column('hdfs_path', sa.String(length=512), nullable=False),
    sa.Column('state', sa.String(length=16), nullable=False),
    sa.Column('created_at', sa.DateTime(), nullable=False),
    sa.Column('removed_at', sa.DateTime(), nullable=True),
    sa.PrimaryKeyConstraint('id')
    )
    with op.batch_alter_table('chunks', schema=None) as batch_op:
        batch_op.create_index(batch_op.f('ix_chunks_sha256'), ['sha256'], unique=True)
        batch_op.create_index(batch_op.f('ix_chunks_state'), ['state'], unique=False)

    op.create_table('dataset_sources',
    sa.Column('id', sa.String(length=32), nullable=False),
    sa.Column('title', sa.String(length=255), nullable=False),
    sa.Column('author', sa.String(length=255), nullable=True),
    sa.Column('ebook_id', sa.Integer(), nullable=True),
    sa.Column('source_url', sa.String(length=512), nullable=True),
    sa.Column('download_url', sa.String(length=512), nullable=True),
    sa.Column('organization', sa.String(length=255), nullable=True),
    sa.Column('downloaded_at', sa.DateTime(), nullable=True),
    sa.Column('file_format', sa.String(length=32), nullable=False),
    sa.Column('size_bytes', sa.BigInteger(), nullable=False),
    sa.Column('sha256', sa.String(length=64), nullable=False),
    sa.Column('kind', sa.String(length=24), nullable=False),
    sa.Column('derived_from_id', sa.String(length=32), nullable=True),
    sa.Column('transformation', sa.Text(), nullable=True),
    sa.Column('local_path', sa.String(length=512), nullable=False),
    sa.Column('license_note', sa.Text(), nullable=True),
    sa.CheckConstraint("kind = 'original' OR derived_from_id IS NOT NULL OR transformation IS NOT NULL",
name='ck_dataset_derived_is_documented'),
    sa.CheckConstraint("kind IN ('original','workload_copy','transformed')", name='ck_dataset_kind'),
    sa.ForeignKeyConstraint(['derived_from_id'], ['dataset_sources.id'], ondelete='SET NULL'),
    sa.PrimaryKeyConstraint('id')
    )
    with op.batch_alter_table('dataset_sources', schema=None) as batch_op:
        batch_op.create_index(batch_op.f('ix_dataset_sources_sha256'), ['sha256'], unique=False)

    op.create_table('deduplication_jobs',
    sa.Column('id', sa.String(length=32), nullable=False),
    sa.Column('kind', sa.String(length=24), nullable=False),
    sa.Column('mode', sa.String(length=8), nullable=True),
    sa.Column('chunk_size', sa.Integer(), nullable=True),
    sa.Column('status', sa.String(length=16), nullable=False),
    sa.Column('progress_done', sa.Integer(), nullable=False),
    sa.Column('progress_total', sa.Integer(), nullable=False),
    sa.Column('message', sa.Text(), nullable=True),
    sa.Column('params', sa.JSON(), nullable=True),
    sa.Column('result', sa.JSON(), nullable=True),
    sa.Column('error', sa.Text(), nullable=True),
    sa.Column('resumed_from_id', sa.String(length=32), nullable=True),
    sa.Column('created_at', sa.DateTime(), nullable=False),
    sa.Column('started_at', sa.DateTime(), nullable=True),
    sa.Column('finished_at', sa.DateTime(), nullable=True),
    sa.Column('heartbeat_at', sa.DateTime(), nullable=True),
    sa.PrimaryKeyConstraint('id')
    )
    with op.batch_alter_table('deduplication_jobs', schema=None) as batch_op:
        batch_op.create_index(batch_op.f('ix_deduplication_jobs_created_at'), ['created_at'], unique=False)
        batch_op.create_index(batch_op.f('ix_deduplication_jobs_kind'), ['kind'], unique=False)
        batch_op.create_index(batch_op.f('ix_deduplication_jobs_status'), ['status'], unique=False)

    op.create_table('experiment_runs',
    sa.Column('id', sa.String(length=32), nullable=False),
    sa.Column('scenario', sa.String(length=48), nullable=False),
    sa.Column('params', sa.JSON(), nullable=True),
    sa.Column('status', sa.String(length=16), nullable=False),
    sa.Column('result', sa.JSON(), nullable=True),
    sa.Column('error', sa.Text(), nullable=True),
    sa.Column('job_id', sa.String(length=32), nullable=True),
    sa.Column('started_at', sa.DateTime(), nullable=False),
    sa.Column('finished_at', sa.DateTime(), nullable=True),
    sa.PrimaryKeyConstraint('id')
    )
    with op.batch_alter_table('experiment_runs', schema=None) as batch_op:
        batch_op.create_index(batch_op.f('ix_experiment_runs_scenario'), ['scenario'], unique=False)
        batch_op.create_index(batch_op.f('ix_experiment_runs_started_at'), ['started_at'], unique=False)
```

```
    op.create_table('file_manifests',
    sa.Column('id', sa.String(length=32), nullable=False),
    sa.Column('file_sha256', sa.String(length=64), nullable=False),
    sa.Column('file_size', sa.BigInteger(), nullable=False),
    sa.Column('chunk_size', sa.Integer(), nullable=False),
    sa.Column('chunk_count', sa.Integer(), nullable=False),
    sa.Column('manifest_bytes', sa.Integer(), nullable=False),
    sa.Column('hdfs_path', sa.String(length=512), nullable=False),
    sa.Column('state', sa.String(length=16), nullable=False),
    sa.Column('created_at', sa.DateTime(), nullable=False),
    sa.Column('removed_at', sa.DateTime(), nullable=True),
    sa.PrimaryKeyConstraint('id'),
    sa.UniqueConstraint('file_sha256', 'chunk_size', name='uq_manifest_content')
    )
    with op.batch_alter_table('file_manifests', schema=None) as batch_op:
        batch_op.create_index(batch_op.f('ix_file_manifests_file_sha256'), ['file_sha256'], unique=False)
        batch_op.create_index(batch_op.f('ix_file_manifests_state'), ['state'], unique=False)

    op.create_table('objects',
    sa.Column('id', sa.String(length=32), nullable=False),
    sa.Column('sha256', sa.String(length=64), nullable=False),
    sa.Column('size_bytes', sa.BigInteger(), nullable=False),
    sa.Column('hdfs_path', sa.String(length=512), nullable=False),
    sa.Column('state', sa.String(length=16), nullable=False),
    sa.Column('created_at', sa.DateTime(), nullable=False),
    sa.Column('removed_at', sa.DateTime(), nullable=True),
    sa.PrimaryKeyConstraint('id')
    )
    with op.batch_alter_table('objects', schema=None) as batch_op:
        batch_op.create_index(batch_op.f('ix_objects_sha256'), ['sha256'], unique=True)
        batch_op.create_index(batch_op.f('ix_objects_state'), ['state'], unique=False)

    op.create_table('performance_samples',
    sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),
    sa.Column('operation', sa.String(length=32), nullable=False),
    sa.Column('mode', sa.String(length=8), nullable=True),
    sa.Column('bytes_processed', sa.BigInteger(), nullable=False),
    sa.Column('duration_ms', sa.Float(), nullable=False),
    sa.Column('file_id', sa.String(length=32), nullable=True),
    sa.Column('created_at', sa.DateTime(), nullable=False),
    sa.PrimaryKeyConstraint('id')
    )
    with op.batch_alter_table('performance_samples', schema=None) as batch_op:
        batch_op.create_index(batch_op.f('ix_performance_samples_created_at'), ['created_at'], unique=False)
        batch_op.create_index(batch_op.f('ix_performance_samples_operation'), ['operation'], unique=False)

    op.create_table('replication_operations',
    sa.Column('id', sa.String(length=32), nullable=False),
    sa.Column('previous_factor', sa.Integer(), nullable=True),
    sa.Column('requested_factor', sa.Integer(), nullable=False),
    sa.Column('status', sa.String(length=16), nullable=False),
    sa.Column('files_total', sa.Integer(), nullable=False),
    sa.Column('files_changed', sa.Integer(), nullable=False),
    sa.Column('space_consumed_before', sa.BigInteger(), nullable=True),
    sa.Column('space_consumed_after', sa.BigInteger(), nullable=True),
    sa.Column('details', sa.JSON(), nullable=True),
    sa.Column('error', sa.Text(), nullable=True),
    sa.Column('requested_at', sa.DateTime(), nullable=False),
    sa.Column('completed_at', sa.DateTime(), nullable=True),
    sa.PrimaryKeyConstraint('id')
    )
    with op.batch_alter_table('replication_operations', schema=None) as batch_op:
        batch_op.create_index(batch_op.f('ix_replication_operations_requested_at'), ['requested_at'], unique=False)

    op.create_table('storage_snapshots',
    sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),
    sa.Column('taken_at', sa.DateTime(), nullable=False),
    sa.Column('reason', sa.String(length=48), nullable=False),
    sa.Column('logical_files', sa.Integer(), nullable=False),
    sa.Column('logical_bytes', sa.BigInteger(), nullable=False),
    sa.Column('stored_bytes', sa.BigInteger(), nullable=False),
    sa.Column('unique_files', sa.Integer(), nullable=False),
    sa.PrimaryKeyConstraint('id')
    )
    with op.batch_alter_table('storage_snapshots', schema=None) as batch_op:
        batch_op.create_index(batch_op.f('ix_storage_snapshots_taken_at'), ['taken_at'], unique=False)

    op.create_table('file_chunks',
    sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),
    sa.Column('manifest_id', sa.String(length=32), nullable=False),
    sa.Column('chunk_id', sa.String(length=32), nullable=False),
    sa.Column('ordinal', sa.Integer(), nullable=False),
    sa.Column('offset_bytes', sa.BigInteger(), nullable=False),
    sa.CheckConstraint('ordinal >= 0', name='ck_ordinal_nonnegative'),
    sa.ForeignKeyConstraint(['chunk_id'], ['chunks.id'], ondelete='RESTRICT'),
    sa.ForeignKeyConstraint(['manifest_id'], ['file_manifests.id'], ondelete='CASCADE'),
    sa.PrimaryKeyConstraint('id'),
    sa.UniqueConstraint('manifest_id', 'ordinal', name='uq_manifest_ordinal')
    )
    with op.batch_alter_table('file_chunks', schema=None) as batch_op:
        batch_op.create_index(batch_op.f('ix_file_chunks_chunk_id'), ['chunk_id'], unique=False)
        batch_op.create_index(batch_op.f('ix_file_chunks_manifest_id'), ['manifest_id'], unique=False)
```

```
    op.create_table('files',
    sa.Column('id', sa.String(length=32), nullable=False),
    sa.Column('name', sa.String(length=255), nullable=False),
    sa.Column('folder', sa.String(length=512), nullable=False),
    sa.Column('size_bytes', sa.BigInteger(), nullable=False),
    sa.Column('sha256', sa.String(length=64), nullable=False),
    sa.Column('mime_type', sa.String(length=128), nullable=True),
    sa.Column('extension', sa.String(length=32), nullable=True),
    sa.Column('storage_mode', sa.String(length=8), nullable=False),
    sa.Column('status', sa.String(length=8), nullable=False),
    sa.Column('raw_path', sa.String(length=512), nullable=True),
    sa.Column('object_id', sa.String(length=32), nullable=True),
    sa.Column('manifest_id', sa.String(length=32), nullable=True),
    sa.Column('provenance_label', sa.String(length=24), nullable=False),
    sa.Column('dataset_source_id', sa.String(length=32), nullable=True),
    sa.Column('created_at', sa.DateTime(), nullable=False),
    sa.Column('updated_at', sa.DateTime(), nullable=False),
    sa.Column('deleted_at', sa.DateTime(), nullable=True),
    sa.Column('purged_at', sa.DateTime(), nullable=True),
    sa.Column('last_verified_at', sa.DateTime(), nullable=True),
    sa.Column('last_verified_ok', sa.Boolean(), nullable=True),
    sa.CheckConstraint("status = 'purged' OR (storage_mode = 'raw' AND raw_path IS NOT NULL AND object_id IS NULL AND manifest_id IS
NULL) OR (storage_mode = 'file' AND raw_path IS NULL AND object_id IS NOT NULL AND manifest_id IS NULL) OR (storage_mode = 'chunk'
AND raw_path IS NULL AND object_id IS NULL AND manifest_id IS NOT NULL)", name='ck_file_storage_pointer'),
    sa.CheckConstraint('size_bytes >= 0', name='ck_file_size'),
    sa.ForeignKeyConstraint(['dataset_source_id'], ['dataset_sources.id'], ondelete='SET NULL'),
    sa.ForeignKeyConstraint(['manifest_id'], ['file_manifests.id'], ondelete='RESTRICT'),
    sa.ForeignKeyConstraint(['object_id'], ['objects.id'], ondelete='RESTRICT'),
    sa.PrimaryKeyConstraint('id')
    )
    with op.batch_alter_table('files', schema=None) as batch_op:
        batch_op.create_index(batch_op.f('ix_files_folder'), ['folder'], unique=False)
        batch_op.create_index(batch_op.f('ix_files_manifest_id'), ['manifest_id'], unique=False)
        batch_op.create_index(batch_op.f('ix_files_object_id'), ['object_id'], unique=False)
        batch_op.create_index(batch_op.f('ix_files_sha256'), ['sha256'], unique=False)
        batch_op.create_index(batch_op.f('ix_files_status'), ['status'], unique=False)
        batch_op.create_index('ix_files_status_created', ['status', 'created_at'], unique=False)
        batch_op.create_index('uq_active_file_name', ['folder', 'name'], unique=True, sqlite_where=sa.text("status = 'active'"))

    op.create_table('optimization_history',
    sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),
    sa.Column('job_id', sa.String(length=32), nullable=False),
    sa.Column('mode', sa.String(length=8), nullable=False),
    sa.Column('chunk_size', sa.Integer(), nullable=True),
    sa.Column('before', sa.JSON(), nullable=False),
    sa.Column('after', sa.JSON(), nullable=False),
    sa.Column('files_processed', sa.Integer(), nullable=False),
    sa.Column('files_failed', sa.Integer(), nullable=False),
    sa.Column('bytes_reclaimed_logical', sa.BigInteger(), nullable=False),
    sa.Column('duration_ms', sa.Float(), nullable=False),
    sa.Column('created_at', sa.DateTime(), nullable=False),
    sa.ForeignKeyConstraint(['job_id'], ['deduplication_jobs.id'], ondelete='CASCADE'),
    sa.PrimaryKeyConstraint('id')
    )
    with op.batch_alter_table('optimization_history', schema=None) as batch_op:
        batch_op.create_index(batch_op.f('ix_optimization_history_created_at'), ['created_at'], unique=False)
        batch_op.create_index(batch_op.f('ix_optimization_history_job_id'), ['job_id'], unique=False)

    op.create_table('integrity_checks',
    sa.Column('id', sa.Integer(), autoincrement=True, nullable=False),
    sa.Column('file_id', sa.String(length=32), nullable=False),
    sa.Column('ok', sa.Boolean(), nullable=False),
    sa.Column('expected_sha256', sa.String(length=64), nullable=False),
    sa.Column('actual_sha256', sa.String(length=64), nullable=True),
    sa.Column('bytes_read', sa.BigInteger(), nullable=False),
    sa.Column('chunks_checked', sa.Integer(), nullable=False),
    sa.Column('missing', sa.JSON(), nullable=True),
    sa.Column('corrupt', sa.JSON(), nullable=True),
    sa.Column('detail', sa.Text(), nullable=True),
    sa.Column('duration_ms', sa.Float(), nullable=False),
    sa.Column('checked_at', sa.DateTime(), nullable=False),
    sa.ForeignKeyConstraint(['file_id'], ['files.id'], ondelete='CASCADE'),
    sa.PrimaryKeyConstraint('id')
    )
    with op.batch_alter_table('integrity_checks', schema=None) as batch_op:
        batch_op.create_index(batch_op.f('ix_integrity_checks_checked_at'), ['checked_at'], unique=False)
        batch_op.create_index(batch_op.f('ix_integrity_checks_file_id'), ['file_id'], unique=False)

    # ### end Alembic commands ###


def downgrade() -> None:
    # ### commands auto generated by Alembic - please adjust! ###
    with op.batch_alter_table('integrity_checks', schema=None) as batch_op:
        batch_op.drop_index(batch_op.f('ix_integrity_checks_file_id'))
        batch_op.drop_index(batch_op.f('ix_integrity_checks_checked_at'))

    op.drop_table('integrity_checks')
    with op.batch_alter_table('optimization_history', schema=None) as batch_op:
        batch_op.drop_index(batch_op.f('ix_optimization_history_job_id'))
        batch_op.drop_index(batch_op.f('ix_optimization_history_created_at'))
```

```
    op.drop_table('optimization_history')
    with op.batch_alter_table('files', schema=None) as batch_op:
        batch_op.drop_index('uq_active_file_name', sqlite_where=sa.text("status = 'active'"))
        batch_op.drop_index('ix_files_status_created')
        batch_op.drop_index(batch_op.f('ix_files_status'))
        batch_op.drop_index(batch_op.f('ix_files_sha256'))
        batch_op.drop_index(batch_op.f('ix_files_object_id'))
        batch_op.drop_index(batch_op.f('ix_files_manifest_id'))
        batch_op.drop_index(batch_op.f('ix_files_folder'))

    op.drop_table('files')
    with op.batch_alter_table('file_chunks', schema=None) as batch_op:
        batch_op.drop_index(batch_op.f('ix_file_chunks_manifest_id'))
        batch_op.drop_index(batch_op.f('ix_file_chunks_chunk_id'))

    op.drop_table('file_chunks')
    with op.batch_alter_table('storage_snapshots', schema=None) as batch_op:
        batch_op.drop_index(batch_op.f('ix_storage_snapshots_taken_at'))

    op.drop_table('storage_snapshots')
    with op.batch_alter_table('replication_operations', schema=None) as batch_op:
        batch_op.drop_index(batch_op.f('ix_replication_operations_requested_at'))

    op.drop_table('replication_operations')
    with op.batch_alter_table('performance_samples', schema=None) as batch_op:
        batch_op.drop_index(batch_op.f('ix_performance_samples_operation'))
        batch_op.drop_index(batch_op.f('ix_performance_samples_created_at'))

    op.drop_table('performance_samples')
    with op.batch_alter_table('objects', schema=None) as batch_op:
        batch_op.drop_index(batch_op.f('ix_objects_state'))
        batch_op.drop_index(batch_op.f('ix_objects_sha256'))

    op.drop_table('objects')
    with op.batch_alter_table('file_manifests', schema=None) as batch_op:
        batch_op.drop_index(batch_op.f('ix_file_manifests_state'))
        batch_op.drop_index(batch_op.f('ix_file_manifests_file_sha256'))

    op.drop_table('file_manifests')
    with op.batch_alter_table('experiment_runs', schema=None) as batch_op:
        batch_op.drop_index(batch_op.f('ix_experiment_runs_started_at'))
        batch_op.drop_index(batch_op.f('ix_experiment_runs_scenario'))

    op.drop_table('experiment_runs')
    with op.batch_alter_table('deduplication_jobs', schema=None) as batch_op:
        batch_op.drop_index(batch_op.f('ix_deduplication_jobs_status'))
        batch_op.drop_index(batch_op.f('ix_deduplication_jobs_kind'))
        batch_op.drop_index(batch_op.f('ix_deduplication_jobs_created_at'))

    op.drop_table('deduplication_jobs')
    with op.batch_alter_table('dataset_sources', schema=None) as batch_op:
        batch_op.drop_index(batch_op.f('ix_dataset_sources_sha256'))

    op.drop_table('dataset_sources')
    with op.batch_alter_table('chunks', schema=None) as batch_op:
        batch_op.drop_index(batch_op.f('ix_chunks_state'))
        batch_op.drop_index(batch_op.f('ix_chunks_sha256'))

    op.drop_table('chunks')
    with op.batch_alter_table('audit_events', schema=None) as batch_op:
        batch_op.drop_index(batch_op.f('ix_audit_events_severity'))
        batch_op.drop_index(batch_op.f('ix_audit_events_occurred_at'))
        batch_op.drop_index(batch_op.f('ix_audit_events_event_type'))

    op.drop_table('audit_events')
    op.drop_table('app_settings')
    # ### end Alembic commands ###
```

11.3 Backend: deduplication and services

backend/app/services/analytics.py 305 lines

```
"""Storage metrics. Each number states whether the database derived it or HDFS measured it.

    Savings (%)          = (Original Logical Bytes - Stored Content Bytes) / Original Logical Bytes x 100
    Deduplication Ratio  = Original Logical Bytes / Stored Content Bytes

Both are undefined (None) when their denominator is zero.
"""

from __future__ import annotations

from sqlalchemy import case, distinct, func, select
from sqlalchemy.orm import Session

from app.core.errors import HdfsError
from app.models.entities import (
    Chunk,
    ContentObject,
    ContentState,
    FileChunk,
    FileManifest,
    FileStatus,
    IntegrityCheck,
```

```
    LogicalFile,
    OptimizationHistory,
    PerformanceSample,
    StorageMode,
    StorageSnapshot,
)
from app.services import settings_store
from app.services.context import AppContext


def savings_percent(logical: int, stored: int) -> float | None:
    return None if logical <= 0 else (logical - stored) / logical * 100


def dedup_ratio(logical: int, stored: int) -> float | None:
    return None if stored <= 0 else logical / stored


def storage_statistics(session: Session) -> dict:
    """Values derived from metadata relations, describing application-level content."""
    active = LogicalFile.status == FileStatus.ACTIVE
    logical_files, logical_bytes = session.execute(
        select(func.count(), func.coalesce(func.sum(LogicalFile.size_bytes), 0)).where(active)
    ).one()
    deleted_files, deleted_bytes = session.execute(
        select(func.count(), func.coalesce(func.sum(LogicalFile.size_bytes), 0)).where(
            LogicalFile.status == FileStatus.DELETED
        )
    ).one()

    distinct_contents = select(LogicalFile.sha256,
func.max(LogicalFile.size_bytes).label("size")).where(active).group_by(LogicalFile.sha256).subquery()
    unique_files, unique_bytes = session.execute(
        select(func.count(), func.coalesce(func.sum(distinct_contents.c.size), 0))
    ).one()

    raw_files, raw_bytes = session.execute(
        select(func.count(), func.coalesce(func.sum(LogicalFile.size_bytes), 0)).where(
            LogicalFile.storage_mode == StorageMode.RAW, LogicalFile.status != FileStatus.PURGED
        )
    ).one()
    object_count, object_bytes = session.execute(
        select(func.count(), func.coalesce(func.sum(ContentObject.size_bytes), 0)).where(
            ContentObject.state == ContentState.ACTIVE
        )
    ).one()
    chunk_count, chunk_bytes = session.execute(
        select(func.count(), func.coalesce(func.sum(Chunk.size_bytes), 0)).where(Chunk.state == ContentState.ACTIVE)
    ).one()
    manifest_count, manifest_bytes = session.execute(
        select(func.count(), func.coalesce(func.sum(FileManifest.manifest_bytes), 0)).where(
            FileManifest.state == ContentState.ACTIVE
        )
    ).one()
    chunk_references = session.scalar(
        select(func.count())
        .select_from(FileChunk)
        .join(FileManifest, FileManifest.id == FileChunk.manifest_id)
        .where(FileManifest.state == ContentState.ACTIVE)
    )
    referenced_chunks = session.scalar(
        select(func.count(distinct(FileChunk.chunk_id)))
        .join(FileManifest, FileManifest.id == FileChunk.manifest_id)
        .where(FileManifest.state == ContentState.ACTIVE)
    )
    by_mode = dict(
        session.execute(select(LogicalFile.storage_mode, func.count()).where(active).group_by(LogicalFile.storage_mode)).all()
    )

    stored = raw_bytes + object_bytes + chunk_bytes
    return {
        "logical_files": logical_files,
        "logical_bytes": logical_bytes,
        "unique_files": unique_files,
        "unique_content_bytes": unique_bytes,
        "duplicate_files": logical_files - unique_files,
        "deleted_retained_files": deleted_files,
        "deleted_retained_bytes": deleted_bytes,
        "raw_files": raw_files,
        "raw_bytes": raw_bytes,
        "object_count": object_count,
        "object_bytes": object_bytes,
        "chunk_count": chunk_count,
        "chunk_bytes": chunk_bytes,
        "manifest_count": manifest_count,
        "manifest_bytes": manifest_bytes,
        "chunk_references": chunk_references,
        "duplicate_chunk_references": chunk_references - referenced_chunks,
        "stored_content_bytes": stored,
        "bytes_avoided": logical_bytes - stored,
        "savings_percent": savings_percent(logical_bytes, stored),
        "dedup_ratio": dedup_ratio(logical_bytes, stored),
        "files_by_mode": by_mode,
```

```
    }


def measured_hdfs(ctx: AppContext) -> dict:
    """Values the NameNode reports for the managed namespace. spaceConsumed includes replication."""
    try:
        summary = ctx.hdfs.content_summary(ctx.paths.root)
        areas = {}
        for name, path in (
            ("objects", ctx.paths.objects),
            ("chunks", ctx.paths.chunks),
            ("manifests", ctx.paths.manifests),
            ("raw", ctx.paths.raw),
            ("staging", ctx.paths.staging),
            ("recovery", ctx.paths.recovery),
        ):
            area = ctx.hdfs.content_summary(path)
            areas[name] = {
                "length": area.length,
                "space_consumed": area.space_consumed,
                "file_count": area.file_count,
            }
    except HdfsError as exc:
        return {"available": False, "error": exc.message}
    return {
        "available": True,
        "length": summary.length,
        "space_consumed": summary.space_consumed,
        "file_count": summary.file_count,
        "directory_count": summary.directory_count,
        "areas": areas,
    }


def full_statistics(ctx: AppContext, session: Session) -> dict:
    stats = storage_statistics(session)
    measured = measured_hdfs(ctx)
    target = settings_store.get_value(ctx, session, "target_replication")
    factor = target
    if factor is None:
        try:
            factor = ctx.hdfs.default_replication()
        except HdfsError:
            factor = None
    return {
        "derived": stats,
        "measured": measured,
        "estimated": {
            "replication_factor": factor,
            "replication_adjusted_bytes": None if factor is None else stats["stored_content_bytes"] * factor,
            "note": "Stored content multiplied by the replication factor. This excludes block checksums and "
            "filesystem overhead; see the measured values for what the cluster reports.",
        },
    }


def record_snapshot(session: Session, reason: str) -> None:
    session.flush()
    stats = storage_statistics(session)
    session.add(
        StorageSnapshot(
            reason=reason,
            logical_files=stats["logical_files"],
            logical_bytes=stats["logical_bytes"],
            stored_bytes=stats["stored_content_bytes"],
            unique_files=stats["unique_files"],
        )
    )


def history(session: Session, limit: int = 500) -> list[dict]:
    rows = list(session.scalars(select(StorageSnapshot).order_by(StorageSnapshot.taken_at.desc(),
StorageSnapshot.id.desc()).limit(limit)))
    rows.reverse()
    return [
        {
            "taken_at": row.taken_at,
            "reason": row.reason,
            "logical_files": row.logical_files,
            "logical_bytes": row.logical_bytes,
            "stored_bytes": row.stored_bytes,
            "unique_files": row.unique_files,
            "saved_bytes": row.logical_bytes - row.stored_bytes,
            "savings_percent": savings_percent(row.logical_bytes, row.stored_bytes),
        }
        for row in rows
    ]


def distributions(session: Session) -> dict:
    active = LogicalFile.status == FileStatus.ACTIVE
    by_type = session.execute(
        select(
```

```
            func.coalesce(LogicalFile.extension, "none"),
            func.count(),
            func.coalesce(func.sum(LogicalFile.size_bytes), 0),
        )
        .where(active)
        .group_by(LogicalFile.extension)
        .order_by(func.sum(LogicalFile.size_bytes).desc())
    ).all()
    groups = session.execute(select(func.count()).where(active).group_by(LogicalFile.sha256)).all()
    histogram: dict[int, int] = {}
    for (copies,) in groups:
        histogram[copies] = histogram.get(copies, 0) + 1
    upload_day = func.date(LogicalFile.created_at)
    uploads = session.execute(
        select(upload_day, func.count(), func.coalesce(func.sum(LogicalFile.size_bytes), 0))
        .where(LogicalFile.status != FileStatus.PURGED)
        .group_by(upload_day)
        .order_by(upload_day)
    ).all()
    return {
        "file_types": [{"extension": ext, "files": files, "bytes": size} for ext, files, size in by_type],
        "duplicate_distribution": [{"copies": copies, "groups": histogram[copies]} for copies in sorted(histogram)],
        "upload_trend": [{"day": day, "files": files, "bytes": size} for day, files, size in uploads],
    }


def optimization_history(session: Session, limit: int = 50) -> list[dict]:
    rows = session.scalars(select(OptimizationHistory).order_by(OptimizationHistory.created_at.desc()).limit(limit))
    return [
        {
            "job_id": row.job_id,
            "mode": row.mode,
            "chunk_size": row.chunk_size,
            "before": row.before,
            "after": row.after,
            "files_processed": row.files_processed,
            "files_failed": row.files_failed,
            "bytes_reclaimed_logical": row.bytes_reclaimed_logical,
            "duration_ms": row.duration_ms,
            "created_at": row.created_at,
        }
        for row in rows
    ]


def performance(session: Session) -> dict:
    """Aggregates over measured samples. Throughput is total bytes divided by total time."""
    rows = session.execute(
        select(
            PerformanceSample.operation,
            PerformanceSample.mode,
            func.count(),
            func.sum(PerformanceSample.bytes_processed),
            func.sum(PerformanceSample.duration_ms),
            func.min(PerformanceSample.duration_ms),
            func.max(PerformanceSample.duration_ms),
        ).group_by(PerformanceSample.operation, PerformanceSample.mode)
    ).all()
    operations = [
        {
            "operation": op,
            "mode": mode,
            "samples": count,
            "bytes": total_bytes,
            "total_ms": total_ms,
            "min_ms": low,
            "max_ms": high,
            "mean_ms": total_ms / count,
            "throughput_mib_s": (total_bytes / (1024 * 1024)) / (total_ms / 1000) if total_ms else None,
        }
        for op, mode, count, total_bytes, total_ms, low, high in rows
    ]
    return {"operations": operations, "integrity": integrity_summary(session)}


def integrity_summary(session: Session) -> dict:
    total, passed = session.execute(
        select(func.count(), func.coalesce(func.sum(case((IntegrityCheck.ok.is_(True), 1), else_=0)), 0))
    ).one()
    verified_files = session.scalar(
        select(func.count()).select_from(LogicalFile).where(
            LogicalFile.status == FileStatus.ACTIVE, LogicalFile.last_verified_ok.is_(True)
        )
    )
    failed_files = session.scalar(
        select(func.count()).select_from(LogicalFile).where(
            LogicalFile.status == FileStatus.ACTIVE, LogicalFile.last_verified_ok.is_(False)
        )
    )
    active_files = session.scalar(select(func.count()).select_from(LogicalFile).where(LogicalFile.status == FileStatus.ACTIVE))
    return {
        "checks_total": total,
        "checks_passed": passed,
```

```
        "checks_failed": total - passed,
        "files_verified_ok": verified_files,
        "files_verified_failed": failed_files,
        "files_never_verified": active_files - verified_files - failed_files,
    }
```

backend/app/services/audit.py 57 lines

```python
"""Append-only audit trail."""

from __future__ import annotations

from sqlalchemy.orm import Session, sessionmaker

from app.core.db import session_scope
from app.models.entities import AuditEvent, PerformanceSample


def record(
    session: Session,
    event_type: str,
    message: str,
    *,
    severity: str = "info",
    entity_type: str | None = None,
    entity_id: str | None = None,
    details: dict | None = None,
) -> None:
    session.add(
        AuditEvent(
            event_type=event_type,
            severity=severity,
            entity_type=entity_type,
            entity_id=entity_id,
            message=message,
            details=details,
        )
    )


def record_failure(factory: sessionmaker[Session], event_type: str, message: str, **kwargs) -> None:
    """Record a failure in its own transaction, so it survives the rollback of the failed operation."""
    with session_scope(factory) as session:
        record(session, event_type, message, severity="error", **kwargs)


def record_sample(
    session: Session,
    operation: str,
    bytes_processed: int,
    duration_ms: float,
    *,
    mode: str | None = None,
    file_id: str | None = None,
) -> None:
    session.add(
        PerformanceSample(
            operation=operation,
            bytes_processed=bytes_processed,
            duration_ms=duration_ms,
            mode=mode,
            file_id=file_id,
        )
    )
```

backend/app/services/content_store.py 304 lines

```python
"""Content-addressable storage primitives.

HDFS and SQLite cannot share a transaction, so every write follows the same order:

    1. write to /staging/<random>       (a crash here leaves only an unreferenced staging file)
    2. check length and, optionally, the SHA-256 of what HDFS now holds
    3. atomically rename into the content-addressed path
    4. commit the metadata row          (a crash before this leaves a detectable, unreferenced orphan)

Metadata therefore never points at bytes that are not fully stored and checked.
"""

from __future__ import annotations

import hashlib
import json
import time
import uuid
from collections.abc import Callable, Iterable, Iterator
from dataclasses import dataclass, field
from pathlib import Path

from sqlalchemy import select
from sqlalchemy.orm import Session

from app.core.errors import HdfsError, IntegrityFailure
from app.models.entities import (
    Chunk,
```

```
    ContentObject,
    ContentState,
    FileChunk,
    FileManifest,
    LogicalFile,
    StorageMode,
)
from app.services import settings_store
from app.services.context import AppContext
from app.services.hashing import (
    fixed_size_chunks,
    read_file_blocks,
    sha256_bytes,
    sha256_stream,
)


def parent_of(path: str) -> str:
    return path.rsplit("/", 1)[0] or "/"


def target_replication(ctx: AppContext, session: Session) -> int | None:
    return settings_store.get_value(ctx, session, "target_replication")


def put_content(
    ctx: AppContext,
    final_path: str,
    data_factory: Callable[[], bytes | Iterable[bytes]],
    expected_sha256: str,
    size: int,
    *,
    verify: bool,
    replication: int | None = None,
) -> bool:
    """Store bytes at ``final_path`` following the staged protocol. Returns True if new bytes were written."""
    hdfs = ctx.hdfs
    existing = hdfs.status(final_path)
    if existing is not None:
        # Possibly left by a crash between rename and commit. Adopt it only if it proves correct.
        actual, length = sha256_stream(hdfs.open(final_path))
        if actual == expected_sha256 and length == size:
            return False
        quarantine = ctx.paths.recovery_orphan_path(f"corrupt-{uuid.uuid4().hex}")
        hdfs.mkdirs(parent_of(quarantine))
        hdfs.rename(final_path, quarantine)

    staging = ctx.paths.staging_path(uuid.uuid4().hex)
    try:
        hdfs.create(staging, data_factory(), length=size, overwrite=False, replication=replication)
        status = hdfs.status(staging)
        if status is None or status.length != size:
            found = None if status is None else status.length
            raise IntegrityFailure(f"Staged write to {staging} has length {found}, expected {size}.")
        if verify:
            actual, _ = sha256_stream(hdfs.open(staging))
            if actual != expected_sha256:
                raise IntegrityFailure(f"Staged write has SHA-256 {actual}, expected {expected_sha256}.")
        hdfs.mkdirs(parent_of(final_path))
        if not hdfs.rename(staging, final_path):
            # HDFS refuses to rename onto an existing path, so another writer got there first.
            if hdfs.status(final_path) is None:
                raise HdfsError(f"HDFS refused the rename to {final_path}.")
            hdfs.delete(staging)
            return False
        return True
    except Exception:
        _delete_quietly(ctx, staging)
        raise


def _delete_quietly(ctx: AppContext, path: str) -> None:
    try:
        ctx.hdfs.delete(path)
    except HdfsError:
        pass  # staging is swept by recovery, so this must not mask the original error


def find_active_object(session: Session, sha256: str) -> ContentObject | None:
    return session.scalar(
        select(ContentObject).where(ContentObject.sha256 == sha256, ContentObject.state == ContentState.ACTIVE)
    )


def find_active_manifest(session: Session, sha256: str, chunk_size: int | None = None) -> FileManifest | None:
    query = select(FileManifest).where(FileManifest.file_sha256 == sha256, FileManifest.state == ContentState.ACTIVE)
    if chunk_size is not None:
        query = query.where(FileManifest.chunk_size == chunk_size)
    return session.scalars(query.order_by(FileManifest.created_at)).first()


def register_object(session: Session, sha256: str, size: int, hdfs_path: str) -> ContentObject:
    """Create or reactivate the row for a content value. Removed rows are reused, keeping one per hash."""
```

```
    row = session.scalar(select(ContentObject).where(ContentObject.sha256 == sha256))
    if row is None:
        row = ContentObject(sha256=sha256, size_bytes=size, hdfs_path=hdfs_path)
        session.add(row)
    else:
        row.state = ContentState.ACTIVE
        row.removed_at = None
        row.hdfs_path = hdfs_path
    session.flush()
    return row


@dataclass
class ManifestBuild:
    manifest: FileManifest
    reused_manifest: bool = False
    chunks_total: int = 0
    chunks_new: int = 0
    chunks_reused: int = 0
    bytes_written: int = 0
    chunking_ms: float = 0.0
    new_chunk_hashes: list[str] = field(default_factory=list)


def build_manifest(
    ctx: AppContext,
    session: Session,
    source: Iterable[bytes],
    file_sha256: str,
    file_size: int,
    chunk_size: int,
    *,
    replication: int | None = None,
) -> ManifestBuild:
    """Chunk a stream, store only unseen chunks, and record the ordered recipe.

    Rows are added only after all HDFS writes finish, so the SQLite write lock is held briefly.
    The caller must hold ``ctx.content_lock`` so garbage collection cannot remove a chunk being reused.
    """
    existing = find_active_manifest(session, file_sha256, chunk_size)
    if existing is not None:
        return ManifestBuild(manifest=existing, reused_manifest=True, chunks_total=existing.chunk_count)

    whole = hashlib.sha256()
    plan: list[tuple[str, int, int]] = []  # (sha256, length, offset)
    seen: dict[str, Chunk | None] = {}  # chunks settled in this run; None means written but no row yet
    build = ManifestBuild(manifest=None)  # type: ignore[arg-type]
    offset = 0
    started = time.perf_counter()

    for block in fixed_size_chunks(source, chunk_size):
        digest = sha256_bytes(block)
        whole.update(block)
        plan.append((digest, len(block), offset))
        offset += len(block)
        build.chunks_total += 1

        if digest in seen:
            build.chunks_reused += 1
            continue
        path = ctx.paths.chunk_path(digest)
        row = session.scalar(select(Chunk).where(Chunk.sha256 == digest))
        if row is not None and row.state == ContentState.ACTIVE:
            status = ctx.hdfs.status(row.hdfs_path)
            if status is not None and status.length == len(block):
                seen[digest] = row
                build.chunks_reused += 1
                continue
            # The metadata says it exists but HDFS disagrees. These bytes are at hand, so repair it.
            ctx.hdfs.delete(row.hdfs_path)
        written = put_content(ctx, path, lambda b=block: b, digest, len(block), verify=False, replication=replication)
        seen[digest] = row
        if written:
            build.chunks_new += 1
            build.bytes_written += len(block)
            build.new_chunk_hashes.append(digest)
        else:
            build.chunks_reused += 1

    if whole.hexdigest() != file_sha256 or offset != file_size:
        raise IntegrityFailure(
            f"Source changed while chunking: expected {file_sha256} ({file_size} bytes), "
            f"read {whole.hexdigest()} ({offset} bytes)."
        )

    manifest_path = ctx.paths.manifest_path(file_sha256, chunk_size)
    document = json.dumps(
        {
            "version": 1,
            "algorithm": "sha256",
            "file_sha256": file_sha256,
            "file_size": file_size,
            "chunk_size": chunk_size,
```

```
            "chunks": [{"sha256": sha, "length": length, "offset": off} for sha, length, off in plan],
        },
        separators=(",", ":"),
    ).encode()
    put_content(
        ctx, manifest_path, lambda: document, sha256_bytes(document), len(document), verify=False, replication=replication
    )
    build.bytes_written += len(document)

    # All bytes are durable in HDFS, so metadata can now be written in one short transaction.
    rows: dict[str, Chunk] = {}
    for digest, length, _ in plan:
        if digest in rows:
            continue
        row = seen.get(digest) or session.scalar(select(Chunk).where(Chunk.sha256 == digest))
        if row is None:
            row = Chunk(sha256=digest, size_bytes=length, hdfs_path=ctx.paths.chunk_path(digest))
            session.add(row)
        else:
            row.state = ContentState.ACTIVE
            row.removed_at = None
        rows[digest] = row

    manifest = session.scalar(
        select(FileManifest).where(FileManifest.file_sha256 == file_sha256, FileManifest.chunk_size == chunk_size)
    )
    if manifest is None:
        manifest = FileManifest(file_sha256=file_sha256, chunk_size=chunk_size, file_size=file_size, chunk_count=0,
hdfs_path=manifest_path)
        session.add(manifest)
    manifest.file_size = file_size
    manifest.chunk_count = len(plan)
    manifest.manifest_bytes = len(document)
    manifest.hdfs_path = manifest_path
    manifest.state = ContentState.ACTIVE
    manifest.removed_at = None
    session.flush()  # assigns chunk and manifest primary keys
    manifest.entries = [
        FileChunk(chunk_id=rows[digest].id, ordinal=index, offset_bytes=off)
        for index, (digest, _, off) in enumerate(plan)
    ]
    session.flush()
    build.manifest = manifest
    build.chunking_ms = (time.perf_counter() - started) * 1000
    return build


def repair_manifest(
    ctx: AppContext, session: Session, manifest: FileManifest, local_path: Path, replication: int | None
) -> int:
    """Rewrite any missing or truncated chunks of a manifest from a byte-identical local copy."""
    written = 0
    chunks = manifest_chunks(session, manifest.id)
    for chunk, block in zip(chunks, fixed_size_chunks(read_file_blocks(local_path), manifest.chunk_size), strict=True):
        status = ctx.hdfs.status(chunk.hdfs_path)
        if status is not None and status.length == chunk.size_bytes:
            continue
        if status is not None:
            ctx.hdfs.delete(chunk.hdfs_path)
        put_content(ctx, chunk.hdfs_path, lambda b=block: b, chunk.sha256, len(block), verify=False, replication=replication)
        written += len(block)
    return written


def manifest_chunks(session: Session, manifest_id: str) -> list[Chunk]:
    return list(
        session.scalars(
            select(Chunk)
            .join(FileChunk, FileChunk.chunk_id == Chunk.id)
            .where(FileChunk.manifest_id == manifest_id)
            .order_by(FileChunk.ordinal)
        )
    )


def iter_file_bytes(ctx: AppContext, session: Session, file: LogicalFile) -> Iterator[bytes]:
    """Yield a logical file's original bytes. In chunk mode, every chunk is checked as it is read."""
    if file.storage_mode == StorageMode.RAW:
        yield from ctx.hdfs.open(file.raw_path)
    elif file.storage_mode == StorageMode.FILE:
        yield from ctx.hdfs.open(file.content_object.hdfs_path)
    elif file.storage_mode == StorageMode.CHUNK:
        for chunk in manifest_chunks(session, file.manifest_id):
            data = b"".join(ctx.hdfs.open(chunk.hdfs_path))
            actual = sha256_bytes(data)
            if actual != chunk.sha256:
                raise IntegrityFailure(f"Chunk {chunk.sha256} is corrupt in HDFS (read {actual}).")
            yield data
    else:
        raise IntegrityFailure(f"File {file.id} has no stored content (mode {file.storage_mode}).")
```

backend/app/services/context.py 85 lines

```
"""Everything a service needs, passed explicitly so experiments and tests can use isolated instances."""

from __future__ import annotations

import threading
from dataclasses import dataclass, field
from typing import Any

from sqlalchemy.orm import Session, sessionmaker

from app.core.config import Settings
from app.hdfs.base import HdfsClient


@dataclass(frozen=True)
class StoragePaths:
    """The internal HDFS namespace. Paths come from hashes and generated IDs, never from user input."""

    root: str

    @property
    def objects(self) -> str:
        return f"{self.root}/objects"

    @property
    def chunks(self) -> str:
        return f"{self.root}/chunks"

    @property
    def manifests(self) -> str:
        return f"{self.root}/manifests"

    @property
    def staging(self) -> str:
        return f"{self.root}/staging"

    @property
    def recovery(self) -> str:
        return f"{self.root}/recovery"

    @property
    def raw(self) -> str:
        return f"{self.root}/raw"

    def all_dirs(self) -> list[str]:
        return [self.objects, self.chunks, self.manifests, self.staging, self.recovery, self.raw]

    def object_path(self, sha256: str) -> str:
        return f"{self.objects}/{sha256[:2]}/{sha256[2:4]}/{sha256}"

    def chunk_path(self, sha256: str) -> str:
        return f"{self.chunks}/{sha256[:2]}/{sha256[2:4]}/{sha256}"

    def manifest_path(self, sha256: str, chunk_size: int) -> str:
        return f"{self.manifests}/{sha256[:2]}/{sha256}.{chunk_size}.json"

    def staging_path(self, token: str) -> str:
        return f"{self.staging}/{token}"

    def raw_path(self, file_id: str) -> str:
        return f"{self.raw}/{file_id[:2]}/{file_id}"

    def recovery_raw_path(self, file_id: str) -> str:
        return f"{self.recovery}/deleted/{file_id}"

    def recovery_orphan_path(self, name: str) -> str:
        return f"{self.recovery}/orphans/{name}"


@dataclass
class AppContext:
    settings: Settings
    session_factory: sessionmaker[Session]
    hdfs: HdfsClient
    paths: StoragePaths
    monitor: Any = None  # ClusterMonitor; typed loosely to avoid a circular import
    jobs: Any = None  # JobManager
    # Guards content-addressed state so garbage collection can never race a new reference to the same
    # content. This assumes one API process, a limitation documented in the report.
    content_lock: threading.RLock = field(default_factory=threading.RLock)

    def ensure_namespace(self) -> None:
        for path in self.paths.all_dirs():
            self.hdfs.mkdirs(path)
```

backend/app/services/datasets.py 376 lines

```
"""Authentic dataset handling with machine-readable provenance.

Source files come only from Project Gutenberg, downloaded as published. Two clearly labelled kinds of
workload may be derived from them, and are never presented as separately published originals:

  * exact copies: the same local file ingested under another logical name (label ``workload_copy``)
```

```
  * transformed files: a documented byte-level edit of one original (kind ``transformed``)

No text is invented. The only bytes a transformation adds are a single newline or one replaced byte.
"""

from __future__ import annotations

import json
import time
from dataclasses import dataclass
from datetime import UTC, datetime
from pathlib import Path
from urllib.parse import urlsplit

import httpx
from sqlalchemy import select
from sqlalchemy.orm import Session

from app.core.db import session_scope
from app.core.errors import AppError, NotFoundError, ValidationFailure
from app.models.entities import DatasetSource
from app.services import audit, ingest
from app.services.context import AppContext
from app.services.hashing import sha256_file

ORGANIZATION = "Project Gutenberg (Project Gutenberg Literary Archive Foundation)"
LICENSE_NOTE = (
    "Public domain in the USA, distributed under the Project Gutenberg License "
    "(https://www.gutenberg.org/policy/license.html)."
)
ALLOWED_HOSTS = {"gutenberg.pglaf.org"}
# Project Gutenberg's website is for human readers: automated access to it is not permitted, and its policy
# directs automated tools to mirrors (https://www.gutenberg.org/policy/robot_access.html). This tool therefore
# downloads from the mirror run by the Project Gutenberg Literary Archive Foundation, with a delay between requests.
MIRROR_BASE = "https://gutenberg.pglaf.org/cache/epub"
USER_AGENT = "hdfs-optimizer-academic-project/1.0 (dataset download for storage evaluation)"
POLITE_DELAY_SECONDS = 2.0


@dataclass(frozen=True)
class CatalogEntry:
    ebook_id: int
    title: str
    author: str
    formats: tuple[str, ...] = ("txt",)


# The first three are the documents suggested by the project brief.
CATALOG: tuple[CatalogEntry, ...] = (
    CatalogEntry(1342, "Pride and Prejudice", "Jane Austen", ("txt", "html", "epub")),
    CatalogEntry(11, "Alice's Adventures in Wonderland", "Lewis Carroll", ("txt", "html", "epub")),
    CatalogEntry(1661, "The Adventures of Sherlock Holmes", "Arthur Conan Doyle", ("txt", "html", "epub")),
    CatalogEntry(84, "Frankenstein; Or, The Modern Prometheus", "Mary Wollstonecraft Shelley"),
    CatalogEntry(345, "Dracula", "Bram Stoker"),
    CatalogEntry(98, "A Tale of Two Cities", "Charles Dickens"),
    CatalogEntry(2701, "Moby Dick; Or, The Whale", "Herman Melville"),
    CatalogEntry(100, "The Complete Works of William Shakespeare", "William Shakespeare"),
    CatalogEntry(2600, "War and Peace", "Leo Tolstoy"),
)

_MIRROR_SUFFIX = {"txt": ".txt", "html": "-images.html", "epub": "-images-3.epub"}
_FORMAT_EXTENSION = {"txt": "txt", "html": "html", "epub": "epub"}

TRANSFORMATIONS = {
    "truncate_90": "Kept the first 90% of the original bytes and removed the rest.",
    "prefix_newline": "Inserted one newline byte (0x0A) before the first original byte, shifting every later byte by one.",
    "replace_mid_byte": "Replaced the single byte at the file's midpoint with a different byte, keeping the length unchanged.",
}


def source_page(ebook_id: int) -> str:
    return f"https://www.gutenberg.org/ebooks/{ebook_id}"


def download_url(ebook_id: int, file_format: str) -> str:
    return f"{MIRROR_BASE}/{ebook_id}/pg{ebook_id}{_MIRROR_SUFFIX[file_format]}"


def _read_manifest(path: Path) -> dict:
    if path.exists():
        return json.loads(path.read_text())
    return {
        "description": "Provenance of every dataset file used by HDFS Optimizer. Generated by the download "
        "tool; do not edit by hand.",
        "files": [],
    }


def _write_manifest(path: Path, manifest: dict) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    manifest["number_of_files"] = len(manifest["files"])
    manifest["number_of_original_files"] = sum(1 for f in manifest["files"] if f["kind"] == "original")
    manifest["total_size_bytes"] = sum(f["size_bytes"] for f in manifest["files"])
```

```
    path.write_text(json.dumps(manifest, indent=2) + "\n")


def _header_title(path: Path) -> str | None:
    """Read the title Project Gutenberg prints in a plain-text file, to confirm the right book arrived."""
    if path.suffix != ".txt":
        return None
    with open(path, encoding="utf-8", errors="replace") as handle:
        for line in handle.read(4096).splitlines():
            if line.lstrip("").startswith("The Project Gutenberg eBook of "):
                return line.lstrip("").removeprefix("The Project Gutenberg eBook of ").strip()
            if line.lstrip("").startswith("Title: "):
                return line.lstrip("").removeprefix("Title: ").strip()
    return None


def download_catalog(
    dataset_dir: Path,
    provenance_path: Path,
    *,
    ebook_ids: list[int] | None = None,
    log=print,
) -> dict:
    """Download originals that are not already present. Existing records and bytes are never rewritten."""
    originals = dataset_dir / "originals"
    originals.mkdir(parents=True, exist_ok=True)
    manifest = _read_manifest(provenance_path)
    known = {(f["ebook_id"], f["file_format"]): f for f in manifest["files"] if f["kind"] == "original"}
    selected = [e for e in CATALOG if ebook_ids is None or e.ebook_id in ebook_ids]
    summary = {"downloaded": 0, "already_present": 0, "failed": []}

    with httpx.Client(follow_redirects=True, timeout=120, headers={"User-Agent": USER_AGENT}) as client:
        for entry in selected:
            for file_format in entry.formats:
                filename = f"pg{entry.ebook_id}.{_FORMAT_EXTENSION[file_format]}"
                target = originals / filename
                record = known.get((entry.ebook_id, file_format))
                if record and target.exists() and sha256_file(target)[0] == record["sha256"]:
                    summary["already_present"] += 1
                    continue
                url = download_url(entry.ebook_id, file_format)
                try:
                    response = client.get(url)
                    response.raise_for_status()
                    final_url = str(response.url)
                    host = urlsplit(final_url).hostname
                    if host not in ALLOWED_HOSTS:
                        raise AppError(f"Redirected to an unexpected host: {host}")
                except (httpx.HTTPError, AppError) as exc:
                    summary["failed"].append({"url": url, "error": str(exc)})
                    log(f"FAILED {url}: {exc}")
                    continue
                target.write_bytes(response.content)
                digest, size = sha256_file(target)
                new_record = {
                    "title": entry.title,
                    "author": entry.author,
                    "ebook_id": entry.ebook_id,
                    "source_organization": ORGANIZATION,
                    "source_url": source_page(entry.ebook_id),
                    "requested_url": url,
                    "actual_download_url": final_url,
                    "download_date": datetime.now(UTC).isoformat(),
                    "file_format": file_format,
                    "file_size": size,
                    "size_bytes": size,
                    "sha256": digest,
                    "kind": "original",
                    "transformations": "None. Stored exactly as served by the publisher.",
                    "header_title_in_file": _header_title(target),
                    "local_path": f"originals/{filename}",
                    "license_note": LICENSE_NOTE,
                }
                manifest["files"] = [
                    f for f in manifest["files"]
                    if not (f.get("ebook_id") == entry.ebook_id and f.get("file_format") == file_format and f["kind"] == "original")
                ] + [new_record]
                _write_manifest(provenance_path, manifest)
                summary["downloaded"] += 1
                log(f"downloaded {filename}: {size} bytes, sha256 {digest}")
                time.sleep(POLITE_DELAY_SECONDS)
    _write_manifest(provenance_path, manifest)
    return summary


def build_transformed_workload(dataset_dir: Path, provenance_path: Path, *, ebook_id: int = 1342) -> list[dict]:
    """Create documented partial-overlap files from one original plain-text book. Repeatable."""
    manifest = _read_manifest(provenance_path)
    source = next(
        (f for f in manifest["files"] if f["kind"] == "original" and f["ebook_id"] == ebook_id and f["file_format"] == "txt"),
        None,
    )
    if source is None:
```

```
        raise ValidationFailure(f"Download the plain-text original of ebook {ebook_id} first.")
    original = (dataset_dir / source["local_path"]).read_bytes()
    workload = dataset_dir / "workload"
    workload.mkdir(parents=True, exist_ok=True)
    midpoint = len(original) // 2
    replacement = b"X" if original[midpoint:midpoint + 1] != b"X" else b"Y"
    variants = {
        "truncate_90": original[: int(len(original) * 0.9)],
        "prefix_newline": b"\n" + original,
        "replace_mid_byte": original[:midpoint] + replacement + original[midpoint + 1 :],
    }
    created = []
    manifest["files"] = [
        f for f in manifest["files"]
        if not (f["kind"] == "transformed" and f.get("derived_from_sha256") == source["sha256"])
    ]
    for name, data in variants.items():
        filename = f"pg{ebook_id}.{name}.txt"
        path = workload / filename
        path.write_bytes(data)
        digest, size = sha256_file(path)
        record = {
            "title": f"{source['title']} (test workload: {name})",
            "author": source["author"],
            "ebook_id": ebook_id,
            "source_organization": "Derived locally by this project; not published by the source organization.",
            "source_url": source["source_url"],
            "requested_url": None,
            "actual_download_url": None,
            "download_date": None,
            "file_format": "txt",
            "file_size": size,
            "size_bytes": size,
            "sha256": digest,
            "kind": "transformed",
            "derived_from_sha256": source["sha256"],
            "transformations": TRANSFORMATIONS[name],
            "local_path": f"workload/{filename}",
            "license_note": LICENSE_NOTE,
        }
        manifest["files"].append(record)
        created.append(record)
    _write_manifest(provenance_path, manifest)
    return created


# -- database integration ----------------------------------------------------------------------


def sync_provenance(ctx: AppContext, session: Session) -> int:
    """Mirror the provenance file into the database. Returns the number of new rows."""
    manifest = _read_manifest(ctx.settings.provenance_path)
    existing = {row.sha256: row for row in session.scalars(select(DatasetSource))}
    added = 0
    for record in manifest["files"]:
        if record["sha256"] in existing:
            continue
        row = DatasetSource(
            title=record["title"],
            author=record.get("author"),
            ebook_id=record.get("ebook_id"),
            source_url=record.get("source_url"),
            download_url=record.get("actual_download_url"),
            organization=record.get("source_organization"),
            downloaded_at=_parse_date(record.get("download_date")),
            file_format=record["file_format"],
            size_bytes=record["size_bytes"],
            sha256=record["sha256"],
            kind=record["kind"],
            transformation=record.get("transformations") if record["kind"] != "original" else None,
            local_path=record["local_path"],
            license_note=record.get("license_note"),
        )
        session.add(row)
        existing[row.sha256] = row
        added += 1
    session.flush()
    for record in manifest["files"]:
        parent = existing.get(record.get("derived_from_sha256") or "")
        if parent is not None:
            existing[record["sha256"]].derived_from_id = parent.id
    session.flush()
    return added


def _parse_date(value: str | None) -> datetime | None:
    return datetime.fromisoformat(value).replace(tzinfo=None) if value else None


def source_dict(row: DatasetSource, *, local_present: bool | None = None) -> dict:
    return {
        "id": row.id,
        "title": row.title,
```

```
        "author": row.author,
        "ebook_id": row.ebook_id,
        "source_url": row.source_url,
        "download_url": row.download_url,
        "organization": row.organization,
        "downloaded_at": row.downloaded_at,
        "file_format": row.file_format,
        "size_bytes": row.size_bytes,
        "sha256": row.sha256,
        "kind": row.kind,
        "derived_from_id": row.derived_from_id,
        "transformation": row.transformation,
        "license_note": row.license_note,
        "local_present": local_present,
    }


def run_import(ctx: AppContext, job_id: str, params: dict, reporter) -> dict:
    """Job: download from the publisher, derive workload files, record provenance."""
    ebook_ids = params.get("ebook_ids")
    reporter.progress(0, 2, "Downloading from Project Gutenberg")
    summary = download_catalog(ctx.settings.dataset_dir, ctx.settings.provenance_path, ebook_ids=ebook_ids, log=lambda _: None)
    reporter.progress(1, 2, "Building documented workload files")
    derived = []
    if (ebook_ids is None or 1342 in ebook_ids) and not summary["failed"]:
        derived = build_transformed_workload(ctx.settings.dataset_dir, ctx.settings.provenance_path)
    with session_scope(ctx.session_factory) as session:
        added = sync_provenance(ctx, session)
        audit.record(
            session,
            "dataset.imported",
            f"Downloaded {summary['downloaded']} files ({summary['already_present']} already present, "
            f"{len(summary['failed'])} failed).",
            severity="warning" if summary["failed"] else "info",
            details={"failed": summary["failed"]},
        )
    reporter.progress(2, 2, "Done")
    return {
        **summary,
        "derived_files": len(derived),
        "rows_added": added,
        "errors": [f"{f['url']}: {f['error']}" for f in summary["failed"]],
    }


def run_ingest(ctx: AppContext, job_id: str, params: dict, reporter) -> dict:
    """Job: load dataset files into HDFS through the normal upload pipeline, labelling every copy."""
    source_ids: list[str] = params["source_ids"]
    copies: int = params.get("copies", 0)
    folder = params.get("folder", "/datasets")
    results, errors = [], []
    total = len(source_ids) * (copies + 1)
    done = 0
    reporter.progress(0, total, "Ingesting dataset files")
    for source_id in source_ids:
        with session_scope(ctx.session_factory) as session:
            source = session.get(DatasetSource, source_id)
            if source is None:
                raise NotFoundError(f"No dataset source {source_id}.")
            local, expected, kind = ctx.settings.dataset_dir / source.local_path, source.sha256, source.kind
        if not local.exists():
            errors.append(f"{local.name}: not present locally. Import the dataset first.")
            continue
        hash_started = time.perf_counter()
        digest = sha256_file(local)
        hash_ms = (time.perf_counter() - hash_started) * 1000
        if digest[0] != expected:
            errors.append(f"{local.name}: local bytes no longer match the recorded SHA-256; refusing to ingest.")
            continue
        for copy_index in range(copies + 1):
            if copy_index == 0:
                name, label = local.name, kind if kind != "original" else "dataset_original"
            else:
                name, label = f"{local.stem}__workload-copy-{copy_index}{local.suffix}", "workload_copy"
            outcome = ingest.ingest_local_file(
                ctx,
                local,
                name,
                folder=folder,
                mode=params.get("mode"),
                chunk_size=params.get("chunk_size"),
                provenance_label=label,
                dataset_source_id=source_id,
                known_hash=digest,
                known_hash_ms=hash_ms if copy_index == 0 else None,  # copies reuse the same hash
            )
            results.append({"file_id": outcome.file.id, "name": outcome.file.name, "outcome": outcome.outcome, "label": label})
            done += 1
            reporter.progress(done, total, f"Ingested {done} of {total}")
    return {"ingested": len(results), "files": results, "errors": errors}
```

backend/app/services/experiments.py 538 lines

```
"""Reproducible experiments against the real cluster.

Each run uses its own HDFS directory and its own temporary metadata database, so it can never touch
managed user data. Every number returned is measured during the run.
"""

from __future__ import annotations

import csv
import hashlib
import io
import shutil
import tempfile
import time
import uuid
from collections.abc import Callable
from pathlib import Path

from sqlalchemy import select

from app.core.db import Base, make_engine, make_session_factory, session_scope, utcnow
from app.core.errors import ValidationFailure
from app.models.entities import ExperimentRun
from app.services import (
    audit,
    datasets,
    ingest,
    integrity,
    lifecycle,
    optimizer,
    replication,
)
from app.services.analytics import dedup_ratio, savings_percent, storage_statistics
from app.services.context import AppContext, StoragePaths
from app.services.jobs import JobManager

EXPERIMENT_ROOT = "/hdfs-optimizer-experiments"
KIB = 1024
MIB = 1024 * 1024
DEFAULT_CHUNK_SIZES = (16 * KIB, 64 * KIB, 256 * KIB, 1 * MIB, 4 * MIB)
JOB_HANDLERS = {
    "analyze": optimizer.run_analysis,
    "optimize": optimizer.run_optimization,
    "replication": replication.run_replication_change,
}


class IsolatedRun:
    """A throwaway copy of the application state, backed by the real HDFS cluster."""

    def __init__(self, ctx: AppContext):
        self.parent = ctx
        self.run_id = uuid.uuid4().hex
        self.root = f"{EXPERIMENT_ROOT}/{self.run_id}"
        self.workdir = Path(tempfile.mkdtemp(prefix="hdfsopt-exp-"))

    def __enter__(self) -> AppContext:
        engine = make_engine(self.workdir / "metadata.db")
        Base.metadata.create_all(engine)
        self.engine = engine
        self.ctx = AppContext(
            settings=self.parent.settings,
            session_factory=make_session_factory(engine),
            hdfs=self.parent.hdfs,
            paths=StoragePaths(root=self.root),
            monitor=self.parent.monitor,
        )
        self.ctx.jobs = JobManager(self.ctx, JOB_HANDLERS, synchronous=True)
        self.ctx.ensure_namespace()
        return self.ctx

    def __exit__(self, *_exc) -> None:
        if not self.root.startswith(f"{EXPERIMENT_ROOT}/") or len(self.root) <= len(EXPERIMENT_ROOT) + 8:
            raise RuntimeError(f"Refusing to clean unexpected path: {self.root}")
        try:
            self.parent.hdfs.delete(self.root, recursive=True)
        finally:
            self.engine.dispose()
            shutil.rmtree(self.workdir, ignore_errors=True)


def snapshot(ctx: AppContext) -> dict:
    """Metadata-derived and NameNode-measured values at this moment."""
    with session_scope(ctx.session_factory) as session:
        stats = storage_statistics(session)
    summary = ctx.hdfs.content_summary(ctx.paths.root)
    return {
        "logical_files": stats["logical_files"],
        "unique_files": stats["unique_files"],
        "duplicate_files": stats["duplicate_files"],
        "logical_bytes": stats["logical_bytes"],
        "stored_content_bytes": stats["stored_content_bytes"],
```

```
        "bytes_avoided": stats["bytes_avoided"],
        "savings_percent": stats["savings_percent"],
        "dedup_ratio": stats["dedup_ratio"],
        "chunk_count": stats["chunk_count"],
        "chunk_references": stats["chunk_references"],
        "duplicate_chunk_references": stats["duplicate_chunk_references"],
        "manifest_count": stats["manifest_count"],
        "manifest_bytes": stats["manifest_bytes"],
        "hdfs_length_bytes": summary.length,
        "hdfs_space_consumed_bytes": summary.space_consumed,
        "hdfs_file_count": summary.file_count,
        "hdfs_directory_count": summary.directory_count,
    }


def stable_disk(probe: Callable[[], int], *, interval: float = 3.0, needed: int = 3, timeout: float = 120.0) -> int:
    """Read disk usage once it stops changing.

    DataNodes delete blocks asynchronously after the NameNode's delete returns, so a single reading can
    still include blocks from earlier work. Raises rather than returning a value that never stabilised.
    """
    deadline = time.monotonic() + timeout
    previous, repeats = probe(), 1
    while repeats < needed:
        if time.monotonic() > deadline:
            raise RuntimeError(f"DataNode disk usage did not stabilise within {timeout} s.")
        time.sleep(interval)
        current = probe()
        repeats = repeats + 1 if current == previous else 1
        previous = current
    return previous


def namespace_cost(ctx: AppContext) -> dict:
    """Objects this run adds to the NameNode. Read from uncached sources for this run's directory only;
    NameNode JMX values such as FilesTotal are cached for about ten seconds and include the whole cluster."""
    summary = ctx.hdfs.content_summary(ctx.paths.root)
    fsck = ctx.monitor.fsck(ctx.paths.root)
    blocks = fsck["total_blocks"] or 0
    return {
        "namespace_files": summary.file_count,
        "namespace_directories": summary.directory_count,
        "namespace_blocks": blocks,
        "namespace_objects": summary.file_count + summary.directory_count + blocks,
    }


def _load_sources(ctx: AppContext, kinds: tuple[str, ...] = ("original",), formats: tuple[str, ...] = ("txt",)) -> list[dict]:
    manifest = datasets._read_manifest(ctx.settings.provenance_path)
    rows = [f for f in manifest["files"] if f["kind"] in kinds and f["file_format"] in formats]
    usable = []
    for row in rows:
        path = ctx.settings.dataset_dir / row["local_path"]
        if path.exists():
            usable.append({**row, "path": path})
    if not usable:
        raise ValidationFailure("No authentic dataset files are available. Import the dataset first.")
    usable.sort(key=lambda r: r["size_bytes"])
    return usable


def _verify_all(ctx: AppContext) -> dict:
    with session_scope(ctx.session_factory) as session:
        rows = session.execute(select(integrity.LogicalFile.id).where(integrity.LogicalFile.status == "active")).scalars().all()
    started = time.perf_counter()
    passed = 0
    total_bytes = 0
    for file_id in rows:
        check = integrity.verify_file(ctx, file_id)
        passed += int(check.ok)
        total_bytes += check.bytes_read
    return {
        "files_checked": len(rows),
        "files_passed": passed,
        "bytes_read": total_bytes,
        "duration_s": time.perf_counter() - started,
    }


def _throughput(total_bytes: int, seconds: float) -> float | None:
    return total_bytes / MIB / seconds if seconds > 0 else None


def _ingest_timed(ctx: AppContext, items: list[tuple[Path, str, str]], mode: str, chunk_size: int | None = None) -> dict:
    total_bytes, hash_ms = 0, 0.0
    started = time.perf_counter()
    for path, name, label in items:
        outcome = ingest.ingest_local_file(ctx, path, name, folder="/experiment", mode=mode, chunk_size=chunk_size,
provenance_label=label)
        total_bytes += outcome.file.size_bytes
        hash_ms += outcome.hash_ms
    seconds = time.perf_counter() - started
    return {
```

```
        "upload_duration_s": seconds,
        "hash_duration_s": hash_ms / 1000,
        "upload_bytes": total_bytes,
        "upload_throughput_mib_s": _throughput(total_bytes, seconds),
    }


def _workload_copies(sources: list[dict], copies: int) -> list[tuple[Path, str, str]]:
    items = []
    for row in sources:
        path = row["path"]
        items.append((path, path.name, "dataset_original"))
        for index in range(1, copies + 1):
            items.append((path, f"{path.stem}__workload-copy-{index}{path.suffix}", "workload_copy"))
    return items


# -- scenarios ---------------------------------------------------------------------------------


def scenario_a_unique(ctx: AppContext, params: dict) -> dict:
    """A: only distinct originals. Deduplication has nothing to find; savings should be about zero."""
    sources = _load_sources(ctx)
    rows = []
    for mode in ("file", "chunk"):
        with IsolatedRun(ctx) as run:
            timing = _ingest_timed(run, _workload_copies(sources, 0), mode, MIB)
            rows.append({"mode": mode, **timing, **snapshot(run)})
    return {"description": "Distinct authentic files only.", "sources": len(sources), "rows": rows}


def scenario_b_exact_duplicates(ctx: AppContext, params: dict) -> dict:
    """B: originals plus N byte-identical workload copies, with and without deduplication."""
    sources = _load_sources(ctx)
    rows = []
    for copies in params.get("copies", [1, 3, 5]):
        for mode in ("raw", "file"):
            with IsolatedRun(ctx) as run:
                timing = _ingest_timed(run, _workload_copies(sources, copies), mode)
                rows.append({"copies": copies, "mode": mode, **timing, **snapshot(run)})
    return {"description": "Each original plus N exact workload copies.", "sources": len(sources), "rows": rows}


def scenario_c_many_names(ctx: AppContext, params: dict) -> dict:
    """C: one authentic file under many logical names must yield exactly one stored object."""
    source = _load_sources(ctx)[0]
    names = params.get("names", 20)
    items = [(source["path"], f"name-{i:03d}{source['path'].suffix}", "workload_copy") for i in range(names)]
    with IsolatedRun(ctx) as run:
        timing = _ingest_timed(run, items, "file")
        result = {**timing, **snapshot(run), "verification": _verify_all(run)}
    return {
        "description": "One authentic file referenced under many logical names.",
        "source": source["title"],
        "source_bytes": source["size_bytes"],
        "logical_names": names,
        "rows": [result],
    }


def scenario_d_chunk_sizes(ctx: AppContext, params: dict, physical_probe: Callable[[], int] | None = None) -> dict:
    """D: chunk-level deduplication on originals plus documented partial-overlap files, per chunk size."""
    originals = _load_sources(ctx)
    transformed = _load_sources(ctx, kinds=("transformed",))
    items = [(r["path"], r["path"].name, "dataset_original") for r in originals]
    items += [(r["path"], r["path"].name, "transformed") for r in transformed]

    def measure(run: AppContext, mode: str, chunk_size: int | None) -> dict:
        disk_before = stable_disk(physical_probe) if physical_probe else None
        timing = _ingest_timed(run, items, mode, chunk_size)
        disk_after = stable_disk(physical_probe) if physical_probe else None
        return {
            "mode": mode,
            "chunk_size": chunk_size,
            **timing,
            **snapshot(run),
            **namespace_cost(run),
            "disk_used_delta": None if physical_probe is None else disk_after - disk_before,
        }

    rows = []
    with IsolatedRun(ctx) as run:
        rows.append(measure(run, "file", None))
    for chunk_size in params.get("chunk_sizes", DEFAULT_CHUNK_SIZES):
        with IsolatedRun(ctx) as run:
            rows.append(measure(run, "chunk", chunk_size))
    return {
        "description": "Originals plus truncated, one-byte-shifted and one-byte-edited variants of one book.",
        "transformations": datasets.TRANSFORMATIONS,
        "rows": rows,
    }
```

```python
def scenario_d2_boundary_shift(ctx: AppContext, params: dict) -> dict:
    """D2: isolates the known weakness of fixed-size chunking, one variant at a time."""
    original = next(r for r in _load_sources(ctx) if r["ebook_id"] == 1342)
    variants = _load_sources(ctx, kinds=("transformed",))
    chunk_size = params.get("chunk_size", 64 * KIB)
    rows = []
    for variant in variants:
        with IsolatedRun(ctx) as run:
            ingest.ingest_local_file(run, original["path"], original["path"].name, mode="chunk", chunk_size=chunk_size)
            outcome = ingest.ingest_local_file(run, variant["path"], variant["path"].name, mode="chunk", chunk_size=chunk_size)
            rows.append(
                {
                    "variant": variant["local_path"].split("/")[-1],
                    "transformation": variant["transformations"],
                    "chunk_size": chunk_size,
                    "chunks_total": outcome.chunks_total,
                    "chunks_reused": outcome.chunks_reused,
                    "chunks_new": outcome.chunks_new,
                    "reuse_percent": outcome.chunks_reused / outcome.chunks_total * 100 if outcome.chunks_total else None,
                    "bytes_written_including_manifest": outcome.bytes_written,
                    "variant_bytes": outcome.file.size_bytes,
                }
            )
    return {"description": "How much of each variant reuses the original's chunks.", "rows": rows}


def scenario_e_roundtrip(ctx: AppContext, params: dict) -> dict:
    """E: download every file in each mode and compare SHA-256 with the source on disk."""
    sources = _load_sources(ctx, formats=("txt", "html", "epub"))
    rows = []
    for mode in ("raw", "file", "chunk"):
        with IsolatedRun(ctx) as run:
            ids = {}
            for row in sources:
                outcome = ingest.ingest_local_file(run, row["path"], row["path"].name, mode=mode, chunk_size=256 * KIB)
                ids[outcome.file.id] = row
            matched, bytes_read = 0, 0
            started = time.perf_counter()
            for file_id, row in ids.items():
                digest = hashlib.sha256()
                for block in integrity.stream_verified(run, file_id):
                    digest.update(block)
                    bytes_read += len(block)
                matched += int(digest.hexdigest() == row["sha256"])
            seconds = time.perf_counter() - started
            rows.append(
                {
                    "mode": mode,
                    "files": len(ids),
                    "files_matching_source_sha256": matched,
                    "bytes_read": bytes_read,
                    "restore_duration_s": seconds,
                    "restore_throughput_mib_s": _throughput(bytes_read, seconds),
                }
            )
    return {"description": "Byte-for-byte restore compared with the publisher's bytes on disk.", "rows": rows}


def scenario_f_shared_deletion(ctx: AppContext, params: dict) -> dict:
    """F: deleting one logical file must leave shared content readable for the others."""
    source = _load_sources(ctx)[0]
    rows = []
    for mode in ("file", "chunk"):
        with IsolatedRun(ctx) as run:
            first = ingest.ingest_local_file(run, source["path"], "a.txt", mode=mode, chunk_size=64 * KIB)
            second = ingest.ingest_local_file(run, source["path"], "b.txt", mode=mode, chunk_size=64 * KIB)
            before = snapshot(run)
            lifecycle.soft_delete(run, first.file.id)
            lifecycle.purge(run, first.file.id)
            after_first = snapshot(run)
            survivor_ok = integrity.verify_file(run, second.file.id).ok
            lifecycle.soft_delete(run, second.file.id)
            restored = lifecycle.restore(run, second.file.id)
            restored_ok = integrity.verify_file(run, restored.id).ok
            lifecycle.soft_delete(run, second.file.id)
            lifecycle.purge(run, second.file.id)
            after_last = snapshot(run)
            rows.append(
                {
                    "mode": mode,
                    "stored_bytes_two_references": before["stored_content_bytes"],
                    "stored_bytes_after_purging_one": after_first["stored_content_bytes"],
                    "survivor_passes_sha256": survivor_ok,
                    "soft_delete_then_restore_passes_sha256": restored_ok,
                    "stored_bytes_after_purging_last": after_last["stored_content_bytes"],
                    "hdfs_files_after_purging_last": after_last["hdfs_file_count"],
                }
            )
    return {"description": "Reference-aware deletion of shared content.", "source": source["title"], "rows": rows}


def scenario_g_restore_after_optimize(ctx: AppContext, params: dict) -> dict:
    """G: upload without deduplication, optimize, then verify every restored file."""
```

```
    return scenario_i_before_after(ctx, {**params, "copies": params.get("copies", 2), "scenario": "G"})


def scenario_h_replication(ctx: AppContext, params: dict) -> dict:
    """H: real replication changes on an isolated directory, verified with fsck."""
    source = _load_sources(ctx)[0]
    factors = params.get("factors", [2, 3])
    live = ctx.monitor.status()["datanodes"]["live"]
    rows = []
    with IsolatedRun(ctx) as run:
        ingest.ingest_local_file(run, source["path"], source["path"].name, mode="file")
        rows.append({"step": "initial", "factor": run.hdfs.default_replication(), **_replication_measure(run)})
        for factor in factors:
            if factor > live:
                rows.append({"step": "skipped", "factor": factor, "reason": f"only {live} live DataNodes"})
                continue
            started = time.perf_counter()
            replication.request_change(run, factor, acknowledge_reduced_protection=True)
            rows.append(
                {"step": "changed", "factor": factor, "converged_in_s": time.perf_counter() - started, **_replication_measure(run)}
            )
        rejected = None
        try:
            replication.request_change(run, live + 1)
        except ValidationFailure as exc:
            rejected = exc.message
    return {"description": "Replication factor changes and measured effect.", "live_datanodes": live, "rows": rows,
"rejected_over_limit": rejected}


def _replication_measure(run: AppContext) -> dict:
    summary = run.hdfs.content_summary(run.paths.root)
    fsck = run.monitor.fsck(run.paths.root)
    return {
        "length_bytes": summary.length,
        "space_consumed_bytes": summary.space_consumed,
        "multiplier": summary.space_consumed / summary.length if summary.length else None,
        "average_block_replication": fsck["average_block_replication"],
        "under_replicated_blocks": fsck["under_replicated_blocks"],
        "fsck_status": fsck["status"],
    }


def scenario_i_before_after(ctx: AppContext, params: dict, physical_probe: Callable[[], int] | None = None) -> dict:
    """I: identical data stored raw, then optimized in place, measured before and after by HDFS itself."""
    sources = _load_sources(ctx)
    copies = params.get("copies", 2)
    items = _workload_copies(sources, copies)
    rows = []
    for mode, chunk_size in (("file", None), ("chunk", 256 * KIB)):
        with IsolatedRun(ctx) as run:
            disk_baseline = stable_disk(physical_probe) if physical_probe else None
            timing = _ingest_timed(run, items, "raw")
            disk_before = stable_disk(physical_probe) if physical_probe else None
            before = snapshot(run)
            cost_before = namespace_cost(run)

            analysis_id = run.jobs.submit("analyze", mode=mode, chunk_size=chunk_size)
            preview = _job_result(run, analysis_id)
            started = time.perf_counter()
            job_id = run.jobs.submit("optimize", mode=mode, chunk_size=chunk_size)
            optimize_result = _job_result(run, job_id)
            optimize_seconds = time.perf_counter() - started
            disk_after = stable_disk(physical_probe) if physical_probe else None
            after = snapshot(run)
            cost_after = namespace_cost(run)
            verification = _verify_all(run)

            rows.append(
                {
                    "mode": mode,
                    "chunk_size": chunk_size,
                    "files": before["logical_files"],
                    "logical_bytes": before["logical_bytes"],
                    "stored_before": before["stored_content_bytes"],
                    "stored_after": after["stored_content_bytes"],
                    "predicted_stored_after": preview["predicted_stored_bytes"],
                    "hdfs_length_before": before["hdfs_length_bytes"],
                    "hdfs_length_after": after["hdfs_length_bytes"],
                    "hdfs_space_consumed_before": before["hdfs_space_consumed_bytes"],
                    "hdfs_space_consumed_after": after["hdfs_space_consumed_bytes"],
                    "hdfs_files_before": before["hdfs_file_count"],
                    "hdfs_files_after": after["hdfs_file_count"],
                    "namespace_objects_before": cost_before["namespace_objects"],
                    "namespace_objects_after": cost_after["namespace_objects"],
                    "disk_used_delta_before": None if physical_probe is None else disk_before - disk_baseline,
                    "disk_used_delta_after": None if physical_probe is None else disk_after - disk_baseline,
                    "savings_percent_after": savings_percent(after["logical_bytes"], after["stored_content_bytes"]),
                    "dedup_ratio_after": dedup_ratio(after["logical_bytes"], after["stored_content_bytes"]),
                    "upload_duration_s": timing["upload_duration_s"],
                    "optimize_duration_s": optimize_seconds,
                    "optimize_throughput_mib_s": _throughput(before["logical_bytes"], optimize_seconds),
                    "files_failed": optimize_result["files_failed"],
```

```
                    "verification": verification,
                }
            )
    return {"description": f"{copies} exact workload copies per original, stored raw, then optimized.", "sources": len(sources),
"rows": rows}


def _job_result(run: AppContext, job_id: str) -> dict:
    with session_scope(run.session_factory) as session:
        job = session.get(optimizer.DeduplicationJob, job_id)
        if job.result is None:
            raise RuntimeError(f"Job {job.kind} failed: {job.error}")
        return job.result


SCENARIOS: dict[str, tuple[str, Callable[[AppContext, dict], dict]]] = {
    "A_unique_files": ("Unique authentic files", scenario_a_unique),
    "B_exact_duplicates": ("Exact duplicates of authentic files", scenario_b_exact_duplicates),
    "C_many_names": ("Many logical names, identical content", scenario_c_many_names),
    "D_chunk_sizes": ("Chunk-level deduplication by chunk size", scenario_d_chunk_sizes),
    "D2_boundary_shift": ("Fixed-size chunking boundary shift", scenario_d2_boundary_shift),
    "E_roundtrip": ("Download and checksum verification", scenario_e_roundtrip),
    "F_shared_deletion": ("Logical deletion with shared content", scenario_f_shared_deletion),
    "G_restore_after_optimize": ("Restoration after optimization", scenario_g_restore_after_optimize),
    "H_replication": ("Replication changes", scenario_h_replication),
    "I_before_after": ("Storage before and after optimization", scenario_i_before_after),
}


def run_scenario(ctx: AppContext, job_id: str, params: dict, reporter) -> dict:
    """Job: run one scenario and store its result in the main metadata database."""
    scenario = params["scenario"]
    title, function = SCENARIOS[scenario]
    with session_scope(ctx.session_factory) as session:
        row = ExperimentRun(scenario=scenario, params=params.get("options"), job_id=job_id)
        session.add(row)
        session.flush()
        run_id = row.id
    reporter.progress(0, 1, f"Running: {title}")
    started = time.perf_counter()
    try:
        result = function(ctx, params.get("options") or {})
    except Exception as exc:
        with session_scope(ctx.session_factory) as session:
            row = session.get(ExperimentRun, run_id)
            row.status, row.error, row.finished_at = "failed", getattr(exc, "message", str(exc)), utcnow()
        raise
    result["duration_s"] = time.perf_counter() - started
    with session_scope(ctx.session_factory) as session:
        row = session.get(ExperimentRun, run_id)
        row.status, row.result, row.finished_at = "succeeded", result, utcnow()
        audit.record(session, "experiment.completed", f"Experiment {scenario} completed in {result['duration_s']:.1f} s.",
entity_type="experiment", entity_id=run_id)
    reporter.progress(1, 1, "Done")
    return {"experiment_run_id": run_id, "scenario": scenario}


def rows_to_csv(rows: list[dict]) -> str:
    """Flatten result rows to CSV. Nested values become dotted columns."""
    flat = []
    for row in rows:
        out: dict = {}
        for key, value in row.items():
            if isinstance(value, dict):
                out.update({f"{key}.{k}": v for k, v in value.items()})
            else:
                out[key] = value
        flat.append(out)
    columns: list[str] = []
    for row in flat:
        columns.extend(c for c in row if c not in columns)
    buffer = io.StringIO()
    writer = csv.DictWriter(buffer, fieldnames=columns)
    writer.writeheader()
    writer.writerows(flat)
    return buffer.getvalue()
```

backend/app/services/hashing.py 58 lines

```
"""Streaming SHA-256 and deterministic fixed-size chunking."""

from __future__ import annotations

import hashlib
from collections.abc import Iterable, Iterator
from pathlib import Path

BUFFER_SIZE = 1024 * 1024


def sha256_bytes(data: bytes) -> str:
    return hashlib.sha256(data).hexdigest()
```

```
def sha256_file(path: Path | str) -> tuple[str, int]:
    """Hash a local file with constant memory. Returns (hex digest, size in bytes)."""
    digest = hashlib.sha256()
    size = 0
    with open(path, "rb") as handle:
        while block := handle.read(BUFFER_SIZE):
            digest.update(block)
            size += len(block)
    return digest.hexdigest(), size


def sha256_stream(stream: Iterable[bytes]) -> tuple[str, int]:
    digest = hashlib.sha256()
    size = 0
    for block in stream:
        digest.update(block)
        size += len(block)
    return digest.hexdigest(), size


def read_file_blocks(path: Path | str, block_size: int = BUFFER_SIZE) -> Iterator[bytes]:
    with open(path, "rb") as handle:
        while block := handle.read(block_size):
            yield block


def fixed_size_chunks(stream: Iterable[bytes], chunk_size: int) -> Iterator[bytes]:
    """Re-slice a byte stream into chunks of exactly ``chunk_size``, except possibly the last.

    The result depends only on the bytes and the chunk size, not on how the input stream was buffered.
    Memory use is bounded by one chunk plus one input block.
    """
    if chunk_size <= 0:
        raise ValueError("chunk_size must be positive")
    pending = bytearray()
    for block in stream:
        pending += block
        while len(pending) >= chunk_size:
            yield bytes(pending[:chunk_size])
            del pending[:chunk_size]
    if pending:
        yield bytes(pending)
```

backend/app/services/ingest.py 304 lines

```
"""Upload pipeline: hash, look up, store only what is new, then commit the logical reference."""

from __future__ import annotations

import mimetypes
import posixpath
import time
from dataclasses import dataclass
from pathlib import Path

from sqlalchemy import func, select
from sqlalchemy.orm import Session

from app.core.db import session_scope
from app.core.errors import ValidationFailure
from app.models.entities import (
    FileManifest,
    FileStatus,
    LogicalFile,
    StorageMode,
    new_id,
)
from app.services import analytics, audit, content_store, settings_store
from app.services.context import AppContext
from app.services.hashing import read_file_blocks, sha256_file


def normalize_folder(folder: str | None) -> str:
    """Canonical virtual folder path. Folders exist only in metadata, never as user-named HDFS paths."""
    raw = (folder or "/").strip().replace("\\", "/")
    if any(ord(ch) < 32 for ch in raw):
        raise ValidationFailure("Folder path contains control characters.")
    parts = [part for part in raw.split("/") if part]
    if any(part in {".", ".."} for part in parts):
        raise ValidationFailure("Folder path may not contain '.' or '..' segments.")
    normalized = "/" + "/".join(parts)
    if len(normalized) > 512:
        raise ValidationFailure("Folder path is too long.")
    return normalized


def validate_filename(name: str | None) -> str:
    cleaned = posixpath.basename((name or "").replace("\\", "/")).strip()
    if not cleaned or cleaned in {".", ".."}:
        raise ValidationFailure("A file name is required.")
    if any(ord(ch) < 32 for ch in cleaned):
        raise ValidationFailure("File name contains control characters.")
    if len(cleaned) > 255:
```

```
        raise ValidationFailure("File name is too long.")
    return cleaned


def unique_name(session: Session, folder: str, name: str) -> str:
    """Identity comes from content, so a repeated name is simply a different logical file: 'a (1).txt'."""
    stem, extension = posixpath.splitext(name)
    candidate, counter = name, 0
    while session.scalar(
        select(func.count())
        .select_from(LogicalFile)
        .where(LogicalFile.folder == folder, LogicalFile.name == candidate, LogicalFile.status == FileStatus.ACTIVE)
    ):
        counter += 1
        candidate = f"{stem} ({counter}){extension}"
    return candidate


@dataclass
class IngestResult:
    file: LogicalFile
    outcome: str  # stored_new | duplicate_reference | chunks_shared | stored_raw
    duplicate_of_existing: bool
    bytes_written: int
    chunks_total: int
    chunks_new: int
    chunks_reused: int
    hash_ms: float
    store_ms: float
    total_ms: float


def ingest_local_file(
    ctx: AppContext,
    local_path: Path,
    name: str,
    *,
    folder: str = "/",
    mode: str | None = None,
    chunk_size: int | None = None,
    provenance_label: str = "user_upload",
    dataset_source_id: str | None = None,
    known_hash: tuple[str, int] | None = None,
    known_hash_ms: float | None = None,
) -> IngestResult:
    """Store one already-spooled local file. Identical bytes under any name yield one stored copy."""
    started = time.perf_counter()
    name = validate_filename(name)
    folder = normalize_folder(folder)

    with session_scope(ctx.session_factory) as session:
        options = settings_store.get_all(ctx, session)
    mode = settings_store.validate_mode(mode or options["default_ingest_mode"])
    chunk_size = settings_store.validate_chunk_size(ctx, chunk_size or options["default_chunk_size"])
    verify = options["verify_on_write"]
    replication = options["target_replication"]

    # A caller that already hashed the bytes passes its own measured time. Without one, no hashing happened
    # here, so no sample is recorded: timing nothing would report an absurd throughput.
    hash_ms = known_hash_ms
    if known_hash is None:
        hash_started = time.perf_counter()
        file_sha256, size = sha256_file(local_path)
        hash_ms = (time.perf_counter() - hash_started) * 1000
    else:
        file_sha256, size = known_hash

    with ctx.content_lock, session_scope(ctx.session_factory) as session:
        store_started = time.perf_counter()
        duplicate_of_existing = bool(
            session.scalar(
                select(func.count())
                .select_from(LogicalFile)
                .where(LogicalFile.sha256 == file_sha256, LogicalFile.status != FileStatus.PURGED)
            )
        )
        record = LogicalFile(
            id=new_id(),
            name=unique_name(session, folder, name),
            folder=folder,
            size_bytes=size,
            sha256=file_sha256,
            mime_type=mimetypes.guess_type(name)[0],
            extension=(posixpath.splitext(name)[1].lstrip(".").lower() or None),
            storage_mode=mode,
            provenance_label=provenance_label,
            dataset_source_id=dataset_source_id,
        )
        bytes_written, chunks_total, chunks_new, chunks_reused = 0, 0, 0, 0
        chunking_ms = 0.0

        if mode == StorageMode.RAW:
            # The deliberate baseline: a private copy, as a plain HDFS upload would produce.
```

```
            record.raw_path = ctx.paths.raw_path(record.id)
            content_store.put_content(
                ctx,
                record.raw_path,
                lambda: read_file_blocks(local_path),
                file_sha256,
                size,
                verify=verify,
                replication=replication,
            )
            bytes_written, outcome = size, "stored_raw"
        else:
            reused = _reference_existing_content(ctx, session, record, file_sha256, size, mode, chunk_size, local_path, verify,
replication)
            if reused is not None:
                bytes_written = reused
                outcome = "duplicate_reference"
                chunks_total = record.manifest.chunk_count if record.manifest else 0
                chunks_reused = chunks_total
            elif mode == StorageMode.FILE:
                path = ctx.paths.object_path(file_sha256)
                content_store.put_content(
                    ctx,
                    path,
                    lambda: read_file_blocks(local_path),
                    file_sha256,
                    size,
                    verify=verify,
                    replication=replication,
                )
                record.content_object = content_store.register_object(session, file_sha256, size, path)
                bytes_written, outcome = size, "stored_new"
            else:
                build = content_store.build_manifest(
                    ctx,
                    session,
                    read_file_blocks(local_path),
                    file_sha256,
                    size,
                    chunk_size,
                    replication=replication,
                )
                record.manifest = build.manifest
                bytes_written = build.bytes_written
                chunks_total, chunks_new, chunks_reused = build.chunks_total, build.chunks_new, build.chunks_reused
                chunking_ms = build.chunking_ms
                outcome = "chunks_shared" if chunks_reused else "stored_new"

        session.add(record)
        session.flush()
        store_ms = (time.perf_counter() - store_started) * 1000
        total_ms = (time.perf_counter() - started) * 1000

        if hash_ms is not None:
            audit.record_sample(session, "hash", size, hash_ms, mode=mode, file_id=record.id)
        audit.record_sample(session, "upload", size, total_ms, mode=record.storage_mode, file_id=record.id)
        if chunking_ms:
            audit.record_sample(session, "chunking", size, chunking_ms, mode=mode, file_id=record.id)
        audit.record(
            session,
            "file.uploaded",
            f"Uploaded {record.name} ({size} bytes): {outcome}, {bytes_written} bytes written to HDFS.",
            entity_type="file",
            entity_id=record.id,
            details={"sha256": file_sha256, "outcome": outcome, "bytes_written": bytes_written, "mode": record.storage_mode},
        )
        if duplicate_of_existing:
            audit.record(
                session,
                "dedup.duplicate_detected",
                f"{record.name} is byte-identical to content already managed (SHA-256 {file_sha256[:12]}...).",
                entity_type="file",
                entity_id=record.id,
                details={"sha256": file_sha256, "stored_again": outcome == "stored_raw"},
            )
        analytics.record_snapshot(session, "upload")
        session.flush()

    return IngestResult(
        file=record,
        outcome=outcome,
        duplicate_of_existing=duplicate_of_existing,
        bytes_written=bytes_written,
        chunks_total=chunks_total,
        chunks_new=chunks_new,
        chunks_reused=chunks_reused,
        hash_ms=hash_ms or 0.0,
        store_ms=store_ms,
        total_ms=total_ms,
    )


def _reference_existing_content(
```

```
    ctx: AppContext,
    session: Session,
    record: LogicalFile,
    file_sha256: str,
    size: int,
    mode: str,
    chunk_size: int,
    local_path: Path,
    verify: bool,
    replication: int | None,
) -> int | None:
    """Point at an already-stored form of this content. Returns repair bytes, or None if nothing is stored."""
    manifest: FileManifest | None = content_store.find_active_manifest(
        session, file_sha256, chunk_size if mode == StorageMode.CHUNK else None
    )
    obj = content_store.find_active_object(session, file_sha256)

    # Prefer the requested representation, then fall back to the other, so nothing is stored twice.
    if obj is not None and (mode == StorageMode.FILE or manifest is None):
        status = ctx.hdfs.status(obj.hdfs_path)
        repaired = 0
        if status is None or status.length != size or obj.size_bytes != size:
            # Metadata says present but HDFS disagrees. Hash equality means the upload can repair it.
            if status is not None:
                ctx.hdfs.delete(obj.hdfs_path)
            content_store.put_content(
                ctx, obj.hdfs_path, lambda: read_file_blocks(local_path), file_sha256, size, verify=verify, replication=replication
            )
            repaired = size
            audit.record(
                session,
                "storage.object_repaired",
                f"Stored object {file_sha256[:12]}... was missing or truncated and was rewritten from an upload.",
                severity="warning",
                entity_type="object",
                entity_id=obj.id,
            )
        record.storage_mode = StorageMode.FILE
        record.content_object = obj
        return repaired

    fallback = manifest or content_store.find_active_manifest(session, file_sha256)
    if fallback is not None:
        chunks = content_store.manifest_chunks(session, fallback.id)
        if _all_chunks_present(ctx, chunks):
            record.storage_mode = StorageMode.CHUNK
            record.manifest = fallback
            return 0
        # A damaged manifest must not gain references. Rewriting its chunks from this upload heals it.
        repaired = content_store.repair_manifest(ctx, session, fallback, local_path, replication)
        audit.record(
            session,
            "storage.manifest_repaired",
            f"Manifest for {file_sha256[:12]}... had missing chunks that were rewritten from an upload.",
            severity="warning",
            entity_type="manifest",
            entity_id=fallback.id,
        )
        record.storage_mode = StorageMode.CHUNK
        record.manifest = fallback
        return repaired
    return None


def _all_chunks_present(ctx: AppContext, chunks: list) -> bool:
    for chunk in chunks:
        status = ctx.hdfs.status(chunk.hdfs_path)
        if status is None or status.length != chunk.size_bytes:
            return False
    return True
```

backend/app/services/integrity.py 182 lines

```
"""Integrity verification and verified restoration: rebuild the original bytes, then compare SHA-256."""

from __future__ import annotations

import hashlib
import json
import time
from collections.abc import Iterator
from pathlib import Path

from sqlalchemy import select
from sqlalchemy.orm import Session

from app.core.db import session_scope, utcnow
from app.core.errors import HdfsFileNotFound, IntegrityFailure, NotFoundError
from app.models.entities import FileStatus, IntegrityCheck, LogicalFile, StorageMode
from app.services import audit, content_store
from app.services.context import AppContext
from app.services.hashing import sha256_bytes
```

```
def get_file(session: Session, file_id: str) -> LogicalFile:
    record = session.get(LogicalFile, file_id)
    if record is None or record.status == FileStatus.PURGED:
        raise NotFoundError(f"No file with id {file_id}.")
    return record


def verify_file(ctx: AppContext, file_id: str) -> IntegrityCheck:
    """Read every byte back from HDFS and compare. HDFS being down is an error, not a failed check."""
    started = time.perf_counter()
    with session_scope(ctx.session_factory) as session:
        record = get_file(session, file_id)
        digest = hashlib.sha256()
        missing: list[str] = []
        corrupt: list[str] = []
        bytes_read = 0
        chunks_checked = 0
        detail = None

        if record.storage_mode == StorageMode.CHUNK:
            chunks = content_store.manifest_chunks(session, record.manifest_id)
            for chunk in chunks:
                chunks_checked += 1
                try:
                    data = b"".join(ctx.hdfs.open(chunk.hdfs_path))
                except HdfsFileNotFound:
                    missing.append(chunk.sha256)
                    continue
                if sha256_bytes(data) != chunk.sha256:
                    corrupt.append(chunk.sha256)
                digest.update(data)
                bytes_read += len(data)
            detail = _compare_durable_manifest(ctx, record, [c.sha256 for c in chunks])
        else:
            path = record.raw_path if record.storage_mode == StorageMode.RAW else record.content_object.hdfs_path
            try:
                for block in ctx.hdfs.open(path):
                    digest.update(block)
                    bytes_read += len(block)
            except HdfsFileNotFound:
                missing.append(path)

        actual = None if missing else digest.hexdigest()
        ok = not missing and not corrupt and actual == record.sha256 and bytes_read == record.size_bytes and detail is None
        duration_ms = (time.perf_counter() - started) * 1000
        check = IntegrityCheck(
            file_id=record.id,
            ok=ok,
            expected_sha256=record.sha256,
            actual_sha256=actual,
            bytes_read=bytes_read,
            chunks_checked=chunks_checked,
            missing=missing or None,
            corrupt=corrupt or None,
            detail=detail,
            duration_ms=duration_ms,
        )
        session.add(check)
        record.last_verified_at = utcnow()
        record.last_verified_ok = ok
        audit.record_sample(session, "verify", bytes_read, duration_ms, mode=record.storage_mode, file_id=record.id)
        audit.record(
            session,
            "integrity.passed" if ok else "integrity.failed",
            f"{record.name}: SHA-256 {'matches' if ok else 'does NOT match'} "
            f"({bytes_read} bytes, {len(missing)} missing, {len(corrupt)} corrupt).",
            severity="info" if ok else "error",
            entity_type="file",
            entity_id=record.id,
        )
        session.flush()
    return check


def _compare_durable_manifest(ctx: AppContext, record: LogicalFile, expected_hashes: list[str]) -> str | None:
    """The manifest in HDFS is the durable recipe. It must agree with the copy in the database."""
    try:
        document = json.loads(b"".join(ctx.hdfs.open(record.manifest.hdfs_path)))
    except HdfsFileNotFound:
        return f"Manifest file is missing from HDFS: {record.manifest.hdfs_path}"
    except ValueError:
        return "Manifest file in HDFS is not valid JSON."
    if [entry["sha256"] for entry in document.get("chunks", [])] != expected_hashes:
        return "Manifest file in HDFS disagrees with the metadata database."
    return None


def preflight(ctx: AppContext, session: Session, record: LogicalFile) -> None:
    """Fail before any response headers are sent if the start of the content is not readable."""
    if record.storage_mode == StorageMode.CHUNK:
        return  # chunks are checked one by one as they are streamed
    path = record.raw_path if record.storage_mode == StorageMode.RAW else record.content_object.hdfs_path
    if ctx.hdfs.status(path) is None:
```

```
        raise IntegrityFailure(f"Stored content for {record.name} is missing from HDFS.")


def stream_verified(ctx: AppContext, file_id: str) -> Iterator[bytes]:
    """Stream a file and compare its whole hash at the end.

    A failure raises mid-stream, which aborts the HTTP response, so a client can never mistake
    damaged data for a complete download.
    """
    started = time.perf_counter()
    digest = hashlib.sha256()
    total = 0
    with session_scope(ctx.session_factory) as session:
        record = get_file(session, file_id)
        mode, expected = record.storage_mode, record.sha256
        for block in content_store.iter_file_bytes(ctx, session, record):
            digest.update(block)
            total += len(block)
            yield block
    if digest.hexdigest() != expected:
        audit.record_failure(
            ctx.session_factory,
            "file.restore_failed",
            f"Download of {file_id} failed: read {digest.hexdigest()}, expected {expected}.",
            entity_type="file",
            entity_id=file_id,
        )
        raise IntegrityFailure(f"Reconstructed file {file_id} failed SHA-256 verification.")
    with session_scope(ctx.session_factory) as session:
        audit.record_sample(session, "restore", total, (time.perf_counter() - started) * 1000, mode=mode, file_id=file_id)
        audit.record(session, "file.downloaded", f"Restored {total} bytes; SHA-256 verified.", entity_type="file",
entity_id=file_id)


def restore_to_path(ctx: AppContext, file_id: str, destination: Path) -> str:
    """Rebuild a file on the local filesystem through the verified stream. Returns its digest."""
    digest = hashlib.sha256()
    with open(destination, "wb") as handle:
        for block in stream_verified(ctx, file_id):
            digest.update(block)
            handle.write(block)
    return digest.hexdigest()


def latest_checks(session: Session, file_id: str, limit: int = 10) -> list[IntegrityCheck]:
    return list(
        session.scalars(
            select(IntegrityCheck)
            .where(IntegrityCheck.file_id == file_id)
            .order_by(IntegrityCheck.checked_at.desc(), IntegrityCheck.id.desc())
            .limit(limit)
        )
    )


def run_verify_all(ctx: AppContext, job_id: str, params: dict, reporter) -> dict:
    with session_scope(ctx.session_factory) as session:
        ids = list(session.scalars(select(LogicalFile.id).where(LogicalFile.status == FileStatus.ACTIVE)))
    passed, failed = 0, []
    reporter.progress(0, len(ids), "Verifying files")
    for index, file_id in enumerate(ids, start=1):
        check = verify_file(ctx, file_id)
        if check.ok:
            passed += 1
        else:
            failed.append(file_id)
        reporter.progress(index, len(ids), f"Verified {index} of {len(ids)}")
    return {"files_checked": len(ids), "passed": passed, "failed": failed, "errors": [f"{f}: verification failed" for f in failed]}
```

backend/app/services/jobs.py 117 lines

```
"""Background job runner. Jobs run one at a time and persist their state in the database."""

from __future__ import annotations

import logging
from collections.abc import Callable
from concurrent.futures import ThreadPoolExecutor

from app.core.db import session_scope, utcnow
from app.core.errors import AppError
from app.models.entities import DeduplicationJob, JobStatus, new_id
from app.services import audit
from app.services.context import AppContext

log = logging.getLogger("hdfs_optimizer.jobs")

JobHandler = Callable[[AppContext, str, dict, "JobReporter"], dict]


class JobReporter:
    """Writes progress to the database so the API can report what is really happening."""

    def __init__(self, ctx: AppContext, job_id: str):
```

```
        self.ctx = ctx
        self.job_id = job_id

    def progress(self, done: int, total: int, message: str | None = None) -> None:
        with session_scope(self.ctx.session_factory) as session:
            job = session.get(DeduplicationJob, self.job_id)
            job.progress_done = done
            job.progress_total = total
            job.heartbeat_at = utcnow()
            if message:
                job.message = message


class JobManager:
    def __init__(self, ctx: AppContext, handlers: dict[str, JobHandler], *, synchronous: bool = False):
        self.ctx = ctx
        self.handlers = handlers
        self.synchronous = synchronous  # tests run jobs inline for determinism
        self.executor = None if synchronous else ThreadPoolExecutor(max_workers=1, thread_name_prefix="job")

    def submit(
        self,
        kind: str,
        *,
        mode: str | None = None,
        chunk_size: int | None = None,
        params: dict | None = None,
        resumed_from_id: str | None = None,
    ) -> str:
        if kind not in self.handlers:
            raise AppError(f"Unknown job kind: {kind}")
        job_id = new_id()
        merged = {**(params or {}), "mode": mode, "chunk_size": chunk_size}
        with session_scope(self.ctx.session_factory) as session:
            session.add(
                DeduplicationJob(
                    id=job_id,
                    kind=kind,
                    mode=mode,
                    chunk_size=chunk_size,
                    params=merged,
                    resumed_from_id=resumed_from_id,
                )
            )
        if self.synchronous:
            self._run(job_id, kind, merged)
        else:
            self.executor.submit(self._run, job_id, kind, merged)
        return job_id

    def _run(self, job_id: str, kind: str, params: dict) -> None:
        with session_scope(self.ctx.session_factory) as session:
            job = session.get(DeduplicationJob, job_id)
            job.status = JobStatus.RUNNING
            job.started_at = job.heartbeat_at = utcnow()
        try:
            result = self.handlers[kind](self.ctx, job_id, params, JobReporter(self.ctx, job_id))
            status = JobStatus.PARTIAL if result.get("errors") else JobStatus.SUCCEEDED
            error = None
        except Exception as exc:  # noqa: BLE001 - any failure becomes a visible job state
            log.exception("job %s failed", job_id)
            result, status = None, JobStatus.FAILED
            error = getattr(exc, "message", None) or f"{type(exc).__name__}: {exc}"
            audit.record_failure(
                self.ctx.session_factory, f"job.{kind}_failed", error, entity_type="job", entity_id=job_id
            )
        with session_scope(self.ctx.session_factory) as session:
            job = session.get(DeduplicationJob, job_id)
            job.status = status
            job.result = _json_safe(result)
            job.error = error
            job.finished_at = utcnow()
            if status == JobStatus.SUCCEEDED:
                job.message = "Completed"
            elif status == JobStatus.PARTIAL:
                job.message = "Completed with errors"
            else:
                job.message = error

    def shutdown(self) -> None:
        if self.executor:
            self.executor.shutdown(wait=False, cancel_futures=True)


def _json_safe(value):
    """Job results hold plain data; convert any datetime values that metrics may contain."""
    if isinstance(value, dict):
        return {str(k): _json_safe(v) for k, v in value.items()}
    if isinstance(value, list | tuple | set):
        return [_json_safe(v) for v in value]
    if hasattr(value, "isoformat"):
        return value.isoformat()
    return value
```

backend/app/services/lifecycle.py 255 lines

```python
"""Reference-aware deletion, restoration, garbage collection and crash recovery.

Deleting a logical file never touches shared content directly. Content is removed only by the
collector, and only when a query over the current relations proves nothing still refers to it.
A soft-deleted file keeps its content, because it can still be restored.
"""

from __future__ import annotations

import posixpath
from dataclasses import asdict, dataclass, field

from sqlalchemy import select
from sqlalchemy.orm import Session

from app.core.db import session_scope, utcnow
from app.core.errors import ConflictError, HdfsError
from app.hdfs.base import walk_files
from app.models.entities import (
    Chunk,
    ContentObject,
    ContentState,
    DeduplicationJob,
    FileChunk,
    FileManifest,
    FileStatus,
    JobStatus,
    LogicalFile,
    StorageMode,
)
from app.services import analytics, audit
from app.services.content_store import parent_of
from app.services.context import AppContext
from app.services.ingest import unique_name
from app.services.integrity import get_file


def soft_delete(ctx: AppContext, file_id: str) -> LogicalFile:
    """Recoverable delete. A private raw copy moves to the recovery area; shared content stays in place."""
    with ctx.content_lock, session_scope(ctx.session_factory) as session:
        record = get_file(session, file_id)
        if record.status == FileStatus.DELETED:
            raise ConflictError(f"{record.name} is already deleted.")
        if record.storage_mode == StorageMode.RAW:
            target = ctx.paths.recovery_raw_path(record.id)
            ctx.hdfs.mkdirs(parent_of(target))
            if not ctx.hdfs.rename(record.raw_path, target):
                raise HdfsError(f"Could not move {record.raw_path} to the recovery area.")
            record.raw_path = target
        record.status = FileStatus.DELETED
        record.deleted_at = utcnow()
        audit.record(session, "file.deleted", f"Deleted {record.name} (recoverable).", entity_type="file", entity_id=record.id)
        analytics.record_snapshot(session, "delete")
    return record


def restore(ctx: AppContext, file_id: str) -> LogicalFile:
    with ctx.content_lock, session_scope(ctx.session_factory) as session:
        record = get_file(session, file_id)
        if record.status != FileStatus.DELETED:
            raise ConflictError(f"{record.name} is not deleted.")
        if record.storage_mode == StorageMode.RAW:
            target = ctx.paths.raw_path(record.id)
            ctx.hdfs.mkdirs(parent_of(target))
            if not ctx.hdfs.rename(record.raw_path, target):
                raise HdfsError(f"Could not restore {record.raw_path}.")
            record.raw_path = target
        original = record.name
        record.name = unique_name(session, record.folder, original)
        record.status = FileStatus.ACTIVE
        record.deleted_at = None
        audit.record(
            session,
            "file.restored",
            f"Restored {original}" + (f" as {record.name}." if record.name != original else "."),
            entity_type="file",
            entity_id=record.id,
        )
        analytics.record_snapshot(session, "restore")
    return record


def purge(ctx: AppContext, file_id: str) -> dict:
    """Permanently remove a soft-deleted file. Shared content stays while anything else refers to it."""
    with ctx.content_lock:
        with session_scope(ctx.session_factory) as session:
            record = get_file(session, file_id)
            if record.status != FileStatus.DELETED:
                raise ConflictError("Only a deleted file can be permanently removed. Delete it first.")
            raw_path = record.raw_path
            record.status = FileStatus.PURGED
            record.purged_at = utcnow()
```

```
            record.raw_path = None
            record.object_id = None
            record.manifest_id = None
            audit.record(session, "file.purged", f"Permanently removed {record.name}.", entity_type="file", entity_id=record.id)
        # The commit above came first, so a crash here leaves an orphan for reconciliation, never a dangling pointer.
        if raw_path:
            ctx.hdfs.delete(raw_path)
        return collect_garbage(ctx)


@dataclass
class GarbageReport:
    objects: list[str] = field(default_factory=list)
    manifests: list[str] = field(default_factory=list)
    chunks: list[str] = field(default_factory=list)
    reclaimable_bytes: int = 0
    removed: bool = False

    def as_dict(self) -> dict:
        data = asdict(self)
        data["object_count"] = len(self.objects)
        data["manifest_count"] = len(self.manifests)
        data["chunk_count"] = len(self.chunks)
        return data


def _find_garbage(session: Session) -> tuple[list[ContentObject], list[FileManifest], list[Chunk]]:
    holds_content = LogicalFile.status != FileStatus.PURGED
    objects = list(
        session.scalars(
            select(ContentObject).where(
                ContentObject.state == ContentState.ACTIVE,
                ContentObject.id.not_in(
                    select(LogicalFile.object_id).where(holds_content, LogicalFile.object_id.is_not(None))
                ),
            )
        )
    )
    referenced_manifests = select(LogicalFile.manifest_id).where(holds_content, LogicalFile.manifest_id.is_not(None))
    manifests = list(
        session.scalars(
            select(FileManifest).where(FileManifest.state == ContentState.ACTIVE, FileManifest.id.not_in(referenced_manifests))
        )
    )
    # A chunk is live while any manifest that survives this collection uses it.
    chunks = list(
        session.scalars(
            select(Chunk).where(
                Chunk.state == ContentState.ACTIVE,
                Chunk.id.not_in(
                    select(FileChunk.chunk_id)
                    .join(FileManifest, FileManifest.id == FileChunk.manifest_id)
                    .where(FileManifest.state == ContentState.ACTIVE, FileManifest.id.in_(referenced_manifests))
                ),
            )
        )
    )
    return objects, manifests, chunks


def collect_garbage(ctx: AppContext, *, dry_run: bool = False) -> dict:
    """Remove only content that no restorable file refers to. Dry-run reports without changing anything."""
    with ctx.content_lock:
        with session_scope(ctx.session_factory) as session:
            objects, manifests, chunks = _find_garbage(session)
            report = GarbageReport(
                objects=[o.sha256 for o in objects],
                manifests=[m.file_sha256 for m in manifests],
                chunks=[c.sha256 for c in chunks],
                reclaimable_bytes=sum(o.size_bytes for o in objects) + sum(c.size_bytes for c in chunks),
            )
            if dry_run or not (objects or manifests or chunks):
                return report.as_dict()
            # Mark first: from here, an interrupted run is finished by recover_on_startup.
            for row in [*objects, *manifests, *chunks]:
                row.state = ContentState.GC_PENDING
        _finish_pending_removals(ctx)
        with session_scope(ctx.session_factory) as session:
            audit.record(
                session,
                "storage.gc",
                f"Removed {len(report.objects)} objects, {len(report.manifests)} manifests and "
                f"{len(report.chunks)} chunks that nothing refers to ({report.reclaimable_bytes} bytes before replication).",
                details={k: len(getattr(report, k)) for k in ("objects", "manifests", "chunks")},
            )
            analytics.record_snapshot(session, "gc")
        report.removed = True
        return report.as_dict()


def _finish_pending_removals(ctx: AppContext) -> None:
    with session_scope(ctx.session_factory) as session:
        for model in (ContentObject, FileManifest, Chunk):
```

```
            for row in session.scalars(select(model).where(model.state == ContentState.GC_PENDING)):
                ctx.hdfs.delete(row.hdfs_path)
                row.state = ContentState.REMOVED
                row.removed_at = utcnow()
                if model is FileManifest:
                    row.entries.clear()


def recover_on_startup(ctx: AppContext) -> dict:
    """Bring metadata and HDFS back to a consistent state after an unclean stop."""
    summary = {"jobs_interrupted": 0, "staging_removed": 0, "pending_gc": "skipped", "hdfs_available": True}
    with session_scope(ctx.session_factory) as session:
        stale = session.scalars(
            select(DeduplicationJob).where(DeduplicationJob.status.in_([JobStatus.RUNNING, JobStatus.QUEUED]))
        )
        for job in stale:
            job.status = JobStatus.INTERRUPTED
            job.finished_at = utcnow()
            job.message = "The server stopped while this job was active. Completed files are safe; it can be resumed."
            summary["jobs_interrupted"] += 1
            audit.record(session, "job.interrupted", f"Job {job.id} ({job.kind}) was interrupted.", severity="warning",
entity_type="job", entity_id=job.id)
    try:
        ctx.ensure_namespace()
        with ctx.content_lock:
            _finish_pending_removals(ctx)
            summary["pending_gc"] = "finished"
            for entry in ctx.hdfs.list_dir(ctx.paths.staging):
                ctx.hdfs.delete(entry.path, recursive=True)
                summary["staging_removed"] += 1
    except HdfsError as exc:
        # Starting without HDFS is allowed: the UI must be able to report that the cluster is down.
        summary["hdfs_available"] = False
        summary["error"] = exc.message
    return summary


def reconcile(ctx: AppContext, *, apply: bool = False) -> dict:
    """Compare HDFS with the metadata. Orphans are quarantined, never deleted, and only when apply=True."""
    with ctx.content_lock, session_scope(ctx.session_factory) as session:
        expected: set[str] = set()
        expected.update(session.scalars(select(ContentObject.hdfs_path).where(ContentObject.state == ContentState.ACTIVE)))
        expected.update(session.scalars(select(Chunk.hdfs_path).where(Chunk.state == ContentState.ACTIVE)))
        expected.update(session.scalars(select(FileManifest.hdfs_path).where(FileManifest.state == ContentState.ACTIVE)))
        expected.update(
            path
            for path in session.scalars(select(LogicalFile.raw_path).where(LogicalFile.status != FileStatus.PURGED,
LogicalFile.raw_path.is_not(None)))
        )
        present: set[str] = set()
        for area in (ctx.paths.objects, ctx.paths.chunks, ctx.paths.manifests, ctx.paths.raw, f"{ctx.paths.recovery}/deleted"):
            present.update(status.path for status in walk_files(ctx.hdfs, area))
        orphans = sorted(present - expected)
        missing = sorted(expected - present)
        moved = 0
        if apply:
            for path in orphans:
                target = ctx.paths.recovery_orphan_path(posixpath.basename(path))
                ctx.hdfs.mkdirs(parent_of(target))
                if ctx.hdfs.rename(path, target):
                    moved += 1
            audit.record(session, "storage.reconciled", f"Quarantined {moved} orphaned HDFS files.", details={"missing":
len(missing)})
        return {
            "paths_expected": len(expected),
            "paths_present": len(present),
            "orphans": orphans[:200],
            "orphan_count": len(orphans),
            "missing": missing[:200],
            "missing_count": len(missing),
            "orphans_quarantined": moved,
        }
```

backend/app/services/monitor.py 172 lines

```
"""Cluster health read from the NameNode's JMX servlet and fsck servlet. Nothing here is estimated."""

from __future__ import annotations

import json
import re
from datetime import UTC, datetime

import httpx

from app.core.errors import HdfsError, HdfsUnavailable


def _now_iso() -> str:
    return datetime.now(UTC).isoformat()


class ClusterMonitor:
    def __init__(self, namenode_url: str, *, user: str = "hadoop", timeout_seconds: float = 10.0):
```

```
        self.base_url = namenode_url.rstrip("/")
        self.user = user
        self._http = httpx.Client(timeout=httpx.Timeout(timeout_seconds, connect=3.0))

    def close(self) -> None:
        self._http.close()

    def _bean(self, name: str) -> dict:
        try:
            response = self._http.get(f"{self.base_url}/jmx", params={"qry": f"Hadoop:service=NameNode,name={name}"})
        except httpx.TransportError as exc:
            raise HdfsUnavailable(f"Cannot reach the NameNode at {self.base_url}: {exc}") from exc
        if not response.is_success:
            raise HdfsError(f"NameNode JMX returned {response.status_code} for {name}.")
        beans = response.json().get("beans", [])
        if not beans:
            raise HdfsError(f"NameNode JMX has no bean named {name}.")
        return beans[0]

    def status(self) -> dict:
        """Current cluster state. A down NameNode is a normal answer here, not an exception."""
        checked_at = _now_iso()
        try:
            fs = self._bean("FSNamesystem")
            state = self._bean("FSNamesystemState")
            info = self._bean("NameNodeInfo")
            nn_status = self._bean("NameNodeStatus")
        except HdfsError as exc:
            return {"reachable": False, "error": exc.message, "checked_at": checked_at, "namenode_url": self.base_url}

        safemode = bool(info.get("Safemode"))
        live = int(fs.get("NumLiveDataNodes", 0))
        dead = int(fs.get("NumDeadDataNodes", 0))
        under = int(fs.get("UnderReplicatedBlocks", 0))
        missing = int(fs.get("MissingBlocks", 0))
        corrupt = int(fs.get("CorruptBlocks", 0))
        problems = []
        if safemode:
            problems.append("The NameNode is in safe mode, so writes are refused.")
        if dead:
            problems.append("1 DataNode is dead." if dead == 1 else f"{dead} DataNodes are dead.")
        if missing:
            problems.append("1 block has no live replica." if missing == 1 else f"{missing} blocks have no live replica.")
        if corrupt:
            problems.append("1 block is corrupt." if corrupt == 1 else f"{corrupt} blocks are corrupt.")
        if under:
            problems.append("1 block is under-replicated." if under == 1 else f"{under} blocks are under-replicated.")
        if live == 0:
            problems.append("No DataNodes are live.")

        return {
            "reachable": True,
            "checked_at": checked_at,
            "namenode_url": self.base_url,
            "health": "healthy" if not problems else ("critical" if missing or live == 0 else "degraded"),
            "problems": problems,
            "namenode": {
                "state": nn_status.get("State"),
                "role": nn_status.get("NNRole"),
                "host_and_port": nn_status.get("HostAndPort"),
                "software_version": info.get("SoftwareVersion"),
                "cluster_id": info.get("ClusterId"),
                "block_pool_id": info.get("BlockPoolId"),
                "started_at": nn_status.get("LastHATransitionTime") or None,
                "fs_state": state.get("FSState"),
                "safemode": safemode,
                "safemode_message": info.get("Safemode") or None,
            },
            "capacity": {
                "total_bytes": int(fs.get("CapacityTotal", 0)),
                "used_bytes": int(fs.get("CapacityUsed", 0)),
                "remaining_bytes": int(fs.get("CapacityRemaining", 0)),
                "non_dfs_used_bytes": int(info.get("NonDfsUsedSpace", 0)),
            },
            "namespace": {
                "files_total": int(fs.get("FilesTotal", 0)),
                "blocks_total": int(fs.get("BlocksTotal", 0)),
                "under_replicated_blocks": under,
                "missing_blocks": missing,
                "corrupt_blocks": corrupt,
                "pending_replication_blocks": int(fs.get("PendingReplicationBlocks", 0)),
                "pending_deletion_blocks": int(fs.get("PendingDeletionBlocks", 0)),
                "excess_blocks": int(fs.get("ExcessBlocks", 0)),
            },
            "datanodes": {
                "live": live,
                "dead": dead,
                "stale": int(fs.get("StaleDataNodes", 0)),
                "decommissioning": int(fs.get("NumDecommissioningDataNodes", 0)),
            },
        }

    def nodes(self) -> dict:
```

```
        checked_at = _now_iso()
        info = self._bean("NameNodeInfo")
        nodes = []
        for state, field in (("live", "LiveNodes"), ("dead", "DeadNodes")):
            for name, node in json.loads(info.get(field) or "{}").items():
                nodes.append(
                    {
                        "name": name.split(":")[0],
                        "transfer_address": name,
                        "state": state,
                        "last_contact_seconds": node.get("lastContact"),
                        "capacity_bytes": node.get("capacity"),
                        "used_bytes": node.get("usedSpace") if state == "live" else None,
                        "remaining_bytes": node.get("remaining"),
                        "non_dfs_used_bytes": node.get("nonDfsUsedSpace"),
                        "blocks": node.get("numBlocks"),
                        "version": node.get("version"),
                        "admin_state": node.get("adminState"),
                        "rack": node.get("location"),
                    }
                )
        nodes.sort(key=lambda n: n["name"])
        return {"checked_at": checked_at, "nodes": nodes}

    def fsck(self, path: str) -> dict:
        """Run fsck through the NameNode's servlet and parse the summary it prints."""
        try:
            response = self._http.get(
                f"{self.base_url}/fsck", params={"ugi": self.user, "path": path}, timeout=60
            )
        except httpx.TransportError as exc:
            raise HdfsUnavailable(f"Cannot reach the NameNode at {self.base_url}: {exc}") from exc
        if not response.is_success:
            raise HdfsError(f"fsck returned {response.status_code}: {response.text[:300]}")
        return parse_fsck(response.text)


def parse_fsck(text: str) -> dict:
    def integer(pattern: str) -> int | None:
        match = re.search(pattern, text, flags=re.MULTILINE)
        return int(match.group(1)) if match else None

    average = re.search(r"Average block replication:\s+([\d.]+)", text)
    status = re.search(r"Status:\s+(\w+)", text)
    return {
        "status": status.group(1) if status else None,
        "total_files": integer(r"Total files:\s+(\d+)"),
        "total_blocks": integer(r"Total blocks \(validated\):\s+(\d+)"),
        "minimally_replicated_blocks": integer(r"Minimally replicated blocks:\s+(\d+)"),
        "over_replicated_blocks": integer(r"Over-replicated blocks:\s+(\d+)"),
        "under_replicated_blocks": integer(r"Under-replicated blocks:\s+(\d+)"),
        "mis_replicated_blocks": integer(r"Mis-replicated blocks:\s+(\d+)"),
        "default_replication": integer(r"Default replication factor:\s+(\d+)"),
        "average_block_replication": float(average.group(1)) if average else None,
        "missing_blocks": integer(r"Missing blocks:\s+(\d+)"),
        "corrupt_blocks": integer(r"Corrupt blocks:\s+(\d+)"),
        "missing_replicas": integer(r"Missing replicas:\s+(\d+)"),
        "datanodes": integer(r"Number of data-nodes:\s+(\d+)"),
    }
```

backend/app/services/optimizer.py 367 lines

```
"""Analysis (dry run) and safe optimization.

Analysis reads and changes nothing permanently: it reports estimated savings. Optimization converts one
file at a time, always in this order:

    1. make the deduplicated form durable in HDFS and verify it against the file's SHA-256
    2. commit the file's new pointer
    3. only then remove the old copy

Every file is its own unit of work, so an interrupted job loses nothing and can simply be run again.
"""

from __future__ import annotations

import hashlib
import time

from sqlalchemy import func, select
from sqlalchemy.orm import Session

from app.core.db import session_scope
from app.core.errors import (
    ConflictError,
    HdfsError,
    HdfsUnavailable,
    IntegrityFailure,
    NotFoundError,
    ValidationFailure,
)
from app.models.entities import (
    Chunk,
    ContentObject,
```

```
    ContentState,
    DeduplicationJob,
    FileStatus,
    JobStatus,
    LogicalFile,
    OptimizationHistory,
    StorageMode,
)
from app.services import analytics, audit, content_store, lifecycle, settings_store
from app.services.content_store import parent_of
from app.services.context import AppContext
from app.services.hashing import fixed_size_chunks, sha256_bytes, sha256_stream


def eligible_query(mode: str):
    sources = [StorageMode.RAW] if mode == StorageMode.FILE else [StorageMode.RAW, StorageMode.FILE]
    return select(LogicalFile).where(LogicalFile.status == FileStatus.ACTIVE, LogicalFile.storage_mode.in_(sources))


def catalog_fingerprint(session: Session) -> str:
    """Changes whenever the set of files or their storage changes, which invalidates an earlier preview."""
    count, total, latest = session.execute(
        select(func.count(), func.coalesce(func.sum(LogicalFile.size_bytes), 0), func.max(LogicalFile.updated_at)).where(
            LogicalFile.status != FileStatus.PURGED
        )
    ).one()
    return hashlib.sha256(f"{count}|{total}|{latest}".encode()).hexdigest()


def _source_path(file: LogicalFile) -> str:
    return file.raw_path if file.storage_mode == StorageMode.RAW else file.content_object.hdfs_path


def capture_metrics(ctx: AppContext) -> dict:
    with session_scope(ctx.session_factory) as session:
        return {"derived": analytics.storage_statistics(session), "measured": analytics.measured_hdfs(ctx)}


# -- analysis ---------------------------------------------------------------------------------


def run_analysis(ctx: AppContext, job_id: str, params: dict, reporter) -> dict:
    mode = params["mode"]
    chunk_size = params.get("chunk_size")
    with session_scope(ctx.session_factory) as session:
        fingerprint = catalog_fingerprint(session)
        stats = analytics.storage_statistics(session)
        files = list(session.scalars(eligible_query(mode).order_by(LogicalFile.created_at)))
        rows = [
            {
                "id": f.id,
                "name": f.name,
                "folder": f.folder,
                "sha256": f.sha256,
                "size_bytes": f.size_bytes,
                "storage_mode": f.storage_mode,
                "source_path": _source_path(f),
            }
            for f in files
        ]
        stored_objects = set(session.scalars(select(ContentObject.sha256).where(ContentObject.state == ContentState.ACTIVE)))
        known_chunks = set(session.scalars(select(Chunk.sha256).where(Chunk.state == ContentState.ACTIVE)))
        manifests_exist = set()
        if mode == StorageMode.CHUNK:
            for sha in {r["sha256"] for r in rows}:
                if content_store.find_active_manifest(session, sha, chunk_size):
                    manifests_exist.add(sha)
        # An object can go away only if every file holding it would be converted by this job.
        holders = dict(
            session.execute(
                select(LogicalFile.object_id, func.count())
                .where(LogicalFile.status != FileStatus.PURGED, LogicalFile.object_id.is_not(None))
                .group_by(LogicalFile.object_id)
            ).all()
        )
        converting_holders = {}
        object_sizes = {}
        if mode == StorageMode.CHUNK:
            for f in files:
                if f.object_id:
                    converting_holders[f.object_id] = converting_holders.get(f.object_id, 0) + 1
                    object_sizes[f.object_id] = f.size_bytes

    reporter.progress(0, len(rows), "Analysing files")
    actions, new_bytes, new_chunks = [], 0, 0
    total_chunks, reused_chunks = 0, 0
    seen_chunks: set[str] = set()
    seen_contents: set[str] = set()
    for index, row in enumerate(rows, start=1):
        sha = row["sha256"]
        first_copy = sha not in seen_contents
        seen_contents.add(sha)
        if mode == StorageMode.FILE:
```

```
            already = sha in stored_objects
            if first_copy and not already:
                new_bytes += row["size_bytes"]
            action = "promote_to_object" if first_copy and not already else "reference_existing_object"
        else:
            action = "reference_existing_manifest" if sha in manifests_exist or not first_copy else "chunk"
            if action == "chunk":
                for block in fixed_size_chunks(ctx.hdfs.open(row["source_path"]), chunk_size):
                    digest = sha256_bytes(block)
                    total_chunks += 1
                    if digest in known_chunks or digest in seen_chunks:
                        reused_chunks += 1
                    else:
                        seen_chunks.add(digest)
                        new_bytes += len(block)
                        new_chunks += 1
        actions.append({k: row[k] for k in ("id", "name", "folder", "sha256", "size_bytes", "storage_mode")} | {"action": action})
        reporter.progress(index, len(rows), f"Analysed {index} of {len(rows)}")

    raw_removed = sum(r["size_bytes"] for r in rows if r["storage_mode"] == StorageMode.RAW)
    objects_removed = sum(
        object_sizes[oid] for oid, count in converting_holders.items() if holders.get(oid, 0) == count
    )
    predicted_stored = stats["stored_content_bytes"] - raw_removed - objects_removed + new_bytes
    return {
        "estimated": True,
        "catalog_fingerprint": fingerprint,
        "mode": mode,
        "chunk_size": chunk_size,
        "files_eligible": len(rows),
        "distinct_contents": len(seen_contents),
        "current_logical_bytes": stats["logical_bytes"],
        "current_stored_bytes": stats["stored_content_bytes"],
        "predicted_stored_bytes": predicted_stored,
        "predicted_reduction_bytes": stats["stored_content_bytes"] - predicted_stored,
        "predicted_savings_percent": analytics.savings_percent(stats["logical_bytes"], predicted_stored),
        "new_content_bytes": new_bytes,
        "chunks_examined": total_chunks,
        "chunks_new": new_chunks,
        "chunks_reused": reused_chunks,
        "actions": actions[:500],
        "actions_truncated": len(actions) > 500,
    }


# -- optimization ------------------------------------------------------------------------------


def _check_source_hash(ctx: AppContext, path: str, file: LogicalFile) -> None:
    actual, length = sha256_stream(ctx.hdfs.open(path))
    if actual != file.sha256 or length != file.size_bytes:
        raise IntegrityFailure(
            f"{file.name}: bytes at {path} hash to {actual}, expected {file.sha256}. The file was left unchanged."
        )


def _convert_to_object(ctx: AppContext, session: Session, file: LogicalFile, verified: set[str]) -> str | None:
    """Returns a leftover raw path that may be deleted once the new pointer has committed."""
    sha = file.sha256
    raw_path = file.raw_path
    target = ctx.paths.object_path(sha)
    existing = content_store.find_active_object(session, sha)
    manifest = None if existing else content_store.find_active_manifest(session, sha)

    if existing is not None:
        if sha not in verified:
            _check_source_hash(ctx, existing.hdfs_path, file)
            verified.add(sha)
        file.content_object, file.storage_mode, file.raw_path = existing, StorageMode.FILE, None
        return raw_path
    if manifest is not None:
        if sha not in verified:
            actual, _ = sha256_stream(content_store.iter_file_bytes(ctx, session, _manifest_view(file, manifest)))
            if actual != sha:
                raise IntegrityFailure(f"{file.name}: existing manifest does not rebuild the file's hash.")
            verified.add(sha)
        file.manifest, file.storage_mode, file.raw_path = manifest, StorageMode.CHUNK, None
        return raw_path

    raw_exists = ctx.hdfs.status(raw_path) is not None
    target_exists = ctx.hdfs.status(target) is not None
    if target_exists:
        # Left by an earlier run that stopped between rename and commit. Adopt it only after proving it.
        _check_source_hash(ctx, target, file)
    elif raw_exists:
        _check_source_hash(ctx, raw_path, file)
        ctx.hdfs.mkdirs(parent_of(target))
        if not ctx.hdfs.rename(raw_path, target):
            raise IntegrityFailure(f"{file.name}: HDFS refused the rename to {target}.")
        raw_exists = False
    else:
        raise IntegrityFailure(f"{file.name}: content is missing from HDFS at {raw_path}.")
    verified.add(sha)
```

```
    file.content_object = content_store.register_object(session, sha, file.size_bytes, target)
    file.storage_mode, file.raw_path = StorageMode.FILE, None
    return raw_path if raw_exists else None


class _manifest_view:  # noqa: N801 - a small read-only stand-in for iter_file_bytes
    def __init__(self, file: LogicalFile, manifest):
        self.storage_mode = StorageMode.CHUNK
        self.manifest_id = manifest.id
        self.id = file.id


def _convert_to_chunks(
    ctx: AppContext, session: Session, file: LogicalFile, chunk_size: int, verified: set[str], replication: int | None
) -> tuple[str | None, int]:
    sha = file.sha256
    raw_path = file.raw_path if file.storage_mode == StorageMode.RAW else None
    build = content_store.build_manifest(
        ctx, session, ctx.hdfs.open(_source_path(file)), sha, file.size_bytes, chunk_size, replication=replication
    )
    session.flush()
    key = f"{sha}:{chunk_size}"
    if key not in verified:
        actual, length = sha256_stream(content_store.iter_file_bytes(ctx, session, _manifest_view(file, build.manifest)))
        if actual != sha or length != file.size_bytes:
            raise IntegrityFailure(f"{file.name}: rebuilding from chunks gave {actual}, expected {sha}.")
        verified.add(key)
    file.manifest = build.manifest
    file.storage_mode = StorageMode.CHUNK
    file.raw_path = None
    file.content_object = None
    return raw_path, build.chunks_new


def run_optimization(ctx: AppContext, job_id: str, params: dict, reporter) -> dict:
    mode = params["mode"]
    chunk_size = params.get("chunk_size")
    started = time.perf_counter()
    before = capture_metrics(ctx)
    with session_scope(ctx.session_factory) as session:
        file_ids =
list(session.scalars(select(LogicalFile.id).where(eligible_query(mode).whereclause).order_by(LogicalFile.created_at)))
        replication = settings_store.get_value(ctx, session, "target_replication")

    processed, errors = 0, []
    orphans_left: list[str] = []
    verified: set[str] = set()
    chunks_new = 0
    reporter.progress(0, len(file_ids), "Optimizing files")
    for index, file_id in enumerate(file_ids, start=1):
        try:
            with ctx.content_lock:
                leftover = None
                with session_scope(ctx.session_factory) as session:
                    file = session.get(LogicalFile, file_id)
                    if file is None or file.status != FileStatus.ACTIVE:
                        continue
                    if mode == StorageMode.FILE:
                        leftover = _convert_to_object(ctx, session, file, verified)
                    else:
                        leftover, new = _convert_to_chunks(ctx, session, file, chunk_size, verified, replication)
                        chunks_new += new
                    file.last_verified_ok = True
                    audit.record(session, "optimize.file_converted", f"{file.name} now stored as {file.storage_mode}.",
entity_type="file", entity_id=file.id)
                    analytics.record_snapshot(session, "optimize")
                if leftover:
                    # The pointer has committed, so this copy is redundant. Failing to remove it leaves an
                    # orphan for reconciliation to find; the file itself converted successfully.
                    try:
                        ctx.hdfs.delete(leftover)
                    except HdfsError as exc:
                        orphans_left.append(leftover)
                        audit.record_failure(
                            ctx.session_factory, "optimize.leftover_not_removed", f"{leftover}: {exc.message}", entity_type="file",
entity_id=file_id
                        )
            processed += 1
        except HdfsUnavailable:
            raise  # the cluster is gone, so continuing would only record misleading per-file failures
        except Exception as exc:  # noqa: BLE001 - per-file isolation: report it and continue with the rest
            message = getattr(exc, "message", str(exc))
            errors.append({"file_id": file_id, "error": message})
            audit.record_failure(ctx.session_factory, "optimize.file_failed", message, entity_type="file", entity_id=file_id)
        reporter.progress(index, len(file_ids), f"Processed {index} of {len(file_ids)}")

    reporter.progress(len(file_ids), len(file_ids), "Collecting content nothing refers to")
    gc_report = lifecycle.collect_garbage(ctx)
    after = capture_metrics(ctx)
    duration_ms = (time.perf_counter() - started) * 1000
    reclaimed = before["derived"]["stored_content_bytes"] - after["derived"]["stored_content_bytes"]
    with session_scope(ctx.session_factory) as session:
        session.add(
```

```
            OptimizationHistory(
                job_id=job_id,
                mode=mode,
                chunk_size=chunk_size,
                before=before,
                after=after,
                files_processed=processed,
                files_failed=len(errors),
                bytes_reclaimed_logical=reclaimed,
                duration_ms=duration_ms,
            )
        )
        audit.record_sample(session, "optimize", before["derived"]["logical_bytes"], duration_ms, mode=mode)
        audit.record(
            session,
            "optimize.completed",
            f"{mode}-level optimization: {processed} files converted, {len(errors)} failed, {reclaimed} bytes of stored content
reclaimed.",
            severity="warning" if errors else "info",
            entity_type="job",
            entity_id=job_id,
        )
    return {
        "estimated": False,
        "files_processed": processed,
        "files_failed": len(errors),
        "errors": errors,
        "orphans_left": orphans_left,
        "chunks_new": chunks_new,
        "before": before,
        "after": after,
        "bytes_reclaimed": reclaimed,
        "garbage_collected": {k: gc_report[k] for k in ("object_count", "manifest_count", "chunk_count", "reclaimable_bytes")},
        "duration_ms": duration_ms,
    }


def validate_analysis_for_optimize(ctx: AppContext, analysis_job_id: str) -> DeduplicationJob:
    """Optimization must follow a successful preview of the same, unchanged catalog."""
    with session_scope(ctx.session_factory) as session:
        job = session.get(DeduplicationJob, analysis_job_id)
        if job is None or job.kind != "analyze":
            raise NotFoundError(f"No analysis job with id {analysis_job_id}.")
        if job.status != JobStatus.SUCCEEDED:
            raise ConflictError("That analysis has not completed successfully, so there is no preview to act on.")
        if job.result["catalog_fingerprint"] != catalog_fingerprint(session):
            raise ConflictError("Files have changed since this preview. Run the analysis again and review the new estimate.")
        return job


def validate_mode(ctx: AppContext, mode: str, chunk_size: int | None) -> tuple[str, int | None]:
    if mode not in (StorageMode.FILE, StorageMode.CHUNK):
        raise ValidationFailure("Optimization mode must be 'file' or 'chunk'.")
    if mode == StorageMode.CHUNK:
        with session_scope(ctx.session_factory) as session:
            chunk_size = chunk_size or settings_store.get_value(ctx, session, "default_chunk_size")
        settings_store.validate_chunk_size(ctx, chunk_size)
        return mode, chunk_size
    return mode, None
```

backend/app/services/replication.py 209 lines

```
"""HDFS replication management.

Replication is fault tolerance, not waste. Deduplication works on identical content; it never removes
a block replica. Changing the factor is a separate, explicit decision with its own safety checks.
"""

from __future__ import annotations

import time

from sqlalchemy import select

from app.core.db import session_scope, utcnow
from app.core.errors import ConflictError, HdfsError, HdfsUnavailable, ValidationFailure
from app.hdfs.base import walk_files
from app.models.entities import ReplicationOperation
from app.services import audit, settings_store
from app.services.context import AppContext

MINIMUM_VERIFY_SECONDS = 180.0
VERIFY_POLL_SECONDS = 3.0
# HDFS defaults: each 3-second cycle schedules about 2 x live DataNodes blocks for re-replication.
REDUNDANCY_INTERVAL_SECONDS = 3.0
WORK_MULTIPLIER_PER_ITERATION = 2


def convergence_timeout(blocks: int, live_datanodes: int, *, safety_factor: float = 3.0) -> float:
    """How long to wait for HDFS, scaled to its re-replication rate, so that a large namespace of small
    chunk files is not reported as a failure while HDFS is still correctly working through it."""
    per_cycle = max(1, WORK_MULTIPLIER_PER_ITERATION * live_datanodes)
    expected = blocks / per_cycle * REDUNDANCY_INTERVAL_SECONDS
    return max(MINIMUM_VERIFY_SECONDS, expected * safety_factor)
```

```
def replication_status(ctx: AppContext) -> dict:
    cluster = ctx.monitor.status()
    with session_scope(ctx.session_factory) as session:
        target = settings_store.get_value(ctx, session, "target_replication")
        operations = [_operation_dict(op) for op in
session.scalars(select(ReplicationOperation).order_by(ReplicationOperation.requested_at.desc()).limit(20))]
    if not cluster["reachable"]:
        return {"reachable": False, "error": cluster.get("error"), "target_replication": target, "operations": operations}

    cluster_default = ctx.hdfs.default_replication()
    effective = target or cluster_default
    live = cluster["datanodes"]["live"]
    fsck = ctx.monitor.fsck(ctx.paths.root)
    summary = ctx.hdfs.content_summary(ctx.paths.root)
    warnings = []
    if effective > live:
        warnings.append(
            f"The target of {effective} replicas exceeds the {live} live DataNodes, so blocks cannot be fully replicated."
        )
    if fsck.get("under_replicated_blocks"):
        count = fsck["under_replicated_blocks"]
        warnings.append("1 managed block is under-replicated." if count == 1 else f"{count} managed blocks are under-replicated.")
    if effective == 1:
        warnings.append("With one replica, losing a single DataNode can lose data.")
    return {
        "reachable": True,
        "cluster_default_replication": cluster_default,
        "target_replication": target,
        "effective_replication": effective,
        "live_datanodes": live,
        "dead_datanodes": cluster["datanodes"]["dead"],
        "max_supported_replication": live,
        "fsck": fsck,
        "stored_length_bytes": summary.length,
        "space_consumed_bytes": summary.space_consumed,
        "measured_replication_multiplier": (summary.space_consumed / summary.length) if summary.length else None,
        "warnings": warnings,
        "operations": operations,
        "checked_at": cluster["checked_at"],
    }


def _operation_dict(op: ReplicationOperation) -> dict:
    return {
        "id": op.id,
        "previous_factor": op.previous_factor,
        "requested_factor": op.requested_factor,
        "status": op.status,
        "files_total": op.files_total,
        "files_changed": op.files_changed,
        "space_consumed_before": op.space_consumed_before,
        "space_consumed_after": op.space_consumed_after,
        "details": op.details,
        "error": op.error,
        "requested_at": op.requested_at,
        "completed_at": op.completed_at,
    }


def request_change(ctx: AppContext, factor: int, *, acknowledge_reduced_protection: bool = False) -> dict:
    """Validate against live cluster state, then apply and verify in a background job."""
    if factor < 1:
        raise ValidationFailure("Replication factor must be at least 1.")
    cluster = ctx.monitor.status()
    if not cluster["reachable"]:
        raise HdfsUnavailable(f"Cannot change replication while the NameNode is unreachable: {cluster.get('error')}")
    live = cluster["datanodes"]["live"]
    if factor > live:
        raise ValidationFailure(
            f"Replication factor {factor} cannot be satisfied by {live} live DataNodes. HDFS would leave "
            "blocks permanently under-replicated.",
            details={"live_datanodes": live},
        )
    with session_scope(ctx.session_factory) as session:
        current = settings_store.get_value(ctx, session, "target_replication") or ctx.hdfs.default_replication()
    if factor < current and not acknowledge_reduced_protection:
        raise ConflictError(
            f"Lowering replication from {current} to {factor} reduces fault tolerance. Confirm this explicitly to continue.",
            details={"current": current, "requested": factor},
        )
    with session_scope(ctx.session_factory) as session:
        operation = ReplicationOperation(previous_factor=current, requested_factor=factor)
        session.add(operation)
        session.flush()
        operation_id = operation.id
        audit.record(session, "replication.requested", f"Replication change requested: {current} to {factor}.",
entity_type="replication", entity_id=operation_id)
    job_id = ctx.jobs.submit("replication", params={"operation_id": operation_id, "factor": factor})
    return {"operation_id": operation_id, "job_id": job_id, "previous_factor": current, "requested_factor": factor}
```

```
def run_replication_change(ctx: AppContext, job_id: str, params: dict, reporter) -> dict:
    operation_id, factor = params["operation_id"], params["factor"]
    try:
        before = ctx.hdfs.content_summary(ctx.paths.root)
        files = list(walk_files(ctx.hdfs, ctx.paths.root))
        reporter.progress(0, len(files), f"Requesting replication {factor} on {len(files)} files")
        changed = 0
        for index, entry in enumerate(files, start=1):
            if entry.replication != factor and ctx.hdfs.set_replication(entry.path, factor):
                changed += 1
            if index % 50 == 0 or index == len(files):
                reporter.progress(index, len(files), f"Requested on {index} of {len(files)} files")
        with session_scope(ctx.session_factory) as session:
            settings_store.update(ctx, session, {"target_replication": factor})
            operation = session.get(ReplicationOperation, operation_id)
            operation.status = "applied"
            operation.files_total, operation.files_changed = len(files), changed
            operation.space_consumed_before = before.space_consumed

        verified, fsck, block_sample = _wait_until_converged(ctx, factor, files, reporter)
        after = ctx.hdfs.content_summary(ctx.paths.root)
        with session_scope(ctx.session_factory) as session:
            operation = session.get(ReplicationOperation, operation_id)
            operation.status = "verified" if verified else "timed_out"
            operation.space_consumed_after = after.space_consumed
            operation.details = {"fsck": fsck, "block_sample": block_sample}
            operation.completed_at = utcnow()
            audit.record(
                session,
                "replication.verified" if verified else "replication.timed_out",
                f"Replication {factor}: "
                + ("every block has the requested number of replicas." if verified else "blocks did not converge before the
timeout."),
                severity="info" if verified else "warning",
                entity_type="replication",
                entity_id=operation_id,
            )
        return {
            "verified": verified,
            "files_total": len(files),
            "files_changed": changed,
            "space_consumed_before": before.space_consumed,
            "space_consumed_after": after.space_consumed,
            "fsck": fsck,
        }
    except HdfsError as exc:
        with session_scope(ctx.session_factory) as session:
            operation = session.get(ReplicationOperation, operation_id)
            operation.status = "failed"
            operation.error = exc.message
            operation.completed_at = utcnow()
        raise


def _wait_until_converged(ctx: AppContext, factor: int, files: list, reporter) -> tuple[bool, dict, list]:
    """HDFS re-replicates asynchronously, so success is observed rather than assumed.

    fsck stops counting a replica as soon as it is marked excess, which is before the DataNode deletes it.
    Convergence therefore also requires the block locations themselves to show the requested count.
    """
    live = ctx.monitor.status().get("datanodes", {}).get("live", 1)
    deadline = time.monotonic() + convergence_timeout(len(files), live)
    fsck: dict = {}
    while True:
        fsck = ctx.monitor.fsck(ctx.paths.root)
        converged = (
            fsck.get("under_replicated_blocks") == 0
            and fsck.get("over_replicated_blocks") == 0
            and fsck.get("missing_blocks") == 0
            and (fsck.get("total_blocks") == 0 or fsck.get("average_block_replication") == float(factor))
        )
        sample = _sample_block_locations(ctx, files)
        if converged and all(len(block["hosts"]) == factor for block in sample):
            return True, fsck, sample
        if time.monotonic() >= deadline:
            return False, fsck, sample
        reporter.progress(len(files), len(files), f"Waiting for HDFS to reach {factor} replicas (average
{fsck.get('average_block_replication')})")
        time.sleep(VERIFY_POLL_SECONDS)


def _sample_block_locations(ctx: AppContext, files: list, limit: int = 25) -> list[dict]:
    sample = []
    for entry in [f for f in files if f.length > 0][:limit]:
        for block in ctx.hdfs.block_locations(entry.path):
            sample.append({"path": entry.path, "block_id": block.block_id, "length": block.length, "hosts": list(block.hosts)})
    return sample
```

backend/app/services/settings_store.py 68 lines

```
"""Runtime settings persisted in the metadata database, with environment defaults."""

from __future__ import annotations
```

```
from sqlalchemy.orm import Session

from app.core.errors import ValidationFailure
from app.models.entities import AppSetting
from app.services.context import AppContext

INGEST_MODES = ("raw", "file", "chunk")


def _defaults(ctx: AppContext) -> dict:
    return {
        "default_ingest_mode": ctx.settings.default_ingest_mode,
        "default_chunk_size": ctx.settings.default_chunk_size,
        "verify_on_write": ctx.settings.verify_on_write,
        # None means "use the cluster's dfs.replication".
        "target_replication": None,
    }


def get_all(ctx: AppContext, session: Session) -> dict:
    values = _defaults(ctx)
    for row in session.query(AppSetting).all():
        if row.key in values:
            values[row.key] = row.value
    return values


def get_value(ctx: AppContext, session: Session, key: str):
    return get_all(ctx, session)[key]


def validate_chunk_size(ctx: AppContext, chunk_size: int) -> int:
    low, high = ctx.settings.min_chunk_size, ctx.settings.max_chunk_size
    if not low <= chunk_size <= high:
        raise ValidationFailure(f"Chunk size must be between {low} and {high} bytes, got {chunk_size}.")
    return chunk_size


def validate_mode(mode: str) -> str:
    if mode not in INGEST_MODES:
        raise ValidationFailure(f"Mode must be one of {', '.join(INGEST_MODES)}, got {mode!r}.")
    return mode


def update(ctx: AppContext, session: Session, changes: dict) -> dict:
    """Validate and save changes. These apply to future writes and never rewrite stored data."""
    if "default_ingest_mode" in changes:
        validate_mode(changes["default_ingest_mode"])
    if "default_chunk_size" in changes:
        validate_chunk_size(ctx, changes["default_chunk_size"])
    if "verify_on_write" in changes and not isinstance(changes["verify_on_write"], bool):
        raise ValidationFailure("verify_on_write must be true or false.")
    allowed = set(_defaults(ctx))
    for key, value in changes.items():
        if key not in allowed:
            raise ValidationFailure(f"Unknown setting: {key}")
        row = session.get(AppSetting, key)
        if row is None:
            session.add(AppSetting(key=key, value=value))
        else:
            row.value = value
    session.flush()
    return get_all(ctx, session)
```

11.4 Backend: API routes

backend/app/api/dedup.py 214 lines

```
"""Duplicate analysis, optimization jobs and garbage collection."""

from __future__ import annotations

from fastapi import APIRouter, Depends, Query
from sqlalchemy import func, select

from app.api.deps import get_ctx
from app.core.db import session_scope
from app.core.errors import ConflictError, NotFoundError
from app.models.entities import (
    Chunk,
    ContentState,
    DeduplicationJob,
    FileChunk,
    FileManifest,
    FileStatus,
    JobStatus,
    LogicalFile,
    StorageMode,
)
from app.schemas.api import AnalyzeRequest, JobAccepted, JobOut, OptimizeRequest
from app.services import lifecycle, optimizer
from app.services.context import AppContext

router = APIRouter(prefix="/api/v1/dedup", tags=["deduplication"])
```

```
CHUNK_MAP_CELLS = 240


def _groups(ctx: AppContext, limit: int) -> dict:
    """Exact duplicates: logical files with the same SHA-256, regardless of name."""
    with session_scope(ctx.session_factory) as session:
        grouped = session.execute(
            select(LogicalFile.sha256, func.count().label("copies"), func.max(LogicalFile.size_bytes).label("size"))
            .where(LogicalFile.status == FileStatus.ACTIVE)
            .group_by(LogicalFile.sha256)
            .having(func.count() > 1)
            .order_by((func.max(LogicalFile.size_bytes) * (func.count() - 1)).desc())
        ).all()
        groups = []
        for sha, copies, size in grouped[:limit]:
            members = list(
                session.scalars(
                    select(LogicalFile).where(LogicalFile.sha256 == sha, LogicalFile.status ==
FileStatus.ACTIVE).order_by(LogicalFile.created_at)
                )
            )
            raw_members = sum(1 for m in members if m.storage_mode == StorageMode.RAW)
            stored_forms = raw_members + (1 if raw_members < len(members) else 0)
            groups.append(
                {
                    "sha256": sha,
                    "copies": copies,
                    "size_bytes": size,
                    "logical_bytes": size * copies,
                    "redundant_logical_bytes": size * (copies - 1),
                    # Bytes optimization could still free. Zero once the group is already deduplicated.
                    "reclaimable_bytes": size * max(stored_forms - 1, 0),
                    "already_deduplicated": stored_forms <= 1,
                    "members": [
                        {
                            "id": m.id,
                            "name": m.name,
                            "folder": m.folder,
                            "storage_mode": m.storage_mode,
                            "provenance_label": m.provenance_label,
                            "created_at": m.created_at,
                        }
                        for m in members
                    ],
                }
            )
        return {
            "group_count": len(grouped),
            "duplicate_files": sum(copies - 1 for _, copies, _ in grouped),
            "redundant_logical_bytes": sum(size * (copies - 1) for _, copies, size in grouped),
            "groups": groups,
        }


@router.get("/groups")
def duplicate_groups(limit: int = Query(100, ge=1, le=500), ctx: AppContext = Depends(get_ctx)) -> dict:
    return _groups(ctx, limit)


@router.get("/duplicates")
def duplicates(limit: int = Query(100, ge=1, le=500), ctx: AppContext = Depends(get_ctx)) -> dict:
    data = _groups(ctx, limit)
    return {"duplicate_files": data["duplicate_files"], "groups": data["groups"]}


@router.get("/shared-chunks")
def shared_chunks(limit: int = Query(100, ge=1, le=500), ctx: AppContext = Depends(get_ctx)) -> dict:
    """Chunks used by more than one manifest: shared content inside files that are not identical overall."""
    with session_scope(ctx.session_factory) as session:
        shared = session.execute(
            select(Chunk.sha256, Chunk.size_bytes, func.count(func.distinct(FileChunk.manifest_id)).label("manifests"),
func.count().label("references"))
            .join(FileChunk, FileChunk.chunk_id == Chunk.id)
            .join(FileManifest, FileManifest.id == FileChunk.manifest_id)
            .where(Chunk.state == ContentState.ACTIVE, FileManifest.state == ContentState.ACTIVE)
            .group_by(Chunk.id)
            .having(func.count() > 1)
            .order_by(func.count().desc())
        ).all()
        manifests = list(
            session.scalars(select(FileManifest).where(FileManifest.state == ContentState.ACTIVE).order_by(FileManifest.created_at))
        )
        usage = dict(
            session.execute(
                select(FileChunk.chunk_id, func.count(func.distinct(FileChunk.manifest_id)))
                .join(FileManifest, FileManifest.id == FileChunk.manifest_id)
                .where(FileManifest.state == ContentState.ACTIVE)
                .group_by(FileChunk.chunk_id)
            ).all()
        )
        map_rows = []
        for manifest in manifests[:limit]:
```

```
            names = list(
                session.scalars(
                    select(LogicalFile.name).where(LogicalFile.manifest_id == manifest.id, LogicalFile.status == FileStatus.ACTIVE)
                )
            )
            cells = list(
                session.scalars(
                    select(FileChunk.chunk_id)
                    .where(FileChunk.manifest_id == manifest.id)
                    .order_by(FileChunk.ordinal)
                    .limit(CHUNK_MAP_CELLS)
                )
            )
            map_rows.append(
                {
                    "id": manifest.id,
                    "file_sha256": manifest.file_sha256,
                    "file_size": manifest.file_size,
                    "chunk_size": manifest.chunk_size,
                    "chunk_count": manifest.chunk_count,
                    "logical_files": names,
                    # True where another distinct manifest also uses this chunk.
                    "chunk_map": [usage.get(chunk_id, 1) > 1 for chunk_id in cells],
                }
            )
        return {
            "shared_chunk_count": len(shared),
            "bytes_saved_by_sharing": sum(size * (references - 1) for _, size, _, references in shared),
            "chunks": [
                {"sha256": sha, "size_bytes": size, "manifests": count, "references": references}
                for sha, size, count, references in shared[:limit]
            ],
            "manifests": map_rows,
            "chunk_map_limit": CHUNK_MAP_CELLS,
        }


@router.post("/analyze", response_model=JobAccepted, status_code=202)
def analyze(body: AnalyzeRequest, ctx: AppContext = Depends(get_ctx)) -> JobAccepted:
    """Dry run. Reports estimated savings and changes nothing in HDFS."""
    mode, chunk_size = optimizer.validate_mode(ctx, body.mode, body.chunk_size)
    return JobAccepted(job_id=ctx.jobs.submit("analyze", mode=mode, chunk_size=chunk_size))


@router.post("/optimize", response_model=JobAccepted, status_code=202)
def optimize(body: OptimizeRequest, ctx: AppContext = Depends(get_ctx)) -> JobAccepted:
    """Start optimization. Requires a successful preview of the catalog exactly as it is now."""
    analysis = optimizer.validate_analysis_for_optimize(ctx, body.analysis_job_id)
    params = {"analysis_job_id": analysis.id}
    return JobAccepted(job_id=ctx.jobs.submit("optimize", mode=analysis.mode, chunk_size=analysis.chunk_size, params=params))


@router.get("/jobs", response_model=list[JobOut])
def list_jobs(
    kind: str | None = None,
    limit: int = Query(30, ge=1, le=200),
    ctx: AppContext = Depends(get_ctx),
) -> list[JobOut]:
    with session_scope(ctx.session_factory) as session:
        query = select(DeduplicationJob)
        if kind:
            query = query.where(DeduplicationJob.kind == kind)
        rows = session.scalars(query.order_by(DeduplicationJob.created_at.desc()).limit(limit))
        return [JobOut.model_validate(row) for row in rows]


@router.get("/jobs/{job_id}", response_model=JobOut)
def get_job(job_id: str, ctx: AppContext = Depends(get_ctx)) -> JobOut:
    with session_scope(ctx.session_factory) as session:
        job = session.get(DeduplicationJob, job_id)
        if job is None:
            raise NotFoundError(f"No job with id {job_id}.")
        return JobOut.model_validate(job)


@router.post("/jobs/{job_id}/resume", response_model=JobAccepted, status_code=202)
def resume_job(job_id: str, ctx: AppContext = Depends(get_ctx)) -> JobAccepted:
    """Run an interrupted or failed optimization again. Finished files are not eligible, so none repeat."""
    with session_scope(ctx.session_factory) as session:
        job = session.get(DeduplicationJob, job_id)
        if job is None or job.kind != "optimize":
            raise NotFoundError(f"No optimization job with id {job_id}.")
        if job.status not in (JobStatus.INTERRUPTED, JobStatus.FAILED, JobStatus.PARTIAL):
            raise ConflictError(f"Only an interrupted, failed or partial job can be resumed; this one is {job.status}.")
        mode, chunk_size = job.mode, job.chunk_size
    return JobAccepted(job_id=ctx.jobs.submit("optimize", mode=mode, chunk_size=chunk_size, resumed_from_id=job_id))


@router.get("/gc")
def preview_garbage(ctx: AppContext = Depends(get_ctx)) -> dict:
    return lifecycle.collect_garbage(ctx, dry_run=True)
```

```
@router.post("/gc")
def run_garbage_collection(ctx: AppContext = Depends(get_ctx)) -> dict:
    return lifecycle.collect_garbage(ctx)
```

backend/app/api/deps.py 20 lines

```
"""Shared API dependencies."""

from __future__ import annotations

from fastapi import Request

from app.core.errors import UnauthorizedError
from app.services.context import AppContext


def get_ctx(request: Request) -> AppContext:
    return request.app.state.ctx


def require_token(request: Request) -> None:
    """Local access control. When a token is configured, requests that change state must present it."""
    expected = request.app.state.ctx.settings.api_token
    if expected and request.method not in {"GET", "HEAD", "OPTIONS"} and request.headers.get("x-api-key") != expected:
        raise UnauthorizedError("This action requires a valid X-API-Key header.")
```

backend/app/api/files.py 299 lines

```
"""Logical file endpoints: upload, list, inspect, download, delete, restore, verify."""

from __future__ import annotations

import hashlib
import tempfile
import time
from pathlib import Path
from urllib.parse import quote

from fastapi import APIRouter, Depends, File, Form, Query, UploadFile
from fastapi.responses import StreamingResponse
from sqlalchemy import func, or_, select

from app.api.deps import get_ctx
from app.core.db import session_scope
from app.core.errors import HdfsError, PayloadTooLarge
from app.models.entities import FileChunk, FileStatus, LogicalFile, StorageMode
from app.schemas.api import (
    FileDetail,
    FileOut,
    FilePage,
    IntegrityOut,
    JobAccepted,
    UploadResult,
)
from app.services import content_store, datasets, ingest, integrity, lifecycle
from app.services.context import AppContext

router = APIRouter(prefix="/api/v1/files", tags=["files"])

SORT_COLUMNS = {
    "name": LogicalFile.name,
    "size": LogicalFile.size_bytes,
    "created": LogicalFile.created_at,
    "mode": LogicalFile.storage_mode,
}
COPY_BUFFER = 1024 * 1024


def _reference_counts(session, hashes: list[str]) -> dict[str, int]:
    if not hashes:
        return {}
    return dict(
        session.execute(
            select(LogicalFile.sha256, func.count())
            .where(LogicalFile.sha256.in_(hashes), LogicalFile.status != FileStatus.PURGED)
            .group_by(LogicalFile.sha256)
        ).all()
    )


def _to_out(record: LogicalFile, references: int) -> FileOut:
    out = FileOut.model_validate(record)
    out.logical_references = references
    out.is_duplicate = references > 1
    return out


@router.post("/upload", response_model=UploadResult, status_code=201)
def upload_file(
    file: UploadFile = File(...),
    folder: str = Form("/"),
    mode: str | None = Form(None, description="raw, file or chunk; defaults to the configured mode"),
    chunk_size: int | None = Form(None),
    ctx: AppContext = Depends(get_ctx),
) -> UploadResult:
```

```
    """Spool to disk while hashing, so memory use does not grow with file size, then ingest."""
    name = ingest.validate_filename(file.filename)
    folder = ingest.normalize_folder(folder)
    ctx.settings.spool_dir.mkdir(parents=True, exist_ok=True)
    digest = hashlib.sha256()
    size = 0
    hash_seconds = 0.0
    with tempfile.NamedTemporaryFile(dir=ctx.settings.spool_dir, prefix="upload-", delete=False) as spool:
        spool_path = Path(spool.name)
        try:
            while block := file.file.read(COPY_BUFFER):
                size += len(block)
                if size > ctx.settings.max_upload_bytes:
                    raise PayloadTooLarge(f"Upload exceeds the limit of {ctx.settings.max_upload_bytes} bytes.")
                started = time.perf_counter()
                digest.update(block)
                hash_seconds += time.perf_counter() - started
                spool.write(block)
        except Exception:
            spool_path.unlink(missing_ok=True)
            raise
    try:
        result = ingest.ingest_local_file(
            ctx,
            spool_path,
            name,
            folder=folder,
            mode=mode,
            chunk_size=chunk_size,
            known_hash=(digest.hexdigest(), size),
            known_hash_ms=hash_seconds * 1000,
        )
    finally:
        spool_path.unlink(missing_ok=True)
    with session_scope(ctx.session_factory) as session:
        counts = _reference_counts(session, [result.file.sha256])
        file_out = _to_out(session.get(LogicalFile, result.file.id), counts.get(result.file.sha256, 1))
    return UploadResult(
        file=file_out,
        outcome=result.outcome,
        duplicate_of_existing=result.duplicate_of_existing,
        bytes_written=result.bytes_written,
        chunks_total=result.chunks_total,
        chunks_new=result.chunks_new,
        chunks_reused=result.chunks_reused,
        hash_ms=result.hash_ms,
        store_ms=result.store_ms,
        total_ms=result.total_ms,
    )


@router.get("", response_model=FilePage)
def list_files(
    folder: str | None = None,
    recursive: bool = False,
    q: str | None = Query(None, description="Matches name or SHA-256 prefix"),
    status: str = Query("active", pattern="^(active|deleted)$"),
    mode: str | None = Query(None, pattern="^(raw|file|chunk)$"),
    extension: str | None = None,
    duplicates_only: bool = False,
    sort: str = Query("created", pattern="^(name|size|created|mode)$"),
    order: str = Query("desc", pattern="^(asc|desc)$"),
    page: int = Query(1, ge=1),
    page_size: int = Query(50, ge=1, le=200),
    ctx: AppContext = Depends(get_ctx),
) -> FilePage:
    with session_scope(ctx.session_factory) as session:
        query = select(LogicalFile).where(LogicalFile.status == status)
        if folder is not None:
            normalized = ingest.normalize_folder(folder)
            if recursive and normalized != "/":
                query = query.where(or_(LogicalFile.folder == normalized, LogicalFile.folder.like(f"{normalized}/%")))
            elif not recursive:
                query = query.where(LogicalFile.folder == normalized)
        if q:
            like = f"%{q.replace('%', '').replace('_', '')}%"
            query = query.where(or_(LogicalFile.name.ilike(like), LogicalFile.sha256.like(f"{q.lower()}%")))
        if mode:
            query = query.where(LogicalFile.storage_mode == mode)
        if extension:
            query = query.where(LogicalFile.extension == extension.lower())
        if duplicates_only:
            duplicated = (
                select(LogicalFile.sha256)
                .where(LogicalFile.status != FileStatus.PURGED)
                .group_by(LogicalFile.sha256)
                .having(func.count() > 1)
            )
            query = query.where(LogicalFile.sha256.in_(duplicated))
        total = session.scalar(select(func.count()).select_from(query.subquery()))
        column = SORT_COLUMNS[sort]
        ordered = column.asc() if order == "asc" else column.desc()
        rows = list(session.scalars(query.order_by(ordered, LogicalFile.id).offset((page - 1) * page_size).limit(page_size)))
```

```
        counts = _reference_counts(session, list({r.sha256 for r in rows}))
        return FilePage(
            items=[_to_out(r, counts.get(r.sha256, 1)) for r in rows],
            total=total,
            page=page,
            page_size=page_size,
        )


@router.get("/folders")
def list_folders(ctx: AppContext = Depends(get_ctx)) -> dict:
    """Virtual folders with their direct file counts and sizes."""
    with session_scope(ctx.session_factory) as session:
        rows = session.execute(
            select(LogicalFile.folder, func.count(), func.coalesce(func.sum(LogicalFile.size_bytes), 0))
            .where(LogicalFile.status == FileStatus.ACTIVE)
            .group_by(LogicalFile.folder)
            .order_by(LogicalFile.folder)
        ).all()
    return {"folders": [{"path": path, "files": files, "bytes": size} for path, files, size in rows]}


@router.post("/verify-all", response_model=JobAccepted, status_code=202)
def verify_all(ctx: AppContext = Depends(get_ctx)) -> JobAccepted:
    """Read every active file back from HDFS and compare SHA-256, as a background job."""
    return JobAccepted(job_id=ctx.jobs.submit("verify_all"))


@router.get("/{file_id}", response_model=FileDetail)
def get_file(file_id: str, ctx: AppContext = Depends(get_ctx)) -> FileDetail:
    with session_scope(ctx.session_factory) as session:
        record = integrity.get_file(session, file_id)
        siblings = list(
            session.scalars(
                select(LogicalFile).where(LogicalFile.sha256 == record.sha256, LogicalFile.status !=
FileStatus.PURGED).order_by(LogicalFile.created_at)
            )
        )
        base = _to_out(record, len(siblings)).model_dump()
        chunks: list[dict] = []
        paths: list[str] = []
        chunk_size = chunk_count = None
        if record.storage_mode == StorageMode.CHUNK:
            manifest = record.manifest
            chunk_size, chunk_count = manifest.chunk_size, manifest.chunk_count
            paths.append(manifest.hdfs_path)
            ordered = content_store.manifest_chunks(session, manifest.id)
            usage = dict(
                session.execute(
                    select(FileChunk.chunk_id, func.count(func.distinct(FileChunk.manifest_id)))
                    .where(FileChunk.chunk_id.in_([c.id for c in ordered[:200]]))
                    .group_by(FileChunk.chunk_id)
                ).all()
            )
            offset = 0
            for index, chunk in enumerate(ordered[:200]):
                chunks.append(
                    {
                        "ordinal": index,
                        "offset_bytes": offset,
                        "sha256": chunk.sha256,
                        "size_bytes": chunk.size_bytes,
                        "shared_by_manifests": usage.get(chunk.id, 1),
                    }
                )
                offset += chunk.size_bytes
        elif record.storage_mode == StorageMode.FILE:
            paths.append(record.content_object.hdfs_path)
        elif record.raw_path:
            paths.append(record.raw_path)

        locations = None
        try:
            if paths and record.storage_mode != StorageMode.CHUNK:
                locations = [
                    {"block_id": b.block_id, "length": b.length, "hosts": list(b.hosts), "corrupt": b.corrupt}
                    for b in ctx.hdfs.block_locations(paths[0])
                ]
        except HdfsError:
            locations = None  # the detail view must still open when HDFS is down

        return FileDetail(
            **base,
            hdfs_paths=paths,
            same_content_files=[
                {"id": s.id, "name": s.name, "folder": s.folder, "status": s.status, "provenance_label": s.provenance_label}
                for s in siblings
                if s.id != record.id
            ],
            chunk_size=chunk_size,
            chunk_count=chunk_count,
            chunks=chunks,
            integrity_history=[IntegrityOut.model_validate(c) for c in integrity.latest_checks(session, file_id)],
```

```
            block_locations=locations,
            dataset_source=datasets.source_dict(record.dataset_source) if record.dataset_source else None,
        )


@router.get("/{file_id}/download")
def download_file(file_id: str, ctx: AppContext = Depends(get_ctx)) -> StreamingResponse:
    with session_scope(ctx.session_factory) as session:
        record = integrity.get_file(session, file_id)
        integrity.preflight(ctx, session, record)
        name, size, sha, mime = record.name, record.size_bytes, record.sha256, record.mime_type
    return StreamingResponse(
        integrity.stream_verified(ctx, file_id),
        media_type=mime or "application/octet-stream",
        headers={
            "Content-Length": str(size),
            "Content-Disposition": f"attachment; filename*=UTF-8''{quote(name)}",
            "X-Content-SHA256": sha,
        },
    )


@router.delete("/{file_id}")
def delete_file(file_id: str, permanent: bool = False, ctx: AppContext = Depends(get_ctx)) -> dict:
    """Soft delete by default. ``permanent`` applies only to an already-deleted file."""
    if permanent:
        gc_report = lifecycle.purge(ctx, file_id)
        return {
            "status": "purged",
            "content_removed": {k: gc_report[k] for k in ("object_count", "manifest_count", "chunk_count", "reclaimable_bytes")},
        }
    record = lifecycle.soft_delete(ctx, file_id)
    return {"status": "deleted", "id": record.id, "recoverable": True}


@router.post("/{file_id}/restore")
def restore_file(file_id: str, ctx: AppContext = Depends(get_ctx)) -> dict:
    record = lifecycle.restore(ctx, file_id)
    return {"status": record.status, "id": record.id, "name": record.name}


@router.post("/{file_id}/verify", response_model=IntegrityOut)
def verify_file(file_id: str, ctx: AppContext = Depends(get_ctx)) -> IntegrityOut:
    check = integrity.verify_file(ctx, file_id)
    return IntegrityOut.model_validate(check)
```

backend/app/api/system.py 405 lines

```
"""Health, storage statistics, cluster state, analytics, datasets, experiments, audit and settings."""

from __future__ import annotations

import time
import uuid
from pathlib import Path

from fastapi import APIRouter, Body, Depends, Query, Response
from sqlalchemy import func, select, text

from app.api.deps import get_ctx
from app.core.db import session_scope
from app.core.errors import HdfsError, NotFoundError, ValidationFailure
from app.models.entities import (
    AuditEvent,
    Chunk,
    ContentObject,
    ContentState,
    DatasetSource,
    ExperimentRun,
    FileManifest,
)
from app.schemas.api import (
    AuditOut,
    AuditPage,
    DatasetImportRequest,
    DatasetIngestRequest,
    ExperimentRequest,
    JobAccepted,
    ReplicationRequest,
    SettingsUpdate,
)
from app.services import (
    analytics,
    audit,
    datasets,
    experiments,
    lifecycle,
    replication,
    settings_store,
)
from app.services.context import AppContext
from app.services.hashing import sha256_bytes
```

```
router = APIRouter(prefix="/api/v1")


# -- health ---------------------------------------------------------------------------------------


@router.get("/health", tags=["health"])
def health(ctx: AppContext = Depends(get_ctx)) -> dict:
    """The API is up. HDFS state is reported separately, so a cluster outage stays visible in the UI."""
    with session_scope(ctx.session_factory) as session:
        session.execute(text("SELECT 1"))
    cluster = ctx.monitor.status()
    return {"status": "ok", "database": "ok", "hdfs_reachable": cluster["reachable"], "hdfs_health": cluster.get("health")}


@router.get("/system/status", tags=["health"])
def system_status(ctx: AppContext = Depends(get_ctx)) -> dict:
    cluster = ctx.monitor.status()
    return {
        "api": "ok",
        "root_path": ctx.paths.root,
        "namenode_url": ctx.settings.namenode_url,
        "cluster": cluster,
    }


@router.get("/system/diagnostics", tags=["health"])
def diagnostics(ctx: AppContext = Depends(get_ctx)) -> dict:
    """Run real checks, including a write, read and delete cycle through a DataNode."""
    checks = []

    def check(name: str, function) -> None:
        started = time.perf_counter()
        try:
            detail = function()
            checks.append({"name": name, "ok": True, "detail": detail, "ms": (time.perf_counter() - started) * 1000})
        except Exception as exc:  # noqa: BLE001 - a diagnostic reports failures as data
            checks.append({"name": name, "ok": False, "detail": getattr(exc, "message", str(exc)), "ms": (time.perf_counter() -
started) * 1000})

    def database() -> str:
        with session_scope(ctx.session_factory) as session:
            result = session.execute(text("PRAGMA integrity_check")).scalar_one()
        if result != "ok":
            raise ValidationFailure(result)
        return "SQLite integrity_check returned ok"

    def namenode() -> str:
        status = ctx.monitor.status()
        if not status["reachable"]:
            raise HdfsError(status["error"])
        return f"{status['namenode']['state']}, Hadoop {status['namenode']['software_version']}, {status['datanodes']['live']} live
DataNodes"

    def write_cycle() -> str:
        payload = f"diagnostic {uuid.uuid4()}".encode()
        path = ctx.paths.staging_path(f"diagnostic-{uuid.uuid4().hex}")
        ctx.hdfs.create(path, payload, length=len(payload))
        try:
            data = b"".join(ctx.hdfs.open(path))
        finally:
            ctx.hdfs.delete(path)
        if sha256_bytes(data) != sha256_bytes(payload):
            raise HdfsError("Bytes read from HDFS differ from bytes written.")
        return f"Wrote, read back and deleted {len(payload)} bytes through WebHDFS"

    def spool() -> str:
        directory: Path = ctx.settings.spool_dir
        directory.mkdir(parents=True, exist_ok=True)
        probe = directory / f".probe-{uuid.uuid4().hex}"
        probe.write_bytes(b"ok")
        probe.unlink()
        return str(directory)

    check("Metadata database", database)
    check("NameNode connection", namenode)
    check("HDFS write, read and delete", write_cycle)
    check("Upload spool directory", spool)
    return {"ok": all(c["ok"] for c in checks), "checks": checks}


# -- storage --------------------------------------------------------------------------------------


@router.get("/storage/statistics", tags=["storage"])
def storage_statistics(ctx: AppContext = Depends(get_ctx)) -> dict:
    with session_scope(ctx.session_factory) as session:
        return analytics.full_statistics(ctx, session)


@router.get("/storage/savings", tags=["storage"])
def storage_savings(ctx: AppContext = Depends(get_ctx)) -> dict:
    with session_scope(ctx.session_factory) as session:
```

```
        stats = analytics.storage_statistics(session)
        return {
            "logical_bytes": stats["logical_bytes"],
            "stored_content_bytes": stats["stored_content_bytes"],
            "bytes_avoided": stats["bytes_avoided"],
            "savings_percent": stats["savings_percent"],
            "dedup_ratio": stats["dedup_ratio"],
            "history": analytics.history(session),
        }


@router.get("/storage/objects", tags=["storage"])
def storage_objects(
    kind: str = Query("objects", pattern="^(objects|chunks|manifests)$"),
    page: int = Query(1, ge=1),
    page_size: int = Query(50, ge=1, le=200),
    ctx: AppContext = Depends(get_ctx),
) -> dict:
    model = {"objects": ContentObject, "chunks": Chunk, "manifests": FileManifest}[kind]
    with session_scope(ctx.session_factory) as session:
        query = select(model).where(model.state == ContentState.ACTIVE)
        total = session.scalar(select(func.count()).select_from(query.subquery()))
        rows = session.scalars(query.order_by(model.created_at.desc()).offset((page - 1) * page_size).limit(page_size))
        items = [
            {
                "id": row.id,
                "sha256": row.sha256 if kind != "manifests" else row.file_sha256,
                "size_bytes": row.size_bytes if kind != "manifests" else row.manifest_bytes,
                "hdfs_path": row.hdfs_path,
                "created_at": row.created_at,
            }
            for row in rows
        ]
    return {"items": items, "total": total, "page": page, "page_size": page_size}


@router.get("/storage/reconcile", tags=["storage"])
def reconcile_report(ctx: AppContext = Depends(get_ctx)) -> dict:
    return lifecycle.reconcile(ctx)


@router.post("/storage/reconcile", tags=["storage"])
def reconcile_apply(ctx: AppContext = Depends(get_ctx)) -> dict:
    return lifecycle.reconcile(ctx, apply=True)


# -- cluster ------------------------------------------------------------------------------------


@router.get("/cluster/status", tags=["cluster"])
def cluster_status(ctx: AppContext = Depends(get_ctx)) -> dict:
    return ctx.monitor.status()


@router.get("/cluster/nodes", tags=["cluster"])
def cluster_nodes(ctx: AppContext = Depends(get_ctx)) -> dict:
    return ctx.monitor.nodes()


@router.get("/cluster/replication", tags=["cluster"])
def cluster_replication(ctx: AppContext = Depends(get_ctx)) -> dict:
    return replication.replication_status(ctx)


@router.post("/cluster/replication", tags=["cluster"], status_code=202)
def change_replication(body: ReplicationRequest, ctx: AppContext = Depends(get_ctx)) -> dict:
    return replication.request_change(ctx, body.factor, acknowledge_reduced_protection=body.acknowledge_reduced_protection)


# -- analytics ----------------------------------------------------------------------------------


@router.get("/analytics/overview", tags=["analytics"])
def analytics_overview(ctx: AppContext = Depends(get_ctx)) -> dict:
    with session_scope(ctx.session_factory) as session:
        return {
            **analytics.full_statistics(ctx, session),
            "distributions": analytics.distributions(session),
            "integrity": analytics.integrity_summary(session),
            "cluster": ctx.monitor.status(),
        }


@router.get("/analytics/history", tags=["analytics"])
def analytics_history(ctx: AppContext = Depends(get_ctx)) -> dict:
    with session_scope(ctx.session_factory) as session:
        return {"snapshots": analytics.history(session), "optimizations": analytics.optimization_history(session)}


@router.get("/analytics/performance", tags=["analytics"])
def analytics_performance(ctx: AppContext = Depends(get_ctx)) -> dict:
    with session_scope(ctx.session_factory) as session:
        return analytics.performance(session)
```

```
@router.get("/analytics/report.csv", tags=["analytics"])
def analytics_report(ctx: AppContext = Depends(get_ctx)) -> Response:
    with session_scope(ctx.session_factory) as session:
        stats = analytics.storage_statistics(session)
        perf = analytics.performance(session)
    rows = [{"section": "storage", "metric": k, "value": v} for k, v in stats.items() if not isinstance(v, dict)]
    for op in perf["operations"]:
        for key in ("samples", "bytes", "total_ms", "mean_ms", "throughput_mib_s"):
            rows.append({"section": f"performance.{op['operation']}.{op['mode'] or 'all'}", "metric": key, "value": op[key]})
    rows.extend({"section": "integrity", "metric": k, "value": v} for k, v in perf["integrity"].items())
    return Response(
        experiments.rows_to_csv(rows),
        media_type="text/csv",
        headers={"Content-Disposition": 'attachment; filename="hdfs-optimizer-report.csv"'},
    )


# -- datasets ------------------------------------------------------------------------------


@router.get("/datasets", tags=["datasets"])
def list_datasets(ctx: AppContext = Depends(get_ctx)) -> dict:
    with session_scope(ctx.session_factory) as session:
        datasets.sync_provenance(ctx, session)
        rows = list(session.scalars(select(DatasetSource).order_by(DatasetSource.kind, DatasetSource.size_bytes)))
        items = [datasets.source_dict(r, local_present=(ctx.settings.dataset_dir / r.local_path).exists()) for r in rows]
    return {
        "items": items,
        "catalog": [
            {
                "ebook_id": entry.ebook_id,
                "title": entry.title,
                "author": entry.author,
                "formats": entry.formats,
                "source_url": datasets.source_page(entry.ebook_id),
            }
            for entry in datasets.CATALOG
        ],
        "transformations": datasets.TRANSFORMATIONS,
    }


@router.post("/datasets/import", tags=["datasets"], response_model=JobAccepted, status_code=202)
def import_dataset(body: DatasetImportRequest = Body(default=DatasetImportRequest()), ctx: AppContext = Depends(get_ctx)) ->
JobAccepted:
    return JobAccepted(job_id=ctx.jobs.submit("dataset_import", params={"ebook_ids": body.ebook_ids}))


@router.post("/datasets/ingest", tags=["datasets"], response_model=JobAccepted, status_code=202)
def ingest_dataset(body: DatasetIngestRequest, ctx: AppContext = Depends(get_ctx)) -> JobAccepted:
    if not body.source_ids:
        raise ValidationFailure("Select at least one dataset file.")
    return JobAccepted(
        job_id=ctx.jobs.submit(
            "dataset_ingest",
            mode=body.mode,
            chunk_size=body.chunk_size,
            params={"source_ids": body.source_ids, "copies": body.copies, "folder": body.folder},
        )
    )


@router.get("/datasets/provenance", tags=["datasets"])
def provenance(ctx: AppContext = Depends(get_ctx)) -> dict:
    return datasets._read_manifest(ctx.settings.provenance_path)


# -- experiments ---------------------------------------------------------------------------


@router.get("/experiments", tags=["experiments"])
def list_experiments(ctx: AppContext = Depends(get_ctx)) -> dict:
    with session_scope(ctx.session_factory) as session:
        runs = session.scalars(select(ExperimentRun).order_by(ExperimentRun.started_at.desc()).limit(50))
        items = [
            {
                "id": r.id,
                "scenario": r.scenario,
                "status": r.status,
                "result": r.result,
                "error": r.error,
                "job_id": r.job_id,
                "started_at": r.started_at,
                "finished_at": r.finished_at,
            }
            for r in runs
        ]
    return {
        "scenarios": [{"id": key, "title": value[0], "description": value[1].__doc__} for key, value in
experiments.SCENARIOS.items()],
        "runs": items,
```

```
    }


@router.post("/experiments/run", tags=["experiments"], response_model=JobAccepted, status_code=202)
def run_experiment(body: ExperimentRequest, ctx: AppContext = Depends(get_ctx)) -> JobAccepted:
    if body.scenario not in experiments.SCENARIOS:
        raise ValidationFailure(f"Unknown scenario: {body.scenario}")
    return JobAccepted(job_id=ctx.jobs.submit("experiment", params={"scenario": body.scenario, "options": body.options}))


@router.get("/experiments/{run_id}/export.csv", tags=["experiments"])
def export_experiment(run_id: str, ctx: AppContext = Depends(get_ctx)) -> Response:
    with session_scope(ctx.session_factory) as session:
        run = session.get(ExperimentRun, run_id)
        if run is None or not run.result:
            raise NotFoundError(f"No completed experiment run {run_id}.")
        csv_text = experiments.rows_to_csv(run.result.get("rows", []))
        scenario = run.scenario
    return Response(
        csv_text,
        media_type="text/csv",
        headers={"Content-Disposition": f'attachment; filename="{scenario}-{run_id[:8]}.csv"'},
    )


# -- audit ---------------------------------------------------------------------------------------


@router.get("/audit/events", tags=["audit"], response_model=AuditPage)
def audit_events(
    event_type: str | None = Query(None, description="Exact type, or a prefix such as 'file.'"),
    severity: str | None = Query(None, pattern="^(info|warning|error)$"),
    page: int = Query(1, ge=1),
    page_size: int = Query(50, ge=1, le=200),
    ctx: AppContext = Depends(get_ctx),
) -> AuditPage:
    with session_scope(ctx.session_factory) as session:
        query = select(AuditEvent)
        if event_type:
            prefix = event_type.replace("%", "").replace("_", r"\_")
            query = query.where(AuditEvent.event_type.like(f"{prefix}%", escape="\\"))
        if severity:
            query = query.where(AuditEvent.severity == severity)
        total = session.scalar(select(func.count()).select_from(query.subquery()))
        rows = session.scalars(query.order_by(AuditEvent.occurred_at.desc(), AuditEvent.id.desc()).offset((page - 1) *
page_size).limit(page_size))
        return AuditPage(items=[AuditOut.model_validate(r) for r in rows], total=total, page=page, page_size=page_size)


# -- settings ------------------------------------------------------------------------------------


@router.get("/settings", tags=["settings"])
def get_settings_view(ctx: AppContext = Depends(get_ctx)) -> dict:
    with session_scope(ctx.session_factory) as session:
        values = settings_store.get_all(ctx, session)
    return {
        "values": values,
        "connection": {
            "namenode_url": ctx.settings.namenode_url,
            "hdfs_user": ctx.settings.hdfs_user,
            "root_path": ctx.paths.root,
            "datanode_host_map": ctx.settings.datanode_map,
            "source": "Read-only here. Set with HDFSOPT_* environment variables and restart the backend.",
        },
        "limits": {
            "min_chunk_size": ctx.settings.min_chunk_size,
            "max_chunk_size": ctx.settings.max_chunk_size,
            "max_upload_bytes": ctx.settings.max_upload_bytes,
        },
        "storage": {
            "database_path": str(ctx.settings.database_path),
            "database_bytes": ctx.settings.database_path.stat().st_size if ctx.settings.database_path.exists() else None,
        },
    }


@router.put("/settings", tags=["settings"])
def update_settings(body: SettingsUpdate, ctx: AppContext = Depends(get_ctx)) -> dict:
    changes = body.model_dump(exclude_none=True)
    with session_scope(ctx.session_factory) as session:
        values = settings_store.update(ctx, session, changes)
        audit.record(session, "settings.changed", f"Settings changed: {', '.join(sorted(changes)) or 'nothing'}.", details=changes)
    return {"values": values}
```

11.5 Frontend: libraries and shared components

`frontend/package.json` 42 lines

```
{
  "name": "hdfs-optimizer-frontend",
  "version": "0.1.0",
  "private": true,
```

```
  "scripts": {
    "dev": "next dev -p 3100",
    "build": "next build",
    "start": "next start -p 3100",
    "lint": "eslint",
    "typecheck": "tsc --noEmit",
    "test:e2e": "playwright test"
  },
  "dependencies": {
    "@base-ui/react": "^1.9.0",
    "@tanstack/react-query": "^5.106.0",
    "class-variance-authority": "^0.7.1",
    "clsx": "^2.1.1",
    "lucide-react": "^1.68.0",
    "motion": "^14.9.1",
    "next": "16.4.5",
    "next-themes": "^0.4.6",
    "react": "19.3.0",
    "react-dom": "19.3.0",
    "recharts": "^3.10.1",
    "shadcn": "^4.23.0",
    "sonner": "^2.0.8",
    "tailwind-merge": "^3.7.0",
    "tw-animate-css": "^1.4.0"
  },
  "devDependencies": {
    "@playwright/test": "^1.64.1",
    "@tailwindcss/turbopack": "^4",
    "@types/node": "^20",
    "@types/react": "^19",
    "@types/react-dom": "^19",
    "eslint": "^9",
    "eslint-config-next": "16.4.5",
    "tailwindcss": "^4",
    "typescript": "^5"
  }
}
```

frontend/tsconfig.json 35 lines

```
{
  "compilerOptions": {
    "target": "ES2017",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "react-jsx",
    "incremental": true,
    "plugins": [
      {
        "name": "next"
      }
    ],
    "paths": {
      "@/*": ["./*"]
    }
  },
  "include": [
    "next-env.d.ts",
    "**/*.ts",
    "**/*.tsx",
    ".next/types/**/*.ts",
    ".next/dev/types/**/*.ts",
    "**/*.mts"
  ],
  "exclude": ["node_modules"]
}
```

frontend/next.config.ts 19 lines

```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  // The container image ships only the files the server needs, which avoids copying all of node_modules.
  output: process.env.NEXT_STANDALONE === "1" ? "standalone" : undefined,
  cacheComponents: true,
  partialPrefetching: true,
  turbopack: {
    rules: {
      "*.css": {
        loaders: ["@tailwindcss/turbopack"],
        as: "*.css",
      },
    },
  },
};

export default nextConfig;
```

frontend/eslint.config.mjs 19 lines

```
import { defineConfig, globalIgnores } from "eslint/config";
import nextVitals from "eslint-config-next/core-web-vitals";
import nextTs from "eslint-config-next/typescript";

const eslintConfig = defineConfig([
  ...nextVitals,
  ...nextTs,
  // Override default ignores of eslint-config-next.
  globalIgnores([
    // Default ignores of eslint-config-next:
    ".next/**",
    "out/**",
    "build/**",
    "next-env.d.ts",
  ]),
]);

export default eslintConfig;
```

frontend/components.json 26 lines

```
{
  "$schema": "https://ui.shadcn.com/schema.json",
  "style": "base-nova",
  "rsc": true,
  "tsx": true,
  "tailwind": {
    "config": "",
    "css": "app/globals.css",
    "baseColor": "neutral",
    "cssVariables": true,
    "prefix": ""
  },
  "iconLibrary": "lucide",
  "rtl": false,
  "aliases": {
    "components": "@/components",
    "utils": "@/lib/utils",
    "ui": "@/components/ui",
    "lib": "@/lib",
    "hooks": "@/hooks"
  },
  "menuColor": "default",
  "menuAccent": "subtle",
  "registries": {}
}
```

frontend/.env.example 5 lines

```
# URL the browser uses to reach the backend
NEXT_PUBLIC_API_URL=http://localhost:8100
# Only if the backend is started with HDFSOPT_API_TOKEN. This value reaches the browser, so it is for local use only.
# NEXT_PUBLIC_API_TOKEN=
```

frontend/app/globals.css 147 lines

```
@import "tailwindcss";
@import "tw-animate-css";
@import "shadcn/tailwind.css";

@custom-variant dark (&:is(.dark *));

@theme inline {
  --color-background: var(--background);
  --color-foreground: var(--foreground);
  --font-sans: var(--font-plex-sans);
  --font-mono: var(--font-plex-mono);
  --font-heading: var(--font-plex-sans);
  --color-card: var(--card);
  --color-card-foreground: var(--card-foreground);
  --color-popover: var(--popover);
  --color-popover-foreground: var(--popover-foreground);
  --color-primary: var(--primary);
  --color-primary-foreground: var(--primary-foreground);
  --color-secondary: var(--secondary);
  --color-secondary-foreground: var(--secondary-foreground);
  --color-muted: var(--muted);
  --color-muted-foreground: var(--muted-foreground);
  --color-subtle: var(--subtle);
  --color-accent: var(--accent);
  --color-accent-foreground: var(--accent-foreground);
  --color-destructive: var(--destructive);
  --color-border: var(--border);
  --color-input: var(--input);
  --color-ring: var(--ring);
  --color-good: var(--status-good);
  --color-warning: var(--status-warning);
  --color-serious: var(--status-serious);
  --color-critical: var(--status-critical);
  --color-series-1: var(--series-1);
  --color-series-2: var(--series-2);
  --color-avoided: var(--avoided);
  --color-avoided-ink: var(--avoided-ink);
  --radius-sm: 4px;
  --radius-md: 6px;
```

```
  --radius-lg: 8px;
  --radius-xl: 10px;
}

/* Chart colors come from a palette validated for color-vision deficiency; dark values are selected, not flipped. */
:root {
  --background: #f9f9f7;
  --foreground: #0b0b0b;
  --card: #fcfcfb;
  --card-foreground: #0b0b0b;
  --popover: #fcfcfb;
  --popover-foreground: #0b0b0b;
  --primary: #1c5cab;
  --primary-foreground: #ffffff;
  --secondary: #f0efec;
  --secondary-foreground: #0b0b0b;
  --muted: #f0efec;
  --muted-foreground: #52514e;
  --subtle: #6f6d67;
  --accent: #f0efec;
  --accent-foreground: #0b0b0b;
  --destructive: #c23232;
  --border: #e1e0d9;
  --input: #d6d5cd;
  --ring: #2a78d6;
  --radius: 8px;

  --status-good: #0a7d0a;
  --status-warning: #a86f00;
  --status-serious: #c9572a;
  --status-critical: #d03b3b;

  --series-1: #2a78d6;
  --series-2: #eb6834;
  --series-3: #1baf7a;
  --avoided: #b7d3f6;
  --avoided-ink: #0b0b0b;
  --chart-grid: #e1e0d9;
  --chart-axis: #c3c2b7;
  --chart-label: #6f6d67;
}

.dark {
  --background: #0d0d0d;
  --foreground: #f0efec;
  --card: #1a1a19;
  --card-foreground: #f0efec;
  --popover: #1f1f1e;
  --popover-foreground: #f0efec;
  --primary: #5598e7;
  --primary-foreground: #0b0b0b;
  --secondary: #2c2c2a;
  --secondary-foreground: #f0efec;
  --muted: #262624;
  --muted-foreground: #c3c2b7;
  --subtle: #9a9890;
  --accent: #2c2c2a;
  --accent-foreground: #f0efec;
  --destructive: #e66767;
  --border: #2c2c2a;
  --input: #383835;
  --ring: #3987e5;

  --status-good: #0ca30c;
  --status-warning: #fab219;
  --status-serious: #ec835a;
  --status-critical: #e66767;

  --series-1: #3987e5;
  --series-2: #d95926;
  --series-3: #199e70;
  --avoided: #184f95;
  --avoided-ink: #ffffff;
  --chart-grid: #2c2c2a;
  --chart-axis: #383835;
  --chart-label: #9a9890;
}

@layer base {
  * {
    @apply border-border outline-ring/50;
  }
  html {
    -webkit-text-size-adjust: 100%;
  }
  body {
    @apply bg-background text-foreground font-sans;
    font-size: 14px;
    line-height: 1.45;
  }
  ::selection {
    background: color-mix(in oklch, var(--series-1) 25%, transparent);
  }
```

```
}

@media (prefers-reduced-motion: reduce) {
  *,
  ::before,
  ::after {
    animation-duration: 0.01ms !important;
    transition-duration: 0.01ms !important;
  }
}

.tabular {
  font-variant-numeric: tabular-nums;
}
```

frontend/app/layout.tsx 28 lines

```
import type { Metadata } from "next";
import { IBM_Plex_Mono, IBM_Plex_Sans } from "next/font/google";

import { Providers } from "@/components/providers";
import { AppShell } from "@/components/shell/app-shell";

import "./globals.css";

const plexSans = IBM_Plex_Sans({ variable: "--font-plex-sans", subsets: ["latin"], weight: ["400", "500", "600"] });
const plexMono = IBM_Plex_Mono({ variable: "--font-plex-mono", subsets: ["latin"], weight: ["400", "500"] });

export const metadata: Metadata = {
  title: "HDFS Optimizer",
  description: "Deduplication and storage optimization for Apache Hadoop HDFS.",
};

export default function RootLayout({ children }: Readonly<{ children: React.ReactNode }>) {
  return (
    <html lang="en" suppressHydrationWarning className={`${plexSans.variable} ${plexMono.variable} antialiased`}>
      <body>
        <Providers>
          <AppShell>{children}</AppShell>
        </Providers>
      </body>
    </html>
  );
}
```

frontend/lib/api.ts 102 lines

```
import type { UploadResult } from "./types";

export const API_URL = process.env.NEXT_PUBLIC_API_URL ?? "http://localhost:8100";
const API_TOKEN = process.env.NEXT_PUBLIC_API_TOKEN;

export class ApiError extends Error {
  constructor(
    message: string,
    public status: number,
    public code: string,
    public details: Record<string, unknown> = {},
  ) {
    super(message);
  }
}

function describeValidation(detail: unknown): string {
  if (Array.isArray(detail)) {
    return detail.map((d) => `${(d.loc ?? []).slice(1).join(".")}: ${d.msg}`).join("; ");
  }
  return String(detail);
}

async function toError(response: Response): Promise<ApiError> {
  const body = await response.json().catch(() => null);
  if (body?.error) return new ApiError(body.error.message, response.status, body.error.code, body.error.details);
  if (body?.detail) return new ApiError(describeValidation(body.detail), response.status, "validation_failed");
  return new ApiError(`The server returned ${response.status}.`, response.status, "http_error");
}

export async function api<T>(path: string, init?: RequestInit): Promise<T> {
  let response: Response;
  try {
    response = await fetch(`${API_URL}${path}`, {
      ...init,
      headers: {
        ...(init?.body ? { "Content-Type": "application/json" } : {}),
        ...(API_TOKEN ? { "X-API-Key": API_TOKEN } : {}),
        ...init?.headers,
      },
    });
  } catch {
    throw new ApiError(`Cannot reach the backend at ${API_URL}. Is it running?`, 0, "backend_unreachable");
  }
  if (!response.ok) throw await toError(response);
  return (await response.json()) as T;
}
```

```
export const post = <T>(path: string, body?: unknown) =>
  api<T>(path, { method: "POST", body: body === undefined ? undefined : JSON.stringify(body) });

export const put = <T>(path: string, body: unknown) => api<T>(path, { method: "PUT", body: JSON.stringify(body) });

export const del = <T>(path: string) => api<T>(path, { method: "DELETE" });

export interface UploadOptions {
  folder: string;
  mode: string;
  chunkSize?: number;
  onProgress: (sentFraction: number) => void;
}

/** XMLHttpRequest is used because fetch cannot report upload progress. The fraction is what the browser really sent. */
export function uploadFile(file: File, options: UploadOptions): Promise<UploadResult> {
  return new Promise((resolve, reject) => {
    const form = new FormData();
    form.append("file", file);
    form.append("folder", options.folder);
    form.append("mode", options.mode);
    if (options.chunkSize) form.append("chunk_size", String(options.chunkSize));

    const request = new XMLHttpRequest();
    request.open("POST", `${API_URL}/api/v1/files/upload`);
    if (API_TOKEN) request.setRequestHeader("X-API-Key", API_TOKEN);
    request.upload.onprogress = (event) => {
      if (event.lengthComputable) options.onProgress(event.loaded / event.total);
    };
    request.onload = () => {
      let body: { error?: { message: string; code: string }; detail?: unknown } | UploadResult | null = null;
      try {
        body = JSON.parse(request.responseText);
      } catch {
        /* handled below */
      }
      if (request.status >= 200 && request.status < 300 && body) {
        resolve(body as UploadResult);
        return;
      }
      const failure = body as { error?: { message: string; code: string }; detail?: unknown } | null;
      reject(
        new ApiError(
          failure?.error?.message ?? (failure?.detail ? describeValidation(failure.detail) : `Upload failed (${request.status}).`),
          request.status,
          failure?.error?.code ?? "upload_failed",
        ),
      );
    };
    request.onerror = () => reject(new ApiError(`Cannot reach the backend at ${API_URL}.`, 0, "backend_unreachable"));
    request.send(form);
  });
}
```

frontend/lib/format.ts 83 lines

```
const UNITS = ["B", "KiB", "MiB", "GiB", "TiB", "PiB"];

export function formatBytes(bytes: number | null | undefined, digits = 1): string {
  if (bytes === null || bytes === undefined) return "N/A";
  const sign = bytes < 0 ? "-" : "";
  let value = Math.abs(bytes);
  let unit = 0;
  while (value >= 1024 && unit < UNITS.length - 1) {
    value /= 1024;
    unit += 1;
  }
  const shown = unit === 0 ? value.toFixed(0) : value.toFixed(value >= 100 ? 0 : digits);
  return `${sign}${shown} ${UNITS[unit]}`;
}

export function formatCount(value: number | null | undefined): string {
  return value === null || value === undefined ? "N/A" : value.toLocaleString("en-US");
}

export function formatPercent(value: number | null | undefined, digits = 1): string {
  return value === null || value === undefined ? "N/A" : `${value.toFixed(digits)}%`;
}

export function formatRatio(value: number | null | undefined): string {
  return value === null || value === undefined ? "N/A" : `${value.toFixed(2)} : 1`;
}

export function formatMs(ms: number | null | undefined): string {
  if (ms === null || ms === undefined) return "N/A";
  if (ms < 1) return "<1 ms";
  if (ms < 1000) return `${ms.toFixed(0)} ms`;
  return `${(ms / 1000).toFixed(ms < 10000 ? 2 : 1)} s`;
}

/** The backend stores naive UTC timestamps. */
export function parseTime(value: string): Date {
  return new Date(/[zZ]|[+-]\d\d:?\d\d$/.test(value) ? value : `${value}Z`);
}
```

```ts
export function formatDateTime(value: string | null | undefined): string {
  if (!value) return "N/A";
  return parseTime(value).toLocaleString("en-GB", {
    day: "2-digit",
    month: "short",
    year: "numeric",
    hour: "2-digit",
    minute: "2-digit",
    second: "2-digit",
  });
}

export function formatTime(value: string): string {
  return parseTime(value).toLocaleTimeString("en-GB", { hour: "2-digit", minute: "2-digit", second: "2-digit" });
}

export function shortHash(hash: string | null | undefined, length = 12): string {
  return hash ? hash.slice(0, length) : "N/A";
}

export const MODE_LABEL: Record<string, string> = {
  raw: "No dedup",
  file: "Whole file",
  chunk: "Chunked",
};

export const PROVENANCE_LABEL: Record<string, string> = {
  user_upload: "Uploaded",
  dataset_original: "Dataset original",
  workload_copy: "Workload copy",
  transformed: "Transformed test file",
};

export const CHUNK_SIZES = [
  { value: 64 * 1024, label: "64 KiB" },
  { value: 256 * 1024, label: "256 KiB" },
  { value: 1024 * 1024, label: "1 MiB" },
  { value: 4 * 1024 * 1024, label: "4 MiB" },
];

export function chunkSizeLabel(bytes: number | null | undefined): string {
  return bytes ? formatBytes(bytes, 0) : "N/A";
}
```

frontend/lib/queries.ts 41 lines

```ts
"use client";

import { useQuery } from "@tanstack/react-query";
import { useEffect, useRef } from "react";

import { api } from "./api";
import type { ClusterStatus, Job } from "./types";

export function useClusterStatus() {
  return useQuery({
    queryKey: ["cluster", "status"],
    queryFn: () => api<ClusterStatus>("/api/v1/cluster/status"),
    refetchInterval: 10_000,
  });
}

const FINISHED = new Set(["succeeded", "partial", "failed", "interrupted"]);

export function isFinished(job: Job<unknown> | undefined): boolean {
  return !!job && FINISHED.has(job.status);
}

/** Polls a real background job until it reaches a final state, then calls onFinished once. */
export function useJob<R>(jobId: string | null, onFinished?: (job: Job<R>) => void) {
  const notified = useRef<string | null>(null);
  const query = useQuery({
    queryKey: ["job", jobId],
    queryFn: () => api<Job<R>>(`/api/v1/dedup/jobs/${jobId}`),
    enabled: !!jobId,
    refetchInterval: (state) => (isFinished(state.state.data as Job<unknown> | undefined) ? false : 700),
  });
  const job = query.data;
  useEffect(() => {
    if (job && isFinished(job as Job<unknown>) && notified.current !== job.id) {
      notified.current = job.id;
      onFinished?.(job);
    }
  }, [job, onFinished]);
  return query;
}
```

frontend/lib/types.ts 467 lines

```ts
// Shapes returned by the FastAPI backend. See http://localhost:8100/docs for the full API.

export type StorageMode = "raw" | "file" | "chunk";

export interface LogicalFile {
```

```
  id: string;
  name: string;
  folder: string;
  size_bytes: number;
  sha256: string;
  mime_type: string | null;
  extension: string | null;
  storage_mode: StorageMode;
  status: "active" | "deleted" | "purged";
  provenance_label: string;
  created_at: string;
  deleted_at: string | null;
  last_verified_at: string | null;
  last_verified_ok: boolean | null;
  logical_references: number;
  is_duplicate: boolean;
}

export interface Page<T> {
  items: T[];
  total: number;
  page: number;
  page_size: number;
}

export interface IntegrityCheck {
  id: number;
  file_id: string;
  ok: boolean;
  expected_sha256: string;
  actual_sha256: string | null;
  bytes_read: number;
  chunks_checked: number;
  missing: string[] | null;
  corrupt: string[] | null;
  detail: string | null;
  duration_ms: number;
  checked_at: string;
}

export interface FileDetail extends LogicalFile {
  hdfs_paths: string[];
  same_content_files: { id: string; name: string; folder: string; status: string; provenance_label: string }[];
  chunk_size: number | null;
  chunk_count: number | null;
  chunks: { ordinal: number; offset_bytes: number; sha256: string; size_bytes: number; shared_by_manifests: number }[];
  integrity_history: IntegrityCheck[];
  block_locations: { block_id: number; length: number; hosts: string[]; corrupt: boolean }[] | null;
  dataset_source: DatasetSource | null;
}

export interface UploadResult {
  file: LogicalFile;
  outcome: "stored_new" | "duplicate_reference" | "chunks_shared" | "stored_raw";
  duplicate_of_existing: boolean;
  bytes_written: number;
  chunks_total: number;
  chunks_new: number;
  chunks_reused: number;
  hash_ms: number;
  store_ms: number;
  total_ms: number;
}

export interface DerivedStats {
  logical_files: number;
  logical_bytes: number;
  unique_files: number;
  unique_content_bytes: number;
  duplicate_files: number;
  deleted_retained_files: number;
  deleted_retained_bytes: number;
  raw_files: number;
  raw_bytes: number;
  object_count: number;
  object_bytes: number;
  chunk_count: number;
  chunk_bytes: number;
  manifest_count: number;
  manifest_bytes: number;
  chunk_references: number;
  duplicate_chunk_references: number;
  stored_content_bytes: number;
  bytes_avoided: number;
  savings_percent: number | null;
  dedup_ratio: number | null;
  files_by_mode: Partial<Record<StorageMode, number>>;
}

export interface MeasuredArea {
  length: number;
  space_consumed: number;
  file_count: number;
```

```
}

export type MeasuredStats =
  | { available: false; error: string }
  | {
      available: true;
      length: number;
      space_consumed: number;
      file_count: number;
      directory_count: number;
      areas: Record<"objects" | "chunks" | "manifests" | "raw" | "staging" | "recovery", MeasuredArea>;
    };

export interface FullStatistics {
  derived: DerivedStats;
  measured: MeasuredStats;
  estimated: { replication_factor: number | null; replication_adjusted_bytes: number | null; note: string };
}

export interface ClusterStatusUp {
  reachable: true;
  checked_at: string;
  namenode_url: string;
  health: "healthy" | "degraded" | "critical";
  problems: string[];
  namenode: {
    state: string;
    role: string;
    host_and_port: string;
    software_version: string;
    cluster_id: string;
    block_pool_id: string;
    fs_state: string;
    safemode: boolean;
    safemode_message: string | null;
  };
  capacity: { total_bytes: number; used_bytes: number; remaining_bytes: number; non_dfs_used_bytes: number };
  namespace: {
    files_total: number;
    blocks_total: number;
    under_replicated_blocks: number;
    missing_blocks: number;
    corrupt_blocks: number;
    pending_replication_blocks: number;
    pending_deletion_blocks: number;
    excess_blocks: number;
  };
  datanodes: { live: number; dead: number; stale: number; decommissioning: number };
}

export type ClusterStatus = ClusterStatusUp | { reachable: false; error: string; checked_at: string; namenode_url: string };

export interface DataNode {
  name: string;
  transfer_address: string;
  state: "live" | "dead";
  last_contact_seconds: number | null;
  capacity_bytes: number | null;
  used_bytes: number | null;
  remaining_bytes: number | null;
  non_dfs_used_bytes: number | null;
  blocks: number | null;
  version: string | null;
  admin_state: string | null;
  rack: string | null;
}

export interface Fsck {
  status: string | null;
  total_files: number | null;
  total_blocks: number | null;
  over_replicated_blocks: number | null;
  under_replicated_blocks: number | null;
  mis_replicated_blocks: number | null;
  default_replication: number | null;
  average_block_replication: number | null;
  missing_blocks: number | null;
  corrupt_blocks: number | null;
  missing_replicas: number | null;
  datanodes: number | null;
}

export interface ReplicationOperation {
  id: string;
  previous_factor: number | null;
  requested_factor: number;
  status: "pending" | "applied" | "verified" | "timed_out" | "failed";
  files_total: number;
  files_changed: number;
  space_consumed_before: number | null;
  space_consumed_after: number | null;
  error: string | null;
  requested_at: string;
```

```
  completed_at: string | null;
}

export type ReplicationStatus =
  | { reachable: false; error: string; target_replication: number | null; operations: ReplicationOperation[] }
  | {
      reachable: true;
      cluster_default_replication: number;
      target_replication: number | null;
      effective_replication: number;
      live_datanodes: number;
      dead_datanodes: number;
      max_supported_replication: number;
      fsck: Fsck;
      stored_length_bytes: number;
      space_consumed_bytes: number;
      measured_replication_multiplier: number | null;
      warnings: string[];
      operations: ReplicationOperation[];
      checked_at: string;
    };

export interface Job<R = Record<string, unknown>> {
  id: string;
  kind: string;
  mode: StorageMode | null;
  chunk_size: number | null;
  status: "queued" | "running" | "succeeded" | "partial" | "failed" | "interrupted";
  progress_done: number;
  progress_total: number;
  message: string | null;
  params: Record<string, unknown> | null;
  result: R | null;
  error: string | null;
  resumed_from_id: string | null;
  created_at: string;
  started_at: string | null;
  finished_at: string | null;
}

export interface AnalysisResult {
  estimated: true;
  catalog_fingerprint: string;
  mode: StorageMode;
  chunk_size: number | null;
  files_eligible: number;
  distinct_contents: number;
  current_logical_bytes: number;
  current_stored_bytes: number;
  predicted_stored_bytes: number;
  predicted_reduction_bytes: number;
  predicted_savings_percent: number | null;
  new_content_bytes: number;
  chunks_examined: number;
  chunks_new: number;
  chunks_reused: number;
  actions: {
    id: string;
    name: string;
    folder: string;
    sha256: string;
    size_bytes: number;
    storage_mode: StorageMode;
    action: string;
  }[];
  actions_truncated: boolean;
}

export interface MetricsPair {
  derived: DerivedStats;
  measured: MeasuredStats;
}

export interface OptimizeResult {
  estimated: false;
  files_processed: number;
  files_failed: number;
  errors: { file_id: string; error: string }[];
  orphans_left: string[];
  chunks_new: number;
  before: MetricsPair;
  after: MetricsPair;
  bytes_reclaimed: number;
  garbage_collected: { object_count: number; manifest_count: number; chunk_count: number; reclaimable_bytes: number };
  duration_ms: number;
}

export interface DuplicateGroup {
  sha256: string;
  copies: number;
  size_bytes: number;
  logical_bytes: number;
  redundant_logical_bytes: number;
```

```
  reclaimable_bytes: number;
  already_deduplicated: boolean;
  members: {
    id: string;
    name: string;
    folder: string;
    storage_mode: StorageMode;
    provenance_label: string;
    created_at: string;
  }[];
}

export interface DuplicateGroups {
  group_count: number;
  duplicate_files: number;
  redundant_logical_bytes: number;
  groups: DuplicateGroup[];
}

export interface SharedChunks {
  shared_chunk_count: number;
  bytes_saved_by_sharing: number;
  chunks: { sha256: string; size_bytes: number; manifests: number; references: number }[];
  manifests: {
    id: string;
    file_sha256: string;
    file_size: number;
    chunk_size: number;
    chunk_count: number;
    logical_files: string[];
    chunk_map: boolean[];
  }[];
  chunk_map_limit: number;
}

export interface Snapshot {
  taken_at: string;
  reason: string;
  logical_files: number;
  logical_bytes: number;
  stored_bytes: number;
  unique_files: number;
  saved_bytes: number;
  savings_percent: number | null;
}

export interface OptimizationRecord {
  job_id: string;
  mode: StorageMode;
  chunk_size: number | null;
  before: MetricsPair;
  after: MetricsPair;
  files_processed: number;
  files_failed: number;
  bytes_reclaimed_logical: number;
  duration_ms: number;
  created_at: string;
}

export interface Overview extends FullStatistics {
  distributions: {
    file_types: { extension: string; files: number; bytes: number }[];
    duplicate_distribution: { copies: number; groups: number }[];
    upload_trend: { day: string; files: number; bytes: number }[];
  };
  integrity: IntegritySummary;
  cluster: ClusterStatus;
}

export interface IntegritySummary {
  checks_total: number;
  checks_passed: number;
  checks_failed: number;
  files_verified_ok: number;
  files_verified_failed: number;
  files_never_verified: number;
}

export interface PerformanceOp {
  operation: string;
  mode: StorageMode | null;
  samples: number;
  bytes: number;
  total_ms: number;
  min_ms: number;
  max_ms: number;
  mean_ms: number;
  throughput_mib_s: number | null;
}

export interface AuditEvent {
  id: number;
  occurred_at: string;
```

```ts
  event_type: string;
  severity: "info" | "warning" | "error";
  entity_type: string | null;
  entity_id: string | null;
  message: string;
  details: Record<string, unknown> | null;
}

export interface DatasetSource {
  id: string;
  title: string;
  author: string | null;
  ebook_id: number | null;
  source_url: string | null;
  download_url: string | null;
  organization: string | null;
  downloaded_at: string | null;
  file_format: string;
  size_bytes: number;
  sha256: string;
  kind: "original" | "workload_copy" | "transformed";
  derived_from_id: string | null;
  transformation: string | null;
  license_note: string | null;
  local_present: boolean | null;
}

export interface DatasetList {
  items: DatasetSource[];
  catalog: { ebook_id: number; title: string; author: string; formats: string[]; source_url: string }[];
  transformations: Record<string, string>;
}

export interface ExperimentRun {
  id: string;
  scenario: string;
  status: "running" | "succeeded" | "failed";
  result: ({ description?: string; rows?: Record<string, unknown>[]; duration_s?: number } & Record<string, unknown>) | null;
  error: string | null;
  job_id: string | null;
  started_at: string;
  finished_at: string | null;
}

export interface Experiments {
  scenarios: { id: string; title: string; description: string }[];
  runs: ExperimentRun[];
}

export interface SettingsView {
  values: {
    default_ingest_mode: StorageMode;
    default_chunk_size: number;
    verify_on_write: boolean;
    target_replication: number | null;
  };
  connection: {
    namenode_url: string;
    hdfs_user: string;
    root_path: string;
    datanode_host_map: Record<string, string>;
    source: string;
  };
  limits: { min_chunk_size: number; max_chunk_size: number; max_upload_bytes: number };
  storage: { database_path: string; database_bytes: number | null };
}

export interface Diagnostics {
  ok: boolean;
  checks: { name: string; ok: boolean; detail: string; ms: number }[];
}

export interface GarbageReport {
  object_count: number;
  manifest_count: number;
  chunk_count: number;
  reclaimable_bytes: number;
  removed: boolean;
}

export interface ReconcileReport {
  paths_expected: number;
  paths_present: number;
  orphans: string[];
  orphan_count: number;
  missing: string[];
  missing_count: number;
  orphans_quarantined: number;
}
```

frontend/lib/utils.ts 7 lines

```ts
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";
```

```ts
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}
```

frontend/components/common.tsx 271 lines

```tsx
"use client";

import { motion } from "motion/react";
import { AlertTriangle, Check, CheckCircle2, Circle, Copy, Info, Loader2, OctagonAlert, type LucideIcon } from "lucide-react";
import { useState, type ReactNode } from "react";

import { Button } from "@/components/ui/button";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Skeleton } from "@/components/ui/skeleton";
import { Tooltip, TooltipContent, TooltipTrigger } from "@/components/ui/tooltip";
import { ApiError } from "@/lib/api";
import { shortHash } from "@/lib/format";
import { cn } from "@/lib/utils";

export function PageHeader({ title, description, actions }: { title: string; description?: ReactNode; actions?: ReactNode }) {
  return (
    <header className="mb-6 flex flex-col gap-3 sm:flex-row sm:items-start sm:justify-between">
      <div className="min-w-0">
        <h1 className="text-xl font-semibold tracking-tight">{title}</h1>
        {description && <p className="mt-1 max-w-3xl text-[13px] text-muted-foreground">{description}</p>}
      </div>
      {actions && <div className="flex shrink-0 flex-wrap items-center gap-2">{actions}</div>}
    </header>
  );
}

export function Panel({
  title,
  subtitle,
  actions,
  children,
  className,
  flush,
}: {
  title?: ReactNode;
  subtitle?: ReactNode;
  actions?: ReactNode;
  children: ReactNode;
  className?: string;
  flush?: boolean;
}) {
  return (
    <section className={cn("min-w-0 rounded-lg border bg-card", className)}>
      {(title || actions) && (
        <div className="flex flex-wrap items-start justify-between gap-2 px-4 pt-3.5 pb-1">
          <div className="min-w-0">
            {title && <h2 className="text-[13px] font-semibold">{title}</h2>}
            {subtitle && <p className="mt-0.5 text-xs text-muted-foreground">{subtitle}</p>}
          </div>
          {actions && <div className="flex shrink-0 items-center gap-2">{actions}</div>}
        </div>
      )}
      <div className={flush ? "" : "p-4"}>{children}</div>
    </section>
  );
}

export type Provenance = "measured" | "derived" | "estimated";

const PROVENANCE: Record<Provenance, { label: string; help: string; dot: string }> = {
  measured: {
    label: "Measured",
    help: "Reported directly by HDFS at the time shown.",
    dot: "bg-foreground",
  },
  derived: {
    label: "Derived",
    help: "Computed exactly from the metadata database, which records what the application stored. Excludes replication.",
    dot: "border-[1.5px] border-foreground",
  },
  estimated: {
    label: "Estimated",
    help: "A prediction or calculation, not an observation. Treat as approximate until verified.",
    dot: "border-[1.5px] border-dashed border-muted-foreground",
  },
};

/** Every headline number states how it is known. */
export function ProvenanceTag({ kind }: { kind: Provenance }) {
  const info = PROVENANCE[kind];
  return (
    <Tooltip>
      <TooltipTrigger
        render={
          <span className="inline-flex cursor-help items-center gap-1 text-[11px] text-muted-foreground">
            <span className={cn("inline-block size-2 rounded-full", info.dot)} aria-hidden />
            {info.label}
```

```
          </span>
        }
      />
      <TooltipContent className="max-w-64">{info.help}</TooltipContent>
    </Tooltip>
  );
}

export function Stat({
  label,
  value,
  note,
  provenance,
}: {
  label: string;
  value: ReactNode;
  note?: ReactNode;
  provenance?: Provenance;
}) {
  return (
    <div className="min-w-0">
      <div className="text-xs leading-tight text-muted-foreground">{label}</div>
      <div className="mt-1 truncate text-xl font-semibold tracking-tight" title={typeof value === "string" ? value : undefined}>
        {value}
      </div>
      <div className="mt-1 flex min-h-4 flex-wrap items-center gap-x-2 gap-y-0.5 text-xs text-muted-foreground">
        {provenance && <ProvenanceTag kind={provenance} />}
        {note && <span>{note}</span>}
      </div>
    </div>
  );
}

const TONES: Record<string, { icon: LucideIcon; className: string }> = {
  good: { icon: CheckCircle2, className: "text-good" },
  warning: { icon: AlertTriangle, className: "text-warning" },
  critical: { icon: OctagonAlert, className: "text-critical" },
  neutral: { icon: Circle, className: "text-muted-foreground" },
  info: { icon: Info, className: "text-muted-foreground" },
  busy: { icon: Loader2, className: "text-primary" },
};

/** Status always pairs an icon and a label with its color. */
export function Status({ tone, children, className }: { tone: keyof typeof TONES; children: ReactNode; className?: string }) {
  const { icon: Icon, className: toneClass } = TONES[tone];
  return (
    <span className={cn("inline-flex items-center gap-1.5 text-[13px]", className)}>
      <Icon className={cn("size-3.5 shrink-0", toneClass, tone === "busy" && "animate-spin")} aria-hidden />
      <span>{children}</span>
    </span>
  );
}

export function EmptyState({ icon: Icon, title, children, action }: { icon?: LucideIcon; title: string; children?: ReactNode;
action?: ReactNode }) {
  return (
    <div className="flex flex-col items-center justify-center px-6 py-10 text-center">
      {Icon && <Icon className="mb-3 size-5 text-muted-foreground" aria-hidden />}
      <p className="text-[13px] font-medium">{title}</p>
      {children && <p className="mt-1 max-w-md text-xs text-muted-foreground">{children}</p>}
      {action && <div className="mt-4">{action}</div>}
    </div>
  );
}

export function ErrorNotice({ error, onRetry }: { error: unknown; onRetry?: () => void }) {
  const message = error instanceof ApiError || error instanceof Error ? error.message : "Something went wrong.";
  const hdfsDown = error instanceof ApiError && error.code === "hdfs_unavailable";
  return (
    <div role="alert" className="flex items-start gap-2.5 rounded-md border border-critical/30 bg-critical/5 px-3 py-2.5 text-
[13px]">
      <OctagonAlert className="mt-0.5 size-4 shrink-0 text-critical" aria-hidden />
      <div className="min-w-0 flex-1">
        <p className="font-medium">{hdfsDown ? "HDFS is unavailable" : "This could not be completed"}</p>
        <p className="mt-0.5 break-words text-muted-foreground">{message}</p>
      </div>
      {onRetry && (
        <Button size="sm" variant="outline" onClick={onRetry}>
          Retry
        </Button>
      )}
    </div>
  );
}

export function Notice({ tone = "info", children }: { tone?: "info" | "warning"; children: ReactNode }) {
  const Icon = tone === "warning" ? AlertTriangle : Info;
  return (
    <div className={cn("flex items-start gap-2 rounded-md border px-3 py-2 text-xs text-muted-foreground", tone === "warning" &&
"border-warning/40 bg-warning/5")}>
      <Icon className={cn("mt-px size-3.5 shrink-0", tone === "warning" && "text-warning")} aria-hidden />
      <div className="min-w-0">{children}</div>
    </div>
```

```
  );
}

export function RowsSkeleton({ rows = 5 }: { rows?: number }) {
  return (
    <div className="space-y-2 p-4" aria-label="Loading">
      {Array.from({ length: rows }).map((_, i) => (
        <Skeleton key={i} className="h-5 w-full" />
      ))}
    </div>
  );
}

export function HashText({ hash, length = 12 }: { hash: string; length?: number }) {
  const [copied, setCopied] = useState(false);
  return (
    <button
      type="button"
      className="group inline-flex items-center gap-1 rounded font-mono text-xs text-muted-foreground hover:text-foreground"
      title={hash}
      aria-label={`Copy SHA-256 ${hash}`}
      onClick={(event) => {
        event.stopPropagation();
        navigator.clipboard?.writeText(hash);
        setCopied(true);
        setTimeout(() => setCopied(false), 1200);
      }}
    >
      {shortHash(hash, length)}
      {copied ? <Check className="size-3" aria-hidden /> : <Copy className="size-3 opacity-0 group-hover:opacity-100" aria-hidden />}
    </button>
  );
}

/** A determinate bar that shows real job progress. */
export function ProgressBar({ value, className, label }: { value: number; className?: string; label?: string }) {
  const percent = Math.max(0, Math.min(1, value)) * 100;
  return (
    <div
      className={cn("h-1.5 w-full overflow-hidden rounded-full bg-muted", className)}
      role="progressbar"
      aria-valuenow={Math.round(percent)}
      aria-valuemin={0}
      aria-valuemax={100}
      aria-label={label}
    >
      <motion.div className="h-full rounded-full bg-series-1" animate={{ width: `${percent}%` }} transition={{ duration: 0.25, ease: "easeOut" }} />
    </div>
  );
}

/** A compact select. Options carry labels so the trigger always shows the human-readable choice. */
export function Choice<T extends string>({
  value,
  onChange,
  options,
  ariaLabel,
  className,
}: {
  value: T;
  onChange: (value: T) => void;
  options: { value: T; label: string }[];
  ariaLabel: string;
  className?: string;
}) {
  return (
    <Select items={options} value={value} onValueChange={(next) => next !== null && onChange(next as T)}>
      <SelectTrigger aria-label={ariaLabel} className={cn("h-8 text-[13px]", className)}>
        <SelectValue />
      </SelectTrigger>
      <SelectContent>
        {options.map((option) => (
          <SelectItem key={option.value} value={option.value}>
            {option.label}
          </SelectItem>
        ))}
      </SelectContent>
    </Select>
  );
}

export function KeyValue({ rows }: { rows: { label: string; value: ReactNode; hint?: ReactNode }[] }) {
  return (
    <dl className="divide-y text-[13px]">
      {rows.map((row) => (
        <div key={row.label} className="flex items-baseline justify-between gap-4 py-1.5 first:pt-0 last:pb-0">
          <dt className="shrink-0 text-muted-foreground">{row.label}</dt>
          <dd className="min-w-0 text-right break-words">{row.value}</dd>
        </div>
      ))}
    </dl>
```

```
  );
}
```

frontend/components/providers.tsx 26 lines

```tsx
"use client";

import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import { ThemeProvider } from "next-themes";
import { useState, type ReactNode } from "react";

import { Toaster } from "@/components/ui/sonner";
import { TooltipProvider } from "@/components/ui/tooltip";

export function Providers({ children }: { children: ReactNode }) {
  const [client] = useState(
    () =>
      new QueryClient({
        defaultOptions: { queries: { retry: 1, staleTime: 2_000, refetchOnWindowFocus: true } },
      }),
  );
  return (
    <ThemeProvider attribute="class" defaultTheme="system" enableSystem disableTransitionOnChange>
      <QueryClientProvider client={client}>
        <TooltipProvider delay={150}>{children}</TooltipProvider>
        <Toaster position="bottom-right" />
      </QueryClientProvider>
    </ThemeProvider>
  );
}
```

frontend/components/shell/app-shell.tsx 206 lines

```tsx
"use client";

import {
  Activity,
  ChartNoAxesCombined,
  CopyCheck,
  FlaskConical,
  FolderTree,
  Gauge,
  Layers3,
  Menu,
  Moon,
  Server,
  Settings2,
  Sparkles,
  Sun,
  type LucideIcon,
} from "lucide-react";
import { motion } from "motion/react";
import Link from "next/link";
import { usePathname } from "next/navigation";
import { useTheme } from "next-themes";
import { useState, useSyncExternalStore, type ReactNode } from "react";

import { Button } from "@/components/ui/button";
import { Sheet, SheetContent, SheetTitle } from "@/components/ui/sheet";
import { useClusterStatus } from "@/lib/queries";
import { cn } from "@/lib/utils";

const NAV: { group: string; items: { href: string; label: string; icon: LucideIcon }[] }[] = [
  {
    group: "Overview",
    items: [{ href: "/", label: "Dashboard", icon: Gauge }],
  },
  {
    group: "Data",
    items: [
      { href: "/files", label: "Files", icon: FolderTree },
      { href: "/duplicates", label: "Duplicates", icon: CopyCheck },
      { href: "/optimize", label: "Optimization", icon: Sparkles },
    ],
  },
  {
    group: "Cluster",
    items: [
      { href: "/replication", label: "Replication", icon: Layers3 },
      { href: "/cluster", label: "Monitoring", icon: Server },
    ],
  },
  {
    group: "Insight",
    items: [
      { href: "/analytics", label: "Analytics", icon: ChartNoAxesCombined },
      { href: "/activity", label: "Activity", icon: Activity },
      { href: "/lab", label: "Dataset lab", icon: FlaskConical },
    ],
  },
  {
    group: "System",
    items: [{ href: "/settings", label: "Settings", icon: Settings2 }],
  },
];
```

```
function BrandMark() {
  // Several logical files resolving to one stored copy.
  return (
    <svg viewBox="0 0 24 24" className="size-5" fill="none" aria-hidden>
      <path d="M3 5h6c3 0 3 7 6 7M3 12h12M3 19h6c3 0 3-7 6-7" stroke="currentColor" strokeWidth="1.7" strokeLinecap="round" />
      <rect x="15" y="9" width="6" height="6" rx="1.5" fill="var(--series-1)" />
    </svg>
  );
}

function ClusterIndicator() {
  const { data, isError } = useClusterStatus();
  let dot = "bg-muted-foreground";
  let text = "Checking cluster";
  if (isError) {
    dot = "bg-critical";
    text = "Backend unreachable";
  } else if (data && !data.reachable) {
    dot = "bg-critical";
    text = "NameNode unreachable";
  } else if (data?.reachable) {
    dot = data.health === "healthy" ? "bg-good" : data.health === "degraded" ? "bg-warning" : "bg-critical";
    text = `${data.health === "healthy" ? "Healthy" : data.health === "degraded" ? "Degraded" : "Critical"} · ${data.datanodes.live}
DataNode${data.datanodes.live === 1 ? "" : "s"}`;
  }
  return (
    <Link href="/cluster" className="flex min-w-0 items-center gap-2 rounded-md px-2 py-1.5 text-xs text-muted-foreground hover:bg-
accent hover:text-foreground">
      <span className={cn("size-2 shrink-0 rounded-full", dot)} aria-hidden />
      <span className="truncate">{text}</span>
    </Link>
  );
}

function ThemeToggle() {
  const { resolvedTheme, setTheme } = useTheme();
  // The theme is unknown during server rendering; true only in the browser.
  const mounted = useSyncExternalStore(
    () => () => {},
    () => true,
    () => false,
  );
  const dark = mounted && resolvedTheme === "dark";
  return (
    <Button
      variant="ghost"
      size="icon-sm"
      aria-label={dark ? "Switch to light mode" : "Switch to dark mode"}
      onClick={() => setTheme(dark ? "light" : "dark")}
    >
      {dark ? <Sun /> : <Moon />}
    </Button>
  );
}

function Navigation({ onNavigate }: { onNavigate?: () => void }) {
  const pathname = usePathname();
  return (
    <nav aria-label="Main" className="space-y-4">
      {NAV.map((section) => (
        <div key={section.group}>
          <p className="mb-1 px-2 text-[11px] font-medium text-subtle">{section.group}</p>
          <ul className="space-y-px">
            {section.items.map(({ href, label, icon: Icon }) => {
              const active = href === "/" ? pathname === "/" : pathname.startsWith(href);
              return (
                <li key={href}>
                  <Link
                    href={href}
                    onClick={onNavigate}
                    aria-current={active ? "page" : undefined}
                    className={cn(
                      "flex items-center gap-2.5 rounded-md px-2 py-1.5 text-[13px] transition-colors",
                      active ? "bg-accent font-medium text-foreground" : "text-muted-foreground hover:bg-accent/60 hover:text-
foreground",
                    )}
                  >
                    <Icon className="size-4 shrink-0" aria-hidden />
                    {label}
                  </Link>
                </li>
              );
            })}
          </ul>
        </div>
      ))}
    </nav>
  );
}

export function AppShell({ children }: { children: ReactNode }) {
  const [open, setOpen] = useState(false);
```

```
  const pathname = usePathname();
  return (
    <div className="min-h-screen lg:grid lg:grid-cols-[216px_minmax(0,1fr)]">
      <aside className="hidden border-r bg-card lg:block">
        <div className="sticky top-0 flex h-screen flex-col">
          <div className="flex h-14 items-center gap-2 px-4">
            <BrandMark />
            <span className="text-sm font-semibold tracking-tight">HDFS Optimizer</span>
          </div>
          <div className="flex-1 overflow-y-auto px-2 py-2">
            <Navigation />
          </div>
          <div className="flex items-center justify-between gap-1 border-t px-2 py-2">
            <ClusterIndicator />
            <ThemeToggle />
          </div>
        </div>
      </aside>

      <div className="min-w-0">
        <div className="sticky top-0 z-30 flex h-12 items-center justify-between border-b bg-card px-3 lg:hidden">
          <div className="flex items-center gap-2">
            <Button variant="ghost" size="icon-sm" aria-label="Open navigation" onClick={() => setOpen(true)}>
              <Menu />
            </Button>
            <BrandMark />
            <span className="text-sm font-semibold tracking-tight">HDFS Optimizer</span>
          </div>
          <ThemeToggle />
        </div>
        <Sheet open={open} onOpenChange={setOpen}>
          <SheetContent side="left" className="w-64 p-3">
            <SheetTitle className="px-2 pt-1 pb-2 text-sm">HDFS Optimizer</SheetTitle>
            <Navigation onNavigate={() => setOpen(false)} />
            <div className="mt-4 border-t pt-2">
              <ClusterIndicator />
            </div>
          </SheetContent>
        </Sheet>

        <motion.main
          key={pathname}
          initial={{ opacity: 0, y: 4 }}
          animate={{ opacity: 1, y: 0 }}
          transition={{ duration: 0.16, ease: "easeOut" }}
          className="mx-auto w-full max-w-[1320px] px-4 py-6 sm:px-6 lg:px-8"
        >
          {children}
        </motion.main>
      </div>
    </div>
  );
}
```

frontend/components/charts/index.tsx 316 lines

```
"use client";

import { Table2, ChartColumn } from "lucide-react";
import { useState, type ReactNode } from "react";
import {
  Area,
  AreaChart,
  Bar,
  BarChart,
  CartesianGrid,
  ResponsiveContainer,
  Tooltip,
  XAxis,
  YAxis,
} from "recharts";

import { Panel } from "@/components/common";
import { Button } from "@/components/ui/button";
import { formatBytes, formatCount, parseTime } from "@/lib/format";

const axisTick = { fill: "var(--chart-label)", fontSize: 11 };
const GRID = "var(--chart-grid)";
const AXIS = "var(--chart-axis)";

export interface TableColumn {
  key: string;
  label: string;
  align?: "right";
  format?: (value: never) => string;
}

/** Every chart has a table twin, so no value depends on hovering or on color. */
export function ChartPanel({
  title,
  subtitle,
  legend,
  columns,
  rows,
```

```
  children,
  empty,
}: {
  title: string;
  subtitle?: ReactNode;
  legend?: { label: string; color: string }[];
  columns: TableColumn[];
  rows: Record<string, unknown>[];
  children: ReactNode;
  empty?: ReactNode;
}) {
  const [showTable, setShowTable] = useState(false);
  return (
    <Panel
      title={title}
      subtitle={subtitle}
      actions={
        rows.length > 0 && (
          <Button
            variant="ghost"
            size="icon-sm"
            aria-label={showTable ? "Show chart" : "Show table"}
            title={showTable ? "Show chart" : "Show table"}
            onClick={() => setShowTable((v) => !v)}
          >
            {showTable ? <ChartColumn /> : <Table2 />}
          </Button>
        )
      }
    >
      {rows.length === 0 ? (
        empty
      ) : showTable ? (
        <div className="max-h-56 overflow-auto">
          <table className="w-full text-xs">
            <thead>
              <tr className="border-b text-muted-foreground">
                {columns.map((c) => (
                  <th key={c.key} className={`py-1.5 pr-3 font-medium ${c.align === "right" ? "text-right" : "text-left"}`}>
                    {c.label}
                  </th>
                ))}
              </tr>
            </thead>
            <tbody className="tabular">
              {rows.map((row, i) => (
                <tr key={i} className="border-b last:border-0">
                  {columns.map((c) => (
                    <td key={c.key} className={`py-1 pr-3 ${c.align === "right" ? "text-right" : ""}`}>
                      {c.format ? c.format(row[c.key] as never) : String(row[c.key] ?? "N/A")}
                    </td>
                  ))}
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      ) : (
        <>
          {legend && (
            <ul className="mb-2 flex flex-wrap gap-x-4 gap-y-1 text-xs text-muted-foreground">
              {legend.map((item) => (
                <li key={item.label} className="flex items-center gap-1.5">
                  <span className="inline-block h-0.5 w-3 rounded-full" style={{ background: item.color }} aria-hidden />
                  {item.label}
                </li>
              ))}
            </ul>
          )}
          {children}
        </>
      )}
    </Panel>
  );
}

function TooltipBox({ title, rows }: { title: string; rows: { label: string; value: string; color?: string }[] }) {
  return (
    <div className="rounded-md border bg-popover px-2.5 py-2 text-xs shadow-sm">
      <p className="mb-1 font-medium">{title}</p>
      {rows.map((row) => (
        <p key={row.label} className="tabular flex items-center justify-between gap-4 text-muted-foreground">
          <span className="flex items-center gap-1.5">
            {row.color && <span className="inline-block size-2 rounded-full" style={{ background: row.color }} aria-hidden />}
            {row.label}
          </span>
          <span className="text-foreground">{row.value}</span>
        </p>
      ))}
    </div>
  );
}
```

```
/** Logical and stored bytes over time, on one axis. The gap between the lines is the saving. */
export function StorageOverTimeChart({ data, height = 220 }: { data: { taken_at: string; logical_bytes: number; stored_bytes:
number }[]; height?: number }) {
  const series = data.map((d, index) => ({ ...d, index, label: parseTime(d.taken_at).toLocaleString("en-GB", { day: "2-digit",
month: "short", hour: "2-digit", minute: "2-digit", second: "2-digit" }) }));
  return (
    <ResponsiveContainer width="100%" height={height}>
      <AreaChart data={series} margin={{ top: 6, right: 8, bottom: 0, left: 0 }}>
        <CartesianGrid stroke={GRID} vertical={false} />
        <XAxis dataKey="index" tick={false} axisLine={{ stroke: AXIS }} tickLine={false} height={8} />
        <YAxis tick={axisTick} tickFormatter={(v) => formatBytes(v, 0)} axisLine={false} tickLine={false} width={62} />
        <Tooltip
          cursor={{ stroke: AXIS }}
          content={({ active, payload }) => {
            if (!active || !payload?.length) return null;
            const point = payload[0].payload as (typeof series)[number];
            return (
              <TooltipBox
                title={point.label}
                rows={[
                  { label: "Logical", value: formatBytes(point.logical_bytes), color: "var(--chart-label)" },
                  { label: "Stored", value: formatBytes(point.stored_bytes), color: "var(--series-1)" },
                ]}
              />
            );
          }}
        />
        <Area type="stepAfter" dataKey="logical_bytes" stroke="var(--chart-label)" strokeWidth={2} fill="none"
isAnimationActive={false} />
        <Area type="stepAfter" dataKey="stored_bytes" stroke="var(--series-1)" strokeWidth={2} fill="var(--series-1)"
fillOpacity={0.1} isAnimationActive={false} />
      </AreaChart>
    </ResponsiveContainer>
  );
}

type Row = Record<string, unknown>;

export function ColumnChart({
  data,
  x,
  y,
  xFormat,
  yFormat = formatCount,
  seriesLabel,
}: {
  data: Row[];
  x: string;
  y: string;
  xFormat?: (value: never) => string;
  yFormat?: (value: number) => string;
  seriesLabel: string;
}) {
  return (
    <ResponsiveContainer width="100%" height={200}>
      <BarChart data={data} margin={{ top: 6, right: 8, bottom: 0, left: 0 }}>
        <CartesianGrid stroke={GRID} vertical={false} />
        <XAxis dataKey={x} tick={axisTick} tickFormatter={xFormat as never} axisLine={{ stroke: AXIS }} tickLine={false}
interval={0} height={24} />
        <YAxis tick={axisTick} tickFormatter={(v) => yFormat(v)} axisLine={false} tickLine={false} width={56}
allowDecimals={false} />
        <Tooltip
          cursor={{ fill: "var(--muted)", opacity: 0.6 }}
          content={({ active, payload }) => {
            if (!active || !payload?.length) return null;
            const point = payload[0].payload as Row;
            const heading = xFormat ? xFormat(point[x] as never) : String(point[x]);
            return <TooltipBox title={heading} rows={[{ label: seriesLabel, value: yFormat(point[y] as number), color: "var(--
series-1)" }]} />;
          }}
        />
        <Bar dataKey={y} fill="var(--series-1)" maxBarSize={24} radius={[4, 4, 0, 0]} isAnimationActive={false} />
      </BarChart>
    </ResponsiveContainer>
  );
}

export function HorizontalBars({
  data,
  category,
  value,
  format,
  seriesLabel,
}: {
  data: Row[];
  category: string;
  value: string;
  format: (value: number) => string;
  seriesLabel: string;
}) {
  const height = Math.max(120, data.length * 34 + 10);
  return (
    <ResponsiveContainer width="100%" height={height}>
```

```
      <BarChart layout="vertical" data={data} margin={{ top: 0, right: 70, bottom: 0, left: 0 }}>
        <XAxis type="number" hide />
        <YAxis type="category" dataKey={category} tick={axisTick} axisLine={{ stroke: AXIS }} tickLine={false} width={70} />
        <Tooltip
          cursor={{ fill: "var(--muted)", opacity: 0.6 }}
          content={({ active, payload }) => {
            if (!active || !payload?.length) return null;
            const point = payload[0].payload as Row;
            return <TooltipBox title={String(point[category])} rows={[{ label: seriesLabel, value: format(point[value] as number),
color: "var(--series-1)" }]} />;
          }}
        />
        <Bar
          dataKey={value}
          fill="var(--series-1)"
          maxBarSize={18}
          radius={[0, 4, 4, 0]}
          isAnimationActive={false}
          label={{ position: "right", fill: "var(--chart-label)", fontSize: 11, formatter: (v: unknown) => format(v as number) }}
        />
      </BarChart>
    </ResponsiveContainer>
  );
}

/** One whole split into parts: what is kept and what deduplication avoided. */
export function ReductionBar({ logical, stored }: { logical: number; stored: number }) {
  if (logical <= 0) return null;
  const kept = Math.min(stored, logical);
  const avoided = Math.max(logical - stored, 0);
  const keptShare = (kept / logical) * 100;
  const avoidedShare = (avoided / logical) * 100;
  return (
    <div>
      <div
        className="flex h-7 w-full gap-0.5"
        role="img"
        aria-label={`${formatBytes(kept)} stored, ${keptShare.toFixed(1)} percent. ${formatBytes(avoided)} avoided,
${avoidedShare.toFixed(1)} percent.`}
      >
        {kept > 0 && (
          <div className="flex items-center overflow-hidden rounded bg-series-1 px-2 text-xs font-medium text-white" style={{ width:
`${keptShare}%` }} title={`Stored: ${formatBytes(kept)}`}>
            {keptShare >= 12 && <span>{formatBytes(kept)}</span>}
          </div>
        )}
        {avoided > 0 && (
          <div className="flex items-center overflow-hidden rounded bg-avoided px-2 text-xs font-medium text-avoided-ink"
style={{ width: `${avoidedShare}%` }} title={`Avoided: ${formatBytes(avoided)}`}>
            {avoidedShare >= 12 && <span>{formatBytes(avoided)}</span>}
          </div>
        )}
      </div>
      <ul className="mt-2 flex flex-wrap gap-x-4 gap-y-1 text-xs text-muted-foreground">
        <li className="flex items-center gap-1.5">
          <span className="inline-block size-2 rounded-sm bg-series-1" aria-hidden />
          Stored content
        </li>
        <li className="flex items-center gap-1.5">
          <span className="inline-block size-2 rounded-sm bg-avoided" aria-hidden />
          Avoided by deduplication
        </li>
      </ul>
    </div>
  );
}

/** Two comparable values as thin bars on one shared scale. */
export function BeforeAfter({
  rows,
  format = formatBytes,
}: {
  rows: { label: string; before: number; after: number }[];
  format?: (value: number) => string;
}) {
  return (
    <div className="space-y-4">
      {rows.map((row) => {
        const max = Math.max(row.before, row.after, 1);
        return (
          <div key={row.label}>
            <p className="mb-1.5 text-xs text-muted-foreground">{row.label}</p>
            <div className="space-y-1">
              {[
                { name: "Before", value: row.before, className: "bg-[var(--chart-label)]" },
                { name: "After", value: row.after, className: "bg-series-1" },
              ].map((part) => (
                <div key={part.name} className="flex items-center gap-2">
                  <span className="w-10 shrink-0 text-[11px] text-muted-foreground">{part.name}</span>
                  <div className="h-3 min-w-0 flex-1">
                    <div className={`h-full rounded-r ${part.className}`} style={{ width: `${(part.value / max) * 100}%`, minWidth:
part.value > 0 ? 2 : 0 }} />
                  </div>
```

```
                <span className="tabular w-20 shrink-0 text-right text-xs">{format(part.value)}</span>
              </div>
            ))}
          </div>
        </div>
      );
    })}
  </div>
);
}
```

11.6 Frontend: pages

frontend/app/page.tsx 320 lines

```
"use client";

import { useQuery } from "@tanstack/react-query";
import { ArrowRight, FolderTree, Inbox } from "lucide-react";
import Link from "next/link";

import { ColumnChart, ChartPanel, HorizontalBars, ReductionBar, StorageOverTimeChart } from "@/components/charts";
import { EmptyState, ErrorNotice, PageHeader, Panel, ProvenanceTag, RowsSkeleton, Stat, Status, type Provenance } from "@/
components/common";
import { Button } from "@/components/ui/button";
import { api } from "@/lib/api";
import { formatBytes, formatCount, formatDateTime, formatPercent, formatRatio, formatTime, MODE_LABEL, chunkSizeLabel } from "@/lib/
format";
import type { AuditEvent, OptimizationRecord, Overview, Page, Snapshot } from "@/lib/types";

function Ladder({ data }: { data: Overview }) {
  const { derived, measured, estimated } = data;
  const rows: { label: string; hint: string; value: string; kind: Provenance }[] = [
    { label: "Logical data", hint: "What users see: every file at full size", value: formatBytes(derived.logical_bytes), kind:
"derived" },
    { label: "Stored content", hint: "Unique data kept, before HDFS replicates it", value:
formatBytes(derived.stored_content_bytes), kind: "derived" },
    {
      label: `Replication-adjusted (×${estimated.replication_factor ?? "?"})`,
      hint: "Stored content times the replication factor",
      value: formatBytes(estimated.replication_adjusted_bytes),
      kind: "estimated",
    },
    {
      label: "HDFS space consumed",
      hint: "NameNode content summary, including replicas",
      value: measured.available ? formatBytes(measured.space_consumed) : "Unavailable",
      kind: "measured",
    },
    {
      label: "Metadata in HDFS",
      hint: "Chunk manifests, before replication",
      value: measured.available ? formatBytes(measured.areas.manifests.length) : "Unavailable",
      kind: "measured",
    },
  ];
  return (
    <ul className="divide-y">
      {rows.map((row) => (
        <li key={row.label} className="flex items-center justify-between gap-3 py-2 first:pt-0 last:pb-0">
          <div className="min-w-0">
            <p className="text-[13px]">{row.label}</p>
            <p className="truncate text-[11px] text-muted-foreground">{row.hint}</p>
          </div>
          <div className="shrink-0 text-right">
            <p className="tabular text-[13px] font-medium">{row.value}</p>
            <ProvenanceTag kind={row.kind} />
          </div>
        </li>
      ))}
    </ul>
  );
}

function ClusterCard({ data }: { data: Overview }) {
  const cluster = data.cluster;
  if (!cluster.reachable) {
    return (
      <Panel title="Cluster">
        <Status tone="critical">NameNode unreachable</Status>
        <p className="mt-2 text-xs break-words text-muted-foreground">{cluster.error}</p>
      </Panel>
    );
  }
  const tone = cluster.health === "healthy" ? "good" : cluster.health === "degraded" ? "warning" : "critical";
  return (
    <Panel
      title="Cluster"
      actions={
        <Link href="/cluster" className="text-xs text-muted-foreground hover:text-foreground">
          Details
        </Link>
      }
    >
```

```
      <Status tone={tone} className="font-medium">
        {cluster.health === "healthy" ? "Healthy" : cluster.health === "degraded" ? "Degraded" : "Critical"}
      </Status>
      <dl className="mt-3 grid grid-cols-2 gap-x-3 gap-y-2 text-xs">
        <div>
          <dt className="text-muted-foreground">Active DataNodes</dt>
          <dd className="text-[13px] font-medium">
            {cluster.datanodes.live} live{cluster.datanodes.dead ? `, ${cluster.datanodes.dead} dead` : ""}
          </dd>
        </div>
        <div>
          <dt className="text-muted-foreground">Replication</dt>
          <dd className="text-[13px] font-medium">Factor {data.estimated.replication_factor ?? "N/A"}</dd>
        </div>
        <div>
          <dt className="text-muted-foreground">Under-replicated</dt>
          <dd className="text-[13px] font-medium">{formatCount(cluster.namespace.under_replicated_blocks)} blocks</dd>
        </div>
        <div>
          <dt className="text-muted-foreground">Hadoop</dt>
          <dd className="text-[13px] font-medium">{cluster.namenode.software_version}</dd>
        </div>
      </dl>
      {cluster.problems.length > 0 && <p className="mt-2 text-xs text-muted-foreground">{cluster.problems[0]}</p>}
    </Panel>
  );
}

export default function DashboardPage() {
  const overview = useQuery({ queryKey: ["overview"], queryFn: () => api<Overview>("/api/v1/analytics/overview"), refetchInterval:
15_000 });
  const history = useQuery({
    queryKey: ["history"],
    queryFn: () => api<{ snapshots: Snapshot[]; optimizations: OptimizationRecord[] }>("/api/v1/analytics/history"),
    refetchInterval: 15_000,
  });
  const events = useQuery({ queryKey: ["audit", "recent"], queryFn: () => api<Page<AuditEvent>>("/api/v1/audit/events?page_size=7"),
refetchInterval: 15_000 });

  if (overview.isError) {
    return (
      <>
        <PageHeader title="Dashboard" />
        <ErrorNotice error={overview.error} onRetry={() => overview.refetch()} />
      </>
    );
  }
  if (!overview.data) {
    return (
      <>
        <PageHeader title="Dashboard" />
        <RowsSkeleton rows={8} />
      </>
    );
  }

  const data = overview.data;
  const { derived, distributions } = data;
  const empty = derived.logical_files === 0;
  const snapshots = history.data?.snapshots ?? [];
  const optimizations = history.data?.optimizations ?? [];

  return (
    <>
      <PageHeader title="Dashboard" description="Live storage state of the managed HDFS namespace. Every value comes from the
backend, and states how it is known." />

      <div className="grid gap-4 xl:grid-cols-[minmax(0,1fr)_300px]">
        <Panel>
          {empty ? (
            <EmptyState
              icon={FolderTree}
              title="No files are managed yet"
              action={
                <div className="flex flex-wrap justify-center gap-2">
                  <Button size="sm" render={<Link href="/files" />}>
                    Upload files
                  </Button>
                  <Button size="sm" variant="outline" render={<Link href="/lab" />}>
                    Import the dataset
                  </Button>
                </div>
              }
            >
              Savings cannot be calculated until something is stored, so they show as N/A instead of 0%.
            </EmptyState>
          ) : (
            <>
              <div className="flex flex-wrap items-end justify-between gap-x-8 gap-y-3">
                <div>
                  <p className="text-xs text-muted-foreground">Storage reduction</p>
                  <p className="mt-1 text-5xl font-semibold tracking-tight">{formatPercent(derived.savings_percent)}</p>
                </div>
```

```
              <p className="max-w-md pb-1 text-[13px] text-muted-foreground">
                {formatBytes(derived.logical_bytes)} of logical data is held in {formatBytes(derived.stored_content_bytes)} of
stored content, before replication.
              </p>
            </div>
            <div className="mt-4">
              <ReductionBar logical={derived.logical_bytes} stored={derived.stored_content_bytes} />
            </div>
          </>
        )}
        <div className="mt-5 grid grid-cols-2 gap-x-4 gap-y-4 border-t pt-4 sm:grid-cols-3 lg:grid-cols-6">
          <Stat label="Logical files" value={formatCount(derived.logical_files)} provenance="derived" />
          <Stat label="Unique contents" value={formatCount(derived.unique_files)} provenance="derived" />
          <Stat label="Duplicates detected" value={formatCount(derived.duplicate_files)} provenance="derived" />
          <Stat label="Unique data retained" value={formatBytes(derived.stored_content_bytes)} provenance="derived" />
          <Stat label="Bytes avoided" value={empty ? "N/A" : formatBytes(derived.bytes_avoided)} provenance="derived" />
          <Stat label="Dedup ratio" value={formatRatio(derived.dedup_ratio)} provenance="derived" />
        </div>
      </Panel>
      <ClusterCard data={data} />
    </div>

    <div className="mt-4 grid gap-4 xl:grid-cols-[minmax(0,1fr)_380px]">
      <ChartPanel
        title="Logical and stored data over time"
        subtitle="Each point is a snapshot taken after an upload, deletion or optimization. The gap is the saving."
        legend={[
          { label: "Logical", color: "var(--chart-label)" },
          { label: "Stored", color: "var(--series-1)" },
        ]}
        columns={[
          { key: "taken_at", label: "Time", format: (v: string) => formatDateTime(v) },
          { key: "reason", label: "After" },
          { key: "logical_bytes", label: "Logical", align: "right", format: (v: number) => formatBytes(v) },
          { key: "stored_bytes", label: "Stored", align: "right", format: (v: number) => formatBytes(v) },
        ]}
        rows={snapshots as never}
        empty={<EmptyState title="No snapshots yet">The first upload records the first point.</EmptyState>}
      >
        <StorageOverTimeChart data={snapshots} />
      </ChartPanel>
      <Panel title="Where the bytes are" subtitle="From logical size down to what HDFS holds">
        <Ladder data={data} />
      </Panel>
    </div>

    <div className="mt-4 grid gap-4 md:grid-cols-2 xl:grid-cols-3">
      <ChartPanel
        title="Duplicate distribution"
        subtitle="Distinct contents, by number of logical copies"
        columns={[
          { key: "copies", label: "Copies" },
          { key: "groups", label: "Contents", align: "right" },
        ]}
        rows={distributions.duplicate_distribution as never}
        empty={<EmptyState title="Nothing to show yet" />}
      >
        <ColumnChart data={distributions.duplicate_distribution as never} x="copies" y="groups" seriesLabel="Contents"
xFormat={((v: number) => `${v}×`) as never} />
      </ChartPanel>
      <ChartPanel
        title="File types"
        subtitle="Logical bytes per extension"
        columns={[
          { key: "extension", label: "Type" },
          { key: "files", label: "Files", align: "right" },
          { key: "bytes", label: "Size", align: "right", format: (v: number) => formatBytes(v) },
        ]}
        rows={distributions.file_types.slice(0, 6) as never}
        empty={<EmptyState title="Nothing to show yet" />}
      >
        <HorizontalBars data={distributions.file_types.slice(0, 6) as never} category="extension" value="bytes" format={(v) =>
formatBytes(v)} seriesLabel="Size" />
      </ChartPanel>
      <ChartPanel
        title="Upload trend"
        subtitle="Files uploaded per day (UTC)"
        columns={[
          { key: "day", label: "Day" },
          { key: "files", label: "Files", align: "right" },
          { key: "bytes", label: "Size", align: "right", format: (v: number) => formatBytes(v) },
        ]}
        rows={distributions.upload_trend as never}
        empty={<EmptyState title="Nothing to show yet" />}
      >
        <ColumnChart data={distributions.upload_trend as never} x="day" y="files" seriesLabel="Files" />
      </ChartPanel>
    </div>

    <div className="mt-4 grid gap-4 xl:grid-cols-2">
      <Panel title="Optimization history" subtitle="Verified results from completed jobs" flush>
        {optimizations.length === 0 ? (
          <EmptyState title="No optimization has run">
```

```
              <Link href="/optimize" className="underline underline-offset-2">
                Review a dry run
              </Link>{" "}
              to see what could be reclaimed.
            </EmptyState>
          ) : (
            <div className="overflow-x-auto px-4 pb-3">
              <table className="w-full text-[13px]">
                <thead>
                  <tr className="border-b text-left text-xs text-muted-foreground">
                    <th className="py-2 pr-3 font-medium">Finished</th>
                    <th className="py-2 pr-3 font-medium">Mode</th>
                    <th className="py-2 pr-3 text-right font-medium">Files</th>
                    <th className="py-2 pr-3 text-right font-medium">Stored before</th>
                    <th className="py-2 text-right font-medium">Stored after</th>
                  </tr>
                </thead>
                <tbody className="tabular">
                  {optimizations.slice(0, 5).map((row) => (
                    <tr key={row.job_id} className="border-b last:border-0">
                      <td className="py-1.5 pr-3 whitespace-nowrap">{formatDateTime(row.created_at)}</td>
                      <td className="py-1.5 pr-3 whitespace-nowrap">
                        {MODE_LABEL[row.mode]}
                        {row.chunk_size ? ` (${chunkSizeLabel(row.chunk_size)})` : ""}
                      </td>
                      <td className="py-1.5 pr-3 text-right">{row.files_processed}</td>
                      <td className="py-1.5 pr-3 text-right">{formatBytes(row.before.derived.stored_content_bytes)}</td>
                      <td className="py-1.5 text-right">{formatBytes(row.after.derived.stored_content_bytes)}</td>
                    </tr>
                  ))}
                </tbody>
              </table>
            </div>
          )}
        </Panel>
        <Panel
          title="Recent operations"
          flush
          actions={
            <Link href="/activity" className="inline-flex items-center gap-1 text-xs text-muted-foreground hover:text-foreground">
              All activity <ArrowRight className="size-3" aria-hidden />
            </Link>
          }
        >
          {events.data?.items.length ? (
            <ul className="divide-y px-4 pb-2">
              {events.data.items.map((event) => (
                <li key={event.id} className="flex items-baseline gap-3 py-1.5 text-xs">
                  <span className="tabular shrink-0 text-muted-foreground">{formatTime(event.occurred_at)}</span>
                  <span className="min-w-0 truncate" title={event.message}>
                    {event.severity !== "info" && <span className="mr-1 font-medium capitalize">{event.severity}:</span>}
                    {event.message}
                  </span>
                </li>
              ))}
            </ul>
          ) : (
            <EmptyState icon={Inbox} title="No operations recorded yet" />
          )}
        </Panel>
      </div>
    </>
  );
}
```

frontend/app/activity/page.tsx 135 lines

```
"use client";

import { keepPreviousData, useQuery } from "@tanstack/react-query";
import { ChevronLeft, ChevronRight, ScrollText } from "lucide-react";
import { useState } from "react";

import { Choice, EmptyState, ErrorNotice, PageHeader, Panel, RowsSkeleton, Status } from "@/components/common";
import { Button } from "@/components/ui/button";
import { api } from "@/lib/api";
import { formatDateTime } from "@/lib/format";
import type { AuditEvent, Page } from "@/lib/types";
import { cn } from "@/lib/utils";

const TYPE_FILTERS = [
  { value: "all", label: "All events" },
  { value: "file.", label: "Files" },
  { value: "dedup.", label: "Duplicate detections" },
  { value: "optimize.", label: "Optimization" },
  { value: "integrity.", label: "Integrity checks" },
  { value: "replication.", label: "Replication" },
  { value: "storage.", label: "Storage maintenance" },
  { value: "dataset.", label: "Datasets" },
  { value: "experiment.", label: "Experiments" },
  { value: "job.", label: "Jobs" },
  { value: "request.", label: "Failed requests" },
  { value: "settings.", label: "Settings" },
];
```

```
const TONE = { info: "info", warning: "warning", error: "critical" } as const;

export default function ActivityPage() {
  const [type, setType] = useState("all");
  const [severity, setSeverity] = useState("all");
  const [page, setPage] = useState(1);
  const pageSize = 40;
  const params = new URLSearchParams({ page: String(page), page_size: String(pageSize) });
  if (type !== "all") params.set("event_type", type);
  if (severity !== "all") params.set("severity", severity);

  const events = useQuery({
    queryKey: ["audit", params.toString()],
    queryFn: () => api<Page<AuditEvent>>(`/api/v1/audit/events?${params}`),
    placeholderData: keepPreviousData,
    refetchInterval: 10_000,
  });
  const totalPages = events.data ? Math.max(1, Math.ceil(events.data.total / pageSize)) : 1;

  return (
    <>
      <PageHeader title="Activity and audit" description="An append-only record of uploads, duplicate detections, jobs, deletions,
restorations, integrity checks, replication changes and failures." />
      <Panel flush>
        <div className="flex flex-wrap items-center gap-2 border-b p-3">
          <Choice
            ariaLabel="Filter by event type"
            value={type}
            onChange={(v) => {
              setType(v);
              setPage(1);
            }}
            options={TYPE_FILTERS}
          />
          <Choice
            ariaLabel="Filter by severity"
            value={severity}
            onChange={(v) => {
              setSeverity(v);
              setPage(1);
            }}
            options={[
              { value: "all", label: "Any severity" },
              { value: "info", label: "Info" },
              { value: "warning", label: "Warnings" },
              { value: "error", label: "Errors" },
            ]}
          />
        </div>
        {events.isError ? (
          <div className="p-4">
            <ErrorNotice error={events.error} onRetry={() => events.refetch()} />
          </div>
        ) : !events.data ? (
          <RowsSkeleton rows={10} />
        ) : events.data.items.length === 0 ? (
          <EmptyState icon={ScrollText} title="No events match">
            Operations appear here as they happen.
          </EmptyState>
        ) : (
          <div className={cn("overflow-x-auto transition-opacity", events.isPlaceholderData && "opacity-60")}>
            <table className="w-full text-[13px]">
              <thead>
                <tr className="border-b text-left text-xs text-muted-foreground">
                  <th className="px-3 py-2 font-medium">Time</th>
                  <th className="px-3 py-2 font-medium">Severity</th>
                  <th className="px-3 py-2 font-medium">Event</th>
                  <th className="px-3 py-2 font-medium">Details</th>
                </tr>
              </thead>
              <tbody>
                {events.data.items.map((event) => (
                  <tr key={event.id} className="border-b align-baseline last:border-0">
                    <td className="tabular px-3 py-1.5 text-xs whitespace-nowrap text-muted-
foreground">{formatDateTime(event.occurred_at)}</td>
                    <td className="px-3 py-1.5 whitespace-nowrap">
                      <Status tone={TONE[event.severity]} className="text-xs capitalize">
                        {event.severity}
                      </Status>
                    </td>
                    <td className="px-3 py-1.5 font-mono text-xs whitespace-nowrap">{event.event_type}</td>
                    <td className="min-w-80 px-3 py-1.5 text-xs break-words">{event.message}</td>
                  </tr>
                ))}
              </tbody>
            </table>
          </div>
        )}
        {events.data && events.data.total > 0 && (
          <div className="flex items-center justify-between border-t px-3 py-2 text-xs text-muted-foreground">
            <span className="tabular">{events.data.total.toLocaleString("en-US")} events</span>
            <div className="flex items-center gap-2">
```

```
              <span className="tabular">
                Page {page} of {totalPages}
              </span>
              <Button variant="outline" size="icon-sm" aria-label="Previous page" disabled={page <= 1} onClick={() => setPage(page -
1)}>
                <ChevronLeft />
              </Button>
              <Button variant="outline" size="icon-sm" aria-label="Next page" disabled={page >= totalPages} onClick={() =>
setPage(page + 1)}>
                <ChevronRight />
              </Button>
            </div>
          </div>
        )}
      </Panel>
    </>
  );
}
```

frontend/app/analytics/page.tsx 195 lines

```
"use client";

import { useQuery } from "@tanstack/react-query";
import { BarChart3, Download } from "lucide-react";

import { ChartPanel, StorageOverTimeChart } from "@/components/charts";
import { EmptyState, ErrorNotice, PageHeader, Panel, RowsSkeleton, Stat } from "@/components/common";
import { Button } from "@/components/ui/button";
import { API_URL, api } from "@/lib/api";
import { formatBytes, formatCount, formatDateTime, formatMs, formatPercent, formatRatio, MODE_LABEL } from "@/lib/format";
import type { IntegritySummary, OptimizationRecord, Overview, PerformanceOp, Snapshot } from "@/lib/types";

const OPERATION_LABEL: Record<string, string> = {
  hash: "SHA-256 hashing",
  upload: "Upload, end to end",
  chunking: "Chunking and storing",
  restore: "Restore (download)",
  verify: "Integrity verification",
  optimize: "Optimization job",
};

export default function AnalyticsPage() {
  const overview = useQuery({ queryKey: ["overview"], queryFn: () => api<Overview>("/api/v1/analytics/overview") });
  const performance = useQuery({ queryKey: ["performance"], queryFn: () => api<{ operations: PerformanceOp[]; integrity:
IntegritySummary }>("/api/v1/analytics/performance") });
  const history = useQuery({ queryKey: ["history"], queryFn: () => api<{ snapshots: Snapshot[]; optimizations:
OptimizationRecord[] }>("/api/v1/analytics/history") });

  if (overview.isError) {
    return (
      <>
        <PageHeader title="Analytics" />
        <ErrorNotice error={overview.error} onRetry={() => overview.refetch()} />
      </>
    );
  }
  if (!overview.data) {
    return (
      <>
        <PageHeader title="Analytics" />
        <RowsSkeleton rows={8} />
      </>
    );
  }

  const { derived, measured } = overview.data;
  const operations = performance.data?.operations ?? [];
  const integrity = performance.data?.integrity;
  const restore = operations.filter((o) => o.operation === "restore").reduce((sum, o) => sum + o.samples, 0);
  const throughput = operations
    .filter((o) => o.throughput_mib_s !== null)
    .map((o) => ({ label: `${OPERATION_LABEL[o.operation] ?? o.operation}${o.mode ? `, ${MODE_LABEL[o.mode].toLowerCase()}` : ""}`,
value: o.throughput_mib_s as number }))
    .sort((a, b) => b.value - a.value);
  const hashMs = operations.filter((o) => o.operation === "hash").reduce((sum, o) => sum + o.total_ms, 0);
  const uploadMs = operations.filter((o) => o.operation === "upload").reduce((sum, o) => sum + o.total_ms, 0);
  const chunkingMs = operations.filter((o) => o.operation === "chunking").reduce((sum, o) => sum + o.total_ms, 0);

  return (
    <>
      <PageHeader
        title="Analytics and reports"
        description="Volume, deduplication and performance, from measurements recorded by each operation."
        actions={
          <Button size="sm" variant="outline" render={<a href={`${API_URL}/api/v1/analytics/report.csv`} />}>
            <Download /> Download CSV summary
          </Button>
        }
      />

      <Panel className="mb-4">
        <div className="grid grid-cols-2 gap-4 sm:grid-cols-3 lg:grid-cols-6">
          <Stat label="Logical volume" value={formatBytes(derived.logical_bytes)} provenance="derived" />
```

```
          <Stat label="Unique data stored" value={formatBytes(derived.stored_content_bytes)} provenance="derived" />
          <Stat label="Saved bytes" value={formatBytes(derived.bytes_avoided)} provenance="derived" />
          <Stat label="Deduplication rate" value={formatPercent(derived.savings_percent)} provenance="derived" />
          <Stat label="Deduplication ratio" value={formatRatio(derived.dedup_ratio)} provenance="derived" />
          <Stat label="Duplicate files" value={formatCount(derived.duplicate_files)} provenance="derived" />
        </div>
        <div className="mt-4 grid grid-cols-2 gap-4 border-t pt-4 sm:grid-cols-3 lg:grid-cols-6">
          <Stat label="HDFS length" value={measured.available ? formatBytes(measured.length) : "Unavailable"}
provenance="measured" />
          <Stat label="HDFS space consumed" value={measured.available ? formatBytes(measured.space_consumed) : "Unavailable"}
provenance="measured" />
          <Stat label="HDFS files" value={measured.available ? formatCount(measured.file_count) : "Unavailable"}
provenance="measured" />
          <Stat label="Chunks stored" value={formatCount(derived.chunk_count)} provenance="derived" />
          <Stat label="Duplicate chunk refs" value={formatCount(derived.duplicate_chunk_references)} provenance="derived" />
          <Stat label="Manifest metadata" value={formatBytes(derived.manifest_bytes)} provenance="derived" />
        </div>
      </Panel>

      <div className="grid gap-4 xl:grid-cols-2">
        <ChartPanel
          title="Throughput by operation"
          subtitle="Total bytes divided by total time. Measured on this machine and cluster."
          columns={[
            { key: "label", label: "Operation" },
            { key: "value", label: "MiB/s", align: "right", format: (v: number) => v.toFixed(1) },
          ]}
          rows={throughput}
          empty={<EmptyState icon={BarChart3} title="No operations measured yet" />}
        >
          <ThroughputBars data={throughput} />
        </ChartPanel>
        <ChartPanel
          title="Saved data over time"
          legend={[
            { label: "Logical", color: "var(--chart-label)" },
            { label: "Stored", color: "var(--series-1)" },
          ]}
          columns={[
            { key: "taken_at", label: "Time", format: (v: string) => formatDateTime(v) },
            { key: "logical_bytes", label: "Logical", align: "right", format: (v: number) => formatBytes(v) },
            { key: "stored_bytes", label: "Stored", align: "right", format: (v: number) => formatBytes(v) },
            { key: "saved_bytes", label: "Saved", align: "right", format: (v: number) => formatBytes(v) },
          ]}
          rows={(history.data?.snapshots ?? []) as never}
          empty={<EmptyState title="No snapshots yet" />}
        >
          <StorageOverTimeChart data={history.data?.snapshots ?? []} height={400} />
        </ChartPanel>
      </div>

      <div className="mt-4 grid gap-4 xl:grid-cols-[minmax(0,1fr)_360px]">
        <Panel title="Measured operations" flush>
          {operations.length === 0 ? (
            <EmptyState title="No operations measured yet" />
          ) : (
            <div className="overflow-x-auto px-4 pb-3">
              <table className="w-full text-[13px]">
                <thead>
                  <tr className="border-b text-left text-xs text-muted-foreground">
                    <th className="py-2 pr-3 font-medium">Operation</th>
                    <th className="py-2 pr-3 font-medium">Mode</th>
                    <th className="py-2 pr-3 text-right font-medium">Samples</th>
                    <th className="py-2 pr-3 text-right font-medium">Bytes</th>
                    <th className="py-2 pr-3 text-right font-medium">Mean</th>
                    <th className="py-2 pr-3 text-right font-medium">Range</th>
                    <th className="py-2 text-right font-medium">MiB/s</th>
                  </tr>
                </thead>
                <tbody className="tabular">
                  {operations.map((op) => (
                    <tr key={`${op.operation}-${op.mode}`} className="border-b last:border-0">
                      <td className="py-1.5 pr-3 whitespace-nowrap">{OPERATION_LABEL[op.operation] ?? op.operation}</td>
                      <td className="py-1.5 pr-3 whitespace-nowrap">{op.mode ? MODE_LABEL[op.mode] : "N/A"}</td>
                      <td className="py-1.5 pr-3 text-right">{formatCount(op.samples)}</td>
                      <td className="py-1.5 pr-3 text-right">{formatBytes(op.bytes)}</td>
                      <td className="py-1.5 pr-3 text-right">{formatMs(op.mean_ms)}</td>
                      <td className="py-1.5 pr-3 text-right whitespace-nowrap">
                        {formatMs(op.min_ms)} to {formatMs(op.max_ms)}
                      </td>
                      <td className="py-1.5 text-right">{op.throughput_mib_s?.toFixed(1) ?? "N/A"}</td>
                    </tr>
                  ))}
                </tbody>
              </table>
            </div>
          )}
        </Panel>
        <div className="space-y-4">
          <Panel title="Integrity and restoration">
            <div className="grid grid-cols-2 gap-4">
              <Stat label="Checks passed" value={integrity ? `${integrity.checks_passed} of ${integrity.checks_total}` : "N/A"}
provenance="measured" />
```

```
              <Stat label="Checks failed" value={formatCount(integrity?.checks_failed)} provenance="measured" />
              <Stat label="Files never verified" value={formatCount(integrity?.files_never_verified)} />
              <Stat label="Verified restores" value={formatCount(restore)} provenance="measured" note="Downloads that passed
SHA-256" />
            </div>
          </Panel>
          <Panel title="Time spent">
            <div className="grid grid-cols-2 gap-4">
              <Stat label="Hashing, all uploads" value={formatMs(hashMs)} provenance="measured" note={uploadMs ? `${((hashMs /
uploadMs) * 100).toFixed(1)}% of upload time` : undefined} />
              <Stat label="Chunking and chunk writes" value={formatMs(chunkingMs)} provenance="measured" />
            </div>
          </Panel>
        </div>
      </div>
    </>
  );
}

function ThroughputBars({ data }: { data: { label: string; value: number }[] }) {
  // Labels here are long, so they sit above each bar rather than in a narrow axis.
  const max = Math.max(...data.map((d) => d.value), 1);
  return (
    <ul className="space-y-2">
      {data.map((row) => (
        <li key={row.label}>
          <div className="mb-0.5 flex items-baseline justify-between gap-3 text-xs">
            <span className="truncate text-muted-foreground">{row.label}</span>
            <span className="tabular shrink-0">{row.value.toFixed(1)} MiB/s</span>
          </div>
          <div className="h-2 rounded-r bg-series-1" style={{ width: `${(row.value / max) * 100}%`, minWidth: 2 }} />
        </li>
      ))}
    </ul>
  );
}
```

frontend/app/cluster/page.tsx 199 lines

```
"use client";

import { useQuery } from "@tanstack/react-query";
import { ExternalLink, RefreshCw, Server, ServerOff } from "lucide-react";

import { EmptyState, ErrorNotice, KeyValue, Notice, PageHeader, Panel, RowsSkeleton, Stat, Status } from "@/components/common";
import { Button } from "@/components/ui/button";
import { api } from "@/lib/api";
import { formatBytes, formatCount, formatDateTime } from "@/lib/format";
import { useClusterStatus } from "@/lib/queries";
import type { DataNode } from "@/lib/types";

function CapacityMeter({ used, nonDfs, total }: { used: number; nonDfs: number; total: number }) {
  const usedShare = total ? (used / total) * 100 : 0;
  const otherShare = total ? (nonDfs / total) * 100 : 0;
  return (
    <div>
      <div className="flex h-2 w-full gap-0.5 overflow-hidden rounded-full bg-muted" role="img" aria-label={`${usedShare.toFixed(2)}
percent used by HDFS, ${otherShare.toFixed(1)} percent used by other data`}>
        <div className="h-full rounded-full bg-series-1" style={{ width: `${usedShare}%`, minWidth: used > 0 ? 3 : 0 }} />
        <div className="h-full rounded-full bg-[var(--chart-label)]" style={{ width: `${otherShare}%` }} />
      </div>
      <ul className="mt-2 flex flex-wrap gap-x-4 gap-y-1 text-xs text-muted-foreground">
        <li className="flex items-center gap-1.5">
          <span className="inline-block size-2 rounded-sm bg-series-1" aria-hidden /> HDFS: {formatBytes(used)}
        </li>
        <li className="flex items-center gap-1.5">
          <span className="inline-block size-2 rounded-sm bg-[var(--chart-label)]" aria-hidden /> Other data on disk:
{formatBytes(nonDfs)}
        </li>
      </ul>
    </div>
  );
}

export default function ClusterPage() {
  const status = useClusterStatus();
  const nodes = useQuery({ queryKey: ["cluster", "nodes"], queryFn: () => api<{ checked_at: string; nodes: DataNode[] }>("/api/v1/
cluster/nodes"), refetchInterval: 10_000, retry: false });
  const data = status.data;

  const refresh = () => {
    status.refetch();
    nodes.refetch();
  };

  const actions = (
    <>
      {data && <span className="text-xs text-muted-foreground">Updated {formatDateTime(data.checked_at)}</span>}
      <Button size="sm" variant="outline" onClick={refresh}>
        <RefreshCw className={status.isFetching ? "animate-spin" : ""} /> Refresh
      </Button>
      {data?.namenode_url && (
        <Button size="sm" variant="outline" render={<a href={data.namenode_url} target="_blank" rel="noreferrer" />}>
```

```
          NameNode UI <ExternalLink />
        </Button>
      )}
    </>
  );

  if (status.isError) {
    return (
      <>
        <PageHeader title="Cluster monitoring" actions={actions} />
        <ErrorNotice error={status.error} onRetry={refresh} />
      </>
    );
  }
  if (!data) {
    return (
      <>
        <PageHeader title="Cluster monitoring" />
        <RowsSkeleton rows={8} />
      </>
    );
  }
  if (!data.reachable) {
    return (
      <>
        <PageHeader title="Cluster monitoring" actions={actions} />
        <Panel>
          <EmptyState icon={ServerOff} title="The NameNode is not responding">
            {data.error}
          </EmptyState>
        </Panel>
      </>
    );
  }

  const live = nodes.data?.nodes.filter((n) => n.state === "live").length ?? data.datanodes.live;
  const total = nodes.data?.nodes.length ?? live;
  const tone = data.health === "healthy" ? "good" : data.health === "degraded" ? "warning" : "critical";

  return (
    <>
      <PageHeader title="Cluster monitoring" description="Read from the NameNode's JMX interface on each refresh."
actions={actions} />

      {data.problems.length > 0 && (
        <div className="mb-4 space-y-2">
          {data.problems.map((problem) => (
            <Notice key={problem} tone="warning">
              {problem}
            </Notice>
          ))}
        </div>
      )}

      <Panel className="mb-4">
        <div className="grid grid-cols-2 gap-4 lg:grid-cols-5">
          <Stat label="Health" value={<Status tone={tone} className="text-xl font-semibold capitalize">{data.health}</Status>}
provenance="measured" />
          <Stat label="NameNode" value={<span className="capitalize">{data.namenode.state}</span>} provenance="measured" />
          <Stat label="DataNodes" value={`${data.datanodes.live} live`} provenance="measured" note={`${data.datanodes.dead} dead,
${data.datanodes.stale} stale`} />
          <Stat label="HDFS files" value={formatCount(data.namespace.files_total)} provenance="measured" note="Includes
directories" />
          <Stat label="Blocks" value={formatCount(data.namespace.blocks_total)} provenance="measured" />
        </div>
      </Panel>

      <div className="grid gap-4 lg:grid-cols-[minmax(0,1fr)_360px]">
        <Panel title="Capacity" subtitle="As reported by DataNodes">
          <div className="mb-4 grid grid-cols-3 gap-4">
            <Stat label="Configured" value={formatBytes(data.capacity.total_bytes)} />
            <Stat label="Used by HDFS" value={formatBytes(data.capacity.used_bytes)} />
            <Stat label="Remaining" value={formatBytes(data.capacity.remaining_bytes)} />
          </div>
          <CapacityMeter used={data.capacity.used_bytes} nonDfs={data.capacity.non_dfs_used_bytes}
total={data.capacity.total_bytes} />
          <p className="mt-3 text-xs text-muted-foreground">On a single host, every DataNode reports the same disk, so configured
capacity counts that disk once per DataNode.</p>
        </Panel>
        <Panel title="NameNode">
          <KeyValue
            rows={[
              { label: "Address", value: data.namenode.host_and_port },
              { label: "Hadoop version", value: data.namenode.software_version },
              { label: "Cluster ID", value: data.namenode.cluster_id },
              { label: "Filesystem state", value: data.namenode.fs_state },
              { label: "Safe mode", value: data.namenode.safemode ? "On" : "Off" },
              { label: "Connection", value: data.namenode_url },
            ]}
          />
        </Panel>
      </div>
```

```
      <Panel title={`DataNodes: ${live} live of ${total}`} className="mt-4" flush>
        {nodes.isError ? (
          <div className="p-4">
            <ErrorNotice error={nodes.error} />
          </div>
        ) : !nodes.data ? (
          <RowsSkeleton rows={3} />
        ) : nodes.data.nodes.length === 0 ? (
          <EmptyState icon={Server} title="No DataNodes have registered" />
        ) : (
          <div className="overflow-x-auto px-4 pb-3">
            <table className="w-full text-[13px]">
              <thead>
                <tr className="border-b text-left text-xs text-muted-foreground">
                  <th className="py-2 pr-3 font-medium">Node</th>
                  <th className="py-2 pr-3 font-medium">State</th>
                  <th className="py-2 pr-3 text-right font-medium">Last contact</th>
                  <th className="py-2 pr-3 text-right font-medium">Blocks</th>
                  <th className="py-2 pr-3 text-right font-medium">HDFS used</th>
                  <th className="py-2 pr-3 text-right font-medium">Remaining</th>
                  <th className="py-2 pr-3 font-medium">Admin state</th>
                  <th className="py-2 font-medium">Version</th>
                </tr>
              </thead>
              <tbody className="tabular">
                {nodes.data.nodes.map((node) => (
                  <tr key={node.name} className="border-b last:border-0">
                    <td className="py-1.5 pr-3 font-medium">{node.name}</td>
                    <td className="py-1.5 pr-3">
                      <Status tone={node.state === "live" ? "good" : "critical"} className="text-xs capitalize">
                        {node.state}
                      </Status>
                    </td>
                    <td className="py-1.5 pr-3 text-right">{node.last_contact_seconds ?? "N/A"} s ago</td>
                    <td className="py-1.5 pr-3 text-right">{formatCount(node.blocks)}</td>
                    <td className="py-1.5 pr-3 text-right">{formatBytes(node.used_bytes)}</td>
                    <td className="py-1.5 pr-3 text-right">{formatBytes(node.remaining_bytes)}</td>
                    <td className="py-1.5 pr-3">{node.admin_state ?? "N/A"}</td>
                    <td className="py-1.5">{node.version ?? "N/A"}</td>
                  </tr>
                ))}
              </tbody>
            </table>
          </div>
        )}
      </Panel>

      <Panel title="Block health" className="mt-4">
        <div className="grid grid-cols-2 gap-4 sm:grid-cols-3 lg:grid-cols-6">
          <Stat label="Under-replicated" value={formatCount(data.namespace.under_replicated_blocks)} />
          <Stat label="Missing" value={formatCount(data.namespace.missing_blocks)} />
          <Stat label="Corrupt" value={formatCount(data.namespace.corrupt_blocks)} />
          <Stat label="Pending replication" value={formatCount(data.namespace.pending_replication_blocks)} />
          <Stat label="Pending deletion" value={formatCount(data.namespace.pending_deletion_blocks)} />
          <Stat label="Excess replicas" value={formatCount(data.namespace.excess_blocks)} />
        </div>
      </Panel>
    </>
  );
}
```

frontend/app/duplicates/page.tsx 266 lines

```
"use client";

import { useQuery } from "@tanstack/react-query";
import { ChevronDown, ChevronRight, CopyCheck, RefreshCw, Sparkles } from "lucide-react";
import Link from "next/link";
import { useState } from "react";

import { EmptyState, ErrorNotice, HashText, Notice, PageHeader, Panel, ProvenanceTag, RowsSkeleton, Stat } from "@/components/
common";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Tabs, TabsContent, TabsList, TabsTrigger } from "@/components/ui/tabs";
import { api } from "@/lib/api";
import { chunkSizeLabel, formatBytes, formatCount, MODE_LABEL, PROVENANCE_LABEL, shortHash } from "@/lib/format";
import type { DuplicateGroup, DuplicateGroups, SharedChunks } from "@/lib/types";

/**
 * Shows how logical names resolve to stored bytes: converging on one box when deduplicated,
 * one box each when every copy is still stored separately.
 */
function RelationshipDiagram({ group }: { group: DuplicateGroup }) {
  const shown = group.members.slice(0, 6);
  const hidden = group.members.length - shown.length;
  const rowHeight = 30;
  const rows = shown.length + (hidden > 0 ? 1 : 0);
  const height = Math.max(rows * rowHeight + 8, 92);
  const y = (index: number) => 4 + rowHeight * index + rowHeight / 2 - 3;
  const middle = height / 2;
  const raw = group.members.filter((m) => m.storage_mode === "raw").length;
  const deduped = group.members.length - raw;
```

```tsx
  const label = (text: string, max: number) => (text.length > max ? `${text.slice(0, max - 1)}…` : text);

  return (
    <div className="overflow-x-auto">
      <svg
        viewBox={`0 0 640 ${height}`}
        className="h-auto w-full max-w-[800px] min-w-[520px]"
        role="img"
        aria-label={`${group.copies} logical files share SHA-256 ${shortHash(group.sha256)}. ${raw} stored as separate copies,
${deduped} share one stored form.`}
      >
        {shown.map((member, index) => {
          const isRaw = member.storage_mode === "raw";
          const targetY = isRaw ? y(index) : middle;
          return (
            <g key={member.id}>
              <rect x={0} y={y(index) - 11} width={190} height={22} rx={4} fill="var(--muted)" />
              <text x={8} y={y(index) + 4} fontSize={11} fill="var(--foreground)">
                {label(member.name, 30)}
              </text>
              <path d={`M190 ${y(index)} C 240 ${y(index)}, 240 ${middle}, 290 ${middle}`} fill="none" stroke="var(--chart-axis)"
strokeWidth={1.5} />
              <path d={`M430 ${middle} C 470 ${middle}, 470 ${targetY}, 510 ${targetY}`} fill="none" stroke={isRaw ? "var(--
series-2)" : "var(--series-1)"} strokeWidth={1.5} opacity={isRaw || index === 0 ? 1 : 0.35} />
              {isRaw && (
                <>
                  <rect x={510} y={y(index) - 11} width={130} height={22} rx={4} fill="none" stroke="var(--series-2)"
strokeWidth={1.5} />
                  <text x={520} y={y(index) + 4} fontSize={11} fill="var(--foreground)">
                    Own stored copy
                  </text>
                </>
              )}
            </g>
          );
        })}
        {hidden > 0 && (
          <text x={8} y={y(shown.length) + 4} fontSize={11} fill="var(--chart-label)">
            and {hidden} more
          </text>
        )}
        <rect x={290} y={middle - 22} width={140} height={44} rx={6} fill="var(--card)" stroke="var(--chart-axis)" />
        <text x={360} y={middle - 4} fontSize={10} textAnchor="middle" fill="var(--chart-label)">
          SHA-256
        </text>
        <text x={360} y={middle + 11} fontSize={11} textAnchor="middle" fontFamily="var(--font-plex-mono)" fill="var(--foreground)">
          {shortHash(group.sha256, 16)}
        </text>
        {deduped > 0 && (
          <>
            <rect x={510} y={middle - 15} width={130} height={30} rx={4} fill="var(--series-1)" />
            <text x={575} y={middle + 4} fontSize={11} textAnchor="middle" fill="#ffffff">
              One shared copy
            </text>
          </>
        )}
      </svg>
    </div>
  );
}

function GroupRow({ group, defaultOpen }: { group: DuplicateGroup; defaultOpen: boolean }) {
  const [open, setOpen] = useState(defaultOpen);
  return (
    <li className="border-b last:border-0">
      <button type="button" aria-expanded={open} onClick={() => setOpen(!open)} className="flex w-full items-center gap-3 px-4
py-2.5 text-left hover:bg-accent/50">
        {open ? <ChevronDown className="size-4 shrink-0 text-muted-foreground" aria-hidden /> : <ChevronRight className="size-4
shrink-0 text-muted-foreground" aria-hidden />}
        <span className="min-w-0 flex-1 truncate text-[13px]" title={group.members[0]?.name}>
          {group.members[0]?.name}
          {group.copies > 1 && <span className="text-muted-foreground"> and {group.copies - 1} more</span>}
        </span>
        <span className="tabular hidden shrink-0 text-xs text-muted-foreground sm:inline">
          {group.copies} × {formatBytes(group.size_bytes)}
        </span>
        <span className="tabular w-28 shrink-0 text-right text-xs">{group.already_deduplicated ? <Badge
variant="secondary">Deduplicated</Badge> : `${formatBytes(group.reclaimable_bytes)} reclaimable`}</span>
      </button>
      {open && (
        <div className="space-y-3 px-4 pt-1 pb-4">
          <div className="flex flex-wrap items-center gap-x-3 gap-y-1 text-xs text-muted-foreground">
            <span>Matching hash</span> <HashText hash={group.sha256} length={32} />
          </div>
          <RelationshipDiagram group={group} />
          <div className="overflow-x-auto rounded-md border">
            <table className="w-full text-xs">
              <thead>
                <tr className="border-b text-left text-muted-foreground">
                  <th className="px-3 py-1.5 font-medium">Logical file</th>
                  <th className="px-3 py-1.5 font-medium">Folder</th>
                  <th className="px-3 py-1.5 font-medium">Stored as</th>
                  <th className="px-3 py-1.5 font-medium">Origin</th>
```

```
              </tr>
            </thead>
            <tbody>
              {group.members.map((member) => (
                <tr key={member.id} className="border-b last:border-0">
                  <td className="px-3 py-1">{member.name}</td>
                  <td className="px-3 py-1 text-muted-foreground">{member.folder}</td>
                  <td className="px-3 py-1">{MODE_LABEL[member.storage_mode]}</td>
                  <td className="px-3 py-1 text-muted-foreground">{PROVENANCE_LABEL[member.provenance_label] ??
member.provenance_label}</td>
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      </div>
    )}
  </li>
);
}

function ChunkMap({ data }: { data: SharedChunks }) {
  return (
    <ul className="space-y-4">
      {data.manifests.map((manifest) => {
        const shared = manifest.chunk_map.filter(Boolean).length;
        return (
          <li key={manifest.id}>
            <div className="mb-1 flex flex-wrap items-baseline justify-between gap-x-3 text-xs">
              <span className="min-w-0 truncate font-medium" title={manifest.logical_files.join(", ")}>
                {manifest.logical_files[0] ?? "No active file"}
                {manifest.logical_files.length > 1 && <span className="font-normal text-muted-foreground"> and
{manifest.logical_files.length - 1} more</span>}
              </span>
              <span className="tabular text-muted-foreground">
                {manifest.chunk_count} chunks of {chunkSizeLabel(manifest.chunk_size)}
                {manifest.chunk_map.length > 0 && `, ${((shared / manifest.chunk_map.length) * 100).toFixed(0)}% shared`}
              </span>
            </div>
            <div className="flex flex-wrap gap-px" role="img" aria-label={`${shared} of ${manifest.chunk_map.length} chunks shown
are shared with another manifest`}>
              {manifest.chunk_map.map((isShared, index) => (
                <span key={index} className={isShared ? "h-3 w-1.5 rounded-[1px] bg-series-1" : "h-3 w-1.5 rounded-[1px] bg-[var(--
chart-axis)]"} />
              ))}
            </div>
            {manifest.chunk_count > data.chunk_map_limit && <p className="mt-1 text-[11px] text-muted-foreground">First
{data.chunk_map_limit} chunks shown.</p>}
          </li>
        );
      })}
    </ul>
  );
}

export default function DuplicatesPage() {
  const groups = useQuery({ queryKey: ["dup-groups"], queryFn: () => api<DuplicateGroups>("/api/v1/dedup/groups") });
  const chunks = useQuery({ queryKey: ["shared-chunks"], queryFn: () => api<SharedChunks>("/api/v1/dedup/shared-chunks") });
  const reclaimable = groups.data?.groups.reduce((sum, g) => sum + g.reclaimable_bytes, 0) ?? 0;

  return (
    <>
      <PageHeader
        title="Duplicates"
        description="Identity is the SHA-256 of the content, never the file name."
        actions={
          <>
            <Button
              size="sm"
              variant="outline"
              onClick={() => {
                groups.refetch();
                chunks.refetch();
              }}
            >
              <RefreshCw className={groups.isFetching ? "animate-spin" : ""} /> Scan again
            </Button>
            <Button size="sm" render={<Link href="/optimize" />}>
              <Sparkles /> Review optimization
            </Button>
          </>
        }
      />

      <Panel className="mb-4">
        <div className="grid grid-cols-2 gap-4 lg:grid-cols-4">
          <Stat label="Duplicate groups" value={formatCount(groups.data?.group_count)} provenance="derived" />
          <Stat label="Redundant logical copies" value={formatCount(groups.data?.duplicate_files)} provenance="derived" />
          <Stat label="Redundant logical bytes" value={formatBytes(groups.data?.redundant_logical_bytes)} provenance="derived" />
          <Stat label="Still reclaimable" value={formatBytes(reclaimable)} provenance="estimated" note={reclaimable > 0 ? "until
optimized" : undefined} />
        </div>
      </Panel>
```

```
      <Tabs defaultValue="exact">
        <TabsList>
          <TabsTrigger value="exact">Exact duplicates</TabsTrigger>
          <TabsTrigger value="chunks">Shared chunks</TabsTrigger>
        </TabsList>

        <TabsContent value="exact" className="mt-3 space-y-3">
          <Notice>
            <strong className="font-medium text-foreground">Exact duplicates</strong> are whole files with identical bytes. These
are not HDFS replicas: replicas are block copies HDFS keeps for fault tolerance, and deduplication never removes them.
          </Notice>
          <Panel flush>
            {groups.isError ? (
              <div className="p-4">
                <ErrorNotice error={groups.error} onRetry={() => groups.refetch()} />
              </div>
            ) : !groups.data ? (
              <RowsSkeleton />
            ) : groups.data.groups.length === 0 ? (
              <EmptyState icon={CopyCheck} title="No exact duplicates found">
                Every managed file has distinct content. Upload a file twice, or ingest dataset files with workload copies, to see a
group.
              </EmptyState>
            ) : (
              <ul>
                {groups.data.groups.map((group, index) => (
                  <GroupRow key={group.sha256} group={group} defaultOpen={index === 0} />
                ))}
              </ul>
            )}
          </Panel>
        </TabsContent>

        <TabsContent value="chunks" className="mt-3 space-y-3">
          <Notice>
            <strong className="font-medium text-foreground">Shared chunks</strong> are identical fixed-size pieces inside files that
may differ overall. These are application-level chunks, each stored as its own HDFS file; they are not HDFS blocks. Only chunk-mode
files appear here.
          </Notice>
          <Panel
            title="Chunk map"
            subtitle="Each mark is one chunk, in file order. Blue is also used by another distinct file."
            actions={
              chunks.data && (
                <span className="flex items-center gap-2 text-xs text-muted-foreground">
                  {formatBytes(chunks.data.bytes_saved_by_sharing)} saved by sharing <ProvenanceTag kind="derived" />
                </span>
              )
            }
          >
            {chunks.isError ? (
              <ErrorNotice error={chunks.error} onRetry={() => chunks.refetch()} />
            ) : !chunks.data ? (
              <RowsSkeleton rows={3} />
            ) : chunks.data.manifests.length === 0 ? (
              <EmptyState title="No chunk-mode files yet">Upload with chunk-level dedup, or run chunk-level optimization.</
EmptyState>
            ) : (
              <ChunkMap data={chunks.data} />
            )}
          </Panel>
        </TabsContent>
      </Tabs>
    </>
  );
}
```

frontend/app/files/page.tsx 655 lines

```
"use client";

import { useQuery, useQueryClient } from "@tanstack/react-query";
import { keepPreviousData } from "@tanstack/react-query";
import {
  ArchiveRestore,
  ArrowDown,
  ArrowUp,
  CheckCircle2,
  ChevronLeft,
  ChevronRight,
  Download,
  FileText,
  Folder,
  Loader2,
  OctagonAlert,
  Search,
  ShieldCheck,
  Trash2,
  Upload,
  X,
} from "lucide-react";
import { useCallback, useRef, useState } from "react";
import { toast } from "sonner";
```

```tsx
import { Choice, EmptyState, ErrorNotice, HashText, KeyValue, Notice, PageHeader, Panel, ProgressBar, RowsSkeleton, Status } from
"@/components/common";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Dialog, DialogContent, DialogDescription, DialogFooter, DialogHeader, DialogTitle } from "@/components/ui/dialog";
import { Input } from "@/components/ui/input";
import { Sheet, SheetContent, SheetDescription, SheetHeader, SheetTitle } from "@/components/ui/sheet";
import { Switch } from "@/components/ui/switch";
import { API_URL, api, del, post, uploadFile } from "@/lib/api";
import { CHUNK_SIZES, chunkSizeLabel, formatBytes, formatDateTime, formatMs, MODE_LABEL, PROVENANCE_LABEL } from "@/lib/format";
import type { FileDetail, IntegrityCheck, LogicalFile, Page, SettingsView, StorageMode, UploadResult } from "@/lib/types";
import { cn } from "@/lib/utils";

const OUTCOME_TEXT: Record<UploadResult["outcome"], string> = {
  stored_new: "New content stored",
  duplicate_reference: "Duplicate: no new data stored",
  chunks_shared: "Stored; shares chunks with existing files",
  stored_raw: "Stored without deduplication",
};

interface UploadItem {
  key: string;
  name: string;
  size: number;
  sent: number;
  state: "queued" | "sending" | "storing" | "done" | "failed";
  result?: UploadResult;
  error?: string;
}

function Uploader({ folder, onUploaded }: { folder: string; onUploaded: () => void }) {
  const settings = useQuery({ queryKey: ["settings"], queryFn: () => api<SettingsView>("/api/v1/settings") });
  const [items, setItems] = useState<UploadItem[]>([]);
  const [mode, setMode] = useState<StorageMode | null>(null);
  const [chunkSize, setChunkSize] = useState<number | null>(null);
  const [dragging, setDragging] = useState(false);
  const input = useRef<HTMLInputElement>(null);

  const activeMode = mode ?? settings.data?.values.default_ingest_mode ?? "file";
  const activeChunk = chunkSize ?? settings.data?.values.default_chunk_size ?? 1024 * 1024;

  const update = (key: string, patch: Partial<UploadItem>) => setItems((list) => list.map((item) => (item.key === key ?
{ ...item, ...patch } : item)));

  const start = useCallback(
    async (files: File[]) => {
      const added = files.map((file, index) => ({ key: `${Date.now()}-${index}-${file.name}`, name: file.name, size: file.size,
sent: 0, state: "queued" as const }));
      setItems((list) => [...added, ...list].slice(0, 30));
      // One at a time: the server stores uploads serially, and per-file progress stays meaningful.
      for (const [index, file] of files.entries()) {
        const key = added[index].key;
        update(key, { state: "sending" });
        try {
          const result = await uploadFile(file, {
            folder,
            mode: activeMode,
            chunkSize: activeMode === "chunk" ? activeChunk : undefined,
            onProgress: (fraction) => update(key, { sent: fraction, state: fraction >= 1 ? "storing" : "sending" }),
          });
          update(key, { state: "done", sent: 1, result });
        } catch (error) {
          update(key, { state: "failed", error: error instanceof Error ? error.message : "Upload failed" });
        }
        onUploaded();
      }
    },
    [folder, activeMode, activeChunk, onUploaded],
  );

  return (
    <Panel
      title="Upload"
      subtitle={`Into ${folder}`}
      actions={
        <div className="flex flex-wrap items-center gap-2">
          <Choice
            ariaLabel="Storage mode for new uploads"
            value={activeMode}
            onChange={setMode}
            options={[
              { value: "file", label: "Whole-file dedup" },
              { value: "chunk", label: "Chunk-level dedup" },
              { value: "raw", label: "No dedup (baseline)" },
            ]}
          />
          {activeMode === "chunk" && (
            <Choice
              ariaLabel="Chunk size"
              value={String(activeChunk)}
              onChange={(v) => setChunkSize(Number(v))}
              options={[...CHUNK_SIZES, ...(CHUNK_SIZES.some((c) => c.value === activeChunk) ? [] : [{ value: activeChunk, label:
```

```
chunkSizeLabel(activeChunk) }])].map((c) => ({
                value: String(c.value),
                label: c.label,
              }))}
            />
          )}
        </div>
      }
    >
      <div
        onDragOver={(e) => {
          e.preventDefault();
          setDragging(true);
        }}
        onDragLeave={() => setDragging(false)}
        onDrop={(e) => {
          e.preventDefault();
          setDragging(false);
          if (e.dataTransfer.files.length) start(Array.from(e.dataTransfer.files));
        }}
        className={cn("flex flex-col items-center justify-center rounded-md border border-dashed px-4 py-6 text-center transition-
colors", dragging ? "border-series-1 bg-series-1/5" : "border-input")}
      >
        <Upload className="mb-2 size-5 text-muted-foreground" aria-hidden />
        <p className="text-[13px]">Drag files here, or</p>
        <Button size="sm" variant="outline" className="mt-2" onClick={() => input.current?.click()}>
          Choose files
        </Button>
        <input
          ref={input}
          data-testid="file-input"
          type="file"
          multiple
          className="sr-only"
          aria-label="Choose files to upload"
          onChange={(e) => {
            if (e.target.files?.length) start(Array.from(e.target.files));
            e.target.value = "";
          }}
        />
      </div>
      {items.length > 0 && (
        <ul className="mt-3 divide-y" aria-live="polite">
          {items.map((item) => (
            <li key={item.key} className="py-2 first:pt-0 last:pb-0">
              <div className="flex items-center justify-between gap-3 text-[13px]">
                <span className="min-w-0 truncate" title={item.name}>
                  {item.name}
                </span>
                <span className="tabular shrink-0 text-xs text-muted-foreground">{formatBytes(item.size)}</span>
              </div>
              {(item.state === "sending" || item.state === "queued") && <ProgressBar value={item.sent} className="mt-1.5"
label={`Sending ${item.name}`} />}
              {item.state === "storing" && (
                <p className="mt-1 flex items-center gap-1.5 text-xs text-muted-foreground">
                  <Loader2 className="size-3 animate-spin" aria-hidden /> Received. Hashing and writing to HDFS.
                </p>
              )}
              {item.state === "done" && item.result && (
                <p className="mt-1 flex flex-wrap items-center gap-x-2 gap-y-0.5 text-xs text-muted-foreground">
                  <CheckCircle2 className="size-3 text-good" aria-hidden />
                  <span>{OUTCOME_TEXT[item.result.outcome]}</span>
                  <span className="tabular">
                    · {formatBytes(item.result.bytes_written)} written
                    {item.result.chunks_total > 0 && ` · ${item.result.chunks_reused} of ${item.result.chunks_total} chunks reused`}
· {formatMs(item.result.total_ms)}
                  </span>
                </p>
              )}
              {item.state === "failed" && (
                <p className="mt-1 flex items-start gap-1.5 text-xs break-words text-muted-foreground">
                  <OctagonAlert className="mt-px size-3 shrink-0 text-critical" aria-hidden /> {item.error}
                </p>
              )}
            </li>
          ))}
        </ul>
      )}
    </Panel>
  );
}

function VerifyResult({ check }: { check: IntegrityCheck }) {
  return (
    <div className="rounded-md border px-3 py-2 text-xs">
      <Status tone={check.ok ? "good" : "critical"} className="font-medium">
        {check.ok ? "SHA-256 matches" : "Verification failed"}
      </Status>
      <p className="mt-1 text-muted-foreground">
        {formatDateTime(check.checked_at)} · {formatBytes(check.bytes_read)} read · {formatMs(check.duration_ms)}
        {check.chunks_checked > 0 && ` · ${check.chunks_checked} chunks`}
      </p>
      {!check.ok && (
```

```
        <p className="mt-1 break-words text-muted-foreground">
          {check.missing && `${check.missing.length} missing. `}
          {check.corrupt && `${check.corrupt.length} corrupt. `}
          {check.detail}
        </p>
      )}
    </div>
  );
}

function FileDrawer({ fileId, onClose, onChanged }: { fileId: string | null; onClose: () => void; onChanged: () => void }) {
  const client = useQueryClient();
  const detail = useQuery({ queryKey: ["file", fileId], queryFn: () => api<FileDetail>(`/api/v1/files/${fileId}`), enabled: !!
fileId });
  const [busy, setBusy] = useState<string | null>(null);
  const [confirmPurge, setConfirmPurge] = useState(false);
  const file = detail.data;

  const run = async (name: string, action: () => Promise<unknown>, done?: string) => {
    setBusy(name);
    try {
      await action();
      if (done) toast.success(done);
      await client.invalidateQueries({ queryKey: ["file", fileId] });
      onChanged();
    } catch (error) {
      toast.error(error instanceof Error ? error.message : "That failed");
    } finally {
      setBusy(null);
    }
  };

  return (
    <Sheet open={!!fileId} onOpenChange={(open) => !open && onClose()}>
      <SheetContent side="right" className="w-full gap-0 overflow-y-auto sm:max-w-lg">
        <SheetHeader className="border-b">
          <SheetTitle className="truncate pr-6 text-sm" title={file?.name}>
            {file?.name ?? "File details"}
          </SheetTitle>
          <SheetDescription className="text-xs">{file ? `${file.folder} · ${formatBytes(file.size_bytes)}` : "Loading"}</
SheetDescription>
        </SheetHeader>
        {detail.isError && (
          <div className="p-4">
            <ErrorNotice error={detail.error} />
          </div>
        )}
        {file && (
          <div className="space-y-5 p-4">
            <div className="flex flex-wrap gap-2">
              {file.status === "active" ? (
                <>
                  <Button size="sm" render={<a href={`${API_URL}/api/v1/files/${file.id}/download`} />}>
                    <Download /> Download
                  </Button>
                  <Button size="sm" variant="outline" disabled={!!busy} onClick={() => run("verify", () => post(`/api/v1/files/
${file.id}/verify`))}>
                    {busy === "verify" ? <Loader2 className="animate-spin" /> : <ShieldCheck />} Verify integrity
                  </Button>
                  <Button size="sm" variant="outline" disabled={!!busy} onClick={() => run("delete", () => del(`/api/v1/files/
${file.id}`), "File deleted. It can be restored.").then(onClose)}>
                    <Trash2 /> Delete
                  </Button>
                </>
              ) : (
                <>
                  <Button size="sm" disabled={!!busy} onClick={() => run("restore", () => post(`/api/v1/files/${file.id}/restore`),
"File restored.").then(onClose)}>
                    <ArchiveRestore /> Restore
                  </Button>
                  <Button size="sm" variant="destructive" disabled={!!busy} onClick={() => setConfirmPurge(true)}>
                    <Trash2 /> Delete permanently
                  </Button>
                </>
              )}
            </div>

            <KeyValue
              rows={[
                { label: "SHA-256", value: <HashText hash={file.sha256} length={16} /> },
                {
                  label: "Stored as",
                  value: `${MODE_LABEL[file.storage_mode]}${file.chunk_size ? `, ${file.chunk_count} chunks of
${chunkSizeLabel(file.chunk_size)}` : ""}`,
                },
                { label: "Logical references", value: `${file.logical_references}` },
                { label: "Origin", value: PROVENANCE_LABEL[file.provenance_label] ?? file.provenance_label },
                { label: "Uploaded", value: formatDateTime(file.created_at) },
                ...(file.deleted_at ? [{ label: "Deleted", value: formatDateTime(file.deleted_at) }] : []),
              ]}
            />

            <section>
```

```
              <h3 className="mb-1.5 text-xs font-medium">Location in HDFS</h3>
              {file.hdfs_paths.map((path) => (
                <p key={path} className="font-mono text-[11px] break-all text-muted-foreground">
                  {path}
                </p>
              ))}
              {file.block_locations?.map((block) => (
                <p key={block.block_id} className="mt-1 text-xs text-muted-foreground">
                  Block {block.block_id} ({formatBytes(block.length)}): replicas on {block.hosts.join(", ") || "no live DataNode"}
                </p>
              ))}
              {file.storage_mode === "chunk" && <p className="mt-1 text-xs text-muted-foreground">Each chunk below is its own HDFS
file, with its own blocks and replicas.</p>}
            </section>

            {file.dataset_source && (
              <section>
                <h3 className="mb-1.5 text-xs font-medium">Dataset provenance</h3>
                <p className="text-xs text-muted-foreground">
                  {file.dataset_source.title}
                  {file.dataset_source.source_url && (
                    <>
                      {" · "}
                      <a className="underline underline-offset-2" href={file.dataset_source.source_url} target="_blank"
rel="noreferrer">
                        source
                      </a>
                    </>
                  )}
                </p>
              </section>
            )}

            {file.same_content_files.length > 0 && (
              <section>
                <h3 className="mb-1.5 text-xs font-medium">Files with identical content</h3>
                <ul className="space-y-0.5 text-xs text-muted-foreground">
                  {file.same_content_files.slice(0, 12).map((other) => (
                    <li key={other.id} className="truncate">
                      {other.folder === "/" ? "/" : `${other.folder}/`}
                      {other.name}
                      {other.status === "deleted" && " (deleted, restorable)"}
                    </li>
                  ))}
                </ul>
              </section>
            )}

            {file.chunks.length > 0 && (
              <section>
                <h3 className="mb-1.5 text-xs font-medium">Chunks</h3>
                <div className="max-h-44 overflow-auto rounded-md border">
                  <table className="w-full text-[11px]">
                    <thead className="sticky top-0 bg-card text-muted-foreground">
                      <tr className="border-b text-left">
                        <th className="px-2 py-1 font-medium">#</th>
                        <th className="px-2 py-1 font-medium">SHA-256</th>
                        <th className="px-2 py-1 text-right font-medium">Size</th>
                        <th className="px-2 py-1 text-right font-medium">Used by</th>
                      </tr>
                    </thead>
                    <tbody className="tabular">
                      {file.chunks.map((chunk) => (
                        <tr key={chunk.ordinal} className="border-b last:border-0">
                          <td className="px-2 py-0.5">{chunk.ordinal}</td>
                          <td className="px-2 py-0.5 font-mono">{chunk.sha256.slice(0, 16)}</td>
                          <td className="px-2 py-0.5 text-right">{formatBytes(chunk.size_bytes)}</td>
                          <td className="px-2 py-0.5 text-right">
                            {chunk.shared_by_manifests} {chunk.shared_by_manifests === 1 ? "manifest" : "manifests"}
                          </td>
                        </tr>
                      ))}
                    </tbody>
                  </table>
                </div>
              </section>
            )}

            <section>
              <h3 className="mb-1.5 text-xs font-medium">Integrity history</h3>
              {file.integrity_history.length === 0 ? (
                <p className="text-xs text-muted-foreground">Not verified since upload. Verify reads every byte back from HDFS.</p>
              ) : (
                <div className="space-y-2">
                  {file.integrity_history.slice(0, 4).map((check) => (
                    <VerifyResult key={check.id} check={check} />
                  ))}
                </div>
              )}
            </section>
          </div>
        )}
```

```
        <Dialog open={confirmPurge} onOpenChange={setConfirmPurge}>
          <DialogContent>
            <DialogHeader>
              <DialogTitle>Delete {file?.name} permanently?</DialogTitle>
              <DialogDescription>
                This cannot be undone. Shared content stays while other files use it{file && file.logical_references > 1 ? `
(${file.logical_references - 1} other reference${file.logical_references > 2 ? "s" : ""} exist)` : ""}. Content nothing else uses is
removed from HDFS.
              </DialogDescription>
            </DialogHeader>
            <DialogFooter>
              <Button variant="outline" onClick={() => setConfirmPurge(false)}>
                Cancel
              </Button>
              <Button
                variant="destructive"
                onClick={() => {
                  setConfirmPurge(false);
                  if (file) run("purge", () => del(`/api/v1/files/${file.id}?permanent=true`), "Permanently
removed.").then(onClose);
                }}
              >
                Delete permanently
              </Button>
            </DialogFooter>
          </DialogContent>
        </Dialog>
      </SheetContent>
    </Sheet>
  );
}

type SortKey = "name" | "size" | "created" | "mode";

function SortHeader({ label, column, sort, order, onSort, align }: { label: string; column: SortKey; sort: SortKey; order: "asc" |
"desc"; onSort: (c: SortKey) => void; align?: "right" }) {
  const active = sort === column;
  return (
    <th className={cn("px-3 py-2 font-medium", align === "right" && "text-right")} aria-sort={active ? (order === "asc" ?
"ascending" : "descending") : "none"}>
      <button type="button" onClick={() => onSort(column)} className={cn("inline-flex items-center gap-1 hover:text-foreground",
active && "text-foreground")}>
        {label}
        {active && (order === "asc" ? <ArrowUp className="size-3" aria-hidden /> : <ArrowDown className="size-3" aria-hidden />)}
      </button>
    </th>
  );
}

export default function FilesPage() {
  const client = useQueryClient();
  const [folder, setFolder] = useState<string | null>(null);
  const [search, setSearch] = useState("");
  const [status, setStatus] = useState<"active" | "deleted">("active");
  const [mode, setMode] = useState("all");
  const [duplicatesOnly, setDuplicatesOnly] = useState(false);
  const [sort, setSort] = useState<SortKey>("created");
  const [order, setOrder] = useState<"asc" | "desc">("desc");
  const [page, setPage] = useState(1);
  const [selected, setSelected] = useState<string | null>(null);
  const pageSize = 25;

  const params = new URLSearchParams({ status, sort, order, page: String(page), page_size: String(pageSize) });
  if (folder) params.set("folder", folder);
  if (search) params.set("q", search);
  if (mode !== "all") params.set("mode", mode);
  if (duplicatesOnly) params.set("duplicates_only", "true");

  const files = useQuery({
    queryKey: ["files", params.toString()],
    queryFn: () => api<Page<LogicalFile>>(`/api/v1/files?${params}`),
    placeholderData: keepPreviousData,
  });
  const folders = useQuery({ queryKey: ["folders"], queryFn: () => api<{ folders: { path: string; files: number; bytes: number }[] }
>("/api/v1/files/folders") });

  const refresh = useCallback(() => {
    client.invalidateQueries({ queryKey: ["files"] });
    client.invalidateQueries({ queryKey: ["folders"] });
    client.invalidateQueries({ queryKey: ["overview"] });
  }, [client]);

  const onSort = (column: SortKey) => {
    setPage(1);
    if (sort === column) setOrder(order === "asc" ? "desc" : "asc");
    else {
      setSort(column);
      setOrder(column === "name" ? "asc" : "desc");
    }
  };

  const totalPages = files.data ? Math.max(1, Math.ceil(files.data.total / pageSize)) : 1;
  const uploadFolder = folder ?? "/";
```

```
  const filtering = !!search || mode !== "all" || duplicatesOnly || status === "deleted";

  return (
    <>
      <PageHeader title="Files" description="Logical files. Each resolves to stored content in HDFS, so several files can share the
same bytes." />

      <div className="grid gap-4 lg:grid-cols-[220px_minmax(0,1fr)]">
        <div className="space-y-4">
          <Panel title="Folders" flush>
            <ul className="px-2 pb-2">
              <li>
                <button
                  type="button"
                  onClick={() => {
                    setFolder(null);
                    setPage(1);
                  }}
                  className={cn("flex w-full items-center gap-2 rounded px-2 py-1 text-left text-[13px] hover:bg-accent", folder ===
null && "bg-accent font-medium")}
                >
                  <Folder className="size-3.5 shrink-0 text-muted-foreground" aria-hidden /> All folders
                </button>
              </li>
              {folders.data?.folders.map((item) => (
                <li key={item.path}>
                  <button
                    type="button"
                    onClick={() => {
                      setFolder(item.path);
                      setPage(1);
                    }}
                    className={cn("flex w-full items-center justify-between gap-2 rounded px-2 py-1 text-left text-[13px] hover:bg-
accent", folder === item.path && "bg-accent font-medium")}
                    style={{ paddingLeft: 8 + (item.path.split("/").filter(Boolean).length - 1) * 10 }}
                  >
                    <span className="flex min-w-0 items-center gap-2">
                      <Folder className="size-3.5 shrink-0 text-muted-foreground" aria-hidden />
                      <span className="truncate">{item.path}</span>
                    </span>
                    <span className="tabular shrink-0 text-[11px] text-muted-foreground">{item.files}</span>
                  </button>
                </li>
              ))}
            </ul>
          </Panel>
        </div>

        <div className="min-w-0 space-y-4">
          <Uploader folder={uploadFolder} onUploaded={refresh} />

          <Panel flush>
            <div className="flex flex-wrap items-center gap-2 border-b p-3">
              <div className="relative min-w-40 flex-1">
                <Search className="absolute top-2 left-2 size-4 text-muted-foreground" aria-hidden />
                <Input
                  value={search}
                  onChange={(e) => {
                    setSearch(e.target.value);
                    setPage(1);
                  }}
                  placeholder="Search by name or SHA-256 prefix"
                  aria-label="Search files"
                  className="h-8 pl-8 text-[13px]"
                />
                {search && (
                  <button type="button" aria-label="Clear search" className="absolute top-2 right-2 text-muted-foreground
hover:text-foreground" onClick={() => setSearch("")}>
                    <X className="size-4" aria-hidden />
                  </button>
                )}
              </div>
              <Choice
                ariaLabel="Filter by storage mode"
                value={mode}
                onChange={(v) => {
                  setMode(v);
                  setPage(1);
                }}
                options={[
                  { value: "all", label: "All modes" },
                  { value: "raw", label: "No dedup" },
                  { value: "file", label: "Whole file" },
                  { value: "chunk", label: "Chunked" },
                ]}
              />
              <Choice
                ariaLabel="Active or deleted files"
                value={status}
                onChange={(v) => {
                  setStatus(v);
                  setPage(1);
                }}
```

```
                options={[
                  { value: "active", label: "Active" },
                  { value: "deleted", label: "Recently deleted" },
                ]}
              />
              <label className="flex items-center gap-2 text-xs text-muted-foreground">
                <Switch
                  checked={duplicatesOnly}
                  onCheckedChange={(v) => {
                    setDuplicatesOnly(v);
                    setPage(1);
                  }}
                  aria-label="Show only duplicates"
                />
                Duplicates only
              </label>
            </div>

            {files.isError ? (
              <div className="p-4">
                <ErrorNotice error={files.error} onRetry={() => files.refetch()} />
              </div>
            ) : !files.data ? (
              <RowsSkeleton />
            ) : files.data.items.length === 0 ? (
              <EmptyState icon={FileText} title={filtering ? "No files match these filters" : "No files here yet"}>
                {filtering ? "Change or clear the filters." : "Upload a file above, or import the authentic dataset from the Dataset
lab."}
              </EmptyState>
            ) : (
              <div className={cn("overflow-x-auto transition-opacity", files.isPlaceholderData && "opacity-60")}>
                <table className="w-full text-[13px]">
                  <thead>
                    <tr className="border-b text-left text-xs text-muted-foreground">
                      <SortHeader label="Name" column="name" sort={sort} order={order} onSort={onSort} />
                      <SortHeader label="Size" column="size" sort={sort} order={order} onSort={onSort} align="right" />
                      <th className="px-3 py-2 font-medium">SHA-256</th>
                      <SortHeader label="Stored as" column="mode" sort={sort} order={order} onSort={onSort} />
                      <th className="px-3 py-2 font-medium">Duplicates</th>
                      <SortHeader label="Uploaded" column="created" sort={sort} order={order} onSort={onSort} />
                    </tr>
                  </thead>
                  <tbody>
                    {files.data.items.map((file) => (
                      <tr key={file.id} className="cursor-pointer border-b last:border-0 hover:bg-accent/50" onClick={() =>
setSelected(file.id)}>
                        <td className="max-w-72 px-3 py-1.5">
                          <button type="button" className="block max-w-full truncate text-left hover:underline" title={file.name}
onClick={() => setSelected(file.id)}>
                            {file.name}
                          </button>
                          <span className="block truncate text-[11px] text-muted-foreground">
                            {file.folder}
                            {file.provenance_label === "workload_copy" && " · workload copy"}
                            {file.provenance_label === "transformed" && " · transformed test file"}
                          </span>
                        </td>
                        <td className="tabular px-3 py-1.5 text-right whitespace-nowrap">{formatBytes(file.size_bytes)}</td>
                        <td className="px-3 py-1.5">
                          <HashText hash={file.sha256} />
                        </td>
                        <td className="px-3 py-1.5 whitespace-nowrap">{MODE_LABEL[file.storage_mode]}</td>
                        <td className="px-3 py-1.5">
                          {file.is_duplicate ? <Badge variant="secondary">{file.logical_references} references</Badge> : <span
className="text-xs text-muted-foreground">Unique</span>}
                        </td>
                        <td className="tabular px-3 py-1.5 text-xs whitespace-nowrap text-muted-
foreground">{formatDateTime(file.created_at)}</td>
                      </tr>
                    ))}
                  </tbody>
                </table>
              </div>
            )}

            {files.data && files.data.total > 0 && (
              <div className="flex items-center justify-between border-t px-3 py-2 text-xs text-muted-foreground">
                <span className="tabular">
                  {files.data.total.toLocaleString("en-US")} file{files.data.total === 1 ? "" : "s"}
                </span>
                <div className="flex items-center gap-2">
                  <span className="tabular">
                    Page {page} of {totalPages}
                  </span>
                  <Button variant="outline" size="icon-sm" aria-label="Previous page" disabled={page <= 1} onClick={() =>
setPage(page - 1)}>
                    <ChevronLeft />
                  </Button>
                  <Button variant="outline" size="icon-sm" aria-label="Next page" disabled={page >= totalPages} onClick={() =>
setPage(page + 1)}>
                    <ChevronRight />
                  </Button>
                </div>
```

```
              </div>
            )}
          </Panel>

          {status === "deleted" && <Notice>Deleted files keep their content, so they still count towards stored bytes until
permanently removed.</Notice>}
        </div>
      </div>

      <FileDrawer fileId={selected} onClose={() => setSelected(null)} onChanged={refresh} />
    </>
  );
}
```

frontend/app/lab/page.tsx 345 lines

```
"use client";

import { useQuery, useQueryClient } from "@tanstack/react-query";
import { Download, ExternalLink, FlaskConical, Loader2, PackagePlus, Play } from "lucide-react";
import { useMemo, useState } from "react";
import { toast } from "sonner";

import { Choice, EmptyState, ErrorNotice, HashText, Notice, PageHeader, Panel, ProgressBar, RowsSkeleton, Stat, Status } from "@/
components/common";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Dialog, DialogContent, DialogDescription, DialogFooter, DialogHeader, DialogTitle } from "@/components/ui/dialog";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { API_URL, api, post } from "@/lib/api";
import { CHUNK_SIZES, formatBytes, formatCount, formatDateTime } from "@/lib/format";
import { isFinished, useJob } from "@/lib/queries";
import type { DatasetList, ExperimentRun, Experiments, Job } from "@/lib/types";

const KIND_TEXT = { original: "Authentic original", transformed: "Transformed test file", workload_copy: "Workload copy" } as const;

function ResultValue({ value }: { value: unknown }) {
  if (value === null || value === undefined) return <span className="text-muted-foreground">N/A</span>;
  if (typeof value === "boolean") return <>{value ? "Yes" : "No"}</>;
  if (typeof value === "number") return <>{Number.isInteger(value) ? value.toLocaleString("en-US") : value.toFixed(2)}</>;
  if (typeof value === "object") return <span className="font-mono text-[10px] break-all">{JSON.stringify(value)}</span>;
  return <>{String(value)}</>;
}

function RunResult({ run }: { run: ExperimentRun }) {
  const rows = run.result?.rows;
  const columns = useMemo(() => {
    const seen: string[] = [];
    rows?.forEach((row) => Object.keys(row).forEach((key) => !seen.includes(key) && seen.push(key)));
    return seen;
  }, [rows]);

  return (
    <div>
      <div className="mb-2 flex flex-wrap items-center justify-between gap-2 text-xs text-muted-foreground">
        <span>
          {run.result?.description}
          {run.result?.duration_s !== undefined && ` Ran in ${run.result.duration_s.toFixed(1)} s on
${formatDateTime(run.finished_at)}.`}
        </span>
        <Button size="xs" variant="outline" render={<a href={`${API_URL}/api/v1/experiments/${run.id}/export.csv`} />}>
          <Download /> CSV
        </Button>
      </div>
      <div className="max-h-72 overflow-auto rounded-md border">
        <table className="text-[11px]">
          <thead className="sticky top-0 bg-card text-muted-foreground">
            <tr className="border-b text-left">
              {columns.map((column) => (
                <th key={column} className="px-2 py-1 font-medium whitespace-nowrap">
                  {column.replaceAll("_", " ")}
                </th>
              ))}
            </tr>
          </thead>
          <tbody className="tabular">
            {rows?.map((row, index) => (
              <tr key={index} className="border-b last:border-0">
                {columns.map((column) => (
                  <td key={column} className="px-2 py-0.5 whitespace-nowrap">
                    <ResultValue value={row[column]} />
                  </td>
                ))}
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    </div>
  );
}

export default function LabPage() {
```

```
  const client = useQueryClient();
  const datasets = useQuery({ queryKey: ["datasets"], queryFn: () => api<DatasetList>("/api/v1/datasets") });
  const experiments = useQuery({ queryKey: ["experiments"], queryFn: () => api<Experiments>("/api/v1/experiments") });

  const [selected, setSelected] = useState<Set<string>>(new Set());
  const [ingestOpen, setIngestOpen] = useState(false);
  const [copies, setCopies] = useState(2);
  const [mode, setMode] = useState<"raw" | "file" | "chunk">("raw");
  const [chunkSize, setChunkSize] = useState(256 * 1024);
  const [activeJob, setActiveJob] = useState<string | null>(null);
  const [activeScenario, setActiveScenario] = useState<string | null>(null);

  const job = useJob<{ errors?: string[] }>(activeJob, (finished) => {
    client.invalidateQueries({ queryKey: ["datasets"] });
    client.invalidateQueries({ queryKey: ["experiments"] });
    client.invalidateQueries({ queryKey: ["files"] });
    client.invalidateQueries({ queryKey: ["overview"] });
    setActiveScenario(null);
    if (finished.status === "succeeded") toast.success(`${finished.kind.replace("_", " ")} finished.`);
    else toast.error(finished.error ?? `Finished with status ${finished.status}.`);
  });
  const running = !!activeJob && !isFinished(job.data as Job<unknown> | undefined);

  const start = async (path: string, body?: unknown) => {
    try {
      const { job_id } = await post<{ job_id: string }>(path, body);
      setActiveJob(job_id);
    } catch (error) {
      setActiveScenario(null);
      toast.error(error instanceof Error ? error.message : "Could not start");
    }
  };

  const items = datasets.data?.items ?? [];
  const originals = items.filter((i) => i.kind === "original");
  const totalBytes = originals.reduce((sum, i) => sum + i.size_bytes, 0);
  const latest = (scenario: string) => experiments.data?.runs.find((run) => run.scenario === scenario);

  return (
    <>
      <PageHeader
        title="Dataset and experiment lab"
        description="Authentic files from Project Gutenberg, with recorded provenance, and reproducible experiments that run on the
real cluster."
        actions={
          <Button size="sm" variant="outline" disabled={running} onClick={() => start("/api/v1/datasets/import", {})}>
            <Download /> Download from Project Gutenberg
          </Button>
        }
      />

      {running && job.data && (
        <div className="mb-4 rounded-lg border bg-card px-4 py-3">
          <div className="mb-1.5 flex items-center justify-between gap-3 text-xs text-muted-foreground">
            <Status tone="busy">{job.data.message ?? "Queued"}</Status>
            {job.data.progress_total > 0 && (
              <span className="tabular">
                {job.data.progress_done} of {job.data.progress_total}
              </span>
            )}
          </div>
          <ProgressBar value={job.data.progress_total ? job.data.progress_done / job.data.progress_total : 0} label="Job progress" /
>
        </div>
      )}

      <Panel className="mb-4">
        <div className="grid grid-cols-2 gap-4 lg:grid-cols-4">
          <Stat label="Authentic original files" value={formatCount(originals.length)} />
          <Stat label="Original data" value={formatBytes(totalBytes)} />
          <Stat label="Transformed test files" value={formatCount(items.filter((i) => i.kind === "transformed").length)} />
          <Stat label="Source" value="Project Gutenberg" note={<a className="underline underline-offset-2" href="https://www.
gutenberg.org/" target="_blank" rel="noreferrer">gutenberg.org</a>} />
        </div>
      </Panel>

      <Notice>
        Nothing here is synthesized. <strong className="font-medium text-foreground">Originals</strong> are stored exactly as the
publisher served them. <strong className="font-medium text-foreground">Workload copies</strong> are the same bytes under another
logical name. <strong className="font-medium text-foreground">Transformed test files</strong> are byte-level edits of one original,
documented below. None is presented as a separately published work.
      </Notice>

      <Panel
        title="Dataset files and provenance"
        className="mt-4"
        flush
        actions={
          <Button size="sm" disabled={selected.size === 0 || running} onClick={() => setIngestOpen(true)}>
            <PackagePlus /> Ingest {selected.size > 0 ? selected.size : ""} into HDFS
          </Button>
        }
      >
```

```
        {datasets.isError ? (
          <div className="p-4">
            <ErrorNotice error={datasets.error} onRetry={() => datasets.refetch()} />
          </div>
        ) : !datasets.data ? (
          <RowsSkeleton />
        ) : items.length === 0 ? (
          <EmptyState icon={FlaskConical} title="No dataset files yet">
            Use the download button to fetch the catalog from Project Gutenberg. {datasets.data.catalog.length} works are
configured.
          </EmptyState>
        ) : (
          <div className="overflow-x-auto px-4 pb-3">
            <table className="w-full text-[13px]">
              <thead>
                <tr className="border-b text-left text-xs text-muted-foreground">
                  <th className="w-8 py-2 pr-2">
                    <input
                      type="checkbox"
                      aria-label="Select all available files"
                      checked={selected.size > 0 && selected.size === items.filter((i) => i.local_present).length}
                      onChange={(e) => setSelected(e.target.checked ? new Set(items.filter((i) => i.local_present).map((i) =>
i.id)) : new Set())}
                    />
                  </th>
                  <th className="py-2 pr-3 font-medium">Title</th>
                  <th className="py-2 pr-3 font-medium">Kind</th>
                  <th className="py-2 pr-3 font-medium">Format</th>
                  <th className="py-2 pr-3 text-right font-medium">Size</th>
                  <th className="py-2 pr-3 font-medium">SHA-256</th>
                  <th className="py-2 pr-3 font-medium">Downloaded</th>
                  <th className="py-2 font-medium">Source</th>
                </tr>
              </thead>
              <tbody>
                {items.map((item) => (
                  <tr key={item.id} className="border-b align-baseline last:border-0">
                    <td className="py-1.5 pr-2">
                      <input
                        type="checkbox"
                        aria-label={`Select ${item.title}`}
                        disabled={!item.local_present}
                        checked={selected.has(item.id)}
                        onChange={(e) => {
                          const next = new Set(selected);
                          if (e.target.checked) next.add(item.id);
                          else next.delete(item.id);
                          setSelected(next);
                        }}
                      />
                    </td>
                    <td className="max-w-80 py-1.5 pr-3">
                      <span className="block truncate" title={item.title}>
                        {item.title}
                      </span>
                      <span className="block truncate text-[11px] text-muted-foreground" title={item.transformation ??
item.author ?? ""}>
                        {item.transformation ?? item.author}
                      </span>
                    </td>
                    <td className="py-1.5 pr-3 whitespace-nowrap">
                      <Badge variant={item.kind === "original" ? "secondary" : "outline"}>{KIND_TEXT[item.kind]}</Badge>
                    </td>
                    <td className="py-1.5 pr-3 uppercase text-xs text-muted-foreground">{item.file_format}</td>
                    <td className="tabular py-1.5 pr-3 text-right whitespace-nowrap">{formatBytes(item.size_bytes)}</td>
                    <td className="py-1.5 pr-3">
                      <HashText hash={item.sha256} />
                    </td>
                    <td className="tabular py-1.5 pr-3 text-xs whitespace-nowrap text-muted-foreground">{item.downloaded_at ?
formatDateTime(item.downloaded_at) : "Derived locally"}</td>
                    <td className="py-1.5 whitespace-nowrap">
                      {item.source_url && (
                        <a className="inline-flex items-center gap-1 text-xs underline underline-offset-2" href={item.source_url}
target="_blank" rel="noreferrer">
                          #{item.ebook_id} <ExternalLink className="size-3" aria-hidden />
                        </a>
                      )}
                    </td>
                  </tr>
                ))}
              </tbody>
            </table>
          </div>
        )}
      </Panel>

      <Panel title="Experiments" subtitle="Each run uses its own HDFS directory and temporary database, so it never touches managed
files." className="mt-4" flush>
        {!experiments.data ? (
          <RowsSkeleton />
        ) : (
          <ul className="divide-y border-t">
            {experiments.data.scenarios.map((scenario) => {
```

```
              const run = latest(scenario.id);
              const busy = activeScenario === scenario.id && running;
              return (
                <li key={scenario.id} className="px-4 py-3">
                  <div className="flex flex-wrap items-start justify-between gap-3">
                    <div className="min-w-0 flex-1">
                      <p className="text-[13px] font-medium">{scenario.title}</p>
                      <p className="text-xs text-muted-foreground">{scenario.description}</p>
                    </div>
                    <div className="flex items-center gap-3">
                      {run && !busy && (
                        <Status tone={run.status === "succeeded" ? "good" : run.status === "failed" ? "critical" : "busy"}
className="text-xs capitalize">
                          {run.status}
                        </Status>
                      )}
                      <Button
                        size="sm"
                        variant="outline"
                        disabled={running || items.length === 0}
                        onClick={() => {
                          setActiveScenario(scenario.id);
                          start("/api/v1/experiments/run", { scenario: scenario.id });
                        }}
                      >
                        {busy ? <Loader2 className="animate-spin" /> : <Play />} Run
                      </Button>
                    </div>
                  </div>
                  {run?.status === "failed" && <p className="mt-2 text-xs break-words text-muted-foreground">{run.error}</p>}
                  {run?.status === "succeeded" && (
                    <div className="mt-3">
                      <RunResult run={run} />
                    </div>
                  )}
                </li>
              );
            })}
          </ul>
        )}
      </Panel>

      <Dialog open={ingestOpen} onOpenChange={setIngestOpen}>
        <DialogContent>
          <DialogHeader>
            <DialogTitle>Ingest {selected.size} dataset file{selected.size === 1 ? "" : "s"}</DialogTitle>
            <DialogDescription>Files go through the normal upload pipeline into the folder /datasets. The bytes of each file are
checked against its recorded SHA-256 first.</DialogDescription>
          </DialogHeader>
          <div className="grid gap-3">
            <div className="grid gap-1.5">
              <Label htmlFor="copies" className="text-xs">
                Exact workload copies per file
              </Label>
              <Input id="copies" type="number" min={0} max={20} value={copies} onChange={(e) => setCopies(Math.max(0, Math.min(20,
Number(e.target.value))))} className="h-8 w-24" />
              <p className="text-[11px] text-muted-foreground">Copies are named and labelled as workload copies, so they cannot be
mistaken for originals.</p>
            </div>
            <div className="grid gap-1.5">
              <Label className="text-xs">Storage mode</Label>
              <Choice
                ariaLabel="Storage mode for ingestion"
                value={mode}
                onChange={setMode}
                options={[
                  { value: "raw", label: "No dedup (shows what optimization can reclaim)" },
                  { value: "file", label: "Whole-file dedup" },
                  { value: "chunk", label: "Chunk-level dedup" },
                ]}
                className="w-full"
              />
            </div>
            {mode === "chunk" && (
              <div className="grid gap-1.5">
                <Label className="text-xs">Chunk size</Label>
                <Choice ariaLabel="Chunk size for ingestion" value={String(chunkSize)} onChange={(v) => setChunkSize(Number(v))}
options={CHUNK_SIZES.map((c) => ({ value: String(c.value), label: c.label }))} className="w-full" />
              </div>
            )}
          </div>
          <DialogFooter>
            <Button variant="outline" onClick={() => setIngestOpen(false)}>
              Cancel
            </Button>
            <Button
              onClick={() => {
                setIngestOpen(false);
                start("/api/v1/datasets/ingest", { source_ids: Array.from(selected), copies, mode, chunk_size: mode === "chunk" ?
chunkSize : null });
              }}
            >
              Ingest
```

```
            </Button>
          </DialogFooter>
        </DialogContent>
      </Dialog>
    </>
  );
}
```

frontend/app/optimize/page.tsx 417 lines

```tsx
"use client";

import { useQuery, useQueryClient } from "@tanstack/react-query";
import { CheckCircle2, Loader2, Play, RotateCcw, ScanSearch, ShieldCheck } from "lucide-react";
import { useCallback, useState } from "react";
import { toast } from "sonner";

import { BeforeAfter } from "@/components/charts";
import { Choice, EmptyState, ErrorNotice, Notice, PageHeader, Panel, ProgressBar, ProvenanceTag, Stat, Status } from "@/components/
common";
import { Button } from "@/components/ui/button";
import { Dialog, DialogContent, DialogDescription, DialogFooter, DialogHeader, DialogTitle } from "@/components/ui/dialog";
import { api, post } from "@/lib/api";
import { CHUNK_SIZES, chunkSizeLabel, formatBytes, formatCount, formatDateTime, formatMs, formatPercent, MODE_LABEL, shortHash }
from "@/lib/format";
import { isFinished, useJob } from "@/lib/queries";
import type { AnalysisResult, Job, OptimizeResult } from "@/lib/types";
import { cn } from "@/lib/utils";

const ACTION_TEXT: Record<string, string> = {
  promote_to_object: "Becomes the stored object",
  reference_existing_object: "Points at the stored object; old copy removed",
  chunk: "Split into chunks; new chunks stored",
  reference_existing_manifest: "Points at an existing manifest",
};

function Step({ number, title, state, children }: { number: number; title: string; state: "todo" | "active" | "done"; children?:
React.ReactNode }) {
  return (
    <section className="relative grid grid-cols-[28px_minmax(0,1fr)] gap-3">
      <div className="flex flex-col items-center">
        <span
          className={cn(
            "flex size-6 shrink-0 items-center justify-center rounded-full border text-xs font-medium",
            state === "active" && "border-transparent bg-primary text-primary-foreground",
            state === "done" && "border-transparent bg-muted",
            state === "todo" && "text-muted-foreground",
          )}
        >
          {state === "done" ? <CheckCircle2 className="size-4 text-good" aria-label="Done" /> : number}
        </span>
        <span className="mt-1 w-px flex-1 bg-border" aria-hidden />
      </div>
      <div className="min-w-0 pb-6">
        <h2 className={cn("text-[13px] leading-6 font-semibold", state === "todo" && "text-muted-foreground")}>{title}</h2>
        {children && <div className="mt-2">{children}</div>}
      </div>
    </section>
  );
}

function JobStatusLine({ job }: { job: Job<unknown> }) {
  const fraction = job.progress_total ? job.progress_done / job.progress_total : 0;
  return (
    <div>
      <div className="mb-1.5 flex items-center justify-between gap-3 text-xs text-muted-foreground">
        <Status tone="busy">{job.message ?? "Queued"}</Status>
        <span className="tabular">
          {job.progress_done} of {job.progress_total || "?"}
        </span>
      </div>
      <ProgressBar value={fraction} label="Job progress" />
    </div>
  );
}

function AnalysisPreview({ result }: { result: AnalysisResult }) {
  return (
    <div className="space-y-3">
      <div className="grid grid-cols-2 gap-4 rounded-md border p-3 lg:grid-cols-4">
        <Stat label="Files to convert" value={formatCount(result.files_eligible)} note={`${result.distinct_contents} distinct`} />
        <Stat label="Stored content now" value={formatBytes(result.current_stored_bytes)} provenance="derived" />
        <Stat label="Expected afterwards" value={formatBytes(result.predicted_stored_bytes)} provenance="estimated" />
        <Stat label="Expected reduction" value={formatBytes(result.predicted_reduction_bytes)} provenance="estimated"
note={`${formatPercent(result.predicted_savings_percent)} saving`} />
      </div>
      {result.mode === "chunk" && (
        <p className="tabular text-xs text-muted-foreground">
          Read {formatCount(result.chunks_examined)} chunks of {chunkSizeLabel(result.chunk_size)}: {formatCount(result.chunks_new)}
would be new, {formatCount(result.chunks_reused)} would reuse a stored chunk. Each new chunk also becomes an HDFS file, which adds
NameNode metadata.
        </p>
      )}
      {result.actions.length > 0 && (
```

```
        <div className="max-h-56 overflow-auto rounded-md border">
          <table className="w-full text-xs">
            <thead className="sticky top-0 bg-card text-muted-foreground">
              <tr className="border-b text-left">
                <th className="px-3 py-1.5 font-medium">File</th>
                <th className="px-3 py-1.5 font-medium">Now</th>
                <th className="px-3 py-1.5 font-medium">SHA-256</th>
                <th className="px-3 py-1.5 font-medium">Planned action</th>
              </tr>
            </thead>
            <tbody>
              {result.actions.map((action) => (
                <tr key={action.id} className="border-b last:border-0">
                  <td className="max-w-56 truncate px-3 py-1" title={action.name}>
                    {action.name}
                  </td>
                  <td className="px-3 py-1 whitespace-nowrap">{MODE_LABEL[action.storage_mode]}</td>
                  <td className="px-3 py-1 font-mono text-muted-foreground">{shortHash(action.sha256, 10)}</td>
                  <td className="px-3 py-1 text-muted-foreground">{ACTION_TEXT[action.action] ?? action.action}</td>
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      )}
      {result.actions_truncated && <p className="text-xs text-muted-foreground">Showing the first 500 files.</p>}
    </div>
  );
}

function VerifiedResult({ job }: { job: Job<OptimizeResult> }) {
  const client = useQueryClient();
  const [verifyJobId, setVerifyJobId] = useState<string | null>(null);
  const verifyJob = useJob<{ files_checked: number; passed: number; failed: string[] }>(verifyJobId);
  const result = job.result;
  if (!result) return null;
  const { before, after } = result;
  const measuredBefore = before.measured.available ? before.measured : null;
  const measuredAfter = after.measured.available ? after.measured : null;

  return (
    <div className="space-y-4">
      <div className="grid grid-cols-2 gap-4 rounded-md border p-3 lg:grid-cols-4">
        <Stat label="Files converted" value={formatCount(result.files_processed)} note={result.files_failed ?
`${result.files_failed} failed` : "All verified first"} />
        <Stat label="Stored content reclaimed" value={formatBytes(result.bytes_reclaimed)} provenance="derived" />
        <Stat label="Savings now" value={formatPercent(after.derived.savings_percent)} provenance="derived" />
        <Stat label="Duration" value={formatMs(result.duration_ms)} provenance="measured" />
      </div>

      <div className="grid gap-4 lg:grid-cols-2">
        <div>
          <p className="mb-2 flex items-center gap-2 text-xs font-medium">
            Application view <ProvenanceTag kind="derived" />
          </p>
          <BeforeAfter
            rows={[
              { label: "Stored content", before: before.derived.stored_content_bytes, after: after.derived.stored_content_bytes },
            ]}
          />
        </div>
        {measuredBefore && measuredAfter && (
          <div>
            <p className="mb-2 flex items-center gap-2 text-xs font-medium">
              HDFS view, including replicas <ProvenanceTag kind="measured" />
            </p>
            <BeforeAfter rows={[{ label: "Space consumed", before: measuredBefore.space_consumed, after:
measuredAfter.space_consumed }]} />
            <p className="tabular mt-2 text-xs text-muted-foreground">
              Files in namespace: {formatCount(measuredBefore.file_count)} before, {formatCount(measuredAfter.file_count)} after.
HDFS frees disk blocks asynchronously after a delete, so DataNode usage catches up shortly.
            </p>
          </div>
        )}
      </div>

      {result.errors.length > 0 && (
        <div>
          <p className="mb-1 text-xs font-medium">Errors</p>
          <ul className="space-y-1">
            {result.errors.map((error) => (
              <li key={error.file_id} className="rounded-md border border-critical/30 bg-critical/5 px-2.5 py-1.5 text-xs break-
words">
                {error.error}
              </li>
            ))}
          </ul>
          <p className="mt-1 text-xs text-muted-foreground">Files with errors were left unchanged.</p>
        </div>
      )}
      {result.orphans_left?.length > 0 && <Notice tone="warning">{result.orphans_left.length} old copies could not be removed. They
are harmless and can be quarantined from Settings.</Notice>}
```

```
      <div className="flex flex-wrap items-center gap-3">
        <Button
          size="sm"
          variant="outline"
          disabled={!!verifyJobId && !isFinished(verifyJob.data as Job<unknown> | undefined)}
          onClick={async () => {
            const { job_id } = await post<{ job_id: string }>("/api/v1/files/verify-all");
            setVerifyJobId(job_id);
            client.invalidateQueries({ queryKey: ["overview"] });
          }}
        >
          <ShieldCheck /> Verify all restored bytes
        </Button>
        {verifyJob.data && !isFinished(verifyJob.data as Job<unknown>) && <span className="tabular text-xs text-muted-
foreground">Checked {verifyJob.data.progress_done} of {verifyJob.data.progress_total}</span>}
        {verifyJob.data?.result && (
          <Status tone={verifyJob.data.result.failed.length ? "critical" : "good"}>
            {verifyJob.data.result.passed} of {verifyJob.data.result.files_checked} files match their SHA-256
          </Status>
        )}
      </div>
    </div>
  );
}

export default function OptimizePage() {
  const client = useQueryClient();
  const [mode, setMode] = useState<"file" | "chunk">("file");
  const [chunkSize, setChunkSize] = useState(256 * 1024);
  const [analysisId, setAnalysisId] = useState<string | null>(null);
  const [optimizeId, setOptimizeId] = useState<string | null>(null);
  const [confirming, setConfirming] = useState(false);
  const [error, setError] = useState<unknown>(null);

  const history = useQuery({
    queryKey: ["jobs", "optimize"],
    queryFn: () => api<Job<OptimizeResult>[]>("/api/v1/dedup/jobs?kind=optimize&limit=8"),
  });
  const resumable = history.data?.find((job) => ["interrupted", "failed", "partial"].includes(job.status));

  const refreshAll = useCallback(() => {
    client.invalidateQueries({ queryKey: ["jobs"] });
    client.invalidateQueries({ queryKey: ["overview"] });
    client.invalidateQueries({ queryKey: ["files"] });
    client.invalidateQueries({ queryKey: ["dup-groups"] });
  }, [client]);

  const analysis = useJob<AnalysisResult>(analysisId);
  const optimization = useJob<OptimizeResult>(optimizeId, refreshAll);

  const analysisDone = analysis.data?.status === "succeeded";
  const analysisFailed = analysis.data?.status === "failed";
  const analyzing = !!analysisId && !isFinished(analysis.data as Job<unknown> | undefined);
  const optimizing = !!optimizeId && !isFinished(optimization.data as Job<unknown> | undefined);
  const optimized = !!optimizeId && isFinished(optimization.data as Job<unknown> | undefined);

  const changeMethod = (next: { mode?: "file" | "chunk"; chunkSize?: number }) => {
    if (next.mode) setMode(next.mode);
    if (next.chunkSize) setChunkSize(next.chunkSize);
    // An earlier preview describes a different method, so it no longer applies.
    setAnalysisId(null);
    setOptimizeId(null);
  };

  const startAnalysis = async () => {
    setError(null);
    setOptimizeId(null);
    try {
      const { job_id } = await post<{ job_id: string }>("/api/v1/dedup/analyze", { mode, chunk_size: mode === "chunk" ? chunkSize :
null });
      setAnalysisId(job_id);
    } catch (e) {
      setError(e);
    }
  };

  const startOptimization = async () => {
    setConfirming(false);
    setError(null);
    try {
      const { job_id } = await post<{ job_id: string }>("/api/v1/dedup/optimize", { analysis_job_id: analysisId });
      setOptimizeId(job_id);
    } catch (e) {
      setError(e);
    }
  };

  const nothingToDo = analysisDone && analysis.data?.result?.files_eligible === 0;

  return (
    <>
      <PageHeader title="Optimization center" description="Preview first, then convert one file at a time. A file's old copy is
removed only after its new form has been read back and its SHA-256 checked." />
```

```
      {resumable && !optimizeId && (
        <div className="mb-4 flex flex-wrap items-center justify-between gap-3 rounded-lg border bg-card px-4 py-2.5">
          <Status tone="warning">
            A {MODE_LABEL[resumable.mode ?? "file"].toLowerCase()} job from {formatDateTime(resumable.created_at)} ended as
{resumable.status}. Finished files are safe.
          </Status>
          <Button
            size="sm"
            variant="outline"
            onClick={async () => {
              try {
                const { job_id } = await post<{ job_id: string }>(`/api/v1/dedup/jobs/${resumable.id}/resume`);
                setOptimizeId(job_id);
              } catch (e) {
                toast.error(e instanceof Error ? e.message : "Could not resume");
              }
            }}
          >
            <RotateCcw /> Resume
          </Button>
        </div>
      )}

      {!!error && (
        <div className="mb-4">
          <ErrorNotice error={error} />
        </div>
      )}

      <Panel>
        <Step number={1} title="Choose a method" state={analysisId ? "done" : "active"}>
          <div className="grid gap-2 sm:grid-cols-2">
            {(
              [
                { value: "file", title: "File-level", text: "Keeps one stored object per distinct file. Fast, and adds no namespace
overhead. Finds only whole-file duplicates." },
                { value: "chunk", title: "Chunk-level", text: "Splits content into fixed-size chunks and shares identical ones. Also
finds overlap between different files, but creates many small HDFS files." },
              ] as const
            ).map((option) => (
              <button
                key={option.value}
                type="button"
                aria-pressed={mode === option.value}
                onClick={() => changeMethod({ mode: option.value })}
                className={cn("rounded-md border px-3 py-2.5 text-left transition-colors hover:bg-accent/50", mode === option.value
&& "border-primary ring-1 ring-primary")}
              >
                <span className="text-[13px] font-medium">{option.title}</span>
                <span className="mt-0.5 block text-xs text-muted-foreground">{option.text}</span>
              </button>
            ))}
          </div>
          {mode === "chunk" && (
            <div className="mt-3 flex items-center gap-2 text-xs text-muted-foreground">
              Chunk size
              <Choice ariaLabel="Chunk size" value={String(chunkSize)} onChange={(v) => changeMethod({ chunkSize: Number(v) })}
options={CHUNK_SIZES.map((c) => ({ value: String(c.value), label: c.label }))} />
            </div>
          )}
          <Button size="sm" className="mt-3" onClick={startAnalysis} disabled={analyzing || optimizing}>
            {analyzing ? <Loader2 className="animate-spin" /> : <ScanSearch />} {analysisId ? "Analyze again" : "Run dry-run
analysis"}
          </Button>
        </Step>

        <Step number={2} title="Review the preview" state={analysisDone ? (optimizeId ? "done" : "active") : "todo"}>
          {analyzing && analysis.data && <JobStatusLine job={analysis.data as Job<unknown>} />}
          {analysisFailed && <ErrorNotice error={new Error(analysis.data?.error ?? "The analysis failed.")} />}
          {analysisDone && analysis.data?.result && (
            <>
              <p className="mb-2 flex items-center gap-2 text-xs text-muted-foreground">
                <ProvenanceTag kind="estimated" /> Nothing has changed yet. These figures are predictions.
              </p>
              {nothingToDo ? (
                <EmptyState icon={CheckCircle2} title="Nothing to convert">
                  No active file would change under this method.
                </EmptyState>
              ) : (
                <>
                  <AnalysisPreview result={analysis.data.result} />
                  {!optimizeId && (
                    <Button size="sm" className="mt-3" onClick={() => setConfirming(true)}>
                      <Play /> Start optimization
                    </Button>
                  )}
                </>
              )}
            </>
          )}
        </Step>
```

```
        <Step number={3} title="Convert and verify" state={optimized ? "done" : optimizing ? "active" : "todo"}>
          {optimizing && optimization.data && <JobStatusLine job={optimization.data as Job<unknown>} />}
          {optimized && optimization.data?.status === "failed" && <ErrorNotice error={new Error(optimization.data.error ?? "The job
failed.")} />}
          {optimized && optimization.data?.result && (
            <>
              <p className="mb-2 flex items-center gap-2 text-xs text-muted-foreground">
                <ProvenanceTag kind="measured" /> Results below are observed, not predicted.
              </p>
              <VerifiedResult job={optimization.data} />
            </>
          )}
        </Step>
      </Panel>

      <Panel title="Job history" className="mt-4" flush>
        {history.data?.length ? (
          <div className="overflow-x-auto px-4 pb-3">
            <table className="w-full text-[13px]">
              <thead>
                <tr className="border-b text-left text-xs text-muted-foreground">
                  <th className="py-2 pr-3 font-medium">Started</th>
                  <th className="py-2 pr-3 font-medium">Method</th>
                  <th className="py-2 pr-3 font-medium">Outcome</th>
                  <th className="py-2 pr-3 text-right font-medium">Converted</th>
                  <th className="py-2 pr-3 text-right font-medium">Reclaimed</th>
                  <th className="py-2 text-right font-medium">Duration</th>
                </tr>
              </thead>
              <tbody className="tabular">
                {history.data.map((job) => (
                  <tr key={job.id} className="border-b last:border-0">
                    <td className="py-1.5 pr-3 whitespace-nowrap">{formatDateTime(job.created_at)}</td>
                    <td className="py-1.5 pr-3 whitespace-nowrap">
                      {MODE_LABEL[job.mode ?? "file"]}
                      {job.chunk_size ? ` (${chunkSizeLabel(job.chunk_size)})` : ""}
                    </td>
                    <td className="py-1.5 pr-3">
                      <Status tone={job.status === "succeeded" ? "good" : job.status === "running" || job.status === "queued" ?
"busy" : job.status === "partial" || job.status === "interrupted" ? "warning" : "critical"} className="text-xs capitalize">
                        {job.status}
                      </Status>
                    </td>
                    <td className="py-1.5 pr-3 text-right">{job.result?.files_processed ?? "N/A"}</td>
                    <td className="py-1.5 pr-3 text-right">{job.result ? formatBytes(job.result.bytes_reclaimed) : "N/A"}</td>
                    <td className="py-1.5 text-right">{job.result ? formatMs(job.result.duration_ms) : "N/A"}</td>
                  </tr>
                ))}
              </tbody>
            </table>
          </div>
        ) : (
          <EmptyState title="No optimization jobs have run" />
        )}
      </Panel>

      <Dialog open={confirming} onOpenChange={setConfirming}>
        <DialogContent>
          <DialogHeader>
            <DialogTitle>Start {mode}-level optimization?</DialogTitle>
            <DialogDescription>
              {analysis.data?.result?.files_eligible} files will be converted. Each is verified by SHA-256 before its old copy is
removed, and any file that fails verification stays as it is. HDFS replication is not changed.
            </DialogDescription>
          </DialogHeader>
          <DialogFooter>
            <Button variant="outline" onClick={() => setConfirming(false)}>
              Cancel
            </Button>
            <Button onClick={startOptimization}>Start optimization</Button>
          </DialogFooter>
        </DialogContent>
      </Dialog>
    </>
  );
}
```

frontend/app/replication/page.tsx 215 lines

```
"use client";

import { useQuery, useQueryClient } from "@tanstack/react-query";
import { Layers3, Loader2, Minus, Plus } from "lucide-react";
import { useState } from "react";

import { EmptyState, ErrorNotice, KeyValue, Notice, PageHeader, Panel, ProgressBar, ProvenanceTag, RowsSkeleton, Stat, Status } from
"@/components/common";
import { Button } from "@/components/ui/button";
import { Dialog, DialogContent, DialogDescription, DialogFooter, DialogHeader, DialogTitle } from "@/components/ui/dialog";
import { api, post } from "@/lib/api";
import { formatBytes, formatCount, formatDateTime } from "@/lib/format";
import { isFinished, useJob } from "@/lib/queries";
import type { Job, ReplicationStatus } from "@/lib/types";
```

```tsx
const OP_TONE = { verified: "good", applied: "busy", pending: "busy", timed_out: "warning", failed: "critical" } as const;
const OP_TEXT = { verified: "Verified", applied: "Applied, converging", pending: "Pending", timed_out: "Did not converge in time",
failed: "Failed" } as const;

export default function ReplicationPage() {
  const client = useQueryClient();
  const status = useQuery({ queryKey: ["replication"], queryFn: () => api<ReplicationStatus>("/api/v1/cluster/replication"),
refetchInterval: 10_000 });
  const [requested, setRequested] = useState<number | null>(null);
  const [confirming, setConfirming] = useState(false);
  const [jobId, setJobId] = useState<string | null>(null);
  const [error, setError] = useState<unknown>(null);
  const job = useJob(jobId, () => client.invalidateQueries({ queryKey: ["replication"] }));
  const running = !!jobId && !isFinished(job.data as Job<unknown> | undefined);
  const data = status.data;

  const submit = async (acknowledge: boolean) => {
    if (!requested) return;
    setConfirming(false);
    setError(null);
    try {
      const response = await post<{ job_id: string }>("/api/v1/cluster/replication", { factor: requested,
acknowledge_reduced_protection: acknowledge });
      setJobId(response.job_id);
    } catch (e) {
      setError(e);
    }
  };

  if (status.isError) {
    return (
      <>
        <PageHeader title="Replication manager" />
        <ErrorNotice error={status.error} onRetry={() => status.refetch()} />
      </>
    );
  }
  if (!data) {
    return (
      <>
        <PageHeader title="Replication manager" />
        <RowsSkeleton rows={8} />
      </>
    );
  }
  if (!data.reachable) {
    return (
      <>
        <PageHeader title="Replication manager" />
        <ErrorNotice error={new Error(data.error)} onRetry={() => status.refetch()} />
      </>
    );
  }

  const factor = requested ?? data.effective_replication;
  const lowering = factor < data.effective_replication;
  const changed = factor !== data.effective_replication;

  return (
    <>
      <PageHeader title="Replication manager" description="HDFS keeps block replicas so that data survives a DataNode failure.
Replicas are not duplicates: deduplication works above them and never removes one." />

      {data.warnings.length > 0 && (
        <div className="mb-4 space-y-2">
          {data.warnings.map((warning) => (
            <Notice key={warning} tone="warning">
              {warning}
            </Notice>
          ))}
        </div>
      )}

      <Panel className="mb-4">
        <div className="grid grid-cols-2 gap-4 lg:grid-cols-4">
          <Stat label="Effective replication" value={`Factor ${data.effective_replication}`} note={data.target_replication ?
"Application target" : "Cluster default"} />
          <Stat label="Live DataNodes" value={formatCount(data.live_datanodes)} provenance="measured" note={data.dead_datanodes ?
`${data.dead_datanodes} dead` : undefined} />
          <Stat label="Average block replicas" value={data.fsck.average_block_replication?.toFixed(2) ?? "N/A"}
provenance="measured" note="fsck" />
          <Stat label="Under-replicated blocks" value={formatCount(data.fsck.under_replicated_blocks)} provenance="measured"
note="fsck" />
        </div>
      </Panel>

      <div className="grid gap-4 lg:grid-cols-2">
        <Panel title="Change replication factor" subtitle="Applies to all managed data and to new writes">
          <div className="flex items-center gap-3">
            <Button variant="outline" size="icon" aria-label="Decrease replication factor" disabled={factor <= 1 || running}
onClick={() => setRequested(factor - 1)}>
              <Minus />
            </Button>
```

```
            <output className="w-10 text-center text-2xl font-semibold" aria-live="polite">
              {factor}
            </output>
            <Button variant="outline" size="icon" aria-label="Increase replication factor" disabled={factor >=
data.max_supported_replication || running} onClick={() => setRequested(factor + 1)}>
              <Plus />
            </Button>
            <span className="text-xs text-muted-foreground">Maximum {data.max_supported_replication}, limited by live DataNodes.</
span>
          </div>

          <div className="mt-4 rounded-md border p-3">
            <p className="mb-2 flex items-center gap-2 text-xs text-muted-foreground">
              Space at factor {factor} <ProvenanceTag kind={changed ? "estimated" : "measured"} />
            </p>
            <p className="text-lg font-semibold tracking-tight">{formatBytes(changed ? data.stored_length_bytes * factor :
data.space_consumed_bytes)}</p>
            <p className="mt-1 text-xs text-muted-foreground">
              {formatBytes(data.stored_length_bytes)} of data in the managed namespace. Each block survives the loss of {factor - 1}
DataNode{factor === 2 ? "" : "s"}.
            </p>
          </div>

          {!!error && (
            <div className="mt-3">
              <ErrorNotice error={error} />
            </div>
          )}
          {running && job.data && (
            <div className="mt-3">
              <p className="mb-1.5 flex items-center gap-1.5 text-xs text-muted-foreground">
                <Loader2 className="size-3 animate-spin" aria-hidden /> {job.data.message ?? "Queued"}
              </p>
              <ProgressBar value={job.data.progress_total ? job.data.progress_done / job.data.progress_total : 0} label="Replication
progress" />
            </div>
          )}

          <Button className="mt-4" size="sm" disabled={!changed || running} onClick={() => (lowering ? setConfirming(true) :
submit(false))}>
            Apply factor {factor}
          </Button>
        </Panel>

        <Panel title="Measured state of the managed namespace">
          <KeyValue
            rows={[
              { label: "Cluster default (dfs.replication)", value: data.cluster_default_replication },
              { label: "Data length", value: formatBytes(data.stored_length_bytes) },
              { label: "Space consumed, with replicas", value: formatBytes(data.space_consumed_bytes) },
              { label: "Measured multiplier", value: data.measured_replication_multiplier ?
`${data.measured_replication_multiplier.toFixed(2)} ×` : "N/A" },
              { label: "fsck status", value: data.fsck.status ?? "N/A" },
              { label: "Files / blocks checked", value: `${formatCount(data.fsck.total_files)} /
${formatCount(data.fsck.total_blocks)}` },
              { label: "Over-replicated blocks", value: formatCount(data.fsck.over_replicated_blocks) },
              { label: "Missing blocks", value: formatCount(data.fsck.missing_blocks) },
              { label: "Corrupt blocks", value: formatCount(data.fsck.corrupt_blocks) },
            ]}
          />
          <p className="mt-3 text-[11px] text-muted-foreground">Checked {formatDateTime(data.checked_at)}</p>
        </Panel>
      </div>

      <Panel title="Replication changes" className="mt-4" flush>
        {data.operations.length === 0 ? (
          <EmptyState icon={Layers3} title="No replication changes have been made" />
        ) : (
          <div className="overflow-x-auto px-4 pb-3">
            <table className="w-full text-[13px]">
              <thead>
                <tr className="border-b text-left text-xs text-muted-foreground">
                  <th className="py-2 pr-3 font-medium">Requested</th>
                  <th className="py-2 pr-3 font-medium">Change</th>
                  <th className="py-2 pr-3 font-medium">Result</th>
                  <th className="py-2 pr-3 text-right font-medium">Files changed</th>
                  <th className="py-2 pr-3 text-right font-medium">Space before</th>
                  <th className="py-2 text-right font-medium">Space after</th>
                </tr>
              </thead>
              <tbody className="tabular">
                {data.operations.map((op) => (
                  <tr key={op.id} className="border-b last:border-0">
                    <td className="py-1.5 pr-3 whitespace-nowrap">{formatDateTime(op.requested_at)}</td>
                    <td className="py-1.5 pr-3 whitespace-nowrap">
                      {op.previous_factor} to {op.requested_factor}
                    </td>
                    <td className="py-1.5 pr-3">
                      <Status tone={OP_TONE[op.status]} className="text-xs">
                        {OP_TEXT[op.status]}
                      </Status>
                    </td>
                    <td className="py-1.5 pr-3 text-right">{formatCount(op.files_changed)}</td>
```

```
                    <td className="py-1.5 pr-3 text-right">{formatBytes(op.space_consumed_before)}</td>
                    <td className="py-1.5 text-right">{formatBytes(op.space_consumed_after)}</td>
                  </tr>
                ))}
              </tbody>
            </table>
          </div>
        )}
      </Panel>

      <Dialog open={confirming} onOpenChange={setConfirming}>
        <DialogContent>
          <DialogHeader>
            <DialogTitle>Lower replication to {factor}?</DialogTitle>
            <DialogDescription>
              This reduces fault tolerance: {factor === 1 ? "losing one DataNode could lose data." : `each block will survive the
loss of ${factor - 1} DataNode${factor === 2 ? "" : "s"} instead of ${data.effective_replication - 1}.`} It saves space, but it is
not deduplication.
            </DialogDescription>
          </DialogHeader>
          <DialogFooter>
            <Button variant="outline" onClick={() => setConfirming(false)}>
              Cancel
            </Button>
            <Button variant="destructive" onClick={() => submit(true)}>
              Lower to {factor}
            </Button>
          </DialogFooter>
        </DialogContent>
      </Dialog>
    </>
  );
}
```

frontend/app/settings/page.tsx 254 lines

```
"use client";

import { useQuery, useQueryClient } from "@tanstack/react-query";
import { Activity, ArchiveX, Loader2, Recycle } from "lucide-react";
import { useState } from "react";
import { toast } from "sonner";

import { Choice, ErrorNotice, KeyValue, Notice, PageHeader, Panel, RowsSkeleton, Status } from "@/components/common";
import { Button } from "@/components/ui/button";
import { Dialog, DialogContent, DialogDescription, DialogFooter, DialogHeader, DialogTitle } from "@/components/ui/dialog";
import { Label } from "@/components/ui/label";
import { Switch } from "@/components/ui/switch";
import { api, post, put } from "@/lib/api";
import { CHUNK_SIZES, chunkSizeLabel, formatBytes, formatMs } from "@/lib/format";
import type { Diagnostics, GarbageReport, ReconcileReport, SettingsView, StorageMode } from "@/lib/types";

export default function SettingsPage() {
  const client = useQueryClient();
  const settings = useQuery({ queryKey: ["settings"], queryFn: () => api<SettingsView>("/api/v1/settings") });
  const [diagnostics, setDiagnostics] = useState<Diagnostics | null>(null);
  const [diagnosing, setDiagnosing] = useState(false);
  const [garbage, setGarbage] = useState<GarbageReport | null>(null);
  const [reconcile, setReconcile] = useState<ReconcileReport | null>(null);
  const [confirm, setConfirm] = useState<null | "gc" | "reconcile">(null);

  const save = async (changes: Record<string, unknown>) => {
    try {
      await put("/api/v1/settings", changes);
      await client.invalidateQueries({ queryKey: ["settings"] });
      toast.success("Saved. This applies to future writes only; stored data is not rewritten.");
    } catch (error) {
      toast.error(error instanceof Error ? error.message : "Could not save");
    }
  };

  const runDiagnostics = async () => {
    setDiagnosing(true);
    try {
      setDiagnostics(await api<Diagnostics>("/api/v1/system/diagnostics"));
    } catch (error) {
      toast.error(error instanceof Error ? error.message : "Diagnostics failed");
    } finally {
      setDiagnosing(false);
    }
  };

  const guarded = async <T,>(action: () => Promise<T>, onDone: (value: T) => void) => {
    try {
      onDone(await action());
    } catch (error) {
      toast.error(error instanceof Error ? error.message : "That failed");
    }
  };

  if (settings.isError) {
    return (
      <>
        <PageHeader title="Settings" />
```

```
        <ErrorNotice error={settings.error} onRetry={() => settings.refetch()} />
      </>
    );
  }
  if (!settings.data) {
    return (
      <>
        <PageHeader title="Settings" />
        <RowsSkeleton rows={8} />
      </>
    );
  }

  const { values, connection, limits, storage } = settings.data;
  const chunkOptions = CHUNK_SIZES.some((c) => c.value === values.default_chunk_size) ? CHUNK_SIZES : [...CHUNK_SIZES, { value:
values.default_chunk_size, label: chunkSizeLabel(values.default_chunk_size) }];

  return (
    <>
      <PageHeader title="Settings" description="Defaults for new data, connection details and maintenance tools." />

      <div className="grid gap-4 lg:grid-cols-2">
        <Panel title="Deduplication defaults" subtitle="Changes apply to future writes. Stored data is never rewritten by a
setting.">
          <div className="space-y-4">
            <div className="flex items-center justify-between gap-4">
              <Label className="text-[13px] font-normal">Default upload mode</Label>
              <Choice
                ariaLabel="Default upload mode"
                value={values.default_ingest_mode}
                onChange={(v: StorageMode) => save({ default_ingest_mode: v })}
                options={[
                  { value: "file", label: "Whole-file dedup" },
                  { value: "chunk", label: "Chunk-level dedup" },
                  { value: "raw", label: "No dedup" },
                ]}
              />
            </div>
            <div className="flex items-center justify-between gap-4">
              <div>
                <Label className="text-[13px] font-normal">Default chunk size</Label>
                <p className="mt-0.5 max-w-xs text-xs text-muted-foreground">Smaller finds more overlap but creates more HDFS files,
which costs NameNode memory.</p>
              </div>
              <Choice ariaLabel="Default chunk size" value={String(values.default_chunk_size)} onChange={(v) =>
save({ default_chunk_size: Number(v) })} options={chunkOptions.map((c) => ({ value: String(c.value), label: c.label }))} />
            </div>
            <div className="flex items-center justify-between gap-4">
              <div>
                <Label className="text-[13px] font-normal" htmlFor="verify-on-write">
                  Verify objects on write
                </Label>
                <p className="mt-0.5 max-w-xs text-xs text-muted-foreground">Reads each whole-file object back and checks its
SHA-256 before committing it. Slower, but safer.</p>
              </div>
              <Switch id="verify-on-write" checked={values.verify_on_write} onCheckedChange={(v) => save({ verify_on_write: v })} />
            </div>
          </div>
          <div className="mt-4 border-t pt-3">
            <KeyValue
              rows={[
                { label: "Replication target", value: values.target_replication ? `Factor ${values.target_replication}` : "Cluster
default. Change it in Replication." },
                { label: "Allowed chunk sizes", value: `${chunkSizeLabel(limits.min_chunk_size)} to
${chunkSizeLabel(limits.max_chunk_size)}` },
                { label: "Upload limit", value: formatBytes(limits.max_upload_bytes) },
              ]}
            />
          </div>
        </Panel>

        <Panel title="HDFS connection" subtitle={connection.source}>
          <KeyValue
            rows={[
              { label: "NameNode", value: <span className="font-mono text-xs">{connection.namenode_url}</span> },
              { label: "HDFS user", value: <span className="font-mono text-xs">{connection.hdfs_user}</span> },
              { label: "Managed root", value: <span className="font-mono text-xs">{connection.root_path}</span> },
              {
                label: "DataNode address map",
                value: Object.keys(connection.datanode_host_map).length ? (
                  <span className="font-mono text-xs">
                    {Object.entries(connection.datanode_host_map).map(([name, target]) => (
                      <span key={name} className="block">
                        {name} → {target}
                      </span>
                    ))}
                  </span>
                ) : (
                  "None (inside the Docker network)"
                ),
              },
              { label: "Metadata database", value: <span className="font-mono text-xs break-all">{storage.database_path}</span> },
              { label: "Database size", value: formatBytes(storage.database_bytes) },
```

```
            ]}
          />
        </Panel>

        <Panel
          title="Health checks"
          subtitle="Runs real checks, including a write, read and delete through a DataNode."
          actions={
            <Button size="sm" variant="outline" onClick={runDiagnostics} disabled={diagnosing}>
              {diagnosing ? <Loader2 className="animate-spin" /> : <Activity />} Run diagnostics
            </Button>
          }
        >
          {diagnostics ? (
            <ul className="divide-y">
              {diagnostics.checks.map((check) => (
                <li key={check.name} className="py-2 first:pt-0 last:pb-0">
                  <div className="flex items-center justify-between gap-3">
                    <Status tone={check.ok ? "good" : "critical"}>{check.name}</Status>
                    <span className="tabular text-xs text-muted-foreground">{formatMs(check.ms)}</span>
                  </div>
                  <p className="mt-0.5 pl-5 text-xs break-words text-muted-foreground">{check.detail}</p>
                </li>
              ))}
            </ul>
          ) : (
            <p className="text-xs text-muted-foreground">Not run yet.</p>
          )}
        </Panel>

        <Panel title="Storage maintenance" subtitle="Each action shows what it would do before anything changes.">
          <div className="space-y-4">
            <div>
              <div className="flex flex-wrap items-center justify-between gap-2">
                <p className="text-[13px]">Unreferenced content</p>
                <div className="flex gap-2">
                  <Button size="sm" variant="outline" onClick={() => guarded(() => api<GarbageReport>("/api/v1/dedup/gc"),
setGarbage)}>
                    Preview
                  </Button>
                  <Button size="sm" variant="outline" disabled={!garbage || garbage.reclaimable_bytes + garbage.manifest_count ===
0} onClick={() => setConfirm("gc")}>
                    <Recycle /> Collect
                  </Button>
                </div>
              </div>
              {garbage && (
                <p className="tabular mt-1 text-xs text-muted-foreground">
                  {garbage.removed ? "Removed" : "Would remove"} {garbage.object_count} objects, {garbage.manifest_count} manifests
and {garbage.chunk_count} chunks ({formatBytes(garbage.reclaimable_bytes)}, before replication). Content a restorable file uses is
never included.
                </p>
              )}
            </div>
            <div className="border-t pt-4">
              <div className="flex flex-wrap items-center justify-between gap-2">
                <p className="text-[13px]">Compare HDFS with metadata</p>
                <div className="flex gap-2">
                  <Button size="sm" variant="outline" onClick={() => guarded(() => api<ReconcileReport>("/api/v1/storage/
reconcile"), setReconcile)}>
                    Check
                  </Button>
                  <Button size="sm" variant="outline" disabled={!reconcile || reconcile.orphan_count === 0} onClick={() =>
setConfirm("reconcile")}>
                    <ArchiveX /> Quarantine orphans
                  </Button>
                </div>
              </div>
              {reconcile && (
                <p className="tabular mt-1 text-xs text-muted-foreground">
                  {reconcile.paths_present} files in HDFS, {reconcile.paths_expected} expected. {reconcile.orphan_count} orphaned,
{reconcile.missing_count} missing.
                  {reconcile.orphans_quarantined > 0 && ` Moved ${reconcile.orphans_quarantined} to the recovery area.`}
                </p>
              )}
              {reconcile && reconcile.missing_count > 0 && (
                <div className="mt-2">
                  <Notice tone="warning">Some stored content is missing from HDFS. Run integrity verification to see which files are
affected.</Notice>
                </div>
              )}
            </div>
          </div>
        </Panel>
      </div>

      <Dialog open={confirm !== null} onOpenChange={(open) => !open && setConfirm(null)}>
        <DialogContent>
          <DialogHeader>
            <DialogTitle>{confirm === "gc" ? "Remove unreferenced content?" : "Quarantine orphaned files?"}</DialogTitle>
            <DialogDescription>
              {confirm === "gc"
                ? "Content that no file refers to, including no deleted-but-restorable file, is permanently removed from HDFS. The
```

```
check is repeated at removal time."
                : "Files in HDFS with no matching metadata are moved to the recovery area. Nothing is deleted."}
            </DialogDescription>
          </DialogHeader>
          <DialogFooter>
            <Button variant="outline" onClick={() => setConfirm(null)}>
              Cancel
            </Button>
            <Button
              variant={confirm === "gc" ? "destructive" : "default"}
              onClick={() => {
                const action = confirm;
                setConfirm(null);
                if (action === "gc") guarded(() => post<GarbageReport>("/api/v1/dedup/gc"), setGarbage);
                if (action === "reconcile") guarded(() => post<ReconcileReport>("/api/v1/storage/reconcile"), setReconcile);
              }}
            >
              {confirm === "gc" ? "Remove content" : "Quarantine"}
            </Button>
          </DialogFooter>
        </DialogContent>
      </Dialog>
    </>
  );
}
```

11.7 Tests

backend/tests/conftest.py 84 lines

```python
from __future__ import annotations

import sys
from pathlib import Path

import pytest
from fastapi.testclient import TestClient

sys.path.insert(0, str(Path(__file__).resolve().parent))

from app.core.config import Settings  # noqa: E402
from app.core.db import Base, make_engine, make_session_factory  # noqa: E402
from app.main import JOB_HANDLERS, create_app  # noqa: E402
from app.services.context import AppContext, StoragePaths  # noqa: E402
from app.services.jobs import JobManager  # noqa: E402
from support.local_fs_adapter import FakeMonitor, LocalFsTestAdapter  # noqa: E402

KIB = 1024


def make_context(tmp_path: Path) -> AppContext:
    settings = Settings(
        namenode_url="http://test.invalid",
        database_path=tmp_path / "metadata.db",
        spool_dir=tmp_path / "spool",
        dataset_dir=tmp_path / "datasets",
        provenance_path=tmp_path / "datasets" / "manifest.json",
        default_chunk_size=4 * KIB,
        min_chunk_size=KIB,
        _env_file=None,
    )
    engine = make_engine(settings.database_path)
    Base.metadata.create_all(engine)
    hdfs = LocalFsTestAdapter(tmp_path / "fake-hdfs")
    ctx = AppContext(
        settings=settings,
        session_factory=make_session_factory(engine),
        hdfs=hdfs,
        paths=StoragePaths(root="/hdfs-optimizer"),
        monitor=FakeMonitor(hdfs),
    )
    ctx.ensure_namespace()
    ctx.jobs = JobManager(ctx, JOB_HANDLERS, synchronous=True)
    return ctx


@pytest.fixture
def ctx(tmp_path: Path) -> AppContext:
    return make_context(tmp_path)


@pytest.fixture
def client(ctx: AppContext):
    with TestClient(create_app(ctx)) as test_client:
        yield test_client


@pytest.fixture
def write_file(tmp_path: Path):
    """Write bytes to a spool file. Test bytes are deterministic and exist only to exercise the logic."""
    directory = tmp_path / "input"
    directory.mkdir(exist_ok=True)
    counter = {"n": 0}
```

```
    def _write(data: bytes) -> Path:
        counter["n"] += 1
        path = directory / f"input-{counter['n']}.bin"
        path.write_bytes(data)
        return path

    return _write


def pattern(seed: int, size: int) -> bytes:
    """Deterministic, non-repeating test bytes for unit tests. Never used as an evaluation dataset."""
    import hashlib

    out = bytearray()
    counter = 0
    while len(out) < size:
        out += hashlib.sha256(f"{seed}:{counter}".encode()).digest()
        counter += 1
    return bytes(out[:size])
```

backend/tests/support/local_fs_adapter.py 198 lines

```
"""TEST ONLY. A local-filesystem stand-in for HDFS, used by fast unit tests.

This is NOT HDFS and is never used by the application: it lives outside the ``app`` package. Behaviour
that depends on real HDFS is covered by ``test_integration_hdfs.py`` against the real cluster.
"""

from __future__ import annotations

import shutil
from collections.abc import Iterable, Iterator
from pathlib import Path

from app.core.errors import HdfsAlreadyExists, HdfsFileNotFound, HdfsUnavailable
from app.hdfs.base import BlockLocation, ContentSummary, FileStatus


class LocalFsTestAdapter:
    """Mimics the HDFS semantics services rely on, such as rename refusing to overwrite."""

    def __init__(self, base: Path, default_replication: int = 3):
        self.base = Path(base)
        self.base.mkdir(parents=True, exist_ok=True)
        self.replication: dict[str, int] = {}
        self._default_replication = default_replication

    def _local(self, path: str) -> Path:
        return self.base / path.lstrip("/")

    def mkdirs(self, path: str) -> None:
        self._local(path).mkdir(parents=True, exist_ok=True)

    def create(self, path, data, *, length=None, overwrite=False, replication=None) -> None:
        target = self._local(path)
        if target.exists() and not overwrite:
            raise HdfsAlreadyExists(f"Already exists: {path}")
        target.parent.mkdir(parents=True, exist_ok=True)
        with open(target, "wb") as handle:
            for block in [data] if isinstance(data, bytes) else data:
                handle.write(block)
        self.replication[path] = replication or self._default_replication

    def open(self, path: str) -> Iterator[bytes]:
        target = self._local(path)
        if not target.is_file():
            raise HdfsFileNotFound(f"Not found: {path}")
        with open(target, "rb") as handle:
            while block := handle.read(64 * 1024):
                yield block

    def _status(self, path: str, target: Path) -> FileStatus:
        return FileStatus(
            path=path,
            is_dir=target.is_dir(),
            length=0 if target.is_dir() else target.stat().st_size,
            replication=0 if target.is_dir() else self.replication.get(path, self._default_replication),
            block_size=134217728,
            modification_time=int(target.stat().st_mtime * 1000),
        )

    def status(self, path: str) -> FileStatus | None:
        target = self._local(path)
        return self._status(path, target) if target.exists() else None

    def list_dir(self, path: str) -> list[FileStatus]:
        target = self._local(path)
        if not target.is_dir():
            raise HdfsFileNotFound(f"Not found: {path}")
        return [self._status(f"{path.rstrip('/')}/{entry.name}", entry) for entry in sorted(target.iterdir())]

    def rename(self, source: str, destination: str) -> bool:
        src, dst = self._local(source), self._local(destination)
        if not src.exists() or dst.exists():
```

```
            return False
        dst.parent.mkdir(parents=True, exist_ok=True)
        src.rename(dst)
        if source in self.replication:
            self.replication[destination] = self.replication.pop(source)
        return True

    def delete(self, path: str, *, recursive: bool = False) -> bool:
        target = self._local(path)
        if not target.exists():
            return False
        if target.is_dir():
            shutil.rmtree(target)
        else:
            target.unlink()
        self.replication.pop(path, None)
        return True

    def set_replication(self, path: str, factor: int) -> bool:
        if not self._local(path).is_file():
            return False
        self.replication[path] = factor
        return True

    def _walk(self, path: str) -> Iterable[tuple[str, Path]]:
        root = self._local(path)
        if not root.exists():
            raise HdfsFileNotFound(f"Not found: {path}")
        for entry in root.rglob("*"):
            if entry.is_file():
                yield "/" + str(entry.relative_to(self.base)), entry

    def content_summary(self, path: str) -> ContentSummary:
        length = consumed = files = 0
        for logical, entry in self._walk(path):
            size = entry.stat().st_size
            length += size
            consumed += size * self.replication.get(logical, self._default_replication)
            files += 1
        directories = sum(1 for entry in self._local(path).rglob("*") if entry.is_dir()) + 1
        return ContentSummary(length=length, space_consumed=consumed, file_count=files, directory_count=directories)

    def block_locations(self, path: str) -> list[BlockLocation]:
        size = self._local(path).stat().st_size
        hosts = tuple(f"fake-dn{i}" for i in range(self.replication.get(path, self._default_replication)))
        return [BlockLocation(block_id=1, length=size, hosts=hosts, corrupt=False)]

    def default_replication(self) -> int:
        return self._default_replication


class FaultInjector:
    """Wraps an adapter and fails the Nth call to a chosen method, simulating a crash or outage."""

    def __init__(self, inner, *, method: str, on_call: int = 1, error: Exception | None = None):
        self._inner = inner
        self._method = method
        self._remaining = on_call
        self._error = error or HdfsUnavailable("Injected outage")
        self.armed = True

    def __getattr__(self, name):
        attribute = getattr(self._inner, name)
        if name != self._method:
            return attribute

        def wrapper(*args, **kwargs):
            if self.armed:
                self._remaining -= 1
                if self._remaining == 0:
                    self.armed = False
                    raise self._error
            return attribute(*args, **kwargs)

        return wrapper


class FakeMonitor:
    """TEST ONLY. Stands in for the NameNode's JMX and fsck servlets."""

    def __init__(self, hdfs: LocalFsTestAdapter, *, live: int = 3, reachable: bool = True):
        self.hdfs = hdfs
        self.live = live
        self.reachable = reachable

    def status(self) -> dict:
        if not self.reachable:
            return {"reachable": False, "error": "Injected outage", "checked_at": "now", "namenode_url": "fake"}
        return {
            "reachable": True,
            "checked_at": "now",
            "namenode_url": "fake",
            "health": "healthy",
```

```
            "problems": [],
            "namenode": {"state": "active", "software_version": "fake", "safemode": False},
            "capacity": {"total_bytes": 1000, "used_bytes": 0, "remaining_bytes": 1000, "non_dfs_used_bytes": 0},
            "namespace": {
                "files_total": 0,
                "blocks_total": 0,
                "under_replicated_blocks": 0,
                "missing_blocks": 0,
                "corrupt_blocks": 0,
                "pending_replication_blocks": 0,
                "pending_deletion_blocks": 0,
                "excess_blocks": 0,
            },
            "datanodes": {"live": self.live, "dead": 0, "stale": 0, "decommissioning": 0},
        }

    def nodes(self) -> dict:
        return {"checked_at": "now", "nodes": []}

    def fsck(self, path: str) -> dict:
        factors = list(self.hdfs.replication.values())
        return {
            "status": "HEALTHY",
            "total_files": len(factors),
            "total_blocks": len(factors),
            "over_replicated_blocks": 0,
            "under_replicated_blocks": 0,
            "missing_blocks": 0,
            "corrupt_blocks": 0,
            "average_block_replication": sum(factors) / len(factors) if factors else 0.0,
            "default_replication": self.hdfs.default_replication(),
        }
```

backend/tests/test_api.py 385 lines

```
"""REST API behaviour through the real FastAPI application."""

import hashlib

import httpx
import pytest
from app.core.errors import HdfsFileNotFound, HdfsUnavailable
from app.hdfs.webhdfs import WebHdfsClient
from app.main import create_app, run_migrations
from app.services.monitor import ClusterMonitor, parse_fsck
from conftest import make_context, pattern
from fastapi.testclient import TestClient


def upload(client, data: bytes, name="a.txt", **form):
    return client.post("/api/v1/files/upload", files={"file": (name, data)}, data=form)


def test_health_reports_api_and_cluster_separately(client):
    body = client.get("/api/v1/health").json()
    assert body["status"] == "ok" and body["hdfs_reachable"] is True


def test_openapi_documents_the_api(client):
    paths = client.get("/openapi.json").json()["paths"]
    for path in ("/api/v1/files/upload", "/api/v1/dedup/analyze", "/api/v1/cluster/replication", "/api/v1/audit/events"):
        assert path in paths


def test_upload_download_round_trip(client):
    data = pattern(1, 300_000)
    response = upload(client, data, "book.txt", mode="file")
    assert response.status_code == 201
    body = response.json()
    assert body["file"]["sha256"] == hashlib.sha256(data).hexdigest()
    download = client.get(f"/api/v1/files/{body['file']['id']}/download")
    assert download.status_code == 200
    assert download.content == data
    assert download.headers["x-content-sha256"] == body["file"]["sha256"]
    assert "book.txt" in download.headers["content-disposition"]


@pytest.mark.parametrize("mode", ["raw", "file", "chunk"])
def test_round_trip_in_every_mode(client, mode):
    data = pattern(2, 50_000)
    file_id = upload(client, data, mode=mode).json()["file"]["id"]
    assert client.get(f"/api/v1/files/{file_id}/download").content == data
    verified = client.post(f"/api/v1/files/{file_id}/verify").json()
    assert verified["ok"] is True


def test_duplicate_upload_is_reported(client):
    data = pattern(3, 1000)
    upload(client, data, "a.txt", mode="file")
    body = upload(client, data, "b.txt", mode="file").json()
    assert body["outcome"] == "duplicate_reference" and body["duplicate_of_existing"] is True
    assert body["file"]["logical_references"] == 2
```

```
def test_list_search_sort_filter_paginate(client):
    for index in range(5):
        upload(client, pattern(10 + index, 100 * (index + 1)), f"file{index}.txt", mode="file", folder="/reports")
    upload(client, pattern(99, 50), "other.log", mode="raw", folder="/logs")

    assert client.get("/api/v1/files").json()["total"] == 6
    page = client.get("/api/v1/files", params={"page_size": 2, "page": 2, "sort": "size", "order": "asc"}).json()
    assert [item["size_bytes"] for item in page["items"]] == [200, 300]
    assert client.get("/api/v1/files", params={"folder": "/logs"}).json()["total"] == 1
    assert client.get("/api/v1/files", params={"q": "file3"}).json()["total"] == 1
    assert client.get("/api/v1/files", params={"mode": "raw"}).json()["total"] == 1
    assert client.get("/api/v1/files", params={"extension": "log"}).json()["total"] == 1
    folders = client.get("/api/v1/files/folders").json()["folders"]
    assert {f["path"] for f in folders} == {"/reports", "/logs"}


def test_search_by_hash_prefix(client):
    data = pattern(20, 100)
    upload(client, data, mode="file")
    prefix = hashlib.sha256(data).hexdigest()[:12]
    assert client.get("/api/v1/files", params={"q": prefix}).json()["total"] == 1


def test_file_detail_shows_references_and_chunks(client):
    data = pattern(21, 20_000)
    first = upload(client, data, "a.bin", mode="chunk").json()["file"]["id"]
    upload(client, data, "b.bin", mode="chunk")
    detail = client.get(f"/api/v1/files/{first}").json()
    assert detail["logical_references"] == 2
    assert detail["same_content_files"][0]["name"] == "b.bin"
    assert detail["chunk_count"] == 5 and len(detail["chunks"]) == 5
    assert detail["chunk_size"] == 4096


def test_delete_restore_and_purge_flow(client):
    file_id = upload(client, pattern(22, 100), mode="file").json()["file"]["id"]
    assert client.post(f"/api/v1/files/{file_id}/restore").status_code == 409
    assert client.delete(f"/api/v1/files/{file_id}", params={"permanent": True}).status_code == 409
    assert client.delete(f"/api/v1/files/{file_id}").json()["recoverable"] is True
    assert client.get("/api/v1/files").json()["total"] == 0
    assert client.get("/api/v1/files", params={"status": "deleted"}).json()["total"] == 1
    assert client.post(f"/api/v1/files/{file_id}/restore").json()["status"] == "active"
    client.delete(f"/api/v1/files/{file_id}")
    purged = client.delete(f"/api/v1/files/{file_id}", params={"permanent": True}).json()
    assert purged["content_removed"]["object_count"] == 1
    assert client.get(f"/api/v1/files/{file_id}").status_code == 404


def test_duplicate_groups_and_reclaimable_estimate(client):
    data = pattern(23, 1000)
    upload(client, data, "a", mode="raw")
    upload(client, data, "b", mode="raw")
    upload(client, data, "c", mode="raw")
    upload(client, pattern(24, 500), "unique", mode="raw")
    body = client.get("/api/v1/dedup/groups").json()
    assert body["group_count"] == 1 and body["duplicate_files"] == 2
    group = body["groups"][0]
    assert group["copies"] == 3 and group["reclaimable_bytes"] == 2000
    assert group["already_deduplicated"] is False


def test_groups_show_nothing_reclaimable_once_deduplicated(client):
    data = pattern(25, 1000)
    upload(client, data, "a", mode="file")
    upload(client, data, "b", mode="file")
    group = client.get("/api/v1/dedup/groups").json()["groups"][0]
    assert group["reclaimable_bytes"] == 0 and group["already_deduplicated"] is True


def test_shared_chunks_endpoint(client, ctx):
    size = ctx.settings.default_chunk_size
    shared = pattern(26, size * 2)
    upload(client, shared + pattern(27, size), "a", mode="chunk")
    upload(client, shared + pattern(28, size), "b", mode="chunk")
    body = client.get("/api/v1/dedup/shared-chunks").json()
    assert body["shared_chunk_count"] == 2
    assert body["bytes_saved_by_sharing"] == size * 2


def test_analyze_optimize_flow_over_http(client):
    data = pattern(29, 8000)
    ids = [upload(client, data, f"{i}", mode="raw").json()["file"]["id"] for i in range(3)]
    assert client.post("/api/v1/dedup/optimize", json={"analysis_job_id": "missing"}).status_code == 404

    analysis = client.post("/api/v1/dedup/analyze", json={"mode": "file"})
    assert analysis.status_code == 202
    analysis_id = analysis.json()["job_id"]
    preview = client.get(f"/api/v1/dedup/jobs/{analysis_id}").json()
    assert preview["result"]["estimated"] is True

    optimize = client.post("/api/v1/dedup/optimize", json={"analysis_job_id": analysis_id})
    assert optimize.status_code == 202
    finished = client.get(f"/api/v1/dedup/jobs/{optimize.json()['job_id']}").json()
```

```
    assert finished["status"] == "succeeded" and finished["result"]["estimated"] is False
    assert finished["result"]["after"]["derived"]["stored_content_bytes"] == 8000
    for file_id in ids:
        assert client.get(f"/api/v1/files/{file_id}/download").content == data

    # The preview no longer describes the catalog, so it cannot be used again.
    assert client.post("/api/v1/dedup/optimize", json={"analysis_job_id": analysis_id}).status_code == 409


def test_invalid_inputs_are_rejected(client):
    assert upload(client, b"x", mode="nonsense").status_code == 422
    assert upload(client, b"x", mode="chunk", chunk_size=1).status_code == 422
    assert upload(client, b"x", folder="/a/../b").status_code == 422
    assert client.post("/api/v1/dedup/analyze", json={"mode": "raw"}).status_code == 422
    assert client.get("/api/v1/files", params={"sort": "evil"}).status_code == 422
    assert client.get("/api/v1/files", params={"page": 0}).status_code == 422
    assert client.post("/api/v1/cluster/replication", json={"factor": 0}).status_code == 422
    assert client.post("/api/v1/cluster/replication", json={"factor": 9}).status_code == 422
    assert client.post("/api/v1/experiments/run", json={"scenario": "nope"}).status_code == 422


def test_missing_file_returns_404_in_the_error_format(client):
    response = client.get("/api/v1/files/does-not-exist")
    assert response.status_code == 404
    assert response.json()["error"]["code"] == "not_found"


def test_upload_limit_is_enforced(ctx):
    ctx.settings.max_upload_bytes = 1000
    with TestClient(create_app(ctx)) as limited:
        response = upload(limited, pattern(30, 5000), mode="file")
        assert response.status_code == 413
        assert limited.get("/api/v1/files").json()["total"] == 0
    assert list(ctx.settings.spool_dir.iterdir()) == []


def test_token_protects_only_mutating_requests(ctx):
    ctx.settings.api_token = "unit-test-token"
    with TestClient(create_app(ctx)) as guarded:
        assert guarded.get("/api/v1/files").status_code == 200
        assert upload(guarded, b"x", mode="file").status_code == 401
        allowed = guarded.post(
            "/api/v1/files/upload", files={"file": ("a", b"x")}, data={"mode": "file"}, headers={"X-API-Key": "unit-test-token"}
        )
        assert allowed.status_code == 201


def test_cors_allows_only_configured_origins(client):
    allowed = client.get("/api/v1/health", headers={"Origin": "http://localhost:3100"})
    assert allowed.headers["access-control-allow-origin"] == "http://localhost:3100"
    denied = client.get("/api/v1/health", headers={"Origin": "http://evil.example"})
    assert "access-control-allow-origin" not in denied.headers


def test_analytics_endpoints_use_backend_data(client):
    data = pattern(31, 1000)
    upload(client, data, "a.txt", mode="file")
    upload(client, data, "b.txt", mode="file")
    overview = client.get("/api/v1/analytics/overview").json()
    assert overview["derived"]["savings_percent"] == 50.0
    assert overview["distributions"]["file_types"][0]["extension"] == "txt"
    assert overview["distributions"]["duplicate_distribution"] == [{"copies": 2, "groups": 1}]
    assert len(client.get("/api/v1/analytics/history").json()["snapshots"]) == 2
    operations = client.get("/api/v1/analytics/performance").json()["operations"]
    assert {o["operation"] for o in operations} >= {"hash", "upload"}
    report = client.get("/api/v1/analytics/report.csv")
    assert report.headers["content-type"].startswith("text/csv") and "savings_percent" in report.text


def test_hash_timing_is_a_real_measurement(client):
    upload(client, pattern(40, 2_000_000), "a.bin", mode="file")
    hash_ops = [o for o in client.get("/api/v1/analytics/performance").json()["operations"] if o["operation"] == "hash"]
    assert len(hash_ops) == 1
    # SHA-256 in software cannot approach this rate. A pre-supplied hash once reported millions of MiB/s.
    assert 0 < hash_ops[0]["throughput_mib_s"] < 50_000


def test_precomputed_hash_without_a_time_records_no_sample(ctx, write_file):
    from app.services import analytics
    from app.services.hashing import sha256_file
    from app.services.ingest import ingest_local_file

    path = write_file(pattern(41, 1000))
    ingest_local_file(ctx, path, "a.bin", mode="file", known_hash=sha256_file(path))
    from app.core.db import session_scope

    with session_scope(ctx.session_factory) as session:
        operations = {o["operation"] for o in analytics.performance(session)["operations"]}
    assert "hash" not in operations and "upload" in operations


def test_empty_system_has_genuine_empty_values(client):
    overview = client.get("/api/v1/analytics/overview").json()
```

```
    assert overview["derived"]["savings_percent"] is None
    assert overview["distributions"]["file_types"] == []
    assert client.get("/api/v1/analytics/history").json()["snapshots"] == []


def test_audit_trail_records_and_filters(client):
    data = pattern(32, 100)
    file_id = upload(client, data, "a", mode="file").json()["file"]["id"]
    upload(client, data, "b", mode="file")
    client.post(f"/api/v1/files/{file_id}/verify")
    client.delete(f"/api/v1/files/{file_id}")
    types = {e["event_type"] for e in client.get("/api/v1/audit/events").json()["items"]}
    assert {"file.uploaded", "dedup.duplicate_detected", "integrity.passed", "file.deleted"} <= types
    assert all(e["event_type"].startswith("file.") for e in client.get("/api/v1/audit/events", params={"event_type":
"file."}).json()["items"])


def test_settings_validate_and_apply(client):
    assert client.put("/api/v1/settings", json={"default_chunk_size": 3}).status_code == 422
    assert client.put("/api/v1/settings", json={"default_ingest_mode": "nope"}).status_code == 422
    assert client.put("/api/v1/settings", json={"default_ingest_mode": "chunk"}).json()["values"]["default_ingest_mode"] == "chunk"
    assert upload(client, pattern(33, 100)).json()["file"]["storage_mode"] == "chunk"


def test_diagnostics_run_real_checks(client):
    body = client.get("/api/v1/system/diagnostics").json()
    assert body["ok"] is True and len(body["checks"]) == 4


def test_cluster_replication_endpoint_validates_against_live_nodes(client):
    response = client.post("/api/v1/cluster/replication", json={"factor": 5})
    assert response.status_code == 422 and response.json()["error"]["details"]["live_datanodes"] == 3


def test_hdfs_outage_returns_503_and_leaves_no_file(tmp_path):
    ctx = make_context(tmp_path)
    ctx.hdfs = WebHdfsClient("http://127.0.0.1:9", connect_timeout_seconds=0.2)
    with TestClient(create_app(ctx)) as test_client:
        response = upload(test_client, b"data", mode="file")
        assert response.status_code == 503
        assert response.json()["error"]["code"] == "hdfs_unavailable"
        assert test_client.get("/api/v1/files").json()["total"] == 0
        events = test_client.get("/api/v1/audit/events", params={"severity": "error"}).json()["items"]
        assert events[0]["event_type"] == "request.failed"
        assert "/api/v1/files/upload" in events[0]["message"]


# -- WebHDFS client and parser, using a mock transport (no network) ---------------------------------


def _client_with(handler) -> WebHdfsClient:
    client = WebHdfsClient("http://nn:9870", datanode_map={"datanode1": "127.0.0.1:19864"})
    client._http = httpx.Client(transport=httpx.MockTransport(handler), follow_redirects=False)
    return client


def test_webhdfs_create_follows_two_step_protocol_and_rewrites_datanode_host():
    seen = []

    def handler(request: httpx.Request) -> httpx.Response:
        seen.append(request)
        if request.url.host == "nn":
            return httpx.Response(307, headers={"location": "http://datanode1:9864/webhdfs/v1/a?op=CREATE"})
        return httpx.Response(201)

    _client_with(handler).create("/a", b"hello", length=5)
    assert seen[0].url.params["op"] == "CREATE" and seen[0].url.params["overwrite"] == "false"
    assert (seen[1].url.host, seen[1].url.port) == ("127.0.0.1", 19864)
    assert seen[1].content == b"hello"


def test_webhdfs_maps_remote_exceptions():
    def not_found(_):
        return httpx.Response(404, json={"RemoteException": {"exception": "FileNotFoundException", "message": "no"}})

    with pytest.raises(HdfsFileNotFound):
        _client_with(not_found).content_summary("/x")
    assert _client_with(not_found).status("/x") is None

    def safemode(_):
        return httpx.Response(403, json={"RemoteException": {"exception": "SafeModeException", "message": "safe"}})

    with pytest.raises(HdfsUnavailable):
        _client_with(safemode).mkdirs("/x")


def test_webhdfs_connection_failure_is_unavailable():
    client = WebHdfsClient("http://127.0.0.1:9", connect_timeout_seconds=0.2)
    with pytest.raises(HdfsUnavailable):
        client.status("/x")


def test_webhdfs_encodes_paths():
```

```
    seen = []

    def handler(request):
        seen.append(request.url.raw_path)
        return httpx.Response(200, json={"boolean": True})

    _client_with(handler).delete("/a b/c")
    assert b"/webhdfs/v1/a%20b/c" in seen[0]


def test_monitor_reports_unreachable_namenode_as_data():
    status = ClusterMonitor("http://127.0.0.1:9", timeout_seconds=0.2).status()
    assert status["reachable"] is False and "Cannot reach" in status["error"]


def test_parse_fsck_summary():
    text = """
 Total size:  1000 B
 Total files: 2
 Total blocks (validated):  2 (avg. block size 500 B)
 Minimally replicated blocks: 2 (100.0 %)
 Over-replicated blocks:  0 (0.0 %)
 Under-replicated blocks: 1 (50.0 %)
 Mis-replicated blocks:   0 (0.0 %)
 Default replication factor:  3
 Average block replication: 2.5
 Missing blocks:    0
 Corrupt blocks:    0
 Missing replicas:    1 (16.6 %)
 Number of data-nodes:    2
The filesystem under path '/x' is HEALTHY
Status: HEALTHY
"""
    parsed = parse_fsck(text)
    assert parsed["status"] == "HEALTHY"
    assert parsed["total_blocks"] == 2 and parsed["under_replicated_blocks"] == 1
    assert parsed["average_block_replication"] == 2.5 and parsed["datanodes"] == 2


def test_migrations_build_the_same_tables_as_the_models(tmp_path):
    from app.core.db import Base
    from sqlalchemy import create_engine, inspect

    database = tmp_path / "migrated.db"
    run_migrations(database)
    tables = set(inspect(create_engine(f"sqlite:///{database}")).get_table_names()) - {"alembic_version"}
    assert tables == set(Base.metadata.tables)
```

backend/tests/test_hashing_chunking.py 57 lines

```
import hashlib

import pytest
from app.services.hashing import (
    fixed_size_chunks,
    sha256_bytes,
    sha256_file,
    sha256_stream,
)
from conftest import pattern


def test_sha256_matches_known_vector():
    # FIPS 180 test vector for "abc".
    assert sha256_bytes(b"abc") == "ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad"


def test_sha256_of_empty_input():
    assert sha256_bytes(b"") == "e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855"


def test_file_and_stream_hash_agree_with_hashlib(tmp_path):
    data = pattern(1, 3 * 1024 * 1024 + 17)
    path = tmp_path / "f.bin"
    path.write_bytes(data)
    expected = hashlib.sha256(data).hexdigest()
    assert sha256_file(path) == (expected, len(data))
    assert sha256_stream([data[:5], data[5:]]) == (expected, len(data))


@pytest.mark.parametrize("input_block", [1, 7, 4096, 100_000])
def test_chunking_does_not_depend_on_input_buffering(input_block):
    data = pattern(2, 50_000)
    stream = [data[i : i + input_block] for i in range(0, len(data), input_block)]
    assert list(fixed_size_chunks(stream, 4096)) == list(fixed_size_chunks([data], 4096))


def test_chunks_have_exact_size_and_rejoin_to_the_original():
    data = pattern(3, 10_001)
    chunks = list(fixed_size_chunks([data], 4096))
    assert [len(c) for c in chunks] == [4096, 4096, 1809]
    assert b"".join(chunks) == data
```

```
def test_exact_multiple_has_no_empty_tail():
    chunks = list(fixed_size_chunks([pattern(4, 8192)], 4096))
    assert [len(c) for c in chunks] == [4096, 4096]


def test_empty_stream_yields_no_chunks():
    assert list(fixed_size_chunks([], 4096)) == []


def test_invalid_chunk_size_is_rejected():
    with pytest.raises(ValueError):
        list(fixed_size_chunks([b"x"], 0))
```

backend/tests/test_ingest_dedup.py 217 lines

```
"""Whole-file and chunk-level deduplication behaviour."""

import pytest
from app.core.db import session_scope
from app.core.errors import ValidationFailure
from app.models.entities import (
    Chunk,
    ContentObject,
    FileChunk,
    FileManifest,
    LogicalFile,
)
from app.services import analytics, integrity
from app.services.hashing import sha256_bytes
from app.services.ingest import ingest_local_file, normalize_folder, validate_filename
from conftest import pattern
from sqlalchemy import func, select


def count(ctx, model) -> int:
    with session_scope(ctx.session_factory) as session:
        return session.scalar(select(func.count()).select_from(model))


def read_back(ctx, file_id) -> bytes:
    return b"".join(integrity.stream_verified(ctx, file_id))


def test_upload_stores_one_object_at_a_content_addressed_path(ctx, write_file):
    data = pattern(1, 10_000)
    result = ingest_local_file(ctx, write_file(data), "a.txt", mode="file")
    sha = sha256_bytes(data)
    assert result.outcome == "stored_new"
    assert result.file.sha256 == sha
    assert result.bytes_written == len(data)
    assert ctx.hdfs.status(ctx.paths.object_path(sha)).length == len(data)
    assert read_back(ctx, result.file.id) == data


def test_identical_bytes_under_different_names_are_stored_once(ctx, write_file):
    data = pattern(2, 10_000)
    first = ingest_local_file(ctx, write_file(data), "report.txt", mode="file")
    second = ingest_local_file(ctx, write_file(data), "completely-different-name.pdf", mode="file")
    assert second.outcome == "duplicate_reference"
    assert second.duplicate_of_existing is True
    assert second.bytes_written == 0
    assert first.file.id != second.file.id
    assert count(ctx, ContentObject) == 1
    assert count(ctx, LogicalFile) == 2
    assert read_back(ctx, second.file.id) == data


def test_different_content_with_same_filename_stays_distinct(ctx, write_file):
    one, two = pattern(3, 5_000), pattern(4, 5_000)
    first = ingest_local_file(ctx, write_file(one), "same.txt", mode="file")
    second = ingest_local_file(ctx, write_file(two), "same.txt", mode="file")
    assert second.outcome == "stored_new"
    assert second.duplicate_of_existing is False
    assert second.file.name == "same (1).txt"
    assert count(ctx, ContentObject) == 2
    assert read_back(ctx, first.file.id) == one
    assert read_back(ctx, second.file.id) == two


def test_raw_mode_deliberately_keeps_every_copy(ctx, write_file):
    data = pattern(5, 2_000)
    first = ingest_local_file(ctx, write_file(data), "a.bin", mode="raw")
    second = ingest_local_file(ctx, write_file(data), "b.bin", mode="raw")
    assert (first.outcome, second.outcome) == ("stored_raw", "stored_raw")
    assert first.file.raw_path != second.file.raw_path
    assert count(ctx, ContentObject) == 0
    with session_scope(ctx.session_factory) as session:
        stats = analytics.storage_statistics(session)
    assert stats["stored_content_bytes"] == 4_000
    assert stats["duplicate_files"] == 1


def test_chunk_mode_shares_identical_chunks_between_different_files(ctx, write_file):
    size = ctx.settings.default_chunk_size
```

```
    shared = pattern(6, size * 3)
    file_a = shared + pattern(7, size)
    file_b = shared + pattern(8, size)
    first = ingest_local_file(ctx, write_file(file_a), "a.bin", mode="chunk")
    second = ingest_local_file(ctx, write_file(file_b), "b.bin", mode="chunk")
    assert (first.chunks_total, first.chunks_new) == (4, 4)
    assert (second.chunks_total, second.chunks_new, second.chunks_reused) == (4, 1, 3)
    assert second.outcome == "chunks_shared"
    assert count(ctx, Chunk) == 5
    assert count(ctx, FileChunk) == 8
    assert read_back(ctx, first.file.id) == file_a
    assert read_back(ctx, second.file.id) == file_b


def test_repeated_chunks_within_one_file_are_stored_once(ctx, write_file):
    size = ctx.settings.default_chunk_size
    block = pattern(9, size)
    data = block * 5
    result = ingest_local_file(ctx, write_file(data), "repeat.bin", mode="chunk")
    assert (result.chunks_total, result.chunks_new, result.chunks_reused) == (5, 1, 4)
    assert read_back(ctx, result.file.id) == data


def test_identical_file_in_chunk_mode_reuses_the_manifest(ctx, write_file):
    data = pattern(10, 20_000)
    ingest_local_file(ctx, write_file(data), "a.bin", mode="chunk")
    second = ingest_local_file(ctx, write_file(data), "b.bin", mode="chunk")
    assert second.outcome == "duplicate_reference"
    assert second.bytes_written == 0
    assert count(ctx, FileManifest) == 1


def test_chunk_upload_of_content_already_stored_whole_writes_nothing(ctx, write_file):
    data = pattern(11, 20_000)
    ingest_local_file(ctx, write_file(data), "a.bin", mode="file")
    second = ingest_local_file(ctx, write_file(data), "b.bin", mode="chunk")
    assert second.bytes_written == 0
    assert second.file.storage_mode == "file"
    assert count(ctx, Chunk) == 0


def test_file_upload_of_content_already_chunked_writes_nothing(ctx, write_file):
    data = pattern(12, 20_000)
    ingest_local_file(ctx, write_file(data), "a.bin", mode="chunk")
    second = ingest_local_file(ctx, write_file(data), "b.bin", mode="file")
    assert second.bytes_written == 0
    assert second.file.storage_mode == "chunk"
    assert count(ctx, ContentObject) == 0


def test_empty_file_round_trip_in_every_mode(ctx, write_file):
    for index, mode in enumerate(("raw", "file", "chunk")):
        result = ingest_local_file(ctx, write_file(b""), f"empty{index}", mode=mode)
        assert result.file.size_bytes == 0
        assert read_back(ctx, result.file.id) == b""
        assert integrity.verify_file(ctx, result.file.id).ok


def test_upload_repairs_an_object_missing_from_hdfs(ctx, write_file):
    data = pattern(13, 3_000)
    first = ingest_local_file(ctx, write_file(data), "a.bin", mode="file")
    ctx.hdfs.delete(ctx.paths.object_path(first.file.sha256))
    second = ingest_local_file(ctx, write_file(data), "b.bin", mode="file")
    assert second.outcome == "duplicate_reference"
    assert second.bytes_written == len(data)
    assert read_back(ctx, first.file.id) == data


def test_upload_repairs_a_chunk_missing_from_hdfs(ctx, write_file):
    data = pattern(14, 20_000)
    first = ingest_local_file(ctx, write_file(data), "a.bin", mode="chunk")
    with session_scope(ctx.session_factory) as session:
        victim = session.scalars(select(Chunk)).first().hdfs_path
    ctx.hdfs.delete(victim)
    ingest_local_file(ctx, write_file(data), "b.bin", mode="chunk")
    assert read_back(ctx, first.file.id) == data


def test_failed_hdfs_write_leaves_no_metadata_and_no_staging(ctx, write_file):
    from support.local_fs_adapter import FaultInjector

    ctx.hdfs = FaultInjector(ctx.hdfs, method="rename")
    with pytest.raises(Exception, match="Injected outage"):
        ingest_local_file(ctx, write_file(pattern(15, 1_000)), "a.bin", mode="file")
    assert count(ctx, LogicalFile) == 0
    assert count(ctx, ContentObject) == 0
    assert ctx.hdfs.list_dir(ctx.paths.staging) == []


def test_partial_write_detected_by_length_check(ctx, write_file):
    """HDFS accepting fewer bytes than sent must never produce a metadata row."""
    inner_create = ctx.hdfs.create
```

```
    def truncating_create(path, data, **kwargs):
        blocks = b"".join([data] if isinstance(data, bytes) else data)
        inner_create(path, blocks[:-1], **kwargs)

    ctx.hdfs.create = truncating_create
    with pytest.raises(Exception, match="expected 1000"):
        ingest_local_file(ctx, write_file(pattern(16, 1_000)), "a.bin", mode="file")
    assert count(ctx, LogicalFile) == 0


def test_corruption_during_staging_detected_by_hash_check(ctx, write_file):
    inner_create = ctx.hdfs.create

    def flipping_create(path, data, **kwargs):
        blocks = bytearray(b"".join([data] if isinstance(data, bytes) else data))
        blocks[0] ^= 0xFF
        inner_create(path, bytes(blocks), **kwargs)

    ctx.hdfs.create = flipping_create
    with pytest.raises(Exception, match="SHA-256"):
        ingest_local_file(ctx, write_file(pattern(17, 1_000)), "a.bin", mode="file")
    assert count(ctx, ContentObject) == 0


@pytest.mark.parametrize("bad", ["", "..", ".", "bad\x00name", "x" * 256])
def test_invalid_file_names_are_rejected(bad):
    with pytest.raises(ValidationFailure):
        validate_filename(bad)


def test_file_name_cannot_escape_through_path_segments():
    assert validate_filename("../../etc/passwd") == "passwd"
    assert validate_filename("..\\..\\secret.txt") == "secret.txt"


@pytest.mark.parametrize("bad", ["/a/../b", "..", "/./x", "/a\x01b"])
def test_invalid_folders_are_rejected(bad):
    with pytest.raises(ValidationFailure):
        normalize_folder(bad)


def test_folders_are_normalized():
    assert normalize_folder(None) == "/"
    assert normalize_folder("reports//2026/") == "/reports/2026"
```

backend/tests/test_integration_hdfs.py 269 lines

```
"""Integration tests against the REAL HDFS cluster, using authentic Project Gutenberg files.

Run with:  pytest -m hdfs
Each test uses a private directory under /hdfs-optimizer-tests and removes it afterwards.
"""

from __future__ import annotations

import hashlib
import json
import time
import uuid
from pathlib import Path

import httpx
import pytest
from app.core.config import PROJECT_DIR, Settings
from app.core.db import Base, make_engine, make_session_factory
from app.core.errors import HdfsFileNotFound
from app.hdfs.webhdfs import WebHdfsClient
from app.main import JOB_HANDLERS
from app.models.entities import DeduplicationJob as JOB_MODEL
from app.models.entities import ReplicationOperation as OPERATION_MODEL
from app.services import integrity, lifecycle, replication
from app.services.context import AppContext, StoragePaths
from app.services.ingest import ingest_local_file
from app.services.jobs import JobManager
from app.services.monitor import ClusterMonitor

pytestmark = pytest.mark.hdfs

NAMENODE = Settings(_env_file=None).namenode_url
ROOT = "/hdfs-optimizer-tests"
MANIFEST = PROJECT_DIR / "datasets" / "provenance" / "manifest.json"


def _cluster_up() -> bool:
    try:
        return httpx.get(f"{NAMENODE}/jmx?qry=Hadoop:service=NameNode,name=NameNodeStatus", timeout=3).is_success
    except httpx.HTTPError:
        return False


@pytest.fixture(scope="module", autouse=True)
def require_cluster():
    if not _cluster_up():
        pytest.skip(f"No HDFS cluster at {NAMENODE}. Start it with: docker compose up -d --wait")
```

```
@pytest.fixture
def real_ctx(tmp_path: Path):
    settings = Settings(
        _env_file=None,
        database_path=tmp_path / "metadata.db",
        spool_dir=tmp_path / "spool",
        dataset_dir=PROJECT_DIR / "datasets" / "raw",
        provenance_path=MANIFEST,
    )
    engine = make_engine(settings.database_path)
    Base.metadata.create_all(engine)
    hdfs = WebHdfsClient(settings.namenode_url, datanode_map=settings.datanode_map)
    run_root = f"{ROOT}/{uuid.uuid4().hex}"
    ctx = AppContext(
        settings=settings,
        session_factory=make_session_factory(engine),
        hdfs=hdfs,
        paths=StoragePaths(root=run_root),
        monitor=ClusterMonitor(settings.namenode_url),
    )
    ctx.jobs = JobManager(ctx, JOB_HANDLERS, synchronous=True)
    ctx.ensure_namespace()
    yield ctx
    hdfs.delete(run_root, recursive=True)
    hdfs.close()
    engine.dispose()


def _source(ebook_id: int = 11, file_format: str = "txt") -> dict:
    if not MANIFEST.exists():
        pytest.skip("Dataset not downloaded. Run scripts/download_dataset.py")
    manifest = json.loads(MANIFEST.read_text())
    for record in manifest["files"]:
        if record["kind"] == "original" and record["ebook_id"] == ebook_id and record["file_format"] == file_format:
            path = PROJECT_DIR / "datasets" / "raw" / record["local_path"]
            if path.exists():
                return {**record, "path": path}
    pytest.skip(f"Authentic file for ebook {ebook_id} is not present.")


def test_authentic_file_matches_recorded_provenance():
    source = _source(1342)
    assert hashlib.sha256(source["path"].read_bytes()).hexdigest() == source["sha256"]
    assert source["source_url"] == "https://www.gutenberg.org/ebooks/1342"
    assert source["header_title_in_file"] == "Pride and Prejudice"


def test_webhdfs_primitives_on_the_real_cluster(real_ctx):
    client, base = real_ctx.hdfs, real_ctx.paths.staging
    path = f"{base}/probe with space.bin"
    client.create(path, b"hello hdfs", length=10)
    status = client.status(path)
    assert status.length == 10 and status.replication >= 1
    assert b"".join(client.open(path)) == b"hello hdfs"
    # A real HDFS rename refuses to replace an existing file; the staged-write protocol depends on this.
    client.create(f"{base}/other", b"x", length=1)
    assert client.rename(path, f"{base}/other") is False
    assert client.rename(path, f"{base}/renamed") is True
    assert client.status(path) is None
    assert client.delete(f"{base}/renamed") is True
    with pytest.raises(HdfsFileNotFound):
        list(client.open(f"{base}/renamed"))


def test_empty_file_can_be_stored_on_the_real_cluster(real_ctx, tmp_path):
    empty = tmp_path / "empty.txt"
    empty.write_bytes(b"")
    for mode in ("raw", "file", "chunk"):
        result = ingest_local_file(real_ctx, empty, f"empty-{mode}", mode=mode)
        assert b"".join(integrity.stream_verified(real_ctx, result.file.id)) == b""


@pytest.mark.parametrize("mode", ["raw", "file", "chunk"])
def test_authentic_file_round_trips_byte_for_byte(real_ctx, tmp_path, mode):
    source = _source()
    result = ingest_local_file(real_ctx, source["path"], source["path"].name, mode=mode, chunk_size=64 * 1024)
    target = tmp_path / "restored"
    digest = integrity.restore_to_path(real_ctx, result.file.id, target)
    assert digest == source["sha256"]
    assert target.read_bytes() == source["path"].read_bytes()
    assert integrity.verify_file(real_ctx, result.file.id).ok


def test_binary_authentic_file_round_trips_through_chunks(real_ctx, tmp_path):
    source = _source(11, "epub")
    result = ingest_local_file(real_ctx, source["path"], source["path"].name, mode="chunk", chunk_size=64 * 1024)
    target = tmp_path / "restored.epub"
    assert integrity.restore_to_path(real_ctx, result.file.id, target) == source["sha256"]


def test_exact_duplicate_consumes_no_additional_hdfs_space(real_ctx):
```

```
    source = _source()
    first = ingest_local_file(real_ctx, source["path"], "original.txt", mode="file")
    before = real_ctx.hdfs.content_summary(real_ctx.paths.root)
    second = ingest_local_file(real_ctx, source["path"], "workload-copy.txt", mode="file", provenance_label="workload_copy")
    after = real_ctx.hdfs.content_summary(real_ctx.paths.root)
    assert second.outcome == "duplicate_reference"
    assert after.length == before.length
    assert after.space_consumed == before.space_consumed
    assert first.file.id != second.file.id


def test_hdfs_replicates_stored_objects(real_ctx):
    source = _source()
    result = ingest_local_file(real_ctx, source["path"], source["path"].name, mode="file")
    path = real_ctx.paths.object_path(result.file.sha256)
    expected = real_ctx.hdfs.default_replication()
    assert real_ctx.hdfs.status(path).replication == expected
    # A write returns once the minimum replication is met. The NameNode learns of the remaining replicas
    # through asynchronous block reports, so the full set appears shortly afterwards.
    deadline = time.monotonic() + 20
    blocks = real_ctx.hdfs.block_locations(path)
    while len(blocks[0].hosts) < expected and time.monotonic() < deadline:
        time.sleep(0.5)
        blocks = real_ctx.hdfs.block_locations(path)
    assert len(blocks) == 1
    assert len(blocks[0].hosts) == expected
    assert len(set(blocks[0].hosts)) == expected  # replicas are on distinct DataNodes


def test_measured_space_includes_replication(real_ctx):
    source = _source()
    ingest_local_file(real_ctx, source["path"], source["path"].name, mode="file")
    summary = real_ctx.hdfs.content_summary(real_ctx.paths.objects)
    assert summary.length == source["size_bytes"]
    assert summary.space_consumed == source["size_bytes"] * real_ctx.hdfs.default_replication()


def test_optimization_reduces_hdfs_measured_space_and_preserves_bytes(real_ctx, tmp_path):
    source = _source()
    ids = [
        ingest_local_file(real_ctx, source["path"], f"copy-{i}.txt", mode="raw", provenance_label="workload_copy").file.id
        for i in range(3)
    ]
    before = real_ctx.hdfs.content_summary(real_ctx.paths.root)
    assert before.length == source["size_bytes"] * 3

    job_id = real_ctx.jobs.submit("optimize", mode="file")
    with real_ctx.session_factory() as session:
        job = session.get(JOB_MODEL, job_id)
        assert job.status == "succeeded", job.error

    after = real_ctx.hdfs.content_summary(real_ctx.paths.root)
    assert after.length == source["size_bytes"]
    assert after.space_consumed == before.space_consumed / 3
    for file_id in ids:
        target = tmp_path / f"{file_id}.txt"
        assert integrity.restore_to_path(real_ctx, file_id, target) == source["sha256"]


def test_chunk_optimization_on_real_hdfs(real_ctx, tmp_path):
    source = _source()
    ids = [ingest_local_file(real_ctx, source["path"], f"c{i}.txt", mode="file").file.id for i in range(2)]
    real_ctx.jobs.submit("optimize", mode="chunk", chunk_size=64 * 1024)
    assert real_ctx.hdfs.content_summary(real_ctx.paths.objects).file_count == 0
    expected_chunks = -(-source["size_bytes"] // (64 * 1024))
    assert real_ctx.hdfs.content_summary(real_ctx.paths.chunks).file_count == expected_chunks
    for file_id in ids:
        target = tmp_path / f"{file_id}.txt"
        assert integrity.restore_to_path(real_ctx, file_id, target) == source["sha256"]


def test_deleting_shared_reference_on_real_hdfs(real_ctx):
    source = _source()
    first = ingest_local_file(real_ctx, source["path"], "a.txt", mode="chunk", chunk_size=64 * 1024)
    second = ingest_local_file(real_ctx, source["path"], "b.txt", mode="chunk", chunk_size=64 * 1024)
    lifecycle.soft_delete(real_ctx, first.file.id)
    lifecycle.purge(real_ctx, first.file.id)
    assert integrity.verify_file(real_ctx, second.file.id).ok
    lifecycle.soft_delete(real_ctx, second.file.id)
    lifecycle.purge(real_ctx, second.file.id)
    assert real_ctx.hdfs.content_summary(real_ctx.paths.root).file_count == 0


def test_cluster_monitor_reads_real_namenode(real_ctx):
    status = real_ctx.monitor.status()
    assert status["reachable"] is True
    assert status["namenode"]["state"] == "active"
    assert status["namenode"]["software_version"].startswith("3.")
    assert status["datanodes"]["live"] >= 1
    nodes = real_ctx.monitor.nodes()["nodes"]
    assert len([n for n in nodes if n["state"] == "live"]) == status["datanodes"]["live"]
    assert all(n["capacity_bytes"] > 0 for n in nodes if n["state"] == "live")
```

```
def test_fsck_servlet_is_parsed(real_ctx):
    source = _source()
    ingest_local_file(real_ctx, source["path"], source["path"].name, mode="file")
    fsck = real_ctx.monitor.fsck(real_ctx.paths.root)
    assert fsck["status"] == "HEALTHY"
    assert fsck["total_blocks"] >= 1
    assert fsck["missing_blocks"] == 0


def test_replication_change_converges_on_real_cluster(real_ctx):
    live = real_ctx.monitor.status()["datanodes"]["live"]
    if live < 2:
        pytest.skip("Needs at least 2 live DataNodes.")
    source = _source()
    result = ingest_local_file(real_ctx, source["path"], source["path"].name, mode="file")
    path = real_ctx.paths.object_path(result.file.sha256)

    change = replication.request_change(real_ctx, 2, acknowledge_reduced_protection=True)
    assert real_ctx.hdfs.status(path).replication == 2
    summary = real_ctx.hdfs.content_summary(real_ctx.paths.objects)
    assert summary.space_consumed == source["size_bytes"] * 2
    with real_ctx.session_factory() as session:
        operation = session.get(OPERATION_MODEL, change["operation_id"])
        assert operation.status == "verified"
    assert len(real_ctx.hdfs.block_locations(path)[0].hosts) == 2


def test_replication_beyond_live_datanodes_is_refused_on_real_cluster(real_ctx):
    from app.core.errors import ValidationFailure

    live = real_ctx.monitor.status()["datanodes"]["live"]
    with pytest.raises(ValidationFailure):
        replication.request_change(real_ctx, live + 1)
```

backend/tests/test_lifecycle_integrity.py 296 lines

```
"""Deletion, restoration, garbage collection, integrity checks and crash recovery."""

import pytest
from app.core.db import session_scope
from app.core.errors import ConflictError, IntegrityFailure, NotFoundError
from app.models.entities import (
    Chunk,
    ContentObject,
    ContentState,
    DeduplicationJob,
    FileManifest,
    IntegrityCheck,
    JobStatus,
    LogicalFile,
)
from app.services import analytics, integrity, lifecycle
from app.services.ingest import ingest_local_file
from conftest import pattern
from sqlalchemy import func, select
from support.local_fs_adapter import FaultInjector


def get_stats(ctx):
    with session_scope(ctx.session_factory) as session:
        return analytics.storage_statistics(session)


def content_state(ctx, model, sha_column, sha):
    with session_scope(ctx.session_factory) as session:
        return session.scalar(select(model.state).where(sha_column == sha))


@pytest.mark.parametrize("mode", ["file", "chunk"])
def test_deleting_one_reference_keeps_content_for_the_other(ctx, write_file, mode):
    data = pattern(1, 20_000)
    first = ingest_local_file(ctx, write_file(data), "a.bin", mode=mode)
    second = ingest_local_file(ctx, write_file(data), "b.bin", mode=mode)
    lifecycle.soft_delete(ctx, first.file.id)
    lifecycle.purge(ctx, first.file.id)
    assert integrity.verify_file(ctx, second.file.id).ok
    assert b"".join(integrity.stream_verified(ctx, second.file.id)) == data


@pytest.mark.parametrize("mode", ["file", "chunk"])
def test_soft_deleted_file_keeps_content_and_can_be_restored(ctx, write_file, mode):
    data = pattern(2, 20_000)
    result = ingest_local_file(ctx, write_file(data), "a.bin", mode=mode)
    lifecycle.soft_delete(ctx, result.file.id)
    report = lifecycle.collect_garbage(ctx)
    assert report["object_count"] == report["chunk_count"] == 0  # deleted but restorable: must not collect
    restored = lifecycle.restore(ctx, result.file.id)
    assert restored.status == "active"
    assert b"".join(integrity.stream_verified(ctx, result.file.id)) == data


def test_raw_soft_delete_moves_to_recovery_and_restore_moves_back(ctx, write_file):
```

```
    data = pattern(3, 5_000)
    result = ingest_local_file(ctx, write_file(data), "a.bin", mode="raw")
    original_path = result.file.raw_path
    lifecycle.soft_delete(ctx, result.file.id)
    assert ctx.hdfs.status(original_path) is None
    assert ctx.hdfs.status(ctx.paths.recovery_raw_path(result.file.id)) is not None
    lifecycle.restore(ctx, result.file.id)
    assert ctx.hdfs.status(original_path).length == len(data)


def test_restore_avoids_a_name_taken_since_deletion(ctx, write_file):
    first = ingest_local_file(ctx, write_file(pattern(4, 100)), "a.bin", mode="file")
    lifecycle.soft_delete(ctx, first.file.id)
    ingest_local_file(ctx, write_file(pattern(5, 100)), "a.bin", mode="file")
    assert lifecycle.restore(ctx, first.file.id).name == "a (1).bin"


def test_purging_the_last_reference_removes_content_from_hdfs(ctx, write_file):
    data = pattern(6, 5_000)
    result = ingest_local_file(ctx, write_file(data), "a.bin", mode="file")
    path = ctx.paths.object_path(result.file.sha256)
    lifecycle.soft_delete(ctx, result.file.id)
    report = lifecycle.purge(ctx, result.file.id)
    assert report["object_count"] == 1
    assert ctx.hdfs.status(path) is None
    assert get_stats(ctx)["stored_content_bytes"] == 0


def test_gc_keeps_chunks_still_used_by_another_manifest(ctx, write_file):
    size = ctx.settings.default_chunk_size
    shared = pattern(7, size * 2)
    file_a = shared + pattern(8, size)
    file_b = shared + pattern(9, size)
    first = ingest_local_file(ctx, write_file(file_a), "a.bin", mode="chunk")
    second = ingest_local_file(ctx, write_file(file_b), "b.bin", mode="chunk")
    lifecycle.soft_delete(ctx, first.file.id)
    report = lifecycle.purge(ctx, first.file.id)
    assert report["chunk_count"] == 1  # only the chunk unique to file A
    assert report["manifest_count"] == 1
    assert b"".join(integrity.stream_verified(ctx, second.file.id)) == file_b
    lifecycle.soft_delete(ctx, second.file.id)
    report = lifecycle.purge(ctx, second.file.id)
    assert report["chunk_count"] == 3
    assert get_stats(ctx)["chunk_count"] == 0


def test_gc_dry_run_changes_nothing(ctx, write_file):
    result = ingest_local_file(ctx, write_file(pattern(10, 1_000)), "a.bin", mode="file")
    with session_scope(ctx.session_factory) as session:
        file = session.get(LogicalFile, result.file.id)
        file.status = "purged"
        file.object_id = None
    report = lifecycle.collect_garbage(ctx, dry_run=True)
    assert report["object_count"] == 1 and report["removed"] is False
    assert ctx.hdfs.status(ctx.paths.object_path(result.file.sha256)) is not None
    assert content_state(ctx, ContentObject, ContentObject.sha256, result.file.sha256) == ContentState.ACTIVE


def test_reuploading_collected_content_reactivates_the_row(ctx, write_file):
    data = pattern(11, 1_000)
    result = ingest_local_file(ctx, write_file(data), "a.bin", mode="file")
    lifecycle.soft_delete(ctx, result.file.id)
    lifecycle.purge(ctx, result.file.id)
    again = ingest_local_file(ctx, write_file(data), "b.bin", mode="file")
    assert again.outcome == "stored_new"
    assert b"".join(integrity.stream_verified(ctx, again.file.id)) == data
    with session_scope(ctx.session_factory) as session:
        assert session.scalar(select(func.count()).select_from(ContentObject)) == 1


def test_gc_interrupted_after_marking_is_finished_at_startup(ctx, write_file):
    result = ingest_local_file(ctx, write_file(pattern(12, 1_000)), "a.bin", mode="file")
    lifecycle.soft_delete(ctx, result.file.id)
    real = ctx.hdfs
    ctx.hdfs = FaultInjector(real, method="delete")
    with pytest.raises(Exception, match="Injected"):
        lifecycle.purge(ctx, result.file.id)
    assert content_state(ctx, ContentObject, ContentObject.sha256, result.file.sha256) == ContentState.GC_PENDING
    ctx.hdfs = real
    summary = lifecycle.recover_on_startup(ctx)
    assert summary["pending_gc"] == "finished"
    assert content_state(ctx, ContentObject, ContentObject.sha256, result.file.sha256) == ContentState.REMOVED
    assert ctx.hdfs.status(ctx.paths.object_path(result.file.sha256)) is None


def test_purge_requires_soft_delete_first(ctx, write_file):
    result = ingest_local_file(ctx, write_file(pattern(13, 100)), "a.bin", mode="file")
    with pytest.raises(ConflictError):
        lifecycle.purge(ctx, result.file.id)


def test_double_delete_and_restore_of_active_are_rejected(ctx, write_file):
    result = ingest_local_file(ctx, write_file(pattern(14, 100)), "a.bin", mode="file")
```

```
    with pytest.raises(ConflictError):
        lifecycle.restore(ctx, result.file.id)
    lifecycle.soft_delete(ctx, result.file.id)
    with pytest.raises(ConflictError):
        lifecycle.soft_delete(ctx, result.file.id)


def test_purged_file_is_gone(ctx, write_file):
    result = ingest_local_file(ctx, write_file(pattern(15, 100)), "a.bin", mode="file")
    lifecycle.soft_delete(ctx, result.file.id)
    lifecycle.purge(ctx, result.file.id)
    with pytest.raises(NotFoundError):
        integrity.verify_file(ctx, result.file.id)


# -- integrity ---------------------------------------------------------------------------------


@pytest.mark.parametrize("mode", ["raw", "file", "chunk"])
def test_verification_passes_on_intact_data_and_records_a_row(ctx, write_file, mode):
    data = pattern(20, 20_000)
    result = ingest_local_file(ctx, write_file(data), "a.bin", mode=mode)
    check = integrity.verify_file(ctx, result.file.id)
    assert check.ok and check.actual_sha256 == result.file.sha256 and check.bytes_read == len(data)
    with session_scope(ctx.session_factory) as session:
        assert session.scalar(select(func.count()).select_from(IntegrityCheck)) == 1
        assert session.get(LogicalFile, result.file.id).last_verified_ok is True


def test_verification_detects_a_corrupted_object(ctx, write_file):
    result = ingest_local_file(ctx, write_file(pattern(21, 1_000)), "a.bin", mode="file")
    path = ctx.paths.object_path(result.file.sha256)
    ctx.hdfs.delete(path)
    ctx.hdfs.create(path, pattern(22, 1_000))
    check = integrity.verify_file(ctx, result.file.id)
    assert not check.ok and check.actual_sha256 != result.file.sha256
    with pytest.raises(IntegrityFailure):
        b"".join(integrity.stream_verified(ctx, result.file.id)) and None


def test_verification_reports_a_missing_object(ctx, write_file):
    result = ingest_local_file(ctx, write_file(pattern(23, 1_000)), "a.bin", mode="file")
    ctx.hdfs.delete(ctx.paths.object_path(result.file.sha256))
    check = integrity.verify_file(ctx, result.file.id)
    assert not check.ok and check.missing and check.actual_sha256 is None


def test_verification_lists_each_missing_and_corrupt_chunk(ctx, write_file):
    size = ctx.settings.default_chunk_size
    result = ingest_local_file(ctx, write_file(pattern(24, size * 4)), "a.bin", mode="chunk")
    with session_scope(ctx.session_factory) as session:
        chunks = list(session.scalars(select(Chunk)))
    ctx.hdfs.delete(chunks[0].hdfs_path)
    ctx.hdfs.delete(chunks[1].hdfs_path)
    ctx.hdfs.create(chunks[1].hdfs_path, pattern(25, size))
    check = integrity.verify_file(ctx, result.file.id)
    assert not check.ok
    assert check.missing == [chunks[0].sha256]
    assert check.corrupt == [chunks[1].sha256]
    assert check.chunks_checked == 4


def test_verification_detects_a_tampered_durable_manifest(ctx, write_file):
    result = ingest_local_file(ctx, write_file(pattern(26, 10_000)), "a.bin", mode="chunk")
    with session_scope(ctx.session_factory) as session:
        path = session.scalars(select(FileManifest)).first().hdfs_path
    ctx.hdfs.delete(path)
    ctx.hdfs.create(path, b'{"chunks": []}')
    check = integrity.verify_file(ctx, result.file.id)
    assert not check.ok and "disagrees" in check.detail


def test_download_aborts_rather_than_serving_a_corrupt_chunk(ctx, write_file):
    size = ctx.settings.default_chunk_size
    result = ingest_local_file(ctx, write_file(pattern(27, size * 3)), "a.bin", mode="chunk")
    with session_scope(ctx.session_factory) as session:
        path = session.scalars(select(Chunk)).first().hdfs_path
    ctx.hdfs.delete(path)
    ctx.hdfs.create(path, pattern(28, size))
    with pytest.raises(IntegrityFailure):
        list(integrity.stream_verified(ctx, result.file.id))


def test_hdfs_outage_during_verify_is_an_error_not_a_failed_check(ctx, write_file):
    from app.core.errors import HdfsUnavailable

    result = ingest_local_file(ctx, write_file(pattern(29, 100)), "a.bin", mode="file")
    ctx.hdfs = FaultInjector(ctx.hdfs, method="open")
    with pytest.raises(HdfsUnavailable):
        integrity.verify_file(ctx, result.file.id)
    with session_scope(ctx.session_factory) as session:
        assert session.scalar(select(func.count()).select_from(IntegrityCheck)) == 0
```

```
# -- crash recovery -------------------------------------------------------------------------


def test_startup_marks_running_jobs_interrupted_and_clears_staging(ctx):
    with session_scope(ctx.session_factory) as session:
        session.add(DeduplicationJob(id="j" * 32, kind="optimize", status=JobStatus.RUNNING))
    ctx.hdfs.create(ctx.paths.staging_path("leftover"), b"partial")
    summary = lifecycle.recover_on_startup(ctx)
    assert summary["jobs_interrupted"] == 1 and summary["staging_removed"] == 1
    with session_scope(ctx.session_factory) as session:
        assert session.get(DeduplicationJob, "j" * 32).status == JobStatus.INTERRUPTED
    assert ctx.hdfs.list_dir(ctx.paths.staging) == []


def test_startup_survives_hdfs_being_down(ctx):
    ctx.hdfs = FaultInjector(ctx.hdfs, method="mkdirs")
    summary = lifecycle.recover_on_startup(ctx)
    assert summary["hdfs_available"] is False


def test_reconcile_finds_orphans_and_missing_without_deleting(ctx, write_file):
    keep = ingest_local_file(ctx, write_file(pattern(30, 100)), "a.bin", mode="file")
    lost = ingest_local_file(ctx, write_file(pattern(31, 100)), "b.bin", mode="file")
    ctx.hdfs.delete(ctx.paths.object_path(lost.file.sha256))
    orphan = ctx.paths.object_path("f" * 64)
    ctx.hdfs.create(orphan, b"orphan bytes")

    report = lifecycle.reconcile(ctx)
    assert report["orphan_count"] == 1 and report["missing_count"] == 1
    assert ctx.hdfs.status(orphan) is not None  # report-only by default

    applied = lifecycle.reconcile(ctx, apply=True)
    assert applied["orphans_quarantined"] == 1
    assert ctx.hdfs.status(orphan) is None
    assert ctx.hdfs.status(ctx.paths.recovery_orphan_path("f" * 64)) is not None
    assert ctx.hdfs.status(ctx.paths.object_path(keep.file.sha256)) is not None


def test_database_rejects_inconsistent_storage_pointers(ctx, write_file):
    from sqlalchemy.exc import IntegrityError

    result = ingest_local_file(ctx, write_file(pattern(32, 100)), "a.bin", mode="file")
    with pytest.raises(IntegrityError), session_scope(ctx.session_factory) as session:
        session.get(LogicalFile, result.file.id).object_id = None


def test_database_rejects_deleting_a_referenced_object(ctx, write_file):
    from sqlalchemy.exc import IntegrityError

    result = ingest_local_file(ctx, write_file(pattern(33, 100)), "a.bin", mode="file")
    with pytest.raises(IntegrityError), session_scope(ctx.session_factory) as session:
        session.delete(session.scalar(select(ContentObject).where(ContentObject.sha256 == result.file.sha256)))
```

backend/tests/test_optimizer_metrics.py 315 lines

```
"""Analysis, optimization, metrics definitions and replication validation."""

import pytest
from app.core.db import session_scope
from app.core.errors import ConflictError, ValidationFailure
from app.models.entities import (
    DeduplicationJob,
    JobStatus,
    OptimizationHistory,
    ReplicationOperation,
)
from app.services import analytics, integrity, lifecycle, optimizer, replication
from app.services.analytics import dedup_ratio, savings_percent
from app.services.ingest import ingest_local_file
from conftest import pattern
from sqlalchemy import select
from support.local_fs_adapter import FaultInjector


def job(ctx, job_id) -> DeduplicationJob:
    with session_scope(ctx.session_factory) as session:
        return session.get(DeduplicationJob, job_id)


def stats(ctx) -> dict:
    with session_scope(ctx.session_factory) as session:
        return analytics.storage_statistics(session)


def upload_duplicates(ctx, write_file, copies=3, size=20_000, seed=1):
    data = pattern(seed, size)
    return data, [ingest_local_file(ctx, write_file(data), f"copy{i}.bin", mode="raw") for i in range(copies)]


# -- formula definitions --------------------------------------------------------------------


def test_savings_formula():
```

```
    assert savings_percent(1000, 250) == 75.0
    assert savings_percent(1000, 1000) == 0.0


def test_ratio_formula():
    assert dedup_ratio(1000, 250) == 4.0


def test_metrics_are_undefined_rather_than_dividing_by_zero():
    assert savings_percent(0, 0) is None
    assert dedup_ratio(0, 0) is None


def test_empty_system_reports_undefined_metrics(ctx):
    current = stats(ctx)
    assert current["logical_files"] == 0
    assert current["savings_percent"] is None and current["dedup_ratio"] is None


def test_statistics_on_a_known_mix(ctx, write_file):
    one, two = pattern(1, 1000), pattern(2, 3000)
    ingest_local_file(ctx, write_file(one), "a", mode="file")
    ingest_local_file(ctx, write_file(one), "b", mode="file")
    ingest_local_file(ctx, write_file(two), "c", mode="raw")
    current = stats(ctx)
    assert current["logical_files"] == 3
    assert current["unique_files"] == 2
    assert current["duplicate_files"] == 1
    assert current["logical_bytes"] == 5000
    assert current["unique_content_bytes"] == 4000
    assert current["stored_content_bytes"] == 4000
    assert current["bytes_avoided"] == 1000
    assert current["savings_percent"] == 20.0
    assert current["dedup_ratio"] == 1.25


def test_measured_values_come_from_the_store_not_from_multiplication(ctx, write_file):
    ingest_local_file(ctx, write_file(pattern(3, 1000)), "a", mode="file")
    ctx.hdfs.set_replication(ctx.paths.object_path(stats_sha(ctx)), 2)
    with session_scope(ctx.session_factory) as session:
        full = analytics.full_statistics(ctx, session)
    assert full["measured"]["available"] is True
    assert full["measured"]["space_consumed"] == 2000  # what the store reports, despite the default of 3
    assert full["estimated"]["replication_adjusted_bytes"] == 3000


def stats_sha(ctx):
    from app.models.entities import ContentObject

    with session_scope(ctx.session_factory) as session:
        return session.scalars(select(ContentObject.sha256)).first()


def test_measured_values_report_an_outage_rather_than_a_guess(ctx):
    ctx.hdfs = FaultInjector(ctx.hdfs, method="content_summary")
    measured = analytics.measured_hdfs(ctx)
    assert measured["available"] is False and "Injected" in measured["error"]


# -- analysis ------------------------------------------------------------------------------------


@pytest.mark.parametrize("mode,chunk_size", [("file", None), ("chunk", 4096)])
def test_analysis_changes_nothing_and_predicts_the_real_outcome(ctx, write_file, mode, chunk_size):
    upload_duplicates(ctx, write_file)
    before = stats(ctx)
    before_paths = ctx.hdfs.content_summary(ctx.paths.root)

    analysis_id = ctx.jobs.submit("analyze", mode=mode, chunk_size=chunk_size)
    preview = job(ctx, analysis_id).result
    assert preview["estimated"] is True
    assert preview["files_eligible"] == 3
    assert stats(ctx) == before
    assert ctx.hdfs.content_summary(ctx.paths.root) == before_paths

    run_id = ctx.jobs.submit("optimize", mode=mode, chunk_size=chunk_size)
    assert job(ctx, run_id).status == JobStatus.SUCCEEDED
    assert stats(ctx)["stored_content_bytes"] == preview["predicted_stored_bytes"]


def test_chunk_analysis_counts_shared_chunks_across_files(ctx, write_file):
    size = ctx.settings.default_chunk_size
    shared = pattern(4, size * 2)
    ingest_local_file(ctx, write_file(shared + pattern(5, size)), "a", mode="raw")
    ingest_local_file(ctx, write_file(shared + pattern(6, size)), "b", mode="raw")
    preview = job(ctx, ctx.jobs.submit("analyze", mode="chunk", chunk_size=size)).result
    assert preview["chunks_examined"] == 6
    assert preview["chunks_new"] == 4
    assert preview["chunks_reused"] == 2


def test_optimization_requires_a_successful_current_preview(ctx, write_file):
    upload_duplicates(ctx, write_file)
```

```
    with pytest.raises(Exception, match="No analysis job"):
        optimizer.validate_analysis_for_optimize(ctx, "does-not-exist")
    analysis_id = ctx.jobs.submit("analyze", mode="file")
    assert optimizer.validate_analysis_for_optimize(ctx, analysis_id).id == analysis_id


def test_preview_goes_stale_when_files_change(ctx, write_file):
    upload_duplicates(ctx, write_file)
    analysis_id = ctx.jobs.submit("analyze", mode="file")
    ingest_local_file(ctx, write_file(pattern(7, 100)), "new", mode="raw")
    with pytest.raises(ConflictError, match="changed"):
        optimizer.validate_analysis_for_optimize(ctx, analysis_id)


# -- optimization ------------------------------------------------------------------------------


def test_file_level_optimization_keeps_one_copy_and_preserves_bytes(ctx, write_file):
    data, uploads = upload_duplicates(ctx, write_file)
    assert ctx.hdfs.content_summary(ctx.paths.raw).file_count == 3
    run_id = ctx.jobs.submit("optimize", mode="file")
    result = job(ctx, run_id).result
    assert result["files_processed"] == 3 and result["files_failed"] == 0
    assert result["bytes_reclaimed"] == 40_000
    assert ctx.hdfs.content_summary(ctx.paths.raw).file_count == 0
    assert ctx.hdfs.content_summary(ctx.paths.objects).file_count == 1
    for upload in uploads:
        assert b"".join(integrity.stream_verified(ctx, upload.file.id)) == data
        assert integrity.verify_file(ctx, upload.file.id).ok


def test_chunk_level_optimization_collects_replaced_objects(ctx, write_file):
    data = pattern(8, 20_000)
    uploads = [ingest_local_file(ctx, write_file(data), f"{i}", mode="file") for i in range(2)]
    run_id = ctx.jobs.submit("optimize", mode="chunk", chunk_size=4096)
    result = job(ctx, run_id).result
    assert result["garbage_collected"]["object_count"] == 1
    assert stats(ctx)["object_count"] == 0
    assert ctx.hdfs.content_summary(ctx.paths.objects).file_count == 0
    for upload in uploads:
        assert b"".join(integrity.stream_verified(ctx, upload.file.id)) == data


def test_optimization_is_recorded_with_verified_before_and_after(ctx, write_file):
    upload_duplicates(ctx, write_file)
    run_id = ctx.jobs.submit("optimize", mode="file")
    with session_scope(ctx.session_factory) as session:
        record = session.scalar(select(OptimizationHistory).where(OptimizationHistory.job_id == run_id))
        assert record.before["derived"]["stored_content_bytes"] == 60_000
        assert record.after["derived"]["stored_content_bytes"] == 20_000
        assert record.before["measured"]["length"] == 60_000
        assert record.after["measured"]["length"] == 20_000
    assert job(ctx, run_id).result["estimated"] is False


def test_corrupted_source_is_not_promoted_and_other_files_continue(ctx, write_file):
    good = ingest_local_file(ctx, write_file(pattern(9, 1000)), "good", mode="raw")
    bad = ingest_local_file(ctx, write_file(pattern(10, 1000)), "bad", mode="raw")
    ctx.hdfs.delete(bad.file.raw_path)
    ctx.hdfs.create(bad.file.raw_path, pattern(11, 1000))

    run = job(ctx, ctx.jobs.submit("optimize", mode="file"))
    assert run.status == JobStatus.PARTIAL
    assert run.result["files_processed"] == 1 and run.result["files_failed"] == 1
    assert run.result["errors"][0]["file_id"] == bad.file.id
    assert ctx.hdfs.status(bad.file.raw_path) is not None  # left exactly where it was
    assert integrity.verify_file(ctx, good.file.id).ok


def test_outage_mid_job_fails_visibly_and_a_resume_finishes_safely(ctx, write_file):
    data_a, uploads_a = upload_duplicates(ctx, write_file, copies=2, seed=20)
    data_b, uploads_b = upload_duplicates(ctx, write_file, copies=2, seed=21)
    real = ctx.hdfs
    # The first content converts, then HDFS "goes down" while reading the second content for verification.
    ctx.hdfs = FaultInjector(real, method="open", on_call=2)
    first = job(ctx, ctx.jobs.submit("optimize", mode="file"))
    assert first.status == JobStatus.FAILED and "Injected" in first.error
    ctx.hdfs = real
    assert stats(ctx)["files_by_mode"] == {"file": 2, "raw": 2}  # converted files stayed converted

    second = job(ctx, ctx.jobs.submit("optimize", mode="file", resumed_from_id=first.id))
    assert second.status == JobStatus.SUCCEEDED
    assert second.result["files_processed"] == 2  # only what was left
    assert stats(ctx)["stored_content_bytes"] == len(data_a) + len(data_b)
    for data, uploads in ((data_a, uploads_a), (data_b, uploads_b)):
        for upload in uploads:
            assert b"".join(integrity.stream_verified(ctx, upload.file.id)) == data


def test_crash_between_rename_and_commit_is_adopted_on_rerun(ctx, write_file):
    """The raw file reached its object path, but the process died before the metadata commit."""
    data, uploads = upload_duplicates(ctx, write_file, copies=1)
    file = uploads[0].file
```

```
    target = ctx.paths.object_path(file.sha256)
    ctx.hdfs.mkdirs(target.rsplit("/", 1)[0])
    assert ctx.hdfs.rename(file.raw_path, target)

    run = job(ctx, ctx.jobs.submit("optimize", mode="file"))
    assert run.status == JobStatus.SUCCEEDED
    assert b"".join(integrity.stream_verified(ctx, file.id)) == data


def test_failure_removing_old_copy_after_commit_leaves_a_detectable_orphan(ctx, write_file):
    from app.core.errors import HdfsError

    data, uploads = upload_duplicates(ctx, write_file, copies=2)
    real = ctx.hdfs
    ctx.hdfs = FaultInjector(real, method="delete", error=HdfsError("delete refused"))
    run = job(ctx, ctx.jobs.submit("optimize", mode="file"))
    ctx.hdfs = real
    assert run.status == JobStatus.SUCCEEDED, run.result["errors"]  # both files really did convert
    assert len(run.result["orphans_left"]) == 1
    for upload in uploads:
        assert b"".join(integrity.stream_verified(ctx, upload.file.id)) == data
    report = lifecycle.reconcile(ctx, apply=True)
    assert report["orphan_count"] == 1 and report["orphans_quarantined"] == 1
    assert ctx.hdfs.content_summary(ctx.paths.raw).file_count == 0


def test_rerunning_a_finished_optimization_is_a_no_op(ctx, write_file):
    upload_duplicates(ctx, write_file)
    ctx.jobs.submit("optimize", mode="file")
    again = job(ctx, ctx.jobs.submit("optimize", mode="file"))
    assert again.result["files_processed"] == 0
    assert again.result["bytes_reclaimed"] == 0


# -- replication ------------------------------------------------------------------------------


def test_replication_above_live_datanodes_is_rejected(ctx):
    with pytest.raises(ValidationFailure, match="cannot be satisfied"):
        replication.request_change(ctx, 4)


def test_replication_three_is_rejected_when_only_two_datanodes_are_live(ctx):
    ctx.monitor.live = 2
    with pytest.raises(ValidationFailure):
        replication.request_change(ctx, 3)


def test_lowering_replication_requires_acknowledgement(ctx):
    with pytest.raises(ConflictError, match="reduces fault tolerance"):
        replication.request_change(ctx, 1)


def test_replication_change_applies_and_is_verified(ctx, write_file):
    result = ingest_local_file(ctx, write_file(pattern(12, 1000)), "a", mode="file")
    change = replication.request_change(ctx, 2, acknowledge_reduced_protection=True)
    path = ctx.paths.object_path(result.file.sha256)
    assert ctx.hdfs.status(path).replication == 2
    with session_scope(ctx.session_factory) as session:
        operation = session.get(ReplicationOperation, change["operation_id"])
        assert operation.status == "verified"
        assert (operation.space_consumed_before, operation.space_consumed_after) == (3000, 2000)


def test_new_writes_use_the_target_replication(ctx, write_file):
    replication.request_change(ctx, 2, acknowledge_reduced_protection=True)
    result = ingest_local_file(ctx, write_file(pattern(13, 100)), "a", mode="file")
    assert ctx.hdfs.status(ctx.paths.object_path(result.file.sha256)).replication == 2


def test_replication_cannot_change_while_namenode_is_down(ctx):
    from app.core.errors import HdfsUnavailable

    ctx.monitor.reachable = False
    with pytest.raises(HdfsUnavailable):
        replication.request_change(ctx, 2, acknowledge_reduced_protection=True)


def test_convergence_timeout_scales_with_the_namespace():
    assert replication.convergence_timeout(10, 3) == replication.MINIMUM_VERIFY_SECONDS
    # 274 blocks at 6 blocks per 3-second cycle needs 137 s at best; allow for that with a margin.
    assert replication.convergence_timeout(274, 3) > 137 * 2
    assert replication.convergence_timeout(10_000, 3) > replication.convergence_timeout(274, 3)


def test_deduplication_never_changes_replication(ctx, write_file):
    _, uploads = upload_duplicates(ctx, write_file)
    ctx.jobs.submit("optimize", mode="file")
    status = ctx.hdfs.status(ctx.paths.object_path(uploads[0].file.sha256))
    assert status.replication == ctx.hdfs.default_replication()
```

frontend/playwright.config.ts 41 lines

```ts
import { defineConfig } from "@playwright/test";
import path from "node:path";
import os from "node:os";

const stateDir = path.join(os.tmpdir(), "hdfs-optimizer-e2e");

// Browser tests drive the real frontend against a second backend instance with its own database and its
// own HDFS directory, so they exercise the real cluster without touching managed data.
export const E2E_BACKEND = "http://127.0.0.1:8199";

export default defineConfig({
  testDir: "./e2e",
  timeout: 120_000,
  workers: 1,
  fullyParallel: false,
  reporter: [["list"]],
  globalTeardown: "./e2e/teardown.ts",
  use: {
    baseURL: "http://localhost:3100",
    channel: "chrome",
    headless: true,
    viewport: { width: 1440, height: 900 },
    acceptDownloads: true,
    trace: "off",
  },
  webServer: [
    {
      command: `rm -rf ${stateDir} && mkdir -p ${stateDir} && cd ../backend && exec .venv/bin/uvicorn app.main:app --host 127.0.0.1
--port 8199`,
      url: `${E2E_BACKEND}/api/v1/health`,
      reuseExistingServer: false,
      env: {
        HDFSOPT_ROOT_PATH: "/hdfs-optimizer-e2e",
        HDFSOPT_DATABASE_PATH: path.join(stateDir, "metadata.db"),
        HDFSOPT_SPOOL_DIR: path.join(stateDir, "spool"),
        HDFSOPT_CORS_ORIGINS: "http://localhost:3100",
      },
    },
    { command: "npm run start", url: "http://localhost:3100", reuseExistingServer: true },
  ],
});
```

frontend/e2e/app.spec.ts 181 lines

```ts
import { createHash } from "node:crypto";
import { readFileSync } from "node:fs";
import path from "node:path";

import { expect, test, type Page } from "@playwright/test";

const DATASET = path.resolve(__dirname, "../../datasets/raw/originals");
const ALICE = path.join(DATASET, "pg11.txt");
const aliceHash = createHash("sha256").update(readFileSync(ALICE)).digest("hex");

const PAGES = [
  ["/", "Dashboard"],
  ["/files", "Files"],
  ["/duplicates", "Duplicates"],
  ["/optimize", "Optimization center"],
  ["/replication", "Replication manager"],
  ["/cluster", "Cluster monitoring"],
  ["/analytics", "Analytics and reports"],
  ["/activity", "Activity and audit"],
  ["/lab", "Dataset and experiment lab"],
  ["/settings", "Settings"],
] as const;

test.beforeEach(async ({ context }) => {
  // The production build talks to port 8100. Send those calls to the isolated test backend.
  await context.route("http://localhost:8100/**", (route) => route.continue({ url: route.request().url().replace("localhost:8100",
"127.0.0.1:8199") }));
});

function failOnConsoleErrors(page: Page) {
  const errors: string[] = [];
  page.on("console", (message) => {
    if (message.type() === "error") errors.push(message.text());
  });
  page.on("pageerror", (error) => errors.push(error.message));
  return errors;
}

async function upload(page: Page, mode: "Whole-file dedup" | "Chunk-level dedup" | "No dedup (baseline)", files: { name: string;
path: string }[]) {
  await page.goto("/files");
  await page.getByRole("combobox", { name: "Storage mode for new uploads" }).click();
  await page.getByRole("option", { name: mode }).click();
  await page.getByTestId("file-input").setInputFiles(files.map((f) => ({ name: f.name, mimeType: "text/plain", buffer:
readFileSync(f.path) })));
}

test.describe.serial("HDFS Optimizer", () => {
  test("every page renders without console errors", async ({ page }) => {
```

```
    const errors = failOnConsoleErrors(page);
    for (const [url, heading] of PAGES) {
      await page.goto(url);
      await expect(page.getByRole("heading", { level: 1, name: heading })).toBeVisible();
      await page.waitForLoadState("networkidle");
    }
    expect(errors).toEqual([]);
  });

  test("an empty system shows N/A, not invented values", async ({ page }) => {
    await page.goto("/");
    await expect(page.getByText("No files are managed yet")).toBeVisible();
    await expect(page.getByText("N/A").first()).toBeVisible();
  });

  test("the cluster page shows real DataNodes", async ({ page }) => {
    await page.goto("/cluster");
    await expect(page.getByRole("cell", { name: "datanode1" })).toBeVisible();
    await expect(page.getByRole("cell", { name: "datanode3" })).toBeVisible();
    await expect(page.getByText("3.5.0").first()).toBeVisible();
  });

  test("uploading the same authentic file twice stores it once", async ({ page }) => {
    await upload(page, "Whole-file dedup", [{ name: "alice.txt", path: ALICE }]);
    await expect(page.getByText("New content stored")).toBeVisible();
    await page.getByTestId("file-input").setInputFiles({ name: "same-book-other-name.txt", mimeType: "text/plain", buffer:
readFileSync(ALICE) });
    await expect(page.getByText("Duplicate: no new data stored")).toBeVisible();
    await expect(page.getByRole("cell", { name: /same-book-other-name\.txt/ })).toBeVisible();
    await expect(page.getByText("2 references").first()).toBeVisible();
  });

  test("download restores identical bytes, and verification passes", async ({ page }) => {
    await page.goto("/files");
    await page.getByRole("button", { name: "same-book-other-name.txt" }).click();
    const downloadPromise = page.waitForEvent("download");
    await page.getByRole("link", { name: "Download" }).click();
    const download = await downloadPromise;
    expect(download.suggestedFilename()).toBe("same-book-other-name.txt");
    const restored = readFileSync(await download.path());
    expect(createHash("sha256").update(restored).digest("hex")).toBe(aliceHash);

    await page.getByRole("button", { name: "Verify integrity" }).click();
    await expect(page.getByText("SHA-256 matches")).toBeVisible();
  });

  test("the duplicates page explains the relationship", async ({ page }) => {
    await page.goto("/duplicates");
    await expect(page.getByRole("img", { name: /2 logical files share SHA-256/ })).toBeVisible();
    await expect(page.getByText("Deduplicated").first()).toBeVisible();
  });

  test("delete keeps shared content and restore brings the file back", async ({ page }) => {
    await page.goto("/files");
    await page.getByRole("button", { name: "alice.txt" }).click();
    await page.getByRole("button", { name: "Delete", exact: true }).click();
    await expect(page.getByRole("button", { name: "alice.txt" })).toBeHidden();

    await page.getByRole("combobox", { name: "Active or deleted files" }).click();
    await page.getByRole("option", { name: "Recently deleted" }).click();
    await page.getByRole("button", { name: "alice.txt" }).click();
    await page.getByRole("button", { name: "Restore" }).click();
    await expect(page.getByText("File restored.")).toBeVisible();
  });

  test("optimization needs a preview, then reclaims space and keeps bytes intact", async ({ page }) => {
    const sherlock = path.join(DATASET, "pg1661.txt");
    await upload(page, "No dedup (baseline)", [
      { name: "sherlock-a.txt", path: sherlock },
      { name: "sherlock-b.txt", path: sherlock },
      { name: "sherlock-c.txt", path: sherlock },
    ]);
    await expect(page.getByText("Stored without deduplication")).toHaveCount(3, { timeout: 30_000 });

    await page.goto("/optimize");
    await expect(page.getByRole("button", { name: "Start optimization" })).toBeHidden();
    await page.getByRole("button", { name: "Run dry-run analysis" }).click();
    await expect(page.getByText("Nothing has changed yet.")).toBeVisible({ timeout: 30_000 });
    await expect(page.getByText("Expected reduction")).toBeVisible();

    await page.getByRole("button", { name: "Start optimization" }).click();
    await page.getByRole("dialog").getByRole("button", { name: "Start optimization" }).click();
    await expect(page.getByText("Results below are observed, not predicted.")).toBeVisible({ timeout: 60_000 });
    await expect(page.getByText("Stored content reclaimed")).toBeVisible();

    await page.getByRole("button", { name: "Verify all restored bytes" }).click();
    await expect(page.getByText(/5 of 5 files match their SHA-256/)).toBeVisible({ timeout: 60_000 });
  });

  test("chunk mode shows shared chunks in the chunk map", async ({ page }) => {
    const workload = path.resolve(__dirname, "../../datasets/raw/workload");
    await upload(page, "Chunk-level dedup", [
      { name: "pride.txt", path: path.join(DATASET, "pg1342.txt") },
```

```
      { name: "pride-truncated.txt", path: path.join(workload, "pg1342.truncate_90.txt") },
    ]);
    await expect(page.getByText(/chunks reused/)).toHaveCount(2, { timeout: 30_000 });
    await page.goto("/duplicates");
    await page.getByRole("tab", { name: "Shared chunks" }).click();
    await expect(page.getByText("pride-truncated.txt")).toBeVisible();
    await expect(page.getByRole("img", { name: /chunks shown are shared/ }).first()).toBeVisible();
  });

  test("replication above live DataNodes is not offered, and lowering asks for confirmation", async ({ page }) => {
    await page.goto("/replication");
    await expect(page.getByRole("button", { name: "Increase replication factor" })).toBeDisabled();
    await page.getByRole("button", { name: "Decrease replication factor" }).click();
    await page.getByRole("button", { name: "Apply factor 2" }).click();
    await expect(page.getByRole("dialog")).toContainText("reduces fault tolerance");
    await page.getByRole("dialog").getByRole("button", { name: "Cancel" }).click();
  });

  test("the audit trail records what happened", async ({ page }) => {
    await page.goto("/activity");
    await expect(page.getByRole("cell", { name: "file.uploaded" }).first()).toBeVisible();
    await expect(page.getByRole("cell", { name: "dedup.duplicate_detected" }).first()).toBeVisible();
    await expect(page.getByRole("cell", { name: "optimize.completed" }).first()).toBeVisible();
  });

  test("diagnostics run real checks against the cluster", async ({ page }) => {
    await page.goto("/settings");
    await page.getByRole("button", { name: "Run diagnostics" }).click();
    await expect(page.getByText(/Wrote, read back and deleted/)).toBeVisible();
    await expect(page.getByText(/SQLite integrity_check returned ok/)).toBeVisible();
  });

  test("mobile layout has no horizontal page scroll", async ({ page }) => {
    await page.setViewportSize({ width: 390, height: 844 });
    for (const [url] of PAGES) {
      await page.goto(url);
      await page.waitForLoadState("networkidle");
      const overflow = await page.evaluate(() => document.documentElement.scrollWidth - document.documentElement.clientWidth);
      expect(overflow, `${url} overflows horizontally`).toBeLessThanOrEqual(0);
    }
  });
});
```

frontend/e2e/teardown.ts 5 lines

```
/** Remove the HDFS directory used by the browser tests. */
export default async function teardown() {
  await fetch("http://127.0.0.1:9870/webhdfs/v1/hdfs-optimizer-e2e?op=DELETE&recursive=true&user.name=hadoop", { method:
"DELETE" });
}
```

frontend/capture/capture.spec.ts 315 lines

```
/**
 * Captures the execution evidence by really using the running application, from an empty system.
 * Nothing is mocked or edited: every image shows the live page at that moment.
 * Run through scripts/capture_evidence.sh, which resets demo data first.
 */
import { execFileSync } from "node:child_process";
import { readFileSync, mkdirSync, writeFileSync, readdirSync } from "node:fs";
import path from "node:path";

import { expect, test, type Page } from "@playwright/test";

const ROOT = path.resolve(__dirname, "../..");
const SHOTS = path.join(ROOT, "screenshots");
const EVIDENCE = path.join(ROOT, "evidence");
const DATASET = path.join(ROOT, "datasets/raw");
const API = "http://localhost:8100/api/v1";

mkdirSync(SHOTS, { recursive: true });

const compose = (...args: string[]) => execFileSync("docker", ["compose", ...args], { cwd: ROOT, stdio: "pipe" });

async function shot(page: Page, name: string, fullPage = true) {
  await page.waitForLoadState("networkidle").catch(() => {});
  await page.evaluate(() => window.scrollTo(0, 0));
  await page.waitForTimeout(350); // let charts and transitions finish
  await page.screenshot({ path: path.join(SHOTS, `${name}.png`), fullPage });
}

async function jmx(): Promise<Record<string, number> | null> {
  try {
    const response = await fetch("http://127.0.0.1:9870/jmx?qry=Hadoop:service=NameNode,name=FSNamesystem");
    return (await response.json()).beans[0];
  } catch {
    return null;
  }
}

async function waitUntil(check: () => Promise<boolean>, seconds: number, what: string) {
  const deadline = Date.now() + seconds * 1000;
  while (Date.now() < deadline) {
    if (await check()) return;
    await new Promise((resolve) => setTimeout(resolve, 2000));
```

```
  }
  throw new Error(`Timed out waiting for ${what}`);
}

test.describe.serial("evidence", () => {
  test.use({ viewport: { width: 1440, height: 900 }, deviceScaleFactor: 2, colorScheme: "light" });

  test("empty system", async ({ page }) => {
    await page.goto("/");
    await expect(page.getByText("No files are managed yet")).toBeVisible();
    await shot(page, "00_dashboard_empty");
  });

  test("dataset provenance and ingestion", async ({ page }) => {
    await page.goto("/lab");
    await expect(page.getByText("Authentic original").first()).toBeVisible();
    await shot(page, "17_dataset_provenance");

    // Ingest all plain-text originals with two labelled workload copies each, without deduplication.
    const rows = page.locator("tbody tr").filter({ hasText: "Authentic original" }).filter({ has: page.locator('td:has-
text("txt")') });
    const count = await rows.count();
    expect(count).toBe(9);
    for (let i = 0; i < count; i += 1) await rows.nth(i).getByRole("checkbox").check();
    await page.getByRole("button", { name: /Ingest 9 into HDFS/ }).click();
    await page.getByLabel("Exact workload copies per file").fill("2");
    await page.screenshot({ path: path.join(SHOTS, "12a_dataset_ingest_dialog.png") });
    await page.getByRole("dialog").getByRole("button", { name: "Ingest" }).click();
    await expect(page.getByText("dataset ingest finished.")).toBeVisible({ timeout: 120_000 });
  });

  test("file upload shows real outcomes", async ({ page }) => {
    await page.goto("/files");
    await page.getByRole("combobox", { name: "Storage mode for new uploads" }).click();
    await page.getByRole("option", { name: "Whole-file dedup" }).click();
    const html = readFileSync(path.join(DATASET, "originals/pg1342.html"));
    await page.getByTestId("file-input").setInputFiles({ name: "pride-and-prejudice.html", mimeType: "text/html", buffer: html });
    await expect(page.getByText("New content stored")).toBeVisible({ timeout: 60_000 });
    await page.getByTestId("file-input").setInputFiles({ name: "same-book-renamed.html", mimeType: "text/html", buffer: html });
    await expect(page.getByText("Duplicate: no new data stored")).toBeVisible({ timeout: 60_000 });
    await shot(page, "02_file_upload", false);
    await shot(page, "03_file_explorer");
  });

  test("file details, block locations, verification and restored checksum", async ({ page }) => {
    await page.goto("/files");
    await page.getByRole("button", { name: "same-book-renamed.html" }).click();
    await page.getByRole("button", { name: "Verify integrity" }).click();
    await expect(page.getByText("SHA-256 matches")).toBeVisible({ timeout: 30_000 });
    await page.waitForTimeout(500);
    await page.screenshot({ path: path.join(SHOTS, "16a_file_details_verified.png") });

    const downloadPromise = page.waitForEvent("download");
    await page.getByRole("link", { name: "Download" }).click();
    const download = await downloadPromise;
    const target = path.join(EVIDENCE, "restored", "same-book-renamed.html");
    await download.saveAs(target);
  });

  test("duplicate analysis", async ({ page }) => {
    await page.goto("/duplicates");
    await expect(page.getByRole("img", { name: /logical files share SHA-256/ }).first()).toBeVisible();
    await shot(page, "04_duplicate_analysis");
  });

  test("dry run, then optimization, then before and after", async ({ page }) => {
    await page.goto("/optimize");
    await page.getByRole("button", { name: "Run dry-run analysis" }).click();
    await expect(page.getByText("Nothing has changed yet.")).toBeVisible({ timeout: 60_000 });
    await shot(page, "05_optimization_preview");

    await page.getByRole("button", { name: "Start optimization" }).click();
    await page.screenshot({ path: path.join(SHOTS, "05a_optimization_confirm.png") });
    await page.getByRole("dialog").getByRole("button", { name: "Start optimization" }).click();
    await expect(page.getByText("Results below are observed, not predicted.")).toBeVisible({ timeout: 120_000 });
    await page.getByRole("button", { name: "Verify all restored bytes" }).click();
    await expect(page.getByText(/files match their SHA-256/)).toBeVisible({ timeout: 120_000 });
    await shot(page, "06_optimization_complete");
  });

  test("chunk-level sharing on documented partial overlap", async ({ page }) => {
    await page.goto("/files");
    await page.getByRole("combobox", { name: "Storage mode for new uploads" }).click();
    await page.getByRole("option", { name: "Chunk-level dedup" }).click();
    await page.getByRole("combobox", { name: "Chunk size" }).click();
    await page.getByRole("option", { name: "64 KiB" }).click();
    const files = ["originals/pg1661.html", "originals/pg11.epub"];
    for (const name of ["truncate_90", "replace_mid_byte", "prefix_newline"]) files.push(`workload/pg1342.${name}.txt`);
    await page.getByTestId("file-input").setInputFiles(
      files.map((file) => ({ name: path.basename(file), mimeType: "application/octet-stream", buffer:
readFileSync(path.join(DATASET, file)) })),
    );
    await expect(page.getByText(/written/)).toHaveCount(5, { timeout: 120_000 });
```

```
    // Convert everything to chunks at the same size, so the original book and its variants can share chunks.
    await page.goto("/optimize");
    await page.getByRole("button", { name: "Chunk-level" }).click();
    await page.getByRole("combobox", { name: "Chunk size" }).click();
    await page.getByRole("option", { name: "64 KiB" }).click();
    await page.getByRole("button", { name: "Run dry-run analysis" }).click();
    await expect(page.getByText("Nothing has changed yet.")).toBeVisible({ timeout: 120_000 });
    await shot(page, "05b_chunk_preview");
    await page.getByRole("button", { name: "Start optimization" }).click();
    await page.getByRole("dialog").getByRole("button", { name: "Start optimization" }).click();
    await expect(page.getByText("Results below are observed, not predicted.")).toBeVisible({ timeout: 300_000 });
    await shot(page, "06b_chunk_complete");

    await page.goto("/duplicates");
    await page.getByRole("tab", { name: "Shared chunks" }).click();
    await expect(page.getByText("Chunk map")).toBeVisible();
    await shot(page, "04b_shared_chunks");
  });

  test("replication change, for real, down and back up", async ({ page }) => {
    test.setTimeout(900_000);
    await page.goto("/replication");
    await expect(page.getByRole("button", { name: "Increase replication factor" })).toBeDisabled();
    await page.getByRole("button", { name: "Decrease replication factor" }).click();
    await page.getByRole("button", { name: "Apply factor 2" }).click();
    await expect(page.getByRole("dialog")).toContainText("reduces fault tolerance");
    await page.screenshot({ path: path.join(SHOTS, "08a_replication_confirm.png") });
    await page.getByRole("dialog").getByRole("button", { name: "Lower to 2" }).click();
    await expect(page.getByRole("cell", { name: "3 to 2" })).toBeVisible({ timeout: 30_000 });
    await expect(page.getByText("Verified").first()).toBeVisible({ timeout: 300_000 });
    await page.reload();
    await expect(page.getByText("Factor 2").first()).toBeVisible();
    await shot(page, "08b_replication_factor_2");

    await page.getByRole("button", { name: "Increase replication factor" }).click();
    await page.getByRole("button", { name: "Apply factor 3" }).click();
    await expect(page.getByRole("cell", { name: "2 to 3" })).toBeVisible({ timeout: 30_000 });
    await waitUntil(
      async () => {
        const data = await (await fetch(`${API}/cluster/replication`)).json();
        return data.operations[0].status === "verified";
      },
      700,
      "replication 3",
    );
    // NameNode JMX values are cached for about ten seconds, so wait for the health summary to catch up.
    await waitUntil(async () => (await (await fetch(`${API}/cluster/status`)).json()).health === "healthy", 60, "healthy status");
    await page.reload();
    await expect(page.getByText("Factor 3").first()).toBeVisible();
    await shot(page, "08_replication_management");
  });

  test("experiments run from the lab", async ({ page }) => {
    await page.goto("/lab");
    for (const title of ["Fixed-size chunking boundary shift", "Logical deletion with shared content"]) {
      const row = page.locator("li").filter({ hasText: title });
      await row.getByRole("button", { name: "Run" }).click();
      await expect(row.getByText("succeeded")).toBeVisible({ timeout: 120_000 });
      await page.waitForTimeout(800);
    }
    await expect(page.locator("[data-sonner-toast]")).toHaveCount(0, { timeout: 15_000 });
    await shot(page, "17a_experiment_lab");
  });

  test("final state pages", async ({ page }) => {
    await page.goto("/");
    await expect(page.getByText("Storage reduction")).toBeVisible();
    await page.waitForTimeout(800);
    await shot(page, "01_dashboard");
    await page.goto("/analytics");
    await page.waitForTimeout(800);
    await shot(page, "07_storage_analytics");
    await page.goto("/cluster");
    await expect(page.getByRole("cell", { name: "datanode1" })).toBeVisible();
    await shot(page, "09_cluster_status");
    await page.goto("/activity");
    await expect(page.getByRole("cell", { name: "optimize.completed" }).first()).toBeVisible();
    await shot(page, "09a_activity_audit", false);
    await page.goto("/settings");
    await page.getByRole("button", { name: "Run diagnostics" }).click();
    await expect(page.getByText(/Wrote, read back and deleted/)).toBeVisible();
    await page.getByRole("button", { name: "Check" }).click();
    await expect(page.getByText(/files in HDFS,/)).toBeVisible();
    await shot(page, "09b_settings_diagnostics");
  });

  test("dark mode and phone width", async ({ browser }) => {
    const dark = await browser.newContext({ baseURL: "http://localhost:3100", viewport: { width: 1440, height: 900 },
deviceScaleFactor: 2, colorScheme: "dark" });
    const darkPage = await dark.newPage();
    await darkPage.goto("/");
    await expect(darkPage.getByText("Storage reduction")).toBeVisible();
    await darkPage.waitForTimeout(800);
```

```
    await shot(darkPage, "01a_dashboard_dark");
    await darkPage.goto("/duplicates");
    await darkPage.waitForTimeout(500);
    await shot(darkPage, "04a_duplicates_dark", false);
    await dark.close();
    const phone = await browser.newContext({ baseURL: "http://localhost:3100", viewport: { width: 390, height: 844 },
deviceScaleFactor: 2, colorScheme: "light" });
    const phonePage = await phone.newPage();
    await phonePage.goto("/");
    await expect(phonePage.getByText("Storage reduction")).toBeVisible();
    await phonePage.waitForTimeout(800);
    await shot(phonePage, "01b_dashboard_phone", false);
    await phone.close();
  });

  test("NameNode, DataNode and API documentation pages", async ({ page }) => {
    await page.goto("http://127.0.0.1:9870/dfshealth.html#tab-overview");
    await expect(page.getByText("Overview")).toBeVisible();
    await page.waitForTimeout(1500);
    await shot(page, "10_namenode_interface");
    await page.goto("http://127.0.0.1:9870/dfshealth.html#tab-datanode");
    await page.waitForTimeout(1500);
    await shot(page, "11_datanode_status");
    await page.goto("http://127.0.0.1:19864/datanode.html");
    await page.waitForTimeout(1500);
    await shot(page, "11a_datanode_interface", false);
    await page.goto("http://localhost:8100/docs");
    await page.waitForTimeout(2500);
    await shot(page, "12b_api_documentation", false);
  });

  test("a real DataNode outage is shown as it happens", async ({ page }) => {
    test.setTimeout(420_000);
    compose("stop", "datanode3");
    try {
      await waitUntil(async () => (await jmx())?.NumDeadDataNodes === 1, 240, "the DataNode to be declared dead");
      // The NameNode removes a dead node's replicas on its own cycle, slightly after flagging it as dead.
      await waitUntil(async () => (await (await fetch(`${API}/cluster/replication`)).json()).fsck.under_replicated_blocks > 0, 120,
"fsck to report under-replication");
      await waitUntil(async () => ((await jmx())?.UnderReplicatedBlocks ?? 0) > 0, 60, "cached JMX values to catch up");
      await page.goto("/cluster");
      await expect(page.getByText("Degraded").first()).toBeVisible({ timeout: 30_000 });
      await shot(page, "09c_cluster_datanode_dead");
      await page.goto("/replication");
      await expect(page.getByText(/under-replicated/).first()).toBeVisible();
      await shot(page, "09d_replication_datanode_dead");
    } finally {
      compose("start", "datanode3");
    }
    await waitUntil(async () => {
      const bean = await jmx();
      if (bean?.NumLiveDataNodes !== 3) return false;
      return (await (await fetch(`${API}/cluster/replication`)).json()).fsck.under_replicated_blocks === 0;
    }, 240, "the DataNode to recover");
  });

  test("a real NameNode outage is shown as it happens", async ({ page }) => {
    test.setTimeout(420_000);
    compose("stop", "namenode");
    try {
      await waitUntil(async () => (await jmx()) === null, 60, "the NameNode to stop");
      await page.goto("/cluster");
      await expect(page.getByText("The NameNode is not responding")).toBeVisible({ timeout: 30_000 });
      await shot(page, "09e_cluster_namenode_down", false);
      await page.goto("/files");
      await page.getByTestId("file-input").setInputFiles({ name: "during-outage.txt", mimeType: "text/plain", buffer:
readFileSync(path.join(DATASET, "originals/pg11.txt")) });
      await expect(page.getByText(/Cannot reach the NameNode/)).toBeVisible({ timeout: 30_000 });
      await shot(page, "09f_upload_during_outage", false);
    } finally {
      compose("start", "namenode");
    }
    await waitUntil(async () => (await jmx())?.NumLiveDataNodes === 3, 240, "the cluster to recover");
    await waitUntil(async () => (await (await fetch(`${API}/system/diagnostics`)).json()).ok === true, 120, "writes to resume");
  });

  test("terminal evidence is rendered from captured output", async ({ page }) => {
    // Output files are written by scripts/capture_terminal.sh. They are displayed unedited.
    for (const file of readdirSync(EVIDENCE).filter((f) => f.endsWith(".txt")).sort()) {
      const text = readFileSync(path.join(EVIDENCE, file), "utf8");
      const lines = text.split("\n");
      const shown = lines.length > 70 ? [...lines.slice(0, 66), `... (${lines.length - 66} more lines in evidence/${file})`] :
lines;
      const escaped = shown.join("\n").replace(/&/g, "&amp;").replace(/</g, "&lt;");
      const html = `<!doctype html><meta charset="utf-8"><style>body{margin:0;background:#141413;color:#e8e7e3;font:13px/1.45
Menlo,Consolas,monospace;padding:18px 22px}pre{margin:0;white-space:pre-wrap;word-break:break-word}</style><pre>${escaped}</pre>`;
      const temp = path.join(EVIDENCE, ".render.html");
      writeFileSync(temp, html);
      await page.setViewportSize({ width: 1180, height: 200 });
      await page.goto(`file://${temp}`);
      await page.screenshot({ path: path.join(SHOTS, `terminal_${file.replace(".txt", "")}.png`), fullPage: true });
    }
```

```
  });
});
```

frontend/capture/playwright.capture.config.ts 14 lines

```ts
import { defineConfig } from "@playwright/test";

// Used only by scripts/capture_evidence.sh, against the running application.
export default defineConfig({
  testDir: ".",
  testMatch: "capture.spec.ts",
  timeout: 360_000,
  workers: 1,
  fullyParallel: false,
  reporter: [["list"]],
  outputDir: "../test-results/capture",
  use: { baseURL: "http://localhost:3100", channel: "chrome", headless: true, acceptDownloads: true },
});
```

11.8 Reproduction and tooling scripts

scripts/build-app-images.sh 28 lines

```sh
#!/usr/bin/env bash
# Builds the application container images, aborting if the host's free disk falls below a safety margin.
# Image builds grow the VM's disk on the host; on a nearly full machine that can make every write fail.
set -uo pipefail
source "$(dirname "$0")/env.sh"
cd "$PROJECT_ROOT"
MINIMUM_FREE_KB=$(( ${MINIMUM_FREE_GB:-2} * 1024 * 1024 ))
free_kb() { df -k "$PROJECT_ROOT" | awk 'NR==2 {print $4}'; }

if [ "$(free_kb)" -lt "$MINIMUM_FREE_KB" ]; then
  echo "Refusing to build: less than ${MINIMUM_FREE_GB:-2} GB free on the host."
  exit 2
fi
docker compose --profile app build backend frontend &
BUILD=$!
while kill -0 "$BUILD" 2>/dev/null; do
  if [ "$(free_kb)" -lt "$MINIMUM_FREE_KB" ]; then
    echo "ABORTED: host free disk fell below the safety margin. Cleaning up the build."
    kill "$BUILD" 2>/dev/null
    wait "$BUILD" 2>/dev/null
    docker image prune -f >/dev/null
    colima ssh -p "$COLIMA_PROFILE" -- sudo fstrim -a >/dev/null 2>&1 || true
    exit 3
  fi
  sleep 2
done
wait "$BUILD"
```

scripts/capture_evidence.sh 17 lines

```sh
#!/usr/bin/env bash
# Rebuilds screenshots/ and evidence/ from a live system. Run with the backend and frontend running on an
# empty, freshly reset system (see scripts/reset-demo.sh). This stops and restarts HDFS containers on purpose.
set -euo pipefail
source "$(dirname "$0")/env.sh"
cd "$PROJECT_ROOT"
mkdir -p evidence/restored screenshots

# 1. Use the application through a real browser: this fills the system and takes the page screenshots.
(cd frontend && npx playwright test -c capture/playwright.capture.config.ts --grep-invert "terminal evidence")
# 2. Record terminal output from the state the browser session left behind.
scripts/capture_terminal.sh
# 3. Render that output to images, unedited.
(cd frontend && npx playwright test -c capture/playwright.capture.config.ts --grep "terminal evidence")
rm -f evidence/.render.html
echo "Screenshots: screenshots/  Terminal logs: evidence/"
```

scripts/capture_terminal.sh 71 lines

```sh
#!/usr/bin/env bash
# Captures real command output into evidence/*.txt, with each command shown above its output.
# Run while the application and HDFS are running, after the UI capture has populated the system.
set -uo pipefail
source "$(dirname "$0")/env.sh"
cd "$PROJECT_ROOT"
mkdir -p evidence

run() {
  # run <file> <command...>: append the command line and its unedited output.
  local file="evidence/$1.txt"; shift
  { printf '$ %s\n' "$*"; "$@" 2>&1; printf '[exit code %s]\n\n' "$?"; } >> "$file"
}
start() { : > "evidence/$1.txt"; printf '# Captured %s\n\n' "$(date -u +%Y-%m-%dT%H:%M:%SZ)" >> "evidence/$1.txt"; }

start 01_docker_cluster
run 01_docker_cluster docker compose ps --format 'table {{.Service}}\t{{.Image}}\t{{.State}}\t{{.Health}}\t{{.Ports}}'
run 01_docker_cluster docker compose exec -T namenode hadoop version
run 01_docker_cluster docker info --format '{{.OperatingSystem}} {{.Architecture}} CPUs={{.NCPU}} MemBytes={{.MemTotal}}'

start 02_hdfs_report
run 02_hdfs_report docker compose exec -T namenode hdfs dfsadmin -report

start 03_hdfs_namespace
run 03_hdfs_namespace docker compose exec -T namenode hdfs dfs -ls /hdfs-optimizer
```

```
run 03_hdfs_namespace docker compose exec -T namenode hdfs dfs -count -q -h /hdfs-optimizer
run 03_hdfs_namespace docker compose exec -T namenode hdfs dfs -du -s -h /hdfs-optimizer/objects /hdfs-optimizer/chunks /hdfs-optimizer/manifests /hdfs-optimizer/raw
run 03_hdfs_namespace docker compose exec -T namenode bash -c 'hdfs dfs -ls -R /hdfs-optimizer/chunks | grep "^-" | head -8'

start 04_hdfs_fsck
run 04_hdfs_fsck docker compose exec -T namenode bash -c 'hdfs fsck /hdfs-optimizer 2>/dev/null | grep -A16 "^Status"; hdfs fsck /hdfs-optimizer 2>/dev/null | tail -2'
run 04_hdfs_fsck docker compose exec -T namenode bash -c 'F=$(hdfs dfs -ls -R /hdfs-optimizer/chunks | grep "^-" | head -1 | awk "{print \$NF}"); hdfs fsck "$F" -files -blocks -locations 2>/dev/null | head -8'

start 05_backend_execution
run 05_backend_execution curl -s http://127.0.0.1:8100/api/v1/health
run 05_backend_execution curl -s http://127.0.0.1:8100/api/v1/storage/savings --output /dev/null -w 'HTTP %{http_code} in %{time_total}s\n'
run 05_backend_execution python3 -c "import json,urllib.request as u; d=json.load(u.urlopen('http://127.0.0.1:8100/api/v1/storage/statistics')); print(json.dumps(d['derived'],indent=1)); print(json.dumps({k:d['measured'][k] for k in ('length','space_consumed','file_count','directory_count')},indent=1))"
run 05_backend_execution tail -n 12 backend/data/backend.log

start 06_restore_checksum
if [ -f evidence/restored/same-book-renamed.html ]; then
  run 06_restore_checksum shasum -a 256 datasets/raw/originals/pg1342.html evidence/restored/same-book-renamed.html
  run 06_restore_checksum cmp datasets/raw/originals/pg1342.html evidence/restored/same-book-renamed.html
fi
run 06_restore_checksum python3 -c "import json;m=json.load(open('datasets/provenance/manifest.json'));print([f['sha256'] for f in m['files'] if f['local_path']=='originals/pg1342.html'][0], 'recorded in provenance manifest')"

start 07_unit_tests
(cd backend && { printf '$ pytest -m "not hdfs" -v\n'; .venv/bin/python -m pytest -m "not hdfs" -q 2>&1 | tail -6; printf '\n'; }) >> evidence/07_unit_tests.txt

start 08_integration_tests
(cd backend && { printf '$ pytest -m hdfs -v  (real HDFS cluster)\n'; .venv/bin/python -m pytest -m hdfs -v 2>&1 | grep -E "PASSED|FAILED|ERROR|SKIPPED|passed|failed" | sed 's/tests\/test_integration_hdfs.py::://'; printf '\n'; }) >> evidence/08_integration_tests.txt

start 09_browser_tests
(cd frontend && { printf '$ npx playwright test  (real browser, real cluster)\n'; npx playwright test 2>&1 | grep -vE "WebServer|^$" ; printf '\n'; }) >> evidence/09_browser_tests.txt

start 10_dataset_provenance
run 10_dataset_provenance python3 -c "import json;m=json.load(open('datasets/provenance/manifest.json'));print('files:',m['number_of_files'],'originals:',m['number_of_original_files'],'bytes:',m['total_size_bytes']); [print(f['kind'][:5],f['file_format'].ljust(4),str(f['size_bytes']).rjust(9),f['sha256'][:16],f['local_path']) for f in m['files']]"
echo "Terminal evidence written to evidence/"

# Keep the replication timings from this session as machine-readable evidence.
python3 - <<'PY'
import json, urllib.request
from datetime import datetime
operations = json.load(urllib.request.urlopen("http://127.0.0.1:8100/api/v1/cluster/replication"))["operations"]
for op in operations:
    start, end = op["requested_at"], op["completed_at"]
    op["duration_seconds"] = (datetime.fromisoformat(end) - datetime.fromisoformat(start)).total_seconds() if end else None
json.dump(operations, open("benchmarks/results/replication_operations.json", "w"), indent=2, default=str)
print("replication operations:", [(o["requested_factor"], o["status"], o["duration_seconds"]) for o in operations])
PY
```

scripts/env.sh 14 lines

```
#!/usr/bin/env bash
# Shared environment for project scripts. Source it; it changes nothing outside this shell.
# A project-local Docker config is used so that a stale Docker Desktop credential
# helper in ~/.docker/config.json cannot break image pulls.
if [ -n "${BASH_VERSION:-}" ]; then _env_self="${BASH_SOURCE[0]}"; else _env_self="${(%):-%x}"; fi
PROJECT_ROOT="$(cd "$(dirname "$_env_self")/.." && pwd)"
unset _env_self
export PROJECT_ROOT
export DOCKER_CONFIG="$PROJECT_ROOT/.docker-local"
export COLIMA_PROFILE="${COLIMA_PROFILE:-hdfsopt}"
if [ -S "$HOME/.colima/$COLIMA_PROFILE/docker.sock" ]; then
  export DOCKER_HOST="unix://$HOME/.colima/$COLIMA_PROFILE/docker.sock"
fi
```

scripts/reset-demo.sh 32 lines

```
#!/usr/bin/env bash
# Deletes disposable demo data only: the managed HDFS directories of this project, and its metadata database.
#
# Paths are fixed literals, not arguments or variables, so this script cannot be pointed at anything else.
# It does not touch the downloaded dataset, source code, results or Docker volumes of other projects.
# Stop the backend first; it must restart with an empty database.
set -euo pipefail
source "$(dirname "$0")/env.sh"
cd "$PROJECT_ROOT"

if [ ! -f backend/app/main.py ] || [ ! -f docker-compose.yml ]; then
  echo "Refusing to run: this does not look like the HDFS Optimizer project."
  exit 1
fi
if curl -fs -m 2 http://127.0.0.1:8100/api/v1/health >/dev/null 2>&1; then
  echo "The backend is still running on port 8100. Stop it first."
  exit 1
fi
```

```
if [ "${1:-}" != "--yes" ]; then
  echo "This permanently deletes all managed files in HDFS under /hdfs-optimizer and the metadata database."
  read -r -p "Type RESET to continue: " answer
  [ "$answer" = "RESET" ] || { echo "Cancelled."; exit 1; }
fi

docker compose exec -T namenode hdfs dfs -rm -r -f -skipTrash \
  /hdfs-optimizer /hdfs-optimizer-experiments /hdfs-optimizer-tests /hdfs-optimizer-e2e /hdfs-optimizer-failure-tests

rm -f backend/data/metadata.db backend/data/metadata.db-wal backend/data/metadata.db-shm
rm -rf backend/data/spool
echo "Reset complete. Start the backend to create a new empty database."
```

scripts/setup.sh 19 lines

```
#!/usr/bin/env bash
# Installs project dependencies and builds the Hadoop image. Writes only inside this project and Docker.
set -euo pipefail
source "$(dirname "$0")/env.sh"
cd "$PROJECT_ROOT"

PYTHON="$(command -v python3.12 || command -v python3.13 || command -v python3)"
echo "Using $("$PYTHON" --version)"
[ -d backend/.venv ] || "$PYTHON" -m venv backend/.venv
backend/.venv/bin/pip install -q --upgrade pip
backend/.venv/bin/pip install -q -r backend/requirements-dev.txt -r reports/source/requirements.txt

(cd frontend && npm ci --no-audit --no-fund && npm run build)

"$PROJECT_ROOT/scripts/start-runtime.sh"
source "$PROJECT_ROOT/scripts/env.sh"
docker compose build namenode
echo "Setup complete. Next: scripts/start-all.sh"
```

scripts/start-all.sh 31 lines

```
#!/usr/bin/env bash
# Starts HDFS, then the backend and frontend in the background of this script. Ctrl+C stops the two app servers.
set -euo pipefail
source "$(dirname "$0")/env.sh"
cd "$PROJECT_ROOT"

"$PROJECT_ROOT/scripts/start-runtime.sh"
source "$PROJECT_ROOT/scripts/env.sh"
docker compose up -d --wait

mkdir -p backend/data
scripts/start-backend.sh > backend/data/backend.log 2>&1 &
BACKEND_PID=$!
scripts/start-frontend.sh > backend/data/frontend.log 2>&1 &
FRONTEND_PID=$!
trap 'kill "$BACKEND_PID" "$FRONTEND_PID" 2>/dev/null || true' EXIT

for _ in $(seq 1 40); do
  if curl -fs -m 2 http://127.0.0.1:8100/api/v1/health >/dev/null && curl -fs -m 2 -o /dev/null http://127.0.0.1:3100/; then break;
fi
  sleep 1
done

cat <<INFO
HDFS Optimizer is running.
  Application      http://localhost:3100
  API docs         http://localhost:8100/docs
  NameNode UI      http://localhost:9870
Logs are in backend/data/. Press Ctrl+C to stop the application; HDFS keeps running (scripts/stop-all.sh stops it).
INFO
wait
```

scripts/start-backend.sh 6 lines

```
#!/usr/bin/env bash
# Starts the FastAPI backend on port 8100. Requires backend/.venv (see scripts/setup.sh).
set -euo pipefail
cd "$(dirname "$0")/../backend"
exec .venv/bin/uvicorn app.main:app --host 127.0.0.1 --port "${BACKEND_PORT:-8100}"
```

scripts/start-frontend.sh 7 lines

```
#!/usr/bin/env bash
# Serves the production build on port 3100. Run `npm run build` in frontend/ first.
set -euo pipefail
cd "$(dirname "$0")/../frontend"
[ -d .next ] || npm run build
exec npx next start -p "${FRONTEND_PORT:-3100}"
```

scripts/start-runtime.sh 11 lines

```
#!/usr/bin/env bash
# Starts a dedicated Colima VM (macOS). Skips itself when any Docker daemon is already reachable.
set -euo pipefail
source "$(dirname "$0")/env.sh"
if docker info >/dev/null 2>&1; then echo "Docker daemon already reachable"; exit 0; fi
command -v colima >/dev/null || { echo "Install a Docker runtime first: brew install colima docker docker-compose"; exit 1; }
colima start "$COLIMA_PROFILE" --cpu "${COLIMA_CPU:-4}" --memory "${COLIMA_MEMORY:-4}" --disk "${COLIMA_DISK:-30}" \
  --vm-type vz --mount-type virtiofs --mount "$PROJECT_ROOT:w"
source "$(dirname "$0")/env.sh"
docker info --format 'Docker {{.ServerVersion}} on {{.Architecture}}, {{.NCPU}} CPUs, {{.MemTotal}} bytes RAM'
```

scripts/stop-all.sh 11 lines

```
#!/usr/bin/env bash
# Clean shutdown of HDFS. Data stays in Docker volumes. Pass --vm to also stop the Colima VM.
set -euo pipefail
source "$(dirname "$0")/env.sh"
cd "$PROJECT_ROOT"
docker compose --profile app stop
if [ "${1:-}" = "--vm" ]; then
  colima stop "$COLIMA_PROFILE"
fi
echo "Stopped. Data is kept. Start again with scripts/start-all.sh"
```

scripts/audit_report.py 126 lines

```
#!/usr/bin/env python3
"""Check the exported report PDF: required sections, source files, figures, references and stray output.

Usage: backend/.venv/bin/python scripts/audit_report.py [--render DIR]
With --render, also writes a contact sheet of the non-code pages for visual inspection.
"""

from __future__ import annotations

import argparse
import json
import re
import sys
from pathlib import Path

import fitz

PROJECT = Path(__file__).resolve().parent.parent
PDF = PROJECT / "reports" / "final" / "HDFS_Storage_Optimizer_Complete_Report.pdf"

REQUIRED = [
    "Certificate", "Declaration of Originality", "Acknowledgements", "Abstract", "Keywords", "Table of Contents",
    "List of Figures", "List of Tables", "Abbreviations",
    "Chapter 1 — Introduction", "Chapter 2 — Problem Statement", "Chapter 3 — Objectives", "Chapter 4 — Literature Review",
    "Chapter 5 — Dataset Description", "Chapter 6 — Dataset Source", "Chapter 7 — Tools and Technologies",
    "Chapter 8 — System Requirements", "Chapter 9 — System Architecture", "Chapter 10 — Implementation and Methodology",
    "Chapter 11 — Complete Source Code", "Chapter 12 — Execution Screenshots", "Chapter 13 — Output and Result Screenshots",
    "Chapter 14 — Result Analysis", "Chapter 15 — Limitations", "Chapter 16 — Future Scope", "Chapter 17 — Conclusion",
    "References", "Appendix A", "Appendix B", "Appendix C", "Appendix D", "Appendix E", "Appendix F", "Appendix G", "Appendix H",
]
FORBIDDEN = [r"\bTODO\b", r"\bFIXME\b", r"lorem ipsum", r"\bNaN\b", r"\bundefined\b(?! when| metrics)", r"#let "]


def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--render", type=Path)
    args = parser.parse_args()

    document = fitz.open(PDF)
    pages = [page.get_text() for page in document]
    text = "\n".join(pages)
    normalized = re.sub(r"\s+", " ", text)
    failures: list[str] = []

    print(f"pages: {len(pages)}, size: {PDF.stat().st_size / 1e6:.1f} MB")
    for section in REQUIRED:
        if section not in normalized:
            failures.append(f"missing section: {section}")

    groups = json.loads((PROJECT / "reports/source/generated/code_files.json").read_text())
    files = [f for group in groups for f in group["files"]]
    for file in files:
        if file["path"] not in normalized:
            failures.append(f"source file heading not found: {file['path']}")
    # Listings are complete if distinctive last lines of each file appear in the PDF.
    squeezed = re.sub(r"\s+", "", text)
    for file in files:
        lines = [line.strip() for line in (PROJECT / file["path"]).read_text().splitlines() if len(line.strip()) > 12]
        if lines and re.sub(r"\s+", "", lines[-1])[:50] not in squeezed:
            failures.append(f"source listing appears truncated: {file['path']}")
    print(f"source files checked: {len(files)}")

    figure_count = len(set(re.findall(r"Figure (\d+):", normalized)))
    table_count = len(set(re.findall(r"Table (\d+):", normalized)))
    images = sum(len(page.get_images()) for page in document)
    print(f"figures: {figure_count}, tables: {table_count}, embedded images: {images}")

    for pattern in FORBIDDEN:
        hits = [i + 1 for i, page in enumerate(pages) if re.search(pattern, page, flags=re.IGNORECASE) and not _is_code_page(page,
files)]
        if hits:
            failures.append(f"pattern {pattern!r} on non-code pages {hits[:10]}")

    outside = []
    for number, page in enumerate(document, start=1):
        rect = page.rect
        for block in page.get_text("blocks"):
            if block[2] > rect.width + 1 or block[3] > rect.height + 1 or block[0] < -1:
                outside.append(number)
                break
```

```
    if outside:
        failures.append(f"text outside the page on pages {outside[:20]}")

    unnumbered = [i + 1 for i, page in enumerate(pages) if i > 4 and not re.search(r"(^|\n)\s*([ivxlc]+|\d+)\s*\n?\s*$", page)]
    print(f"pages without a trailing page number: {len(unnumbered)} {unnumbered[:10]}")

    blank = [i + 1 for i, page in enumerate(document) if len(pages[i].strip()) < 15 and not page.get_images()]
    if blank:
        failures.append(f"blank pages: {blank}")

    if args.render:
        args.render.mkdir(parents=True, exist_ok=True)
        code_start = next(i for i, page in enumerate(pages) if "Complete Source Code" in page and "This chapter contains" in page)
        code_end = next(i for i, page in enumerate(pages) if "Execution Screenshots" in page and "Every image in this chapter" in
page)
        print(f"code pages: {code_start + 1} to {code_end}")
        contact_sheets(document, [i for i in range(len(pages)) if i < code_start + 2 or i >= code_end - 1], args.render)

    for failure in failures:
        print("FAIL", failure)
    print("result:", "PASS" if not failures else f"{len(failures)} problem(s)")
    return 1 if failures else 0


def _is_code_page(page_text: str, files: list[dict]) -> bool:
    return any(file["path"] in page_text for file in files) or "import " in page_text or "def " in page_text or "=>" in page_text


def contact_sheets(document, indexes: list[int], out: Path, per_sheet: int = 12, scale: float = 0.62) -> None:
    columns = 4
    for sheet, start in enumerate(range(0, len(indexes), per_sheet)):
        chunk = indexes[start : start + per_sheet]
        width = 595 * scale
        height = 842 * scale
        rows = (len(chunk) + columns - 1) // columns
        canvas = fitz.open()
        page = canvas.new_page(width=width * columns, height=height * rows)
        for position, index in enumerate(chunk):
            x = (position % columns) * width
            y = (position // columns) * height
            page.show_pdf_page(fitz.Rect(x + 3, y + 3, x + width - 3, y + height - 3), document, index)
        page.get_pixmap(dpi=110).save(out / f"sheet_{sheet:02d}.png")
        print(f"sheet_{sheet:02d}.png: pages {chunk[0] + 1} to {chunk[-1] + 1}")


if __name__ == "__main__":
    sys.exit(main())
```

scripts/build_report.py 117 lines

```
#!/usr/bin/env python3
"""Generate report data from the repository, then compile the PDF with Typst.

The code listings, API table and schema all come from the working source, so the report cannot diverge from it.
"""

from __future__ import annotations

import json
import re
import subprocess
import sys
from pathlib import Path

PROJECT = Path(__file__).resolve().parent.parent
SOURCE = PROJECT / "reports" / "source"
GENERATED = SOURCE / "generated"
FINAL = PROJECT / "reports" / "final" / "HDFS_Storage_Optimizer_Complete_Report.pdf"
sys.path.insert(0, str(PROJECT / "backend"))

LANG = {".py": "python", ".js": "javascript", ".ts": "typescript", ".tsx": "tsx", ".css": "css", ".yml": "yaml", ".sh": "bash",
".xml": "xml", ".json": "json", ".ini": "ini", ".mjs": "javascript", ".mako": "text", ".txt": "text", ".md": "markdown"}

GROUPS = [
    ("Infrastructure and deployment", ["docker-compose.yml", "docker-compose.light.yml", ".env.example", "infrastructure/docker/
hadoop/Dockerfile", "infrastructure/docker/hadoop/entrypoint.sh", "infrastructure/hadoop/*.xml", "backend/Dockerfile",
"backend/.dockerignore", "frontend/Dockerfile", "frontend/.dockerignore"]),
    ("Backend: core, models and HDFS client", ["backend/requirements*.txt", "backend/alembic.ini", "backend/pytest.ini", "backend/
app/main.py", "backend/app/core/*.py", "backend/app/models/*.py", "backend/app/schemas/*.py", "backend/app/hdfs/*.py", "backend/
migrations/env.py", "backend/migrations/script.py.mako", "backend/migrations/versions/*.py"]),
    ("Backend: deduplication and services", ["backend/app/services/*.py"]),
    ("Backend: API routes", ["backend/app/api/*.py"]),
    ("Frontend: libraries and shared components", ["frontend/package.json", "frontend/tsconfig.json", "frontend/next.config.ts",
"frontend/eslint.config.mjs", "frontend/postcss.config.mjs", "frontend/components.json", "frontend/.env.example", "frontend/app/
globals.css", "frontend/app/layout.tsx", "frontend/lib/*.ts", "frontend/components/*.tsx", "frontend/components/shell/*.tsx",
"frontend/components/charts/*.tsx"]),
    ("Frontend: pages", ["frontend/app/page.tsx", "frontend/app/*/page.tsx"]),
    ("Tests", ["backend/tests/conftest.py", "backend/tests/support/*.py", "backend/tests/test_*.py", "frontend/
playwright.config.ts", "frontend/e2e/*.ts", "frontend/capture/*.ts"]),
    ("Reproduction and tooling scripts", ["scripts/*.sh", "scripts/*.py", "presentation/source/prepare_crops.py", "presentation/
source/build_deck.js", "presentation/source/package.json", "reports/source/requirements.txt"]),
]
```

```
def code_files() -> list[dict]:
    groups, seen = [], set()
    for title, patterns in GROUPS:
        files = []
        for pattern in patterns:
            for path in sorted(PROJECT.glob(pattern)):
                relative = path.relative_to(PROJECT).as_posix()
                if not path.is_file() or relative in seen or path.name == "__init__.py":
                    continue
                seen.add(relative)
                text = path.read_text()
                files.append({"path": relative, "lang": "dockerfile" if path.name == "Dockerfile" else LANG.get(path.suffix,
"text"), "lines": text.count("\n") + 1})
        if files:
            groups.append({"title": title, "files": files})
    return groups


def api_routes() -> list[dict]:
    from app.main import create_app

    schema = create_app().openapi()
    routes = []
    for path, methods in schema["paths"].items():
        for method, spec in methods.items():
            description = (spec.get("description") or spec.get("summary") or "").strip().split("\n")[0]
            routes.append({"method": method.upper(), "path": path.removeprefix("/api/v1"), "summary": description})
    return routes


def db_schema() -> list[dict]:
    from app.core.db import Base
    from app.models import entities  # noqa: F401
    from sqlalchemy import CheckConstraint, UniqueConstraint

    tables = []
    for table in Base.metadata.sorted_tables:
        columns = []
        for column in table.columns:
            keys = []
            if column.primary_key:
                keys.append("primary key")
            keys.extend(f"references {fk.target_fullname}" for fk in column.foreign_keys)
            if column.unique:
                keys.append("unique")
            if column.index:
                keys.append("indexed")
            columns.append({"name": column.name, "type": str(column.type), "nullable": bool(column.nullable), "key": ",
".join(keys)})
        constraints = [c.name for c in table.constraints if isinstance(c, CheckConstraint | UniqueConstraint) and c.name]
        constraints += [i.name for i in table.indexes if i.name and i.name.startswith("uq_")]
        tables.append({"name": table.name, "columns": columns, "constraints": constraints})
    return tables


def test_counts() -> dict:
    def parse(name: str) -> dict:
        path = PROJECT / "evidence" / f"{name}.txt"
        text = path.read_text() if path.exists() else ""
        passed = re.findall(r"(\d+) passed", text)
        failed = re.findall(r"(\d+) failed", text)
        return {"passed": int(passed[-1]) if passed else 0, "failed": int(failed[-1]) if failed else 0}

    return {"unit": parse("07_unit_tests"), "integration": parse("08_integration_tests"), "browser": parse("09_browser_tests")}


def main() -> int:
    GENERATED.mkdir(parents=True, exist_ok=True)
    groups = code_files()
    (GENERATED / "code_files.json").write_text(json.dumps(groups, indent=1))
    (GENERATED / "api_routes.json").write_text(json.dumps(api_routes(), indent=1))
    (GENERATED / "db_schema.json").write_text(json.dumps(db_schema(), indent=1))
    (GENERATED / "test_counts.json").write_text(json.dumps(test_counts(), indent=1))
    print("source listing:", sum(len(g["files"]) for g in groups), "files,", sum(f["lines"] for g in groups for f in g["files"]),
"lines")
    FINAL.parent.mkdir(parents=True, exist_ok=True)
    result = subprocess.run(["typst", "compile", "--root", str(PROJECT), str(SOURCE / "main.typ"), str(FINAL)])
    if result.returncode:
        return result.returncode
    print(f"wrote {FINAL} ({FINAL.stat().st_size / 1e6:.1f} MB)")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

scripts/download_dataset.py 43 lines

```
#!/usr/bin/env python3
"""Download authentic source files from Project Gutenberg and write provenance.

Usage:  backend/.venv/bin/python scripts/download_dataset.py [--ebook 1342 --ebook 11]

Safe to re-run: files whose bytes still match their recorded SHA-256 are not requested again.
```

```
"""

import argparse
import sys
from pathlib import Path

PROJECT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(PROJECT / "backend"))

from app.services.datasets import (  # noqa: E402
    build_transformed_workload,
    download_catalog,
)


def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--ebook", type=int, action="append", help="Limit to these ebook ids (default: full catalog)")
    parser.add_argument("--no-workload", action="store_true", help="Skip the documented partial-overlap test files")
    args = parser.parse_args()

    dataset_dir = PROJECT / "datasets" / "raw"
    manifest = PROJECT / "datasets" / "provenance" / "manifest.json"
    summary = download_catalog(dataset_dir, manifest, ebook_ids=args.ebook)
    print(f"downloaded={summary['downloaded']} already_present={summary['already_present']} failed={len(summary['failed'])}")
    for failure in summary["failed"]:
        print(f"  FAILED {failure['url']}: {failure['error']}")
    if not args.no_workload and not summary["failed"] and (args.ebook is None or 1342 in args.ebook):
        for record in build_transformed_workload(dataset_dir, manifest):
            print(f"workload {record['local_path']}: {record['size_bytes']} bytes. {record['transformations']}")
    print(f"provenance: {manifest}")
    return 1 if summary["failed"] else 0


if __name__ == "__main__":
    raise SystemExit(main())
```

scripts/final_audit.py 180 lines

```
#!/usr/bin/env python3
"""Final quality audit. Every line it prints comes from a check it just ran.

Usage: backend/.venv/bin/python scripts/final_audit.py
Needs HDFS, the backend and the frontend running. Output is also saved to evidence/11_final_audit.txt.
"""

from __future__ import annotations

import hashlib
import json
import os
import re
import shutil
import subprocess
import sys
import time
from datetime import UTC, datetime
from pathlib import Path

import fitz
import httpx
from pptx import Presentation

PROJECT = Path(__file__).resolve().parent.parent
API = "http://127.0.0.1:8100/api/v1"
FRONTEND = "http://127.0.0.1:3100"
PDF = PROJECT / "reports/final/HDFS_Storage_Optimizer_Complete_Report.pdf"
PPTX = PROJECT / "presentation/final/HDFS_Storage_Optimizer_Presentation.pptx"
REQUIRED_SCREENSHOTS = [
    "01_dashboard", "02_file_upload", "03_file_explorer", "04_duplicate_analysis", "05_optimization_preview",
    "06_optimization_complete", "07_storage_analytics", "08_replication_management", "09_cluster_status",
    "10_namenode_interface", "11_datanode_status", "terminal_05_backend_execution", "terminal_03_hdfs_namespace",
    "terminal_08_integration_tests", "16a_file_details_verified", "terminal_06_restore_checksum", "17_dataset_provenance",
]
PAGES = ["/", "/files", "/duplicates", "/optimize", "/replication", "/cluster", "/analytics", "/activity", "/lab", "/settings"]

results: list[tuple[bool, str]] = []
lines: list[str] = []


def say(text: str) -> None:
    print(text, flush=True)
    lines.append(text)


def check(ok: bool, text: str) -> bool:
    results.append((bool(ok), text))
    say(f"[{'PASS' if ok else 'FAIL'}] {text}")
    return bool(ok)


def run(command: list[str], cwd: Path) -> tuple[int, str]:
    process = subprocess.run(command, cwd=cwd, capture_output=True, text=True)
    return process.returncode, process.stdout + process.stderr
```

```
def sha256(data: bytes) -> str:
    return hashlib.sha256(data).hexdigest()


def main() -> int:
    say(f"Final audit, {datetime.now(UTC).isoformat()}")

    free = shutil.disk_usage(PROJECT).free / 1024**3
    check(free > 1.0, f"Free disk on the host: {free:.1f} GiB")

    say("\n-- Services")
    code, output = run(["docker", "compose", "ps", "--format", "{{.Service}} {{.State}} {{.Health}}"], PROJECT)
    healthy = [line for line in output.splitlines() if "running healthy" in line]
    check(len(healthy) == 4, f"HDFS containers healthy: {len(healthy)} of 4")
    health = httpx.get(f"{API}/health", timeout=20).json()
    check(health["status"] == "ok" and health["hdfs_reachable"], f"Backend health: {health}")
    diagnostics = httpx.get(f"{API}/system/diagnostics", timeout=60).json()
    check(diagnostics["ok"], "Backend diagnostics, including a real HDFS write, read and delete")
    statuses = {page: httpx.get(f"{FRONTEND}{page}", timeout=30).status_code for page in PAGES}
    check(all(s == 200 for s in statuses.values()), f"Frontend pages returning 200: {sum(s == 200 for s in statuses.values())} of
{len(PAGES)}")
    openapi = httpx.get("http://127.0.0.1:8100/openapi.json", timeout=20).json()
    check(len(openapi["paths"]) > 30, f"API documentation: {len(openapi['paths'])} documented paths")

    say("\n-- Authentic file upload and SHA-256-matching download")
    source = PROJECT / "datasets/raw/originals/pg1661.epub"
    data = source.read_bytes()
    recorded = next(f["sha256"] for f in json.loads((PROJECT / "datasets/provenance/manifest.json").read_text())["files"] if
f["local_path"] == "originals/pg1661.epub")
    check(sha256(data) == recorded, "Source file matches its recorded provenance hash")
    created = []
    for mode, chunk_size in (("file", None), ("chunk", 65536), ("raw", None)):
        form = {"mode": mode, "folder": "/final-audit"}
        if chunk_size:
            form["chunk_size"] = str(chunk_size)
        response = httpx.post(f"{API}/files/upload", files={"file": (f"audit-{mode}.epub", data)}, data=form, timeout=120)
        response.raise_for_status()
        file_id = response.json()["file"]["id"]
        created.append(file_id)
        restored = httpx.get(f"{API}/files/{file_id}/download", timeout=120).content
        check(sha256(restored) == recorded and restored == data, f"Mode {mode}: outcome {response.json()['outcome']}, download is
byte-identical, SHA-256 {sha256(restored)[:16]}...")
        verified = httpx.post(f"{API}/files/{file_id}/verify", timeout=120).json()
        check(verified["ok"], f"Mode {mode}: integrity check reads the bytes back from HDFS and passes")
    for file_id in created:
        httpx.delete(f"{API}/files/{file_id}", timeout=60).raise_for_status()
        httpx.delete(f"{API}/files/{file_id}", params={"permanent": "true"}, timeout=60).raise_for_status()
    reconcile = httpx.get(f"{API}/storage/reconcile", timeout=120).json()
    check(reconcile["orphan_count"] == 0 and reconcile["missing_count"] == 0, f"Database and HDFS agree: {reconcile['orphan_count']}
orphans, {reconcile['missing_count']} missing")
    stats = httpx.get(f"{API}/storage/statistics", timeout=60).json()
    say(f"Current system: {stats['derived']['logical_files']} files, {stats['derived']['savings_percent']:.1f}% savings (derived),
{stats['measured']['space_consumed']} bytes consumed (measured)")
    job = httpx.post(f"{API}/files/verify-all", timeout=30).json()["job_id"]
    deadline = time.time() + 300
    while time.time() < deadline:
        state = httpx.get(f"{API}/dedup/jobs/{job}", timeout=30).json()
        if state["status"] not in ("queued", "running"):
            break
        time.sleep(1)
    check(state["status"] == "succeeded" and not state["result"]["failed"], f"Every managed file verified: {state['result']
['passed']} of {state['result']['files_checked']}")

    say("\n-- Automated tests")
    python = str(PROJECT / "backend/.venv/bin/python")
    for label, args, cwd in (
        ("Unit and API", [python, "-m", "pytest", "-m", "not hdfs", "-q"], PROJECT / "backend"),
        ("Integration on real HDFS", [python, "-m", "pytest", "-m", "hdfs", "-q"], PROJECT / "backend"),
        ("Browser", ["npx", "playwright", "test"], PROJECT / "frontend"),
    ):
        code, output = run(args, cwd)
        passed = re.findall(r"(\d+) passed", output)
        failed = re.findall(r"(\d+) failed", output)
        check(code == 0 and passed and not failed, f"{label} tests: {passed[-1] if passed else 0} passed, {failed[-1] if failed else
0} failed")
    code, output = run(["npx", "tsc", "--noEmit"], PROJECT / "frontend")
    check(code == 0, "Frontend type check")
    code, output = run([str(PROJECT / "backend/.venv/bin/ruff"), "check", "backend/app", "backend/tests", "--select", "E,F,I,B,UP",
"--ignore", "E501,B008,B017,B018"], PROJECT)
    check(code == 0, "Backend lint")

    say("\n-- Evidence")
    missing = [name for name in REQUIRED_SCREENSHOTS if not (PROJECT / "screenshots" / f"{name}.png").exists()]
    check(not missing, f"Required screenshots present: {len(REQUIRED_SCREENSHOTS) - len(missing)} of {len(REQUIRED_SCREENSHOTS)}
{missing or ''}")
    say(f"All screenshots: {len(list((PROJECT / 'screenshots').glob('*.png')))}")
    benchmark_json = list((PROJECT / "benchmarks/results").glob("*.json"))
    documents = [d for d in (json.loads(p.read_text()) for p in benchmark_json) if isinstance(d, dict) and "scenario" in d]
    failed_runs = [d["scenario"] for d in documents if d.get("status") == "failed"]
    inconsistent = [d["scenario"] for d in documents if d.get("inconsistent_values")]
    check(not failed_runs, f"Benchmark result files: {len(benchmark_json)} JSON, {len(list((PROJECT / 'benchmarks/
```

```
results').glob('*.csv'))))} CSV, {len(failed_runs)} failed")
    check(not inconsistent, f"Byte counts identical across repetitions: {len(inconsistent)} scenarios differ")
    failures = json.loads((PROJECT / "benchmarks/results/J_failure_handling.json").read_text())
    check(all(failures[k]["status"] == "completed" for k in ("J1_datanode_failure", "J2_namenode_failure", "J3_process_kill")),
"Failure experiments completed")
    check(failures["J3_process_kill"]["files_restored_byte_identical_after_resume"] == failures["J3_process_kill"]["files"],
"Process kill: every file restored byte-identical after resume")

    say("\n-- Report PDF")
    document = fitz.open(PDF)
    check(len(document) > 200, f"PDF opens: {len(document)} pages, {PDF.stat().st_size / 1e6:.1f} MB")
    code, output = run([python, "scripts/audit_report.py"], PROJECT)
    for line in output.strip().splitlines():
        say(f"  {line}")
    check(code == 0, "Report audit: required sections, every source file, no stray output, nothing outside the page")

    say("\n-- Presentation PPTX")
    deck = Presentation(PPTX)
    slides = list(deck.slides)
    check(len(slides) == 20, f"PPTX opens: {len(slides)} slides, {PPTX.stat().st_size / 1e6:.1f} MB")
    with_notes = sum(1 for s in slides if s.has_notes_slide and s.notes_slide.notes_text_frame.text.strip())
    check(with_notes == 20, f"Slides with speaker notes: {with_notes}")
    charts = sum(1 for s in slides for shape in s.shapes if shape.has_chart)
    pictures = sum(1 for s in slides for shape in s.shapes if shape.shape_type == 13)
    say(f"Native editable charts: {charts}; real screenshots: {pictures}")
    check(charts >= 5 and pictures >= 8, "Deck uses native charts and real screenshots")
    deck_text = " ".join(shape.text_frame.text for s in slides for shape in s.shapes if shape.has_text_frame)
    expected = json.loads((PROJECT / "benchmarks/results/I_before_after.json").read_text())["rows"][0]
    check(f"{expected['hdfs_space_consumed_before'] / 1048576:.0f} MiB" in deck_text and f"{expected['hdfs_space_consumed_after'] /
1048576:.0f} MiB" in deck_text, "Deck figures match the benchmark results used by the report")

    say("\n-- Deliverables")
    for path in (PDF, PPTX, PPTX.with_suffix(".pdf"), PROJECT / "README.md", PROJECT / "PROJECT_STATUS.md", PROJECT /
"documentation/VIVA_PREPARATION.md", PROJECT / "documentation/RESULTS_SUMMARY.md", PROJECT / "datasets/provenance/manifest.json",
PROJECT / "docker-compose.yml", PROJECT / "docker-compose.light.yml", PROJECT / ".env.example"):
        check(path.exists() and path.stat().st_size > 0, f"{path.relative_to(PROJECT)}: {path.stat().st_size} bytes")

    failures_count = sum(1 for ok, _ in results if not ok)
    say(f"\nRESULT: {len(results) - failures_count} of {len(results)} checks passed")
    (PROJECT / "evidence/11_final_audit.txt").write_text("\n".join(lines) + "\n")
    return 1 if failures_count else 0


if __name__ == "__main__":
    os.environ.setdefault("DOCKER_CONFIG", str(PROJECT / ".docker-local"))
    sys.exit(main())
```

scripts/make_charts.py 270 lines

```
#!/usr/bin/env python3
"""Draw charts and write the results summary, directly from benchmarks/results/*.json.

Nothing is typed in by hand: a chart or figure can only show what an experiment recorded.
Charts use one axis each, thin marks, hairline grids and a palette checked for color-vision deficiency.
"""

from __future__ import annotations

import json
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt  # noqa: E402

PROJECT = Path(__file__).resolve().parent.parent
RESULTS = PROJECT / "benchmarks" / "results"
CHARTS = PROJECT / "benchmarks" / "charts"
SUMMARY = PROJECT / "documentation" / "RESULTS_SUMMARY.md"

BLUE, ORANGE, GRAY = "#2a78d6", "#eb6834", "#898781"
INK, SECONDARY, GRID, AXIS = "#0b0b0b", "#52514e", "#e1e0d9", "#c3c2b7"
MIB = 1024 * 1024

plt.rcParams.update(
    {
        "font.family": ["Helvetica Neue", "Arial", "DejaVu Sans"],
        "font.size": 10,
        "axes.edgecolor": AXIS,
        "axes.linewidth": 0.8,
        "axes.titlesize": 11,
        "axes.titleweight": "semibold",
        "axes.titlelocation": "left",
        "axes.labelcolor": SECONDARY,
        "text.color": INK,
        "xtick.color": SECONDARY,
        "ytick.color": SECONDARY,
        "xtick.major.size": 0,
        "ytick.major.size": 0,
        "axes.grid": True,
        "axes.grid.axis": "y",
        "grid.color": GRID,
        "grid.linewidth": 0.8,
```

```
        "axes.axisbelow": True,
        "axes.spines.top": False,
        "axes.spines.right": False,
        "axes.spines.left": False,
        "legend.frameon": False,
        "figure.dpi": 200,
        "savefig.bbox": "tight",
    }
)


def load(name: str) -> dict:
    return json.loads((RESULTS / f"{name}.json").read_text())


def save(fig, name: str) -> None:
    CHARTS.mkdir(parents=True, exist_ok=True)
    fig.savefig(CHARTS / f"{name}.svg")
    fig.savefig(CHARTS / f"{name}.png", dpi=220)
    plt.close(fig)


def mib(value: float) -> float:
    return value / MIB


def size_label(bytes_value: int | None) -> str:
    if bytes_value is None:
        return "whole"
    return f"{bytes_value // 1024} KiB" if bytes_value < MIB else f"{bytes_value // MIB} MiB"


def grouped_bars(ax, categories, series, *, fmt="{:.1f}", width=0.32):
    positions = range(len(categories))
    offset = (len(series) - 1) * width / 2
    for index, (label, values, color) in enumerate(series):
        xs = [p + index * (width + 0.02) - offset for p in positions]
        bars = ax.bar(xs, values, width, label=label, color=color)
        for bar, value in zip(bars, values, strict=True):
            ax.text(bar.get_x() + bar.get_width() / 2, value, fmt.format(value), ha="center", va="bottom", fontsize=8.5,
color=SECONDARY)
    ax.set_xticks(list(positions), categories)


def chart_exact_duplicates():
    rows = load("B_exact_duplicates")["rows"]
    copies = sorted({r["copies"] for r in rows})
    raw = [next(r for r in rows if r["copies"] == c and r["mode"] == "raw") for c in copies]
    file = [next(r for r in rows if r["copies"] == c and r["mode"] == "file") for c in copies]
    fig, ax = plt.subplots(figsize=(6.4, 3.2))
    grouped_bars(
        ax,
        [f"{c} {'copy' if c == 1 else 'copies'} per file" for c in copies],
        [
            ("No deduplication", [mib(r["hdfs_space_consumed_bytes"]) for r in raw], GRAY),
            ("Whole-file deduplication", [mib(r["hdfs_space_consumed_bytes"]) for r in file], BLUE),
        ],
        fmt="{:.0f}",
    )
    ax.set_ylabel("HDFS space consumed (MiB)")
    ax.set_title("Exact duplicates: space measured by the NameNode, replication 3")
    ax.legend(loc="upper left")
    save(fig, "exact_duplicates_space")


def chart_chunk_tradeoff():
    rows = load("D_chunk_sizes")["rows"]
    chunked = [r for r in rows if r["mode"] == "chunk"]
    labels = [size_label(r["chunk_size"]) for r in chunked]
    panels = [
        ("Savings on this workload (%)", [r["savings_percent"] for r in chunked], "{:.1f}", None),
        ("Upload throughput (MiB/s)", [r["upload_throughput_mib_s.mean"] for r in chunked], "{:.1f}",
([r["upload_throughput_mib_s.mean"] - r["upload_throughput_mib_s.min"] for r in chunked], [r["upload_throughput_mib_s.max"] -
r["upload_throughput_mib_s.mean"] for r in chunked])),
        ("NameNode objects (files, directories, blocks)", [r["namespace_objects.mean"] for r in chunked], "{:,.0f}", None),
    ]
    fig, axes = plt.subplots(1, 3, figsize=(10.2, 2.9))
    for ax, (title, values, fmt, err) in zip(axes, panels, strict=True):
        ax.bar(labels, values, 0.5, color=BLUE, yerr=err, ecolor=SECONDARY, error_kw={"linewidth": 1, "capsize": 2})
        top = [v + (err[1][i] if err else 0) for i, v in enumerate(values)]
        for index, value in enumerate(values):
            ax.text(index, top[index], fmt.format(value), ha="center", va="bottom", fontsize=8.5, color=SECONDARY)
        ax.set_title(title, fontsize=10)
        ax.tick_params(axis="x", labelsize=8.5)
    fig.supxlabel("Chunk size", fontsize=9.5, color=SECONDARY)
    fig.tight_layout()
    save(fig, "chunk_size_tradeoff")


def chart_before_after():
    rows = load("I_before_after")["rows"]
    labels = [("Whole-file" if r["mode"] == "file" else f"Chunk, {size_label(r['chunk_size'])}") for r in rows]
    fig, axes = plt.subplots(1, 2, figsize=(8.8, 3.1))
```

```
    grouped_bars(
        axes[0],
        labels,
        [
            ("Before", [mib(r["hdfs_space_consumed_before"]) for r in rows], GRAY),
            ("After", [mib(r["hdfs_space_consumed_after"]) for r in rows], BLUE),
        ],
        fmt="{:.0f}",
    )
    axes[0].set_title("NameNode space consumed (MiB)", fontsize=10)
    grouped_bars(
        axes[1],
        labels,
        [
            ("Before", [mib(r["disk_used_delta_before.mean"]) for r in rows], GRAY),
            ("After", [mib(r["disk_used_delta_after.mean"]) for r in rows], BLUE),
        ],
        fmt="{:.0f}",
    )
    axes[1].set_title("DataNode disk used, measured with du (MiB)", fontsize=10)
    handles, labels_ = axes[0].get_legend_handles_labels()
    fig.legend(handles, labels_, loc="lower center", ncol=2, bbox_to_anchor=(0.5, -0.06))
    fig.tight_layout()
    save(fig, "before_after_optimization")


def chart_restore():
    rows = load("E_roundtrip")["rows"]
    names = {"raw": "No dedup", "file": "Whole-file", "chunk": "Chunk, 256 KiB"}
    fig, ax = plt.subplots(figsize=(5, 2.8))
    values = [r["restore_throughput_mib_s.mean"] for r in rows]
    errors = ([v - r["restore_throughput_mib_s.min"] for v, r in zip(values, rows, strict=True)], [r["restore_throughput_mib_s.max"]
- v for v, r in zip(values, rows, strict=True)])
    ax.bar([names[r["mode"]] for r in rows], values, 0.42, color=BLUE, yerr=errors, ecolor=SECONDARY, error_kw={"linewidth": 1,
"capsize": 3})
    for index, value in enumerate(values):
        ax.text(index, value + errors[1][index], f"{value:.0f}", ha="center", va="bottom", fontsize=9, color=SECONDARY)
    ax.set_ylabel("MiB/s")
    ax.set_title("Verified restore throughput (bars show the range)")
    save(fig, "restore_throughput")


def chart_boundary_shift():
    rows = load("D2_boundary_shift")["rows"]
    names = {
        "pg1342.truncate_90.txt": "Truncated to 90%",
        "pg1342.replace_mid_byte.txt": "One byte replaced",
        "pg1342.prefix_newline.txt": "One byte inserted at start",
    }
    fig, ax = plt.subplots(figsize=(6, 2.3))
    labels = [names[r["variant"]] for r in rows][::-1]
    values = [r["reuse_percent"] for r in rows][::-1]
    ax.barh(labels, values, 0.45, color=BLUE)
    ax.grid(axis="x")
    ax.grid(axis="y", visible=False)
    for index, value in enumerate(values):
        ax.text(value + 1.2, index, f"{value:.1f}%", va="center", fontsize=9, color=SECONDARY)
    ax.set_xlim(0, 110)
    ax.set_xlabel("Chunks reused from the original (%)")
    ax.set_title(f"Fixed-size chunking at {size_label(rows[0]['chunk_size'])}")
    save(fig, "boundary_shift")


def bytes_text(value: float) -> str:
    return f"{value / MIB:.2f} MiB"


def write_summary() -> None:
    env = json.loads((RESULTS / "environment.json").read_text())
    a, b, c, d, d2 = (load(n) for n in ("A_unique_files", "B_exact_duplicates", "C_many_names", "D_chunk_sizes",
"D2_boundary_shift"))
    e, f, h, i = (load(n) for n in ("E_roundtrip", "F_shared_deletion", "H_replication", "I_before_after"))
    lines = [
        "# Results summary",
        "",
        "Generated by `scripts/make_charts.py` from `benchmarks/results/`. Do not edit by hand.",
        "",
        f"Run started {env['run_started']}. {env['deployment']} Hadoop {env['hadoop_version']}, {env['live_datanodes']} DataNodes,
replication {env['default_replication']}, {env['machine']}.",
        "",
        "Savings below describe these workloads, in which duplicates were created on purpose. They do not predict savings on any
real system. Throughput describes this laptop setup.",
        "",
        "## A. Unique files",
        "",
        "| Mode | Files | Logical bytes | Stored bytes | Savings | HDFS files |",
        "|---|---:|---:|---:|---:|---:|",
    ]
    for r in a["rows"]:
        lines.append(f"| {r['mode']} | {r['logical_files']} | {bytes_text(r['logical_bytes'])} |
{bytes_text(r['stored_content_bytes'])} | {r['savings_percent']:.2f}% | {r['hdfs_file_count']} |")
    lines += ["", "## B. Exact duplicates", "", "| Copies per file | Mode | Logical | Stored | Savings | Ratio | HDFS space consumed
| Upload MiB/s (mean, range) |", "|---:|---|---:|---:|---:|---:|---:|---:|"]
```

```
    for r in b["rows"]:
        lines.append(
            f"| {r['copies']} | {r['mode']} | {bytes_text(r['logical_bytes'])} | {bytes_text(r['stored_content_bytes'])} |
{r['savings_percent']:.2f}% | {r['dedup_ratio']:.2f} | {bytes_text(r['hdfs_space_consumed_bytes'])} | "
            f"{r['upload_throughput_mib_s.mean']:.1f} ({r['upload_throughput_mib_s.min']:.1f} to
{r['upload_throughput_mib_s.max']:.1f}) |"
        )
    r = c["rows"][0]
    lines += ["", "## C. One file, many names", "", f"{c['extra']['logical_names']} logical names for one authentic file:
{bytes_text(r['logical_bytes'])} logical, {bytes_text(r['stored_content_bytes'])} stored, {r['hdfs_file_count']} HDFS file, savings
{r['savings_percent']:.2f}%. All {r['verification.files_passed']} of {r['verification.files_checked']} names passed SHA-256
verification."]
    lines += ["", "## D. Chunk size trade-off", "", "| Mode | Chunk size | Stored | Savings | HDFS files | NameNode objects | Disk
used (du) | Upload MiB/s |", "|---|---:|---:|---:|---:|---:|---:|---:|"]
    for r in d["rows"]:
        lines.append(
            f"| {r['mode']} | {size_label(r['chunk_size'])} | {bytes_text(r['stored_content_bytes'])} | {r['savings_percent']:.2f}%
| {r['hdfs_file_count']} | {r['namespace_objects.mean']:.0f} | {bytes_text(r['disk_used_delta.mean'])} |
{r['upload_throughput_mib_s.mean']:.1f} |"
        )
    lines += ["", "## D2. Fixed-size chunking boundary shift", "", "| Variant | Chunks | Reused | Reuse |", "|---|---:|---:|---:|"]
    for r in d2["rows"]:
        lines.append(f"| {r['variant']} | {r['chunks_total']} | {r['chunks_reused']} | {r['reuse_percent']:.1f}% |")
    lines += ["", "## E. Restore and checksum", "", "| Mode | Files | Matching source SHA-256 | Restore MiB/s (mean, range) |",
"|---|---:|---:|---:|"]
    for r in e["rows"]:
        lines.append(f"| {r['mode']} | {r['files']} | {r['files_matching_source_sha256']} | {r['restore_throughput_mib_s.mean']:.1f}
({r['restore_throughput_mib_s.min']:.1f} to {r['restore_throughput_mib_s.max']:.1f}) |")
    lines += ["", "## F. Deletion with shared content", "", "| Mode | Stored, 2 refs | Stored, after purging 1 | Survivor passes |
Delete then restore passes | HDFS files after last purge |", "|---|---:|---:|---|---|---:|"]
    for r in f["rows"]:
        lines.append(f"| {r['mode']} | {bytes_text(r['stored_bytes_two_references'])} |
{bytes_text(r['stored_bytes_after_purging_one'])} | {r['survivor_passes_sha256']} | {r['soft_delete_then_restore_passes_sha256']} |
{r['hdfs_files_after_purging_last']} |")
    lines += ["", "## H. Replication", "", "| Step | Factor | Length | Space consumed | Multiplier | fsck average | Under-replicated
| Converged in |", "|---|---:|---:|---:|---:|---:|---:|---:|"]
    for r in h["rows"]:
        lines.append(f"| {r['step']} | {r['factor']} | {bytes_text(r['length_bytes'])} | {bytes_text(r['space_consumed_bytes'])} |
{r['multiplier']:.2f} | {r['average_block_replication']} | {r['under_replicated_blocks']} | {f'{r["converged_in_s.mean"]:.1f} s' if
r.get('converged_in_s.mean') else 'N/A'} |")
    lines += ["", f"Requesting a factor above the live DataNodes was refused: \"{h['extra']['rejected_over_limit']}\"", "", "## I.
Before and after optimization", "", "| Mode | Files | Logical | Stored before / after | NameNode space before / after | Disk (du)
before / after | HDFS files before / after | Files passing SHA-256 | Optimize time |",
"|---|---:|---:|---:|---:|---:|---:|---:|---:|"]
    for r in i["rows"]:
        lines.append(
            f"| {r['mode']} {size_label(r['chunk_size']) if r['chunk_size'] else ''} | {r['files']} |
{bytes_text(r['logical_bytes'])} | {bytes_text(r['stored_before'])} / {bytes_text(r['stored_after'])} |
{bytes_text(r['hdfs_space_consumed_before'])} / {bytes_text(r['hdfs_space_consumed_after'])} | "
            f"{bytes_text(r['disk_used_delta_before.mean'])} / {bytes_text(r['disk_used_delta_after.mean'])} |
{r['hdfs_files_before']} / {r['hdfs_files_after']} | {r['verification.files_passed']} of {r['verification.files_checked']} |
{r['optimize_duration_s.mean']:.2f} s |"
        )
    failure_path = RESULTS / "J_failure_handling.json"
    if failure_path.exists():
        lines += ["", "## J. Real failures", "", "```json", json.dumps(json.loads(failure_path.read_text()), indent=2), "```"]
    SUMMARY.write_text("\n".join(lines) + "\n")


def main() -> None:
    chart_exact_duplicates()
    chart_chunk_tradeoff()
    chart_before_after()
    chart_restore()
    chart_boundary_shift()
    write_summary()
    print(f"charts in {CHARTS}; summary in {SUMMARY}")


if __name__ == "__main__":
    main()
```

scripts/make_diagrams.py 401 lines

```
#!/usr/bin/env python3
"""Generate the architecture diagrams as SVG files in diagrams/.

Each diagram describes the implemented system: module, table and path names match the source code.
A box refuses to render text that would not fit, so overflow fails the build rather than the page.
"""

from __future__ import annotations

from html import escape
from pathlib import Path

OUT = Path(__file__).resolve().parent.parent / "diagrams"

INK = "#0b0b0b"
MUTED = "#52514e"
LINE = "#6f6d67"
NEUTRAL = ("#f7f7f4", "#c3c2b7")
HDFS = ("#e4effc", "#2a78d6")
META = ("#fdeee6", "#eb6834")
```

```
ACCENT = ("#2a78d6", "#2a78d6")
GOOD = ("#e3f5ec", "#1baf7a")
FONT = "Helvetica Neue, Helvetica, Arial, sans-serif"
MONO = "Menlo, Consolas, monospace"
CHAR = 0.53  # approximate glyph width as a fraction of font size


def text_width(value: str, size: float) -> float:
    return len(value) * size * CHAR


class Diagram:
    def __init__(self, name: str, width: int, height: int):
        self.name, self.width, self.height = name, width, height
        self.parts: list[str] = []

    def text(self, x, y, value, *, size=12, weight="normal", anchor="middle", color=INK, mono=False, halo=False):
        family = MONO if mono else FONT
        outline = ' paint-order="stroke" stroke="#ffffff" stroke-width="4" stroke-linejoin="round"' if halo else ""
        self.parts.append(
            f'<text x="{x}" y="{y}" font-family="{family}" font-size="{size}" font-weight="{weight}" '
            f'text-anchor="{anchor}" fill="{color}"{outline}>{escape(value)}</text>'
        )

    def box(self, x, y, w, h, title, lines=(), *, style=NEUTRAL, title_size=12.5, line_size=10.5, rx=6, title_color=INK,
mono_lines=False):
        fill, stroke = style
        self.parts.append(f'<rect x="{x}" y="{y}" width="{w}" height="{h}" rx="{rx}" fill="{fill}" stroke="{stroke}" stroke-
width="1.2"/>')
        rows = [title] if isinstance(title, str) else list(title)
        total = len(rows) * (title_size + 3) + len(lines) * (line_size + 3.5)
        cursor = y + (h - total) / 2 + title_size
        for row in rows:
            self._fit(row, title_size, w, "title")
            self.text(x + w / 2, cursor, row, size=title_size, weight="600", color=title_color)
            cursor += title_size + 3
        for line in lines:
            self._fit(line, line_size * (1.1 if mono_lines else 1), w, "line")
            self.text(x + w / 2, cursor + 1, line, size=line_size, color=MUTED if style is not ACCENT else "#ffffff",
mono=mono_lines)
            cursor += line_size + 3.5
        return (x, y, w, h)

    def _fit(self, value, size, width, kind):
        if text_width(value, size) > width - 8:
            raise ValueError(f"{self.name}: {kind} {value!r} does not fit in a box {width} wide")

    def group(self, x, y, w, h, label, *, style=NEUTRAL):
        fill, stroke = style
        self.parts.append(f'<rect x="{x}" y="{y}" width="{w}" height="{h}" rx="8" fill="none" stroke="{stroke}" stroke-width="1"
stroke-dasharray="5 4"/>')
        self.text(x + 10, y + 16, label, size=11, weight="600", anchor="start", color=MUTED)
        _ = fill

    def arrow(self, points, label=None, *, dashed=False, color=LINE, label_at=None, label_anchor="middle", both=False):
        path = " ".join(f"{px},{py}" for px, py in points)
        dash = ' stroke-dasharray="5 4"' if dashed else ""
        start = ' marker-start="url(#arrow-start)"' if both else ""
        self.parts.append(f'<polyline points="{path}" fill="none" stroke="{color}" stroke-width="1.3"{dash} marker-
end="url(#arrow)"{start}/>')
        if label:
            (x1, y1), (x2, y2) = points[0], points[1]
            lx, ly = label_at or ((x1 + x2) / 2, (y1 + y2) / 2 - 5)
            self.text(lx, ly, label, size=10, color=MUTED, anchor=label_anchor, halo=True)

    def diamond(self, cx, cy, w, h, lines, style=NEUTRAL):
        fill, stroke = style
        pts = f"{cx},{cy - h / 2} {cx + w / 2},{cy} {cx},{cy + h / 2} {cx - w / 2},{cy}"
        self.parts.append(f'<polygon points="{pts}" fill="{fill}" stroke="{stroke}" stroke-width="1.2"/>')
        top = cy - (len(lines) - 1) * 7 + 4
        for index, line in enumerate(lines):
            if text_width(line, 11) > w * 0.72:
                raise ValueError(f"{self.name}: decision text {line!r} does not fit")
            self.text(cx, top + index * 14, line, size=11, weight="600")

    def write(self):
        OUT.mkdir(exist_ok=True)
        defs = (
            '<defs>'
            f'<marker id="arrow" viewBox="0 0 10 10" refX="9" refY="5" markerWidth="7" markerHeight="7" orient="auto"><path d="M0
0L10 5L0 10z" fill="{LINE}"/></marker>'
            f'<marker id="arrow-start" viewBox="0 0 10 10" refX="1" refY="5" markerWidth="7" markerHeight="7" orient="auto"><path
d="M10 0L0 5L10 10z" fill="{LINE}"/></marker>'
            '</defs>'
        )
        svg = f'<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 {self.width} {self.height}" width="{self.width}"
height="{self.height}">{defs}<rect width="100%" height="100%" fill="#ffffff"/>{"".join(self.parts)}</svg>\n'
        (OUT / f"{self.name}.svg").write_text(svg)


def system_architecture():
    d = Diagram("01_system_architecture", 900, 620)
    d.box(40, 20, 820, 54, "Browser: Next.js + React + TanStack Query", ["10 pages · upload progress via XMLHttpRequest · charts
```

```
from backend data"])
    d.arrow([(450, 74), (450, 104)], "REST /api/v1 (JSON, multipart, streamed download)", label_at=(460, 93), label_anchor="start",
both=True)
    d.box(40, 104, 820, 42, "FastAPI: files · dedup · system routers", ["Validation, pagination, error mapping, CORS, optional X-
API-Key"])
    d.arrow([(450, 146), (450, 170)])
    d.group(40, 170, 820, 232, "Services (backend/app/services)")
    row1 = [("ingest", "Hash, look up, store new"), ("content_store", "Staged CAS, manifests"), ("optimizer", "Dry run, safe
conversion"), ("lifecycle", "Delete, GC, recovery")]
    row2 = [("integrity", "SHA-256 verification"), ("replication", "Set and verify factor"), ("monitor", "NameNode JMX, fsck"),
("analytics", "Derived and measured")]
    row3 = [("jobs", "Background job runner"), ("datasets", "Provenance, workloads"), ("experiments", "Isolated measured runs"),
("audit", "Append-only events")]
    for row_index, row in enumerate((row1, row2, row3)):
        for col, (title, line) in enumerate(row):
            d.box(56 + col * 200, 194 + row_index * 68, 188, 58, title, [line])
    d.arrow([(240, 402), (240, 440)], "SQLAlchemy", label_at=(248, 424), label_anchor="start", both=True)
    d.box(60, 440, 360, 62, "SQLite metadata database (WAL)", ["Files, objects, chunks, manifests, jobs, integrity, audit"],
style=META)
    d.arrow([(660, 402), (660, 440)], "WebHdfsClient", label_at=(668, 424), label_anchor="start", both=True)
    d.box(480, 440, 360, 62, "WebHDFS REST (NameNode :9870, DataNode :9864)", ["Two-step create, open, rename, delete,
setreplication"], style=HDFS, title_size=11.5)
    d.arrow([(660, 502), (660, 536)])
    d.box(420, 536, 440, 62, "Apache Hadoop 3.5.0 HDFS in Docker", ["1 NameNode + 3 DataNodes · replication factor 3 · persistent
volumes"], style=HDFS)
    d.box(40, 536, 340, 62, "Authentic dataset (local)", ["Project Gutenberg files + provenance manifest"])
    d.write()


def hdfs_cluster():
    d = Diagram("02_hdfs_cluster", 900, 470)
    d.box(40, 170, 170, 80, "HDFS Optimizer", ["backend on host", "WebHDFS client"])
    d.group(250, 20, 620, 430, "Colima VM · Docker Compose network")
    d.box(430, 50, 260, 92, "NameNode (hostname: namenode)", ["Namespace and block map in memory", "RPC :8020 · HTTP :9870
(published)", "fsimage + edits on volume namenode-data"], style=HDFS, title_size=11.5)
    for index in range(3):
        x = 270 + index * 200
        d.box(x, 270, 180, 118, f"DataNode {index + 1} (datanode{index + 1})", ["HTTP :9864, published", f"on host port 1986{4 +
index}", f"volume datanode{index + 1}-data", "Blocks + .meta checksums"], style=HDFS, title_size=11)
        d.arrow([(x + 90, 270), (x + 90 if index == 1 else 560 + (index - 1) * 60, 142)], None, dashed=True)
    d.text(758, 170, "Heartbeat every 3 s", size=10, color=MUTED, anchor="start")
    d.text(758, 184, "and block reports", size=10, color=MUTED, anchor="start")
    d.arrow([(210, 190), (430, 96)], "1. metadata, then 307 redirect", label_at=(305, 118))
    d.arrow([(210, 230), (270, 310)], "2. read / write bytes", label_at=(150, 288))
    d.arrow([(450, 345), (470, 345)], both=True)
    d.arrow([(650, 345), (670, 345)], both=True)
    d.text(560, 415, "Write pipeline: each block is copied to 3 DataNodes (dfs.replication = 3)", size=11, color=MUTED)
    d.text(560, 432, "Light profile: docker-compose.light.yml runs 2 DataNodes with factor 2", size=10.5, color=MUTED)
    d.write()


def upload_pipeline():
    d = Diagram("03_upload_pipeline", 900, 440)
    steps = [
        ("Receive", ["multipart upload", "validate name, folder"]),
        ("Spool + hash", ["streamed to disk", "SHA-256, size"]),
        ("Look up", ["objects, manifests", "by content hash"]),
    ]
    for i, (title, lines) in enumerate(steps):
        d.box(30 + i * 215, 30, 185, 76, title, lines)
        if i < 2:
            d.arrow([(215 + i * 215, 68), (245 + i * 215, 68)])
    d.arrow([(645, 68), (675, 68)])
    d.diamond(770, 68, 190, 92, ["Content", "already stored?"])
    d.arrow([(865, 68), (865, 180)], "yes", label_at=(855, 150), label_anchor="end")
    d.box(660, 180, 220, 76, "Reference only", ["Check it exists in HDFS", "Repair from upload if missing"], style=GOOD)
    d.arrow([(770, 114), (770, 150), (130, 150), (130, 180)], "no: write new bytes", label_at=(450, 143))
    labels = [
        ("1. Stage", ["CREATE", "/staging/<uuid>"]),
        ("2. Check", ["length and", "SHA-256 read-back"]),
        ("3. Publish", ["atomic RENAME to", "/objects/ab/cd/<sha>"]),
    ]
    for i, (title, lines) in enumerate(labels):
        d.box(30 + i * 190, 180, 165, 76, title, lines, style=HDFS)
        if i < 2:
            d.arrow([(195 + i * 190, 218), (220 + i * 190, 218)])
    d.arrow([(112 + 380, 256), (112 + 380, 320), (360, 320)])
    d.arrow([(770, 256), (770, 346), (560, 346)])
    d.box(160, 300, 400, 76, "4. Commit metadata in one SQLite transaction", ["LogicalFile row, object or manifest pointer", "Audit
event, performance samples, storage snapshot"], style=META)
    d.text(450, 412, "Crash before step 4: only an unreferenced HDFS file remains, which reconciliation can find. Metadata never
points at missing bytes.", size=11, color=MUTED)
    d.write()


def dedup_pipeline():
    d = Diagram("04_dedup_decision", 900, 500)
    d.box(350, 16, 200, 44, "Upload with SHA-256 = h")
    d.arrow([(450, 60), (450, 80)])
    d.diamond(450, 124, 200, 88, ["Mode =", "raw?"])
    d.arrow([(550, 124), (720, 124), (720, 170)], "yes", label_at=(620, 117))
    d.box(620, 170, 200, 62, "Store private copy", ["/raw/ab/<file id>", "Baseline: no dedup"])
    d.arrow([(450, 168), (450, 200)], "no", label_at=(460, 188), label_anchor="start")
```

```
    d.diamond(450, 244, 230, 88, ["Active form of h", "already stored?"])
    d.arrow([(335, 244), (170, 244), (170, 290)], "yes", label_at=(250, 237))
    d.box(50, 290, 240, 76, "Add a reference", ["New LogicalFile points at the", "existing object or manifest", "0 bytes written"],
style=GOOD)
    d.arrow([(450, 288), (450, 320)], "no", label_at=(460, 308), label_anchor="start")
    d.diamond(450, 362, 180, 80, ["Mode?"])
    d.arrow([(360, 362), (325, 362), (325, 420)], "file", label_at=(344, 355))
    d.box(205, 420, 240, 62, "Store one object", ["/objects/ab/cd/<h>"], style=HDFS)
    d.arrow([(540, 362), (590, 362), (590, 420)], "chunk", label_at=(565, 355))
    d.box(470, 420, 400, 62, "Split at fixed size, hash each chunk", ["Store only unseen chunks, then a JSON manifest"], style=HDFS)
    d.write()


def chunk_model():
    d = Diagram("05_chunk_storage_model", 900, 400)
    d.text(90, 30, "Logical files", size=12, weight="600", color=MUTED)
    d.text(350, 30, "Manifests (ordered chunk lists)", size=12, weight="600", color=MUTED)
    d.text(730, 30, "Unique chunks in HDFS", size=12, weight="600", color=MUTED)
    d.box(20, 50, 150, 44, "pride.txt")
    d.box(20, 110, 150, 44, "pride-copy.txt")
    d.box(20, 240, 150, 44, "pride-edited.txt")
    colors = ["c1", "c2", "c3", "c4"]
    cell = 56
    def strip(y, labels, height=44):
        d.box(220, y, 300, height + 24, "", ())
        for i, label in enumerate(labels):
            style = META if label == "c5" else HDFS
            d.box(232 + i * (cell + 14), y + 14, cell, height - 4, label, style=style, title_size=11, rx=3)
    positions = {"c1": 60, "c2": 124, "c3": 188, "c4": 252, "c5": 316}
    # Links go first, so the boxes sit on top of them.
    for i, label in enumerate(colors):
        d.arrow([(232 + i * (cell + 14) + cell / 2, 100), (640, positions[label] + 25)], dashed=True, color="#b9b8ae")
    for i, label in enumerate(["c1", "c2", "c5", "c4"]):
        d.arrow([(232 + i * (cell + 14) + cell / 2, 260), (640, positions[label] + 25)], dashed=True, color="#b9b8ae")
    strip(68, colors)
    strip(226, ["c1", "c2", "c5", "c4"])
    d.arrow([(170, 72), (220, 95)])
    d.arrow([(170, 132), (220, 108)])
    d.arrow([(170, 262), (220, 260)])
    for label, y in positions.items():
        d.box(640, y, 230, 50, f"/chunks/ab/cd/<sha of {label}>", ["1 HDFS file, replicated separately"], style=META if label ==
"c5" else HDFS, title_size=10.5, line_size=9.5)
    d.text(370, 340, "Two identical files share one manifest. A file with", size=11, color=MUTED)
    d.text(370, 356, "one changed chunk adds one new chunk (c5).", size=11, color=MUTED)
    d.text(370, 380, "Chunks are application-level files, not HDFS blocks.", size=11, weight="600")
    d.write()


def reconstruction():
    d = Diagram("06_file_reconstruction", 900, 330)
    d.box(30, 40, 170, 70, "GET /files/{id}/download", ["Preflight: content exists"], title_size=11)
    d.arrow([(200, 75), (235, 75)])
    d.box(235, 40, 170, 70, "Load manifest", ["file_chunks in", "ordinal order"], style=META)
    d.arrow([(405, 75), (440, 75)])
    d.box(440, 40, 180, 70, "For each chunk", ["OPEN via WebHDFS", "from a live replica"], style=HDFS)
    d.arrow([(620, 75), (655, 75)])
    d.diamond(740, 75, 170, 92, ["Chunk hash", "correct?"])
    d.arrow([(740, 121), (740, 175)], "no", label_at=(750, 150), label_anchor="start")
    d.box(640, 175, 200, 62, "Abort the response", ["Client sees a failed download"], style=META)
    d.arrow([(740, 29), (740, 20), (530, 20), (530, 40)], "yes: next chunk", label_at=(635, 15))
    d.arrow([(530, 110), (530, 175)], "stream bytes, update hash", label_at=(520, 150), label_anchor="end")
    d.box(400, 175, 200, 62, "Whole-file SHA-256", ["computed while streaming"])
    d.arrow([(400, 206), (330, 206)])
    d.box(120, 175, 210, 62, "Compare with file hash", ["Mismatch aborts; match records", "a restore sample and audit event"],
line_size=9.5)
    d.text(450, 280, "Whole-file and raw modes stream one HDFS file through the same final comparison.", size=11, color=MUTED)
    d.text(450, 298, "An HDFS outage is reported as an error, never as a failed integrity check.", size=11, color=MUTED)
    d.write()


def replication_flow():
    d = Diagram("07_replication_workflow", 900, 400)
    d.box(30, 30, 180, 60, "Request factor n", ["POST /cluster/replication"])
    d.arrow([(210, 60), (250, 60)])
    d.diamond(345, 60, 190, 92, ["n ≤ live", "DataNodes?"])
    d.arrow([(345, 106), (345, 156)], "no", label_at=(355, 134), label_anchor="start")
    d.box(250, 156, 190, 54, "Reject: 422", ["Would stay under-replicated"], style=META)
    d.arrow([(440, 60), (480, 60)], "yes", label_at=(458, 52))
    d.diamond(570, 60, 180, 92, ["Lowers", "protection?"])
    d.arrow([(570, 106), (570, 156)], "yes, not confirmed", label_at=(580, 134), label_anchor="start")
    d.box(475, 156, 190, 54, "Reject: 409", ["Ask for explicit confirmation"], style=META)
    d.arrow([(660, 60), (690, 60)], "ok", label_at=(675, 52))
    d.box(690, 30, 180, 60, "Background job", ["SETREPLICATION per file"], style=HDFS)
    d.arrow([(780, 90), (780, 250)])
    d.box(600, 250, 270, 60, "HDFS converges asynchronously", ["Copies new replicas or deletes excess ones"], style=HDFS)
    d.arrow([(600, 280), (530, 280)])
    d.diamond(430, 280, 200, 92, ["fsck and block", "locations = n?"])
    d.arrow([(430, 326), (430, 366), (620, 366), (720, 366), (720, 310)], "no: poll every 3 s; the time limit scales with blocks",
label_at=(585, 360))
    d.arrow([(330, 280), (250, 280)], "yes", label_at=(290, 272))
    d.box(30, 250, 220, 60, "Verified", ["Record measured space", "before and after"], style=GOOD)
    d.text(160, 350, "Timeout: recorded as did not converge.", size=11, color=MUTED)
    d.write()
```

```
def er_diagram():
    d = Diagram("08_er_diagram", 900, 640)

    def entity(x, y, name, fields, style=NEUTRAL, width=210):
        height = 24 + len(fields) * 15 + 8
        fill, stroke = style
        d.parts.append(f'<rect x="{x}" y="{y}" width="{width}" height="{height}" rx="5" fill="#ffffff" stroke="{stroke}" stroke-
width="1.2"/>')
        d.parts.append(f'<rect x="{x}" y="{y}" width="{width}" height="22" rx="5" fill="{fill}" stroke="{stroke}" stroke-
width="1.2"/>')
        d.text(x + width / 2, y + 15, name, size=11.5, weight="600")
        for i, field in enumerate(fields):
            if text_width(field, 10) > width - 10:
                raise ValueError(f"field {field!r} does not fit")
            d.text(x + 8, y + 36 + i * 15, field, size=10, anchor="start", color=MUTED, mono=True)
        return x, y, width, height

    entity(340, 20, "files", ["id PK", "name, folder (unique if active)", "sha256, size_bytes", "storage_mode, status", "raw_path",
"object_id FK", "manifest_id FK", "dataset_source_id FK", "provenance_label"], META, 230)
    entity(40, 40, "objects", ["id PK", "sha256 UNIQUE", "size_bytes, hdfs_path", "state"], HDFS)
    entity(650, 20, "file_manifests", ["id PK", "file_sha256, chunk_size UQ", "chunk_count, manifest_bytes", "hdfs_path, state"],
HDFS)
    entity(650, 170, "file_chunks", ["id PK", "manifest_id FK", "chunk_id FK", "ordinal (unique per manifest)", "offset_bytes"],
HDFS)
    entity(650, 320, "chunks", ["id PK", "sha256 UNIQUE", "size_bytes, hdfs_path", "state"], HDFS)
    entity(40, 190, "dataset_sources", ["id PK", "title, ebook_id, source_url", "download_url, downloaded_at", "sha256, kind",
"derived_from_id FK", "transformation"], NEUTRAL, 230)
    entity(340, 230, "integrity_checks", ["id PK", "file_id FK", "ok, expected, actual sha256", "missing, corrupt, duration_ms"],
NEUTRAL, 230)
    entity(340, 360, "deduplication_jobs", ["id PK", "kind, mode, chunk_size", "status, progress", "params, result, error",
"resumed_from_id"], NEUTRAL, 230)
    entity(340, 510, "optimization_history", ["id PK", "job_id FK", "before, after (JSON)", "files_processed, duration_ms"],
NEUTRAL, 230)
    entity(40, 340, "replication_operations", ["id PK", "previous_factor, requested", "status, files_changed",
"space_consumed_before/after"], NEUTRAL, 230)
    entity(40, 470, "audit_events", ["id PK", "occurred_at, event_type", "severity, entity_id", "message, details"], NEUTRAL, 230)
    entity(650, 470, "Other supporting tables", ["storage_snapshots", "performance_samples", "experiment_runs", "app_settings"],
NEUTRAL)

    d.arrow([(340, 85), (250, 85)], "0..1", label_at=(295, 78))
    d.arrow([(570, 70), (650, 60)], "0..1", label_at=(610, 58))
    d.arrow([(755, 112), (755, 170)], "1..n", label_at=(765, 145), label_anchor="start")
    d.arrow([(755, 277), (755, 320)], "n..1", label_at=(765, 302), label_anchor="start")
    d.arrow([(340, 150), (270, 195)], "0..1", label_at=(290, 165))
    d.arrow([(455, 187), (455, 230)], "1..n", label_at=(465, 212), label_anchor="start")
    d.arrow([(455, 467), (455, 510)], "1..n", label_at=(465, 492), label_anchor="start")
    d.text(450, 628, "A file's pointer is enforced by a CHECK constraint. Foreign keys to content use ON DELETE RESTRICT. References
are counted by query, never stored.", size=10.5, color=MUTED)
    d.write()


def data_flow():
    d = Diagram("09_data_flow", 900, 460)
    d.box(30, 150, 130, 70, "User", ["via browser"])
    d.box(30, 365, 130, 60, "Project Gutenberg", ["mirror"], title_size=11)
    rows = ["P1 Ingest", "P2 Optimize", "P3 Restore and verify", "P4 Delete and GC", "P5 Analytics", "P6 Dataset import"]
    y = {name: 20 + index * 72 for index, name in enumerate(rows)}
    # Flows between processes and the two stores go first, so boxes sit on top of them.
    d1, d2 = (680, 110), (680, 300)
    links = {
        "P1 Ingest": ("both", "both"),
        "P2 Optimize": ("both", "both"),
        "P3 Restore and verify": ("read", "read"),
        "P4 Delete and GC": ("both", "both"),
        "P5 Analytics": ("read", "read"),
        "P6 Dataset import": ("write", None),
    }
    for name, (to_meta, to_hdfs) in links.items():
        start = (450, y[name] + 28)
        if to_meta == "read":
            d.arrow([d1, start])
        elif to_meta:
            d.arrow([start, d1], both=to_meta == "both")
        if to_hdfs == "read":
            d.arrow([d2, start])
        elif to_hdfs:
            d.arrow([start, d2], both=to_hdfs == "both")
    for name in rows:
        d.box(250, y[name], 200, 56, name, rx=28, title_size=11.5)
    d.box(680, 70, 190, 80, "D1 Metadata", ["SQLite: files, content,", "jobs, audit"], style=META)
    d.box(680, 260, 190, 80, "D2 HDFS", ["Objects, chunks,", "manifests, raw copies"], style=HDFS)
    for name, label in (("P1 Ingest", "files"), ("P2 Optimize", "start job"), ("P3 Restore and verify", "download"), ("P4 Delete and
GC", "delete"), ("P5 Analytics", "reports")):
        target = y[name] + 28
        d.arrow([(160, 185), (250, target)], label, label_at=(222, 185 + (target - 185) * 0.72 - 3))
    d.arrow([(160, 395), (250, y["P6 Dataset import"] + 28)], "books", label_at=(205, 392))
    d.text(775, 372, "Writes follow a fixed order:", size=10.5, color=MUTED)
    d.text(775, 387, "bytes to D2, then metadata to D1", size=10.5, color=MUTED)
    d.text(775, 412, "P6 records provenance, then", size=10.5, color=MUTED)
    d.text(775, 427, "stores files through P1", size=10.5, color=MUTED)
    d.write()
```

```python
def sequence():
    d = Diagram("10_sequence_upload", 900, 590)
    actors = [("Browser", 80), ("FastAPI", 250), ("SQLite", 420), ("NameNode", 610), ("DataNode", 810)]
    for name, x in actors:
        style = META if name == "SQLite" else HDFS if "Node" in name else NEUTRAL
        d.box(x - 60, 16, 120, 34, name, style=style)
        d.parts.append(f'<line x1="{x}" y1="50" x2="{x}" y2="552" stroke="#c3c2b7" stroke-width="1"/>')
    steps = [
        (80, 250, "POST /files/upload (multipart)", False),
        (250, 250, "spool to disk, compute SHA-256", False),
        (250, 420, "find active object by hash", False),
        (420, 250, "none found", True),
        (250, 610, "PUT CREATE /staging/<uuid>", False),
        (610, 250, "307 redirect to a DataNode", True),
        (250, 810, "PUT bytes", False),
        (810, 250, "201 after the write pipeline (3 replicas)", True),
        (250, 810, "GET staged bytes, check SHA-256", False),
        (250, 610, "PUT MKDIRS, then RENAME to /objects/ab/cd/<sha>", False),
        (610, 250, "true", True),
        (250, 420, "COMMIT file, object, audit, samples", False),
        (250, 80, "201 stored_new", True),
    ]
    y = 78
    for x1, x2, label, dashed in steps:
        if x1 == x2:
            d.parts.append(f'<rect x="{x1 - 5}" y="{y - 12}" width="10" height="20" fill="#f0efec" stroke="#c3c2b7"/>')
            d.text(x1 + 12, y + 2, label, size=10.5, anchor="start", color=MUTED, halo=True)
        else:
            d.arrow([(x1, y), (x2, y)], None, dashed=dashed)
            middle = (x1 + x2) / 2
            d.text(middle, y - 6, label, size=10.5, color=INK if not dashed else MUTED, halo=True)
        y += 37
    d.text(450, 574, "For a duplicate, the lookup finds an object, HDFS confirms its length, and only the COMMIT step runs: no bytes
are written.", size=10.5, color=MUTED)
    d.write()


def main() -> None:
    for build in (system_architecture, hdfs_cluster, upload_pipeline, dedup_pipeline, chunk_model, reconstruction, replication_flow,
er_diagram, data_flow, sequence):
        build()
    print(f"wrote {len(list(OUT.glob('*.svg')))} diagrams to {OUT}")


if __name__ == "__main__":
    main()
```

scripts/run_benchmarks.py 203 lines

```python
#!/usr/bin/env python3
"""Run the storage experiments on the real cluster and write CSV and JSON results.

Usage:  backend/.venv/bin/python scripts/run_benchmarks.py [--iterations 3] [--only B_exact_duplicates]

Each scenario runs in private HDFS directories and never touches managed data. Durations and throughputs are
repeated and reported as mean, min and max. Byte counts must be identical in every iteration; if they are not,
the run is flagged, because that would mean the system is not deterministic.
"""

from __future__ import annotations

import argparse
import csv
import inspect
import json
import platform
import statistics
import subprocess
import sys
import time
from datetime import UTC, datetime
from pathlib import Path

PROJECT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(PROJECT / "backend"))

from app.core.config import Settings  # noqa: E402
from app.core.db import Base, make_engine, make_session_factory  # noqa: E402
from app.hdfs.webhdfs import WebHdfsClient  # noqa: E402
from app.services import experiments  # noqa: E402
from app.services.context import AppContext, StoragePaths  # noqa: E402
from app.services.monitor import ClusterMonitor  # noqa: E402

RESULTS = PROJECT / "benchmarks" / "results"
TIMING_SUFFIXES = ("_s", "_mib_s")
# Disk readings include background DataNode activity. Directory counts depend on randomly generated file
# ids, because raw paths are prefixed by them. Neither affects stored data, so both are reported as a range.
ALLOWED_TO_VARY = ("disk_used_delta", "directory_count", "namespace_directories", "namespace_objects")


def disk_used_on_datanodes() -> int:
    """Bytes actually allocated on DataNode volumes: block files, checksum files and filesystem rounding."""
```

```
    total = 0
    for node in ("datanode1", "datanode2", "datanode3"):
        output = subprocess.run(
            ["docker", "compose", "exec", "-T", node, "du", "-s", "--block-size=1", "/data/dn/current"],
            cwd=PROJECT, capture_output=True, text=True, check=True,
        ).stdout
        total += int(output.split()[0])
    return total


def build_context(tmp_db: Path) -> AppContext:
    settings = Settings(_env_file=None, database_path=tmp_db)
    engine = make_engine(tmp_db)
    Base.metadata.create_all(engine)
    return AppContext(
        settings=settings,
        session_factory=make_session_factory(engine),
        hdfs=WebHdfsClient(settings.namenode_url, datanode_map=settings.datanode_map, timeout_seconds=120),
        paths=StoragePaths(root="/hdfs-optimizer-experiments/bench-parent"),
        monitor=ClusterMonitor(settings.namenode_url),
    )


def flatten(row: dict) -> dict:
    flat: dict = {}
    for key, value in row.items():
        if isinstance(value, dict):
            flat.update({f"{key}.{inner}": val for inner, val in value.items()})
        else:
            flat[key] = value
    return flat


def is_timing(key: str) -> bool:
    return key.endswith(TIMING_SUFFIXES)


def aggregate(iterations: list[list[dict]]) -> tuple[list[dict], list[str]]:
    """Combine aligned rows. Timings become mean/min/max; everything else must not change."""
    aggregated, inconsistencies = [], []
    for rows in zip(*iterations, strict=True):
        rows = [flatten(r) for r in rows]
        out: dict = {}
        for key in rows[0]:
            values = [r.get(key) for r in rows]
            numeric = [v for v in values if isinstance(v, int | float) and not isinstance(v, bool)]
            if is_timing(key) and numeric:
                out[f"{key}.mean"] = statistics.mean(numeric)
                out[f"{key}.min"] = min(numeric)
                out[f"{key}.max"] = max(numeric)
            elif any(tag in key for tag in ALLOWED_TO_VARY) and numeric:
                out[f"{key}.mean"] = statistics.mean(numeric)
                out[f"{key}.min"] = min(numeric)
                out[f"{key}.max"] = max(numeric)
            else:
                out[key] = values[0]
                if any(v != values[0] for v in values):
                    inconsistencies.append(f"{key}: {values}")
        aggregated.append(out)
    return aggregated, inconsistencies


def write_csv(path: Path, rows: list[dict]) -> None:
    columns: list[str] = []
    for row in rows:
        columns.extend(c for c in row if c not in columns)
    with open(path, "w", newline="") as handle:
        writer = csv.DictWriter(handle, fieldnames=columns)
        writer.writeheader()
        writer.writerows(rows)


def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--iterations", type=int, default=3)
    parser.add_argument("--only", action="append", help="Scenario id; repeatable")
    args = parser.parse_args()

    RESULTS.mkdir(parents=True, exist_ok=True)
    work = RESULTS / ".work"
    work.mkdir(exist_ok=True)
    ctx = build_context(work / "bench-parent.db")
    status = ctx.monitor.status()
    if not status["reachable"]:
        print(f"HDFS is not reachable: {status['error']}")
        return 1

    environment = {
        "run_started": datetime.now(UTC).isoformat(),
        "machine": platform.machine(),
        "os": platform.platform(),
        "python": platform.python_version(),
        "hadoop_version": status["namenode"]["software_version"],
```

```
        "live_datanodes": status["datanodes"]["live"],
        "default_replication": ctx.hdfs.default_replication(),
        "deployment": "All daemons in Docker containers inside one Colima VM on one laptop; the backend runs on the host.",
    }
    (RESULTS / "environment.json").write_text(json.dumps(environment, indent=2) + "\n")
    print(json.dumps(environment, indent=2))

    # Destructive-free, and repeated only where a timing is meaningful.
    plan = {
        "A_unique_files": args.iterations,
        "B_exact_duplicates": args.iterations,
        "C_many_names": args.iterations,
        "D_chunk_sizes": args.iterations,
        "D2_boundary_shift": 1,
        "E_roundtrip": args.iterations,
        "F_shared_deletion": 1,
        "G_restore_after_optimize": 1,
        "H_replication": 1,
        "I_before_after": args.iterations,
    }
    failures = 0
    for scenario, count in plan.items():
        if args.only and scenario not in args.only:
            continue
        _, function = experiments.SCENARIOS[scenario]
        wants_probe = "physical_probe" in inspect.signature(function).parameters
        iterations, last = [], {}
        started = time.perf_counter()
        print(f"--- {scenario}: {count} iteration(s)", flush=True)
        try:
            for index in range(count):
                result = function(ctx, {}, physical_probe=disk_used_on_datanodes) if wants_probe else function(ctx, {})
                iterations.append(result["rows"])
                last = result
                print(f"    iteration {index + 1} done at {time.perf_counter() - started:.1f} s", flush=True)
        except Exception as exc:  # noqa: BLE001 - recorded in the results; a failed experiment is never hidden
            failures += 1
            message = getattr(exc, "message", repr(exc))
            print(f"    FAILED: {message}", flush=True)
            (RESULTS / f"{scenario}.json").write_text(json.dumps({"scenario": scenario, "status": "failed", "error": message},
indent=2) + "\n")
            continue
        rows, inconsistencies = aggregate(iterations)
        for line in inconsistencies:
            print(f"    INCONSISTENT {line}", flush=True)
        document = {
            "scenario": scenario,
            "title": experiments.SCENARIOS[scenario][0],
            "status": "succeeded",
            "description": last.get("description"),
            "iterations": count,
            "inconsistent_values": inconsistencies,
            "extra": {k: v for k, v in last.items() if k not in {"rows", "description"}},
            "rows": rows,
            "environment": environment,
        }
        (RESULTS / f"{scenario}.json").write_text(json.dumps(document, indent=2, default=str) + "\n")
        write_csv(RESULTS / f"{scenario}.csv", rows)
    for leftover in work.iterdir():
        leftover.unlink()
    work.rmdir()
    print(f"finished; {failures} scenario(s) failed")
    return 1 if failures else 0


if __name__ == "__main__":
    raise SystemExit(main())
```

scripts/run_failure_experiments.py 310 lines

```
#!/usr/bin/env python3
"""Experiment J: real failures. Stops real containers and kills a real backend process.

Usage:  backend/.venv/bin/python scripts/run_failure_experiments.py

  J1  A DataNode stops: stored files stay readable, and HDFS re-replication behaviour is observed.
  J2  The NameNode stops: uploads fail cleanly with 503 and leave no metadata; service resumes after restart.
  J3  The backend is killed with SIGKILL during optimization: nothing is lost and the job can be resumed.

Only this project's containers and a backend process started by this script are affected.
Results go to benchmarks/results/J_failure_handling.json.
"""

from __future__ import annotations

import hashlib
import json
import os
import re
import shutil
import signal
import subprocess
import tempfile
import time
```

```
import uuid
from datetime import UTC, datetime
from pathlib import Path

import httpx

PROJECT = Path(__file__).resolve().parent.parent
BACKEND = PROJECT / "backend"
DATASET = PROJECT / "datasets" / "raw" / "originals"
RESULTS = PROJECT / "benchmarks" / "results" / "J_failure_handling.json"
NAMENODE = "http://127.0.0.1:9870"
TEST_ROOT = f"/hdfs-optimizer-failure-tests/{uuid.uuid4().hex}"


def compose(*args: str) -> None:
    subprocess.run(["docker", "compose", *args], cwd=PROJECT, check=True, capture_output=True, text=True)


def jmx() -> dict | None:
    try:
        response = httpx.get(f"{NAMENODE}/jmx?qry=Hadoop:service=NameNode,name=FSNamesystem", timeout=3)
        return response.json()["beans"][0]
    except (httpx.HTTPError, ValueError, KeyError, IndexError):
        return None


def fsck(path: str) -> str:
    return httpx.get(f"{NAMENODE}/fsck", params={"ugi": "hadoop", "path": path}, timeout=60).text


def under_replicated(path: str) -> int:
    match = re.search(r"Under-replicated blocks:\s+(\d+)", fsck(path))
    return int(match.group(1)) if match else 0


def wait_for(predicate, timeout: float, poll: float = 1.0):
    started = time.monotonic()
    while time.monotonic() - started < timeout:
        value = predicate()
        if value:
            return time.monotonic() - started, value
        time.sleep(poll)
    return None, None


class Backend:
    """A private backend process with its own database and HDFS directory."""

    def __init__(self, state: Path, port: int = 8198):
        self.port = port
        self.base = f"http://127.0.0.1:{port}/api/v1"
        self.env = {
            **os.environ,
            "HDFSOPT_ROOT_PATH": TEST_ROOT,
            "HDFSOPT_DATABASE_PATH": str(state / "metadata.db"),
            "HDFSOPT_SPOOL_DIR": str(state / "spool"),
        }
        self.process: subprocess.Popen | None = None

    def start(self) -> None:
        self.process = subprocess.Popen(
            [str(BACKEND / ".venv/bin/uvicorn"), "app.main:app", "--host", "127.0.0.1", "--port", str(self.port), "--log-level",
"warning"],
            cwd=BACKEND, env=self.env, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL,
        )
        elapsed, _ = wait_for(lambda: self._healthy(), 40, 0.5)
        if elapsed is None:
            raise RuntimeError("Backend did not start")

    def _healthy(self) -> bool:
        try:
            return httpx.get(f"{self.base}/health", timeout=3).status_code == 200
        except httpx.HTTPError:
            return False

    def kill(self) -> None:
        self.process.send_signal(signal.SIGKILL)
        self.process.wait()

    def stop(self) -> None:
        if self.process and self.process.poll() is None:
            self.process.terminate()
            self.process.wait(timeout=20)

    def upload(self, path: Path, name: str, mode: str) -> httpx.Response:
        with open(path, "rb") as handle:
            return httpx.post(f"{self.base}/files/upload", files={"file": (name, handle)}, data={"mode": mode}, timeout=120)

    def get(self, path: str, **kwargs) -> httpx.Response:
        return httpx.get(f"{self.base}{path}", timeout=120, **kwargs)

    def post(self, path: str, body=None) -> httpx.Response:
        return httpx.post(f"{self.base}{path}", json=body, timeout=120)
```

```
    def wait_job(self, job_id: str, timeout: float = 300) -> dict:
        deadline = time.monotonic() + timeout
        while time.monotonic() < deadline:
            job = self.get(f"/dedup/jobs/{job_id}").json()
            if job["status"] not in ("queued", "running"):
                return job
            time.sleep(0.5)
        raise TimeoutError(job_id)


def sha256(data: bytes) -> str:
    return hashlib.sha256(data).hexdigest()


def j1_datanode_failure(api: Backend) -> dict:
    source = DATASET / "pg1661.txt"
    expected = sha256(source.read_bytes())
    file_mode = api.upload(source, "datanode-test-whole.txt", "file").json()["file"]["id"]
    chunk_mode = api.upload(DATASET / "pg84.txt", "datanode-test-chunk.txt", "chunk").json()["file"]["id"]
    chunk_expected = sha256((DATASET / "pg84.txt").read_bytes())
    time.sleep(5)

    compose("stop", "datanode3")
    stopped = time.monotonic()
    immediately = sha256(api.get(f"/files/{file_mode}/download").content) == expected

    elapsed, _ = wait_for(lambda: (jmx() or {}).get("NumDeadDataNodes") == 1, 200)
    dead_after = None if elapsed is None else time.monotonic() - stopped
    status = api.get("/cluster/status").json()
    # The "dead" flag is computed from timestamps; the NameNode's heartbeat checker removes the node's replicas
    # from the block map on its own cycle, so fsck reports the loss a little later. Measure that gap.
    under_at_dead = under_replicated(TEST_ROOT)
    elapsed, _ = wait_for(lambda: under_replicated(TEST_ROOT) > 0, 120)
    gap = elapsed
    report = fsck(TEST_ROOT)
    under = [line.strip() for line in report.splitlines() if "Under-replicated blocks" in line or "Missing blocks" in line or
"Average block replication" in line]
    whole_ok = sha256(api.get(f"/files/{file_mode}/download").content) == expected
    chunk_ok = sha256(api.get(f"/files/{chunk_mode}/download").content) == chunk_expected
    over_limit = api.post("/cluster/replication", {"factor": 3})

    compose("start", "datanode3")
    elapsed, _ = wait_for(lambda: (jmx() or {}).get("NumLiveDataNodes") == 3, 120)
    recovered_after = elapsed
    elapsed, _ = wait_for(lambda: under_replicated(TEST_ROOT) == 0, 180)
    return {
        "readable_immediately_after_stop": immediately,
        "seconds_until_namenode_declared_dead": dead_after,
        "cluster_health_while_dead": status.get("health"),
        "live_datanodes_while_dead": status.get("datanodes", {}).get("live"),
        "under_replicated_blocks_when_first_declared_dead": under_at_dead,
        "further_seconds_until_fsck_reports_under_replication": gap,
        "fsck_while_dead": under,
        "whole_file_mode_readable_while_dead": whole_ok,
        "chunk_mode_readable_while_dead": chunk_ok,
        "replication_factor_3_request_while_two_live_http_status": over_limit.status_code,
        "replication_factor_3_request_message": over_limit.json()["error"]["message"],
        "seconds_until_datanode_live_again": recovered_after,
        "seconds_until_no_under_replicated_blocks": elapsed,
    }


def j2_namenode_failure(api: Backend) -> dict:
    source = DATASET / "pg11.txt"
    before = api.get("/files").json()["total"]
    compose("stop", "namenode")
    wait_for(lambda: jmx() is None, 40)

    started = time.perf_counter()
    response = api.upload(source, "during-outage.txt", "file")
    failed_in = time.perf_counter() - started
    files_during = api.get("/files").json()["total"]
    cluster = api.get("/cluster/status").json()
    health = api.get("/health").json()
    stats = api.get("/storage/statistics").json()
    events = [e["event_type"] for e in api.get("/audit/events?page_size=5").json()["items"]]

    compose("start", "namenode")
    restart_elapsed, _ = wait_for(lambda: (jmx() or {}).get("NumLiveDataNodes") == 3, 180)
    safemode_elapsed, _ = wait_for(lambda: _write_ok(api), 120, 2)
    retry = api.upload(source, "after-outage.txt", "file")
    return {
        "upload_http_status_during_outage": response.status_code,
        "upload_error_code": response.json()["error"]["code"],
        "upload_failed_in_seconds": failed_in,
        "logical_files_before": before,
        "logical_files_during_outage": files_during,
        "api_still_answers_health": health["status"],
        "health_reports_hdfs_reachable": health["hdfs_reachable"],
        "cluster_status_reachable": cluster["reachable"],
        "measured_statistics_available_during_outage": stats["measured"]["available"],
        "recent_audit_event_types": events,
```

```
        "seconds_until_namenode_had_3_datanodes": restart_elapsed,
        "seconds_until_writes_accepted": safemode_elapsed,
        "retry_after_restart_http_status": retry.status_code,
        "retry_outcome": retry.json().get("outcome"),
    }


def _write_ok(api: Backend) -> bool:
    try:
        return api.get("/system/diagnostics").json()["ok"]
    except httpx.HTTPError:
        return False


def j3_process_kill(api: Backend) -> dict:
    sources = sorted(p for p in DATASET.glob("*.txt"))
    ids: dict[str, str] = {}
    for copy in range(3):
        for path in sources:
            response = api.upload(path, f"kill-{copy}-{path.name}", "raw")
            ids[response.json()["file"]["id"]] = sha256(path.read_bytes())
    analysis = api.wait_job(api.post("/dedup/analyze", {"mode": "chunk", "chunk_size": 64 * 1024}).json()["job_id"])
    job_id = api.post("/dedup/optimize", {"analysis_job_id": analysis["id"]}).json()["job_id"]

    elapsed, job = wait_for(lambda: _progress(api, job_id), 120, 0.05)
    if elapsed is None:
        raise RuntimeError("Job never made progress")
    api.kill()
    killed_at_progress = f"{job['progress_done']} of {job['progress_total']}"

    api.start()
    after = api.get(f"/dedup/jobs/{job_id}").json()
    catalog = api.get("/files?page_size=200").json()["items"]
    modes: dict[str, int] = {}
    for item in catalog:
        if item["id"] in ids:
            modes[item["storage_mode"]] = modes.get(item["storage_mode"], 0) + 1
    mid_verify = [api.post(f"/files/{file_id}/verify").json()["ok"] for file_id in ids]
    reconcile_before = api.get("/storage/reconcile").json()
    orphan_areas = sorted({path.removeprefix(TEST_ROOT).split("/")[1] for path in reconcile_before["orphans"]})

    resumed_id = api.post(f"/dedup/jobs/{job_id}/resume").json()["job_id"]
    resumed = api.wait_job(resumed_id)
    restored_ok = 0
    for file_id, expected in ids.items():
        if sha256(api.get(f"/files/{file_id}/download").content) == expected:
            restored_ok += 1
    reconcile_after = api.get("/storage/reconcile").json()
    return {
        "files": len(ids),
        "killed_at_progress": killed_at_progress,
        "job_status_after_restart": after["status"],
        "job_message_after_restart": after["message"],
        "storage_modes_after_kill": modes,
        "files_passing_sha256_before_resume": sum(mid_verify),
        "orphans_found_before_resume": reconcile_before["orphan_count"],
        "orphan_areas_before_resume": orphan_areas,
        "missing_found_before_resume": reconcile_before["missing_count"],
        "resumed_job_status": resumed["status"],
        "resumed_job_files_processed": resumed["result"]["files_processed"],
        "files_restored_byte_identical_after_resume": restored_ok,
        "orphans_after_resume": reconcile_after["orphan_count"],
        "missing_after_resume": reconcile_after["missing_count"],
    }


def _progress(api: Backend, job_id: str):
    job = api.get(f"/dedup/jobs/{job_id}").json()
    if job["status"] != "running":
        raise RuntimeError(f"Job ended before it could be interrupted: {job['status']}")
    return job if 3 <= job["progress_done"] < job["progress_total"] - 3 else None


def main() -> int:
    state = Path(tempfile.mkdtemp(prefix="hdfsopt-failure-"))
    api = Backend(state)
    results: dict = {"started": datetime.now(UTC).isoformat(), "scenario": "J_failure_handling"}
    code = 0
    try:
        api.start()
        for name, function in (("J1_datanode_failure", j1_datanode_failure), ("J2_namenode_failure", j2_namenode_failure),
("J3_process_kill", j3_process_kill)):
            print(f"--- {name}", flush=True)
            try:
                results[name] = {"status": "completed", **function(api)}
            except Exception as exc:  # noqa: BLE001 - a failed experiment is recorded, never hidden
                results[name] = {"status": "failed", "error": repr(exc)}
                code = 1
            print(json.dumps(results[name], indent=2), flush=True)
    finally:
        api.stop()
        compose("start", "namenode", "datanode1", "datanode2", "datanode3")
        wait_for(lambda: (jmx() or {}).get("NumLiveDataNodes") == 3, 180)
```

```
        httpx.delete(f"{NAMENODE}/webhdfs/v1{TEST_ROOT.rsplit('/', 1)[0]}", params={"op": "DELETE", "recursive": "true",
"user.name": "hadoop"}, timeout=60)
        shutil.rmtree(state, ignore_errors=True)
        RESULTS.parent.mkdir(parents=True, exist_ok=True)
        RESULTS.write_text(json.dumps(results, indent=2) + "\n")
    return code


if __name__ == "__main__":
    raise SystemExit(main())
```

presentation/source/prepare_crops.py 39 lines

```
#!/usr/bin/env python3
"""Crop real screenshots to the region a slide discusses, so that their text is readable when projected.

Cropping only: no pixel is altered. Regions are in screenshot pixels (left, top, right, bottom).
"""

from pathlib import Path

from PIL import Image

ROOT = Path(__file__).resolve().parents[2]
OUT = Path(__file__).resolve().parent / "crops"

CROPS = {
    "dashboard": ("01_dashboard", (464, 180, 2820, 1490)),
    "upload": ("02_file_upload", (464, 195, 2820, 1170)),
    "file_details": ("16a_file_details_verified", (2115, 0, 2880, 1110)),
    "duplicates": ("04_duplicate_analysis", (464, 190, 2820, 1390)),
    "shared_chunks": ("04b_shared_chunks", (464, 665, 2820, 1650)),
    "optimization": ("06_optimization_complete", (590, 1430, 2800, 2170)),
    "replication": ("08_replication_management", (464, 235, 2820, 1560)),
    "cluster": ("09_cluster_status", (464, 195, 2820, 1350)),
    "provenance": ("17_dataset_provenance", (464, 530, 2820, 1980)),
    "namenode_datanodes": ("11_datanode_status", (150, 0, 2800, 1790)),
}


def main() -> None:
    OUT.mkdir(exist_ok=True)
    for name, (source, box) in CROPS.items():
        with Image.open(ROOT / "screenshots" / f"{source}.png") as image:
            right, bottom = min(box[2], image.width), min(box[3], image.height)
            image.convert("RGB").crop((box[0], box[1], right, bottom)).save(OUT / f"{name}.png", optimize=True)
            print(name, image.size, "->", (right - box[0], bottom - box[1]))


if __name__ == "__main__":
    main()
```

presentation/source/build_deck.js 526 lines

```
#!/usr/bin/env node
/**
 * Builds presentation/final/HDFS_Storage_Optimizer_Presentation.pptx.
 *
 * Text, shapes, diagrams and charts are native and editable. Images are real screenshots from screenshots/.
 * Every figure is read from benchmarks/results or evidence, so the deck cannot disagree with the report.
 */
const fs = require("node:fs");
const path = require("node:path");
const JSZip = require("jszip");
const pptxgen = require("pptxgenjs");

const ROOT = path.resolve(__dirname, "../..");
const OUT = path.join(ROOT, "presentation/final/HDFS_Storage_Optimizer_Presentation.pptx");
const CROPS = path.join(__dirname, "crops");

const results = (name) => JSON.parse(fs.readFileSync(path.join(ROOT, "benchmarks/results", `${name}.json`), "utf8"));
const MIB = 1048576;
const mib = (b, d = 0) => `${(b / MIB).toFixed(d)} MiB`;
const A = results("A_unique_files").rows;
const B = results("B_exact_duplicates").rows;
const C_many = results("C_many_names").rows[0];
const D = results("D_chunk_sizes").rows;
const D2 = results("D2_boundary_shift").rows;
const E = results("E_roundtrip").rows;
const H = results("H_replication").rows;
const I = results("I_before_after").rows;
const J = results("J_failure_handling");
const provenance = JSON.parse(fs.readFileSync(path.join(ROOT, "datasets/provenance/manifest.json"), "utf8"));
const originals = provenance.files.filter((f) => f.kind === "original");
const counts = JSON.parse(fs.readFileSync(path.join(ROOT, "reports/source/generated/test_counts.json"), "utf8"));

const THEME = {
  name: "HDFS Optimizer",
  headFontFace: "Arial",
  bodyFontFace: "Arial",
  colors: {
    dk1: "0B0B0B", lt1: "FFFFFF", dk2: "12161D", lt2: "F0EFEC",
    accent1: "2A78D6", accent2: "EB6834", accent3: "1BAF7A", accent4: "B7D3F6", accent5: "52514E", accent6: "898781",
    hlink: "1C5CAB", folHlink: "52514E",
  },
```

```
};
const HEX = { grid: "E1E0D9", label: "52514E" };

const pres = new pptxgen();
pres.layout = "LAYOUT_WIDE";
pres.theme = { headFontFace: THEME.headFontFace, bodyFontFace: THEME.bodyFontFace };
pres.title = "HDFS Optimizer";
pres.subject = "HDFS-Based Distributed File Replication, Deduplication and Storage Optimization System";
const C = pres.SchemeColor;

pres.defineSlideMaster({
  title: "TITLE_ONLY",
  background: { color: C.background1 },
  objects: [
    { placeholder: { options: { name: "title", type: "title", x: 0.6, y: 0.38, w: 12.13, h: 1.0, fontSize: 30, bold: true, color:
C.text1, align: "left", valign: "top", margin: 0 }, text: "Slide title" } },
    { text: { text: "HDFS Optimizer", options: { x: 0.6, y: 7.05, w: 4, h: 0.25, fontSize: 10, color: C.accent6, margin: 0 } } },
  ],
  slideNumber: { x: 12.23, y: 7.05, w: 0.5, h: 0.25, fontSize: 10, color: C.accent6, align: "right" },
});
pres.defineSlideMaster({
  title: "DARK",
  background: { color: C.text2 },
  objects: [
    { placeholder: { options: { name: "title", type: "title", x: 0.8, y: 1.5, w: 7.3, h: 2.2, fontSize: 44, bold: true, color:
C.background1, align: "left", valign: "top", margin: 0 }, text: "Title" } },
    { placeholder: { options: { name: "body", type: "body", x: 0.8, y: 3.85, w: 7.3, h: 1.1, fontSize: 18, color: C.accent4, align:
"left", valign: "top", margin: 0 }, text: "Subtitle" } },
  ],
});

// -- composition helpers ---------------------------------------------------------------------

function pngSize(file) {
  const data = fs.readFileSync(file);
  return { w: data.readUInt32BE(16), h: data.readUInt32BE(20) };
}

/** Place a real screenshot (cropped, never altered) at a given width, keeping its proportions. Returns its height. */
function screenshot(slide, name, x, y, w, alt) {
  const file = path.join(CROPS, `${name}.png`);
  const size = pngSize(file);
  const h = (size.h / size.w) * w;
  slide.addShape(pres.ShapeType.rect, { x: x - 0.02, y: y - 0.02, w: w + 0.04, h: h + 0.04, fill: { color: C.background2 }, line:
{ color: C.background2 }, objectName: `${name} frame` });
  slide.addImage({ path: file, x, y, w, h, altText: alt, objectName: name });
  return h;
}

function text(slide, value, x, y, w, h, options = {}) {
  slide.addText(value, { x, y, w, h, margin: 0, isTextBox: true, fontSize: 14, color: C.text1, valign: "top", ...options });
}

function box(slide, label, x, y, w, h, { fill = C.background2, color = C.text1, size = 13, bold = true, line, sub } = {}) {
  const runs = sub
    ? [{ text: label, options: { bold, fontSize: size, color, breakLine: true } }, { text: sub, options: { fontSize: Math.max(size -
2, 10), color, bold: false } }]
    : [{ text: label, options: { bold, fontSize: size, color } }];
  slide.addText(runs, { shape: pres.ShapeType.roundRect, rectRadius: 0.05, x, y, w, h, fill: { color: fill }, line: { color: line ??
fill, width: 1 }, align: "center", valign: "middle", margin: 4, objectName: label });
}

function arrow(slide, x1, y1, x2, y2, color = C.accent6) {
  const flipH = x2 < x1;
  const flipV = y2 < y1;
  slide.addShape(pres.ShapeType.line, { x: Math.min(x1, x2), y: Math.min(y1, y2), w: Math.abs(x2 - x1), h: Math.abs(y2 - y1), flipH,
flipV, line: { color, width: 1.5, endArrowType: "triangle" }, objectName: "arrow" });
}

function stat(slide, value, label, x, y, w, { size = 40, color = C.text1, labelColor = C.accent5 } = {}) {
  text(slide, value, x, y, w, 0.8, { fontSize: size, bold: true, color, valign: "bottom" });
  text(slide, label, x, y + 0.86, w, 0.8, { fontSize: 14, color: labelColor });
}

/** The deck's motif, which is also the product's main figure: one whole, split into kept and avoided bytes. */
function segmentBar(slide, x, y, w, kept, { h = 0.34, keptLabel, avoidedLabel, keptColor = C.accent1, avoidedColor = C.accent4 } =
{}) {
  const keptW = w * kept;
  slide.addText(keptLabel ?? "", { shape: pres.ShapeType.roundRect, rectRadius: 0.04, x, y, w: keptW - 0.02, h, fill: { color:
keptColor }, line: { color: keptColor }, fontSize: 12, color: C.background1, bold: true, margin: 4, align: "left", valign: "middle",
objectName: "stored segment" });
  if (kept < 1) {
    slide.addText(avoidedLabel ?? "", { shape: pres.ShapeType.roundRect, rectRadius: 0.04, x: x + keptW + 0.02, y, w: w - keptW -
0.02, h, fill: { color: avoidedColor }, line: { color: avoidedColor }, fontSize: 12, color: C.text1, bold: true, margin: 4, align:
"left", valign: "middle", objectName: "avoided segment" });
  }
}

const newSlide = (title, sectionTitle, notes) => {
  const slide = pres.addSlide({ masterName: "TITLE_ONLY", sectionTitle });
  slide.addText(title, { placeholder: "title" });
  if (notes) slide.addNotes(notes);
  return slide;
```

```
};

const chartOptions = (extra) => ({
  showLegend: false,
  catAxisLabelColor: HEX.label,
  valAxisLabelColor: HEX.label,
  catAxisLabelFontSize: 12,
  valAxisLabelFontSize: 12,
  dataLabelColor: HEX.label,
  dataLabelFontSize: 12,
  legendFontSize: 12,
  legendColor: HEX.label,
  catAxisLabelFontFace: "+mn-lt",
  valAxisLabelFontFace: "+mn-lt",
  dataLabelFontFace: "+mn-lt",
  legendFontFace: "+mn-lt",
  titleFontFace: "+mn-lt",
  titleColor: "0B0B0B",
  titleFontSize: 14,
  valGridLine: { color: HEX.grid, size: 0.75 },
  catGridLine: { style: "none" },
  showValue: true,
  dataLabelPosition: "outEnd",
  ...extra,
});

// -- slides ----------------------------------------------------------------------------------------

// 1. Title
pres.addSection({ title: "Introduction" });
{
  const slide = pres.addSlide({ masterName: "DARK", sectionTitle: "Introduction" });
  slide.addText("HDFS Optimizer", { placeholder: "title" });
  slide.addText("Store it once. Replicate it safely. Prove every byte.", { placeholder: "body" });
  text(slide, "HDFS-Based Distributed File Replication, Deduplication and Storage Optimization System", 0.8, 3.0, 7.0, 0.7,
{ fontSize: 15, color: C.background2 });
  text(slide, "[Student name]  ·  [Roll number]  ·  [Department]  ·  [University]\nGuide: [Supervisor]  ·  Session: [Academic
session]", 0.8, 5.9, 7.3, 0.7, { fontSize: 13, color: C.accent6 });
  // Hero: many logical files resolving to one stored object, drawn with native shapes.
  const rows = [1.4, 2.25, 3.1, 3.95, 4.8];
  ["report.pdf", "report-final.pdf", "report (1).pdf", "copy of report.pdf", "backup.pdf"].forEach((name, index) => {
    const y = rows[index];
    slide.addText(name, { shape: pres.ShapeType.roundRect, rectRadius: 0.05, x: 8.45, y, w: 1.75, h: 0.5, fill: { color: "232A35" },
line: { color: "3A4453", width: 1 }, color: C.accent4, fontSize: 11, align: "center", valign: "middle", margin: 2, objectName:
`logical file ${index + 1}` });
    arrow(slide, 10.2, y + 0.25, 11.05, 3.35, "5B6677");
  });
  slide.addText("One stored copy", { shape: pres.ShapeType.roundRect, rectRadius: 0.08, x: 11.05, y: 2.75, w: 1.5, h: 1.2, fill:
{ color: C.accent1 }, line: { color: C.accent1 }, color: C.background1, fontSize: 13, bold: true, align: "center", valign: "middle",
margin: 4, objectName: "one stored object" });
  slide.addNotes("Introduce the project in one sentence: an application layer over real, unmodified Apache Hadoop that stores
identical content once, leaves HDFS replication alone, and proves every restored file is byte-identical. Replace the bracketed
placeholders with your details before presenting.");
}

// 2. Problem
{
  const slide = newSlide("HDFS protects blocks, but it cannot see duplicate files", "Introduction", "HDFS copies each block for
fault tolerance. That is deliberate. But when the same book is uploaded three times, HDFS treats those as three unrelated files, and
replicates each one. This drawing shows what plain HDFS stores for three identical uploads at replication factor 3: nine block
copies. Only three are needed for the same fault tolerance.");
  const cell = 0.46;
  const gap = 0.08;
  text(slide, "Plain HDFS: three uploads of one book", 0.6, 1.6, 6, 0.35, { fontSize: 16, bold: true });
  ["report.pdf", "report-final.pdf", "copy of report.pdf"].forEach((name, row) => {
    const y = 2.15 + row * 0.82;
    text(slide, name, 0.6, y + 0.08, 2.1, 0.35, { fontSize: 14, color: C.accent5 });
    [0, 1, 2].forEach((col) => slide.addShape(pres.ShapeType.roundRect, { x: 2.8 + col * (cell + gap), y, w: cell, h: cell,
rectRadius: 0.04, fill: { color: row === 0 ? C.accent1 : C.accent2 }, line: { color: row === 0 ? C.accent1 : C.accent2 },
objectName: `replica ${row}-${col}` }));
  });
  text(slide, "9 block copies stored", 2.8, 4.7, 3.5, 0.4, { fontSize: 18, bold: true });
  text(slide, "With HDFS Optimizer", 7.4, 1.6, 5, 0.35, { fontSize: 16, bold: true });
  ["report.pdf", "report-final.pdf", "copy of report.pdf"].forEach((name, row) => {
    const y = 2.15 + row * 0.82;
    text(slide, name, 7.4, y + 0.08, 2.1, 0.35, { fontSize: 14, color: C.accent5 });
    arrow(slide, 9.45, y + 0.23, 10.15, 2.97);
  });
  [0, 1, 2].forEach((col) => slide.addShape(pres.ShapeType.roundRect, { x: 10.2 + col * (cell + gap), y: 2.75, w: cell, h: cell,
rectRadius: 0.04, fill: { color: C.accent1 }, line: { color: C.accent1 }, objectName: `kept replica ${col}` }));
  text(slide, "3 block copies, same fault tolerance", 7.4, 4.7, 5.3, 0.4, { fontSize: 18, bold: true });
  text(slide, "Each square is one replica of one block. Blue copies provide safety. Orange copies come only from duplicate
uploads.", 0.6, 5.55, 12, 0.6, { fontSize: 14, color: C.accent5 });
}

// 3. Why it matters
{
  const raw = B.find((r) => r.copies === 5 && r.mode === "raw");
  const dedup = B.find((r) => r.copies === 5 && r.mode === "file");
  const slide = newSlide("Every accidental copy is multiplied by the replication factor", "Introduction", `This is a measured
example, not an estimate: nine authentic books with five extra copies each. Without deduplication the NameNode reports
${mib(raw.hdfs_space_consumed_bytes)} consumed; with whole-file deduplication, ${mib(dedup.hdfs_space_consumed_bytes)}. The
duplicates were created for the experiment, so this shows the mechanism and does not predict savings on a real system.`);
```

```
  stat(slide, mib(raw.hdfs_space_consumed_bytes), "consumed by 9 books with 5 extra copies each, stored as plain uploads", 0.6,
1.75, 3.7, { color: C.accent2 });
  stat(slide, mib(dedup.hdfs_space_consumed_bytes), "for the same logical files, deduplicated, at the same replication factor", 4.8,
1.75, 3.7, { color: C.accent1 });
  stat(slide, "3 ×", "every stored byte is kept on three DataNodes, including accidental copies", 9.0, 1.75, 3.7);
  text(slide, "Why this is hard to fix by hand", 0.6, 4.2, 12, 0.4, { fontSize: 18, bold: true });
  [
    ["No visibility", "HDFS has no view of identical content across files."],
    ["Risky cleanup", "Deleting a \"duplicate\" by hand can remove what someone needs."],
    ["Confusing numbers", "Logical size, stored size and disk use are three different things."],
  ].forEach(([head, body], index) => {
    const x = 0.6 + index * 4.1;
    slide.addShape(pres.ShapeType.roundRect, { x, y: 4.75, w: 3.9, h: 1.7, rectRadius: 0.06, fill: { color: C.background2 }, line:
{ color: C.background2 }, objectName: `${head} card` });
    text(slide, head, x + 0.2, 4.92, 3.5, 0.35, { fontSize: 16, bold: true });
    text(slide, body, x + 0.2, 5.32, 3.5, 1.0, { fontSize: 14, color: C.accent5 });
  });
  text(slide, "Measured with authentic files on the test cluster. Duplicates were created for the experiment.", 0.6, 6.6, 12, 0.3,
{ fontSize: 11, color: C.accent6 });
}

// 4. Solution
{
  const slide = newSlide("A layer above HDFS that stores each distinct content once", "Introduction", "HDFS is not modified at all.
The application decides what is written, and HDFS still replicates it. Walk through the six capabilities briefly. The point to
stress: each headline number in the interface states whether it is measured, derived or estimated.");
  const items = [
    ["Whole-file deduplication", "SHA-256 identity. Any number of names, one stored object."],
    ["Chunk-level deduplication", "Fixed-size chunks shared across different files."],
    ["Safe optimization", "Dry run first. Each file is verified before its old copy goes."],
    ["Reference-aware deletion", "Recoverable deletes. Shared content stays while it is needed."],
    ["Replication management", "Real factor changes, confirmed with fsck and block locations."],
    ["Honest metrics", "Every headline value says: measured, derived or estimated."],
  ];
  items.forEach(([head, body], index) => {
    const col = index % 3;
    const row = Math.floor(index / 3);
    const x = 0.6 + col * 4.1;
    const y = 1.7 + row * 2.45;
    slide.addShape(pres.ShapeType.roundRect, { x, y, w: 3.9, h: 2.2, rectRadius: 0.06, fill: { color: C.background2 }, line:
{ color: C.background2 }, objectName: `${head} card` });
    slide.addText(String(index + 1), { shape: pres.ShapeType.ellipse, x: x + 0.2, y: y + 0.2, w: 0.42, h: 0.42, fill: { color:
C.accent1 }, line: { color: C.accent1 }, color: C.background1, fontSize: 14, bold: true, align: "center", valign: "middle", margin:
0, objectName: `number ${index + 1}` });
    text(slide, head, x + 0.2, y + 0.75, 3.5, 0.4, { fontSize: 16, bold: true });
    text(slide, body, x + 0.2, y + 1.17, 3.5, 0.9, { fontSize: 14, color: C.accent5 });
  });
}

// 5. Objectives
{
  const slide = newSlide("What the project set out to achieve", "Introduction", "All of these are implemented and tested. Future
work, such as content-defined chunking, is kept separate and is on a later slide.");
  const groups = [
    ["Reduce", ["Detect exact duplicates with SHA-256", "Find identical chunks across files", "Store unique data in content-
addressed form"]],
    ["Protect", ["Reconstruct originals exactly", "Verify with cryptographic checksums", "Delete safely, and recover"]],
    ["Operate", ["Manage supported replication settings", "Monitor NameNode and DataNode health", "Show storage and savings
clearly"]],
    ["Prove", ["Evaluate on authentic datasets", "Compare storage visually", "Document design and results"]],
  ];
  groups.forEach(([head, lines], index) => {
    const x = 0.6 + index * 3.07;
    slide.addShape(pres.ShapeType.roundRect, { x, y: 1.55, w: 2.92, h: 4.05, rectRadius: 0.05, fill: { color: C.background2 }, line:
{ color: C.background2 }, objectName: `${head} card` });
    text(slide, head, x + 0.2, 1.72, 2.5, 0.55, { fontSize: 24, bold: true, color: "1C5CAB" });
    slide.addText(lines.map((line, i) => ({ text: line, options: { bullet: { indent: 14 }, breakLine: i < lines.length - 1,
paraSpaceAfter: 12 } })), { x: x + 0.2, y: 2.38, w: 2.52, h: 3.1, margin: 0, fontSize: 15, color: C.text1, valign: "top", isTextBox:
true });
  });
  text(slide, "Keep the redundancy that protects data. Remove the redundancy that only costs space.", 0.6, 5.85, 12.13, 0.8,
{ fontSize: 18, bold: true });
}

// 6. Technology stack
pres.addSection({ title: "Design" });
{
  const slide = newSlide("A deliberately ordinary, inspectable technology stack", "Design", "Everything here is free and open
source. WebHDFS means the Python backend needs no Java client. SQLite keeps setup simple, and its limits are discussed on a later
slide. The Hadoop image is built from the official Apache tarball because the official images are amd64-only and this laptop is
arm64.");
  const rows = [
    ["Interface", ["Next.js 16", "React 19", "TypeScript", "Tailwind + shadcn/ui", "Recharts"], C.background2, C.text1],
    ["API and logic", ["Python 3.12", "FastAPI", "Pydantic", "HTTPX", "hashlib SHA-256"], C.background2, C.text1],
    ["Metadata", ["SQLite (WAL)", "SQLAlchemy 2", "Alembic migrations"], "FDEEE6", C.text1],
    ["Storage", ["Apache Hadoop 3.5.0", "HDFS", "WebHDFS REST", "JMX and fsck"], C.accent4, C.text1],
    ["Platform and tests", ["Docker Compose", "Colima", "Pytest", "Playwright", "Typst"], C.background2, C.text1],
  ];
  rows.forEach(([label, items, fill, color], index) => {
    const y = 1.65 + index * 1.03;
    text(slide, label, 0.6, y + 0.27, 2.2, 0.4, { fontSize: 15, bold: true });
    const width = (9.9 - (items.length - 1) * 0.12) / items.length;
    items.forEach((item, i) => box(slide, item, 2.83 + i * (width + 0.12), y, width, 0.85, { fill, color, size: 14 }));
```

```
  });
}

// 7. Architecture
{
  const slide = newSlide("Bytes reach HDFS first, and metadata commits afterwards", "Design", "HDFS and SQLite cannot share a
transaction. So every write has a fixed order: stage in HDFS, check it, rename atomically, then commit metadata. A crash can leave
an unreferenced file, which reconciliation finds. It can never leave metadata that points at missing bytes.");
  box(slide, "Browser", 0.6, 3.0, 1.9, 1.2, { sub: "Next.js dashboard" });
  arrow(slide, 2.5, 3.6, 3.1, 3.6);
  box(slide, "REST API", 3.1, 3.0, 1.9, 1.2, { sub: "FastAPI, /api/v1" });
  arrow(slide, 5.0, 3.6, 5.6, 3.6);
  slide.addShape(pres.ShapeType.roundRect, { x: 5.6, y: 1.7, w: 3.3, h: 3.8, rectRadius: 0.05, fill: { color: C.background1 }, line:
{ color: C.accent6, width: 1, dashType: "dash" }, objectName: "services group" });
  text(slide, "Services", 5.8, 1.82, 2.9, 0.3, { fontSize: 14, bold: true, color: C.accent5 });
  ["Ingest and content store", "Optimizer and lifecycle", "Integrity and replication", "Monitor, analytics, audit"].forEach((label,
i) => box(slide, label, 5.8, 2.22 + i * 0.8, 2.9, 0.68, { size: 13 }));
  arrow(slide, 8.9, 2.75, 9.6, 2.45);
  arrow(slide, 8.9, 4.45, 9.6, 4.75);
  box(slide, "SQLite metadata", 9.6, 1.8, 3.13, 1.3, { fill: "FDEEE6", sub: "Files, objects, chunks, manifests, jobs, audit" });
  box(slide, "HDFS through WebHDFS", 9.6, 4.1, 3.13, 1.3, { fill: C.accent4, sub: "1 NameNode, 3 DataNodes, factor 3" });
  text(slide, "Write order:  1 stage in HDFS   2 check bytes   3 rename atomically   4 commit metadata", 0.6, 5.85, 12.1, 0.4,
{ fontSize: 18, bold: true });
  text(slide, "A crash can leave an unreferenced file, but never metadata pointing at missing bytes.", 0.6, 6.3, 12.1, 0.4,
{ fontSize: 14, color: C.accent5 });
}

// 8. Cluster
{
  const slide = newSlide("A real HDFS cluster: one NameNode and three DataNodes", "Design", "This is genuine Apache Hadoop 3.5.0,
running in containers. The screenshot is Hadoop's own interface. It is honest to say this is all on one laptop: it shows the
software's behaviour under failure, but the three DataNodes share one disk, so it gives no real hardware fault tolerance.");
  box(slide, "NameNode", 2.1, 1.75, 3.0, 1.0, { fill: C.accent4, size: 16, sub: "Namespace and block map" });
  ["DataNode 1", "DataNode 2", "DataNode 3"].forEach((name, i) => {
    const x = 0.6 + i * 2.05;
    box(slide, name, x, 3.75, 1.9, 1.0, { fill: C.accent4, size: 14, sub: "Blocks" });
    arrow(slide, x + 0.95, 3.75, 3.6, 2.75);
  });
  text(slide, "Heartbeats every 3 s, and block reports", 0.6, 1.85, 1.35, 0.9, { fontSize: 12, color: C.accent5 });
  text(slide, "Each block is stored on 3 DataNodes. Stopping a DataNode leaves files readable, which we tested for real.", 0.6, 5.0,
6.0, 0.8, { fontSize: 14 });
  text(slide, "Limit: all nodes share one laptop and one disk.", 0.6, 5.85, 6.0, 0.4, { fontSize: 14, color: C.accent5 });
  const shotHeight = screenshot(slide, "namenode_datanodes", 7.0, 1.65, 5.73, "Screenshot of DataNode list in Hadoop's own NameNode
web interface");
  text(slide, "Hadoop's own NameNode interface, showing this cluster", 7.0, 1.65 + shotHeight + 0.1, 5.73, 0.35, { fontSize: 12,
color: C.accent5 });
}

// 9. Whole-file
{
  const slide = newSlide("Whole-file deduplication: same hash, no new bytes", "Design", "Identity depends only on content. Two
different names with the same bytes are duplicates. The same name with different bytes is not: the second file becomes name (1).
Lookup is by a unique index on the hash.");
  box(slide, "Upload", 0.6, 2.2, 1.6, 1.0, { size: 15, sub: "Any file name" });
  arrow(slide, 2.2, 2.7, 2.7, 2.7);
  box(slide, "SHA-256", 2.7, 2.2, 1.6, 1.0, { size: 15, sub: "Streamed" });
  arrow(slide, 4.3, 2.7, 4.8, 2.7);
  slide.addText("Hash already stored?", { shape: pres.ShapeType.diamond, x: 4.8, y: 1.75, w: 2.7, h: 1.9, fill: { color:
C.background1 }, line: { color: C.accent6, width: 1.2 }, fontSize: 14, bold: true, align: "center", valign: "middle", margin: 16,
objectName: "decision" });
  arrow(slide, 7.5, 2.7, 8.5, 2.2);
  text(slide, "Yes", 7.65, 2.1, 0.6, 0.3, { fontSize: 13, bold: true });
  box(slide, "Add a reference", 8.5, 1.65, 4.23, 1.05, { fill: "E3F5EC", size: 15, sub: "New logical file, 0 bytes written" });
  arrow(slide, 7.5, 2.7, 8.5, 3.45);
  text(slide, "No", 7.65, 3.05, 0.6, 0.3, { fontSize: 13, bold: true });
  box(slide, "Stage, check, rename", 8.5, 2.95, 4.23, 1.05, { fill: C.accent4, size: 15, sub: "/objects/ab/cd/<sha256>" });
  const row = C_many;
  stat(slide, `${row.logical_files} names`, "of one authentic file, in a single test", 0.6, 4.35, 3.6);
  stat(slide, `${row.hdfs_file_count} HDFS file`, "stored for all of them", 4.6, 4.35, 3.6, { color: C.accent1 });
  stat(slide, `${row["verification.files_passed"]} of ${row["verification.files_checked"]}`, "names passed SHA-256 verification",
8.6, 4.35, 4.1);
}

// 10. Chunk-level
{
  const slide = newSlide("Chunks find overlap between different files", "Design", "Each file is split at fixed sizes, and each chunk
hashed. A manifest lists the chunks in order. Reconstruction reads them in order, checks each hash, and compares the whole file hash
at the end. Important: these are application-level chunks, each its own HDFS file. They are not HDFS blocks, which we cannot
deduplicate from outside. The right side is a measured result showing the known weakness of fixed-size chunking.");
  const cell = 0.62;
  const draw = (label, y, cells) => {
    text(slide, label, 0.6, y + 0.12, 2.2, 0.4, { fontSize: 14, color: C.accent5 });
    cells.forEach(([name, shared], i) => slide.addText(name, { shape: pres.ShapeType.roundRect, rectRadius: 0.04, x: 2.8 + i * (cell
+ 0.07), y, w: cell, h: 0.55, fill: { color: shared ? C.accent1 : C.accent2 }, line: { color: shared ? C.accent1 : C.accent2 },
color: C.background1, fontSize: 13, bold: true, align: "center", valign: "middle", margin: 0, objectName: `chunk ${name}` }));
  };
  draw("Original file", 1.85, [["c1", true], ["c2", true], ["c3", true], ["c4", true]]);
  draw("Byte-identical file", 2.55, [["c1", true], ["c2", true], ["c3", true], ["c4", true]]);
  draw("One chunk changed", 3.25, [["c1", true], ["c2", true], ["c5", false], ["c4", true]]);
  text(slide, "Stored once each: c1, c2, c3, c4, c5. A manifest lists each file's chunks in order, and rebuilds it exactly.", 0.6,
4.1, 5.0, 0.95, { fontSize: 14 });
  text(slide, "Chunks are application-level HDFS files, not HDFS blocks.", 0.6, 5.1, 5.0, 0.6, { fontSize: 14, bold: true });
  const variantName = { "pg1342.truncate_90.txt": "Truncated to 90%", "pg1342.replace_mid_byte.txt": "One byte replaced",
```

```
"pg1342.prefix_newline.txt": "One byte inserted at start" };
  slide.addChart(pres.charts.BAR, [{ name: "Chunks reused (%)", labels: D2.map((r) => variantName[r.variant]), values: D2.map((r) =>
Number(r.reuse_percent.toFixed(1))) }], {
    x: 6.3, y: 1.6, w: 6.43, h: 4.2, barDir: "bar", chartColors: [THEME.colors.accent1], barGapWidthPct: 80,
    ...chartOptions({ showTitle: true, title: "Measured: chunks reused, 64 KiB chunks (%)", valAxisMaxVal: 100, valAxisMinVal: 0,
catAxisOrientation: "maxMin", valAxisHidden: true, valGridLine: { style: "none" }, dataLabelFormatCode: '0.0"%"' }),
  });
  text(slide, "One inserted byte shifts every boundary. Content-defined chunking would fix this; it is future work.", 6.3, 5.9,
6.43, 0.6, { fontSize: 14, color: C.accent5 });
}

// 11. Dashboard
pres.addSection({ title: "Product" });
{
  const slide = newSlide("The dashboard says how every number is known", "Product", "This is a real screenshot of the running
system, cropped to the top of the page. Point out the storage reduction figure and bar, then the small labels: Derived values come
from metadata, Measured values come from HDFS, and Estimated values are calculations. On an empty system, it shows N/A rather than a
fake zero.");
  const w = 8.3;
  screenshot(slide, "dashboard", 0.6, 1.5, w, "Screenshot of HDFS Optimizer dashboard with savings and cluster health");
  const x = 0.6 + w + 0.35;
  const notes = [["Derived", "Exact, from the metadata database. Excludes replication."], ["Measured", "Reported by HDFS at the time
shown."], ["Estimated", "A calculation or a prediction, and labelled as one."], ["N/A when undefined", "An empty system never shows
a fake zero."]];
  notes.forEach(([head, body], i) => {
    const y = 1.55 + i * 1.18;
    text(slide, head, x, y, 12.73 - x, 0.35, { fontSize: 16, bold: true });
    text(slide, body, x, y + 0.36, 12.73 - x, 0.75, { fontSize: 14, color: C.accent5 });
  });
}

// 12. Files
{
  const slide = newSlide("Upload, browse and inspect files, down to replica locations", "Product", "Left: uploading the same bytes
under a new name is reported as a duplicate, and no new data is written. Right: the detail panel shows the hash, the HDFS path, the
DataNodes holding replicas and a passed SHA-256 check. Progress bars show the bytes the browser has really sent.");
  screenshot(slide, "upload", 0.6, 1.5, 8.5, "Screenshot of file upload showing a detected duplicate");
  screenshot(slide, "file_details", 9.4, 1.5, 3.33, "Screenshot of a file's details with block replicas and passed verification");
  text(slide, "A renamed copy is reported as a duplicate: no new data stored.", 0.6, 5.55, 8.5, 0.5, { fontSize: 15, bold: true });
  text(slide, "Replicas stay on three DataNodes, and the SHA-256 check passes.", 9.4, 6.4, 3.33, 0.55, { fontSize: 12, bold:
true });
}

// 13. Duplicates
{
  const slide = newSlide("Duplicate detection shows how names map to stored bytes", "Product", "This group has one SHA-256 and
several logical names. The diagram shows that each copy is still stored separately, in orange, so space can be reclaimed. After
optimization the lines converge on one blue box. The tab on the right shows shared chunks: a truncated copy and a one-byte edit
share almost everything, and a one-byte insertion at the start shares nothing.");
  screenshot(slide, "duplicates", 0.6, 1.5, 6.25, "Screenshot of duplicate groups and the relationship diagram");
  screenshot(slide, "shared_chunks", 7.05, 1.5, 5.68, "Screenshot of chunk map with shared chunks");
  text(slide, "Exact duplicates", 0.6, 4.85, 6.25, 0.35, { fontSize: 16, bold: true });
  text(slide, "Same SHA-256, different names. In orange, each copy is still stored separately, so its space can be reclaimed.", 0.6,
5.22, 6.25, 0.8, { fontSize: 14, color: C.accent5 });
  text(slide, "Shared chunks", 7.05, 4.85, 5.68, 0.35, { fontSize: 16, bold: true });
  text(slide, "Blue marks are shared with another file. Truncating or editing one byte shares almost everything. Inserting one byte
at the start shares nothing.", 7.05, 5.22, 5.68, 1.1, { fontSize: 14, color: C.accent5 });
}

// 14. Optimization
{
  const file = I.find((r) => r.mode === "file");
  const chunk = I.find((r) => r.mode === "chunk");
  const slide = newSlide("Optimization reclaimed the copies and kept every byte", "Product", `Both runs started from ${file.files}
files stored without deduplication: nine books with two exact copies each. Each is measured by the NameNode and includes
replication. All ${file["verification.files_passed"]} files then passed SHA-256 verification. Chunk mode uses slightly more space
than whole-file, because of its manifests. The chart and figures are from the repeated benchmark. The screenshot is from the
demonstration session, which also holds two uploaded files, so its totals differ slightly.`);
  slide.addChart(pres.charts.BAR, [
    { name: "Before", labels: ["Whole-file", "Chunk, 256 KiB"], values: [file, chunk].map((r) =>
Number((r.hdfs_space_consumed_before / MIB).toFixed(1))) },
    { name: "After", labels: ["Whole-file", "Chunk, 256 KiB"], values: [file, chunk].map((r) =>
Number((r.hdfs_space_consumed_after / MIB).toFixed(1))) },
  ], { x: 0.6, y: 1.6, w: 6.3, h: 4.6, barDir: "col", chartColors: [THEME.colors.accent6, THEME.colors.accent1], barGapWidthPct:
70, ...chartOptions({ showLegend: true, legendPos: "b", showTitle: true, title: "NameNode-measured space consumed (MiB)",
valAxisMinVal: 0, valAxisHidden: true, valGridLine: { style: "none" } }) });
  screenshot(slide, "optimization", 7.2, 1.6, 5.53, "Screenshot of completed optimization with before and after");
  stat(slide, `${file["verification.files_passed"]} of ${file["verification.files_checked"]}`, "files passed SHA-256 afterwards",
7.2, 3.45, 2.65, { size: 30 });
  stat(slide, `${file.hdfs_files_before} to ${file.hdfs_files_after}`, "HDFS files, whole-file mode", 10.1, 3.45, 2.63, { size:
30 });
  stat(slide, `${(file["disk_used_delta_before.mean"] / MIB).toFixed(0)} to ${(file["disk_used_delta_after.mean"] / MIB).toFixed(0)}
MiB`, "on DataNode disks, measured with du", 7.2, 4.95, 5.5, { size: 30 });
}

// 15. Replication and monitoring
{
  const up = H.find((r) => r.step === "changed" && r.factor === 2);
  const slide = newSlide("Replication and cluster health come from HDFS itself", "Product", "A factor above the number of live
DataNodes is refused, and lowering it needs confirmation. Success is reported only when fsck and the block locations agree. An
interesting finding: fsck stops counting a replica as soon as it is marked excess, before it is deleted, so our first version
reported success too early. A real test caught that.");
  screenshot(slide, "replication", 0.6, 1.5, 6.0, "Screenshot of replication manager");
```

```
  screenshot(slide, "cluster", 6.73, 1.5, 6.0, "Screenshot of cluster monitoring page");
  text(slide, `Measured at factor 2: space multiplier ${up.multiplier.toFixed(2)}, with ${up.under_replicated_blocks} under-
replicated blocks. Success is reported only when fsck and block locations agree. A factor above the live DataNodes is refused.`,
0.6, 5.1, 12.13, 0.9, { fontSize: 15 });
}

// 16. Dataset
pres.addSection({ title: "Evaluation" });
{
  const total = originals.reduce((sum, f) => sum + f.size_bytes, 0);
  const slide = newSlide("Authentic files, with provenance for every byte", "Evaluation", "This project does not need a labelled
machine-learning dataset, but it does need real files. The files are from Project Gutenberg, including the three suggested in the
brief. Duplicates are clearly labelled test copies. Be ready to say what this cannot show: it is small, so it says nothing about
production scale, and its duplicate rates are chosen, not observed.");
  stat(slide, String(originals.length), "original files from Project Gutenberg", 0.6, 1.5, 2.6, { size: 44, color: C.accent1 });
  stat(slide, mib(total), "of authentic data, as published", 3.4, 1.5, 3.0, { size: 44 });
  const rows = [
    ["Kind", "What it is", "Used for"],
    ["Original", "Bytes as published, with URL, date and SHA-256", "Everything"],
    ["Workload copy", "Same bytes, another logical name, labelled", "Duplicate tests"],
    ["Transformed", "One documented byte edit of one original", "Overlap tests"],
  ];
  slide.addTable(rows.map((row, r) => row.map((cell) => ({ text: cell, options: { bold: r === 0, fontSize: 14, color: C.text1, fill:
{ color: r === 0 ? C.background2 : C.background1 }, border: [{ type: "none" }, { type: "none" }, { type: "solid", color: HEX.grid,
pt: 1 }, { type: "none" }], margin: [4, 6, 4, 6] } }))), { x: 0.6, y: 3.55, w: 6.2, colW: [1.5, 3.2, 1.5] });
  text(slide, "No data was invented. Savings verify the mechanism on chosen workloads; they do not predict real-world savings.",
0.6, 5.85, 6.2, 0.8, { fontSize: 14, color: C.accent5 });
  screenshot(slide, "provenance", 7.2, 1.6, 5.53, "Screenshot of dataset lab with provenance");
  text(slide, "Every file: source page, download URL, date, SHA-256 and kind.", 7.2, 5.15, 5.53, 0.6, { fontSize: 14, bold: true });
}

// 17. Benchmarks
{
  const chunked = D.filter((r) => r.mode === "chunk");
  const label = (b) => (b < MIB ? `${b / 1024} KiB` : `${b / MIB} MiB`);
  const slide = newSlide("Finer chunks find more overlap, at a steep cost", "Evaluation", "These are measured results for the same
workload at each chunk size: originals plus three edited variants. Savings rise as chunks shrink, but throughput falls and the
number of NameNode objects grows, because each chunk is its own HDFS file. Chunks larger than the book find nothing, since the
variants are smaller than one chunk. Timings describe this laptop, not a production cluster. Each timing is the mean of three
runs.");
  const common = { barDir: "col", chartColors: [THEME.colors.accent1], barGapWidthPct: 60 };
  const labels = chunked.map((r) => label(r.chunk_size));
  slide.addChart(pres.charts.BAR, [{ name: "Savings (%)", labels, values: chunked.map((r) =>
Number(r.savings_percent.toFixed(1))) }], { x: 0.6, y: 1.45, w: 4.0, h: 4.1, ...common, ...chartOptions({ showTitle: true, title:
"Logical savings (%)", valAxisMinVal: 0, valAxisHidden: true, valGridLine: { style: "none" }, dataLabelFormatCode: "0.0" }) });
  slide.addChart(pres.charts.BAR, [{ name: "Upload MiB/s", labels, values: chunked.map((r) =>
Number(r["upload_throughput_mib_s.mean"].toFixed(1))) }], { x: 4.67, y: 1.45, w: 4.0, h:
4.1, ...common, ...chartOptions({ showTitle: true, title: "Upload throughput (MiB/s)", valAxisMinVal: 0, valAxisHidden: true,
valGridLine: { style: "none" }, dataLabelFormatCode: "0.0" }) });
  slide.addChart(pres.charts.BAR, [{ name: "NameNode objects", labels, values: chunked.map((r) =>
Math.round(r["namespace_objects.mean"])) }], { x: 8.73, y: 1.45, w: 4.0, h: 4.1, ...common, ...chartOptions({ showTitle: true,
title: "NameNode objects", valAxisMinVal: 0, valAxisHidden: true, valGridLine: { style: "none" }, dataLabelFormatCode:
"#,##0" }) });
  const baseline = D.find((r) => r.mode === "file")["disk_used_delta.mean"];
  const smallest = chunked[0];
  const higher = ((smallest["disk_used_delta.mean"] - baseline) / baseline) * 100;
  text(slide, `Logical savings can mislead: at ${label(smallest.chunk_size)}, they read ${smallest.savings_percent.toFixed(1)}%, yet
disk use measured with du is ${higher.toFixed(1)}% higher than whole-file storage.`, 0.6, 5.65, 12.13, 0.8, { fontSize: 18, bold:
true });
  text(slide, "Timings: mean of three runs on one laptop. They show the trend, and do not predict a real cluster.", 0.6, 6.5, 12.13,
0.35, { fontSize: 12, color: C.accent5 });
}

// 18. Integrity and testing
{
  const j1 = J.J1_datanode_failure;
  const j2 = J.J2_namenode_failure;
  const j3 = J.J3_process_kill;
  const slide = newSlide("We broke it on purpose, and no data was lost", "Evaluation", `These failures were real: containers were
stopped, and a process was killed with SIGKILL during a job. With a DataNode down, files stayed readable, and the NameNode declared
it dead after about ${Math.round(j1.seconds_until_namenode_declared_dead)} seconds. With the NameNode down, uploads failed cleanly
with HTTP ${j2.upload_http_status_during_outage} and recorded nothing. After the kill, the job was marked
${j3.job_status_after_restart}; resuming it finished the remaining files, and ${j3.files_restored_byte_identical_after_resume} of
${j3.files} files restored byte-identical. The test counts come from the saved test output.`);
  const failures = [
    ["DataNode stopped", `Files stayed readable. A request for factor 3 with two live nodes was refused (HTTP
${j1.replication_factor_3_request_while_two_live_http_status}).`],
    ["NameNode stopped", `Upload failed with HTTP ${j2.upload_http_status_during_outage}. No file recorded. The retry after restart
succeeded (HTTP ${j2.retry_after_restart_http_status}).`],
    ["Backend killed mid-job", `Job marked ${j3.job_status_after_restart}. After resume,
${j3.files_restored_byte_identical_after_resume} of ${j3.files} files restored byte-identical.`],
  ];
  failures.forEach(([head, body], i) => {
    const y = 1.65 + i * 1.5;
    slide.addShape(pres.ShapeType.roundRect, { x: 0.6, y, w: 7.2, h: 1.33, rectRadius: 0.06, fill: { color: C.background2 }, line:
{ color: C.background2 }, objectName: `${head} card` });
    text(slide, head, 0.82, y + 0.14, 6.8, 0.35, { fontSize: 16, bold: true });
    text(slide, body, 0.82, y + 0.52, 6.8, 0.75, { fontSize: 14, color: C.accent5 });
  });
  text(slide, "Automated tests that passed", 8.2, 1.65, 4.53, 0.4, { fontSize: 16, bold: true });
  [[counts.unit.passed, "unit and API tests"], [counts.integration.passed, "on the real HDFS cluster"], [counts.browser.passed,
"real-browser tests"]].forEach(([value, label], i) => {
    const y = 2.15 + i * 1.3;
```

```
    text(slide, String(value), 8.2, y, 1.6, 0.9, { fontSize: 40, bold: true, color: C.accent1, valign: "middle" });
    text(slide, label, 9.85, y, 2.88, 0.9, { fontSize: 15, valign: "middle" });
  });
  text(slide, "Corrupt or missing chunks are reported individually, and a corrupt download aborts.", 8.2, 6.05, 4.53, 0.6,
{ fontSize: 13, color: C.accent5 });
}

// 19. Limits and future
pres.addSection({ title: "Close" });
{
  const slide = newSlide("What we learned, what limits it, and what comes next", "Close", "Be open about limits; it counts for you.
The biggest measured cost is that each chunk is an HDFS file, and packing would address that. Content-defined chunking fixes the
shift weakness. Everything in the right column is future work, and none of it is implemented.");
  const columns = [
    ["Engineering challenges", ["No transaction spans HDFS and SQLite", "fsck reported success before replicas were deleted",
"NameNode JMX values lag by about 10 s", "DataNode deletes are asynchronous, so disk readings need care"]],
    ["Current limits", ["Fixed-size chunks: a shift defeats them", "Each chunk is an HDFS file, which costs NameNode memory", "One
process and SQLite limit scale", "One laptop: no real hardware fault tolerance"]],
    ["Future work", ["Content-defined chunking (FastCDC)", "Packing chunks into container files", "Distributed index; Spark
background jobs", "Kerberos, HA and real multi-machine deployment"]],
  ];
  columns.forEach(([head, lines], index) => {
    const x = 0.6 + index * 4.1;
    slide.addShape(pres.ShapeType.roundRect, { x, y: 1.55, w: 3.9, h: 4.05, rectRadius: 0.05, fill: { color: index === 2 ?
"E4EFFC" : C.background2 }, line: { color: index === 2 ? "E4EFFC" : C.background2 }, objectName: `${head} card` });
    text(slide, head, x + 0.2, 1.72, 3.5, 0.4, { fontSize: 18, bold: true });
    slide.addText(lines.map((line, i) => ({ text: line, options: { bullet: { indent: 14 }, breakLine: i < lines.length - 1,
paraSpaceAfter: 10 } })), { x: x + 0.2, y: 2.25, w: 3.5, h: 3.25, margin: 0, fontSize: 15, color: C.text1, valign: "top", isTextBox:
true });
  });
  text(slide, "The largest measured cost, small chunk files, points directly at the next step: packing chunks into containers.",
0.6, 5.85, 12.13, 0.8, { fontSize: 18, bold: true });
}

// 20. Conclusion
{
  const file = I.find((r) => r.mode === "file");
  const slide = pres.addSlide({ masterName: "DARK", sectionTitle: "Close" });
  slide.addText([{ text: "Duplicates gone.", options: { breakLine: true } }, { text: "Replicas intact.", options: { breakLine:
true } }, { text: "Every byte verified." }], { placeholder: "title", fontSize: 40 });
  slide.addText("A working deduplication layer on real HDFS, evaluated with authentic data, including real failures.",
{ placeholder: "body" });
  text(slide, "Thank you. Questions?", 0.8, 5.5, 7.3, 0.6, { fontSize: 24, bold: true, color: C.background1 });
  text(slide, mib(file.hdfs_space_consumed_before), 8.9, 1.5, 3.9, 0.7, { fontSize: 36, bold: true, color: C.background1 });
  text(slide, "NameNode space consumed before optimization", 8.9, 2.2, 3.9, 0.6, { fontSize: 14, color: C.accent6 });
  text(slide, mib(file.hdfs_space_consumed_after), 8.9, 3.1, 3.9, 0.7, { fontSize: 36, bold: true, color: C.accent4 });
  text(slide, "after, same logical files, same replication", 8.9, 3.8, 3.9, 0.6, { fontSize: 14, color: C.accent6 });
  text(slide, `${file["verification.files_passed"]} of ${file["verification.files_checked"]}`, 8.9, 4.7, 3.9, 0.7, { fontSize: 36,
bold: true, color: C.background1 });
  text(slide, "files passed SHA-256 afterwards", 8.9, 5.4, 3.9, 0.6, { fontSize: 14, color: C.accent6 });
  slide.addNotes("Close with the takeaway: we separated deliberate redundancy from accidental redundancy, removed only the second,
and proved the bytes are intact. These figures are from a controlled workload with labelled copies. Thank the audience and invite
questions.");
}

/** pptxgenjs cannot write theme colors, so set them in the saved file. All theme parts are updated. */
async function applyTheme(file, theme) {
  const zip = await JSZip.loadAsync(fs.readFileSync(file));
  const parts = Object.keys(zip.files).filter((name) => /^ppt\/theme\/theme\d+\.xml$/.test(name));
  for (const part of parts) {
    let xml = await zip.file(part).async("string");
    for (const [slot, value] of Object.entries(theme.colors)) {
      xml = xml.replace(new RegExp(`<a:${slot}>.*?</a:${slot}>`, "s"), `<a:${slot}><a:srgbClr val="${value}"/></a:${slot}>`);
    }
    xml = xml.replace(/<a:clrScheme name="[^"]*">/, `<a:clrScheme name="${theme.name}">`);
    zip.file(part, xml);
  }
  fs.writeFileSync(file, await zip.generateAsync({ type: "nodebuffer", compression: "DEFLATE" }));
}

(async () => {
  fs.mkdirSync(path.dirname(OUT), { recursive: true });
  await pres.writeFile({ fileName: OUT });
  await applyTheme(OUT, THEME);
  console.log(`wrote ${OUT}`);
})();
```

presentation/source/package.json 8 lines

```
{
  "name": "hdfs-optimizer-deck",
  "private": true,
  "description": "Builds the project presentation from measured results and real screenshots",
  "scripts": { "build": "node build_deck.js" },
  "dependencies": { "jszip": "3.10.1", "pptxgenjs": "4.0.1" }
}
```

reports/source/requirements.txt 4 lines

```
matplotlib==3.10.7
pymupdf==1.26.6
Pillow==12.0.0
```

Chapter 12 — Execution Screenshots

Every image in this chapter and the next was captured from the running system by a Playwright script that used the application from an empty state. No image was edited or mocked. Terminal images show command output saved to `evidence/` and rendered as text, unedited. Long outputs are cut at a marked line, and the full text is in that folder.

```
# Captured 2026-11-05T06:41:41Z

$ docker compose ps --format table {{.Service}}\t{{.Image}}\t{{.State}}\t{{.Health}}\t{{.Ports}}
SERVICE     IMAGE                          STATE     <no value>   PORTS
datanode1   hdfs-optimizer/hadoop:3.5.0   running   healthy      127.0.0.1:19864->9864/tcp
datanode2   hdfs-optimizer/hadoop:3.5.0   running   healthy      127.0.0.1:19865->9864/tcp
datanode3   hdfs-optimizer/hadoop:3.5.0   running   healthy      127.0.0.1:19866->9864/tcp
namenode    hdfs-optimizer/hadoop:3.5.0   running   healthy      127.0.0.1:9870->9870/tcp
[exit code 0]

$ docker compose exec -T namenode hadoop version
Hadoop 3.5.0
Source code repository https://github.com/apache/hadoop.git -r dbcc7cd797100e6b32cd84f85b53a5193a5f9af0
Compiled by cnauroth on 2026-03-24T16:57Z
Compiled on platform linux-aarch_64
Compiled with protoc 3.25.5
From source with checksum 64582ed0a62a169c39a19c734a8f33b5
This command was run using /opt/hadoop/share/hadoop/common/hadoop-common-3.5.0.jar
[exit code 0]

$ docker info --format {{.OperatingSystem}} {{.Architecture}} CPUs={{.NCPU}} MemBytes={{.MemTotal}}
Ubuntu 24.04.4 LTS aarch64 CPUs=4 MemBytes=4093943808
[exit code 0]
```

Figure 11: HDFS cluster running in Docker, with Hadoop version output

```
# Captured 2026-11-05T06:41:42Z

$ docker compose exec -T namenode hdfs dfsadmin -report
Configured Capacity: 94572883968 (88.08 GB)
Present Capacity: 82651029210 (76.97 GB)
DFS Remaining: 82596278272 (76.92 GB)
DFS Used: 54750938 (52.21 MB)
DFS Used%: 0.07%
Replicated Blocks:
        Under replicated blocks: 0
        Blocks with corrupt replicas: 0
        Missing blocks: 0
        Missing blocks (with replication factor 1): 0
        Badly Distributed Blocks: 0
        Low redundancy blocks with highest priority to recover: 0
        Pending deletion blocks: 3
Erasure Coded Block Groups:
        Low redundancy block groups: 0
        Block groups with corrupt internal blocks: 0
        Missing block groups: 0
        Badly Distributed Blocks: 0
        Low redundancy blocks with highest priority to recover: 0
        Pending deletion blocks: 0

-------------------------------------------------
Live datanodes (3):

Name: 172.18.0.3:9866 (hdfs-optimizer-datanode2-1.hdfs-optimizer_default)
Hostname: datanode2
Decommission Status : Normal
Configured Capacity: 31524294656 (29.36 GB)
DFS Used: 18427962 (17.57 MB)
Non DFS Used: 2346479558 (2.19 GB)
DFS Remaining: 27532103680 (25.64 GB)
DFS Used%: 0.06%
DFS Remaining%: 87.34%
Configured Cache Capacity: 0 (0 B)
Cache Used: 0 (0 B)
Cache Remaining: 0 (0 B)
Cache Used%: 100.00%
Cache Remaining%: 0.00%
Xceivers: 0
Last contact: Thu Nov 05 06:41:41 UTC 2026
Last Block Report: Thu Nov 05 06:41:32 UTC 2026
Num of Blocks: 274


Name: 172.18.0.4:9866 (hdfs-optimizer-datanode3-1.hdfs-optimizer_default)
Hostname: datanode3
Decommission Status : Normal
Configured Capacity: 31524294656 (29.36 GB)
DFS Used: 18432058 (17.58 MB)
Non DFS Used: 2346508230 (2.19 GB)
DFS Remaining: 27532070912 (25.64 GB)
DFS Used%: 0.06%
DFS Remaining%: 87.34%
Configured Cache Capacity: 0 (0 B)
Cache Used: 0 (0 B)
Cache Remaining: 0 (0 B)
Cache Used%: 100.00%
Cache Remaining%: 0.00%
Xceivers: 1
Last contact: Thu Nov 05 06:41:42 UTC 2026
Last Block Report: Thu Nov 05 06:41:30 UTC 2026
Num of Blocks: 274

... (24 more lines in evidence/02_hdfs_report.txt)
```

Figure 12: `hdfs dfsadmin -report`: live DataNodes and capacity. All DataNodes share one VM disk, so capacity is counted once per node.

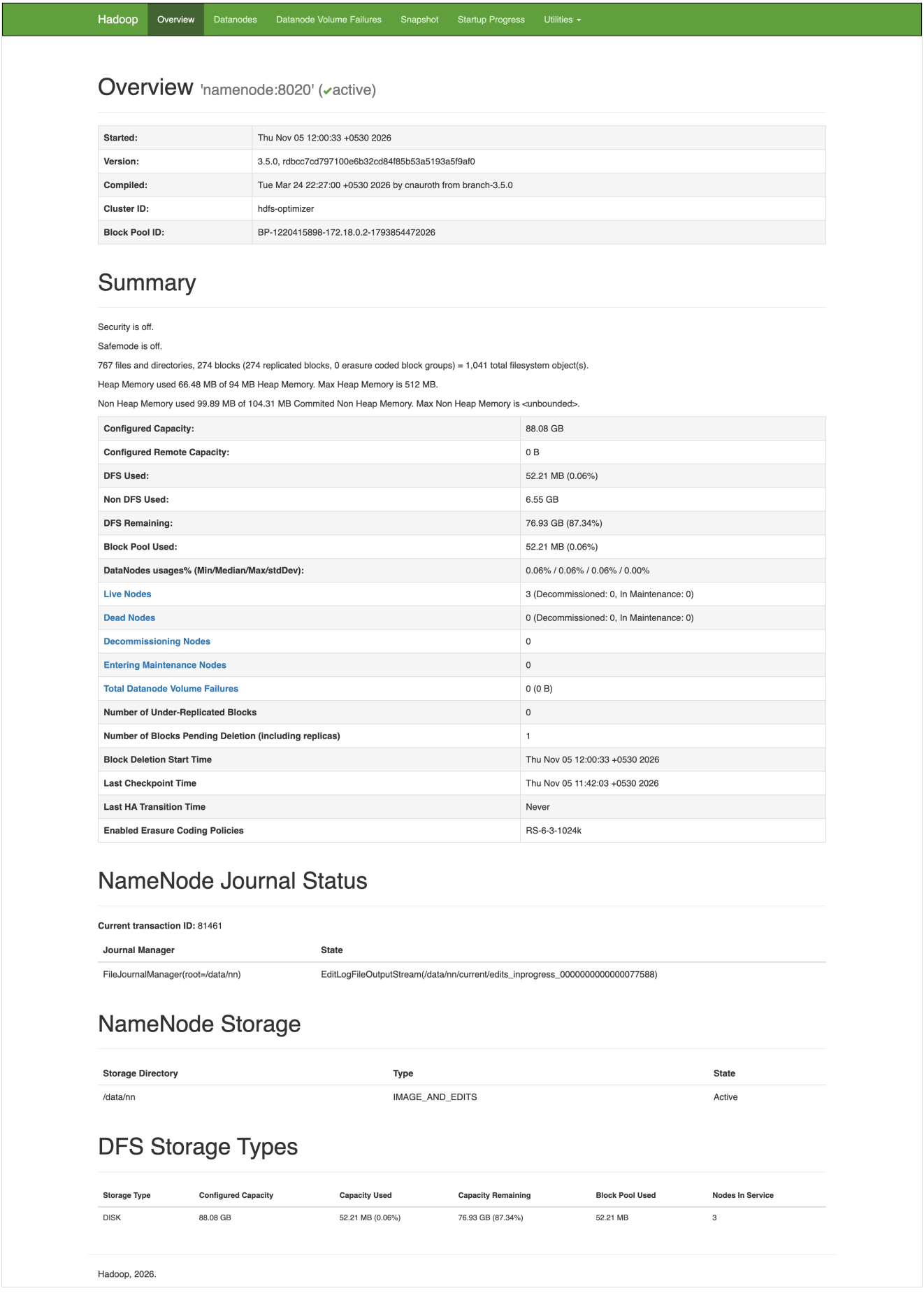

Figure 13: Apache Hadoop's own NameNode web interface

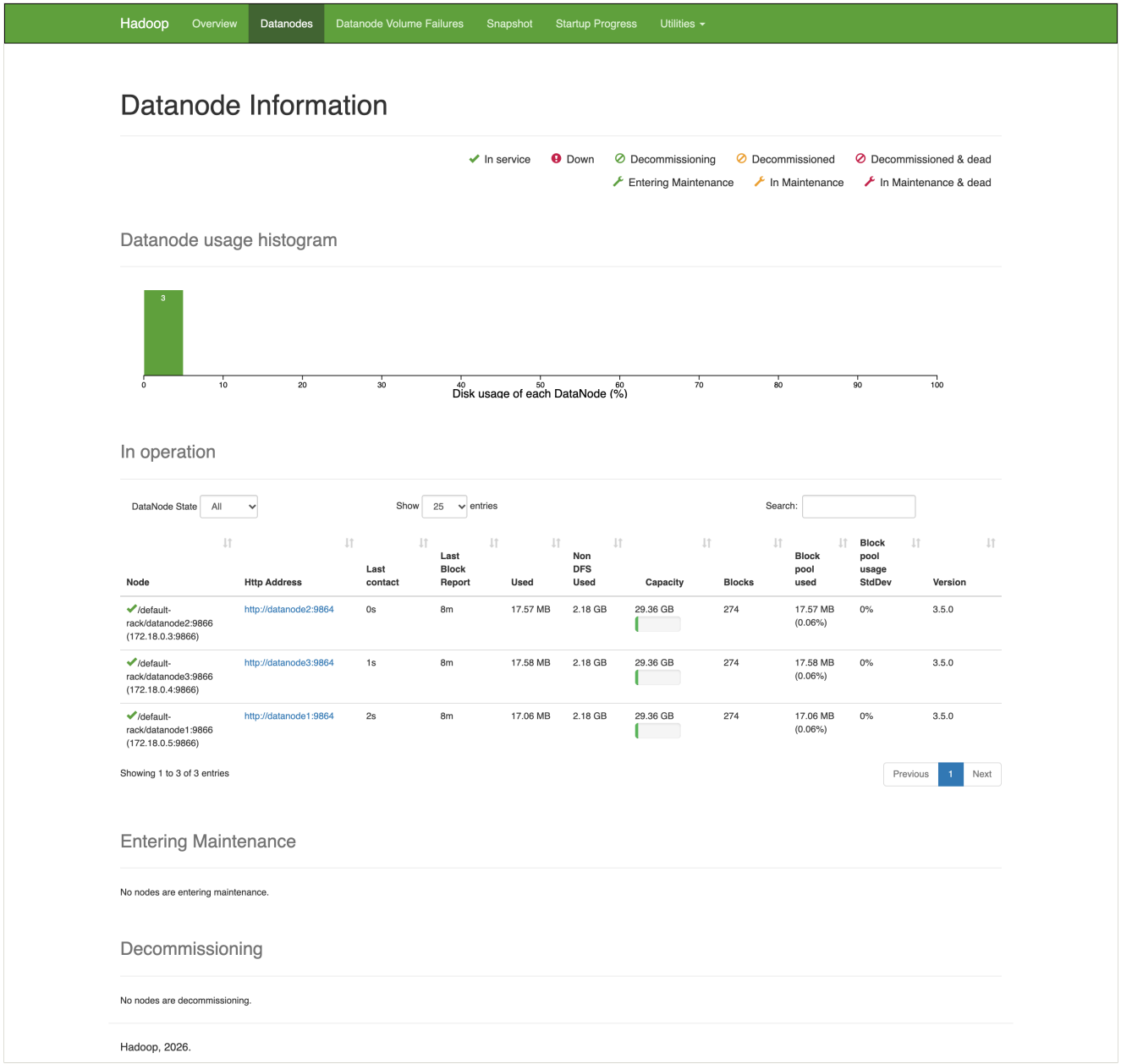

Figure 14: DataNode list in the NameNode interface

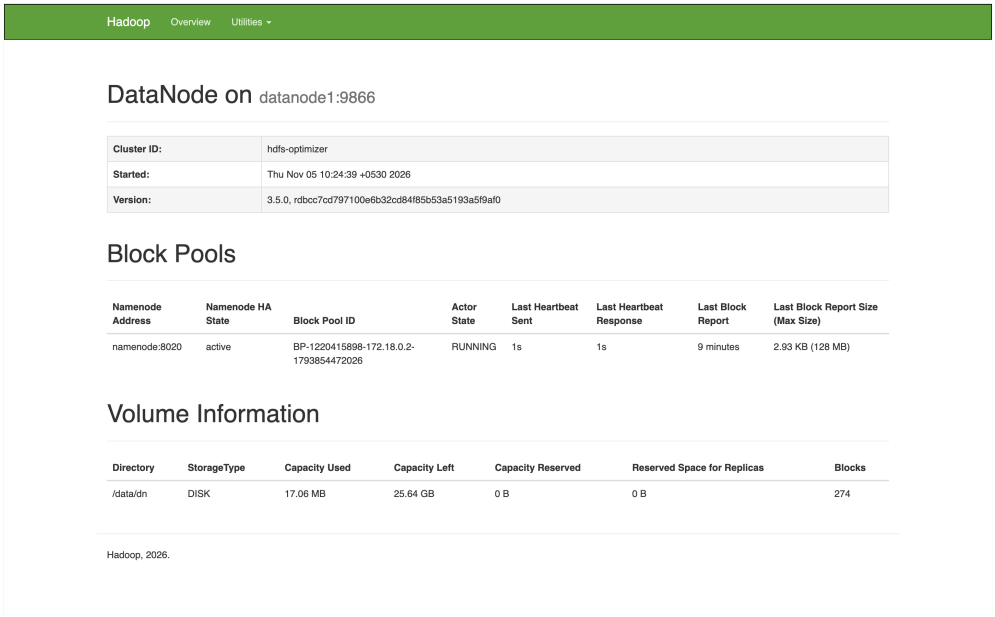

Figure 15: One DataNode's own web interface

```
# Captured 2026-11-05T06:41:52Z

$ curl -s http://127.0.0.1:8100/api/v1/health
{"status":"ok","database":"ok","hdfs_reachable":true,"hdfs_health":"healthy"}[exit code 0]

$ curl -s http://127.0.0.1:8100/api/v1/storage/savings --output /dev/null -w HTTP %{http_code} in %{time_total}s\n
HTTP 200 in 0.017953s
[exit code 0]

$ python3 -c import json,urllib.request as u; d=json.load(u.urlopen('http://127.0.0.1:8100/api/v1/storage/statistics'));
print(json.dumps(d['derived'],indent=1)); print(json.dumps({k:d['measured'][k] for k in
('length','space_consumed','file_count','directory_count')},indent=1))
{
 "logical_files": 34,
 "logical_bytes": 46699993,
 "unique_files": 15,
 "unique_content_bytes": 17897126,
 "duplicate_files": 19,
 "deleted_retained_files": 0,
 "deleted_retained_bytes": 0,
 "raw_files": 0,
 "raw_bytes": 0,
 "object_count": 0,
 "object_bytes": 0,
 "chunk_count": 259,
 "chunk_bytes": 16534916,
 "manifest_count": 15,
 "manifest_bytes": 33001,
 "chunk_references": 280,
 "duplicate_chunk_references": 21,
 "stored_content_bytes": 16534916,
 "bytes_avoided": 30165077,
 "savings_percent": 64.59332231591554,
 "dedup_ratio": 2.824325989923384,
 "files_by_mode": {
  "chunk": 34
 }
}
{
 "length": 16567917,
 "space_consumed": 49703751,
 "file_count": 274,
 "directory_count": 491
}
[exit code 0]

$ tail -n 12 backend/data/backend.log
INFO:     127.0.0.1:55548 - "GET /api/v1/settings HTTP/1.1" 200 OK
INFO:     127.0.0.1:55557 - "GET /api/v1/files/folders HTTP/1.1" 200 OK
INFO:     127.0.0.1:55554 - "GET /api/v1/files?status=active&sort=created&order=desc&page=1&page_size=25 HTTP/1.1" 200 OK
INFO:     127.0.0.1:55554 - "OPTIONS /api/v1/files/upload HTTP/1.1" 200 OK
INFO:     127.0.0.1:55557 - "POST /api/v1/files/upload HTTP/1.1" 503 Service Unavailable
INFO:     127.0.0.1:55554 - "GET /api/v1/files/folders HTTP/1.1" 200 OK
INFO:     127.0.0.1:55557 - "GET /api/v1/files?status=active&sort=created&order=desc&page=1&page_size=25 HTTP/1.1" 200 OK
INFO:     127.0.0.1:55562 - "GET /api/v1/cluster/status HTTP/1.1" 200 OK
INFO:     127.0.0.1:55565 - "GET /api/v1/system/diagnostics HTTP/1.1" 200 OK
INFO:     127.0.0.1:55570 - "GET /api/v1/health HTTP/1.1" 200 OK
INFO:     127.0.0.1:55572 - "GET /api/v1/storage/savings HTTP/1.1" 200 OK
INFO:     127.0.0.1:55573 - "GET /api/v1/storage/statistics HTTP/1.1" 200 OK
[exit code 0]
```

Figure 16: Running backend: health check, live statistics and request log

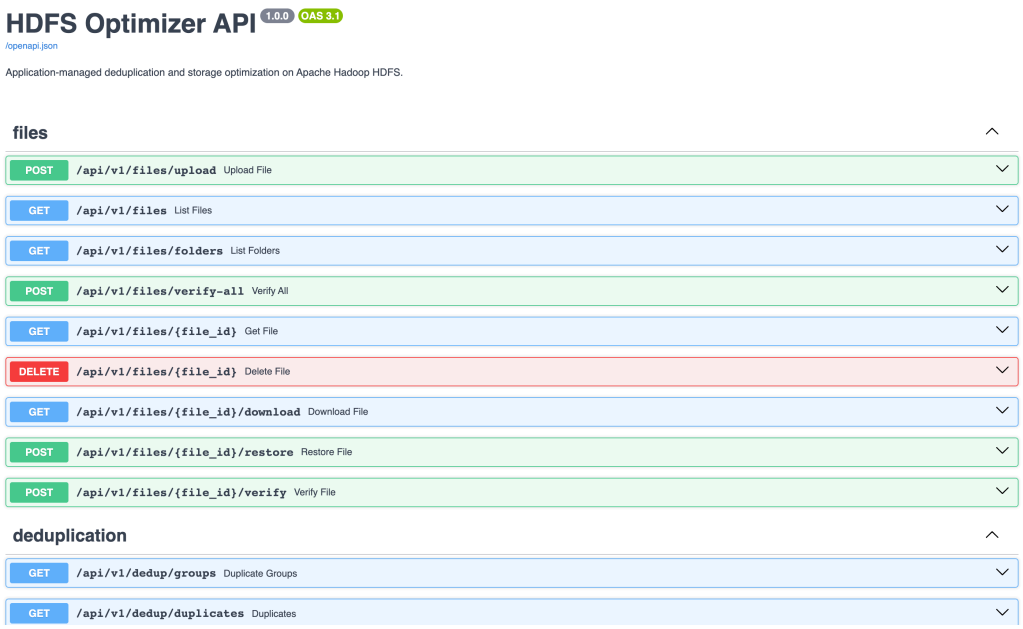

Figure 17: Generated OpenAPI documentation, served by the backend

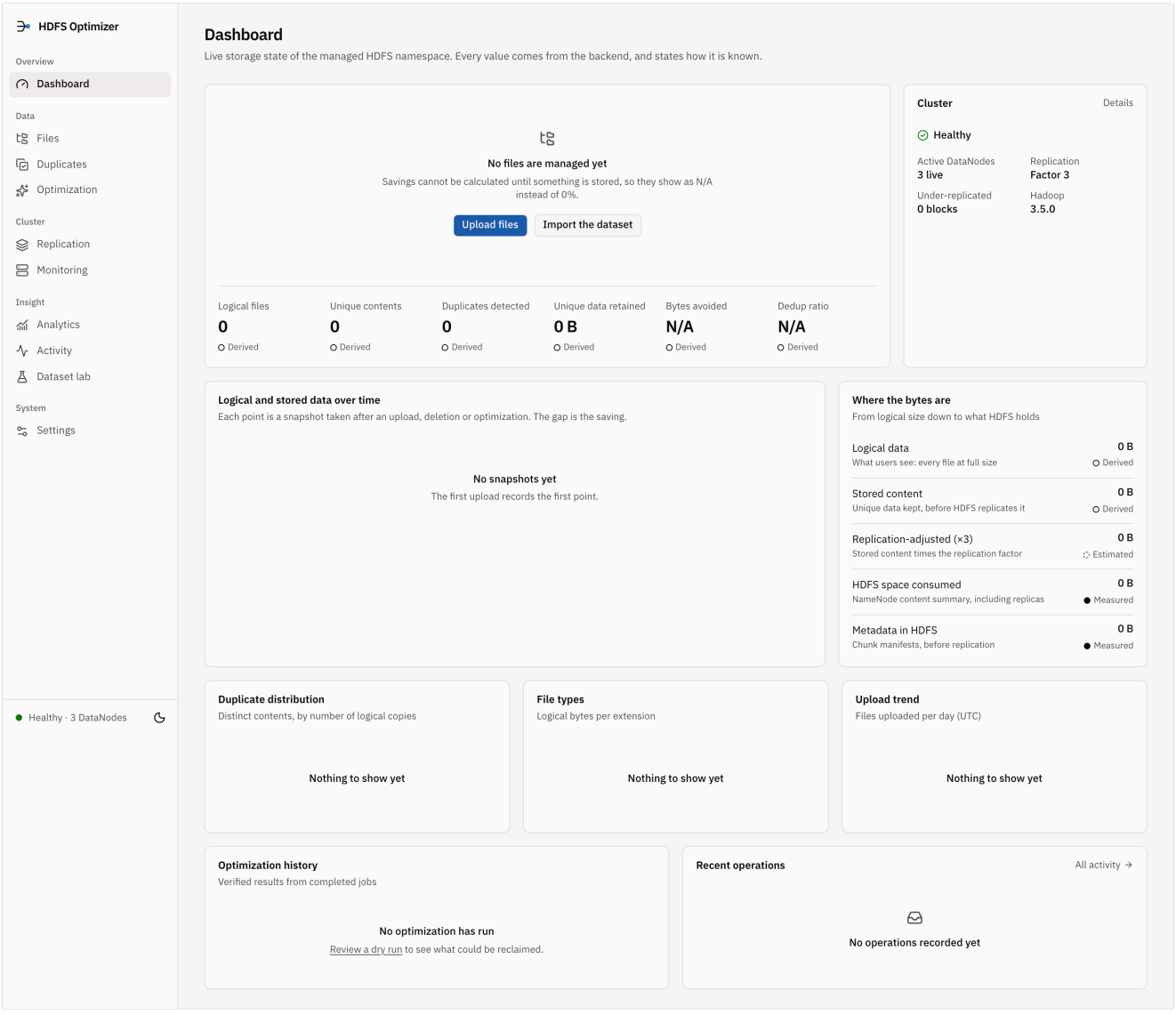

Figure 18: Running frontend on an empty system. Undefined metrics show N/A, not 0%.

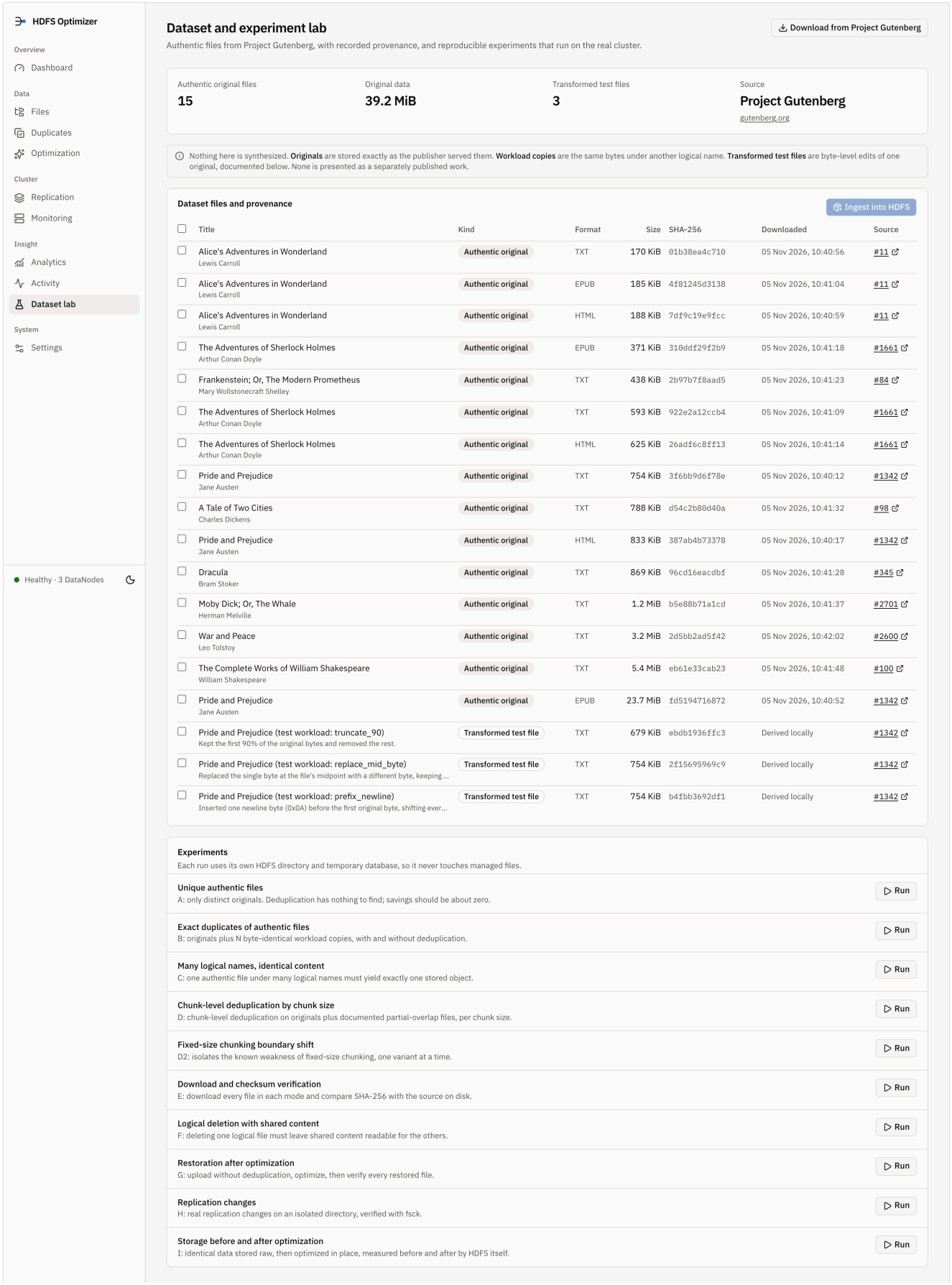

Figure 19: Dataset lab with recorded provenance

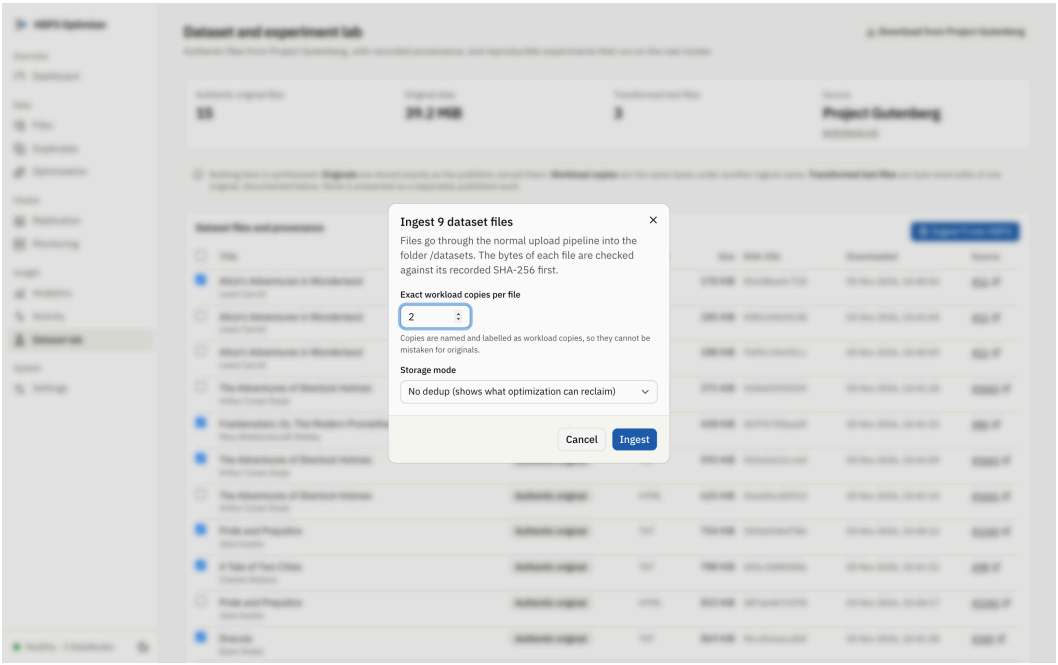

Figure 20: Dataset ingestion with labelled workload copies

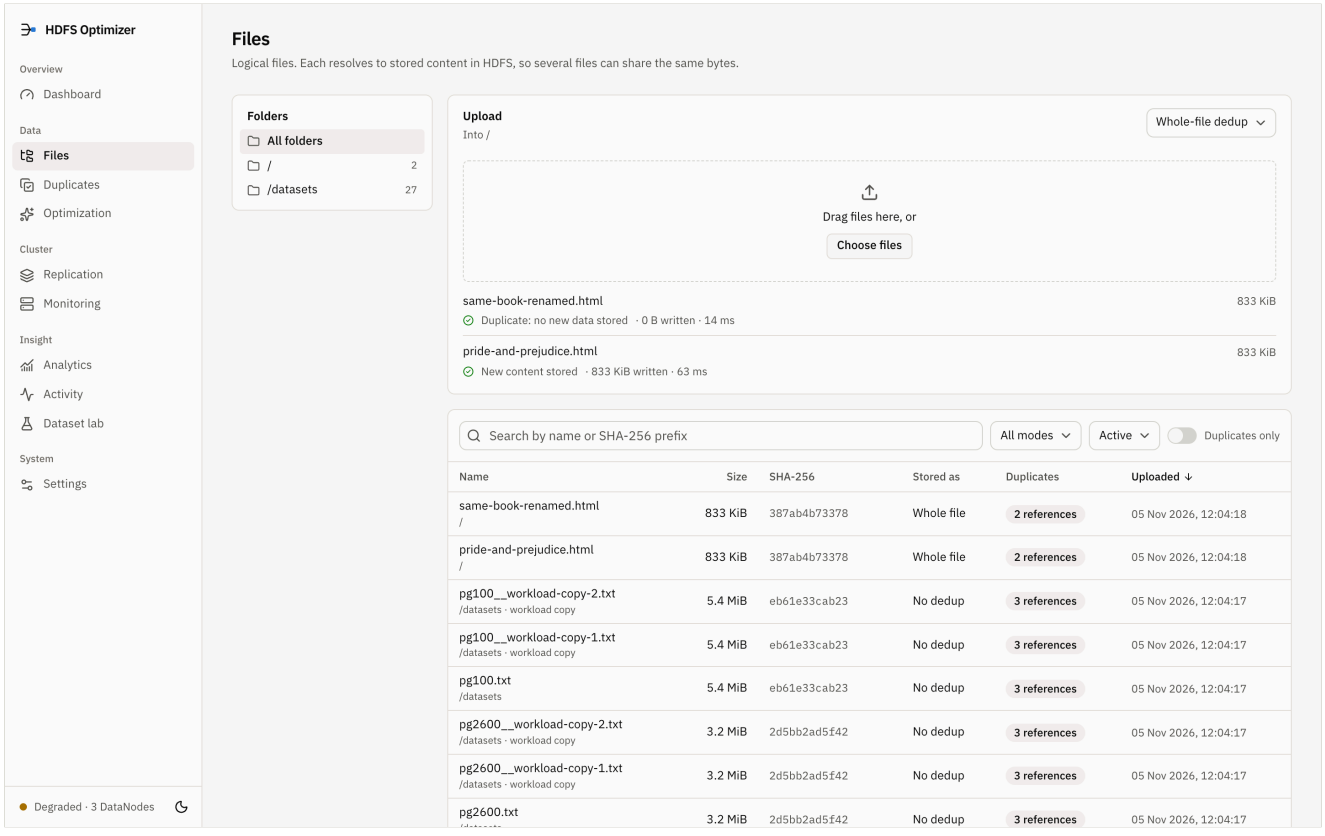

Figure 21: File upload through the interface. The second upload of the same bytes under a new name is reported as a duplicate.

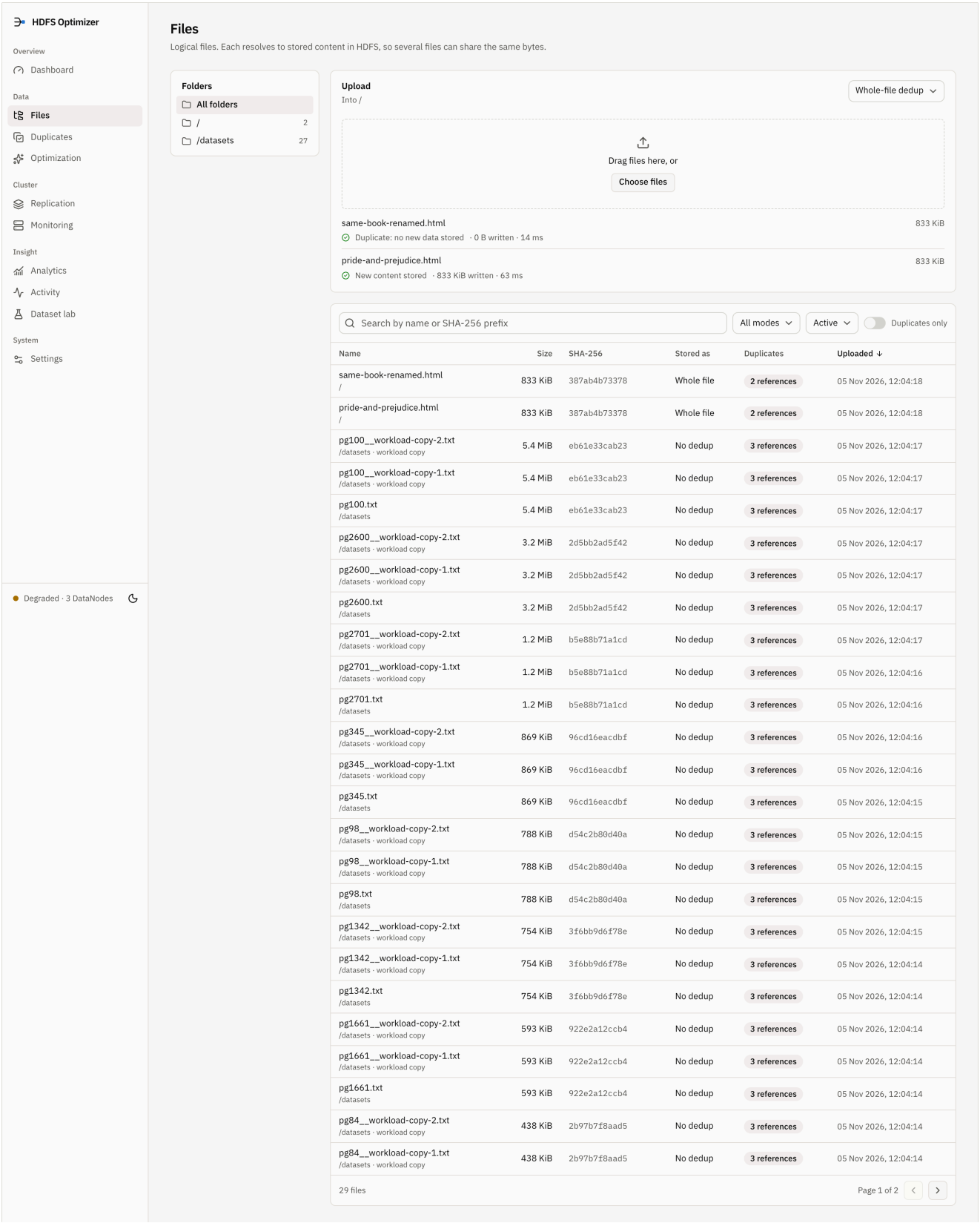

Figure 22: File explorer after ingestion

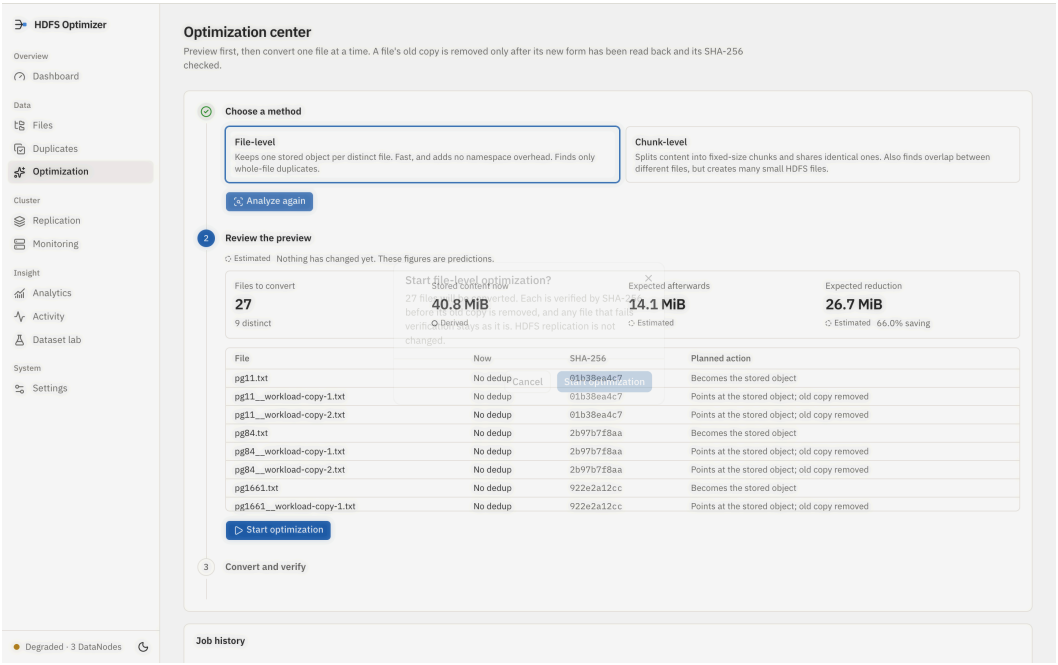

Figure 23: Deduplication execution: confirmation after a reviewed preview

```
# Captured 2026-11-05T06:41:43Z

$ docker compose exec -T namenode hdfs dfs -ls /hdfs-optimizer
Found 6 items
drwxr-xr-x   - hadoop supergroup          0 2026-11-05 06:34 /hdfs-optimizer/chunks
drwxr-xr-x   - hadoop supergroup          0 2026-11-05 06:34 /hdfs-optimizer/manifests
drwxr-xr-x   - hadoop supergroup          0 2026-11-05 06:34 /hdfs-optimizer/objects
drwxr-xr-x   - hadoop supergroup          0 2026-11-05 06:34 /hdfs-optimizer/raw
drwxr-xr-x   - hadoop supergroup          0 2026-11-05 06:34 /hdfs-optimizer/recovery
drwxr-xr-x   - hadoop supergroup          0 2026-11-05 06:41 /hdfs-optimizer/staging
[exit code 0]

$ docker compose exec -T namenode hdfs dfs -count -q -h /hdfs-optimizer
        none             inf            none             inf          491          274             15.8 M /hdfs-optimizer
[exit code 0]

$ docker compose exec -T namenode hdfs dfs -du -s -h /hdfs-optimizer/objects /hdfs-optimizer/chunks /hdfs-optimizer/manifests /hdfs-optimizer/raw
0       0       /hdfs-optimizer/objects
15.8 M  47.3 M  /hdfs-optimizer/chunks
32.2 K  96.7 K  /hdfs-optimizer/manifests
0       0       /hdfs-optimizer/raw
[exit code 0]

$ docker compose exec -T namenode bash -c hdfs dfs -ls -R /hdfs-optimizer/chunks | grep "^-" | head -8
-rw-r--r--   3 hadoop supergroup      65536 2026-11-05 06:34 /hdfs-
optimizer/chunks/00/08/0008e36373b7412301eb430bbdb4c76ce5de833b66d69dd3b0f2654e54720b01
-rw-r--r--   3 hadoop supergroup      65536 2026-11-05 06:34 /hdfs-
optimizer/chunks/01/60/016084b9f0faa660fae389dc654994dd1520d22002a25a3d444361d8a5a67ffe
-rw-r--r--   3 hadoop supergroup      65536 2026-11-05 06:34 /hdfs-
optimizer/chunks/02/1e/021e54a0007fa28423f40b15e3cb491da022dcd1cd1ff5b22659c32e288e151c
-rw-r--r--   3 hadoop supergroup      65536 2026-11-05 06:34 /hdfs-
optimizer/chunks/02/55/0255d3cd0f3c0b97f702b515591fc5dd8cbdcf4658d0da196d70ff2c84fe9ea4
-rw-r--r--   3 hadoop supergroup       2385 2026-11-05 06:34 /hdfs-
optimizer/chunks/04/a4/04a46892613e8fdb2da73fea3859959742d37b8a49cd01ae4a50f7e61f469c1b
-rw-r--r--   3 hadoop supergroup      65536 2026-11-05 06:34 /hdfs-
optimizer/chunks/04/f9/04f95ca1bdae5f2bf5576291ff14446be3c8ed4a3c04759995e3cf82dcaa4deb
-rw-r--r--   3 hadoop supergroup      51490 2026-11-05 06:34 /hdfs-
optimizer/chunks/06/8a/068a6b07975037f297d084551f9d305d174f62e66e75c6d0d5a8f7240e764e14
-rw-r--r--   3 hadoop supergroup      65536 2026-11-05 06:34 /hdfs-
optimizer/chunks/09/b1/09b100ab626e0dde537d60736d9e8e582bcaa89c0db71387b603eb7334872e6a
[exit code 0]
```

Figure 24: HDFS command-line view of the managed namespace after optimization

```
# Captured 2026-11-05T06:41:47Z

$ docker compose exec -T namenode bash -c hdfs fsck /hdfs-optimizer 2>/dev/null | grep -A16 "^Status"; hdfs fsck /hdfs-optimizer 2>/dev/null |
tail -2
Status: HEALTHY
 Number of data-nodes:  3
 Number of racks:               1
 Total dirs:                    491
 Total symlinks:                0

Replicated Blocks:
 Total size:    16567917 B
 Total files:   274
 Total blocks (validated):      274 (avg. block size 60466 B)
 Minimally replicated blocks:   274 (100.0 %)
 Over-replicated blocks:        0 (0.0 %)
 Under-replicated blocks:       0 (0.0 %)
 Mis-replicated blocks:         0 (0.0 %)
 Default replication factor:    3
 Average block replication:     3.0
 Missing blocks:                0

The filesystem under path '/hdfs-optimizer' is HEALTHY
[exit code 0]

$ docker compose exec -T namenode bash -c F=$(hdfs dfs -ls -R /hdfs-optimizer/chunks | grep "^-" | head -1 | awk "{print \$NF}"); hdfs fsck "$F"
-files -blocks -locations 2>/dev/null | head -8
FSCK started by hadoop (auth:SIMPLE) from /172.18.0.2 for path /hdfs-
optimizer/chunks/00/08/0008e36373b7412301eb430bbdb4c76ce5de833b66d69dd3b0f2654e54720b01 at Thu Nov 05 06:41:52 UTC 2026

/hdfs-optimizer/chunks/00/08/0008e36373b7412301eb430bbdb4c76ce5de833b66d69dd3b0f2654e54720b01 65536 bytes, replicated: replication=3, 1 block(s):
OK
0. BP-1220415898-172.18.0.2-1793854472026:blk_1073751754_10930 len=65536 Live_repl=3  [DatanodeInfoWithStorage[172.18.0.5:9866,DS-069e40c2-550b-
48ed-afb6-ece5cd280736,DISK], DatanodeInfoWithStorage[172.18.0.4:9866,DS-f600f7e4-a8f2-4628-877e-0d7ca43dd90a,DISK],
DatanodeInfoWithStorage[172.18.0.3:9866,DS-c6ceee78-740d-44fa-9d8f-981890b460ee,DISK]]


Status: HEALTHY
 Number of data-nodes:  3
[exit code 0]
```

Figure 25: `hdfs fsck` on the managed namespace, and the replica locations of one object

```
# Captured 2026-11-05T06:41:52Z

$ pytest -m "not hdfs" -v
.......................................................................... [ 55%]
.....................................................                    [100%]
129 passed, 17 deselected in 4.62s
```

Figure 26: Unit test execution

```
# Captured 2026-11-05T06:41:59Z

$ pytest -m hdfs -v  (real HDFS cluster)
test_authentic_file_matches_recorded_provenance PASSED [  5%]
test_webhdfs_primitives_on_the_real_cluster PASSED [ 11%]
test_empty_file_can_be_stored_on_the_real_cluster PASSED [ 17%]
test_authentic_file_round_trips_byte_for_byte[raw] PASSED [ 23%]
test_authentic_file_round_trips_byte_for_byte[file] PASSED [ 29%]
test_authentic_file_round_trips_byte_for_byte[chunk] PASSED [ 35%]
test_binary_authentic_file_round_trips_through_chunks PASSED [ 41%]
test_exact_duplicate_consumes_no_additional_hdfs_space PASSED [ 47%]
test_hdfs_replicates_stored_objects PASSED [ 52%]
test_measured_space_includes_replication PASSED [ 58%]
test_optimization_reduces_hdfs_measured_space_and_preserves_bytes PASSED [ 64%]
test_chunk_optimization_on_real_hdfs PASSED [ 70%]
test_deleting_shared_reference_on_real_hdfs PASSED [ 76%]
test_cluster_monitor_reads_real_namenode PASSED [ 82%]
test_fsck_servlet_is_parsed PASSED       [ 88%]
test_replication_change_converges_on_real_cluster PASSED [ 94%]
test_replication_beyond_live_datanodes_is_refused_on_real_cluster PASSED [100%]
==================== 17 passed, 129 deselected in 16.77s ====================
```

Figure 27: Integration tests on the real HDFS cluster

```
# Captured 2026-11-05T06:42:16Z

$ npx playwright test  (real browser, real cluster)
Running 13 tests using 1 worker
  ✓   1 e2e/app.spec.ts:46:7 › HDFS Optimizer › every page renders without console errors (7.1s)
  ✓   2 e2e/app.spec.ts:56:7 › HDFS Optimizer › an empty system shows N/A, not invented values (383ms)
  ✓   3 e2e/app.spec.ts:62:7 › HDFS Optimizer › the cluster page shows real DataNodes (355ms)
  ✓   4 e2e/app.spec.ts:69:7 › HDFS Optimizer › uploading the same authentic file twice stores it once (1.5s)
  ✓   5 e2e/app.spec.ts:78:7 › HDFS Optimizer › download restores identical bytes, and verification passes (1.1s)
  ✓   6 e2e/app.spec.ts:92:7 › HDFS Optimizer › the duplicates page explains the relationship (321ms)
  ✓   7 e2e/app.spec.ts:98:7 › HDFS Optimizer › delete keeps shared content and restore brings the file back (1.3s)
  ✓   8 e2e/app.spec.ts:111:7 › HDFS Optimizer › optimization needs a preview, then reclaims space and keeps bytes intact (4.6s)
  ✓   9 e2e/app.spec.ts:135:7 › HDFS Optimizer › chunk mode shows shared chunks in the chunk map (1.4s)
  ✓  10 e2e/app.spec.ts:148:7 › HDFS Optimizer › replication above live DataNodes is not offered, and lowering asks for confirmation (648ms)
  ✓  11 e2e/app.spec.ts:157:7 › HDFS Optimizer › the audit trail records what happened (313ms)
  ✓  12 e2e/app.spec.ts:164:7 › HDFS Optimizer › diagnostics run real checks against the cluster (580ms)
  ✓  13 e2e/app.spec.ts:171:7 › HDFS Optimizer › mobile layout has no horizontal page scroll (6.4s)
  13 passed (28.6s)
```

Figure 28: Browser tests in Chrome, on the real cluster

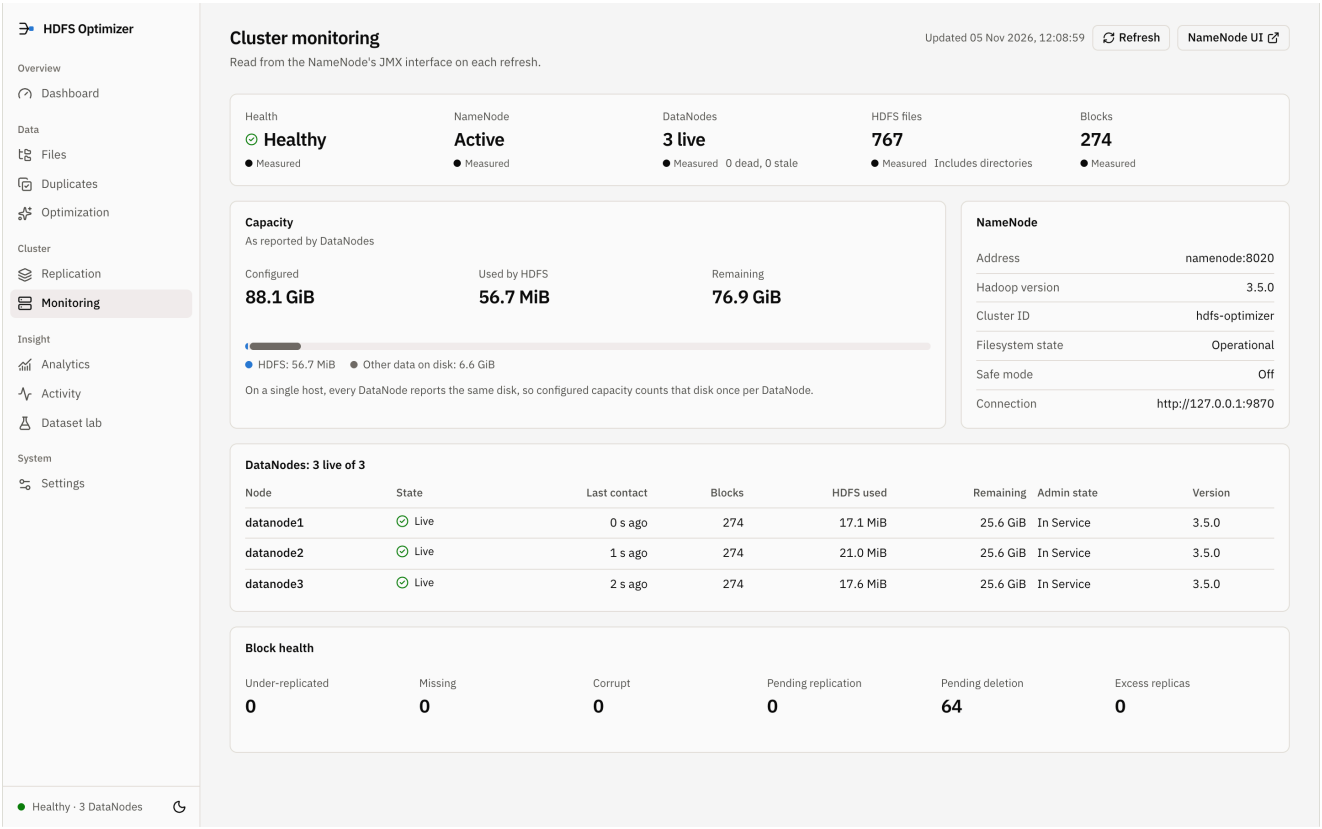

Figure 29: Cluster monitoring page with live NameNode and DataNode data

Chapter 13 — Output and Result Screenshots

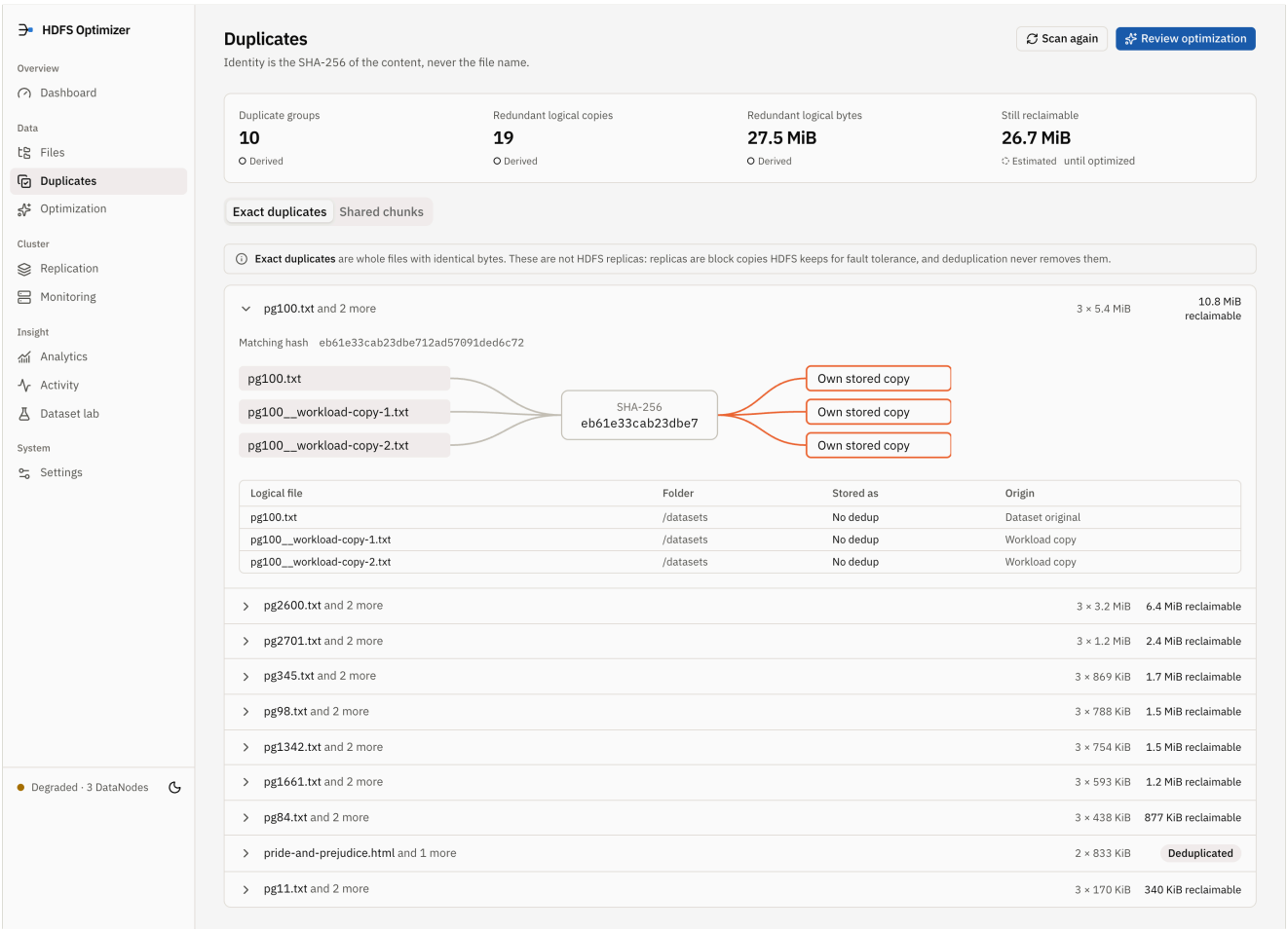

Figure 30: Duplicates found: each group has one SHA-256 and shows how logical names map to stored copies. Here the copies were stored without deduplication.

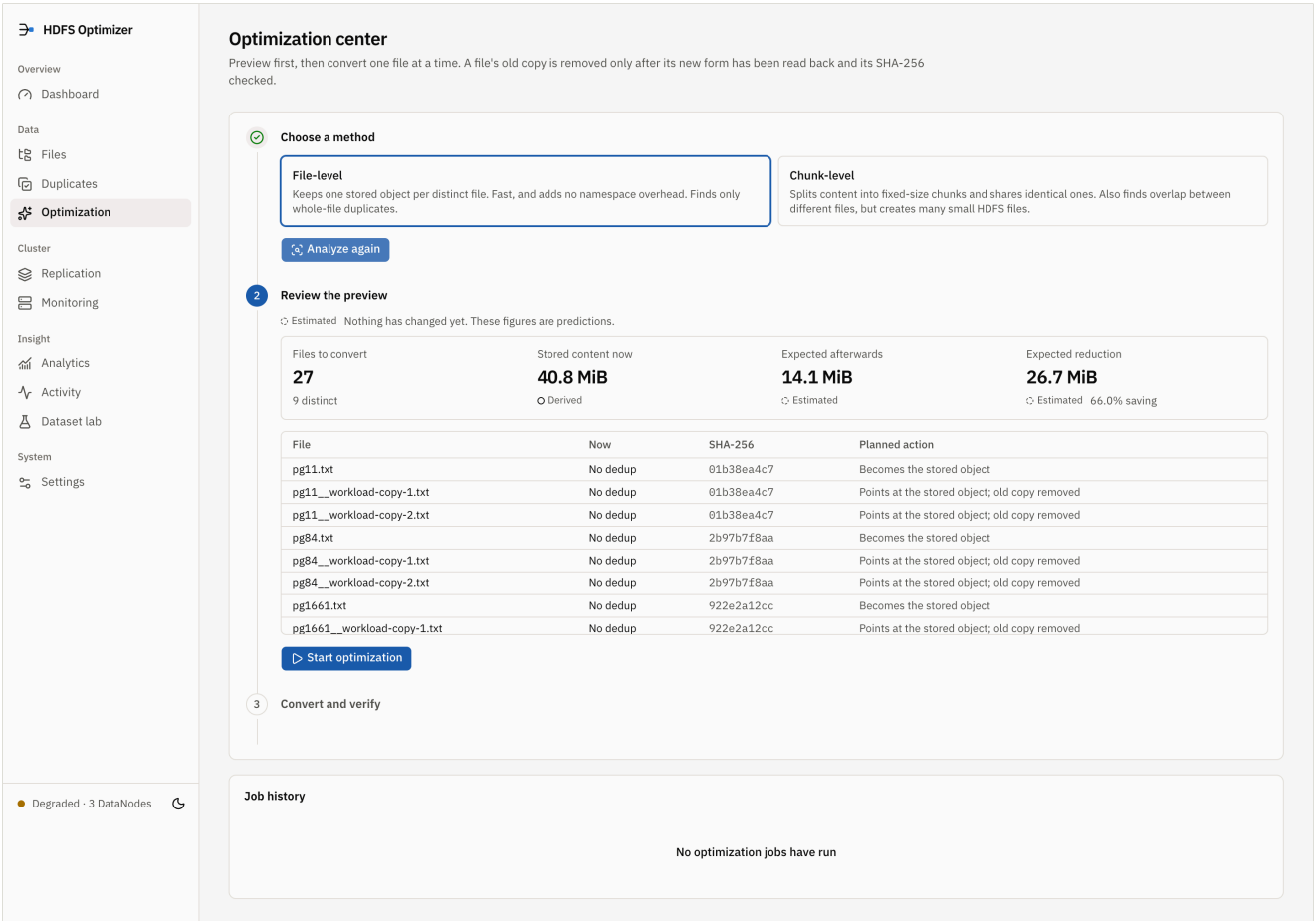

Figure 31: Dry-run preview. Figures are labelled as estimates, and nothing has changed yet.

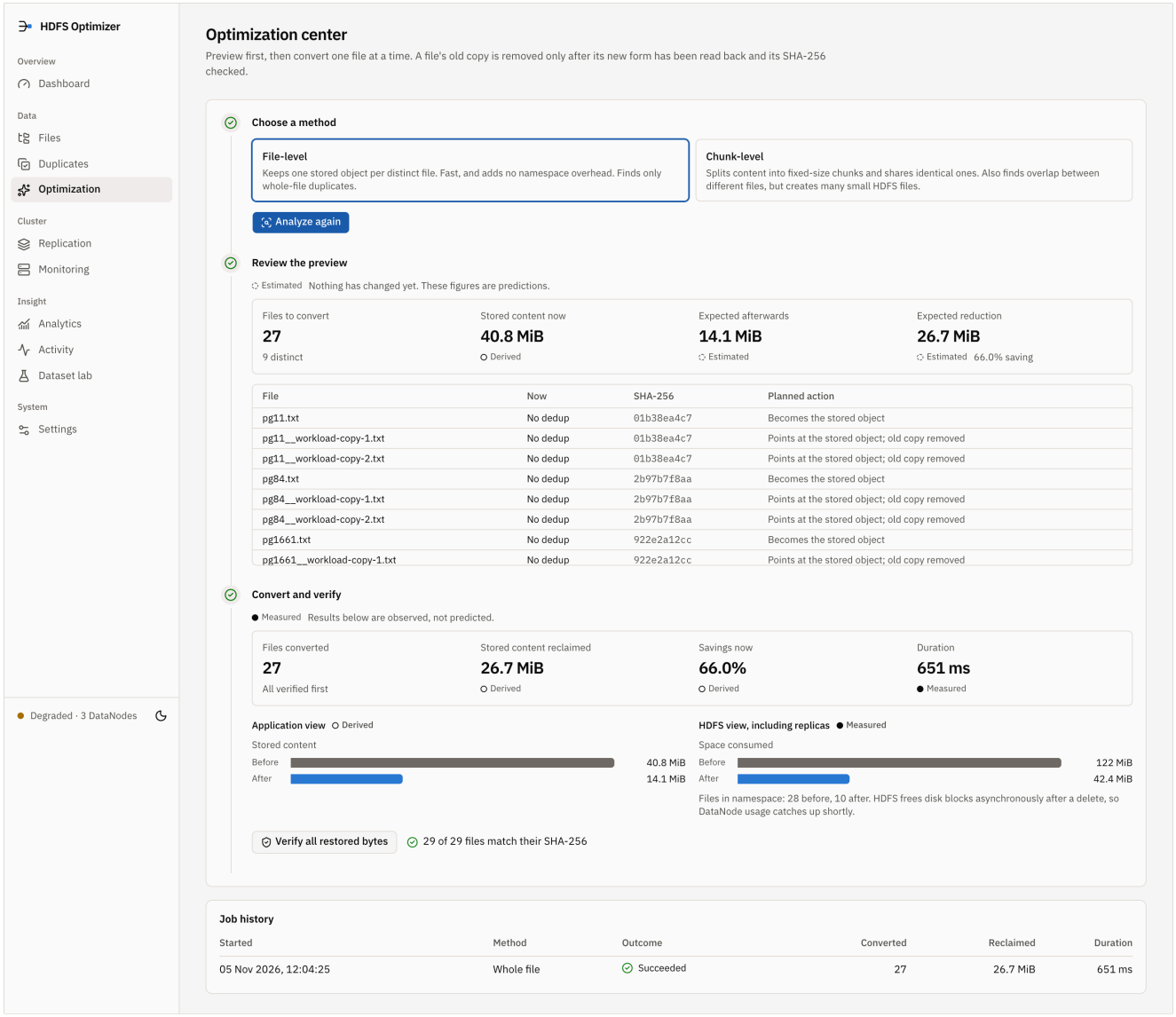

Figure 32: Completed whole-file optimization: measured before and after, and every file verified

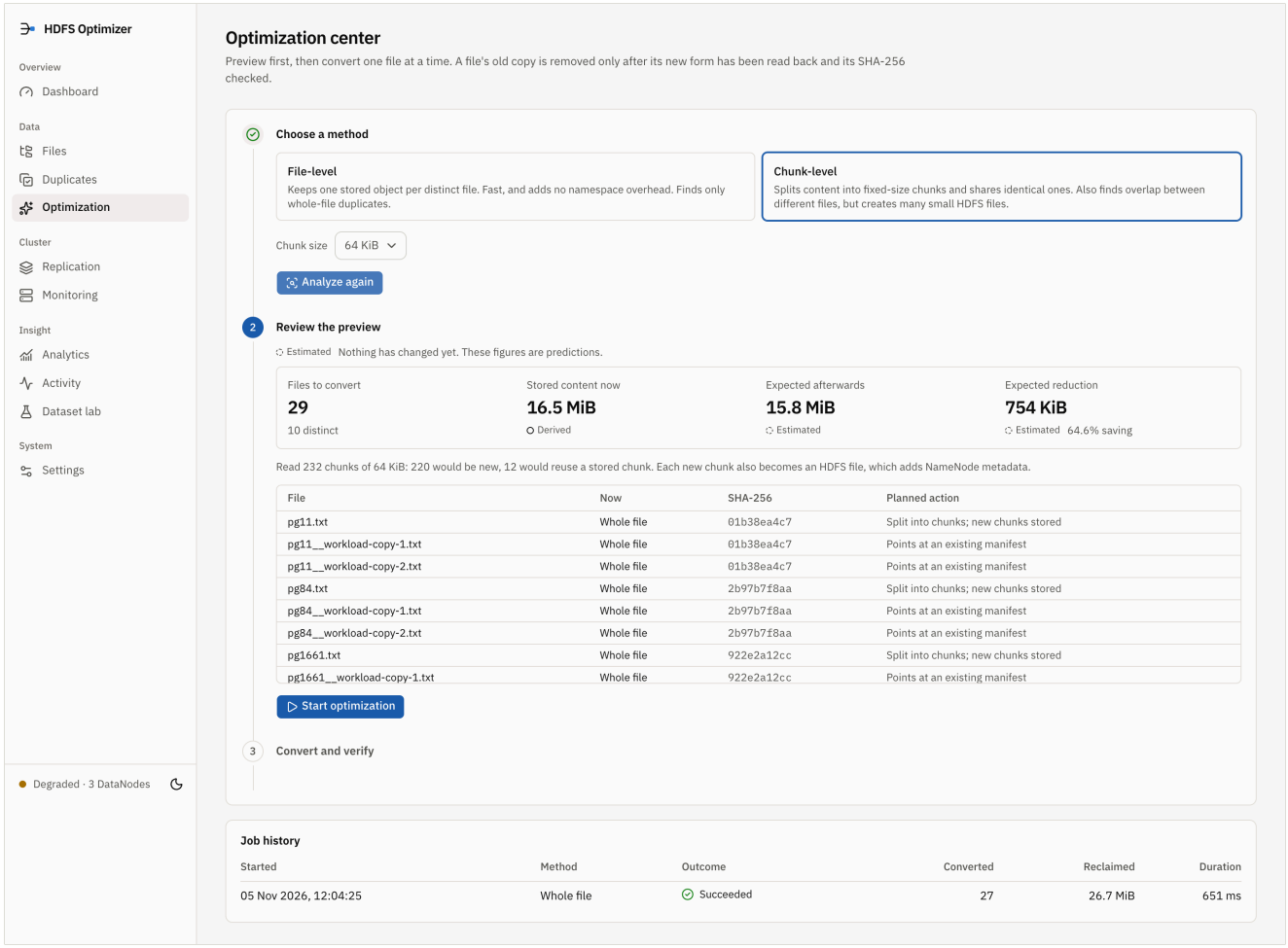

Figure 33: Dry-run preview of chunk-level optimization

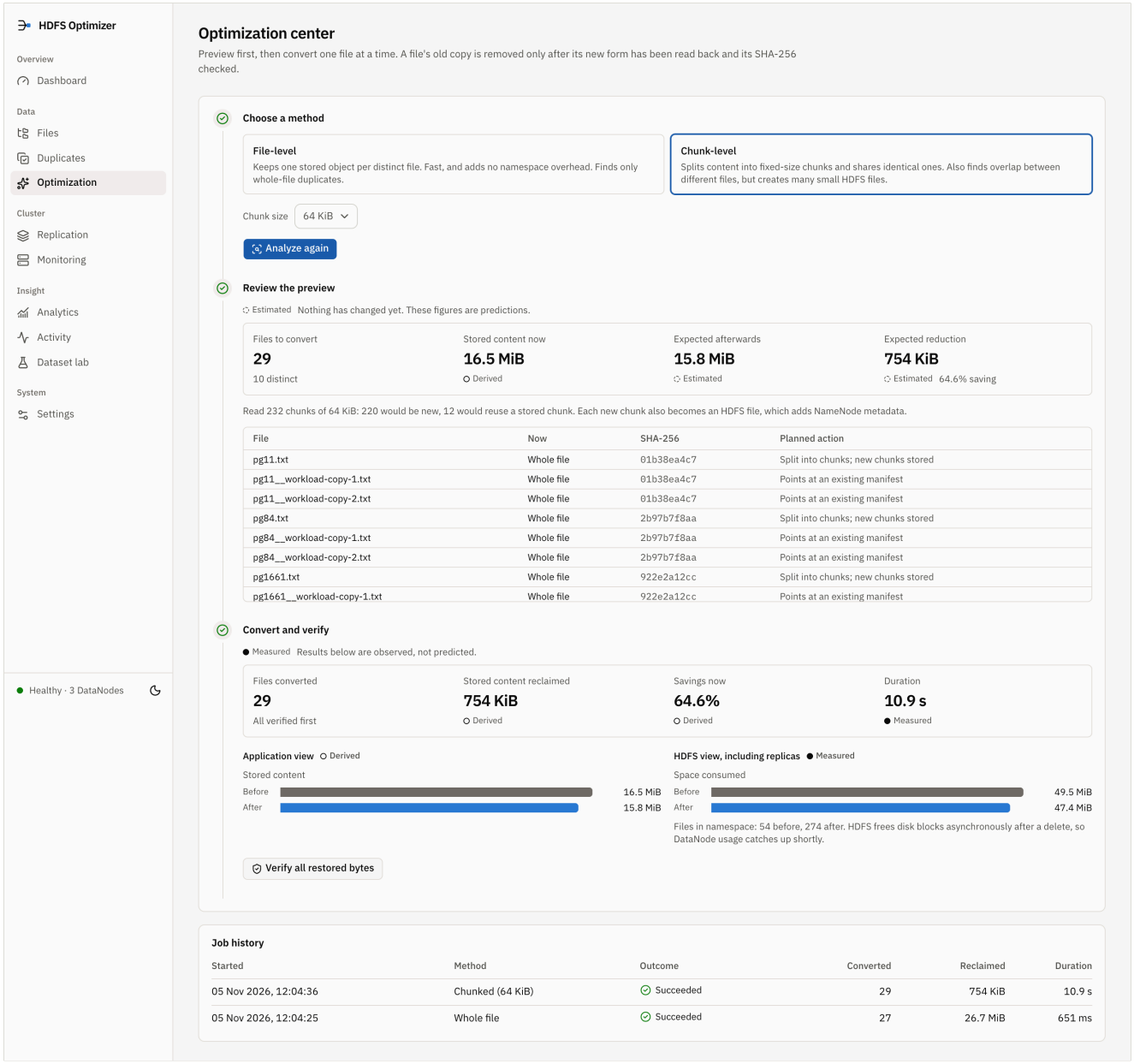

Figure 34: Completed chunk-level optimization

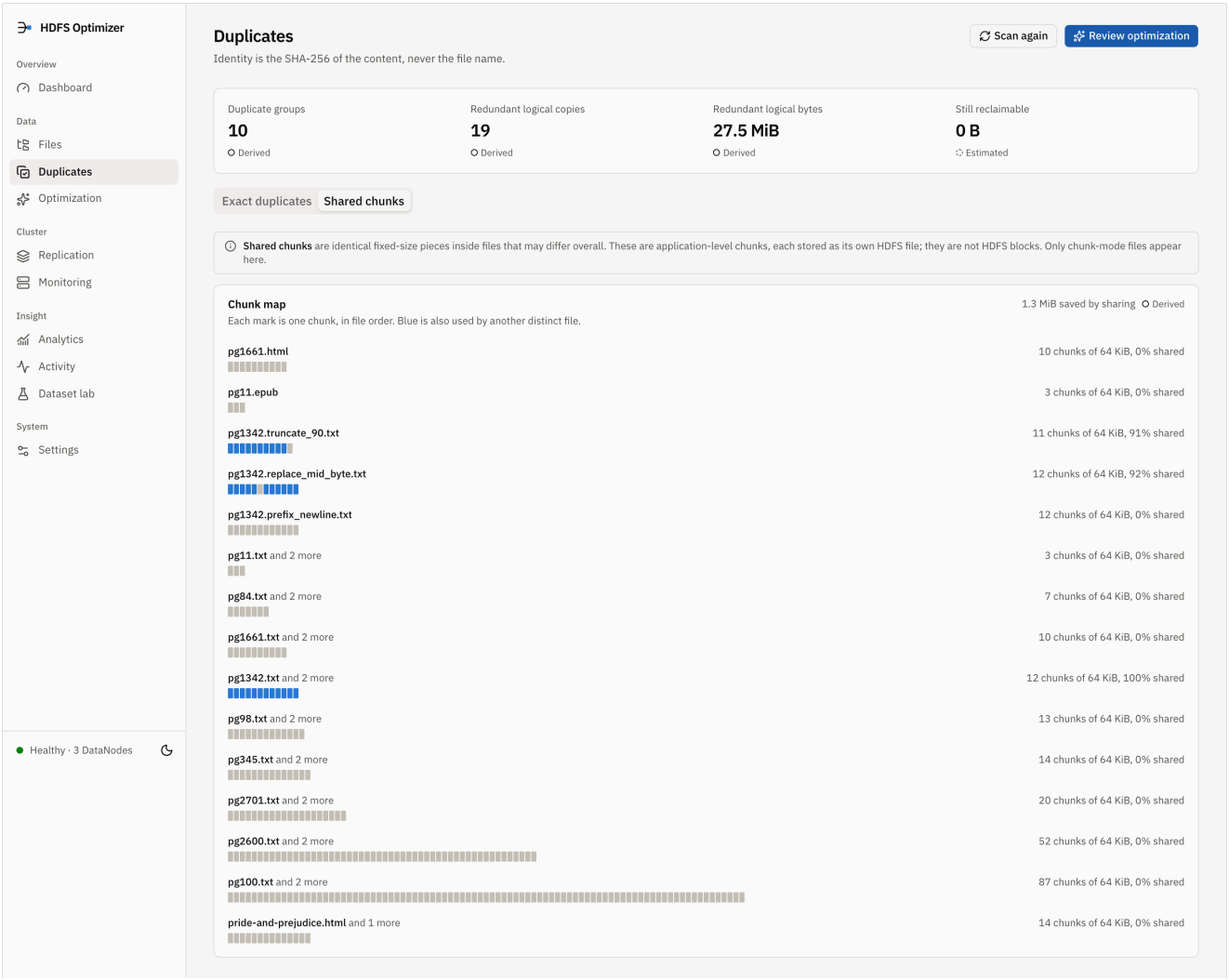

Figure 35: Chunk map: blue marks are chunks shared with another file. A truncated copy and a one-byte edit share almost everything; a one-byte insertion at the start shares nothing.

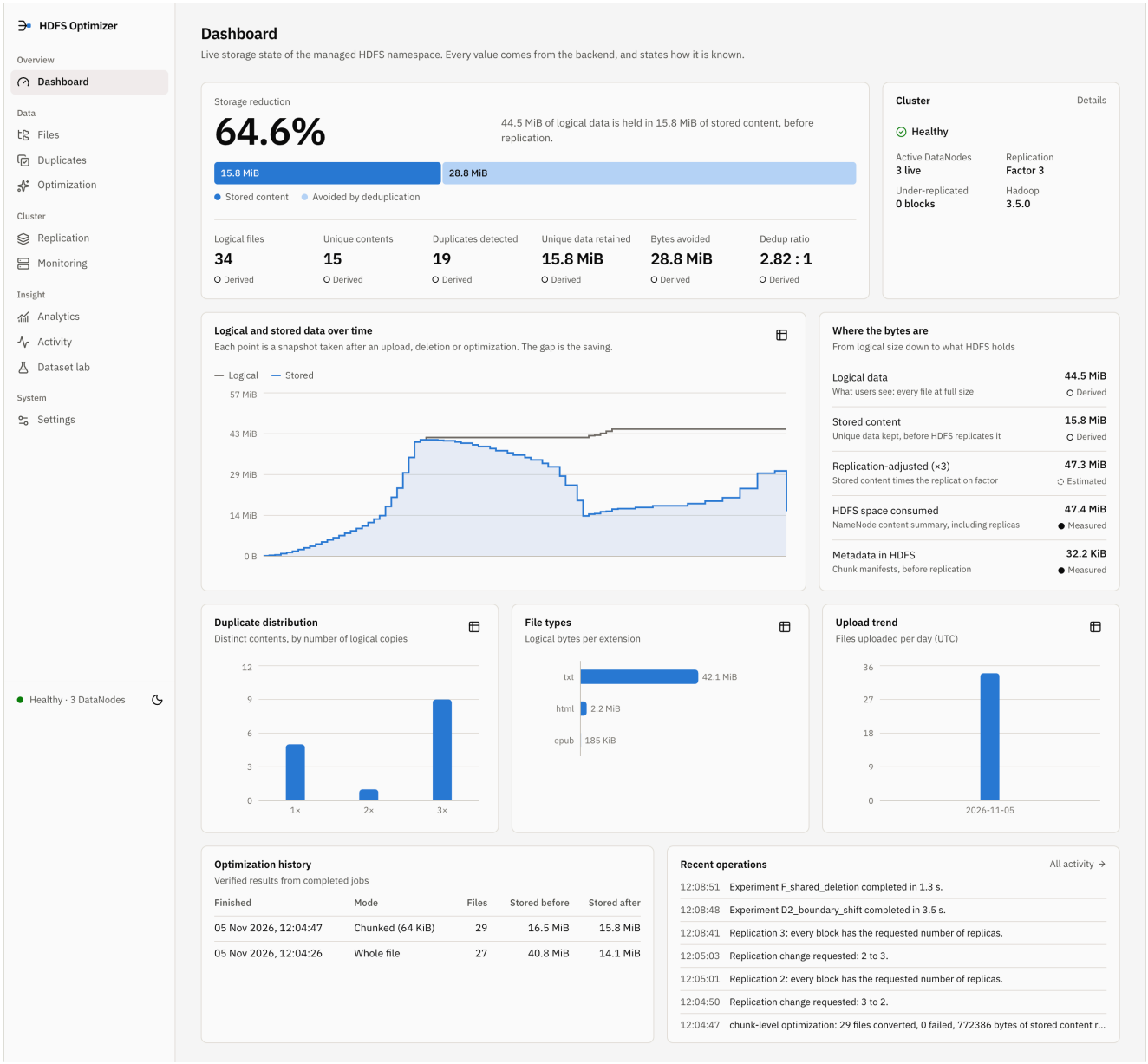

Figure 36: Dashboard with unique content retained and storage savings

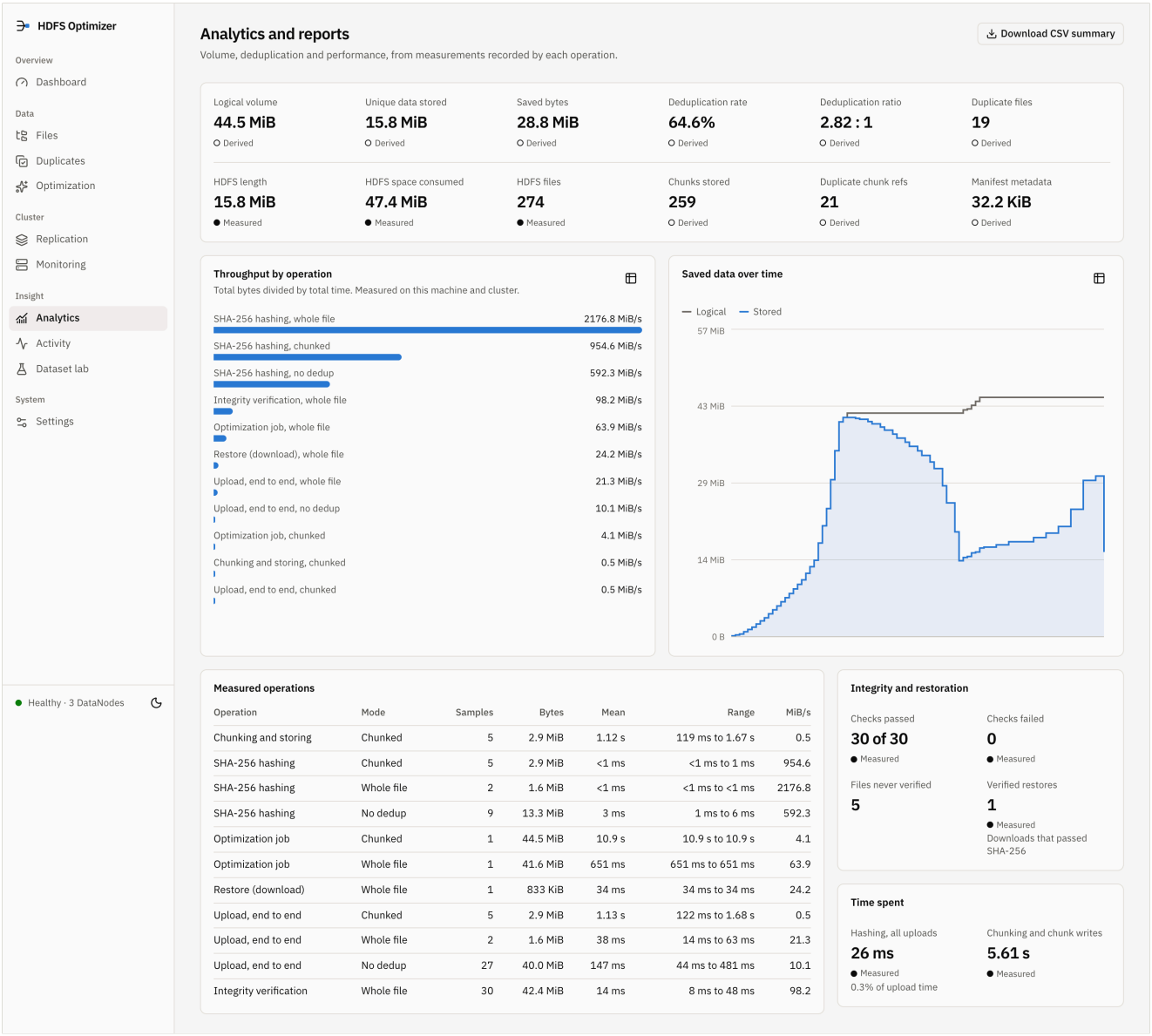

Figure 37: Analytics page with performance measurements and integrity results

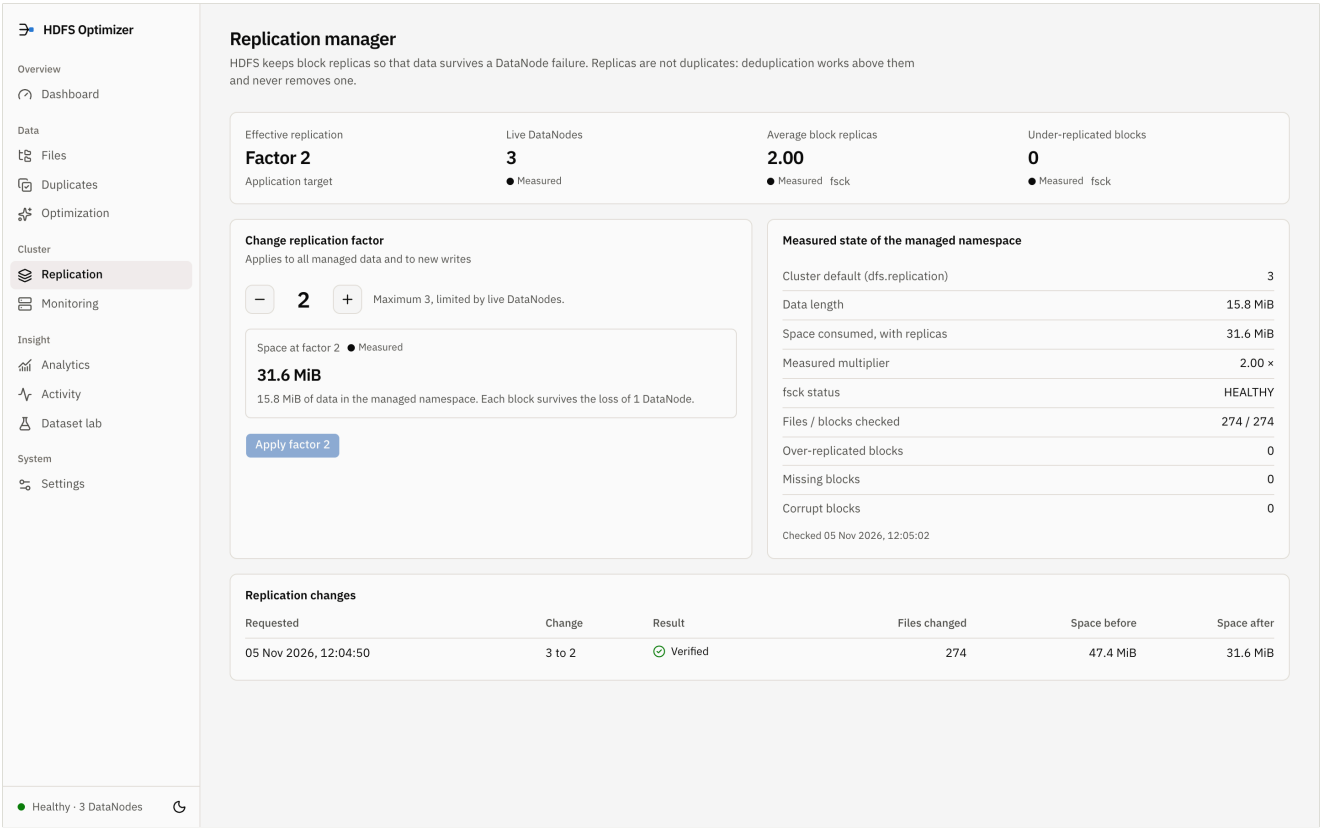

Figure 38: Replication after a verified change to factor 2. Measured space fell accordingly.

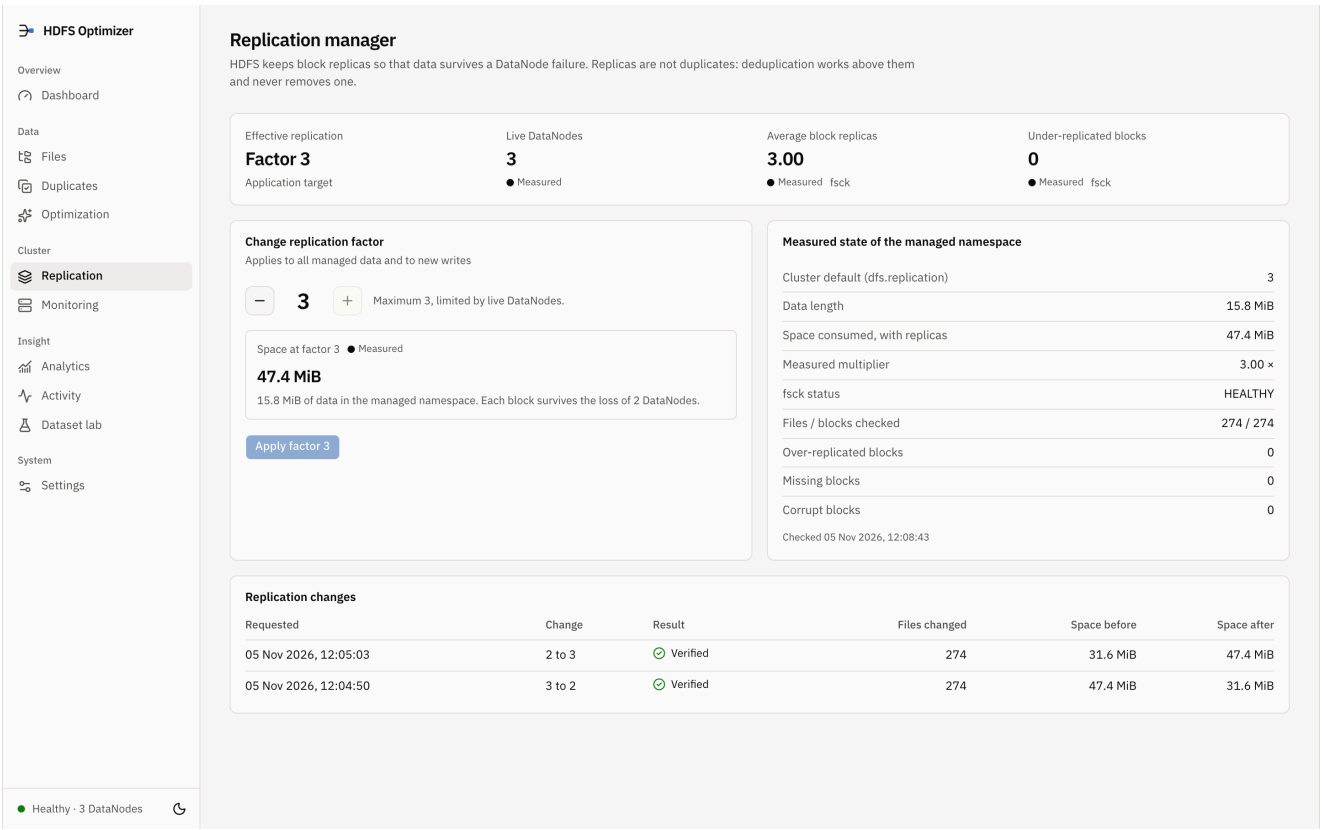

Figure 39: Replication restored to factor 3, with both changes recorded

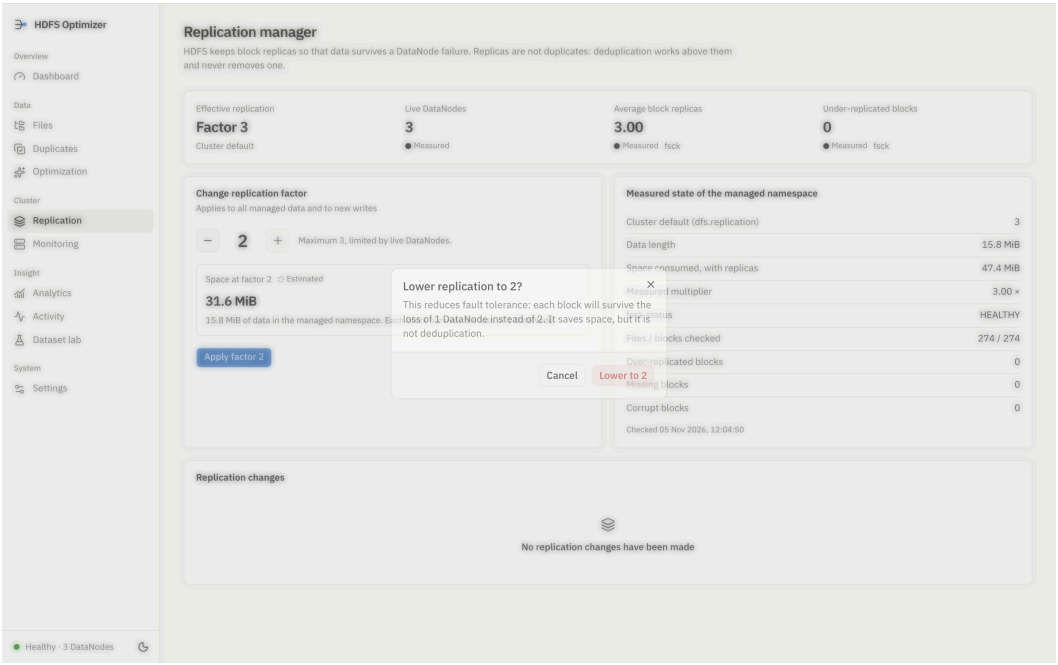

Figure 40: Lowering replication requires explicit confirmation

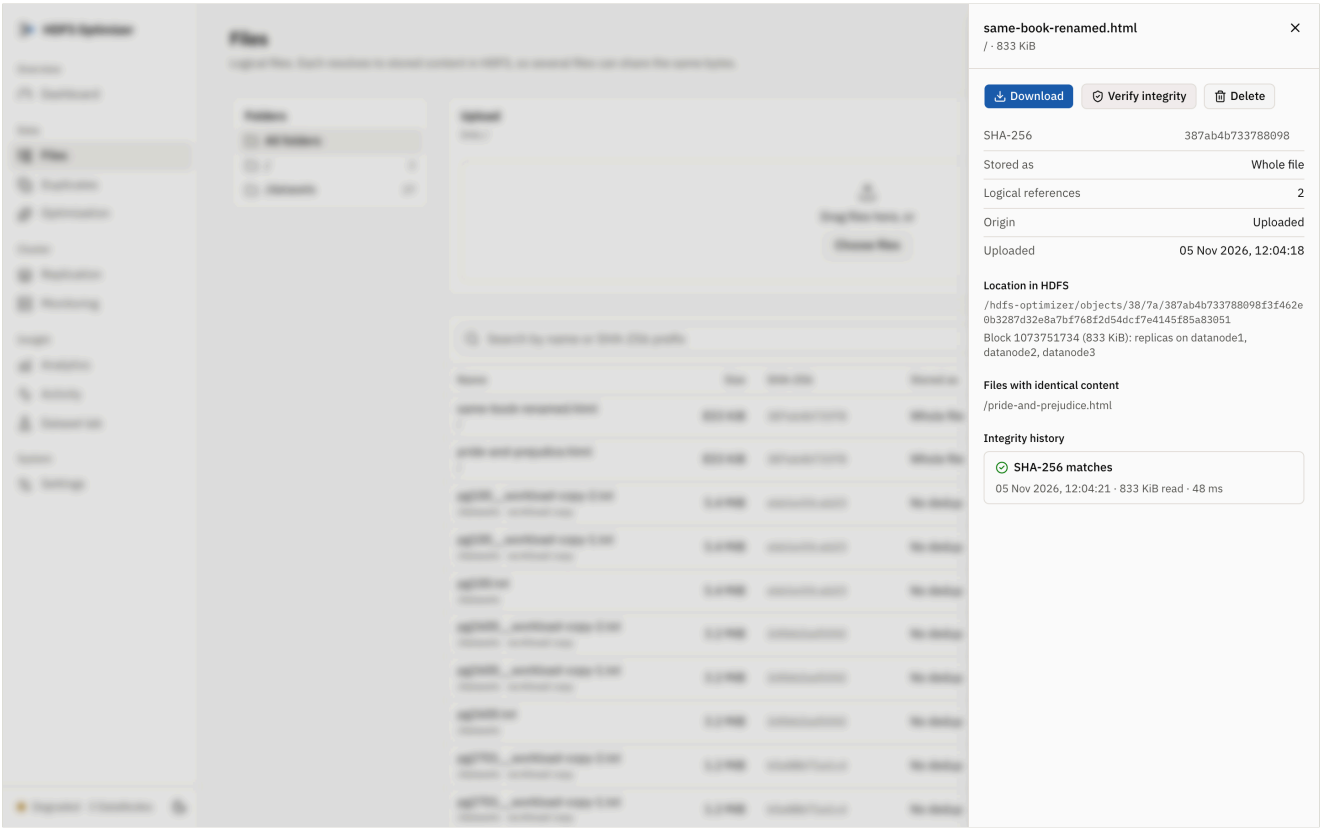

Figure 41: File details: replica locations, and a passed SHA-256 verification

```
# Captured 2026-11-05T06:41:52Z

$ shasum -a 256 datasets/raw/originals/pg1342.html evidence/restored/same-book-renamed.html
387ab4b733788098f3f462e0b3287d32e8a7bf768f2d54dcf7e4145f85a83051  datasets/raw/originals/pg1342.html
387ab4b733788098f3f462e0b3287d32e8a7bf768f2d54dcf7e4145f85a83051  evidence/restored/same-book-renamed.html
[exit code 0]

$ cmp datasets/raw/originals/pg1342.html evidence/restored/same-book-renamed.html
[exit code 0]

$ python3 -c import json;m=json.load(open('datasets/provenance/manifest.json'));print([f['sha256'] for f in m['files'] if
f['local_path']=='originals/pg1342.html'][0], 'recorded in provenance manifest')
387ab4b733788098f3f462e0b3287d32e8a7bf768f2d54dcf7e4145f85a83051 recorded in provenance manifest
[exit code 0]
```

Figure 42: Restored file compared with the publisher's bytes: identical SHA-256, and `cmp` reports no difference

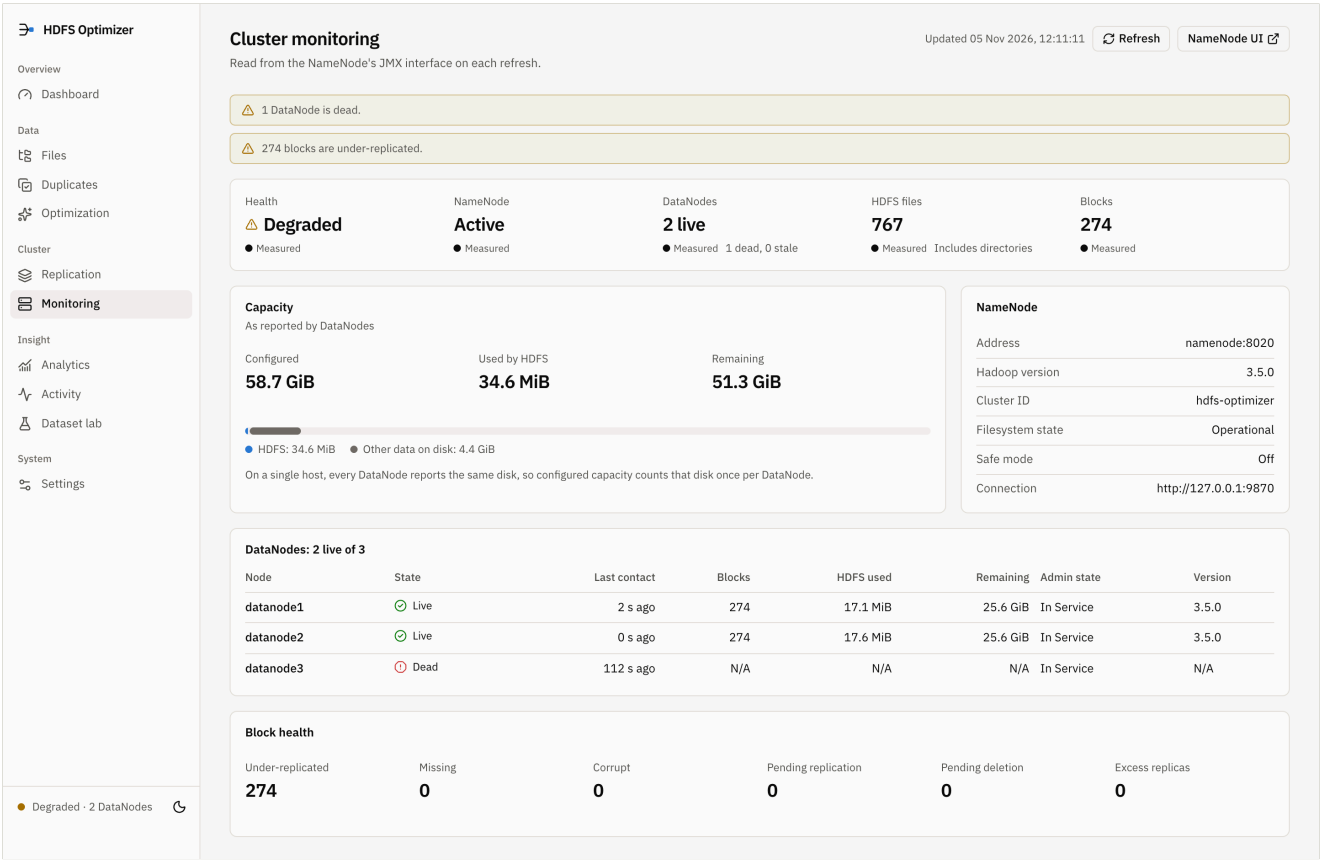

Figure 43: Real failure: a stopped DataNode is reported as dead, and all managed blocks are under-replicated

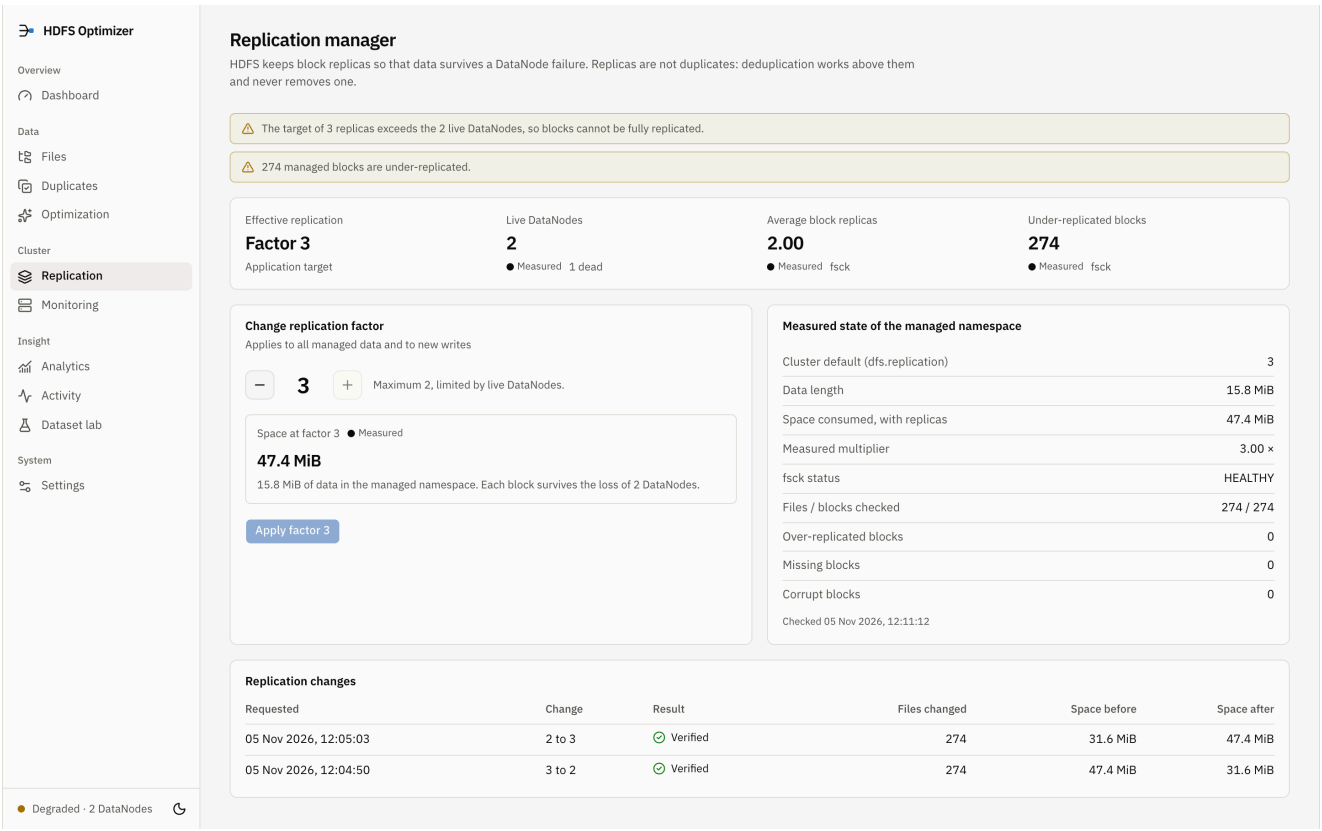

Figure 44: While a DataNode is dead, the maximum selectable factor falls to 2

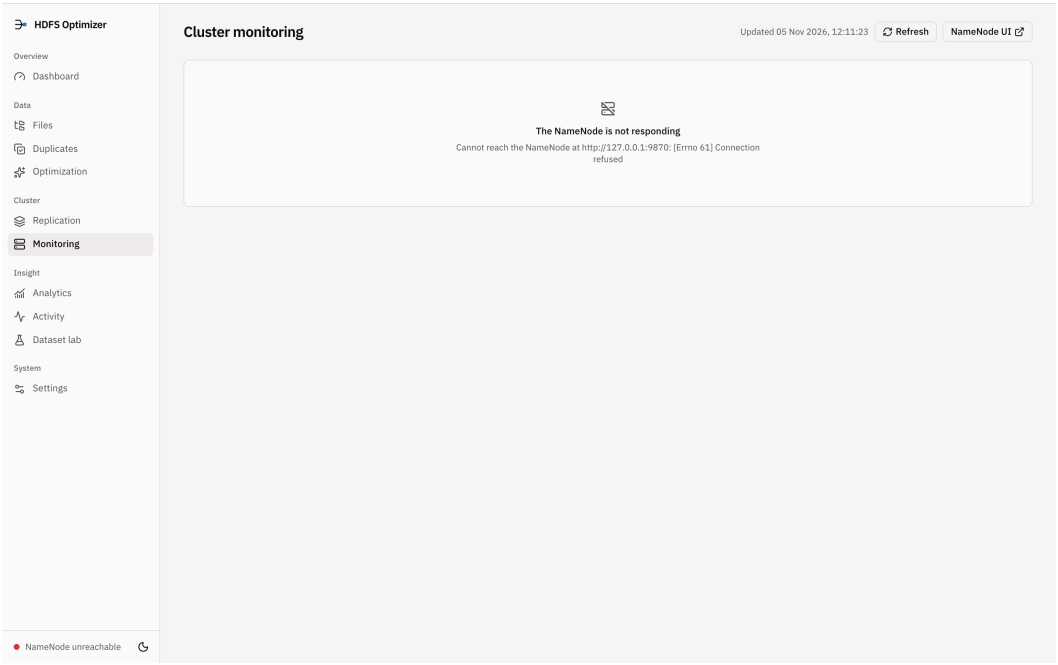

Figure 45: Real failure: the NameNode is stopped, and the interface reports it

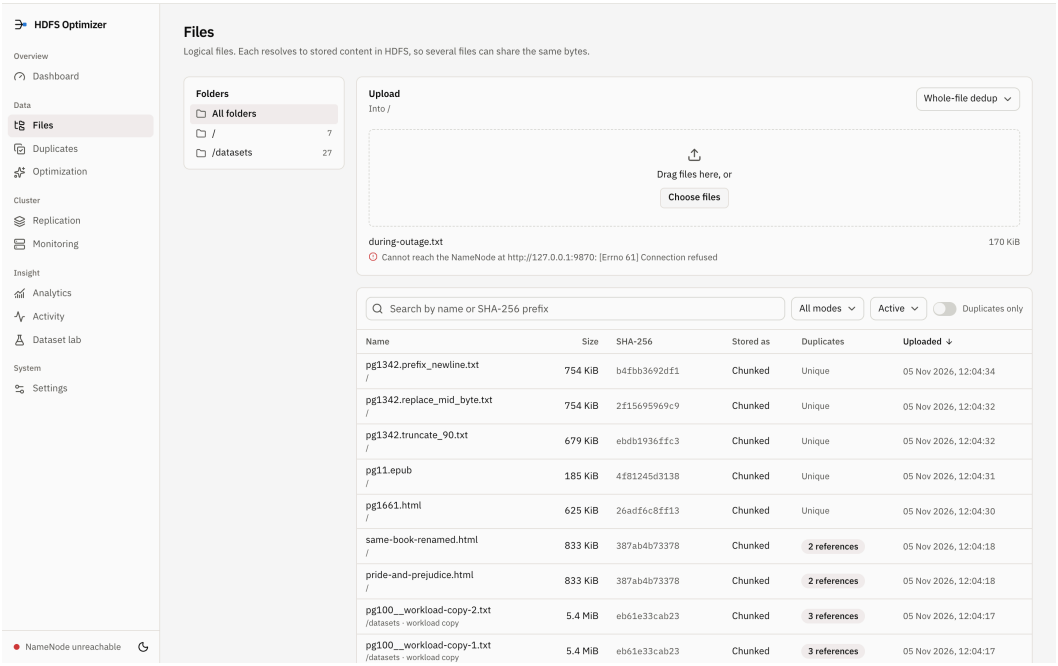

Figure 46: An upload during the NameNode outage fails with a clear error, and no file is recorded

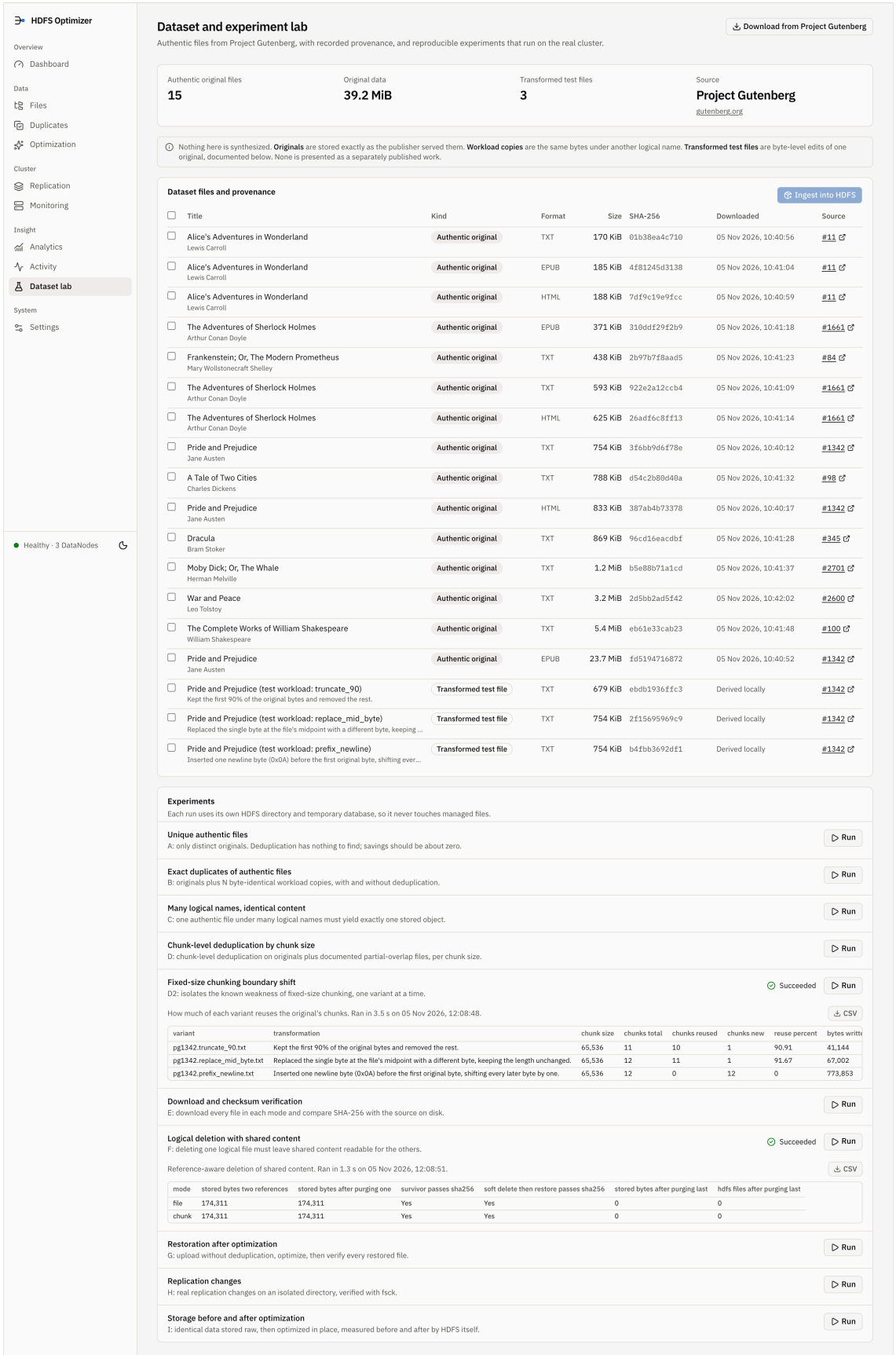

Figure 47: Experiments run from the lab, showing measured results

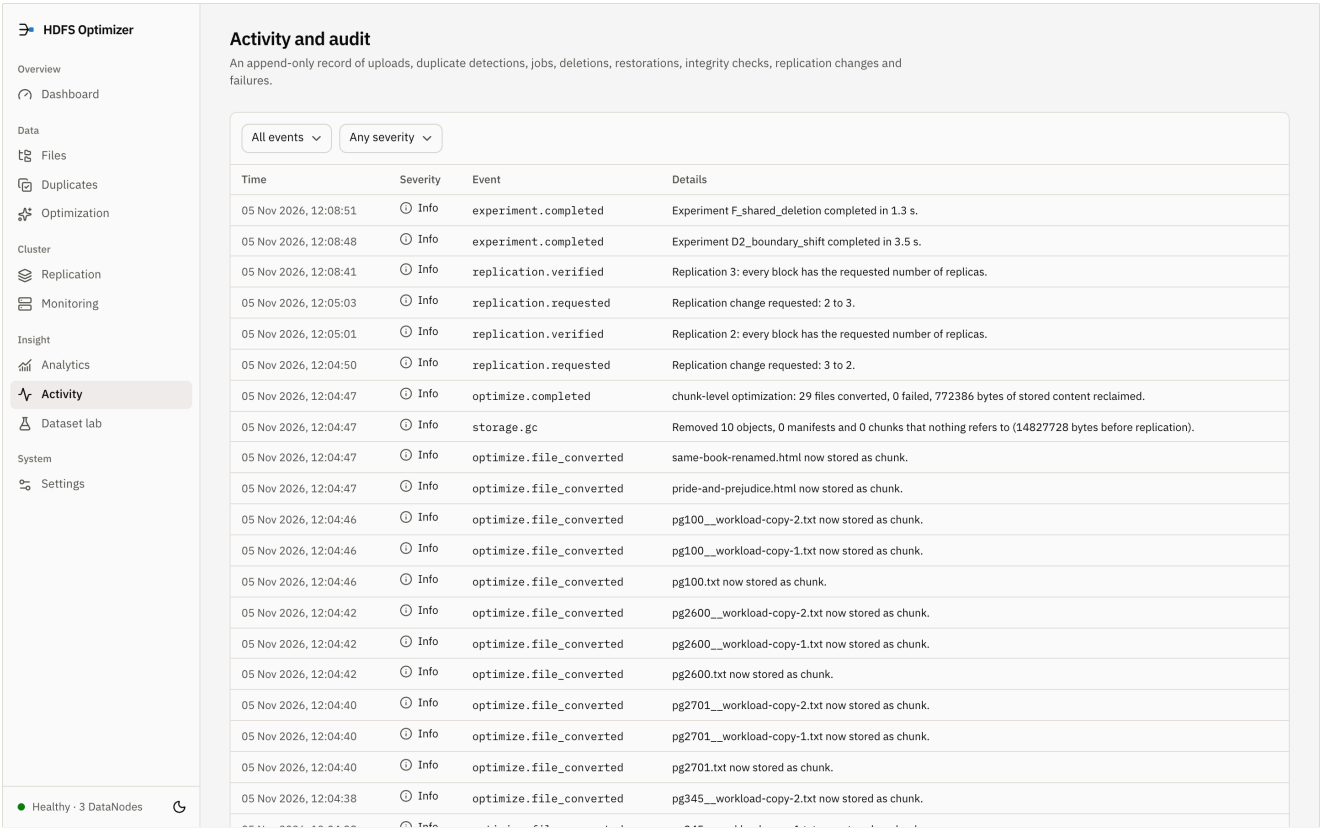

Figure 48: Audit trail of the session

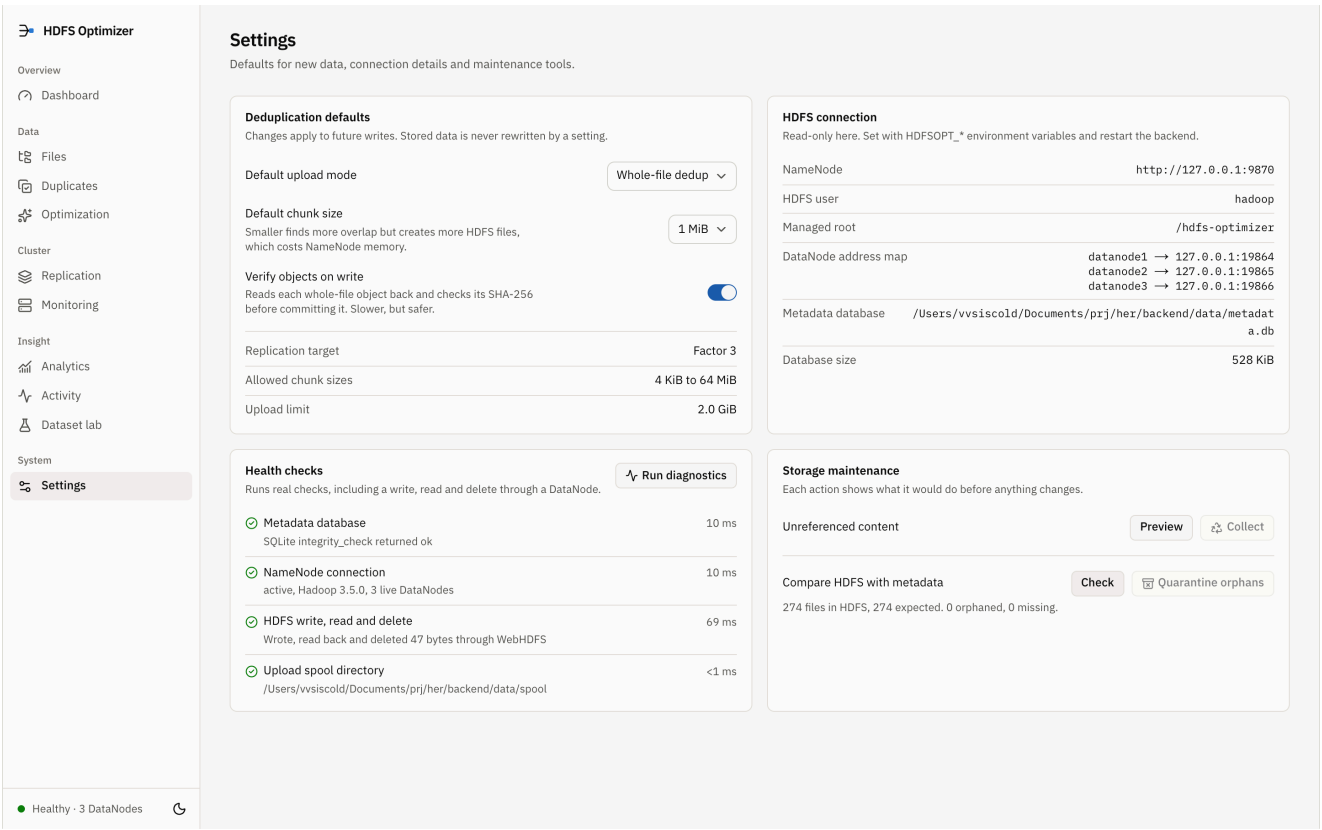

Figure 49: Settings, with real diagnostics and a metadata-to-HDFS comparison

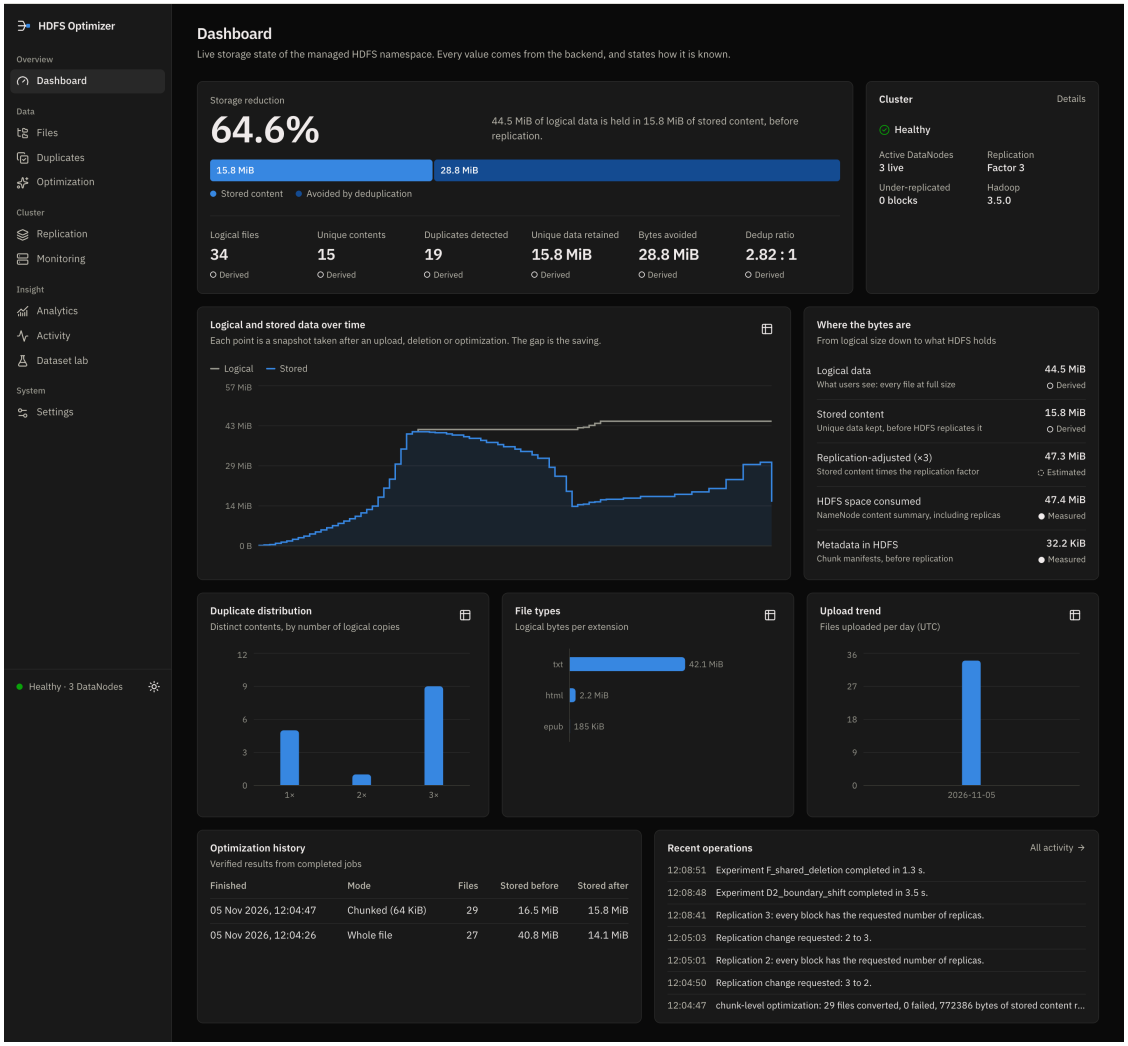

Figure 50: Dashboard in dark mode

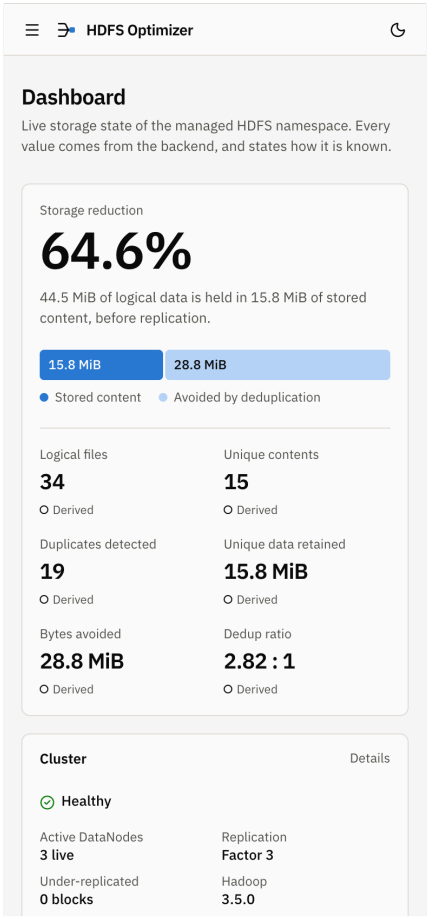

Figure 51: Dashboard at phone width

Chapter 14 — Result Analysis

14.1 Experiment methodology

All experiments ran on the real cluster: Hadoop 3.5.0, 3 DataNodes, replication 3. All daemons in Docker containers inside one Colima VM on one laptop; the backend runs on the host. Each run used a private HDFS directory and a temporary database. Runs with meaningful timings were repeated three times and report the mean with the observed range. Byte counts were required to be identical across iterations, and the benchmark tool flags any difference; no storage value differed. Directory counts do vary, because raw paths use random IDs, and are reported as means. `scripts/run_benchmarks.py` reproduces these runs.

The evaluation deliberately reports three separate quantities, because they answer different questions:

- **Derived (logical):** bytes of content, from the metadata database. Excludes replication and all overhead.
- **Measured by the NameNode:** `GETCONTENTSUMMARY`. `spaceConsumed` is the file length multiplied by replication. It excludes checksums and filesystem rounding.
- **Measured on disk:** `du` over each DataNode's data directory, summed. This includes block files, `.meta` checksum files and 4 KiB filesystem rounding.

Two measurement problems were found and corrected. The first benchmark run produced impossible negative values for files added. The cause was that NameNode JMX values are cached for about ten seconds, and also describe the whole cluster. These values were replaced by uncached queries for each run's own directory. Disk readings were also inconsistent, because DataNodes delete blocks asynchronously after the NameNode's delete returns, so a baseline could include earlier blocks. Each disk reading is now taken only after `du` returns the same value three times, three seconds apart. All results below come from the corrected run.

14.2 Formulas

Savings and ratio follow Equations 1 and 2 in Chapter 10. For N identical copies of each file, whole-file deduplication should give savings of exactly $(N-1)/N$ and a ratio of N.

14.3 Unique files (Experiment A)

With only distinct files, deduplication has nothing to find: savings were 0% in whole-file mode and 0% in chunk mode at 1 MiB. Chunk mode used 27 HDFS files where whole-file mode used 9. This is the correct control result.

14.4 Exact duplicates (Experiments B and C)

Table 3 shows that savings match theory exactly: with 1, 3 and 5 extra copies, N is 2, 4 and 6, and savings are 50%, 75% and 83.33%. NameNode-reported space stays flat at 39.98 MiB regardless of copies, while without deduplication it grows linearly. Figure 52 shows this. Replication stays at 3 throughout.

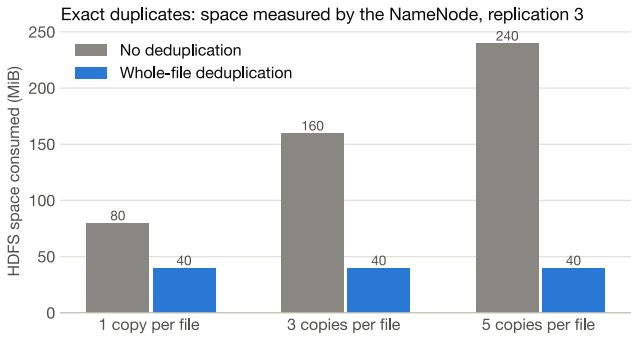

Figure 52: Measured HDFS space for exact duplicates

Deduplication also made these uploads faster, since a duplicate only needs a lookup and a commit. That comes from the workload's high duplicate rate, and it should not be read as a general speed advantage.

In Experiment C, 20 logical names for *Alice's Adventures in Wonderland* produced 1 HDFS file and savings of 95%. All 20 names passed SHA-256 verification.

14.5 Chunk-level deduplication (Experiment D)

This workload contains the 12 files of the originals plus the three documented variants. Table 4 and Figure 53 show results at each chunk size.

Three findings stand out. First, finer chunks find more overlap, but with diminishing returns. Second, chunks of 1 MiB and larger find nothing at all, because the book and its variants are smaller than one chunk, so each is a single differing chunk. Third, and most important, **logical savings overstate physical savings at small chunk sizes**. At 16 KiB, logical savings are 8.91%, but actual disk use is 14.64% *higher* than whole-file storage, because of filesystem rounding and a separate checksum file for every chunk replica. For this workload, 256 KiB gave the lowest disk use among the sizes tested. A system reporting only logical savings would recommend the wrong setting.

Upload throughput falls steeply, because each chunk costs several NameNode and DataNode round trips. The ranges at larger sizes are wide, so small differences between them should not be interpreted.

14.6 Boundary shift (Experiment D2)

Figure 54 isolates fixed-size chunking's weakness at 64 KiB. Truncation reuses 90.9% of chunks, and a one-byte replace-

Table 3: Nine authentic books with $N-1$ labelled exact copies each. Upload shows the mean and observed range of three runs.

Copies	Mode	Logical	Stored	Savings	Ratio	NameNode space	Upload MiB/s
1	No dedup	26.66 MiB	26.66 MiB	0%	1	79.97 MiB	19.5 (12.9–27.2)
1	Whole file	26.66 MiB	13.33 MiB	50%	2	39.98 MiB	44.7 (27.4–58.1)
3	No dedup	53.31 MiB	53.31 MiB	0%	1	159.93 MiB	28.2 (22–33.5)
3	Whole file	53.31 MiB	13.33 MiB	75%	4	39.98 MiB	91 (82.4–100.8)
5	No dedup	79.97 MiB	79.97 MiB	0%	1	239.9 MiB	23.2 (22.5–23.6)
5	Whole file	79.97 MiB	13.33 MiB	83.33%	6	39.98 MiB	95.2 (84.5–107.5)

Table 4: The same files at each chunk size. Disk is du across three DataNodes, including replicas.

Chunk size	Stored	Logical savings	HDFS files	NameNode objects	Disk used (du)	Disk vs whole-file	Upload MiB/s
whole file	15.46 MiB	0%	12	54	46.9 MiB	baseline	25.1 (23.9–27.4)
16 KiB	14.09 MiB	8.91%	921	3009	53.76 MiB	+14.64%	0.7 (0.6–0.8)
64 KiB	14.16 MiB	8.4%	244	894	45.65 MiB	−2.67%	2.9 (2.6–3.3)
256 KiB	14.48 MiB	6.38%	74	286	44.51 MiB	−5.09%	7.7 (6.2–8.4)
1 MiB	15.46 MiB	0%	33	126	47.29 MiB	+0.83%	16.3 (11.6–21.2)
4 MiB	15.46 MiB	0%	25	94	47.2 MiB	+0.63%	16.4 (11.1–19.6)

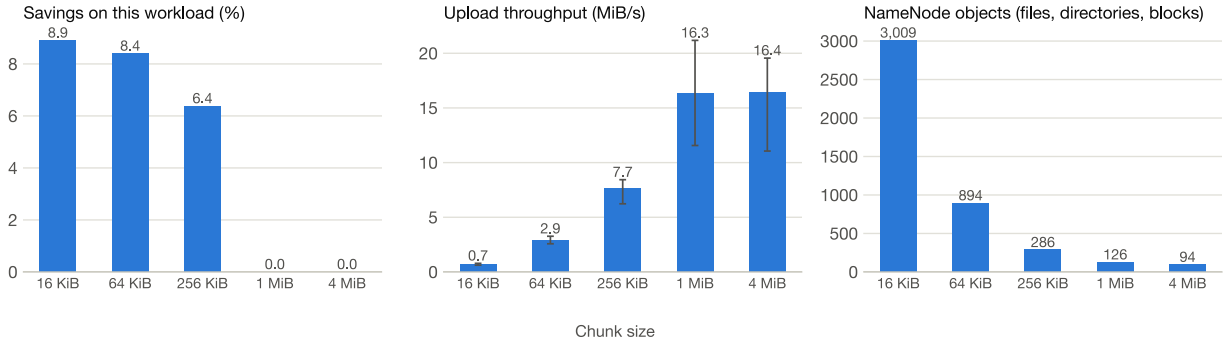

Figure 53: Chunk size trade-off, measured. Bars on throughput show the observed range across three runs.

ment reuses 91.7%, as only one chunk changes. A single byte inserted at the start reuses 0%, since every boundary moves, even though the content is nearly identical. Content-defined chunking [12], [13] exists to solve exactly this.

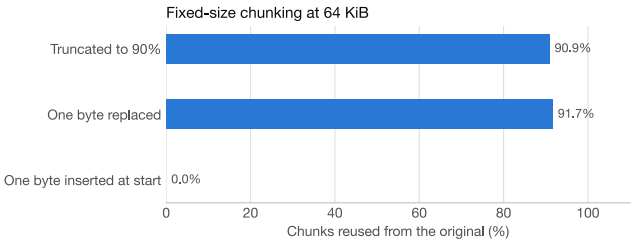

Figure 54: Chunk reuse after three small edits

14.7 Before and after optimization (Experiment I)

Here, 27 files (nine books with two labelled copies each) were first stored without deduplication, then optimized in place. Table 5 and Figure 55 show the results.

Savings were 66.67%, the expected value for three copies. The dry run predicted stored bytes exactly in both modes. All 27 files passed SHA-256 verification afterwards in both modes. Disk use before optimization was 1.06% above NameNode-reported space, consistent with HDFS checksums (4 bytes per 512 bytes, about 0.78%) plus rounding. Chunk mode needed 8.2 times as long as whole-file mode, and it used 66 HDFS files rather than 9. Exact copies alone offer chunk mode no advantage.

Table 5: Optimization in place, measured three ways. Optimize shows the mean and range of three runs.

Mode	Stored before / after	Predicted after	NameNode before / after	Disk (du) before / after	HDFS files before / after	Optimize (s)
Whole file	39.98 MiB / 13.33 MiB	13.33 MiB	119.95 MiB / 39.98 MiB	121.22 MiB / 40.41 MiB	27 / 9	0.39 (0.33–0.45)
Chunk 256 KiB	39.98 MiB / 13.33 MiB	13.33 MiB	119.95 MiB / 40.01 MiB	121.22 MiB / 40.92 MiB	27 / 66	3.21 (2.08–4.17)

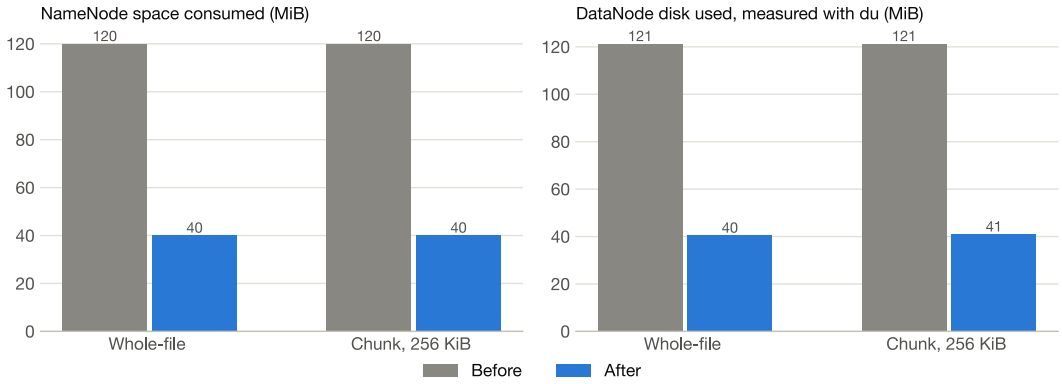

Figure 55: Before and after optimization, as measured by the NameNode and on DataNode disks

14.8 Restoration (Experiments E and G)

Figure 56 shows restore throughput, with whole-file SHA-256 verification active. All 15 files in each mode matched the publisher's bytes on disk: 15, 15, 15 of 15. Chunked restore is slower, at 35.2 MiB/s, because each chunk needs its own lookup and connection. The two faster modes overlap within their ranges.

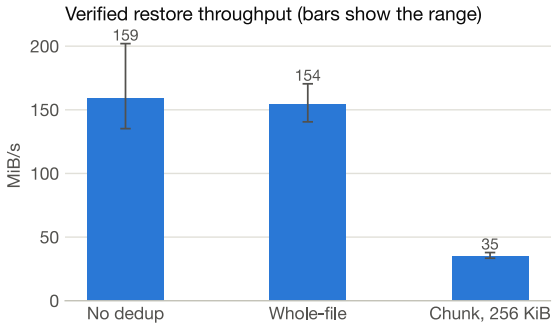

Figure 56: Verified restore throughput

14.9 Deletion with shared content (Experiment F)

In both modes, purging one of two references left stored bytes unchanged, at 0.17 MiB, and the surviving file passed verification. A soft delete followed by a restore also passed. Purging the last reference left 0 HDFS files, so no content leaked.

14.10 Replication (Experiment H)

Changing the factor from 3 to 2 and back, on real data, gave measured multipliers of 3, 2, 3, with zero under-replicated blocks once converged. Requesting a factor of 4 with 3 live DataNodes was refused.

Scale matters here. During the evidence capture, on a namespace of 274 chunk files, lowering the factor took 11 s, and raising it took 217 s. The NameNode schedules about 2 × live DataNodes blocks every 3 seconds for re-replication, so every extra small file also lengthens recovery. An earlier fixed 180-second timeout reported this correct behaviour as a failure, so the timeout now scales with the number of blocks.

14.11 Failure handling (Experiment J)

These were real failures, run with `scripts/run_failure_experiments.py`. Containers were stopped, and a process was killed with `SIGKILL`.

DataNode failure. After stopping a DataNode, files in both modes stayed readable and matched their hashes. The NameNode flagged it as dead after 88 s. `fsck` reported Under-replicated blocks: 3 (100.0 %) only 8.4 s later, because replicas are removed from the block map on the heartbeat checker's own cycle. With no spare node, these blocks stayed under-replicated until the node returned. A request for factor 3 with two live nodes returned HTTP 422.

NameNode failure. An upload returned HTTP 503 (`hdfs_unavailable`) in 0.021 s. The logical file count stayed at 2, the API kept answering, and the failure appeared in the audit trail. After a restart, a retry returned HTTP 201.

Process killed during optimization. The backend was killed with `SIGKILL` at progress 4 of 29. On restart the job was marked interrupted. Before resuming, 27 of 27 test files passed SHA-256, and reconciliation found 0 missing paths. The resumed job processed the remaining 25 files, and 27 of 27 then restored byte-identical, with 0 orphans left. The database and HDFS agreed afterwards.

14.12 Small-file overhead

At 16 KiB, this 15.46 MiB workload needed 3009 NameNode objects, against 54 for whole files. Using the 150-byte rule of thumb [7] as an **estimate**, 1 TiB of unique data at 64 KiB means about 16.8 million chunks, so about 33.6 million file and block objects and roughly 4.7 GiB of NameNode heap. That is an extrapolation and was not measured. It shows why packing chunks into containers is the most valuable next step.

14.13 Logical savings versus physical usage

The values differ for four reasons, all observed here. Replication multiplies data (× 3 here). Checksums and rounding add to it, by about 1% for large files and much more for small ones. Deletion is asynchronous, so disk use falls some time after the metadata says it should. And metadata has its own cost in the NameNode, which `du` on DataNodes does not show at all.

14.14 Timing validity

Timings come from one laptop running everything in a single VM, in an uncontrolled desktop environment, with three repetitions. The wide ranges in some rows reflect that. They support comparisons across large differences, such as chunk size, but not small ones, and they do not predict the performance of a real cluster.

14.15 Summary of observations

- Exact-duplicate savings match theory exactly, and hold in space measured by HDFS and on disk.
- Dry-run predictions of stored bytes were exact.
- Every restored file, in every experiment, matched its original SHA-256.
- Replication was unaffected by deduplication.
- Small chunks can make physical use worse while logical savings look better.
- No data was lost in any failure tested.

Chapter 15 — Limitations

15.1 Small-file overhead

Each chunk is an HDFS file, with its own block and replicas, and every file, directory and block is an object in NameNode memory. Chapter 14 measures this. Small chunks are expensive in a way that a block-level deduplicator inside a storage system would not be.

15.2 Metadata costs

Manifests are stored in HDFS and in SQLite, which adds bytes and a row per chunk reference. At the scale tested this is negligible, but it grows linearly with the number of chunks.

15.3 Docker resource limitations

All daemons share one 4 GB VM on one laptop. Measured throughput reflects that environment, and also includes a virtualised network and disk.

Deployment coverage differs by component. The standard cluster and the light two-DataNode profile were both run and tested. The backend image was built and tested in a container against the real cluster, including an upload and a SHA-256-matched download. The frontend image could **not** be built on the evaluation machine: its first build attempt exhausted the host's free disk, causing I/O errors in the build. HDFS was checked afterwards and was unharmed. The image was slimmed, and a build script was added that aborts below a free-disk margin, which it then did. Its build steps and server command were instead tested directly on the host, serving all ten pages, so the final image itself remains unverified. Evidence is in `evidence/12_light_cluster.txt` and `evidence/13_containerised_app.txt`.

15.4 Single-machine versus multi-machine deployments

All DataNodes share one disk, so a disk or host failure would lose everything. Configured capacity counts that disk once per DataNode. Stopping a container shows HDFS's software behaviour under failure, but this deployment provides no real hardware fault tolerance. With three DataNodes at factor 3, there is also no spare node to re-replicate onto, so blocks stay under-replicated until the node returns.

15.5 Chunking limitations

Fixed-size chunking is defeated by a shift, as measured. A chunk larger than a file also finds no overlap with its variants, so the best chunk size depends on the data.

15.6 Hashing overhead

Each upload hashes all bytes, and whole-file writes are read back to verify. Chunk mode also hashes every chunk, so a verified restore costs CPU proportional to file size.

15.7 Concurrency limitations

One process, one job worker, and a lock held during the storage step, mean ingestion is serial. Running several backend processes against one database is unsafe, because the content lock is in-process.

15.8 Scalability considerations

SQLite is a single-writer database on one disk. A full index of chunk hashes becomes the bottleneck at scale, which is the problem addressed by Zhu et al. [14].

15.9 HDFS-specific restrictions

The application cannot deduplicate HDFS blocks, since these are internal to HDFS. HDFS block checksums already catch disk corruption on read, so the SHA-256 checks here add end-to-end protection rather than replace them. JMX values are cached for about 10 seconds, so monitoring lags slightly. The NameNode remains a single point of failure, because HA is not configured.

15.10 Security

Access control is an optional shared token, intended for local use, and the API binds to localhost. WebHDFS uses Hadoop's simple authentication, which trusts the stated user name. The system should not be exposed to a network as it stands.

Chapter 16 — Future Scope

16.1 Content-defined chunking

Replacing fixed boundaries with FastCDC [13] would make deduplication robust to insertions. The experiment here gives a baseline to compare against.

16.2 Object packing

Writing chunks into large container files, with an offset index, would keep the savings of fine chunking while using few HDFS files and blocks. This addresses the largest measured cost.

16.3 Large-scale distributed indexing

A distributed key-value store, with a Bloom filter in front as in Data Domain [14], would remove the SQLite limit.

16.4 Apache Spark integration

A Spark job [19] could scan an existing HDFS directory in parallel, computing hashes close to the data rather than moving it through one Python process.

16.5 Stronger multi-user authorization

Per-user accounts and Kerberos would be needed. Deduplicating across users also raises an information leak, since a fast “duplicate” response reveals that someone else has the same file. That would need analysis.

16.6 Incremental background deduplication

Ingest could accept data immediately and deduplicate it in the background, using continuous rather than manual jobs.

16.7 More advanced monitoring

Time series of cluster metrics and alerts could be exported to a monitoring system such as Prometheus.

16.8 Enterprise deployment

This includes separate machines, NameNode HA, TLS, and a server database with backup. Rack-aware placement could then be tested on real hardware.

Chapter 17 — Conclusion

17.1 What was built

This project built a working deduplication layer on an unmodified Apache Hadoop 3.5.0 cluster: a FastAPI backend, a relational metadata model, whole-file and fixed-size chunk deduplication through content-addressable storage, safe optimization, replication management, monitoring, a ten-page dashboard and reproducible experiments on authentic files.

17.2 What was verified

129 unit and API tests, 17 integration tests on the real cluster and 13 real-browser tests passed. Every restored file in every experiment matched its original SHA-256. Real DataNode, NameNode and process-kill failures caused no data loss, and an interrupted job resumed safely.

17.3 Results achieved

Savings on exact duplicates matched theory exactly. On 27 files comprising three copies of each of nine books, whole-file optimization reduced NameNode-reported space from 119.95 MiB to 39.98 MiB, with disk measurements in agreement, and all files verified. These figures describe controlled workloads whose duplicates were created deliberately. They verify the mechanism, and they do not predict savings on a real system.

17.4 Technical significance

Three lessons came from running on real HDFS rather than a simulation. Logical savings can mislead: at small chunk sizes, disk use rose even as logical savings improved. Consistency between two independent stores can be achieved by ordering operations carefully, with no shared transaction. And asynchronous behaviour must be observed rather than assumed: `fsck`, JMX caching, block reports and DataNode deletion each surprised an early version of this system, and each was exposed by testing against the real cluster.

17.5 Future possibilities

Content-defined chunking and chunk packing would address the two largest measured weaknesses. A distributed index, parallel scanning with Apache Spark, and deployment on separate machines would make it possible to evaluate the design at a scale this project could not reach.

References

[1] S. Ghemawat, H. Gobioff, and S.-T. Leung, "The Google File System," in *Proceedings of the Nineteenth ACM Symposium on Operating Systems Principles (SOSP)*, 2003, pp. 29–43. doi: 10.1145/945445.945450.

[2] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, "The Hadoop Distributed File System," in *2010 IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST)*, 2010, pp. 1–10. doi: 10.1109/MSST.2010.5496972.

[3] The Apache Software Foundation, "HDFS Architecture (Apache Hadoop 3.5.0 documentation)." Accessed: Nov. 05, 2026. [Online]. Available: https://hadoop.apache.org/docs/r3.5.0/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html

[4] D. T. Meyer and W. J. Bolosky, "A Study of Practical Deduplication," *ACM Transactions on Storage*, vol. 7, no. 4, pp. 1–20, 2012, doi: 10.1145/2078861.2078864.

[5] J. Paulo and J. Pereira, "A Survey and Classification of Storage Deduplication Systems," *ACM Computing Surveys*, vol. 47, no. 1, pp. 1–30, 2014, doi: 10.1145/2611778.

[6] W. Xia *et al.*, "A Comprehensive Study of the Past, Present, and Future of Data Deduplication," *Proceedings of the IEEE*, vol. 104, no. 9, pp. 1681–1710, 2016, doi: 10.1109/JPROC.2016.2571298.

[7] Cloudera, "The Small Files Problem." Accessed: Nov. 05, 2026. [Online]. Available: https://blog.cloudera.com/the-small-files-problem/

[8] The Apache Software Foundation, "WebHDFS REST API (Apache Hadoop 3.5.0 documentation)." Accessed: Nov. 05, 2026. [Online]. Available: https://hadoop.apache.org/docs/r3.5.0/hadoop-project-dist/hadoop-hdfs/WebHDFS.html

[9] S. Quinlan and S. Dorward, "Venti: A New Approach to Archival Data Storage," in *1st USENIX Conference on File and Storage Technologies (FAST)*, 2002. [Online]. Available: https://www.usenix.org/conference/fast-02/venti-new-approach-archival-data-storage

[10] National Institute of Standards and Technology, "Secure Hash Standard (SHS), FIPS PUB 180-4," 2015. doi: 10.6028/NIST.FIPS.180-4.

[11] M. Stevens, E. Bursztein, P. Karpman, A. Albertini, and Y. Markov, "The First Collision for Full SHA-1," in *Advances in Cryptology, CRYPTO 2017, Lecture Notes in Computer Science*, 2017, pp. 570–596. doi: 10.1007/978-3-319-63688-7_19.

[12] A. Muthitacharoen, B. Chen, and D. Mazières, "A Low-bandwidth Network File System," in *Proceedings of the Eighteenth ACM Symposium on Operating Systems Principles (SOSP)*, 2001, pp. 174–187. doi: 10.1145/502034.502052.

[13] W. Xia *et al.*, "FastCDC: A Fast and Efficient Content-Defined Chunking Approach for Data Deduplication," in *2016 USENIX Annual Technical Conference (ATC)*, 2016. [Online]. Available: https://www.usenix.org/conference/atc16/technical-sessions/presentation/xia

[14] B. Zhu, K. Li, and H. Patterson, "Avoiding the Disk Bottleneck in the Data Domain Deduplication File System," in *6th USENIX Conference on File and Storage Technologies (FAST)*, 2008. [Online]. Available: https://www.usenix.org/conference/fast-08/avoiding-disk-bottleneck-data-domain-deduplication-file-system

[15] Project Gutenberg Literary Archive Foundation, "Project Gutenberg." Accessed: Nov. 05, 2026. [Online]. Available: https://www.gutenberg.org/

[16] Project Gutenberg Literary Archive Foundation, "Information About Robot Access to our Pages." Accessed: Nov. 05, 2026. [Online]. Available: https://www.gutenberg.org/policy/robot_access.html

[17] Project Gutenberg Literary Archive Foundation, "The Project Gutenberg License." Accessed: Nov. 05, 2026. [Online]. Available: https://www.gutenberg.org/policy/license.html

[18] S. Ramírez, "FastAPI documentation." Accessed: Nov. 05, 2026. [Online]. Available: https://fastapi.tiangolo.com/

[19] M. Zaharia *et al.*, "Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing," in *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2012. [Online]. Available: https://www.usenix.org/conference/nsdi12/technical-sessions/presentation/zaharia

Appendix A: Setup Commands

The commands below were used to build and run the system on the evaluation machine.

```
brew install colima docker docker-compose typst    # macOS without Docker Desktop
scripts/setup.sh                                   # dependencies, frontend build, Hadoop image
scripts/start-all.sh                               # HDFS, backend, frontend
backend/.venv/bin/python scripts/download_dataset.py
cd backend && .venv/bin/python -m pytest -m "not hdfs"
cd backend && .venv/bin/python -m pytest -m hdfs
cd frontend && npx playwright test
source scripts/env.sh && backend/.venv/bin/python scripts/run_benchmarks.py --iterations 3
source scripts/env.sh && backend/.venv/bin/python scripts/run_failure_experiments.py
backend/.venv/bin/python scripts/make_charts.py
scripts/capture_evidence.sh
backend/.venv/bin/python scripts/build_report.py
cd presentation/source && ../../backend/.venv/bin/python prepare_crops.py && npm install --no-audit --
no-fund && node build_deck.js
```

Evaluation environment: arm64, macOS-26.6.2-arm64-arm-64bit, Python 3.12.4, Hadoop 3.5.0, 3 DataNodes, replication 3. All daemons in Docker containers inside one Colima VM on one laptop; the backend runs on the host.

Appendix B: API Documentation

Generated from the application's OpenAPI schema. Interactive documentation is served at `http://localhost:8100/docs`.

Method	Path	Purpose
POST	`/files/upload`	Spool to disk while hashing, so memory use does not grow with file size, then ingest.
GET	`/files`	List Files
GET	`/files/folders`	Virtual folders with their direct file counts and sizes.
POST	`/files/verify-all`	Read every active file back from HDFS and compare SHA-256, as a background job.
GET	`/files/{file_id}`	Get File
DELETE	`/files/{file_id}`	Soft delete by default. ``permanent`` applies only to an already-deleted file.
GET	`/files/{file_id}/download`	Download File
POST	`/files/{file_id}/restore`	Restore File
POST	`/files/{file_id}/verify`	Verify File
GET	`/dedup/groups`	Duplicate Groups
GET	`/dedup/duplicates`	Duplicates
GET	`/dedup/shared-chunks`	Chunks used by more than one manifest: shared content inside files that are not identical overall.
POST	`/dedup/analyze`	Dry run. Reports estimated savings and changes nothing in HDFS.
POST	`/dedup/optimize`	Start optimization. Requires a successful preview of the catalog exactly as it is now.
GET	`/dedup/jobs`	List Jobs
GET	`/dedup/jobs/{job_id}`	Get Job
POST	`/dedup/jobs/{job_id}/resume`	Run an interrupted or failed optimization again. Finished files are not eligible, so none repeat.
GET	`/dedup/gc`	Preview Garbage
POST	`/dedup/gc`	Run Garbage Collection
GET	`/health`	The API is up. HDFS state is reported separately, so a cluster outage stays visible in the UI.
GET	`/system/status`	System Status
GET	`/system/diagnostics`	Run real checks, including a write, read and delete cycle through a DataNode.
GET	`/storage/statistics`	Storage Statistics
GET	`/storage/savings`	Storage Savings
GET	`/storage/objects`	Storage Objects
GET	`/storage/reconcile`	Reconcile Report
POST	`/storage/reconcile`	Reconcile Apply
GET	`/cluster/status`	Cluster Status
GET	`/cluster/nodes`	Cluster Nodes
GET	`/cluster/replication`	Cluster Replication
POST	`/cluster/replication`	Change Replication
GET	`/analytics/overview`	Analytics Overview
GET	`/analytics/history`	Analytics History
GET	`/analytics/performance`	Analytics Performance
GET	`/analytics/report.csv`	Analytics Report
GET	`/datasets`	List Datasets
POST	`/datasets/import`	Import Dataset
POST	`/datasets/ingest`	Ingest Dataset
GET	`/datasets/provenance`	Provenance
GET	`/experiments`	List Experiments
POST	`/experiments/run`	Run Experiment
GET	`/experiments/{run_id}/export.csv`	Export Experiment
GET	`/audit/events`	Audit Events

Method	Path	Purpose
GET	`/settings`	Get Settings View
PUT	`/settings`	Update Settings

Appendix C: Configuration

All settings use environment variables with the prefix `HDFSOPT_`. See `.env.example`, which is also listed in Chapter 11. HDFS settings are in `infrastructure/hadoop/hdfs-site.xml`. Values that differ from Hadoop defaults for demonstration:

Property	Value	Reason
`dfs.namenode.heartbeat.recheck-interval`	30000 ms	A stopped DataNode is declared dead after about 90 s instead of 10.5 minutes
`dfs.namenode.safemode.extension`	5000 ms	Faster restart in demonstrations
`dfs.client.block.write.replace-datanode-on-failure.best-effort`	true	Small clusters have no spare node for a failed write pipeline

Appendix D: Database Schema

Generated from the SQLAlchemy models.

app_settings

Column	Type	Null	Key
key	VARCHAR(64)	no	primary key
value	JSON	yes	
updated_at	DATETIME	no	

audit_events

Column	Type	Null	Key
id	INTEGER	no	primary key
occurred_at	DATETIME	no	indexed
event_type	VARCHAR(48)	no	indexed
severity	VARCHAR(8)	no	indexed
entity_type	VARCHAR(24)	yes	
entity_id	VARCHAR(64)	yes	
message	TEXT	no	
details	JSON	yes	

chunks

Column	Type	Null	Key
id	VARCHAR(32)	no	primary key
sha256	VARCHAR(64)	no	unique, indexed
size_bytes	BIGINT	no	
hdfs_path	VARCHAR(512)	no	
state	VARCHAR(16)	no	indexed
created_at	DATETIME	no	
removed_at	DATETIME	yes	

dataset_sources

Column	Type	Null	Key
id	VARCHAR(32)	no	primary key
title	VARCHAR(255)	no	
author	VARCHAR(255)	yes	
ebook_id	INTEGER	yes	
source_url	VARCHAR(512)	yes	
download_url	VARCHAR(512)	yes	
organization	VARCHAR(255)	yes	
downloaded_at	DATETIME	yes	
file_format	VARCHAR(32)	no	
size_bytes	BIGINT	no	
sha256	VARCHAR(64)	no	indexed
kind	VARCHAR(24)	no	
derived_from_id	VARCHAR(32)	yes	references dataset_sources.id
transformation	TEXT	yes	
local_path	VARCHAR(512)	no	
license_note	TEXT	yes	

Constraints and indexes: ck_dataset_derived_is_documented; ck_dataset_kind.

deduplication_jobs

Column	Type	Null	Key
id	VARCHAR(32)	no	primary key
kind	VARCHAR(24)	no	indexed

Column	Type	Null	Key
mode	VARCHAR(8)	yes	
chunk_size	INTEGER	yes	
status	VARCHAR(16)	no	indexed
progress_done	INTEGER	no	
progress_total	INTEGER	no	
message	TEXT	yes	
params	JSON	yes	
result	JSON	yes	
error	TEXT	yes	
resumed_from_id	VARCHAR(32)	yes	
created_at	DATETIME	no	indexed
started_at	DATETIME	yes	
finished_at	DATETIME	yes	
heartbeat_at	DATETIME	yes	

experiment_runs

Column	Type	Null	Key
id	VARCHAR(32)	no	primary key
scenario	VARCHAR(48)	no	indexed
params	JSON	yes	
status	VARCHAR(16)	no	
result	JSON	yes	
error	TEXT	yes	
job_id	VARCHAR(32)	yes	
started_at	DATETIME	no	indexed
finished_at	DATETIME	yes	

file_manifests

Column	Type	Null	Key
id	VARCHAR(32)	no	primary key
file_sha256	VARCHAR(64)	no	indexed
file_size	BIGINT	no	
chunk_size	INTEGER	no	
chunk_count	INTEGER	no	
manifest_bytes	INTEGER	no	
hdfs_path	VARCHAR(512)	no	
state	VARCHAR(16)	no	indexed
created_at	DATETIME	no	
removed_at	DATETIME	yes	

Constraints and indexes: uq_manifest_content.

objects

Column	Type	Null	Key
id	VARCHAR(32)	no	primary key
sha256	VARCHAR(64)	no	unique, indexed
size_bytes	BIGINT	no	
hdfs_path	VARCHAR(512)	no	
state	VARCHAR(16)	no	indexed
created_at	DATETIME	no	
removed_at	DATETIME	yes	

performance_samples

Column	Type	Null	Key
id	INTEGER	no	primary key

Column	Type	Null	Key
operation	VARCHAR(32)	no	indexed
mode	VARCHAR(8)	yes	
bytes_processed	BIGINT	no	
duration_ms	FLOAT	no	
file_id	VARCHAR(32)	yes	
created_at	DATETIME	no	indexed

replication_operations

Column	Type	Null	Key
id	VARCHAR(32)	no	primary key
previous_factor	INTEGER	yes	
requested_factor	INTEGER	no	
status	VARCHAR(16)	no	
files_total	INTEGER	no	
files_changed	INTEGER	no	
space_consumed_before	BIGINT	yes	
space_consumed_after	BIGINT	yes	
details	JSON	yes	
error	TEXT	yes	
requested_at	DATETIME	no	indexed
completed_at	DATETIME	yes	

storage_snapshots

Column	Type	Null	Key
id	INTEGER	no	primary key
taken_at	DATETIME	no	indexed
reason	VARCHAR(48)	no	
logical_files	INTEGER	no	
logical_bytes	BIGINT	no	
stored_bytes	BIGINT	no	
unique_files	INTEGER	no	

file_chunks

Column	Type	Null	Key
id	INTEGER	no	primary key
manifest_id	VARCHAR(32)	no	references file_manifests.id, indexed
chunk_id	VARCHAR(32)	no	references chunks.id, indexed
ordinal	INTEGER	no	
offset_bytes	BIGINT	no	

Constraints and indexes: uq_manifest_ordinal; ck_ordinal_nonnegative.

files

Column	Type	Null	Key
id	VARCHAR(32)	no	primary key
name	VARCHAR(255)	no	
folder	VARCHAR(512)	no	indexed
size_bytes	BIGINT	no	
sha256	VARCHAR(64)	no	indexed
mime_type	VARCHAR(128)	yes	
extension	VARCHAR(32)	yes	
storage_mode	VARCHAR(8)	no	
status	VARCHAR(8)	no	indexed
raw_path	VARCHAR(512)	yes	
object_id	VARCHAR(32)	yes	references objects.id, indexed

Column	Type	Null	Key
manifest_id	VARCHAR(32)	yes	references file_manifests.id, indexed
provenance_label	VARCHAR(24)	no	
dataset_source_id	VARCHAR(32)	yes	references dataset_sources.id
created_at	DATETIME	no	
updated_at	DATETIME	no	
deleted_at	DATETIME	yes	
purged_at	DATETIME	yes	
last_verified_at	DATETIME	yes	
last_verified_ok	BOOLEAN	yes	

Constraints and indexes: ck_file_storage_pointer; ck_file_size; uq_active_file_name.

optimization_history

Column	Type	Null	Key
id	INTEGER	no	primary key
job_id	VARCHAR(32)	no	references deduplication_jobs.id, indexed
mode	VARCHAR(8)	no	
chunk_size	INTEGER	yes	
before	JSON	no	
after	JSON	no	
files_processed	INTEGER	no	
files_failed	INTEGER	no	
bytes_reclaimed_logical	BIGINT	no	
duration_ms	FLOAT	no	
created_at	DATETIME	no	indexed

integrity_checks

Column	Type	Null	Key
id	INTEGER	no	primary key
file_id	VARCHAR(32)	no	references files.id, indexed
ok	BOOLEAN	no	
expected_sha256	VARCHAR(64)	no	
actual_sha256	VARCHAR(64)	yes	
bytes_read	BIGINT	no	
chunks_checked	INTEGER	no	
missing	JSON	yes	
corrupt	JSON	yes	
detail	TEXT	yes	
duration_ms	FLOAT	no	
checked_at	DATETIME	no	indexed

Appendix E: Test Results

Counts are parsed from the saved test output in `evidence/` when this report is built.

Suite	Passed	Failed
Unit and API tests (stand-in storage, labelled test-only)	129	0
Integration tests on the real HDFS cluster	17	0
Browser tests in Chrome on the real cluster	13	0

The tests cover file hashing, upload, exact duplicate detection, different names with identical contents, different contents with the same name, object creation, shared chunk references, reconstruction, SHA-256 integrity, logical deletion, shared content preservation, garbage collection safety, invalid input, HDFS connection errors, replication validation, API responses, database constraints, partial upload failure, interrupted jobs and metric correctness.

Appendix F: Benchmark Records

Machine-readable records are in `benchmarks/results/`, as a CSV and a JSON file per scenario, and `documentation/RESULTS_SUMMARY.md` is generated from them. To reproduce them, run the benchmark commands in Appendix A. Timings will differ between machines; byte counts should not.

Appendix G: Dataset Provenance

From `datasets/provenance/manifest.json`. Full SHA-256 values and URLs are in that file.

Kind	Title and source page	Format	Bytes	SHA-256 prefix
original	Pride and Prejudice https://www.gutenberg.org/ebooks/1342	txt	772386	`3f6bb9d6f78e0293`
original	Pride and Prejudice https://www.gutenberg.org/ebooks/1342	html	852589	`387ab4b733788098`
original	Pride and Prejudice https://www.gutenberg.org/ebooks/1342	epub	24864457	`fd519471687295a8`
original	Alice's Adventures in Wonderland https://www.gutenberg.org/ebooks/11	txt	174311	`01b38ea4c710a84b`
original	Alice's Adventures in Wonderland https://www.gutenberg.org/ebooks/11	html	192539	`7df9c19e9fccac5b`
original	Alice's Adventures in Wonderland https://www.gutenberg.org/ebooks/11	epub	189249	`4f81245d31380ac1`
original	The Adventures of Sherlock Holmes https://www.gutenberg.org/ebooks/1661	txt	607606	`922e2a12ccb43a4c`
original	The Adventures of Sherlock Holmes https://www.gutenberg.org/ebooks/1661	html	640229	`26adf6c8ff138457`
original	The Adventures of Sherlock Holmes https://www.gutenberg.org/ebooks/1661	epub	379503	`310ddf29f2b98b87`
original	Frankenstein; Or, The Modern Prometheus https://www.gutenberg.org/ebooks/84	txt	448885	`2b97b7f8aad50864`
original	Dracula https://www.gutenberg.org/ebooks/345	txt	890348	`96cd16eacdbfebae`
original	A Tale of Two Cities https://www.gutenberg.org/ebooks/98	txt	807197	`d54c2b80d40a40b9`
original	Moby Dick; Or, The Whale https://www.gutenberg.org/ebooks/2701	txt	1276315	`b5e88b71a1cda068`
original	The Complete Works of William Shakespeare https://www.gutenberg.org/ebooks/100	txt	5638481	`eb61e33cab23dbe7`
original	War and Peace https://www.gutenberg.org/ebooks/2600	txt	3359610	`2d5bb2ad5f422765`
transformed	Pride and Prejudice (test workload: truncate_90) https://www.gutenberg.org/ebooks/1342	txt	695147	`ebdb1936ffc30d8c`
transformed	Pride and Prejudice (test workload: prefix_newline) https://www.gutenberg.org/ebooks/1342	txt	772387	`b4fbb3692df1e4d1`
transformed	Pride and Prejudice (test workload: replace_mid_byte) https://www.gutenberg.org/ebooks/1342	txt	772386	`2f15695969c98b30`

Original files were downloaded from `www.gutenberg.org/cache/epub/` on 2026-11-05. Organization: Project Gutenberg (Project Gutenberg Literary Archive Foundation). Public domain in the USA, distributed under the Project Gutenberg License (https://www.gutenberg.org/policy/license.html).

Appendix H: Troubleshooting

Symptom	Cause and fix
`docker compose` not found	Run `source scripts/env.sh`, which uses the project's Docker configuration.
Image pull fails on a credential helper	The home Docker configuration refers to a removed Docker Desktop. The same command avoids it.
Uploads return 503 just after a restart	The NameNode is briefly in safe mode. Wait a few seconds and retry.
Monitoring lags by a few seconds	NameNode JMX values are cached for about 10 seconds.
Replication shows "did not converge"	Check for a dead DataNode, then try again.
A download stops partway	The file failed verification, or HDFS went down. Check the Activity page.